Регламент-ТС

Исходный код программы

Автоматизированная система учёта и контроля технических средств тыла
Один офлайн-файл index.html и приложение для Windows (Electron)

Версия 1.0.0 · вложенные видео и изображения (base64) и стили в листинг не включены

Оглавление

1. Шина приложения и справочник технических средств (index.html, скрипт 1) 3
2. QR-код: собственный генератор (index.html, скрипт 2) 60
3. Платформа: модули, целостность, события (index.html, скрипт 3) 63
4. Модуль учёта: парк, ТО, моточасы, сохранность, пункт управления, сезоны, фото и докум… 67
5. Обзор и быстрая навигация (index.html, скрипт 5) 165
6. Общий поиск в шапке (index.html, скрипт 6) 167
7. Окно печати и справка (index.html, скрипт 7) 169
8. Витрина и навигация по разделам (index.html, скрипт 8) 186
9. app/main.js 187
10. app/preload.js 189
11. app/storage.js 190
12. app/package.json 192
13. app/scripts/prepare-www.js 193
14. app/scripts/build-installer.js 194
15. app/installer/installer.nsi 195
16. .github/workflows/build-windows.yml 197
17. tests/platform.spec.js 198
18. tests/qr.spec.js 201
19. tests/season.spec.js 204
20. tests/vfiles.spec.js 206
21. app/test/storage.test.js 208
22. app/test/e2e.test.js 210
23. app/test/portable.test.js 212

1. Шина приложения и справочник технических средств (index.html, скрипт 1)

```
/* ========================================================================
   ОБЩАЯ ШИНА ПРИЛОЖЕНИЯ — window.TST
   ------------------------------------------------------------------------
   Единственная точка связи между блоками. Модули не лезут в DOM и
   переменные друг друга, а общаются через:
     • TST.setWorkspace(ws)  — единый роутер разделов 'overview' | 'catalog' | 'fleet'
                               (раньше это делали три независимых обработчика
                               кликов, и состояние расходилось: в «Обзоре»
                               оставались видны поиск/колокол чужого раздела,
                               aria-selected указывал не на ту вкладку);
     • TST.showApp / showLanding — переход лендинг ⇄ приложение;
     • TST.on / TST.emit     — события: 'workspace', 'fleet-changed';
     • TST.catalog           — API справочника (объявлен ниже);
     • TST.fleet             — API модуля учёта (объявляет модуль учёта).
   ======================================================================== */
const TST = window.TST = (() => {
  const WORKSPACES = ['overview', 'catalog', 'fleet'];
  const LS_WS = 'spravochnik-tst-workspace';
  const WS_TITLES = { overview: 'Центр управления', catalog: 'Справочник', fleet: 'Учёт техники' };
  const LANDING_TITLE = document.title;
  const bus = new EventTarget();
  let current = null;

  /* Адрес экрана в hash: #overview, #catalog/ka-220/to2, #fleet/vehicle/<id>.
     Модули регистрируют:
       routeProviders[ws]() → строка маршрута текущего состояния;
       routeHandlers[ws](parts) → подготовить состояние по маршруту.
     routing — идёт применение маршрута, обратная запись адреса не нужна;
     lastHash — адрес, выставленный нами (его hashchange игнорируется) */
  let routing = false;
  let lastHash = null;

  const api = {
    catalog: null,
    fleet: null,
    get workspace() { return current; },
    on(type, fn) { bus.addEventListener(type, e => fn(e.detail || {})); },
    emit(type, detail) { bus.dispatchEvent(new CustomEvent(type, { detail })); },

    setWorkspace(ws) {
      if (WORKSPACES.indexOf(ws) === -1) ws = 'overview';
      const prev = current;
      current = ws;
      document.body.dataset.ws = ws;
      document.title = (WS_TITLES[ws] || 'Справочник') + ' — Регламент-ТС';
      document.querySelectorAll('[data-ws-btn]').forEach(b => {
        const on = b.dataset.wsBtn === ws;
        b.classList.toggle('is-active', on);
        b.setAttribute('aria-selected', String(on));
      });
      try { localStorage.setItem(LS_WS, ws); } catch (e) { /* приватный режим */ }
      api.emit('workspace', { ws, prev });
      api.syncRoute();
    },

    routeProviders: {},
    routeHandlers: {},

    /* Записать текущий экран в адрес страницы (новая запись истории —
       кнопка «Назад» браузера вернёт на предыдущий экран) */
    syncRoute() {
      if (routing || !api.isApp() || !current) return;
      const p = api.routeProviders[current];
      const h = '#' + (p ? p() : current);
      if (location.hash === h) return;
      lastHash = h;
      location.hash = h;
    },

    /* Открыть экран по адресу. Пустой адрес — лендинг */
    applyRoute(hash) {
      const parts = String(hash || '').replace(/^#\/?/, '').split('/').filter(Boolean).map(decodeURIComponent);
      if (!parts.length || WORKSPACES.indexOf(parts[0]) === -1) {
        if (api.isApp()) { routing = true; try { api.showLanding(); } finally { routing = false; } }
        return;
      }
      routing = true;
      try {
        const handler = api.routeHandlers[parts[0]];
        if (handler) handler(parts.slice(1));
        api.showApp(parts[0]);
      } finally {
        routing = false;
      }
      // Несуществующий id и т.п. — адрес приводится к фактическому экрану
      const p = api.routeProviders[current];
      const h = '#' + (p ? p() : current);
      if (location.hash !== h) { lastHash = h; location.replace(h); }
    },

```

```
      initRouter() {
        window.addEventListener('hashchange', () => {
          if (location.hash === lastHash) { lastHash = null; return; }
          lastHash = null;
          api.applyRoute(location.hash);
        });
        if (location.hash && location.hash !== '#') api.applyRoute(location.hash);
      },

      savedWorkspace() {
        try {
          const ws = localStorage.getItem(LS_WS);
          return WORKSPACES.indexOf(ws) === -1 ? 'overview' : ws;
        } catch (e) { return 'overview'; }
      },

      isApp() { return document.documentElement.classList.contains('is-app'); },

      showApp(ws) {
        const root = document.documentElement;
        root.classList.remove('is-landing');
        root.classList.add('is-app');
        document.querySelector('#landing video')?.pause();
        api.setWorkspace(ws || api.savedWorkspace());
      },

      showLanding() {
        const root = document.documentElement;
        root.classList.remove('is-app');
        root.classList.add('is-landing');
        document.title = LANDING_TITLE;
        document.querySelector('#landing video')?.play().catch(() => {});
        if (!routing && location.hash && location.hash !== '#') { lastHash = ''; location.hash = ''; }
      }
    };
    return api;
  })();
  /* ============================================================================
     ВСТРОЕННАЯ БАЗА ДАННЫХ
     Полностью оффлайн. Структура: catalog → группы → элементы.
     Каждый элемент содержит 5 видов обслуживания (СО, ТО-1, ТО-2, ТО-3, НЗ).
     ============================================================================ */

  /* Хелпер для типовых записей персонала, чтобы сократить дублирование */
  const P = (role, count) => ({ role, count });

  /* Виды обслуживания и их человекочитаемые названия */
  const MAINT_TYPES = [
    { key: 'eo',  short: 'ЕО',    full: 'Ежедневное обслуживание',
      desc: 'Контрольный осмотр и работы перед выходом, в ходе и после работы; периодичность и объём — по РЭ образца (раздел «Нормативная база»)' },
    { key: 'so',  short: 'СО',    full: 'Сезонное обслуживание',
      desc: 'Подготовка к летнему / зимнему периоду эксплуатации; сроки — по РЭ образца (раздел «Нормативная база»)' },
    { key: 'to1', short: 'ТО-1',  full: 'Техническое обслуживание №1',
      desc: 'Периодичность — по РЭ образца (раздел «Нормативная база»)' },
    { key: 'to2', short: 'ТО-2',  full: 'Техническое обслуживание №2',
      desc: 'Периодичность — по РЭ образца (раздел «Нормативная база»)' },
    { key: 'to3', short: 'ТО-3',  full: 'Техническое обслуживание №3',
      desc: 'Применяется, если предусмотрено РЭ образца' },
    { key: 'nz',  short: 'НЗ',    full: 'Снятие с неприкосновенного запаса',
      desc: 'Расконсервация, проверка комплектности и работоспособности после длительного хранения' }
  ];

  /* ============================================================================
     ОСНОВНОЙ КЛАССИФИКАТОР ТЕХНИКИ
     ============================================================================ */
  const CATALOG = [
    {
      id: 'food',
      name: 'Технические средства продовольственной службы',
      type: 'service',
      children: [

        /* ---------- ГРУППА 1: Приготовление, транспортирование и приём пищи ---------- */
        {
          id: 'food-cooking',
          name: 'Средства приготовления, транспортирования и приёма пищи',
          type: 'group',
          children: [

            /* === КУХНИ АВТОМОБИЛЬНЫЕ КА-220 / ПАК-200М === */
            {
              id: 'ka-220',
              name: 'Кухни автомобильные КА-220 и ПАК-200М',
              description: 'Автомобильные полевые кухни на шасси грузовых автомобилей, предназначенные для приготовления горячей пищи в полевых условиях на 220 (КА-220) и 200 (ПАК-200М) человек.',
              specs: {
                groups: [
                  { title: 'КА-220', params: [
                    { label: 'Базовое шасси', value: 'КамАЗ-4310' },
```

1. Шина приложения и справочник технических средств (index.html, скрипт 1) (продолжение)

```
                    { label: 'Количество питающихся, чел.', value: '220-250' },
                    { label: 'Котлы: количество, шт.', value: '3' },
                    { label: 'Котлы: материал', value: 'сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для первого блюда', value: '165/150' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для второго блюда', value: '2x70=140/125' },
                    { label: 'Кипятильники: количество, шт.', value: '2' },
                    { label: 'Кипятильники: производительность каждого, л/ч (на жидком топливе/на твердом топливе)', value: '60/55' },
                    { label: 'Вместимость сборника кипятка, л.', value: '100' },
                    { label: 'Жарочный шкаф, м²', value: '2x0,17=0,34' },
                    { label: 'Форсунки: тип', value: 'ФК-01' },
                    { label: 'Форсунки: количество (установленных/запасных), шт.', value: '2/1' },
                    { label: 'Баки топливные: количество, шт./вместимость, л.', value: '3/72' },
                    { label: 'Бак для воды: количество, шт./вместимость, л.', value: '1/480' },
                    { label: 'Расход топлива, кг/ч (жидкого/твердого)', value: '7-10/48' }
                  ]},
                  { title: 'ПАК-200М', params: [
                    { label: 'База монтажа', value: 'ЗИЛ-131 (УРАЛ-4320)' },
                    { label: 'Тип кузова-фургона', value: 'цельнометаллический' },
                    { label: 'Количество питающихся, чел.', value: '200' },
                    { label: 'Котлы: количество, шт. / материал', value: '3 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '150/140' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '125/110' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для кипятка', value: '150/140' },
                    { label: 'Масло в рубашке котла 2-го блюда: марка / масса, кг', value: 'АК-15 / 9' },
                    { label: 'Кипятильник: количество, шт. / вместимость, л.', value: '1 / 100/85' },
                    { label: 'Плита жарочная с площадью настила, кв. м', value: '-' },
                    { label: 'Жарочный шкаф: количество, шт. / размеры, мм', value: '1 / 420x300x300' },
                    { label: 'Форсунка: тип / количество (установленных/запасных)', value: 'ФК-01 / 3/2' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: '3 / 54' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого, кг/ч (дрова влажностью 25%)', value: '18 / 20-25' },
                    { label: 'Бак для воды: количество, шт. / вместимость, л', value: '1 / 350' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '50-55' },
                    { label: 'Масса, кг: сухая / в снаряженном состоянии', value: '7400 / 8770' }
                  ]},
                  { title: 'ПАК-200М(04)', params: [
                    { label: 'База монтажа', value: 'КАМАЗ-5350' },
                    { label: 'Тип кузова-фургона', value: 'цельнометаллический' },
                    { label: 'Количество питающихся, чел.', value: '200' },
                    { label: 'Котлы: количество, шт. / материал', value: '3 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '165/150' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '165/150' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для кипятка', value: '88/80' },
                    { label: 'Масло в рубашке котла 2-го блюда: марка / масса, кг', value: 'АК-15 / 9' },
                    { label: 'Кипятильник: количество, шт. / вместимость, л.', value: '1 / 100/85' },
                    { label: 'Плита жарочная с площадью настила, кв. м', value: '1' },
                    { label: 'Жарочный шкаф: количество, шт. / размеры, мм', value: '1 / 420x300x300' },
                    { label: 'Форсунка: тип / количество (установленных/запасных)', value: 'ФК-01 / 3/2' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: '3 / 54' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого, кг/ч (дрова влажностью 25%)', value: '12 / 20-25' },
                    { label: 'Бак для воды: количество, шт. / вместимость, л', value: '1 / 450' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '70' },
                    { label: 'Масса, кг: сухая / в снаряженном состоянии', value: '12780 / 15575' }
                  ]}
                ]
              },
              maintenance: {
                so: {
                  actions: [
                    'Перевод котлов и горелок на сезонный режим работы (летний/зимний)',
                    'Очистка дымоходов, котлов и крышек от нагара, накипи и сажи',
                    'Замена сезонных смазочных материалов в редукторах вентилятора и привода',
                    'Проверка теплоизоляции кухонного блока, восстановление повреждённых участков',
                    'Регулировка топливной системы под сезонные температуры окружающего воздуха',
                    'Опрессовка топливопроводов рабочим давлением, выявление мест подтекания',
                    'Покраска повреждённых участков термостойкой краской'
                  ],
                  labor: 12.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь по ремонту топливной аппаратуры', 1), P('Механик-водитель', 1)],
                  tools: ['Набор слесарного инструмента ИН-3', 'Манометр МТП-160', 'Динамометрический ключ', 'Набор щупов', 'Скребок для удаления нагара', 'Щётка металлическая'],
                  materials: ['Прокладки уплотнительные котла (комплект)', 'Смазка ЛИТОЛ-24', 'Керосин ТС-1', 'Термостойкая краска ВТМ-1', 'Ветошь обтирочная', 'Проволока стальная Ø 1,2 мм']
                },
                to1: {
                  actions: [
                    'Внешний осмотр оборудования, очистка от пыли и нагара',
                    'Проверка крепления узлов кухонного блока к раме автомобиля',
                    'Очистка форсунок горелок ГК-5 от засорений',
                    'Контроль уровня масла в редукторе привода вентилятора',
                    'Проверка работы запорной арматуры подачи топлива',
                    'Промывка котлов и крышек горячим раствором кальцинированной соды',
                    'Контроль состояния резиновых уплотнений крышек котлов'
                  ],
                  labor: 4.0,
                  personnel: [P('Повар', 1), P('Слесарь по ремонту топливной аппаратуры', 1)],
                  tools: ['Набор слесарного инструмента', 'Скребок', 'Щётка волосяная', 'Шприц для смазки'],
```

```
                  materials: ['Сода кальцинированная техническая', 'Мыло хозяйственное', 'Ветошь обтирочная', 'Керосин ТС-1', 'Смазка солидол синтетический']
                },
                to2: {
                  actions: [
                    'Полная разборка горелок с дефектацией деталей',
                    'Замена изношенных форсунок и сопел',
                    'Проверка герметичности котлов гидравлической опрессовкой',
                    'Замена прокладок и уплотнений в системе подачи топлива',
                    'Регулировка зазоров в механизме крышек котлов',
                    'Проверка работы термометров и манометров с поверкой',
                    'Ревизия дымососа и вентилятора с заменой подшипников при необходимости',
                    'Замена масла в редукторе привода вентилятора'
                  ],
                  labor: 10.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь по ремонту топливной аппаратуры', 1), P('Электрик', 1)],
                  tools: ['Набор слесарного инструмента ИН-3', 'Опрессовочный насос ручной', 'Динамометрический ключ', 'Мультиметр универсальный', 'Набор отвёрток', 'Ключи комбинированные'],
                  materials: ['Форсунки запасные ФК-01 (комплект)', 'Прокладки уплотнительные (комплект)', 'Уплотнения резиновые', 'Масло индустриальное И-20А', 'Смазка ЛИТОЛ-24', 'Припой ПОС-40', 'Флюс паяльный']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр всех систем кухонного блока',
                    'Демонтаж и ревизия дымососа с заменой подшипников и крыльчатки',
                    'Полная разборка топливной аппаратуры с заменой изношенных узлов',
                    'Замена теплоизоляции котлов при потере свойств',
                    'Балансировка крыльчаток вентиляторов на стенде',
                    'Замена силовых электрических кабелей и проводки',
                    'Восстановление лакокрасочного покрытия с грунтовкой',
                    'Контрольная топка с замером эксплуатационных параметров'
                  ],
                  labor: 20.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь по ремонту топливной аппаратуры', 1), P('Электрик', 1), P('Сварщик', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат инверторный', 'Балансировочный станок', 'Мегаомметр М4100/3', 'Набор съёмников', 'Газовая горелка'],
                  materials: ['Подшипники (комплект)', 'Теплоизоляционные плиты МКРР-130', 'Кабель силовой КГ', 'Краска эмаль НЦ-132', 'Электроды сварочные УОНИ-13/55', 'Ветошь обтирочная', 'Грунт ГФ-021']
                },
                nz: {
                  actions: [
                    'Удаление консервационной смазки с наружных и внутренних поверхностей',
                    'Расконсервация топливной системы продувкой сжатым воздухом',
                    'Заправка системы топливом и опрессовка рабочим давлением',
                    'Очистка котлов от защитной плёнки промывкой',
                    'Контроль комплектности по описи имущества',
                    'Опробование на холостом ходу с замером параметров',
                    'Контрольная варка пробного блюда (вода + крупа)',
                    'Замена расходных материалов с истёкшим сроком хранения'
                  ],
                  labor: 16.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь по ремонту топливной аппаратуры', 1), P('Механик-водитель', 1)],
                  tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Манометр контрольный', 'Термометр контактный', 'Секундомер', 'Компрессор переносной'],
                  materials: ['Топливо дизельное Л-0,2-62', 'Керосин ТС-1', 'Фильтры топливные тонкой очистки', 'Прокладки (комплект)', 'Ветошь обтирочная', 'Моющее средство МС-37']
                }
              }
            },

            /* === КУХНЯ ПРИЦЕПНАЯ КП-130-11Л === */
            {
              id: 'kp-130',
              name: 'Кухня прицепная КП-130-11Л',
              description: 'Кухня прицепная (по учебному пособию — КП-130, КП-130-11М) для приготовления горячей пищи в полевых условиях, а также в железнодорожных вагонах и на судах, на 130 человек, как на месте, так и в движении. Оборудована четырьмя котлами из нержавеющей стали.',
              specs: {
                groups: [
                  { title: 'КП-130, КП-130-11М', params: [
                    { label: 'База монтажа', value: 'специальный кухонный прицеп ГКБ-8912' },
                    { label: 'Количество питающихся, чел.', value: '130' },
                    { label: 'Котлы: количество, шт.', value: '4' },
                    { label: 'Котлы: материал', value: 'сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '2×100/80' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '85/75' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для кипятка', value: '100/80' },
                    { label: 'Масло в рубашке котла 2-го блюда: марка / масса, кг.', value: 'МТ-16п / 13' },
                    { label: 'Водонагреватель: количество, шт. / вместимость, л.', value: '1 / 36/34' },
                    { label: 'Жарочный шкаф: количество, шт. / размеры, мм', value: '1 / 430x260x95' },
                    { label: 'Форсунка: тип / количество, шт.', value: 'ФК-01 / 2/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л. / дополнительная ёмкость, л.', value: '2 / 48 / 65' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого, кг/ч', value: '12 / 28-32' }
                  ]}
                ]
              },
```

1. Шина приложения и справочник технических средств (index.html, скрипт 1) (продолжение)

```
              maintenance: {
                so: {
                  actions: [
                    'Подготовка ходовой части прицепа к сезонной эксплуатации',
                    'Проверка и регулировка давления в шинах под сезон',
                    'Перевод горелок на летний/зимний режим работы',
                    'Очистка котлов и топочного отделения от нагара',
                    'Замена смазки в ступицах колёс прицепа',
                    'Проверка тормозной системы прицепа и стояночного тормоза',
                    'Покраска и антикоррозионная обработка рамы и кузова'
                  ],
                  labor: 9.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь по ремонту топливной аппаратуры', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр шинный', 'Шприц для смазки', 'Скребок', 'Кисть малярная'],
                  materials: ['Смазка ЛИТОЛ-24', 'Краска защитная ХВ-518', 'Грунт ГФ-021', 'Ветошь', 'Керосин ТС-1']
                },
                to1: {
                  actions: [
                    'Внешний осмотр узлов и агрегатов прицепа',
                    'Проверка состояния сцепного устройства',
                    'Очистка горелок и форсунок от засорений',
                    'Контроль крепления котлов в кожухе',
                    'Проверка работы дымоходных заслонок',
                    'Смазка осей колёс и шарниров сцепки'
                  ],
                  labor: 3.0,
                  personnel: [P('Повар', 1), P('Слесарь', 1)],
                  tools: ['Набор ключей', 'Шприц для смазки', 'Скребок', 'Щётка металлическая'],
                  materials: ['Солидол синтетический', 'Ветошь обтирочная', 'Керосин ТС-1']
                },
                to2: {
                  actions: [
                    'Разборка горелок и дефектация форсунок',
                    'Проверка герметичности котлов опрессовкой',
                    'Замена прокладок крышек котлов',
                    'Регулировка тормозной системы прицепа',
                    'Проверка работы редуктора подачи топлива',
                    'Замена сальников в ступицах колёс',
                    'Балансировка колёс при наличии биения'
                  ],
                  labor: 7.5,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь по ремонту топливной аппаратуры', 1)],
                  tools: ['Набор слесарного инструмента', 'Динамометрический ключ', 'Опрессовочный насос', 'Съёмник подшипников', 'Манометр'],
                  materials: ['Прокладки уплотнительные (комплект)', 'Сальники колёс', 'Тормозные накладки', 'Смазка ЛИТОЛ-24', 'Топливо дизельное']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр ходовой части прицепа',
                    'Замена подшипников ступиц колёс',
                    'Полная ревизия топочного отделения',
                    'Замена теплоизоляции при необходимости',
                    'Восстановление лакокрасочного покрытия',
                    'Проверка и регулировка тормозных механизмов',
                    'Контрольный заезд с замером тормозного пути'
                  ],
                  labor: 14.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь по ремонту автомобилей', 1), P('Слесарь по ремонту топливной аппаратуры', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Домкрат гидравлический', 'Балансировочный станок', 'Сварочный аппарат', 'Набор съёмников'],
                  materials: ['Подшипники ступиц (комплект)', 'Теплоизоляция МКРР-130', 'Эмаль НЦ-132', 'Грунт ГФ-021', 'Электроды сварочные', 'Смазка ЛИТОЛ-24']
                },
                nz: {
                  actions: [
                    'Удаление консервационной смазки',
                    'Расконсервация топливной системы',
                    'Проверка комплектности по описи',
                    'Контрольная варка пробного блюда',
                    'Опробование ходовой части на полигоне',
                    'Дозаправка системы топливом и водой',
                    'Замена расходных материалов'
                  ],
                  labor: 11.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь', 1), P('Механик-водитель', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр', 'Опрессовочный насос', 'Компрессор'],
                  materials: ['Топливо дизельное Л-0,2-62', 'Керосин ТС-1', 'Фильтры топливные', 'Прокладки', 'Ветошь']
                }
              }
            },

            /* === КУХОННЫЙ НАБОР КН-5 === */
            {
              id: 'kn-5',
              name: 'Кухонный набор КН-5',
              description: 'Переносной кухонный набор для приготовления пищи в полевых условиях для подразделения до 5 человек. Включает котелки, горелку, посуду и инвентарь.',
```

1. Шина приложения и справочник технических средств (index.html, скрипт 1) (продолжение)

```
              maintenance: {
                so: {
                  actions: [
                    'Перевод горелки на сезонное топливо',
                    'Очистка котелков от нагара и накипи',
                    'Проверка целостности укладочного ящика',
                    'Смазка резьбовых соединений горелки',
                    'Проверка комплектности по описи'
                  ],
                  labor: 2.0,
                  personnel: [P('Повар', 1)],
                  tools: ['Набор слесарного инструмента малый', 'Щётка металлическая', 'Скребок'],
                  materials: ['Керосин ТС-1', 'Сода кальцинированная', 'Смазка ЛИТОЛ-24', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр и чистка комплекта',
                    'Промывка котелков горячей водой',
                    'Проверка работы запорных кранов горелки',
                    'Контроль уплотнений горелки'
                  ],
                  labor: 1.0,
                  personnel: [P('Повар', 1)],
                  tools: ['Щётка', 'Скребок', 'Ключ для горелки'],
                  materials: ['Моющее средство', 'Ветошь', 'Керосин ТС-1']
                },
                to2: {
                  actions: [
                    'Разборка горелки с очисткой форсунок',
                    'Замена уплотнений горелки',
                    'Проверка целостности котелков на герметичность',
                    'Заправка горелки топливом'
                  ],
                  labor: 2.5,
                  personnel: [P('Повар', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Игла для прочистки форсунок'],
                  materials: ['Уплотнения горелки', 'Керосин ТС-1', 'Прокладки', 'Ветошь']
                },
                to3: {
                  actions: [
                    'Полная разборка горелки с заменой изношенных деталей',
                    'Замена резиновых уплотнений',
                    'Восстановление лакокрасочного покрытия котелков',
                    'Дефектация и замена комплектующих'
                  ],
                  labor: 4.5,
                  personnel: [P('Повар', 1), P('Слесарь', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Опрессовочный насос ручной'],
                  materials: ['Запчасти горелки (комплект)', 'Эмаль термостойкая', 'Уплотнения', 'Керосин ТС-1']
                },
                nz: {
                  actions: [
                    'Удаление консервационной смазки',
                    'Проверка комплектности по описи',
                    'Опробование горелки',
                    'Замена расходных материалов'
                  ],
                  labor: 2.0,
                  personnel: [P('Повар', 1)],
                  tools: ['Набор слесарного инструмента малый'],
                  materials: ['Керосин ТС-1', 'Ветошь', 'Уплотнения горелки']
                }
              }
            },

            /* === КУХОННЫЙ НАБОР КН-10 === */
            {
              id: 'kn-10',
              name: 'Кухонный набор КН-10',
              description: 'Переносной кухонный набор для приготовления пищи в полевых условиях для подразделения до 10 человек. Расширенная комплектация с двумя горелками.',
              maintenance: {
                so: {
                  actions: [
                    'Перевод обеих горелок на сезонное топливо',
                    'Очистка увеличенных котелков от нагара и накипи',
                    'Проверка целостности укладочного ящика и ремней',
                    'Смазка всех резьбовых соединений',
                    'Проверка комплектности по описи (двойной комплект посуды)'
                  ],
                  labor: 3.0,
                  personnel: [P('Повар', 1)],
                  tools: ['Набор слесарного инструмента', 'Щётка металлическая', 'Скребок'],
                  materials: ['Керосин ТС-1', 'Сода кальцинированная', 'Смазка ЛИТОЛ-24', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр всего комплекта',
                    'Промывка котелков и термосов горячей водой',
                    'Проверка работы запорных кранов на обеих горелках',
```

```
                    'Контроль уплотнений горелок и термосов'
                  ],
                  labor: 1.5,
                  personnel: [P('Повар', 1)],
                  tools: ['Щётка', 'Скребок', 'Ключ для горелки'],
                  materials: ['Моющее средство', 'Ветошь', 'Керосин ТС-1']
                },
                to2: {
                  actions: [
                    'Разборка обеих горелок с очисткой форсунок',
                    'Замена уплотнений на горелках и термосах',
                    'Проверка герметичности термосов',
                    'Заправка горелок топливом и опрессовка'
                  ],
                  labor: 3.5,
                  personnel: [P('Повар', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Игла для прочистки', 'Манометр'],
                  materials: ['Уплотнения горелки (двойной комплект)', 'Керосин ТС-1', 'Прокладки', 'Ветошь']
                },
                to3: {
                  actions: [
                    'Полная разборка горелок с заменой изношенных деталей',
                    'Замена резиновых уплотнений термосов',
                    'Восстановление лакокрасочного покрытия посуды',
                    'Дефектация и замена комплектующих',
                    'Проверка вакуумной изоляции термосов'
                  ],
                  labor: 6.0,
                  personnel: [P('Повар-инструктор', 1), P('Слесарь', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Опрессовочный насос', 'Вакуумметр'],
                  materials: ['Запчасти горелок (комплект)', 'Эмаль термостойкая', 'Уплотнения', 'Керосин ТС-1']
                },
                nz: {
                  actions: [
                    'Удаление консервационной смазки со всех узлов',
                    'Проверка комплектности по описи',
                    'Опробование обеих горелок поочерёдно',
                    'Контрольная варка пробной порции',
                    'Замена расходных материалов'
                  ],
                  labor: 3.0,
                  personnel: [P('Повар', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр'],
                  materials: ['Керосин ТС-1', 'Ветошь', 'Уплотнения горелки (двойной)']
                }
              }
            },

            /* === УСТРОЙСТВА ДЛЯ СЖИГАНИЯ ЖИДКОГО ТОПЛИВА === */
            {
              id: 'fuel-burners',
              name: 'Устройства для сжигания жидкого топлива',
              type: 'subgroup',
              children: [

                /* === ГОРЕЛКА КУХОННАЯ ГК-5 === */
                {
                  id: 'gk-5',
                  name: 'Горелка кухонная ГК-5',
                  description: 'Универсальная топливная горелка с распылением жидкого топлива (керосина, дизельного) под избыточным давлением воздуха. Применяется в составе кухонь КА-220 и ПАК-200М.',
                  maintenance: {
                    so: {
                      actions: [
                        'Перевод горелки на летний/зимний режим работы',
                        'Замена сезонного топлива и продувка системы',
                        'Регулировка соотношения «топливо–воздух»',
                        'Проверка плотности соединений топливопровода',
                        'Очистка воздушного фильтра компрессора'
                      ],
                      labor: 2.5,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1)],
                      tools: ['Набор ключей', 'Манометр МТП-100', 'Иглы для прочистки форсунок'],
                      materials: ['Керосин ТС-1', 'Топливо дизельное Л-0,2-62', 'Уплотнительные прокладки', 'Ветошь']
                    },
                    to1: {
                      actions: [
                        'Внешний осмотр горелки',
                        'Очистка форсунки от засорений',
                        'Проверка давления подачи воздуха',
                        'Контроль герметичности соединений',
                        'Промывка топливного фильтра'
                      ],
                      labor: 1.0,
                      personnel: [P('Слесарь', 1)],
                      tools: ['Игла для прочистки форсунок', 'Манометр', 'Ключ рожковый'],
                      materials: ['Керосин ТС-1', 'Прокладки', 'Ветошь']
                    },
                    to2: {
                      actions: [
```

```
                        'Полная разборка горелки',
                        'Дефектация форсунки и сопла',
                        'Замена уплотнительных колец',
                        'Регулировка форсунки на стенде',
                        'Проверка работы запорного клапана',
                        'Опрессовка топливной магистрали'
                      ],
                      labor: 3.0,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1)],
                      tools: ['Набор слесарного инструмента', 'Стенд испытания форсунок', 'Манометр', 'Динамометрический ключ'],
                      materials: ['Форсунка запасная', 'Уплотнительные кольца (комплект)', 'Прокладки', 'Керосин ТС-1', 'Дизельное топливо']
                    },
                    to3: {
                      actions: [
                        'Капитальный осмотр горелки',
                        'Замена форсунки и сопла на новые',
                        'Замена всех уплотнений',
                        'Восстановление лакокрасочного покрытия корпуса',
                        'Проверка и регулировка на эксплуатационных режимах',
                        'Замена воздушного фильтра компрессора'
                      ],
                      labor: 5.5,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1), P('Слесарь-инструментальщик', 1)],
                      tools: ['Полный набор слесарного инструмента', 'Стенд испытания форсунок', 'Манометр контрольный'],
                      materials: ['Форсунка комплектная', 'Сопло запасное', 'Уплотнения (полный комплект)', 'Краска термостойкая', 'Воздушный фильтр']
                    },
                    nz: {
                      actions: [
                        'Удаление консервационной смазки',
                        'Расконсервация форсунки и топливных каналов',
                        'Заправка системы топливом',
                        'Опробование на холостом и рабочем режимах',
                        'Регулировка давления подачи'
                      ],
                      labor: 2.0,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1)],
                      tools: ['Набор слесарного инструмента', 'Манометр', 'Игла для прочистки'],
                      materials: ['Керосин ТС-1', 'Прокладки', 'Уплотнительные кольца', 'Ветошь']
                    }
                  }
                },

                /* === ФОРСУНКИ КУХОННЫЕ ФК-01 И ФК-02 === */
                {
                  id: 'fk-01-02',
                  name: 'Форсунки кухонные ФК-01 и ФК-02',
                  description: 'Топливные форсунки с механическим (ФК-01) и пневматическим (ФК-02) распылением для горелок кухонного оборудования полевых кухонь.',
                  maintenance: {
                    so: {
                      actions: [
                        'Регулировка форсунок под сезонную вязкость топлива',
                        'Замена сезонных уплотнительных колец',
                        'Контроль факела распыления',
                        'Проверка пропускной способности'
                      ],
                      labor: 1.5,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1)],
                      tools: ['Стенд испытания форсунок', 'Манометр', 'Набор регулировочных шайб'],
                      materials: ['Уплотнительные кольца', 'Керосин ТС-1', 'Ветошь']
                    },
                    to1: {
                      actions: [
                        'Очистка распылителя от засорений',
                        'Проверка факела распыления визуально',
                        'Контроль герметичности корпуса',
                        'Замена топливного фильтра тонкой очистки'
                      ],
                      labor: 0.8,
                      personnel: [P('Слесарь', 1)],
                      tools: ['Игла для прочистки', 'Лупа', 'Ключ рожковый'],
                      materials: ['Фильтр топливный', 'Керосин ТС-1', 'Ветошь']
                    },
                    to2: {
                      actions: [
                        'Полная разборка форсунки',
                        'Притирка иглы по корпусу',
                        'Проверка хода иглы и герметичности',
                        'Регулировка давления открытия',
                        'Замена уплотнений (комплект)'
                      ],
                      labor: 2.5,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1)],
                      tools: ['Набор слесарного инструмента', 'Стенд испытания форсунок КП-1809А', 'Манометр контрольный', 'Микрометр', 'Притирочный материал'],
                      materials: ['Уплотнения (полный комплект)', 'Притирочная паста', 'Керосин ТС-1', 'Регулировочные
```

```
шайбы']
                    },
                    to3: {
                      actions: [
                        'Полная дефектация форсунки',
                        'Замена иглы и распылителя (комплект)',
                        'Восстановление гидроплотности',
                        'Регулировка на эксплуатационных режимах',
                        'Проверка качества распыления на стенде',
                        'Маркировка с указанием даты ремонта'
                      ],
                      labor: 4.5,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1), P('Слесарь-инструментальщик', 1)],
                      tools: ['Полный набор слесарного инструмента', 'Стенд КП-1809А', 'Микрометр', 'Манометр контрольный'],
                      materials: ['Распылитель в сборе (комплект)', 'Игла форсунки', 'Уплотнения (полный комплект)', 'Притирочная паста', 'Керосин ТС-1']
                    },
                    nz: {
                      actions: [
                        'Расконсервация форсунок',
                        'Проверка пропускной способности на стенде',
                        'Проверка факела распыления',
                        'Регулировка давления открытия',
                        'Замена уплотнений с истёкшим сроком'
                      ],
                      labor: 1.5,
                      personnel: [P('Слесарь по ремонту топливной аппаратуры', 1)],
                      tools: ['Стенд испытания форсунок', 'Манометр', 'Игла для прочистки'],
                      materials: ['Уплотнения', 'Керосин ТС-1', 'Ветошь']
                    }
                  }
                }
              ]
            },

            /* === КП-20 (КП-30), данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'kp-20-30',
              name: 'Кухня переносная КП-20 (КП-30)',
              description: 'Предназначена для приготовления горячей пищи из 2-х блюд и кипятка в полевых условиях: КП-20 — на 20 человек, КП-30 — на 30 человек. Применение: КП-30 — воинским подразделениям, находящимся временно или постоянно в отрыве от своих частей, на каждые 30 человек по 1 шт.; КП-20 — подразделениям численностью до 20 человек, по 1 шт.',
              specs: {
                groups: [
                  { title: 'КП-30', params: [
                    { label: 'Количество питающихся, чел.', value: '30' },
                    { label: 'Котлы: количество, шт. / материал', value: '3 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '24/20' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '17/15' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для кипятка', value: '11,5/10' },
                    { label: 'Масло в рубашке для котла 2-го блюда: марка / масса, кг.', value: 'МТ-16п / 4,5' },
                    { label: 'Форсунка: тип / количество (установленная/запасная)', value: 'ФК-02 / 1/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: 'установлен в тепловом блоке / 1 / 15' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого (дрова влажностью 25%), кг/ч', value: '2 / 11' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '30-40' },
                    { label: 'Масса сухая, кг.', value: '106' }
                  ]},
                  { title: 'КП-20', params: [
                    { label: 'Количество питающихся, чел.', value: '20' },
                    { label: 'Котлы: количество, шт. / материал', value: '3 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '17,5/16,0' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '13,5/10,0' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для кипятка', value: '17,5/16,0' },
                    { label: 'Масло в рубашке для котла 2-го блюда: марка / масса, кг.', value: 'МТ-16п / 3' },
                    { label: 'Форсунка: тип / количество (установленная/запасная)', value: 'ФК-02 / 1/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: 'установлен в тепловом блоке / 1 / 16' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого (дрова влажностью 25%), кг/ч', value: '2 / 6' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '20-30' },
                    { label: 'Масса сухая, кг.', value: '108' }
                  ]}
                ]
              }
            },

            /* === МК-10 (КП-10), данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'mk-10',
              name: 'Кухня переносная МК-10 (КП-10)',
              description: 'Предназначена для приготовления горячей пищи в полевых условиях, а также в железнодорожных вагонах, из 2-х блюд и кипятка на 10 человек питающихся. Применение: для обеспечения малочисленных подразделений и команд численностью до 10 человек.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '10' },
                    { label: 'Котлы: количество, шт. / материал', value: '3 / алюминий' },
```

```
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го, 2-го блюда, для кипятка', value: '5/4,5/4,5/4' },
                    { label: 'Масло в рубашке для котла 2-го блюда: марка / масса, кг.', value: 'МТ-16п / 3' },
                    { label: 'Форсунка: тип / количество (установленных/запасных)', value: 'КГФ-4М / 1/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: 'топл. колонка / 1/8' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого (дрова влажностью 25%), кг/ч', value: '0,6-2,0 / 9' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '20-25' },
                    { label: 'Масса сухая, кг.', value: '44' }
                  ]}
                ]
              }
            },

            /* === МК-30, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'mk-30',
              name: 'Кухня переносная МК-30',
              description: 'Предназначена для приготовления пищи в полевых условиях, а также в железнодорожных вагонах и на судах при перевозке воинских подразделений, из 2-х блюд на 30 человек питающихся. Применение: воинским подразделениям, находящимся временно или постоянно в отрыве от своих частей, на каждые 30 человек по 1 шт.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '30' },
                    { label: 'Котлы: количество, шт. / материал', value: '2 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '26/22' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '16/14' },
                    { label: 'Масло в рубашке для котла 2-го блюда: марка / масса, кг.', value: 'АК-15 / 3,0' },
                    { label: 'Форсунка: тип / количество (установленных/запасных)', value: 'КГФ-4М / 1/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: 'топливная колонка / 1 / 8' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого (дрова влажностью 25%), кг/ч', value: '0,6-2,0 / 9' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '25-30' },
                    { label: 'Масса сухая, кг.', value: '83,5' }
                  ]}
                ]
              }
            },

            /* === КО-75, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'ko-75',
              name: 'Кухня переносная КО-75',
              description: 'Предназначена для приготовления пищи в полевых условиях, а также в железнодорожных вагонах и на судах при перевозке воинских подразделений, из двух блюд на 75 человек. Состав: тепловой блок, дымовая труба, котёл для первого блюда, котёл для второго блюда, водонагреватель, жарочный шкаф, салазки, вентиль форсунки, резинотканевый топливопровод, топливная установка, ручной насос, щит с манометром. Применение: продовольственным складам, прачечным отрядам, хлебозаводам ПМХ, мясопунктам ПМ-40, ремонтным мастерским — на каждые 75 человек 1 штука.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '75' },
                    { label: 'Котлы: количество, шт. / материал', value: '2 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда / для 2-го блюда', value: '56/53 / 40/34' },
                    { label: 'Водонагреватель: количество, шт. / вместимость, л.', value: '1 / 30/26' },
                    { label: 'Масло в рубашке для котла 2-го блюда: марка / масса, кг.', value: 'МТ-16п / 7' },
                    { label: 'Жарочный шкаф: количество, шт. / размеры, мм', value: '1 / 340x277x257' },
                    { label: 'Форсунка: тип / количество (установленных/запасных)', value: 'ФК-01 / 1/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: 'топливная установка / 1 / 48' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого (дрова влажностью 25%), кг/ч', value: '6 / 18' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '30-35' },
                    { label: 'Масса сухая, кг.', value: '210' }
                  ]}
                ]
              }
            },

            /* === КВ-75Д, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'kv-75d',
              name: 'Кухня переносная КВ-75Д',
              description: 'Предназначена для приготовления пищи в полевых условиях, а также в железнодорожных вагонах и на судах при перевозке воинских подразделений, из двух блюд на 75 человек. В отличие от кухни КО-75М не имеет жарочного шкафа и водонагревателя, корпус усилен и приспособлен для десантирования в кузове автомобиля ГАЗ-66Б; топливная система вынесена в виде топливной колонки ёмкостью 24 литра, также приспособленной для десантирования. Применение: подразделениям ВДВ на 80 человек по 1 штуке.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '75' },
                    { label: 'Котлы: количество, шт. / материал', value: '2 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '56/53' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '40/34' },
                    { label: 'Масло в рубашке для котла 2-го блюда: марка / масса, кг.', value: 'МТ-16п / 7' },
                    { label: 'Форсунка: тип / количество (установленных/запасных)', value: 'ФК-01 / 1/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: 'топливная колонка / 1 / 24' },
```

```
                    { label: 'Расход топлива: жидкого, л/ч / твердого (дрова влажностью 25%), кг/ч', value: '6 / 18' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '25-30' },
                    { label: 'Масса сухая, кг.', value: '170' }
                  ]}
                ]
              }
            },

            /* === ОПК-150, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'opk-150',
              name: 'Очажный пищеварочный котёл ОПК-150',
              description: 'Предназначен для приготовления горячей пищи в полевых условиях из одного блюда на 150 человек питающихся. Применение: для подразделений на марше, на местах стоянки. Учебное пособие отмечает: данный вид снят с производства и не используется в настоящее время.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '150' },
                    { label: 'Котлы: количество, шт. / материал', value: '1 / чугун' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '150/130' },
                    { label: 'Расход топлива: твердого (дрова влажностью 25%), кг/ч', value: '40' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '40-50' },
                    { label: 'Масса сухая, кг.', value: '125' }
                  ]}
                ]
              }
            },

            /* === МВК-50, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'mvk-50',
              name: 'Малолитражная вьючная кухня МВК-50',
              description: 'Предназначена для приготовления горячей пищи в полевых условиях, а также в железнодорожных вагонах и на судах при перевозках личного состава, из 1-го блюда и кипятка на 25 человек питающихся. Применение: выдаётся подразделениям, выполняющим задачи в труднодоступных и труднопроходимых районах, где используется вьючный транспорт, как правило в горных местностях.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '25' },
                    { label: 'Котлы: количество, шт. / материал', value: '1 / сталь' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '25/21' },
                    { label: 'Масса сухая, кг.', value: '16' }
                  ]}
                ]
              }
            },

            /* === КП-2-48, КП-2-49, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'kp-2-48',
              name: 'Кухня прицепная КП-2-48, КП-2-49',
              description: 'Предназначена для приготовления горячей пищи в полевых условиях, а также в железнодорожных вагонах и на судах при перевозках личного состава — трёхразовое питание на 150 (КП-2-48) / 210 (КП-2-49) человек, как на месте, так и в движении. Применение: мотострелковым, танковым, зенитным, зенитно-ракетным, строительным, железнодорожным частям и учреждениям.',
              specs: {
                groups: [
                  { title: 'КП-2-48', params: [
                    { label: 'База монтажа', value: 'ИАПЗ-739К' },
                    { label: 'Количество питающихся', value: '150' },
                    { label: 'Котлы: количество, шт. / материал', value: '2 / сталь нержавеющая / чугун' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '125/110' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '75/65' },
                    { label: 'Форсунка: тип / количество, шт.', value: 'КГФ-3М / 2/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: '1 / 24' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого, кг/ч', value: '7-10 / 28-32' }
                  ]},
                  { title: 'КП-2-49', params: [
                    { label: 'База монтажа', value: 'ИАПЗ-739К' },
                    { label: 'Количество питающихся', value: '210' },
                    { label: 'Котлы: количество, шт. / материал', value: '2 / сталь нержавеющая / чугун' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '170/145' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '110/95' },
                    { label: 'Форсунка: тип / количество, шт.', value: 'КГФ-3М / 2/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л.', value: '1 / 24' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого, кг/ч', value: '7-10 / 28-32' }
                  ]}
                ]
              }
            },

            /* === КП-125М, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'kp-125m',
              name: 'Кухня прицепная КП-125М',
              description: 'Предназначена для приготовления горячей пищи в полевых условиях, а также в железнодорожных вагонах и на судах при перевозках личного состава, как на месте, так и в движении. Применение: мотострелковым, танковым, зенитным, зенитно-ракетным, строительным, железнодорожным частям и учреждениям на каждые 125 чел. — 1 шт.',
```

```
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'База монтажа', value: 'ИАПЗ-739К' },
                    { label: 'Количество питающихся, чел.', value: '125' },
                    { label: 'Котлы: количество, шт. / материал', value: '3 / сталь нержавеющая' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '2×56/50' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '56/50' },
                    { label: 'Кипятильник: количество, шт. / вместимость, л.', value: '1 / 125/115' },
                    { label: 'Жарочный шкаф: количество, шт. / размеры, мм', value: '1 / 455x525x440' },
                    { label: 'Форсунка: тип / количество, шт.', value: 'ФК-01 / 2/1' },
                    { label: 'Бак топливный: количество, шт. / вместимость, л. / дополнительная ёмкость, л.', value: '2 / 48 / нет' },
                    { label: 'Расход топлива: жидкого, л/ч / твердого, кг/ч', value: '12 / 28-32' }
                  ]}
                ]
              }
            },

            /* === КПБМ-150, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'kpbm-150',
              name: 'Кухня прицепная блочно-модульная КПБМ-150',
              description: 'Предназначена для приготовления горячей пищи в полевых условиях, а также в железнодорожных вагонах и на судах при перевозках личного состава. Транспортируется автомобильным, воздушным, железнодорожным транспортом; обладает стойкими эксплуатационными характеристиками к внешним воздействующим факторам. Применение: для штатных или сводных подразделений численностью 150 человек питающихся.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '150' },
                    { label: 'Котлы: количество, шт. / материал', value: '3 / нержавеющая сталь' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 1-го блюда', value: '88/80' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для 2-го блюда', value: '88/80' },
                    { label: 'Вместимость котлов (полная/рабочая), л. — для кипятка', value: '88/80' },
                    { label: 'Бак для воды для технических нужд, шт.', value: '1' },
                    { label: 'Жарочный шкаф, шт.', value: '1' },
                    { label: 'Топливный бак: количество, шт. / объём, л.', value: '2 / 68' },
                    { label: 'Расход топлива: твердого (дрова влажностью 25%), кг/ч', value: '30' },
                    { label: 'Время закипания воды в котлах на жидком топливе, мин.', value: '40-50' },
                    { label: 'Тяговый автомобиль', value: 'Урал, КамАЗ' },
                    { label: 'Масса сухая, кг.', value: '2700' }
                  ]}
                ]
              }
            },

            /* === КСВК-240/24, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'ksvk-240',
              name: 'Кухня-столовая возимая в кузове-контейнере переменного объёма КСВК-240/24',
              description: 'Предназначена для приготовления и приёма горячей пищи в полевых условиях личным составом подразделений численностью до 240 чел. Состоит из кузова-контейнера переменного объёма с установленным оборудованием для приготовления и хранения пищи, а также комплекта мебели для организации зоны приёма пищи.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '240' },
                    { label: 'Количество посадочных мест, чел.', value: '24' },
                    { label: 'Количество обслуживающего персонала, чел.', value: '3' },
                    { label: 'Габаритные размеры в транспортном положении (длина, ширина, высота), мм', value: '6845x250x2650' },
                    { label: 'Габаритные размеры в рабочем положении (длина, ширина, высота), мм', value: '6845x6790x3000' },
                    { label: 'Снаряженная масса, кг.', value: '8500' },
                    { label: 'Полная масса, кг.', value: '10000' },
                    { label: 'Полная масса с учётом ТС (КАМАЗ-6350), кг.', value: '20000' },
                    { label: 'Возимые запасы продовольствия (сутодач), не менее', value: '1' },
                    { label: 'Возимые запасы воды в пищеварочных котлах, не менее, л.', value: '300' },
                    { label: 'Возимые запасы дизельного топлива в баках, л.', value: '48' },
                    { label: 'Полная/полезная вместимость пищеварочных котлов тепловых блоков, л.', value: '330/300' },
                    { label: 'В т.ч. для 1-го блюда, л.', value: '165/150' },
                    { label: 'В т.ч. для 2-го блюда, л.', value: '165/150' },
                    { label: 'Время разогрева воды в пищеварочных котлах от 20°С до 95°С с учётом розжига, мин', value: '70' },
                    { label: 'Расход дизельного топлива, кг/час', value: '9 (на 2 форсунки)' },
                    { label: 'Количество кипятка от двух водонагревателей тепловых блоков, л/час', value: '100' },
                    { label: 'Номинальная мощность подключаемых электропотребителей, кВт', value: '4' },
                    { label: 'Номинальное напряжение, В', value: '230' },
                    { label: 'Температура эксплуатации', value: 'от -45°С до +40°С' }
                  ]}
                ]
              }
            },

            /* === ПП-40, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'pp-40',
              name: 'Плита переносная ПП-40',
              description: 'Предназначена для приготовления пищи из двух первых, двух-трёх вторых блюд и кипятка для чая
```

```
 для офицерского состава и выздоравливающих больных полевых госпиталей. Возможность: приготовление пищи из одного первого и второго блюда для 40 питающихся.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Количество питающихся, чел.', value: '40' },
                    { label: 'Площадь жарочной поверхности, м²', value: '0,3' },
                    { label: 'Вместимость водонагревателя, л.', value: '10' },
                    { label: 'Время закипания воды, мин.', value: '55-75' },
                    { label: 'Размеры жарочного шкафа, мм', value: '365x400x432' },
                    { label: 'Форсунка: тип', value: 'ФК-02' },
                    { label: 'Топливные баки верхние: количество, шт. / вместимость, л.', value: '1 / 12' },
                    { label: 'Расход топлива: жидкого, л/ч / дров (влажностью 21-33%), кг/ч', value: '2 / 9' },
                    { label: 'Габаритные размеры (походное/рабочее положение), мм — длина', value: '1170/1250' },
                    { label: 'Габаритные размеры (походное/рабочее положение), мм — ширина', value: '520/760' },
                    { label: 'Габаритные размеры (походное/рабочее положение), мм — высота', value: '750/3300' },
                    { label: 'Масса, кг: в снаряженном состоянии', value: '130' }
                  ]}
                ]
              }
            },

            /* === ТЕРМОСЫ Т-4, ТВН-12, ТН-36, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'termosy',
              name: 'Термосы Т-4, ТВН-12, ТН-36',
              description: 'Предназначены для доставки (переноски, перевозки) и кратковременного хранения горячей пищи в полевых условиях. Блок-термос Т-4 имеет блочную конструкцию и состоит из трёх термосов Т-4, размещённых в общей кассете, что обеспечивает кратковременное хранение и переноску в полевых условиях горячей пищи из нескольких блюд. Термосы ТВН-12, ТН-36 обеспечивают доставку и кратковременное хранение в полевых условиях горячей пищи из одного блюда.',
              specs: {
                groups: [
                  { title: 'ТВН-12', params: [
                    { label: 'Бачок: количество, шт. / материал / вместимость полезная, л.', value: '1 / ст. нержав. / 12' },
                    { label: 'Время поддержания температуры пищи от 90°С до 50°С (при 15°С), ч.', value: '6' },
                    { label: 'Габаритные размеры, мм — длина', value: '344' },
                    { label: 'Габаритные размеры, мм — ширина', value: '234' },
                    { label: 'Габаритные размеры, мм — высота', value: '423' },
                    { label: 'Масса, кг: сухая / в снаряженном состоянии', value: '8 / 20' }
                  ]},
                  { title: 'ТН-36', params: [
                    { label: 'Бачок: количество, шт. / материал / вместимость полезная, л.', value: '1 / ст. нержав. / 36' },
                    { label: 'Время поддержания температуры пищи от 90°С до 50°С (при 15°С), ч.', value: '6' },
                    { label: 'Габаритные размеры, мм — длина', value: '492' },
                    { label: 'Габаритные размеры, мм — ширина', value: '393' },
                    { label: 'Габаритные размеры, мм — высота', value: '670' },
                    { label: 'Масса, кг: сухая / в снаряженном состоянии', value: '17 / 43' }
                  ]},
                  { title: 'Т-4', params: [
                    { label: 'Бачок: количество, шт. / материал / вместимость полезная, л.', value: '1 / алюмин. / 4' },
                    { label: 'Время поддержания температуры пищи от 90°С до 50°С (при 15°С), ч.', value: '6' },
                    { label: 'Габаритные размеры, мм — длина', value: '185' },
                    { label: 'Габаритные размеры, мм — ширина', value: '215' },
                    { label: 'Габаритные размеры, мм — высота', value: '312' },
                    { label: 'Масса, кг: сухая / в снаряженном состоянии', value: '2,4 / 6,4' }
                  ]}
                ]
              }
            }
          ]
        },

        /* ---------- ГРУППА 2: Полевое хлебопечение ---------- */
        {
          id: 'food-bakery',
          name: 'Технические средства полевого хлебопечения',
          type: 'group',
          children: [

            /* === БЛОК ХЛЕБОПЕКАРНЫЙ ПРИЦЕПНОЙ ПХБ-0,4 === */
            {
              id: 'phb-04',
              name: 'Блок хлебопекарный прицепной ПХБ-0,4',
              description: 'Прицепной хлебопекарный блок (учебное пособие описывает также вариант ПХБ-1,0) предназначен для выработки формового пшеничного и ржаного хлеба в полевых условиях. Смонтирован в закрытом кузове-фургоне на базовом шасси прицепа.',
              specs: {
                groups: [
                  { title: 'ПХБ-0,4', params: [
                    { label: 'Производительность, т/сут. — по ржаному хлебу массой 1,7 кг', value: '0,435 кг' },
                    { label: 'Производительность, т/сут. — по пшеничному хлебу массой 1,2 кг', value: '0,300 кг' },
                    { label: 'Масса смонтированного оборудования, кг', value: '1323 кг' },
                    { label: 'Базовое шасси', value: 'прицеп СМЗ-8325, кузов-фургон КП-2' }
                  ]}
                ]
              },
              maintenance: {
```

```
                so: {
                  actions: [
                    'Подготовка ходовой части прицепа к сезону',
                    'Перевод печи на сезонный режим работы',
                    'Очистка пекарной камеры и расстойного шкафа',
                    'Смазка ходовой части и привода тестомеса',
                    'Проверка тормозной системы прицепа',
                    'Регулировка горелочного устройства',
                    'Антикоррозионная обработка форм для выпечки'
                  ],
                  labor: 11.0,
                  personnel: [P('Хлебопёк-инструктор', 1), P('Слесарь по ремонту топливной аппаратуры', 1), P('Механик-водитель', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр шинный', 'Шприц для смазки', 'Щётка пекарная', 'Термометр контактный'],
                  materials: ['Смазка ЛИТОЛ-24', 'Масло пищевое для смазки форм', 'Краска защитная', 'Уплотнения', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр блока и прицепа',
                    'Очистка пекарной камеры от мучной пыли',
                    'Проверка работы привода тестомесильной машины',
                    'Контроль температуры в расстойном шкафу',
                    'Смазка шарниров и подшипников',
                    'Проверка сцепного устройства прицепа'
                  ],
                  labor: 4.0,
                  personnel: [P('Хлебопёк', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Шприц для смазки', 'Термометр контактный', 'Щётка пекарная'],
                  materials: ['Смазка пищевая', 'Ветошь', 'Моющее средство пищевое МС-37']
                },
                to2: {
                  actions: [
                    'Ревизия тестомесильной машины с разборкой',
                    'Проверка герметичности пекарной камеры',
                    'Регулировка терморегуляторов и температурных датчиков',
                    'Замена уплотнений на дверцах печи и шкафа',
                    'Проверка электрооборудования и заземления',
                    'Балансировка месильного органа',
                    'Чистка дымовых каналов'
                  ],
                  labor: 9.0,
                  personnel: [P('Хлебопёк-инструктор', 1), P('Слесарь', 1), P('Электрик', 1)],
                  tools: ['Набор слесарного инструмента', 'Мультиметр', 'Термометр контрольный', 'Мегаомметр', 'Динамометрический ключ'],
                  materials: ['Уплотнения дверцы печи (комплект)', 'Прокладки', 'Смазка ЛИТОЛ-24', 'Смазка пищевая', 'Изоляционная лента']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр всего блока',
                    'Замена подшипников тестомеса',
                    'Полная ревизия электрооборудования',
                    'Замена теплоизоляции печи при необходимости',
                    'Восстановление лакокрасочного покрытия',
                    'Проверка ходовой части прицепа с заменой подшипников',
                    'Контрольная выпечка с замером параметров'
                  ],
                  labor: 22.0,
                  personnel: [P('Хлебопёк-инструктор', 1), P('Слесарь по ремонту автомобилей', 1), P('Электрик', 1), P('Сварщик', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Балансировочный станок', 'Сварочный аппарат', 'Мегаомметр', 'Тестер изоляции'],
                  materials: ['Подшипники (комплект)', 'Теплоизоляция', 'Электроды сварочные', 'Эмаль термостойкая', 'Кабель силовой', 'Смазка пищевая']
                },
                nz: {
                  actions: [
                    'Расконсервация всех узлов и агрегатов',
                    'Проверка комплектности по описи',
                    'Опробование тестомеса на холостом ходу',
                    'Прогрев пекарной камеры',
                    'Контрольная выпечка пробной партии',
                    'Замена расходных материалов с истёкшим сроком'
                  ],
                  labor: 14.0,
                  personnel: [P('Хлебопёк-инструктор', 1), P('Слесарь', 1), P('Электрик', 1)],
                  tools: ['Набор слесарного инструмента', 'Мультиметр', 'Термометр', 'Манометр'],
                  materials: ['Смазка пищевая', 'Уплотнения', 'Ветошь', 'Дрожжи прессованные (для контрольной выпечки)']
                }
              }
            },

            /* === ОТДЕЛЕНИЕ ПОДВИЖНОГО ХЛЕБОЗАВОДА ПХК-7 === */
            {
              id: 'phk-7',
              name: 'Отделение подвижного хлебозавода в кузовах-контейнерах ПХК-7',
              description: 'Передвижной хлебозавод для производства в полевых условиях формового хлеба из пшеничной и ржано-пшеничной муки. Мобильный хлебозавод в составе нескольких модулей (кузовов-контейнеров), развёртываемых в местах
```

```
 дислокации войск. Суточная производительность — от 7000 до 8000 кг хлеба.',
            specs: {
              groups: [
                { title: null, params: [
                  { label: 'Размер площадки для развёртывания, м. — при минимальном удалении от электростанций (15 м) и жилого модуля (2 м), не менее', value: '30x43' },
                  { label: 'Размер площадки для развёртывания, м. — при максимальном удалении от электростанций (40 м) и жилого модуля (20 м), не менее', value: '30x83' },
                  { label: 'Суточная производительность выпечки формового хлеба (пшеничного массой 0,75 кг и ржано-пшеничного массой от 0,8 до 0,85 кг), кг', value: 'от 7000 до 8000' },
                  { label: 'Продолжительность непрерывной работы (при сменной организации персонала и технологических перерывах), ч/сут', value: '24' },
                  { label: 'Численность одной смены, чел.', value: '10-12' },
                  { label: 'Время развёртывания хлебозавода, ч.', value: '2,5-3' },
                  { label: 'Время первой выпечки белого хлеба ускоренным способом, ч.', value: '4' },
                  { label: 'Производительность мукопросеивателя, кг/ч', value: '1500' },
                  { label: 'Масса модулей без автотранспорта, кг.', value: 'до 7000' },
                  { label: 'Количество автотранспорта для перевозки модулей, шт.', value: '5' },
                  { label: 'Количество мест для отдыха в жилом модуле, шт.', value: '15' },
                  { label: 'Базовые автотранспортные средства — автомобили', value: 'КамАЗ-6350 (8x8), Урал-532361 (8x8)' },
                  { label: 'Базовые автотранспортные средства — прицепы', value: 'ЧМЗАП-83357 (2-ПН-8)' },
                  { label: 'Условия эксплуатации (температура окружающей среды)', value: 'от -40 до +45°C' },
                  { label: 'Электроснабжение', value: 'от прицепных дизельных электростанций, других передвижных источников или стационарной сети переменного трёхфазного тока напряжением 400/230 (380/220) В, частотой 50 Гц' }
                ]}
              ]
            },
            maintenance: {
              so: {
                actions: [
                  'Подготовка автономных систем энергоснабжения к сезону',
                  'Сезонная регулировка пекарных печей',
                  'Очистка систем вентиляции и кондиционирования контейнеров',
                  'Перевод водоснабжения на сезонный режим (утепление)',
                  'Регулировка дизель-генераторов',
                  'Проверка тепло- и звукоизоляции контейнеров',
                  'Регулировка тестоприготовительных линий'
                ],
                labor: 28.0,
                personnel: [P('Хлебопёк-инструктор', 2), P('Дизелист', 1), P('Электрик', 1), P('Слесарь', 2)],
                tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Мегаомметр', 'Манометр', 'Термометр', 'Шумомер'],
                materials: ['Уплотнения дверей контейнеров', 'Смазка ЛИТОЛ-24', 'Масло пищевое', 'Изоляционные материалы', 'Антифриз ОЖ-65', 'Фильтры воздушные']
              },
              to1: {
                actions: [
                  'Внешний осмотр всех контейнеров',
                  'Очистка пекарных камер от мучной пыли',
                  'Проверка работы тестомесов и формовщиков',
                  'Контроль параметров расстойных шкафов',
                  'Проверка работы дизель-генераторов под нагрузкой',
                  'Очистка систем вентиляции'
                ],
                labor: 10.0,
                personnel: [P('Хлебопёк', 2), P('Дизелист', 1), P('Электрик', 1)],
                tools: ['Набор слесарного инструмента', 'Мультиметр', 'Термометр', 'Щётки технические'],
                materials: ['Смазка пищевая', 'Ветошь', 'Моющее средство МС-37', 'Фильтры воздушные']
              },
              to2: {
                actions: [
                  'Ревизия тестомесильных машин',
                  'Замена подшипников делителей теста',
                  'Регулировка температурных режимов печей',
                  'Проверка автоматики и средств контроля',
                  'Дефектация электрооборудования',
                  'Замена масла в дизель-генераторах',
                  'Проверка систем пожарной сигнализации'
                ],
                labor: 22.0,
                personnel: [P('Хлебопёк-инструктор', 2), P('Слесарь', 2), P('Электрик', 1), P('Дизелист', 1)],
                tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Мегаомметр', 'Термометр контрольный', 'Балансировочный станок'],
                materials: ['Подшипники (комплект)', 'Масло моторное М10Г2К', 'Фильтры масляные и топливные', 'Уплотнения', 'Смазка пищевая', 'Электроды сварочные']
              },
              to3: {
                actions: [
                  'Капитальный осмотр всех технологических линий',
                  'Замена изношенных узлов тестомесов',
                  'Полная ревизия дизель-генераторов',
                  'Замена теплоизоляции печей',
                  'Восстановление лакокрасочного покрытия контейнеров',
                  'Проверка и замена силовых кабелей',
                  'Контрольная выпечка с замером всех параметров',
                  'Балансировка валов и крыльчаток'
                ],
                labor: 56.0,
                personnel: [P('Хлебопёк-инструктор', 2), P('Слесарь по ремонту автомобилей', 2), P('Дизелист', 1),
```

```
      P('Электрик', 2), P('Сварщик', 1)],
1240                  tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Балансировочный станок', 'Мегаомметр', 'Стенд испытания форсунок', 'Подъёмное оборудование'],
1241                  materials: ['Запчасти комплект тестомеса', 'Подшипники', 'Изоляция тепловая', 'Кабель силовой КГ', 'Эмаль', 'Электроды', 'Масло моторное', 'Антифриз']
1242                },
1243                nz: {
1244                  actions: [
1245                    'Расконсервация всего комплекса',
1246                    'Развёртывание контейнеров на местности',
1247                    'Подключение коммуникаций (вода, электричество)',
1248                    'Опробование всех технологических линий',
1249                    'Контрольная выпечка пробной партии',
1250                    'Проверка комплектности по описи',
1251                    'Замена расходных материалов'
1252                  ],
1253                  labor: 32.0,
1254                  personnel: [P('Хлебопёк-инструктор', 2), P('Слесарь', 2), P('Дизелист', 1), P('Электрик', 1)],
1255                  tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Манометр', 'Термометр', 'Грузоподъёмные приспособления'],
1256                  materials: ['Топливо дизельное', 'Масло моторное', 'Дрожжи прессованные', 'Мука пшеничная (для контрольной выпечки)', 'Соль', 'Ветошь', 'Уплотнения']
1257                }
1258              }
1259            },
1260
1261            /* === ПМХ, данные из учебного пособия; регламент ТО не приведён === */
1262            {
1263              id: 'pmh',
1264              name: 'Передвижной механизированный хлебозавод ПМХ',
1265              description: 'Предназначен для выпечки хлеба в полевых условиях и, как правило, является штатным для соединения. С 1958 года ПМХ стал основным хлебозаводом в Советской Армии. Включается в штаты и табели: мсд (ОБМО) на БТР (из двух отделений хлебопечения); продовольственных складов (из 3-х отделений хлебопечения). Оснащается: печь хлебопекарная прицепная ХПК-50 — 2 шт., агрегат тестоприготавливающий прицепной ТМ-3 — 1 шт., машина тесторазделяющая переносная ПТД — 1 шт., машина просеивающая переносная ПМ-60 — 1 шт., прицеп-цистерна ЦВ-1,2 — 1 шт., электростанция — 2 шт., палатка-цех для ПМХ — 1 шт., комплект инвентаря и приборов.',
1266              specs: {
1267                groups: [
1268                  { title: null, params: [
1269                    { label: 'Производительность при 16 подооборотах по хлебу из муки ржаной обойной массой 1,7 кг', value: '5,87 т/сутки' },
1270                    { label: 'Производительность при 16 подооборотах по хлебу из муки пшеничной обойной массой 1,6 кг', value: '5,5 т/сутки' },
1271                    { label: 'Производительность при 16 подооборотах по хлебу из муки пшеничной 1-го сорта массой 1,2 кг', value: '4,15 т/сутки' },
1272                    { label: 'Время развёртывания в рабочее положение на заранее подготовленной площадке — в летнее время', value: '1,5-2 часа' },
1273                    { label: 'Время развёртывания в рабочее положение на заранее подготовленной площадке — в зимнее время', value: '2-3 часа' },
1274                    { label: 'Время свёртывания в походное положение — в летнее время', value: '1-1,5 часа' },
1275                    { label: 'Время свёртывания в походное положение — в зимнее время', value: '1,5-2 часа' },
1276                    { label: 'Расход топлива на выпечку 1 т хлеба — жидкого (дизельное, керосин)', value: '64-77 кг' },
1277                    { label: 'Расход топлива на выпечку 1 т хлеба — твёрдого (дрова влажностью 21-33%)', value: '0,5-0,6 м³' },
1278                    { label: 'Площадь пола палатки-цеха', value: '84,75 м²' },
1279                    { label: 'Общая масса оборудования, палаток, принадлежностей, инвентаря и другого имущества', value: '17 250 кг' },
1280                    { label: 'Электроснабжение', value: 'от внешней сети 220/230 В' }
1281                  ]}
1282                ]
1283              }
1284            },
1285
1286            /* === ПХП-0,4, данные из учебного пособия; регламент ТО не приведён === */
1287            {
1288              id: 'php-04',
1289              name: 'Печь хлебопекарная переносная ПХП-0,4',
1290              description: 'Предназначена для расстойки теста и выпечки хлеба из ржаной, ржано-пшеничной и пшеничной муки I сорта при работе на жидком (дизельное топливо) и твёрдом топливе. Входит в состав хлебопекарного блока прицепного ПХБ-0,4.',
1291              specs: {
1292                groups: [
1293                  { title: null, params: [
1294                    { label: 'Производительность по хлебу (при 16 подооборотах в сутки), т/сутки — из муки ржаной обойной (буханка массой 1,7 кг)', value: '0,436' },
1295                    { label: 'Производительность по хлебу (при 16 подооборотах в сутки), т/сутки — из муки пшеничной 1 сорта (буханка массой 1,2 кг)', value: '0,3' },
1296                    { label: 'Количество хлебопекарных форм, расположенных одновременно в пекарной и расстойной камерах, шт.', value: '32' },
1297                    { label: 'Вид топлива', value: 'дизельное, твёрдое топливо (дрова, уголь), природный газ' },
1298                    { label: 'Расход топлива на 1 подооборот — дизельное топливо, кг.', value: '1,3-1,5' },
1299                    { label: 'Расход топлива на 1 подооборот — твёрдое топливо (дрова, уголь), кг.', value: '3-4' },
1300                    { label: 'Расход топлива на 1 подооборот — природный газ, м³/час', value: '1,5' },
1301                    { label: 'Габаритные размеры печи (длина x ширина x высота) в рабочем положении, мм', value: '1330x910x4000' },
1302                    { label: 'Масса, кг.', value: '350' }
1303                  ]}
1304                ]
1305              }
1306            },
```

```
            /* === Мельница прицепная ПМ-14, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'pm-14',
              name: 'Мельница прицепная ПМ-14',
              description: 'Предназначена для переработки зерна ржи и пшеницы в муку обойного помола без отбора отрубей в полевых условиях.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Производительность по зерну пшеницы, кг/ч', value: '800' },
                    { label: 'Выход муки из очищенного зерна, %', value: '98' },
                    { label: 'Содержание металлопримесей в 1 кг муки, мг', value: '3' }
                  ]}
                ]
              }
            },

            /* === Мясопункт прицепной ПМ-40М, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'pm-40m',
              name: 'Мясопункт прицепной ПМ-40М',
              description: 'Предназначен для убоя крупного рогатого скота, свиней и овец, а также для переработки и охлаждения туш в полевых условиях.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Производительность, голов в смену — крупного рогатого скота', value: '40-45' },
                    { label: 'Производительность, голов в смену — свиней', value: '120' },
                    { label: 'Производительность, голов в смену — овец', value: '210' },
                    { label: 'Выработка мяса', value: '4-5 тонн' }
                  ]}
                ]
              }
            }
          ]
        },

        /* ---------- ГРУППА 3: Подвоз, хранение продовольствия и воды ---------- */
        {
          id: 'food-transport',
          name: 'Технические средства подвоза, хранения продовольствия и воды',
          type: 'group',
          children: [

            /* === АФИ-53 === */
            {
              id: 'afi-53',
              name: 'Автомобиль-фургон изотермический семейства АФИ-53',
              description: 'Автомобиль с изотермическим кузовом для перевозки скоропортящихся продуктов. Сохраняет заданную температуру в течение нормативного времени без активного охлаждения. По учебному пособию к семейству относятся модели АФИ-5-4320 и АФИ-2,9.',
              specs: {
                groups: [
                  { title: 'АФИ-5-4320', params: [
                    { label: 'Масса перевозимого груза, кг.', value: '5000' },
                    { label: 'Масса кузова, кг.', value: '1760' },
                    { label: 'Масса снаряжённого фургона, кг.', value: '10070' },
                    { label: 'Полная масса автофургона, кг.', value: '15070' },
                    { label: 'Распределение нагрузки на дорогу от полной массы, кг. — на переднюю ось', value: '5800' },
                    { label: 'Распределение нагрузки на дорогу от полной массы, кг. — на заднюю ось', value: '9270' },
                    { label: 'Габаритные размеры автофургона (длина/ширина/высота), мм', value: '9330/2800/3815' },
                    { label: 'Габаритные размеры кузова (длина/ширина/высота), мм', value: '5500/2500/2205' },
                    { label: 'Внутренние размеры кузова (длина/ширина/высота), мм', value: '5340/2380/2005' },
                    { label: 'Погрузочная высота, мм', value: '1450' },
                    { label: 'Размеры проёма задней двери кузова (высота/ширина), мм', value: '2000/2355' }
                  ]},
                  { title: 'АФИ-2,9', params: [
                    { label: 'Масса перевозимого груза, кг.', value: '3200' },
                    { label: 'Масса кузова, кг.', value: '1825' },
                    { label: 'Масса снаряжённого фургона, кг.', value: '8105' },
                    { label: 'Полная масса автофургона, кг.', value: '11505' },
                    { label: 'Распределение нагрузки на дорогу от полной массы, кг. — на переднюю ось', value: '5249' },
                    { label: 'Распределение нагрузки на дорогу от полной массы, кг. — на заднюю ось', value: '6256' },
                    { label: 'Габаритные размеры автофургона (длина/ширина/высота), мм', value: '7900/2600/3365' },
                    { label: 'Габаритные размеры кузова (длина/ширина/высота), мм', value: '4280/2450/2020' },
                    { label: 'Внутренние размеры кузова (длина/ширина/высота), мм', value: '4340/2450/1920' },
                    { label: 'Погрузочная высота, мм', value: '1500' },
                    { label: 'Внутренний объём кузова, куб.м', value: '15-16' },
                    { label: 'Температура эксплуатации', value: 'от -40°C до +40°C' }
                  ]}
                ]
              },
              maintenance: {
                so: {
                  actions: [
                    'Сезонная подготовка изотермического кузова',
                    'Проверка теплоизоляции стен, потолка и пола',
                    'Замена сезонных уплотнений двери',
                    'Сезонное обслуживание шасси автомобиля (масла, фильтры)',
```

```
                    'Антикоррозионная обработка внутренней поверхности',
                    'Проверка термометра контроля температуры',
                    'Очистка дренажной системы кузова'
                  ],
                  labor: 8.5,
                  personnel: [P('Механик-водитель', 1), P('Слесарь по ремонту автомобилей', 1)],
                  tools: ['Набор слесарного инструмента', 'Термометр контрольный', 'Щуп для проверки толщины изоляции', 'Шприц для смазки'],
                  materials: ['Уплотнение двери резиновое', 'Антикоррозионный состав «Мовиль»', 'Масло моторное М10Г2К', 'Фильтры моторные', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр кузова и шасси',
                    'Проверка целостности теплоизоляции',
                    'Контроль герметичности уплотнения двери',
                    'Очистка дренажных отверстий',
                    'Проверка показаний термометра контроля',
                    'Смазка петель и замков двери'
                  ],
                  labor: 3.0,
                  personnel: [P('Механик-водитель', 1)],
                  tools: ['Набор инструментов автомобильный', 'Термометр', 'Шприц для смазки', 'Фонарь технический'],
                  materials: ['Смазка ЛИТОЛ-24', 'Ветошь', 'Моющее средство МС-37']
                },
                to2: {
                  actions: [
                    'Тщательный осмотр теплоизоляции',
                    'Замер термоизоляционных свойств кузова',
                    'Замена изношенных уплотнений',
                    'Регулировка дверных замков и петель',
                    'Дезинфекция внутренней поверхности кузова',
                    'Проверка крепления кузова к шасси'
                  ],
                  labor: 6.5,
                  personnel: [P('Механик-водитель', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Тепловизор / термометр', 'Динамометрический ключ', 'Опрыскиватель'],
                  materials: ['Уплотнения дверей (комплект)', 'Дезинфицирующий раствор', 'Смазка ЛИТОЛ-24', 'Прокладки']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр кузова',
                    'Замена утратившей свойства теплоизоляции',
                    'Восстановление внутренней обшивки',
                    'Замена петель и замков двери',
                    'Капитальный осмотр шасси по нормативам АФИ-53',
                    'Антикоррозионная обработка днища и каркаса',
                    'Контрольный заезд с тестом удержания температуры'
                  ],
                  labor: 18.0,
                  personnel: [P('Механик-водитель', 1), P('Слесарь по ремонту автомобилей', 1), P('Жестянщик', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Тепловизор', 'Гидравлический пресс'],
                  materials: ['Теплоизоляционные плиты', 'Обшивочный материал', 'Антикоррозионный состав', 'Уплотнения', 'Замки и петли', 'Краска']
                },
                nz: {
                  actions: [
                    'Удаление консервационной смазки',
                    'Расконсервация двигателя и систем',
                    'Заправка ГСМ',
                    'Проверка теплоизоляции кузова',
                    'Контрольный заезд с замером температуры',
                    'Проверка комплектности по описи'
                  ],
                  labor: 10.0,
                  personnel: [P('Механик-водитель', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Термометр контрольный', 'Манометр'],
                  materials: ['Топливо дизельное', 'Масло моторное', 'Антифриз', 'Тормозная жидкость', 'Ветошь']
                }
              }
            },

            /* === АФК-53 === */
            {
              id: 'afk-53',
              name: 'Автомобиль-фургон комбинированный семейства АФК-53',
              description: 'Автомобиль-фургон с комбинированным кузовом для перевозки различного продовольствия. По учебному пособию к семейству относятся модели АФК-3307/АФК-3308 (подвоз продовольствия и хлеба) и АФК-2,9 (перевозка скоропортящихся продуктов и хлеба).',
              specs: {
                groups: [
                  { title: 'АФК-3307, АФК-3308', params: [
                    { label: 'Базовое шасси', value: 'ГАЗ-3307, ГАЗ-3309, ГАЗ-3308, ГАЗ-33081' },
                    { label: 'Габаритные размеры кузова-фургона (длина/ширина/высота), мм', value: '3491x2440x1772' },
                    { label: 'Количество хлебных лотков, шт.', value: '80' },
                    { label: 'Нижний порог температуры при эксплуатации, °C', value: '-50' },
                    { label: 'Верхний порог температуры при эксплуатации, °C', value: '+50' },
                    { label: 'Масса перевозимого груза, кг.', value: '3205' }
```

```
                ]},
                { title: 'АФК-2,9', params: [
                  { label: 'Базовое шасси', value: 'КамАЗ-4350, Урал-43206' },
                  { label: 'Габаритные размеры кузова-фургона (длина/ширина/высота), мм', value: '4375x2515x2125' },
                  { label: 'Количество хлебных лотков, шт.', value: '133' },
                  { label: 'Нижний порог температуры при эксплуатации, °C', value: '-50' },
                  { label: 'Верхний порог температуры при эксплуатации, °C', value: '+50' },
                  { label: 'Масса перевозимого груза, кг.', value: '3200/2400' }
                ]}
              ]
            },
            maintenance: {
              so: {
                actions: [
                  'Сезонная подготовка холодильного агрегата',
                  'Заправка хладагентом по сезону',
                  'Проверка работы рефрижераторной установки',
                  'Очистка испарителя и конденсатора',
                  'Подготовка шасси автомобиля к сезону',
                  'Проверка автономного питания холодильника',
                  'Контроль перегородок между отсеками'
                ],
                labor: 14.0,
                personnel: [P('Механик-водитель', 1), P('Слесарь-холодильщик', 1), P('Электрик', 1)],
                tools: ['Набор слесарного инструмента', 'Манометрический коллектор', 'Вакуумный насос', 'Течеискатель галогенный', 'Мультиметр'],
                materials: ['Хладагент R-134a', 'Масло холодильное', 'Уплотнения', 'Фильтр-осушитель', 'Изоляционная лента', 'Ветошь']
              },
              to1: {
                actions: [
                  'Внешний осмотр кузова и холодильной установки',
                  'Контроль давления хладагента',
                  'Очистка испарителя и конденсатора',
                  'Проверка работы вентиляторов холодильной установки',
                  'Контроль температуры в каждом отсеке',
                  'Смазка петель и замков'
                ],
                labor: 4.0,
                personnel: [P('Механик-водитель', 1), P('Слесарь-холодильщик', 1)],
                tools: ['Манометрический коллектор', 'Термометр', 'Щётка для конденсатора', 'Мультиметр'],
                materials: ['Смазка ЛИТОЛ-24', 'Ветошь', 'Очиститель конденсатора']
              },
              to2: {
                actions: [
                  'Ревизия холодильного компрессора',
                  'Проверка герметичности контура хладагента',
                  'Замена фильтра-осушителя',
                  'Регулировка терморегуляторов',
                  'Замена уплотнений дверей отсеков',
                  'Проверка электропроводки и автоматики'
                ],
                labor: 11.0,
                personnel: [P('Слесарь-холодильщик', 1), P('Электрик', 1), P('Механик-водитель', 1)],
                tools: ['Набор слесарного инструмента', 'Манометрический коллектор', 'Вакуумный насос', 'Течеискатель', 'Мегаомметр'],
                materials: ['Хладагент R-134a', 'Фильтр-осушитель', 'Масло холодильное', 'Уплотнения дверей (комплект)', 'Кабель электрический']
              },
              to3: {
                actions: [
                  'Капитальный осмотр холодильной системы',
                  'Замена компрессора при необходимости',
                  'Полная замена хладагента',
                  'Восстановление теплоизоляции отсеков',
                  'Восстановление лакокрасочного покрытия',
                  'Капитальный осмотр шасси автомобиля',
                  'Контрольный заезд с замером параметров'
                ],
                labor: 30.0,
                personnel: [P('Слесарь-холодильщик', 1), P('Слесарь по ремонту автомобилей', 1), P('Электрик', 1), P('Жестянщик', 1)],
                tools: ['Полный набор слесарного инструмента', 'Грузоподъёмное оборудование', 'Сварочный аппарат', 'Стенд испытания компрессоров'],
                materials: ['Компрессор холодильный', 'Хладагент R-134a (полная заправка)', 'Теплоизоляция', 'Уплотнения', 'Кабель силовой', 'Краска']
              },
              nz: {
                actions: [
                  'Расконсервация автомобиля и холодильной установки',
                  'Заправка ГСМ и хладагентом',
                  'Опробование рефрижератора',
                  'Контроль температуры во всех отсеках',
                  'Проверка комплектности',
                  'Контрольный заезд'
                ],
                labor: 14.0,
                personnel: [P('Механик-водитель', 1), P('Слесарь-холодильщик', 1)],
                tools: ['Набор слесарного инструмента', 'Манометрический коллектор', 'Термометр', 'Вакуумный насос'],
                materials: ['Топливо дизельное', 'Масло моторное', 'Хладагент R-134a', 'Антифриз', 'Ветошь']
```

```
                }
              }
            },

            /* === АФХ-53 === */
            {
              id: 'afh-53',
              name: 'Автомобиль-фургон хлебный семейства АФХ-53',
              description: 'Специализированный автомобиль-фургон для перевозки готовой хлебобулочной продукции, оборудованный лотками-кассетами и вентиляцией. По учебному пособию к семейству относятся модели АФХО-131, АФХ-3,5-3307П и АФХ-2,6.',
              specs: {
                groups: [
                  { title: 'АФХО-131', params: [
                    { label: 'База монтажа', value: 'ЗИЛ-131' },
                    { label: 'Полезная нагрузка, кг.', value: '2400' },
                    { label: 'Погрузочная высота, мм — минимальная (по нижним лоткам)', value: '1410' },
                    { label: 'Погрузочная высота, мм — максимальная (по верхним лоткам)', value: '2460' },
                    { label: 'Кузов: конструкция', value: 'смешанная (каркас - деревянный, обшивка - металлическая)' },
                    { label: 'Габаритные размеры, мм — длина', value: '4300' },
                    { label: 'Габаритные размеры, мм — ширина', value: '2460' },
                    { label: 'Габаритные размеры, мм — высота', value: '1300' },
                    { label: 'Термоизоляция', value: 'пенопласт' },
                    { label: 'Количество лотков, шт.', value: '150' },
                    { label: 'Отопительно-вентиляционная установка: тип', value: 'ОВ-65' },
                    { label: 'Отопительно-вентиляционная установка: максимальная теплоотдача, ккал/ч', value: '6500' },
                    { label: 'Отопительно-вентиляционная установка: расход топлива, л/ч', value: '1,0' },
                    { label: 'Отопительно-вентиляционная установка: применяемое топливо', value: 'ДТ' },
                    { label: 'Отопительно-вентиляционная установка: вместимость топл. бака, л.', value: '2,0' }
                  ]},
                  { title: 'АФХ-3,5-3307П', params: [
                    { label: 'База монтажа', value: 'ГАЗ-307' },
                    { label: 'Полезная нагрузка, кг.', value: '2560' },
                    { label: 'Погрузочная высота, мм — минимальная (по нижним лоткам)', value: '1230' },
                    { label: 'Погрузочная высота, мм — максимальная (по верхним лоткам)', value: '2880' },
                    { label: 'Кузов: конструкция', value: 'смешанная (каркас - деревянный, обшивка - металлическая)' },
                    { label: 'Габаритные размеры, мм — длина', value: '3430' },
                    { label: 'Габаритные размеры, мм — ширина', value: '2380' },
                    { label: 'Габаритные размеры, мм — высота', value: '1475' },
                    { label: 'Термоизоляция', value: 'пенопласт' },
                    { label: 'Количество лотков, шт.', value: '160' }
                  ]},
                  { title: 'АФХ-2,6', params: [
                    { label: 'Базовое шасси', value: 'КамАЗ-4350' },
                    { label: 'Масса перевозимого хлеба, кг.', value: '2600' },
                    { label: 'Колёсная формула', value: '4x4' },
                    { label: 'Колёсная база, мм', value: '4180' },
                    { label: 'Снаряженная масса шасси, кг.', value: '6700' },
                    { label: 'Максимально допустимая масса надстройки с грузом, кг.', value: '5000' },
                    { label: 'Полная масса автомобиля (максимально допустимая), кг.', value: '11900' },
                    { label: 'Масса снаряжённого автофургона, кг.', value: '8508' },
                    { label: 'Полная масса автофургона, кг.', value: '11312' },
                    { label: 'Масса снаряжённого кузова автофургона, кг.', value: '2228' },
                    { label: 'Масса лотков, кг.', value: '594' },
                    { label: 'Количество лотков, шт.', value: '220' },
                    { label: 'Габаритные размеры хлебных лотков, мм', value: '740x450x85' },
                    { label: 'Габаритные размеры автофургона (длина/ширина/высота), мм', value: '7830/2900/3440' },
                    { label: 'Габаритные размеры кузова-фургона (длина/ширина/высота), мм', value: '4375/2515/2125' },
                    { label: 'Погрузочная высота, мм — минимальная (по нижним лоткам)', value: '1480' },
                    { label: 'Погрузочная высота, мм — максимальная (по верхним лоткам)', value: '3060' },
                    { label: 'Температура эксплуатации', value: 'от -40°C до +40°C' }
                  ]}
                ]
              },
              maintenance: {
                so: {
                  actions: [
                    'Подготовка кузова к сезону',
                    'Проверка системы вентиляции кузова',
                    'Сезонное обслуживание шасси',
                    'Дезинфекция внутренней поверхности и лотков',
                    'Проверка крепления лотков-направляющих',
                    'Антикоррозионная обработка днища'
                  ],
                  labor: 6.5,
                  personnel: [P('Механик-водитель', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Шприц для смазки', 'Опрыскиватель для дезинфекции'],
                  materials: ['Дезинфицирующий раствор', 'Антикоррозионный состав', 'Смазка ЛИТОЛ-24', 'Масло моторное', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр кузова',
                    'Проверка вентиляционных решёток',
                    'Очистка и санитарная обработка кузова',
                    'Контроль крепления кассет-лотков',
                    'Проверка замков двери и стопоров'
                  ],
                  labor: 2.5,
                  personnel: [P('Механик-водитель', 1)],
```

```
                  tools: ['Набор инструментов', 'Щётка', 'Опрыскиватель'],
                  materials: ['Дезинфицирующее средство', 'Ветошь', 'Смазка']
                },
                to2: {
                  actions: [
                    'Ревизия лотков-кассет',
                    'Замена изношенных направляющих',
                    'Регулировка вентиляции',
                    'Замена уплотнений дверей',
                    'Проверка крепления кузова к шасси'
                  ],
                  labor: 5.0,
                  personnel: [P('Механик-водитель', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Динамометрический ключ', 'Сварочный аппарат'],
                  materials: ['Направляющие лотков', 'Уплотнения', 'Электроды сварочные', 'Краска']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр кузова',
                    'Замена изношенных лотков-кассет',
                    'Восстановление обшивки',
                    'Капитальный осмотр шасси',
                    'Восстановление лакокрасочного покрытия',
                    'Контрольный пробег с грузом'
                  ],
                  labor: 14.0,
                  personnel: [P('Механик-водитель', 1), P('Слесарь по ремонту автомобилей', 1), P('Жестянщик', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Гидравлический пресс'],
                  materials: ['Лотки-кассеты', 'Обшивочный материал', 'Краска', 'Электроды сварочные', 'Антикоррозионный состав']
                },
                nz: {
                  actions: [
                    'Расконсервация автомобиля',
                    'Заправка ГСМ',
                    'Санитарная обработка кузова',
                    'Проверка крепления лотков',
                    'Контрольный заезд'
                  ],
                  labor: 8.0,
                  personnel: [P('Механик-водитель', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Опрыскиватель'],
                  materials: ['Топливо', 'Масло моторное', 'Дезсредство', 'Ветошь']
                }
              }
            },

            /* === ККПр-3,5Х === */
            {
              id: 'kkpr-35',
              name: 'Кузов-контейнер для подвоза продовольствия ККПр-3,5Х',
              description: 'Универсальный кузов-контейнер с креплениями под грузовое шасси, предназначен для подвоза продовольствия и другого имущества. Внимание: обозначение «ККПр-3,5Х» в учебном пособии не встречается — в разделе о составе передвижного хлебозавода ПХК-7 упоминается близкий по назначению мобильный склад-контейнер продовольственный «СКПр-30», однако его числовые характеристики (ёмкость, масса, габариты) в учебном пособии не приведены.',
              maintenance: {
                so: {
                  actions: [
                    'Сезонная подготовка кузова к эксплуатации',
                    'Проверка уплотнений и замков',
                    'Антикоррозионная обработка наружных поверхностей',
                    'Проверка крепления узлов фиксации на шасси',
                    'Смазка шарниров и петель'
                  ],
                  labor: 4.0,
                  personnel: [P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Шприц для смазки', 'Динамометрический ключ'],
                  materials: ['Краска защитная', 'Антикоррозионный состав «Мовиль»', 'Смазка ЛИТОЛ-24', 'Уплотнения', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр контейнера',
                    'Контроль герметичности дверей',
                    'Проверка стопоров и замков',
                    'Очистка внутреннего пространства',
                    'Смазка подвижных соединений'
                  ],
                  labor: 1.5,
                  personnel: [P('Слесарь', 1)],
                  tools: ['Набор инструментов', 'Шприц для смазки'],
                  materials: ['Смазка ЛИТОЛ-24', 'Ветошь']
                },
                to2: {
                  actions: [
                    'Ревизия замков и петель',
                    'Замена изношенных уплотнений',
                    'Подкраска повреждённых участков',
                    'Проверка элементов крепления к шасси'
                  ],
```

```
                  labor: 3.5,
                  personnel: [P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Кисть малярная', 'Динамометрический ключ'],
                  materials: ['Уплотнения (комплект)', 'Краска', 'Замки запасные', 'Смазка']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр кузова',
                    'Замена изношенных узлов крепления',
                    'Восстановление лакокрасочного покрытия',
                    'Антикоррозионная обработка всех поверхностей',
                    'Замена замков и петель'
                  ],
                  labor: 8.0,
                  personnel: [P('Слесарь', 1), P('Жестянщик', 1), P('Сварщик', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Краскораспылитель'],
                  materials: ['Краска эмаль', 'Грунт', 'Электроды сварочные', 'Замки комплект', 'Антикоррозионный состав']
                },
                nz: {
                  actions: [
                    'Удаление консервационной смазки',
                    'Проверка комплектности',
                    'Контроль работы замков',
                    'Опробование на шасси'
                  ],
                  labor: 3.0,
                  personnel: [P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Шприц для смазки'],
                  materials: ['Смазка ЛИТОЛ-24', 'Ветошь', 'Уплотнения']
                }
              }
            },

            /* === АЦПТ-5,6 === */
            {
              id: 'atspt-56',
              name: 'Автомобиль-цистерна для питьевой воды АЦПТ-5,6',
              description: 'Автомобильная цистерна предназначена для транспортирования и кратковременного хранения питьевой воды; специальное оборудование смонтировано на автомобиле повышенной проходимости. Термоизолированная цистерна состоит из двух секций, основания и отсека обслуживания в задней части. Имеет две горловины для наполнения, дезинфекции и осмотра внутренней поверхности, каждая закрывается термоизолированной крышкой; трубопроводы наполнения-слива защищены от обмерзания зимой. Оборудована поручнями и площадками обслуживания с противоскользящей поверхностью. Упоминается в учебном пособии в составе группы «АВЦ-1,7, АЦПТ-4,1, АЦПТ-5, АЦПТ-5,6»; числовые характеристики (объём, масса, габариты) в учебном пособии для этой группы не приведены.',
              maintenance: {
                so: {
                  actions: [
                    'Сезонная подготовка цистерны и трубопроводов',
                    'Проверка системы обогрева цистерны (зимний режим)',
                    'Регулировка крана раздачи и насоса',
                    'Контроль герметичности цистерны опрессовкой',
                    'Дезинфекция внутренней поверхности цистерны',
                    'Сезонное обслуживание шасси',
                    'Замена воздушного фильтра дыхательного клапана'
                  ],
                  labor: 11.0,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь по ремонту автомобилей', 1), P('Механик-водитель', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр для опрессовки', 'Опрессовочный насос', 'Опрыскиватель для дезинфекции', 'Динамометрический ключ'],
                  materials: ['Дезинфицирующий состав (гипохлорит натрия)', 'Уплотнения (комплект)', 'Масло моторное М10Г2К', 'Фильтр дыхательного клапана', 'Тосол ОЖ-65', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр цистерны и арматуры',
                    'Контроль герметичности соединений',
                    'Проверка работы насоса и крана раздачи',
                    'Очистка фильтров водозабора и раздачи',
                    'Проверка показаний водомера',
                    'Санитарная обработка раздаточных рукавов'
                  ],
                  labor: 3.5,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр', 'Щётка для шлангов'],
                  materials: ['Прокладки', 'Дезсредство', 'Смазка ЛИТОЛ-24', 'Ветошь']
                },
                to2: {
                  actions: [
                    'Гидравлическая опрессовка цистерны',
                    'Ревизия водяного насоса',
                    'Замена уплотнений крана раздачи',
                    'Поверка водомера',
                    'Дезинфекция цистерны и магистралей',
                    'Регулировка предохранительного клапана',
                    'Проверка анодной защиты от коррозии'
                  ],
                  labor: 8.5,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь по ремонту топливной аппаратуры', 1), P('Электрик',
```

```
1)],
                tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Манометр контрольный', 'Динамометрический ключ', 'Мультиметр'],
                materials: ['Уплотнения насоса (комплект)', 'Прокладки крана', 'Гипохлорит натрия', 'Анод протекторный', 'Смазка ЛИТОЛ-24']
              },
              to3: {
                actions: [
                  'Капитальный осмотр цистерны и трубопроводов',
                  'Замена водяного насоса при необходимости',
                  'Восстановление антикоррозионной защиты',
                  'Замена изношенных раздаточных рукавов',
                  'Капитальный осмотр шасси',
                  'Контрольный пробег с заливкой воды',
                  'Поверка водомера в специализированной лаборатории'
                ],
                labor: 22.0,
                personnel: [P('Оператор водовоза', 1), P('Слесарь по ремонту автомобилей', 1), P('Слесарь-сантехник', 1), P('Электрик', 1)],
                tools: ['Полный набор слесарного инструмента', 'Опрессовочный насос', 'Грузоподъёмное оборудование', 'Сварочный аппарат для нержавеющей стали'],
                materials: ['Водяной насос', 'Рукава раздаточные', 'Уплотнения (полный комплект)', 'Краска эпоксидная', 'Электроды для нерж. стали', 'Анод протекторный']
              },
              nz: {
                actions: [
                  'Расконсервация автомобиля и цистерны',
                  'Дезинфекция цистерны и магистралей',
                  'Заправка ГСМ',
                  'Опробование насоса и крана раздачи',
                  'Контрольная заливка воды и опробование раздачи',
                  'Проверка комплектности по описи'
                ],
                labor: 12.0,
                personnel: [P('Оператор водовоза', 1), P('Слесарь', 1), P('Механик-водитель', 1)],
                tools: ['Набор слесарного инструмента', 'Манометр', 'Опрыскиватель'],
                materials: ['Топливо дизельное', 'Масло моторное', 'Гипохлорит натрия', 'Уплотнения', 'Ветошь']
              }
            }
          },

          /* === АЦПТ-4,7 === */
          {
            id: 'atspt-47',
            name: 'Автомобиль-цистерна для питьевой воды АЦПТ-4,7',
            description: 'Автомобильная цистерна предназначена для транспортирования и кратковременного хранения питьевой воды на шасси автомобиля повышенной проходимости. Конструктивно подобна АЦПТ-5,6: термоизолированная цистерна из двух секций с отсеком обслуживания, две горловины для наполнения и осмотра, защита трубопроводов от обмерзания. Внимание: в учебном пособии в группе автомобильных цистерн указаны обозначения «АВЦ-1,7, АЦПТ-4,1, АЦПТ-5, АЦПТ-5,6» — обозначение «АЦПТ-4,7» и числовые характеристики (объём, масса, габариты) в пособии не встречаются; наиболее близкое по пособию — АЦПТ-4,1.',
            maintenance: {
              so: {
                actions: [
                  'Сезонная подготовка цистерны',
                  'Проверка системы утепления (зима)',
                  'Регулировка раздаточной арматуры',
                  'Контроль герметичности опрессовкой',
                  'Дезинфекция внутренней поверхности',
                  'Сезонное обслуживание шасси'
                ],
                labor: 9.5,
                personnel: [P('Оператор водовоза', 1), P('Слесарь', 1), P('Механик-водитель', 1)],
                tools: ['Набор слесарного инструмента', 'Манометр', 'Опрессовочный насос', 'Опрыскиватель'],
                materials: ['Гипохлорит натрия', 'Уплотнения', 'Масло моторное', 'Тосол', 'Ветошь']
              },
              to1: {
                actions: [
                  'Внешний осмотр цистерны',
                  'Контроль герметичности соединений',
                  'Очистка фильтров',
                  'Проверка работы насоса',
                  'Санитарная обработка раздаточных устройств'
                ],
                labor: 3.0,
                personnel: [P('Оператор водовоза', 1)],
                tools: ['Набор инструментов', 'Манометр', 'Щётка'],
                materials: ['Прокладки', 'Дезсредство', 'Ветошь']
              },
              to2: {
                actions: [
                  'Гидравлическая опрессовка цистерны',
                  'Ревизия насоса',
                  'Замена уплотнений крана раздачи',
                  'Дезинфекция цистерны',
                  'Регулировка предохранительного клапана'
                ],
                labor: 7.5,
                personnel: [P('Оператор водовоза', 1), P('Слесарь', 1)],
                tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Динамометрический ключ'],
```

```
                  materials: ['Уплотнения (комплект)', 'Гипохлорит натрия', 'Прокладки', 'Смазка ЛИТОЛ-24']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр цистерны',
                    'Замена насоса при необходимости',
                    'Восстановление антикоррозионной защиты',
                    'Замена раздаточных рукавов',
                    'Капитальный осмотр шасси',
                    'Поверка водомера'
                  ],
                  labor: 20.0,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь по ремонту автомобилей', 1), P('Слесарь-сантехник', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Опрессовочный насос'],
                  materials: ['Водяной насос', 'Рукава', 'Уплотнения', 'Краска эпоксидная', 'Анод протекторный']
                },
                nz: {
                  actions: [
                    'Расконсервация автомобиля и цистерны',
                    'Дезинфекция',
                    'Заправка ГСМ',
                    'Опробование насоса и раздачи',
                    'Контрольная заливка воды'
                  ],
                  labor: 10.5,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр'],
                  materials: ['Топливо', 'Масло моторное', 'Гипохлорит натрия', 'Ветошь']
                }
              }
            },

            /* === ЦВ-1,2 === */
            {
              id: 'cv-12',
              name: 'Прицеп-цистерна ЦВ-1,2',
              description: 'Прицеп для доставки и кратковременного хранения воды подразделениям, учреждениям (организациям). По учебному пособию рассматривается в составе пары ПЦПТ-1,2, ЦВ-1,2. Цистерна выполнена без теплоизоляции из нержавеющей стали АИСИ-304 (или аналог) толщиной 2-3 мм, оборудована ручным насосом и краном раздачи.',
              specs: {
                groups: [
                  { title: 'ПЦПТ-1,2, ЦВ-1,2', params: [
                    { label: 'База монтажа', value: 'ИАПЗ-738 (ИАПЗ-739К)' },
                    { label: 'Ёмкость, л.', value: '1200' },
                    { label: 'Ручной насос для воды: тип', value: 'БКФ-4' },
                    { label: 'Ручной насос для воды: производительность, л/мин', value: '40-50' },
                    { label: 'Ручной насос для воды: высота всасывания, м.', value: '5' },
                    { label: 'Ручной насос для воды: максимальный напор, кгс/кв. см.', value: '3' },
                    { label: 'Габаритные размеры (длина/ширина/высота), мм', value: '3943/2070/1668' },
                    { label: 'Вес, кг.: сухая/полная', value: '950/2150' }
                  ]}
                ]
              },
              maintenance: {
                so: {
                  actions: [
                    'Сезонная подготовка прицепа',
                    'Сезонное обслуживание ходовой части',
                    'Проверка тормозной системы прицепа',
                    'Дезинфекция цистерны',
                    'Контроль герметичности опрессовкой',
                    'Проверка ручного насоса',
                    'Антикоррозионная обработка наружных поверхностей'
                  ],
                  labor: 5.5,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр шинный', 'Опрессовочный насос ручной', 'Шприц для смазки'],
                  materials: ['Смазка ЛИТОЛ-24', 'Гипохлорит натрия', 'Уплотнения', 'Краска защитная', 'Ветошь']
                },
                to1: {
                  actions: [
                    'Внешний осмотр цистерны и прицепа',
                    'Проверка работы крана раздачи',
                    'Контроль герметичности',
                    'Очистка фильтра',
                    'Смазка ходовой части и сцепки'
                  ],
                  labor: 1.5,
                  personnel: [P('Оператор водовоза', 1)],
                  tools: ['Набор инструментов', 'Шприц для смазки'],
                  materials: ['Смазка ЛИТОЛ-24', 'Дезсредство', 'Ветошь']
                },
                to2: {
                  actions: [
                    'Опрессовка цистерны',
                    'Ревизия ручного насоса',
                    'Замена уплотнений крана',
```

```
                    'Дезинфекция цистерны',
                    'Смазка ступиц колёс',
                    'Регулировка тормозов'
                  ],
                  labor: 4.5,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Шприц для смазки'],
                  materials: ['Уплотнения', 'Гипохлорит натрия', 'Смазка ЛИТОЛ-24', 'Прокладки']
                },
                to3: {
                  actions: [
                    'Капитальный осмотр прицепа и цистерны',
                    'Замена ручного насоса при необходимости',
                    'Восстановление антикоррозионного покрытия',
                    'Замена подшипников колёс',
                    'Регулировка тормозной системы',
                    'Опрессовка повышенным давлением'
                  ],
                  labor: 10.0,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь', 1), P('Сварщик', 1)],
                  tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Съёмник подшипников'],
                  materials: ['Ручной насос', 'Подшипники', 'Уплотнения', 'Краска защитная', 'Электроды сварочные',
'Смазка ЛИТОЛ-24']
                },
                nz: {
                  actions: [
                    'Расконсервация прицепа',
                    'Дезинфекция цистерны',
                    'Опробование насоса и крана',
                    'Контрольная заливка воды',
                    'Проверка комплектности'
                  ],
                  labor: 5.0,
                  personnel: [P('Оператор водовоза', 1), P('Слесарь', 1)],
                  tools: ['Набор слесарного инструмента', 'Манометр'],
                  materials: ['Гипохлорит натрия', 'Смазка ЛИТОЛ-24', 'Уплотнения', 'Ветошь']
                }
              }
            },

            /* === Цистерна для воды ЦПТ-0,4 и ЦВ-0,4, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'cptvpt-04',
              name: 'Цистерна для воды ЦПТ-0,4 и ЦВ-0,4',
              description: 'Цистерна водяная ЦВ-0,4 предназначена для хранения и транспортировки, подачи воды как из
собственной цистерны, так и напрямую из других ёмкостей и различного типа водоёмов. ЦПТ-0,4 аналогична по устройству
ЦВ-0,4, но имеет теплоизоляцию (у ЦВ-0,4 теплоизоляции нет). Каждые две цистерны комплектуются ручным насосом БКФ-4 и
комплектом резинотканевых рукавов.',
              specs: {
                groups: [
                  { title: 'ЦВ-4', params: [
                    { label: 'Вместимость, л.', value: '320' },
                    { label: 'Габаритные размеры, мм — длина/ширина/высота', value: '1070/595/870' },
                    { label: 'Масса сухая, кг.', value: '80' }
                  ]},
                  { title: 'ЦПТ-0,4', params: [
                    { label: 'Вместимость, л.', value: '350' },
                    { label: 'Габаритные размеры, мм — длина/ширина/высота', value: '1080/595/870' },
                    { label: 'Масса сухая, кг.', value: '95' }
                  ]}
                ]
              }
            }
          ]
        },

        /* ---------- ГРУППА 4: Полевые ремонтные средства продовольственной службы ---------- */
        {
          id: 'food-repair',
          name: 'Полевые ремонтные средства продовольственной службы',
          type: 'group',
          children: [
            /* === РМТП-9, данные из учебного пособия; регламент ТО не приведён === */
            {
              id: 'rmtp-9',
              name: 'Мастерская ремонта технических средств продовольственной службы РМТП-9',
              description: 'Предназначена для текущего ремонта полевых кухонь, хлебопекарных и холодильных установок и
их технического обслуживания. Состав: модуль слесарно-механический РМТП-9СМ, модуль электромеханический РМТП-9ЭМ.',
              specs: {
                groups: [
                  { title: null, params: [
                    { label: 'Базовый кузов-контейнер', value: 'КК4.2.51.1 или КК4.1' },
                    { label: 'Габаритные размеры (длина/ширина/высота), мм', value: '5000/2438/2250' },
                    { label: 'Количество рабочих мест, шт. — в кузове-контейнере/в палатке', value: '4/3' },
                    { label: 'Размеры площадки для развёртывания мастерской, м.', value: '15x15' },
                    { label: 'Базовые автотранспортные средства — автомобили', value: 'КамАЗ-5350,
Урал-4320-0810/0811-31' },
                    { label: 'Базовые автотранспортные средства — прицепы', value: 'ЧМЗАП-83352, ЧМЗАП-83357' },
                    { label: 'Условия эксплуатации (температура окружающей среды) для умеренного и холодного климата',
value: 'от -50 до +50°C' },
```

```
                    { label: 'Электроснабжение — основное', value: 'от собственных дизель-электрических установок, от внешней стационарной электрической сети или от передвижных источников переменного трёхфазного тока напряжением 380/220 (400/230) В, частотой 50 Гц' },
                    { label: 'Электроснабжение — вспомогательное', value: 'от электрической сети шасси (постоянный ток напряжением 24 В)' }
                  ]}
                ]
              }
            }
          ]
        }
      ]
    },

    /* ============================================================
       СЛУЖБА 2: ВЕЩЕВАЯ СЛУЖБА
       ============================================================ */
    {
      id: 'clothing',
      name: 'Технические средства вещевой службы',
      type: 'service',
      children: [
        /* === ПОЛЕВАЯ БАНЯ БПО-32 === */
        {
          id: 'bpo-32',
          name: 'Полевая баня БПО-32',
          description: 'Предназначена для помывки военнослужащих, находящихся вне пунктов постоянной дислокации в полевых условиях, в том числе на учениях, при вооружённых конфликтах и при ликвидации последствий стихийных бедствий. Штатное предназначение — оснащение банно-прачечных пунктов соединений и воинских частей. Состав: раздевально-помывочный блок (кузов-контейнер переменного объёма КК-4.3), теплопункт (кузов-контейнер постоянного объёма КК-4.2), комплект ЗИП, 2 палатки М-10, резервуар РДВ-5000, межблочное оборудование.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Производительность по помывке, чел. — на 8 штатных душевых сетках', value: '32' },
                { label: 'Производительность по помывке, чел. — на 8 штатных и 8 дополнительных душевых сетках', value: '64' },
                { label: 'Количество обслуживающего персонала, чел.', value: '4' },
                { label: 'Время автономной работы, ч, не менее', value: '12' },
                { label: 'Площадь развёртывания бани, м.', value: '10×25' },
                { label: 'Допустимый уклон площадки, град.', value: '2' },
                { label: 'Время развёртывания (свёртывания), ч., не более', value: '2,5' },
                { label: 'Транспортная база — КамАЗ (6×6) или Урал (6×6), количество, шт.', value: '2' },
                { label: 'Скорость транспортирования, км/ч, не более — по дорогам с асфальтовым и бетонным покрытием', value: '70' },
                { label: 'Скорость транспортирования, км/ч, не более — по дорогам с грунтовым покрытием', value: '50' },
                { label: 'Количество ж/д платформ для транспортирования, шт.', value: '1' },
                { label: 'Электроснабжение', value: 'основное — от внешней стационарной сети или передвижных источников переменного трёхфазного тока напряжением 380/220 (400/230) В, частотой 50 Гц; вспомогательное — от встроенных аккумуляторных батарей (постоянный ток напряжением 24 В)' },
                { label: 'Мощность установленной ДЭУ, кВт', value: '8' },
                { label: 'Вместимость топливных баков, л. — для котлов теплопункта', value: '120' },
                { label: 'Вместимость топливных баков, л. — для ДЭУ и отопителей ОВ-95', value: '60' },
                { label: 'Производительность теплопункта по горячей воде (70…80 °С), л/ч — при работе одного котла / двух котлов', value: '1000/2000' },
                { label: 'Производительность теплопункта по холодной воде (5 °С), л/ч, не менее', value: '6000' },
                { label: 'Производительность теплопункта по помывочной воде (42 °С), л/ч — при работе одного котла / двух котлов', value: '2000/4000' },
                { label: 'Мощность котлов тепловая, кВт', value: '2 × 90' },
                { label: 'Срок службы, лет', value: '15' }
              ]}
            ]
          },
          maintenance: {
            so: {
              actions: [
                'Сезонная подготовка водогрейного котла и парогенератора',
                'Перевод горелочного устройства на сезонное топливо',
                'Проверка теплоизоляции трубопроводов горячей воды',
                'Утепление палаточной части на зимний сезон',
                'Заправка системы антифризом (зимний период)',
                'Регулировка терморегуляторов нагрева воды',
                'Опрессовка котла рабочим давлением',
                'Дезинфекция помывочного отделения и душевых сеток'
              ],
              labor: 12.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь по ремонту котельного оборудования', 1), P('Слесарь по ремонту топливной аппаратуры', 1)],
              tools: ['Набор слесарного инструмента ИН-3', 'Опрессовочный насос', 'Манометр контрольный', 'Термометр', 'Опрыскиватель для дезинфекции'],
              materials: ['Топливо дизельное Л-0,2-62', 'Антифриз ОЖ-65', 'Уплотнения котла (комплект)', 'Дезсредство (гипохлорит натрия)', 'Теплоизоляция трубопроводов', 'Ветошь']
            },
            to1: {
              actions: [
                'Внешний осмотр оборудования бани',
                'Очистка душевых сеток от накипи',
                'Проверка работы запорной арматуры',
                'Контроль давления в системе горячего водоснабжения',
                'Удаление накипи в водогрейном котле',
```

```
                'Промывка фильтров водоподготовки',
                'Санитарная обработка помывочного отделения'
              ],
              labor: 4.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь', 1)],
              tools: ['Набор слесарного инструмента', 'Манометр', 'Щётка для удаления накипи', 'Опрыскиватель'],
              materials: ['Антинакипин', 'Дезсредство', 'Фильтрующие элементы', 'Ветошь']
            },
            to2: {
              actions: [
                'Ревизия водогрейного котла с разборкой',
                'Очистка котла от накипи химическим способом',
                'Проверка предохранительного клапана котла',
                'Замена уплотнений душевых сеток',
                'Ревизия циркуляционного насоса',
                'Поверка термометров и манометров',
                'Регулировка горелочного устройства котла',
                'Проверка системы аварийного отключения'
              ],
              labor: 9.5,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь по ремонту котельного оборудования', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Манометр контрольный', 'Динамометрический ключ', 'Мегаомметр', 'Стенд проверки клапанов'],
              materials: ['Уплотнения (комплект)', 'Душевые сетки запасные', 'Антинакипин', 'Подшипники насоса', 'Изоляционная лента', 'Смазка ЛИТОЛ-24']
            },
            to3: {
              actions: [
                'Капитальный осмотр всего оборудования бани',
                'Замена изношенных трубопроводов',
                'Капитальная очистка котла со снятием обмуровки',
                'Замена циркуляционного насоса при необходимости',
                'Восстановление теплоизоляции системы',
                'Замена электропроводки',
                'Контрольная топка с замером производительности',
                'Восстановление лакокрасочного покрытия'
              ],
              labor: 22.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь по ремонту котельного оборудования', 1), P('Электрик', 1), P('Сварщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Опрессовочный насос', 'Мегаомметр', 'Грузоподъёмные приспособления'],
              materials: ['Трубопроводы (комплект)', 'Циркуляционный насос', 'Теплоизоляция МКРР-130', 'Кабель силовой КГ', 'Электроды сварочные УОНИ-13/55', 'Краска эмаль ПФ-115']
            },
            nz: {
              actions: [
                'Расконсервация оборудования бани',
                'Удаление консервационной смазки',
                'Опрессовка системы рабочим давлением',
                'Заправка топливом и водой',
                'Опробование котла на эксплуатационном режиме',
                'Контрольный замер температуры горячей воды',
                'Дезинфекция всего помывочного оборудования',
                'Проверка комплектности по описи'
              ],
              labor: 14.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Манометр', 'Термометр'],
              materials: ['Топливо дизельное', 'Уплотнения', 'Дезсредство', 'Ветошь']
            }
          }
        },

        /* === ПРАЧЕЧНАЯ ММП-2 === */
        {
          id: 'mmp-2',
          name: 'Механизированная малогабаритная прачечная ММП-2',
          description: 'Предназначена для стирки белья и обмундирования в полевых условиях в климатических районах с температурой окружающей среды от +40°С до -40°С. Основное оборудование по учебному пособию: трёхоперационная стиральная машина (2 шт.) и походная швейная машина 322М (322МА) кл. (1 шт.).',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Производительность прачечной (стирка) за 20 часов работы сухого белья, кг.', value: '500-700' },
                { label: 'Продолжительность технологического цикла обработки белья в одной стиральной машине, ч.', value: '2' },
                { label: 'Расход стирального порошка на 1 кг сухого белья, г.', value: '45-50' },
                { label: 'Численность личного состава при двухсменной работе, чел.', value: '12' },
                { label: 'Площадка для развёртывания прачечной, м.', value: '22x12' },
                { label: 'Время развёртывания прачечной в рабочее положение, ч. (летом/зимой)', value: '1/2' },
                { label: 'Время свёртывания прачечной в походное положение, ч. (летом/зимой)', value: '1/1' },
                { label: 'Трёхоперационная стиральная машина, шт.', value: '2' },
                { label: 'Походная швейная машина 322М (322МА) кл., шт.', value: '1' }
              ]}
            ]
          },
          maintenance: {
```

```
            so: {
              actions: [
                'Сезонная подготовка стиральных машин',
                'Регулировка водогрейного котла под сезонную температуру',
                'Проверка системы подогрева воздуха сушильного шкафа',
                'Утепление трубопроводов холодной воды (зима)',
                'Замена сезонных смазочных материалов',
                'Регулировка центрифуги после длительного простоя',
                'Дезинфекция барабанов стиральных машин'
              ],
              labor: 10.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Манометр', 'Мультиметр', 'Шприц для смазки', 'Опрессовочный насос'],
              materials: ['Смазка ЛИТОЛ-24', 'Уплотнения барабанов', 'Топливо дизельное', 'Дезсредство', 'Ветошь', 'Теплоизоляция']
            },
            to1: {
              actions: [
                'Внешний осмотр оборудования прачечной',
                'Очистка фильтров слива стиральных машин',
                'Проверка работы дозаторов моющих средств',
                'Контроль герметичности водяных соединений',
                'Смазка подшипников барабанов и центрифуги',
                'Проверка балансировки центрифуги',
                'Очистка сушильной камеры от ворса'
              ],
              labor: 3.5,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Шприц для смазки', 'Мультиметр', 'Щётка'],
              materials: ['Смазка ЛИТОЛ-24', 'Уплотнения', 'Ветошь']
            },
            to2: {
              actions: [
                'Ревизия приводов барабанов стиральных машин',
                'Замена ремней привода центрифуги',
                'Проверка работы электромагнитных клапанов подачи воды',
                'Балансировка ротора центрифуги на стенде',
                'Замена уплотнений люков стиральных машин',
                'Регулировка термостата сушильного шкафа',
                'Проверка автоматики управления стиркой',
                'Замена тэнов водогрейного котла при необходимости'
              ],
              labor: 8.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Балансировочный станок', 'Мультиметр', 'Мегаомметр', 'Динамометрический ключ'],
              materials: ['Ремни приводные клиновые (комплект)', 'Уплотнения люков', 'Электромагнитные клапаны (запас)', 'Тэны нагревательные', 'Подшипники барабана', 'Смазка ЛИТОЛ-24']
            },
            to3: {
              actions: [
                'Капитальный осмотр стиральных машин и центрифуги',
                'Замена подшипников барабанов',
                'Полная ревизия электрооборудования',
                'Замена изношенных тэнов нагрева воды',
                'Восстановление антикоррозионной защиты внутренних поверхностей',
                'Замена силовых электрических кабелей',
                'Балансировка валов на специализированном стенде',
                'Контрольная стирка пробной партии белья'
              ],
              labor: 18.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь по ремонту бытовой техники', 1), P('Электрик', 1), P('Сварщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Балансировочный станок', 'Сварочный аппарат', 'Мегаомметр М4100/3', 'Стенд испытаний'],
              materials: ['Подшипники (полный комплект)', 'Тэны нагревательные', 'Кабель силовой', 'Уплотнения', 'Краска эпоксидная', 'Электроды для нержавеющей стали']
            },
            nz: {
              actions: [
                'Расконсервация оборудования прачечной',
                'Удаление консервационной смазки',
                'Заправка водой и опрессовка системы',
                'Опробование стиральных машин на холостом ходу',
                'Контрольная стирка пробной партии',
                'Опробование центрифуги под нагрузкой',
                'Проверка комплектности по описи',
                'Замена расходных материалов'
              ],
              labor: 11.0,
              personnel: [P('Машинист банно-прачечного оборудования', 1), P('Слесарь', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Мультиметр', 'Манометр'],
              materials: ['Смазка ЛИТОЛ-24', 'Уплотнения', 'СМС для контрольной стирки', 'Ветошь']
            }
          }
        },

        /* === ПРАЧЕЧНАЯ МПП-9 === */
        {
```

```
        id: 'mpp-9',
        name: 'Механизированная полевая прачечная МПП-9',
        description: 'Предназначена для стирки, сушки и глажения белья и обмундирования военнослужащих 1-4 степеней загрязненности вне пунктов постоянной дислокации в полевых условиях, в том числе на учениях, при вооружённых конфликтах и при ликвидации последствий стихийных бедствий, с полным соблюдением технологического режима обработки белья, для профилактики педикулёза. Штатное предназначение — оснащение прачечных отрядов, госпитальных баз и банно-прачечных пунктов; заменяет ММП-2. Состав: модули грязного и чистого белья, хранения воды (палатки), теплопункт и модуль ЗИП (контейнеры постоянного объёма), модули стирки и сушки (контейнеры переменного объёма), электростанция.',
        specs: {
          groups: [
            { title: null, params: [
              { label: 'Потребляемая мощность, кВт, не более', value: '100' },
              { label: 'Расход за 1 час работы дизельного топлива, л. — электростанция ЭСД-100-Т/400', value: '28' },
              { label: 'Расход за 1 час работы дизельного топлива, л. — электроагрегат дизельный АД-8С-Т/400', value: '2,9' },
              { label: 'Расход за 1 час работы дизельного топлива, л. — отопитель вентиляционный ОВ-95', value: '1,6' },
              { label: 'Расход за 1 час работы дизельного топлива, л. — котёл водогрейный КВ-ГМ-0,9-95', value: '8,3' },
              { label: 'Расход за 1 час работы дизельного топлива, л. — печь комбинированная «Вологда-2У»', value: '0,375/0,45' },
              { label: 'Расход воды за 1 час работы, л. — расходуется на стирку', value: '1370' },
              { label: 'Расход воды за 1 час работы, л. — при подключении модуля (до 8 душевых сеток) на помывку и стирку', value: '2970' },
              { label: 'Расход стирального порошка за 1 час работы, кг.', value: '2,25' },
              { label: 'Площадь развёртывания прачечной, м.', value: '17x27 (459 м²)' },
              { label: 'Время развёртывания, ч.', value: '4' },
              { label: 'Время свёртывания, ч.', value: '4' },
              { label: 'Марка автомобиля для транспортирования прачечной', value: 'КАМАЗ-43114' },
              { label: 'Количество автомобилей для транспортирования, шт.', value: '4' },
              { label: 'Скорость транспортирования, км/ч, не более — по дорогам с асфальтовым и бетонным покрытием', value: '70' },
              { label: 'Скорость транспортирования, км/ч, не более — по дорогам с булыжным покрытием', value: '50' },
              { label: 'Количество ж/д платформ для транспортирования прачечной', value: '3' },
              { label: 'Производительность технологическая с остаточной влажностью белья после сушки не более 10%, кг/ч', value: 'не менее 50' },
              { label: 'Производительность технологическая за 20 часов работы с остаточной влажностью белья не более 10%, кг.', value: 'не менее 1000' },
              { label: 'Количество обслуживающего персонала, чел. в смену', value: '8' },
              { label: 'Продолжительность работы одной смены, ч.', value: '10' },
              { label: 'Продолжительность непрерывной работы без дозаправки водой и топливом, ч.', value: '6' },
              { label: 'Вместимость топливных баков, л. — для котлов водогрейных КВ-ГМ-0,9-95', value: '2x60' },
              { label: 'Вместимость топливных баков, л. — для АД-8С-Т/400', value: '60' },
              { label: 'Вместимость топливных баков, л. — отопитель вентиляционный ОВ-95', value: '4x60' },
              { label: 'Вместимость топливных баков, л. — для ЭСД-100-Т/400', value: '150' },
              { label: 'Вместимость топливных баков, л. — печь комбинированная «Вологда-2У»', value: '6x18' }
            ]}
          ]
        },
        maintenance: {
          so: {
            actions: [
              'Сезонная подготовка всей поточной линии стирки',
              'Регулировка котельной установки под сезон',
              'Подготовка систем вентиляции и кондиционирования',
              'Проверка автономного дизель-генератора',
              'Замена сезонных смазочных материалов в редукторах',
              'Утепление контейнеров и трубопроводов (зима)',
              'Дезинфекция всего технологического оборудования'
            ],
            labor: 24.0,
            personnel: [P('Машинист банно-прачечного оборудования', 2), P('Дизелист', 1), P('Электрик', 1), P('Слесарь', 2)],
            tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Опрессовочный насос', 'Шумомер', 'Термометр'],
            materials: ['Смазка ЛИТОЛ-24', 'Масло моторное М10Г2К', 'Топливо дизельное', 'Антифриз ОЖ-65', 'Дезсредство', 'Уплотнения', 'Теплоизоляция']
          },
          to1: {
            actions: [
              'Внешний осмотр всех контейнеров и оборудования',
              'Очистка фильтров поточной линии стирки',
              'Проверка работы дозаторов СМС',
              'Контроль герметичности паропроводов',
              'Смазка подшипников и редукторов',
              'Проверка работы дизель-генератора под нагрузкой',
              'Очистка вентиляционных каналов'
            ],
            labor: 8.0,
            personnel: [P('Машинист банно-прачечного оборудования', 2), P('Электрик', 1), P('Дизелист', 1)],
            tools: ['Набор слесарного инструмента', 'Шприц для смазки', 'Мультиметр', 'Манометр'],
            materials: ['Смазка ЛИТОЛ-24', 'Уплотнения', 'Фильтры воздушные', 'Ветошь']
          },
          to2: {
            actions: [
              'Ревизия стиральных машин и центрифуг линии',
              'Замена ремней приводов барабанов',
              'Балансировка роторов центрифуг',
              'Проверка автоматики поточной линии',
              'Замена масла в редукторах приводов',
```

```
                'Ревизия дизель-генератора со сменой фильтров',
                'Проверка пожарной сигнализации',
                'Поверка контрольно-измерительных приборов'
              ],
              labor: 18.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь', 2), P('Электрик', 1), P('Дизелист', 1)],
              tools: ['Полный набор слесарного инструмента', 'Балансировочный станок', 'Мультиметр', 'Мегаомметр', 'Динамометрический ключ'],
              materials: ['Ремни клиновые (комплект)', 'Фильтры масляные и топливные', 'Масло моторное М10Г2К', 'Подшипники', 'Уплотнения', 'Изоляционная лента']
            },
            to3: {
              actions: [
                'Капитальный осмотр всех линий прачечной',
                'Замена изношенных узлов стиральных машин',
                'Полная ревизия дизель-генератора',
                'Замена силовых электрических кабелей',
                'Восстановление антикоррозионной защиты',
                'Замена теплоизоляции паропроводов',
                'Контрольная стирка с замером всех параметров',
                'Восстановление лакокрасочного покрытия контейнеров'
              ],
              labor: 48.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь по ремонту бытовой техники', 2), P('Электрик', 2), P('Дизелист', 1), P('Сварщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Балансировочный станок', 'Мегаомметр', 'Грузоподъёмное оборудование'],
              materials: ['Подшипники (комплект)', 'Тэны нагревательные', 'Теплоизоляция', 'Кабель силовой КГ', 'Эмаль', 'Электроды сварочные', 'Масло моторное']
            },
            nz: {
              actions: [
                'Расконсервация всех контейнеров и оборудования',
                'Развёртывание линии в месте дислокации',
                'Подключение коммуникаций',
                'Опробование всех технологических звеньев',
                'Контрольная стирка пробной партии (300 кг)',
                'Проверка комплектности по описи',
                'Замена расходных материалов с истёкшим сроком'
              ],
              labor: 28.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь', 2), P('Электрик', 1), P('Дизелист', 1)],
              tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Манометр', 'Грузоподъёмные приспособления'],
              materials: ['Топливо дизельное', 'Масло моторное', 'СМС', 'Уплотнения', 'Ветошь']
            }
          }
        },

        /* === ПУНКТ БАННО-ПРАЧЕЧНОГО ОБСЛУЖИВАНИЯ ПБПО-10 === */
        {
          id: 'pbpo-10',
          name: 'Пункт банно-прачечного обслуживания ПБПО-10',
          description: 'Предназначен для помывки личного состава малочисленных подразделений, находящихся в отрыве от главных сил или несущих боевое дежурство, а также для стирки и сушки белья и обмундирования в полевых условиях. Смонтирован в кузове-контейнере на автомобильном двухосном прицепе; в развёрнутом состоянии контейнер имеет агрегатный отсек, производственное отделение и тамбур. Оборудование: система отопления и вентиляции, электрооборудование с устройствами электробезопасности, система водоснабжения.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'База', value: 'автомобильный двухосный прицеп' },
                { label: 'Помывка, чел./ч', value: 'не менее 10' },
                { label: 'Стирка белья, кг/ч', value: 'не менее 10' },
                { label: 'Угол наклона рабочей площадки при развёртывании', value: 'не более 2°' },
                { label: 'Расход дизельного топлива, кг/ч — для передвижной электростанции при номинальной мощности', value: 'не более 20,8' },
                { label: 'Расход дизельного топлива, кг/ч — для отопительно-вентиляционных установок (2 шт.)', value: 'не более 2,64' },
                { label: 'Габариты прицепа, мм', value: '9940x2550x4000' },
                { label: 'Габаритные размеры контейнера в развёрнутом состоянии, мм', value: '5500x4670x2790 (+100)' },
                { label: 'Масса снаряженного контейнера на прицепе, кг.', value: 'не более 10 000' },
                { label: 'Условия эксплуатации для умеренного и холодного климата — температура, °С', value: 'от -40 до +50' },
                { label: 'Условия эксплуатации для умеренного и холодного климата — скорость ветра, м/с', value: '20 при -20 °С' },
                { label: 'Электроснабжение', value: 'от внешней сети 220/230 В или от автономного генератора' }
              ]}
            ]
          },
          maintenance: {
            so: {
              actions: [
                'Сезонная подготовка банного и прачечного отделений',
                'Регулировка водогрейных котлов',
                'Подготовка систем горячего водоснабжения',
                'Проверка водоподготовительной установки',
                'Утепление сооружений на зимний период',
                'Дезинфекция всех помещений',
```

```
                'Подготовка вентиляционных систем'
              ],
              labor: 18.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь по ремонту котельного оборудования', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Манометр', 'Термометр', 'Мультиметр', 'Опрыскиватель'],
              materials: ['Смазка ЛИТОЛ-24', 'Уплотнения котлов', 'Топливо', 'Дезсредство', 'Антинакипин', 'Ветошь']
            },
            to1: {
              actions: [
                'Внешний осмотр всех отделений пункта',
                'Очистка душевых сеток и моечных мест',
                'Проверка стиральных машин и центрифуг',
                'Контроль работы котлов',
                'Очистка фильтров водоподготовки',
                'Санитарная обработка помещений'
              ],
              labor: 6.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь', 1)],
              tools: ['Набор слесарного инструмента', 'Манометр', 'Щётка для удаления накипи'],
              materials: ['Антинакипин', 'Дезсредство', 'Фильтрующие элементы', 'Ветошь']
            },
            to2: {
              actions: [
                'Ревизия водогрейных котлов с очисткой от накипи',
                'Ревизия стиральных машин и центрифуг',
                'Замена уплотнений и прокладок',
                'Поверка контрольно-измерительных приборов',
                'Регулировка автоматики безопасности котлов',
                'Замена ремней приводов прачечного оборудования',
                'Проверка электрооборудования и заземления'
              ],
              labor: 15.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь по ремонту котельного оборудования', 1), P('Электрик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Опрессовочный насос', 'Мегаомметр', 'Стенд проверки клапанов'],
              materials: ['Уплотнения (комплект)', 'Антинакипин', 'Ремни клиновые', 'Подшипники', 'Изоляционная лента', 'Смазка ЛИТОЛ-24']
            },
            to3: {
              actions: [
                'Капитальный осмотр всего пункта',
                'Замена тэнов котлов',
                'Капитальный ремонт стиральных машин',
                'Замена изношенных трубопроводов',
                'Восстановление теплоизоляции системы',
                'Замена силовой электропроводки',
                'Восстановление лакокрасочного покрытия',
                'Контрольное опробование с проведением полного цикла'
              ],
              labor: 38.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь по ремонту котельного оборудования', 1), P('Слесарь по ремонту бытовой техники', 1), P('Электрик', 2), P('Сварщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Мегаомметр', 'Грузоподъёмные приспособления'],
              materials: ['Тэны нагревательные', 'Подшипники', 'Трубопроводы', 'Теплоизоляция', 'Кабель силовой', 'Эмаль', 'Электроды сварочные']
            },
            nz: {
              actions: [
                'Расконсервация банного и прачечного оборудования',
                'Опрессовка котлов рабочим давлением',
                'Опробование всех систем',
                'Контрольная помывка и стирка пробной партии',
                'Дезинфекция всех помещений',
                'Проверка комплектности по описи'
              ],
              labor: 22.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Опрессовочный насос', 'Манометр', 'Мультиметр'],
              materials: ['Топливо', 'Уплотнения', 'Дезсредство', 'СМС', 'Ветошь']
            }
          }
        },

        /* === ПОДВИЖНЫЙ БАННО-ПРАЧЕЧНЫЙ КОМПЛЕКС ПКБП-10 === */
        {
          id: 'pkbp-10',
          name: 'Подвижный банно-прачечный комплекс ПКБП-10',
          description: 'Предназначен для приёма, сортировки и стирки грязного белья, отжима, сушки, глажения и мелкого ремонта выстиранного белья, а также помывки военнослужащих в полевых условиях. Штатное предназначение — оснащение обмо дивизии, рмо бригад и отдельных полков; область применения — обслуживание малочисленных подразделений.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Производительность: стирка и сушка белья, кг/ч', value: 'не менее 10' },
                { label: 'Производительность: помывка личного состава, чел/ч', value: 'не менее 9' },
                { label: 'Максимальная мощность потребителей электроэнергии, кВт', value: '11,2' },
```

```
                { label: 'Скорость транспортирования своим ходом, км/ч', value: 'не более 75' },
                { label: 'Запас хода при скорости 40 км/ч, км., не менее', value: '600' },
                { label: 'Часовой расход топлива автомобиля, л/ч, не более — в движении/на стоянке при отборе мощности на генератор', value: '40/31' },
                { label: 'Часовой расход топлива котла, л/ч, не более — в минимальном (основном)/полном режиме', value: '19/38' },
                { label: 'Часовой расход топлива ОВ-65, л/ч, не более — в частичном/полном режиме', value: '08, /1,2' },
                { label: 'Часовой расход пара, кг/ч', value: 'не более 200' },
                { label: 'Часовой расход воды, м³/ч', value: 'не более 2,2' },
                { label: 'Объём топливных баков, л. — автомобиля', value: '240' },
                { label: 'Объём топливных баков, л. — котла', value: '150' },
                { label: 'Объём топливных баков, л. — отопительно-вентиляционной установки ОВ-65', value: '20' },
                { label: 'Масса, кг.', value: 'не более 18500' },
                { label: 'Масса агрегатов, кг., не более — моечного', value: '11500' },
                { label: 'Масса агрегатов, кг., не более — прачечного', value: '7000' },
                { label: 'Время перевода из транспортного положения в рабочее, мин. — при положительных/отрицательных температурах', value: '45/60' },
                { label: 'Время перевода из рабочего положения в транспортное, мин. — при положительных/отрицательных температурах', value: '45/60' },
                { label: 'Габаритные размеры (в транспортном положении), мм (ДхШхВ)', value: '14630x2900x3464' },
                { label: 'Габаритные размеры агрегата моечного (в транспортном положении), мм (ДхШхВ)', value: '8270x2550x3464' },
                { label: 'Габаритные размеры агрегата прачечного (в транспортном положении), мм (ДхШхВ)', value: '6840x2550x3137' }
              ]}
            ]
          },
          maintenance: {
            so: {
              actions: [
                'Сезонная подготовка автомобильных шасси',
                'Подготовка автономной котельной установки',
                'Регулировка систем водоподготовки',
                'Сезонное обслуживание дизель-генератора',
                'Проверка систем вентиляции контейнеров',
                'Утепление коммуникаций между контейнерами',
                'Дезинфекция всех помещений',
                'Контроль работы автономной системы водоснабжения'
              ],
              labor: 26.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Механик-водитель', 2), P('Дизелист', 1), P('Слесарь по ремонту котельного оборудования', 1), P('Электрик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Опрессовочный насос', 'Манометр', 'Шумомер', 'Термометр'],
              materials: ['Смазка ЛИТОЛ-24', 'Масло моторное М10Г2К', 'Топливо дизельное', 'Антифриз ОЖ-65', 'Уплотнения', 'Дезсредство', 'Антинакипин', 'Теплоизоляция']
            },
            to1: {
              actions: [
                'Внешний осмотр контейнеров и шасси',
                'Проверка работы котла и душевых',
                'Очистка фильтров водоподготовки',
                'Контроль работы прачечного оборудования',
                'Проверка дизель-генератора под нагрузкой',
                'Смазка узлов и агрегатов'
              ],
              labor: 9.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Механик-водитель', 1), P('Дизелист', 1)],
              tools: ['Набор слесарного инструмента', 'Манометр', 'Шприц для смазки', 'Мультиметр'],
              materials: ['Смазка ЛИТОЛ-24', 'Фильтры', 'Уплотнения', 'Ветошь']
            },
            to2: {
              actions: [
                'Ревизия котла с очисткой от накипи',
                'Ревизия стиральных машин и центрифуг',
                'Замена масла в дизель-генераторе',
                'Регулировка автоматики безопасности',
                'Проверка автомобильного шасси по нормативу ТО-2',
                'Поверка контрольно-измерительных приборов',
                'Балансировка центрифуг'
              ],
              labor: 22.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь по ремонту автомобилей', 1), P('Слесарь', 1), P('Электрик', 1), P('Дизелист', 1)],
              tools: ['Полный набор слесарного инструмента', 'Опрессовочный насос', 'Балансировочный станок', 'Мультиметр', 'Динамометрический ключ'],
              materials: ['Масло моторное', 'Фильтры (полный комплект)', 'Уплотнения', 'Ремни приводные', 'Антинакипин', 'Подшипники', 'Смазка ЛИТОЛ-24']
            },
            to3: {
              actions: [
                'Капитальный осмотр всего комплекса',
                'Капитальный ремонт котла',
                'Капитальный ремонт прачечного оборудования',
                'Полная ревизия дизель-генератора',
                'Капитальный осмотр автомобильных шасси',
                'Замена силовой проводки',
                'Восстановление лакокрасочного покрытия',
                'Контрольный заезд и опробование комплекса'
              ],
```

```
              labor: 55.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Слесарь по ремонту автомобилей', 2), P('Слесарь по ремонту котельного оборудования', 1), P('Дизелист', 1), P('Электрик', 2), P('Сварщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Балансировочный станок', 'Мегаомметр', 'Грузоподъёмное оборудование', 'Подъёмник автомобильный'],
              materials: ['Запчасти комплект котла', 'Подшипники', 'Тэны', 'Теплоизоляция', 'Кабель силовой', 'Эмаль', 'Электроды сварочные', 'Масло моторное', 'Антифриз']
            },
            nz: {
              actions: [
                'Расконсервация автомобилей и оборудования',
                'Развёртывание комплекса в месте дислокации',
                'Подключение коммуникаций между контейнерами',
                'Заправка ГСМ и водой',
                'Опробование всех систем комплекса',
                'Контрольная помывка и стирка',
                'Проверка комплектности по описи'
              ],
              labor: 32.0,
              personnel: [P('Машинист банно-прачечного оборудования', 2), P('Механик-водитель', 2), P('Слесарь', 1), P('Дизелист', 1)],
              tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Манометр', 'Грузоподъёмные приспособления'],
              materials: ['Топливо дизельное', 'Масло моторное', 'Антифриз', 'Уплотнения', 'СМС', 'Дезсредство', 'Ветошь']
            }
          }
        },

        /* === ПЕРЕДВИЖНОЙ ПУНКТ БЫТОВОГО ОБСЛУЖИВАНИЯ ПБП === */
        {
          id: 'pbp',
          name: 'Передвижной пункт бытового обслуживания ПБП',
          description: 'Предназначен для бытового обслуживания военнослужащих, находящихся вне пунктов постоянной дислокации в полевых условиях, в том числе на учениях, при вооружённых конфликтах и ликвидации последствий стихийных бедствий. Для обслуживания привлекаются: парикмахер (1 чел.), сапожник (1 чел.), портной (1 чел.). Время развёртывания (свёртывания) — не более 1 часа. Состоит из двух отделений: отделения бытового обслуживания (ОБО) и сушильного отделения (СО), смонтированных в базовом кузове-контейнере КК-4.2.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Масса снаряженного пункта, кг., не более', value: '5150' },
                { label: 'Суммарная потребляемая мощность электропотребителей (при использовании электростанции), кВт, не более', value: '8' },
                { label: 'Базовый кузов-контейнер', value: 'КК-4.2' },
                { label: 'Габаритные размеры (ДхШхВ), мм, не более', value: '5390x2518x2250' },
                { label: 'Внутренние размеры сушильного отделения (ДхШхВ), мм, не более', value: '2210x2320x1980' },
                { label: 'Внутренний объём сушильного отделения, м³, не более', value: '10,15' },
                { label: 'Площадь пола сушильного отделения, м², не более', value: '5,12' },
                { label: 'Количество комплектов обмундирования и обуви в сушильном отделении, шт., не более', value: '30' },
                { label: 'Внутренние размеры отделения бытового обслуживания (ДхШхВ), мм, не более', value: '1980x2320x1980' },
                { label: 'Внутренний объём отделения бытового обслуживания, м³, не более', value: '9,09' },
                { label: 'Площадь пола отделения бытового обслуживания, м², не более', value: '4,59' },
                { label: 'Вентиляторы 1,25ЭВ — номинальная подача воздуха приточным вентилятором в ОБО, м³/ч', value: '280' },
                { label: 'Вентиляторы 1,25ЭВ — суммарная подача воздуха тремя вытяжными вентиляторами из СО, м³/ч', value: '840' },
                { label: 'Тепловентиляторы ТПЦ-5 — номинальная подача воздуха одним тепловентилятором, м³/ч', value: '440' },
                { label: 'Фильтровентиляционная установка ФВУ-100А-24 — подача очищенного воздуха, м³/ч', value: '100' },
                { label: 'Отопительно-вентиляционная установка ОВ-95 — подача воздуха, м³/ч (полный/частичный режим)', value: '350/220' },
                { label: 'Отопительно-вентиляционная установка ОВ-95 — расход дизельного топлива, л/ч, не более', value: '1,6' },
                { label: 'Отопительно-вентиляционная установка ОВ-95 — вместимость топливного бака, л., не более', value: '20' },
                { label: 'Автономный источник электропитания АДП-10-Т/400 ВЛСБ — мощность, кВт, не более', value: '10' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — номинальное напряжение, В', value: '380' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — род тока', value: 'переменный 3-х фазный' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — сила тока, А, не более', value: '14,5' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — частота, Гц', value: '50' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — максимальный расход топлива, л/ч', value: '2,14' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — вместимость топливного бака, л., не более', value: '12,5' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — масса, кг., не более', value: '150,2' },
                { label: 'АИП АДП-10-Т/400 ВЛСБ — габаритные размеры, мм, не более', value: '960x600x715' }
              ]}
            ]
          },
          maintenance: {
            so: {
              actions: [
                'Сезонная подготовка автомобиля и кузова',
                'Подготовка системы отопления кузова',
                'Регулировка швейного и сапожного оборудования',
                'Проверка системы вентиляции',
                'Сезонное обслуживание шасси',
                'Замена сезонных смазочных материалов в швейных машинах'
```

```
              ],
              labor: 7.0,
              personnel: [P('Механик-водитель', 1), P('Мастер-парикмахер', 1), P('Слесарь', 1)],
              tools: ['Набор слесарного инструмента', 'Шприц для смазки', 'Набор отвёрток часовых', 'Мультиметр'],
              materials: ['Масло швейное И-12А', 'Смазка ЛИТОЛ-24', 'Антифриз ОЖ-65', 'Масло моторное', 'Ветошь']
            },
            to1: {
              actions: [
                'Внешний осмотр кузова и оборудования',
                'Очистка швейных машин от текстильной пыли',
                'Смазка швейного и сапожного оборудования',
                'Проверка работы парикмахерских приборов',
                'Контроль работы вентиляции',
                'Заточка парикмахерского инструмента'
              ],
              labor: 2.5,
              personnel: [P('Мастер-парикмахер', 1), P('Слесарь', 1)],
              tools: ['Набор инструментов', 'Шприц для смазки', 'Точильный камень', 'Щётка'],
              materials: ['Масло швейное', 'Смазка', 'Ветошь', 'Дезраствор']
            },
            to2: {
              actions: [
                'Ревизия швейных машин с разборкой',
                'Замена приводных ремней',
                'Регулировка натяжения нити в швейных машинах',
                'Ревизия сапожного оборудования',
                'Проверка электроинструмента (изоляция, сопротивление)',
                'Заточка и поверка профессионального инструмента',
                'Очистка вентиляционных систем'
              ],
              labor: 5.5,
              personnel: [P('Мастер-парикмахер', 1), P('Сапожник', 1), P('Слесарь', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Набор отвёрток часовых', 'Мегаомметр', 'Точильный станок'],
              materials: ['Ремни приводные', 'Иглы для швейных машин', 'Масло швейное', 'Изоляционная лента',
'Дезраствор']
            },
            to3: {
              actions: [
                'Капитальный осмотр кузова и оборудования',
                'Замена изношенных узлов швейных машин',
                'Замена электропроводки оборудования',
                'Восстановление лакокрасочного покрытия кузова',
                'Капитальный осмотр автомобильного шасси',
                'Замена парикмахерского кресла и оборудования при необходимости',
                'Контрольный заезд'
              ],
              labor: 14.0,
              personnel: [P('Механик-водитель', 1), P('Слесарь по ремонту автомобилей', 1), P('Электрик', 1),
P('Жестянщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Мегаомметр', 'Краскораспылитель'],
              materials: ['Запчасти швейных машин', 'Кабель электрический', 'Эмаль', 'Грунт ГФ-021', 'Электроды
сварочные', 'Антикоррозионный состав']
            },
            nz: {
              actions: [
                'Расконсервация автомобиля и оборудования',
                'Заправка ГСМ',
                'Опробование швейного и сапожного оборудования',
                'Проверка электрооборудования',
                'Дезинфекция парикмахерского отделения',
                'Проверка комплектности по описи'
              ],
              labor: 8.0,
              personnel: [P('Механик-водитель', 1), P('Мастер-парикмахер', 1), P('Слесарь', 1)],
              tools: ['Набор слесарного инструмента', 'Мультиметр', 'Шприц для смазки'],
              materials: ['Топливо', 'Масло моторное', 'Масло швейное', 'Дезраствор', 'Ветошь']
            }
          }
        },

        /* === ПЕРЕДВИЖНАЯ МАСТЕРСКАЯ ТЕХНИЧЕСКОГО ОБСЛУЖИВАНИЯ ПМТО-В === */
        {
          id: 'pmto-v',
          name: 'Передвижная мастерская технического обслуживания ПМТО-В',
          description: 'Предназначена для производства технического обслуживания и текущего ремонта технических средств
вещевой службы. Мастерская оборудована рабочими местами: автомеханика по обслуживанию и регулированию дизельной
топливной аппаратуры, карбюраторов и электрооборудования автомобилей; токаря; слесаря по обслуживанию и ремонту
оборудования прачечных; электрослесаря (электромеханика) по обслуживанию и мелкому ремонту электрооборудования
технических средств вещевой службы.',
          specs: {
            groups: [
              { title: 'Состав материальной части', params: [
                { label: 'Силовой агрегат (кузов-фургон КМ-4310 на шасси автомобиля КАМАЗ-4310), к-т', value: '1' },
                { label: 'Ремонтный агрегат (кузов-фургон КП-4А на шасси автоприцепа СМЗ-782-Б), к-т', value: '1' },
                { label: 'Переносное оборудование, к-т', value: '1' },
                { label: 'ИП, к-т', value: '1' }
              ]},
              { title: null, params: [
                { label: 'Площадка для развёртывания, м.', value: '10x10' },
                { label: 'Время развёртывания, ч.', value: '1' },
```

```
                { label: 'Время свёртывания, ч.', value: '0,5-0,75' },
                { label: 'Обслуживающий персонал, чел.', value: '5' },
                { label: 'Запас хода при скорости 60 км/ч, км., не менее — при электроснабжении от внешней сети', value: '800' },
                { label: 'Запас хода при скорости 60 км/ч, км., не менее — при электроснабжении от генератора', value: '400' },
                { label: 'Мощность потребителей электроэнергии, кВт, не более', value: '15,85' }
              ]}
            ]
          },
          maintenance: {
            so: {
              actions: [
                'Сезонная подготовка автомобиля-мастерской',
                'Подготовка станочного парка к сезонной эксплуатации',
                'Проверка системы освещения и вентиляции',
                'Подготовка автономного электропитания',
                'Поверка контрольно-измерительных приборов',
                'Сезонное обслуживание шасси',
                'Замена смазочных материалов в станках'
              ],
              labor: 11.0,
              personnel: [P('Механик-водитель', 1), P('Слесарь-ремонтник', 1), P('Электрик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Шприц для смазки', 'Манометр', 'Тестер изоляции'],
              materials: ['Масло индустриальное И-20А', 'Смазка ЛИТОЛ-24', 'Топливо дизельное', 'Антифриз', 'Изоляционная лента', 'Ветошь']
            },
            to1: {
              actions: [
                'Внешний осмотр мастерской и оборудования',
                'Очистка станков и оборудования от стружки',
                'Смазка станочного парка',
                'Проверка работы электрооборудования',
                'Контроль работы сварочного аппарата',
                'Проверка комплектности инструмента'
              ],
              labor: 3.5,
              personnel: [P('Слесарь-ремонтник', 1), P('Электрик', 1)],
              tools: ['Набор слесарного инструмента', 'Шприц для смазки', 'Мультиметр'],
              materials: ['Масло индустриальное И-20А', 'Смазка ЛИТОЛ-24', 'Ветошь']
            },
            to2: {
              actions: [
                'Ревизия станочного оборудования',
                'Замена масла в редукторах станков',
                'Проверка изоляции сварочного аппарата',
                'Поверка измерительных приборов',
                'Регулировка точности станков',
                'Замена щёток электродвигателей',
                'Проверка автомобильного шасси по ТО-2'
              ],
              labor: 9.0,
              personnel: [P('Слесарь-ремонтник', 1), P('Слесарь по ремонту автомобилей', 1), P('Электрик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Мегаомметр', 'Микрометр', 'Индикатор часового типа', 'Динамометрический ключ'],
              materials: ['Масло индустриальное', 'Масло моторное', 'Щётки электродвигателя (комплект)', 'Уплотнения', 'Фильтры', 'Смазка ЛИТОЛ-24']
            },
            to3: {
              actions: [
                'Капитальный осмотр мастерской и оборудования',
                'Замена изношенных узлов станков',
                'Замена силовой электропроводки',
                'Капитальный осмотр сварочного аппарата',
                'Восстановление лакокрасочного покрытия кузова',
                'Капитальный осмотр автомобильного шасси',
                'Контрольный заезд с замером тормозного пути'
              ],
              labor: 22.0,
              personnel: [P('Механик-водитель', 1), P('Слесарь по ремонту автомобилей', 1), P('Слесарь-ремонтник', 1), P('Электрик', 1), P('Сварщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Мегаомметр', 'Грузоподъёмное оборудование'],
              materials: ['Запчасти станочного парка', 'Кабель силовой КГ', 'Эмаль', 'Электроды сварочные', 'Антикоррозионный состав', 'Масло моторное']
            },
            nz: {
              actions: [
                'Расконсервация мастерской и оборудования',
                'Удаление консервационной смазки со станков',
                'Заправка ГСМ',
                'Опробование станочного парка',
                'Проверка работы сварочного и диагностического оборудования',
                'Поверка контрольно-измерительных приборов',
                'Проверка комплектности по описи'
              ],
              labor: 14.0,
              personnel: [P('Механик-водитель', 1), P('Слесарь-ремонтник', 1), P('Электрик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Мультиметр', 'Мегаомметр', 'Манометр'],
```

```
              materials: ['Топливо', 'Масло моторное', 'Масло индустриальное', 'Смазка ЛИТОЛ-24', 'Ветошь']
            }
          }
        },

        /* === ДДА-66 (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'dda-66',
          name: 'Дезинфекционно-душевая установка ДДА-66',
          description: 'Предназначена для помывки личного состава и дезинфекции (дезинсекции) обмундирования, обуви, белья и постельных принадлежностей в полевых условиях в любое время года при температуре окружающей среды от плюс 40°С до минус 30°С. Придаётся воинским частям РХБ защиты и медицинским подразделениям в звене «батальон».',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Тип парового котла', value: 'РИ-5М' },
                { label: 'Паропроизводительность котла при работе, кг/ч (на дизтопливе/на дровах)', value: '200/135' },
                { label: 'Расход топлива: дизельного, кг/ч / дров, м³/ч', value: '23,5/0,1' },
                { label: 'Ёмкость топливного бака, л.', value: '40+40' },
                { label: 'Расход воды при помывке людей, л/ч (летом/зимой)', value: '3000/2000' },
                { label: 'Количество душевых приборов/сеток, шт.', value: '1/6' },
                { label: 'Количество дезкамер, шт.', value: '1' },
                { label: 'Объём дезкамеры, м³', value: '2,5' },
                { label: 'Расход 40% раствора формалина на одну экспозицию, мл. — при заражении вегетативными формами микробов', value: '188' },
                { label: 'Расход 40% раствора формалина на одну экспозицию, мл. — при заражении спорообразующими формами микробов', value: '625' },
                { label: 'Удаление установки от водоисточника, м.', value: '8/100' },
                { label: 'Масса в походном положении, кг.', value: '5800' },
                { label: 'Габаритные размеры установок в походном положении (ДхШхВ), мм', value: '5800x2350x3000' },
                { label: 'База монтажа', value: 'ГАЗ-66' },
                { label: 'Обслуживающий персонал, чел.', value: '3' },
                { label: 'Площадка для развёртывания установки, м.', value: '30x30' }
              ]},
              { title: 'Возможности', params: [
                { label: 'Пропускная способность: помывка личного состава, чел/ч (летом/зимой)', value: '56/56' },
                { label: 'Помывка личного состава с одновременной дезинсекцией обмундирования, чел/ч (летом/зимой)', value: '56/56' },
                { label: 'Помывка личного состава с одновременной дезинфекцией обмундирования, заражённого вегетативными формами микроорганизмов, чел/ч — летом', value: '40' },
                { label: 'Помывка личного состава с одновременной дезинфекцией обмундирования, заражённого вегетативными формами микроорганизмов, чел/ч — зимой', value: '28' },
                { label: 'Расход котлом дизельного топлива, кг/ч', value: '23,5' },
                { label: 'Потребность в воде (летом/зимой), л/ч', value: '3000/2000' },
                { label: 'Режимы работы котла и ДК — ручной', value: 'Да' },
                { label: 'Режимы работы котла и ДК — автоматический режим розжига котла и контроля параметров', value: 'Нет' },
                { label: 'Режимы работы котла и ДК — ручной режим работы ДК', value: 'Да' },
                { label: 'Режимы работы котла и ДК — частично автоматический режим работы ДК (подача пара)', value: 'Нет' }
              ]}
            ]
          }
        },

        /* === ДДА-3 (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'dda-3',
          name: 'Дезинфекционно-душевая установка ДДА-3',
          description: 'Душевая дезинфекционная установка прицепная предназначена для полной санитарной обработки вещей и одежды с дезинфекцией (дезинсекцией) паровоздушным и пароформалиновым методами в очагах инфекционных заболеваний, обработки зон повышенной загрязнённости, одежды, обуви, белья и постельных принадлежностей, а также для помывки людей в полевых условиях во все времена года. Придаётся воинским частям РХБ защиты и медицинским подразделениям, входит в состав санитарно-эпидемиологических отрядов.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Тип парового котла', value: 'РИ-5М — 2 шт.' },
                { label: 'Паропроизводительность котла при работе, кг/ч (на дизтопливе/на дровах)', value: '2x200 / 2x135' },
                { label: 'Расход топлива: дизельного, кг/ч / дров, м³/ч', value: '2x23,5 / 2x0,1' },
                { label: 'Ёмкость топливного бака, л.', value: '150' },
                { label: 'Расход воды при помывке людей, л/ч (летом/зимой)', value: '6000/5000' },
                { label: 'Количество душевых приборов/сеток, шт.', value: '3/18' },
                { label: 'Количество дезкамер, шт.', value: '2' },
                { label: 'Объём дезкамеры, м³', value: '2,4' },
                { label: 'Расход 40% раствора формалина на одну экспозицию, мл. — при заражении вегетативными формами микробов', value: '180' },
                { label: 'Расход 40% раствора формалина на одну экспозицию, мл. — при заражении спорообразующими формами микробов', value: '625' },
                { label: 'Удаление установки от водоисточника, м.', value: '8/200' },
                { label: 'Масса в походном положении, кг.', value: '14275' },
                { label: 'Габаритные размеры установок в походном положении (длина/ширина/высота), мм', value: '12800/2345/3290' },
                { label: 'База монтажа', value: 'ЗИЛ-131 с автопр. СМЗ-8326' },
                { label: 'Обслуживающий персонал, чел.', value: '3-4' },
                { label: 'Площадка для развёртывания установки, м.', value: '50x30' }
              ]},
              { title: 'Возможности', params: [
                { label: 'Пропускная способность: помывка личного состава, чел/ч (летом/зимой)', value: '144/96' },
```

```
                { label: 'Помывка личного состава с одновременной дезинсекцией обмундирования, чел/ч (летом/зимой)', value: '144/96' },
                { label: 'Помывка личного состава с одновременной дезинфекцией обмундирования, заражённого вегетативными формами микроорганизмов, чел/ч (летом/зимой)', value: '96/48' },
                { label: 'Заражённого спорообразующими формами микроорганизмов, чел/ч (летом/зимой)', value: '24/16' },
                { label: 'Мощность, потребляемая, кВт', value: '3' },
                { label: 'Расход котлом дизельного топлива, кг/ч', value: '47' },
                { label: 'Потребность в воде (летом/зимой), л/ч', value: '6000/5000' },
                { label: 'Режимы работы котла и ДК — ручной', value: 'Да' },
                { label: 'Автоматический режим розжига котла и контроля параметров в работе', value: 'Нет' },
                { label: 'Ручной режим работы ДК', value: 'Да' },
                { label: 'Частично автоматический режим работы ДК (подача пара)', value: 'Нет' },
                { label: 'Помещение санпропускника', value: 'УСТ-56' }
              ]}
            ]
          }
        },

        /* === ДДП-2 (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'ddp-2',
          name: 'Дезинфекционно-душевая установка ДДП-2',
          description: 'Установка на шасси одноосного автоприцепа предназначена для дезинфекции и дезинсекции паровоздушным и пароформалиновым способом одежды, обуви, белья и постельных принадлежностей, а также для помывки людей в автономных (полевых) условиях во все времена года, в любых климатических зонах, кроме районов Крайнего Севера, при температуре окружающего воздуха от минус 30°С до плюс 40°С. Помывка людей может происходить одновременно с дезинсекцией их одежды. Пропускная способность установки указана при работе котла на жидком дизельном топливе; при работе на дровах пропускная способность уменьшается на 30-40%.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Тип парового котла', value: 'РИ-5М (РИ-2Л)' },
                { label: 'Паропроизводительность котла при работе, кг/ч (на дизтопливе/на дровах)', value: '200(150)/135(100)' },
                { label: 'Расход топлива: дизельного, кг/ч / дров, м³/ч', value: '23,5(21)/0,1' },
                { label: 'Ёмкость топливного бака, л.', value: '40+38' },
                { label: 'Расход воды при помывке людей, л/ч (летом/зимой)', value: '3000/2000' },
                { label: 'Количество душевых приборов/сеток, шт.', value: '1/6' },
                { label: 'Количество дезкамер, шт.', value: '1' },
                { label: 'Объём дезкамеры, м³', value: '2,0' },
                { label: 'Удаление установки от водоисточника, м.', value: '8' },
                { label: 'Масса в походном положении, кг.', value: '2930' },
                { label: 'Расход 40% раствора формалина на одну экспозицию, мл. — при заражении вегетативными формами микробов', value: '120' },
                { label: 'Расход 40% раствора формалина на одну экспозицию, мл. — при заражении спорообразующими формами микробов', value: '360' },
                { label: 'Габаритные размеры установок в походном положении (длина/ширина/высота), мм', value: '2975/2070/2363' },
                { label: 'База монтажа', value: 'автоприцеп ИАПЗ-738' },
                { label: 'Обслуживающий персонал, чел.', value: '2' },
                { label: 'Площадка для развёртывания установки, м.', value: '30х30' }
              ]},
              { title: 'Возможности', params: [
                { label: 'Пропускная способность: помывка личного состава, чел/ч (летом/зимой)', value: '48/48' },
                { label: 'Помывка личного состава с одновременной дезинсекцией обмундирования, чел/ч (летом/зимой)', value: '48/48' },
                { label: 'Помывка личного состава с одновременной дезинфекцией обмундирования, заражённого вегетативными формами микроорганизмов, чел/ч (летом/зимой)', value: '32/22' },
                { label: 'Дезинфекция обмундирования (без мойки), чел/ч (летом/зимой)', value: '75/45' }
              ]}
            ]
          }
        },

        /* === ДДК-1 (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'ddk-1',
          name: 'Комплекс дезинфекционно-душевой подвижной ДДК-1',
          description: 'Предназначен для проведения в полевых условиях санитарной обработки или гигиенической помывки раненых и больных (в том числе носилочных) личного состава войск, дезинфекции (дезинсекции) обмундирования и постельных принадлежностей. Комплекс состоит из двух установок: ДДУ-1 (размещена в кузове-фургоне на шасси автомобиля КамАЗ-43101 с комплектом ЗИП) и ДДУ-2 (в кузове-фургоне 2-ПН-4М).',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Полная масса комплекса, кг.', value: '20402' },
                { label: 'В том числе, кг.: ДДУ-1/ДДУ-2', value: '13420/6982' },
                { label: 'Габаритные размеры комплекса в походном положении (длина/ширина/высота), мм', value: '14560/2900/3310' },
                { label: 'Напряжение, В: переменного тока/постоянного тока', value: '380/24' },
                { label: 'Мощность, установленная на комплексе, кВт', value: '16' },
                { label: 'В том числе: ДДУ-1/ДДУ-2, кВт', value: '8/8' },
                { label: 'Время развёртывания комплекса из походного положения в рабочее, мин. (летом/зимой)', value: '60/90' },
                { label: 'Количество работающих душевых сеток, шт.', value: '20' },
                { label: 'В том числе, шт.: ДДУ-1/ДДУ-2', value: '10/10' },
                { label: 'Расход воды через каждую сетку, л/мин.', value: '4-6' }
              ]},
              { title: 'Возможности', params: [
                { label: 'Гигиеническая помывка, чел/ч: ДДК/ДДУ-1/ДДУ-2', value: '160/80/80' },
```

```
                { label: 'Гигиеническая помывка носилочных раненых и больных, чел/ч: ДДК/ДДУ-1/ДДУ-2', value: '18/9/9' },
                { label: 'Гигиеническая помывка с дезинфекцией по вегетативному режиму (дезинсекцией) обмундирования при ликвидации ОБЗ и в очагах инфекционных заболеваний, чел/ч: ДДК/ДДУ-1/ДДУ-2', value: '160/80/80' },
                { label: 'Гигиеническая помывка с дезинфекцией обмундирования по споровому режиму при ликвидации ОБЗ, чел/ч: ДДК/ДДУ-1/ДДУ-2', value: '120/60/60' },
                { label: 'Гигиеническая помывка с дезинфекцией обмундирования по споровому режиму в очагах инфекционных заболеваний, чел/ч: ДДК/ДДУ-1/ДДУ-2', value: '60/30/30' }
              ]}
            ]
          }
        },

        /* === ППБ-32 (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'ppb-32',
          name: 'Передвижная полевая баня ППБ-32',
          description: 'Предназначена для помывки личного состава, находящегося в полевых условиях вне пунктов постоянной дислокации. Система водоподготовки позволяет отказаться от применения традиционного парового котла и максимально упрощает работу обслуживающего персонала; поддержание постоянной температуры воды осуществляется автоматически. Порционная подача воды сокращает расход до 10 л/чел. Помывка организуется в кузове-фургоне, где размещены помывочный отсек с восемью душевыми сетками-дозаторами и раздевальный отсек; кузов-фургон оснащён системами отопления и кондиционирования воздуха.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Производительность по помывке, чел/ч', value: '32' },
                { label: 'Потребляемая мощность, кВт, не более', value: '30' },
                { label: 'Расход воды, л/ч, не более', value: '320' },
                { label: 'Расход воды на помывку 1 чел, л., не более', value: '10' },
                { label: 'Время нагрева воды до 40°С, мин., не более', value: '30' },
                { label: 'Расход дизельного топлива, л/ч, не более', value: '10' },
                { label: 'Запас воды, л. (возимый/выносной)', value: '1500/5000' },
                { label: 'Запас дизельного топлива, л.', value: '250' },
                { label: 'Время развёртывания (свёртывания), мин. (летом/зимой)', value: '60/90' },
                { label: 'Время непрерывной автономной работы (без дозаправки), ч., не менее', value: '8' },
                { label: 'Обслуживающий персонал, чел.', value: '2' },
                { label: 'Автомобильное шасси', value: 'КАМАЗ-6350 или УРАЛ-53236' }
              ]}
            ]
          }
        },

        /* === ППБ-32В (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'ppb-32v',
          name: 'Передвижная полевая баня ППБ-32В',
          description: 'Предназначена для помывки личного состава, находящегося в полевых условиях вне пунктов постоянной дислокации.',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Производительность по помывке, чел/ч — в базовой комплектации без доп. оборудования', value: '32' },
                { label: 'Производительность по помывке, чел/ч — с доп. оборудованием', value: '64' },
                { label: 'Потребляемая мощность (при работе от внешней сети), кВт, не более', value: '5' },
                { label: 'Расход воды на помывку 1 чел, л., не более', value: '10' },
                { label: 'Время нагрева воды до 40°С, мин., не более', value: '30' },
                { label: 'Расход дизельного топлива, кг/ч, не более', value: '16' },
                { label: 'Запас воды, л.', value: '5000' },
                { label: 'Время развёртывания (свёртывания), мин. (летом/зимой)', value: '60/90' },
                { label: 'Время непрерывной автономной работы (без дозаправки), ч., не менее', value: '8' },
                { label: 'Обслуживающий персонал, чел.', value: '2' },
                { label: 'Автомобильное шасси', value: 'КАМАЗ-6350, УРАЛ-532362, 2-ПН-8' },
                { label: 'Площадка для развёртывания, м.', value: '15х17' }
              ]}
            ]
          }
        },

        /* === Поезд БПДП (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'bpdp',
          name: 'Поезд банно-прачечно-дезинфекционный БПДП',
          description: 'Предназначен для санитарной обработки (помывки) личного состава, дезинфекции и стирки белья и обмундирования для войсковых объединений, учреждений и проходящих воинских эшелонов. Рассчитан для работы в полевых условиях при температуре наружного воздуха от -50 до +45°С и относительной влажности до 90%, в условиях прямого воздействия атмосферных осадков. Конструкция вагонов обеспечивает возможность дегазации, дезактивации и дезинфекции путём наружной обработки; для передислокации используется тепловоз или электровоз. Состав: вагоны кадра (вагон-штаб; 3 вагона помывочно-дезинфекционных; 6 вагонов-прачечных; вагон-химчистка; вагон-электростанция; вагон-склад; вагон грузовой; вагон очистки сточных вод) и приписные вагоны (3 вагона пассажирских; вагон-кухня-столовая или вагон-ресторан дальнего следования; 2 вагона-цистерны для воды; вагон-цистерна для дизельного топлива; 2 платформы универсальных).',
          specs: {
            groups: [
              { title: null, params: [
                { label: 'Конструкционная скорость, км/ч', value: 'до 120' },
                { label: 'Количество единиц подвижного состава, вагонов', value: '24' },
                { label: 'Мощность, вырабатываемая электроагрегатами вагона-электростанции, кВт', value: '400' },
                { label: 'Мощность, вырабатываемая электроагрегатами вагона-склада, кВт', value: '200' },
```

```
                { label: 'Штатные возможности поезда — санитарная обработка личного состава, чел/ч', value: '120' },
                { label: 'Гигиеническая помывка личного состава без учёта выносного душевого оборудования, чел/ч', value: '180' },
                { label: 'Гигиеническая помывка личного состава с учётом выносного душевого оборудования, чел/ч', value: '360' },
                { label: 'Стирка белья IV степени загрязнения, кг/ч', value: '300' },
                { label: 'Химическая чистка вещевого имущества, кг/ч', value: '48' },
                { label: 'Ремонт белья, % от общего количества', value: '20' },
                { label: 'Хранение грязного белья, т.', value: '8' },
                { label: 'Хранение чистого белья, т.', value: '9' }
              ]}
            ]
          }
        },

        /* === БПП-К (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'bpp-k',
          name: 'Банно-прачечный пункт БПП-К',
          description: 'Предназначен для помывки личного состава малочисленных воинских подразделений в отрыве от главных сил или при несении боевого дежурства (блокпосты, штабы, комендатуры и т. д.), а также для стирки белья и обмундирования силами военнослужащих в полевых условиях. В транспортном положении — автопоезд из моечного агрегата (кузов-фургон К-4310) и прачечного агрегата (кузов-фургон КП-4А); при переводе в рабочее положение автопоезд не расцепляется, выносное оборудование (насосная станция, резервуар РДВ-5000 и т. д.) размещается на рабочей площадке. В состав входят: моечный агрегат, прачечный агрегат, комплект рукавов и шлангов, комплект электрических кабелей, одиночный комплект ЗИП (ЗИП-0), комплект укладочных средств; для группы из 5 комплексов предусмотрен групповой комплект ЗИП (ЗИП-Г). Рабочая площадка — горизонтальная, не менее 25х20 м, с водоёмом (резервуаром) не далее 30 м от места установки.',
          specs: {
            groups: [
              { title: 'Оборудование, входящее в состав БПП-К', params: [
                { label: 'Контейнер, шт.', value: '1' },
                { label: 'Колонка топливная (водогрейная, в комплекте с горелкой, форсункой, баком для топлива, насосом с манометром), шт.', value: '1' },
                { label: 'Система водоснабжения, шт.', value: '1' },
                { label: 'Бак холодной воды, шт.', value: '1' },
                { label: 'Бак горячей воды, шт.', value: '1' },
                { label: 'Электрический щиток ввода, шт.', value: '1' },
                { label: 'Сиденье, шт.', value: '3' },
                { label: 'Раковина, шт.', value: '1' },
                { label: 'Вешалка, шт.', value: '1' },
                { label: 'Замок висячий, шт.', value: '2' }
              ]},
              { title: 'Основные технические характеристики и возможности', params: [
                { label: 'Время перевода пункта из транспортного положения в рабочее, ч.', value: '2' },
                { label: 'Производительность по помывке личного состава, чел/ч (моются одновременно 2 человека)', value: '8' },
                { label: 'Производительность по стирке белья, кг/ч', value: 'определяется применяемой стиральной машиной' },
                { label: 'Ёмкость баков, л. — холодной воды', value: '250' },
                { label: 'Ёмкость баков, л. — горячей воды', value: '250' },
                { label: 'Ёмкость топливной колонки, л., не менее', value: '24' },
                { label: 'Расход топлива, кг/ч — дизельного или керосина', value: '4,8' },
                { label: 'Расход топлива, кг/ч — дров средней влажности', value: '20' },
                { label: 'Габаритные размеры в рабочем положении (длина х ширина х высота), мм', value: '2710x3520x3000' },
                { label: 'Габаритные размеры при транспортировке (длина х ширина х высота), мм', value: '2710x2270x2735' },
                { label: 'Масса, кг.', value: '3000' }
              ]}
            ]
          }
        },

        /* === ПРМ-2В (данные из учебного пособия; регламент ТО не приведён) === */
        {
          id: 'prm-2v',
          name: 'Передвижная мастерская ремонта вещевого имущества ПРМ-2В',
          description: 'Предназначена для среднего ремонта обмундирования, кожаной и валяной обуви в полевых условиях. Организационно состоит из сортировочно-подготовительного отделения, отделения ремонта обмундирования и отделения ремонта обуви. Каждое отделение размещено на прицепе 2-ПН-4 со специальным раздвижным кузовом; для транспортировки мастерской используется 4 автомобиля.',
          specs: {
            groups: [
              { title: 'Основные технические характеристики и возможности', params: [
                { label: 'Штатная численность мастерской, чел.', value: '23' },
                { label: 'Площадка для развёртывания, м²', value: '23x26' },
                { label: 'Время развёртывания, ч. (зимой/летом)', value: '3/2' },
                { label: 'Время свёртывания, ч. (зимой/летом)', value: '2/1,5' },
                { label: 'Производительность мастерской за 10 часов работы (кожаной обуви / валяная обувь / пальто шерстяное, шт.; х/б, к-т) — как приведено в пособии', value: '170/200/40120' }
              ]},
              { title: 'Состав материальной части', params: [
                { label: 'Палатка УЗ-68 (УСТ-56), к-т', value: '1' },
                { label: 'Установка ОВ-65, шт.', value: '6' },
                { label: 'Электростанция ЭСД-20-Т/400, к-т', value: '1' }
              ]}
            ]
          }
        }
```

```
      ]
    },

    /* ============================================================
       СЛУЖБА 3: ПОЖАРНО-СПАСАТЕЛЬНАЯ И МЕСТНОЙ ОБОРОНЫ
       ============================================================ */
    {
      id: 'fire',
      name: 'Технические средства службы пожарно-спасательной и местной обороны',
      type: 'service',
      children: [
        /* === АЦ-6-40 (Урал-5557) === */
        {
          id: 'ac-6-40',
          name: 'Автоцистерна пожарная АЦ-6,0-40 (Урал-5557)',
          description: 'Пожарная автоцистерна на полноприводном шасси Урал-5557: цистерна для воды 6,0 м³, центробежный пожарный насос нормального давления НЦПН-40/100 (подача 40 л/с, напор 100 м), заднее расположение в насосном отсеке (по данным МЧС России).',
          maintenance: {
            so: {
              actions: [
                'Сезонная подготовка пожарного насоса ПН-40УВ',
                'Перевод цистерны на зимний/летний режим',
                'Заправка системы охлаждения насоса',
                'Подготовка системы пенообразования',
                'Утепление цистерны и трубопроводов (зима)',
                'Сезонное обслуживание шасси Урал-5557',
                'Проверка работы насоса от двигателя автомобиля',
                'Поверка манометров и мановакуумметров'
              ],
              labor: 14.0,
              personnel: [P('Водитель пожарного автомобиля', 1), P('Командир отделения', 1), P('Слесарь по ремонту автомобилей', 1)],
              tools: ['Набор слесарного инструмента', 'Манометр контрольный', 'Мановакуумметр', 'Динамометрический ключ', 'Опрессовочный насос'],
              materials: ['Пенообразователь ПО-6А', 'Масло моторное М10Г2К', 'Антифриз ОЖ-65', 'Уплотнения насоса', 'Топливо дизельное', 'Смазка ЛИТОЛ-24', 'Ветошь']
            },
            to1: {
              actions: [
                'Внешний осмотр автомобиля и пожарного оборудования',
                'Проверка уровня воды в цистерне',
                'Контроль уровня пенообразователя',
                'Проверка работы пожарного насоса (опробование от вакуумной системы)',
                'Контроль герметичности всасывающей линии',
                'Проверка пожарных рукавов на отсутствие повреждений',
                'Очистка фильтров всасывающей линии',
                'Смазка узлов насоса',
                'Контроль работы лафетного ствола'
              ],
              labor: 4.5,
              personnel: [P('Водитель пожарного автомобиля', 1), P('Командир отделения', 1)],
              tools: ['Набор слесарного инструмента', 'Манометр', 'Шприц для смазки', 'Вакуумметр'],
              materials: ['Пенообразователь ПО-6А (доливка)', 'Смазка ЛИТОЛ-24', 'Уплотнения', 'Ветошь']
            },
            to2: {
              actions: [
                'Полное опробование насоса ПН-40УВ на режимах',
                'Проверка коробки отбора мощности',
                'Замена масла в насосе и редукторе',
                'Регулировка газоструйного вакуум-аппарата',
                'Опрессовка цистерны и трубопроводов',
                'Поверка манометров и вакуумметров на стенде',
                'Проверка работы дозатора пенообразователя',
                'Гидравлические испытания пожарных рукавов',
                'Балансировка крыльчатки насоса при необходимости',
                'Регулировка предохранительных клапанов'
              ],
              labor: 12.0,
              personnel: [P('Водитель пожарного автомобиля', 1), P('Командир отделения', 1), P('Слесарь по ремонту пожарной техники', 1), P('Электрик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Стенд испытания насосов', 'Манометр контрольный', 'Вакуумметр', 'Опрессовочный насос', 'Динамометрический ключ', 'Стенд поверки рукавов'],
              materials: ['Масло М-10Г2К для насоса', 'Уплотнения насоса (комплект)', 'Пенообразователь ПО-6А', 'Прокладки', 'Сальники', 'Смазка ЛИТОЛ-24']
            },
            to3: {
              actions: [
                'Капитальный осмотр насоса ПН-40УВ',
                'Замена изношенных рабочих колёс и уплотнений',
                'Капитальный осмотр коробки отбора мощности',
                'Антикоррозионная обработка внутренней поверхности цистерны',
                'Замена пожарных рукавов с истёкшим сроком',
                'Капитальный осмотр шасси Урал-5557 по нормативу ТО-3',
                'Восстановление лакокрасочного покрытия',
                'Контрольный заезд и опробование на максимальных режимах',
                'Замена аварийных огнетушителей'
              ],
              labor: 36.0,
              personnel: [P('Водитель пожарного автомобиля', 1), P('Слесарь по ремонту пожарной техники', 1), P('Слесарь
```

```
 по ремонту автомобилей', 1), P('Электрик', 1), P('Сварщик', 1)],
3205             tools: ['Полный набор слесарного инструмента', 'Сварочный аппарат', 'Стенд испытания насосов',
    'Балансировочный станок', 'Грузоподъёмное оборудование', 'Подъёмник автомобильный'],
3206             materials: ['Рабочее колесо насоса', 'Уплотнения (полный комплект)', 'Подшипники', 'Пожарные рукава
    (комплект)', 'Эмаль красная сигнальная', 'Антикоррозионный состав', 'Электроды сварочные', 'Масло моторное']
3207           },
3208           nz: {
3209             actions: [
3210               'Расконсервация автомобиля и пожарного оборудования',
3211               'Удаление консервационной смазки с насоса',
3212               'Заправка ГСМ, водой, пенообразователем',
3213               'Опробование насоса от вакуумной системы',
3214               'Контрольное опробование подачи воды и пены',
3215               'Проверка комплектности ПТВ (пожарно-технического вооружения) по описи',
3216               'Проверка работы средств связи и сигнализации',
3217               'Контрольный заезд'
3218             ],
3219             labor: 16.0,
3220             personnel: [P('Водитель пожарного автомобиля', 1), P('Командир отделения', 1), P('Слесарь', 1)],
3221             tools: ['Набор слесарного инструмента', 'Манометр', 'Вакуумметр', 'Мультиметр'],
3222             materials: ['Топливо дизельное', 'Масло моторное', 'Антифриз', 'Пенообразователь ПО-6А', 'Уплотнения',
    'Ветошь']
3223           }
3224         }
3225       },
3226
3227       /* === ПОЖАРНАЯ МОТОПОМПА === */
3228       {
3229         id: 'fire-pump',
3230         name: 'Пожарная мотопомпа',
3231         description: 'Переносная пожарная мотопомпа МП-800 (МП-1600) с бензиновым двигателем для подачи воды на
    тушение пожаров при отсутствии стационарной водопроводной сети. Производительность — до 800 (1600) л/мин.',
3232         maintenance: {
3233           so: {
3234             actions: [
3235               'Сезонная подготовка двигателя мотопомпы',
3236               'Замена сезонного масла в двигателе',
3237               'Регулировка карбюратора под сезонные условия',
3238               'Проверка системы зажигания',
3239               'Очистка водопроводящей части насоса',
3240               'Проверка работы вакуумной системы',
3241               'Замена воздушного фильтра'
3242             ],
3243             labor: 3.5,
3244             personnel: [P('Слесарь по ремонту пожарной техники', 1), P('Слесарь по ремонту топливной аппаратуры', 1)],
3245             tools: ['Набор слесарного инструмента', 'Манометр', 'Вакуумметр', 'Свечной ключ', 'Щуп для свечей'],
3246             materials: ['Масло моторное М-12ГИ', 'Свечи зажигания', 'Воздушный фильтр', 'Топливо АИ-92', 'Уплотнения
    насоса', 'Ветошь']
3247           },
3248           to1: {
3249             actions: [
3250               'Внешний осмотр мотопомпы',
3251               'Проверка уровня масла и топлива',
3252               'Контроль работы вакуумной системы',
3253               'Проверка соединений всасывающих рукавов',
3254               'Очистка сетчатого фильтра всасывающего рукава',
3255               'Опробование на холостом ходу и рабочем режиме'
3256             ],
3257             labor: 1.5,
3258             personnel: [P('Слесарь по ремонту пожарной техники', 1)],
3259             tools: ['Набор слесарного инструмента', 'Манометр', 'Вакуумметр'],
3260             materials: ['Масло моторное (доливка)', 'Топливо АИ-92', 'Ветошь']
3261           },
3262           to2: {
3263             actions: [
3264               'Замена масла в двигателе',
3265               'Очистка и регулировка свечей зажигания',
3266               'Ревизия карбюратора с продувкой',
3267               'Регулировка зазоров клапанов',
3268               'Проверка компрессии в цилиндрах',
3269               'Опробование на максимальной производительности',
3270               'Поверка манометра и вакуумметра',
3271               'Очистка топливного бака'
3272             ],
3273             labor: 5.0,
3274             personnel: [P('Слесарь по ремонту пожарной техники', 1), P('Слесарь по ремонту топливной аппаратуры', 1)],
3275             tools: ['Набор слесарного инструмента', 'Компрессометр', 'Свечной ключ', 'Набор щупов', 'Манометр
    контрольный'],
3276             materials: ['Масло моторное М-12ГИ', 'Свечи зажигания (комплект)', 'Прокладки', 'Топливный фильтр',
    'Уплотнения насоса', 'Бензин для промывки']
3277           },
3278           to3: {
3279             actions: [
3280               'Капитальный осмотр двигателя мотопомпы',
3281               'Замена поршневых колец при необходимости',
3282               'Замена изношенного рабочего колеса насоса',
3283               'Замена уплотнений и сальников',
3284               'Восстановление лакокрасочного покрытия',
3285               'Опробование на стенде с замером производительности',
3286               'Замена приборов с истёкшим сроком поверки'
```

```
              ],
              labor: 10.0,
              personnel: [P('Слесарь по ремонту пожарной техники', 1), P('Слесарь-инструментальщик', 1)],
              tools: ['Полный набор слесарного инструмента', 'Стенд испытания мотопомп', 'Компрессометр', 'Микрометр'],
              materials: ['Поршневые кольца', 'Рабочее колесо насоса', 'Сальники', 'Уплотнения', 'Прокладки', 'Эмаль красная', 'Масло моторное']
            },
            nz: {
              actions: [
                'Расконсервация мотопомпы',
                'Удаление консервационной смазки',
                'Заправка ГСМ',
                'Опробование на холостом ходу',
                'Контрольный забор воды из открытого водоисточника',
                'Проверка комплектности (рукава, ствол, сетка) по описи'
              ],
              labor: 2.5,
              personnel: [P('Слесарь по ремонту пожарной техники', 1)],
              tools: ['Набор слесарного инструмента', 'Манометр', 'Вакуумметр', 'Свечной ключ'],
              materials: ['Масло моторное', 'Топливо АИ-92', 'Свечи зажигания', 'Уплотнения', 'Ветошь']
            }
          }
        }
      ]
    }
  ];
  /* ========================================================================
     ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ
     ======================================================================== */

  /* Безопасное экранирование HTML — защита от XSS при формировании разметки */
  function escapeHtml(s) {
    return String(s)
      .replace(/&/g, '&amp;')
      .replace(/</g, '&lt;')
      .replace(/>/g, '&gt;')
      .replace(/"/g, '&quot;')
      .replace(/'/g, '&#39;');
  }

  /* Подсветка совпадения с поисковым запросом */
  function highlight(text, query) {
    const raw = String(text);
    const q = String(query || '').trim();
    if (!q) return escapeHtml(raw);
    // Ищем совпадения в ИСХОДНОМ тексте и экранируем каждый кусок отдельно.
    // Раньше замена шла по уже экранированной строке, и запрос вроде «quot»
    // или «amp» вставлял <mark> внутрь &quot; / &amp;, ломая разметку
    const re = new RegExp(q.replace(/[.*+?^${}()|[\]\\]/g, '\\$&'), 'gi');
    let out = '', last = 0, m;
    while ((m = re.exec(raw)) !== null) {
      out += escapeHtml(raw.slice(last, m.index)) + '<mark>' + escapeHtml(m[0]) + '</mark>';
      last = m.index + m[0].length;
    }
    return out + escapeHtml(raw.slice(last));
  }

  /* Является ли узел листом (конкретным средством, а не группой) */
  function isLeaf(node) {
    return !node.children;
  }

  /* Поиск узла по id рекурсивно */
  function findById(nodes, id) {
    for (const n of nodes) {
      if (n.id === id) return n;
      if (n.children) {
        const f = findById(n.children, id);
        if (f) return f;
      }
    }
    return null;
  }

  /* Получить хлебные крошки (родителей) — список названий по пути до узла */
  function getBreadcrumb(nodes, id, path = []) {
    for (const n of nodes) {
      if (n.id === id) return [...path, n.name];
      if (n.children) {
        const r = getBreadcrumb(n.children, id, [...path, n.name]);
        if (r) return r;
      }
    }
    return null;
  }

  /* Текстовое содержимое узла (для поиска по всем полям) */
  function nodeFullText(node) {
    let parts = [node.name || '', node.description || ''];
    if (node.specs && node.specs.groups) {
```

```
      node.specs.groups.forEach(g => {
        if (g.title) parts.push(g.title);
        (g.params || []).forEach(p => parts.push(p.label, p.value));
      });
    }
    if (node.maintenance) {
      for (const k of Object.keys(node.maintenance)) {
        const m = node.maintenance[k];
        parts.push(...(m.actions || []));
        parts.push(...(m.tools || []));
        parts.push(...(m.materials || []));
        (m.personnel || []).forEach(p => parts.push(p.role));
      }
    }
    return parts.join(' \n ').toLowerCase();
  }

  /* SVG-иконки в виде строк (без внешних ресурсов) */
  const ICONS = {
    folder: '<svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M22 19a2 2 0 0 1-2 2H4a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h5l2 3h9a2 2 0 0 1 2 2z"/></svg>',
    group:  '<svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><rect x="3" y="3" width="7" height="7" rx="1"/><rect x="14" y="3" width="7" height="7" rx="1"/><rect x="3" y="14" width="7" height="7" rx="1"/><rect x="14" y="14" width="7" height="7" rx="1"/></svg>',
    item:   '<svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12" r="4"/><circle cx="12" cy="12" r="9"/></svg>',
    chevron:'<svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"><polyline points="9 6 15 12 9 18"/></svg>',
    actions:'<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polyline points="9 11 12 14 22 4"/><path d="M21 12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11"/></svg>',
    clock:  '<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12" r="10"/><polyline points="12 6 12 12 16 14"/></svg>',
    people: '<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0 7.75"/></svg>',
    wrench: '<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M14.7 6.3a1 1 0 0 0 0 1.4l1.6 1.6a1 1 0 0 0 1.4 0l3.77-3.77a6 6 0 0 1-7.94 7.94l-6.91 6.91a2.12 2.12 0 0 1-3-3l6.91-6.91a6 6 0 0 1 7.94-7.94l-3.76 3.76z"/></svg>',
    box:    '<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M21 16V8a2 2 0 0 0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4A2 2 0 0 0 3 8v8a2 2 0 0 0 1 1.73l7 4a2 2 0 0 0 2 0l7-4A2 2 0 0 0 21 16z"/><polyline points="3.27 6.96 12 12.01 20.73 6.96"/><line x1="12" y1="22.08" x2="12" y2="12"/></svg>',
    summary:'<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="16" y1="13" x2="8" y2="13"/><line x1="16" y1="17" x2="8" y2="17"/></svg>'
  };

  /* ============================================================================
     СОСТОЯНИЕ ПРИЛОЖЕНИЯ
     ============================================================================ */
  const state = {
    selectedId: null,        // id выбранного узла
    activeSection: 'specs',  // верхний раздел карточки: 'specs' | 'maintenance' | 'norms'
    activeTab: 'so',         // активный таб обслуживания
    query: '',               // строка поиска
    expanded: new Set(),     // раскрытые узлы дерева
    theme: 'light'           // текущая тема
  };

  // Спец-значение activeTab для вкладки «Сводка» — не настоящий вид
  // обслуживания, а агрегат по всем node.maintenance[...] сразу
  const SUMMARY_TAB = '__summary__';

  /* Плоский список конечных позиций каталога с путём до них */
  function catalogLeaves(nodes = CATALOG, path = [], out = []) {
    for (const n of nodes) {
      if (n.children) catalogLeaves(n.children, [...path, n.name], out);
      else out.push({ id: n.id, name: n.name, path, service: path[0] || '', node: n });
    }
    return out;
  }
  const CATALOG_LEAVES = catalogLeaves();

  /* ============================================================================
     ПРАВКИ ПОЛЬЗОВАТЕЛЯ В СПРАВОЧНИКЕ
     ----------------------------------------------------------------------------
     Исходный каталог (CATALOG) остаётся эталоном из учебного пособия.
     Правки хранятся в данных учёта (TST.fleet.catalogEdits(), попадают в
     резервную копию и синхронизацию вкладок) и накладываются поверх:
       { id, kind:'edit', catalogId, patch:{name,description,specs,maintenance} }
       { id, kind:'add',  parentId, node:{id,name,description,specs,maintenance} }
     Перед каждым наложением исходные поля восстанавливаются, поэтому
     «Сбросить правки» и откат удаления работают без перезагрузки.
     ============================================================================ */
  const BASE_NODE = new Map();   // id → исходные поля правленой позиции
  let userCatalogSig = '';
```

```
const EDITABLE_FIELDS = ['name', 'description', 'specs', 'maintenance'];

function removeUserNodes(nodes) {
  for (let i = nodes.length - 1; i >= 0; i--) {
    if (nodes[i].userAdded) nodes.splice(i, 1);
    else if (nodes[i].children) removeUserNodes(nodes[i].children);
  }
}

/* Скрытые встроенные позиции физически вынимаются из дерева (чтобы не
   учитывать их ни в поиске, ни в обзоре) и хранятся в стопке для возврата */
const HIDE_STASH = [];   // { parentId, index, node }
function restoreHidden() {
  while (HIDE_STASH.length) {
    const h = HIDE_STASH.pop();
    const arr = h.parentId === '' ? CATALOG : (findById(CATALOG, h.parentId) || {}).children;
    if (arr) arr.splice(Math.min(h.index, arr.length), 0, h.node);
  }
}
function hideNode(id) {
  const walk = (arr, parentId) => {
    for (let i = 0; i < arr.length; i++) {
      if (arr[i].id === id) { HIDE_STASH.push({ parentId, index: i, node: arr[i] }); arr.splice(i, 1); return true; }
      if (arr[i].children && walk(arr[i].children, arr[i].id)) return true;
    }
    return false;
  };
  return walk(CATALOG, '');
}

function applyUserCatalog() {
  const edits = (window.TST && TST.fleet && TST.fleet.catalogEdits && TST.fleet.catalogEdits()) || [];
  const sig = JSON.stringify(edits);
  if (sig === userCatalogSig) return false;
  userCatalogSig = sig;

  restoreHidden();
  removeUserNodes(CATALOG);
  BASE_NODE.forEach((base, id) => {
    const node = findById(CATALOG, id);
    if (!node) return;
    EDITABLE_FIELDS.forEach(f => { if (base[f] === undefined) delete node[f]; else node[f] =
JSON.parse(JSON.stringify(base[f])); });
    node.userEdited = false;
  });

  // Добавленные позиции и разделы. Раздел может лежать в другом добавленном
  // разделе — повторяем, пока что-то удаётся поставить на место
  let pending = edits.filter(e => e.kind === 'add' && e.node);
  while (pending.length) {
    const rest = [];
    pending.forEach(e => {
      const holder = e.parentId === '' ? CATALOG : (findById(CATALOG, e.parentId) || {}).children;
      if (!holder) { rest.push(e); return; }
      const n = JSON.parse(JSON.stringify(e.node));
      if (n.isGroup) { delete n.isGroup; n.type = e.parentId === '' ? 'service' : 'group'; n.children = []; }
      holder.push(Object.assign({ userAdded: true, userEdited: true }, n));
    });
    if (rest.length === pending.length) break;
    pending = rest;
  }

  edits.filter(e => e.kind === 'edit' && e.patch).forEach(e => {
    const node = findById(CATALOG, e.catalogId);
    if (!node || node.userAdded) return;
    // У разделов правится только наименование
    const fields = node.children ? ['name'] : EDITABLE_FIELDS;
    if (!BASE_NODE.has(node.id)) {
      const base = {};
      fields.forEach(f => { if (node[f] !== undefined) base[f] = JSON.parse(JSON.stringify(node[f])); });
      BASE_NODE.set(node.id, base);
    }
    fields.forEach(f => {
      if (!(f in e.patch)) return;
      const v = e.patch[f];
      if (f === 'maintenance' && (!v || !Object.keys(v).length)) delete node.maintenance;
      else if (f === 'specs' && (!v || !(v.groups || []).length)) delete node.specs;
      else node[f] = JSON.parse(JSON.stringify(v));
    });
    node.userEdited = true;
  });

  edits.filter(e => e.kind === 'hide').forEach(e => hideNode(e.catalogId));

  CATALOG_LEAVES.length = 0;
  CATALOG_LEAVES.push(...catalogLeaves());
  return true;
}

/* Автоопределение позиции справочника по наименованию / марке техники.
```

```
       Из текста берутся обозначения вида «КА-220», «ПАК-200М», «АЦ-6-40» и
       ищутся в наименованиях позиций как целые слова. Побеждает позиция, где
       совпало больше обозначений («Урал-5557 / АЦ-6-40» → АЦ-6-40, хотя шасси
       Урал-5557 есть у нескольких позиций). При равенстве — пусто: лучше не
       привязать, чем привязать не к тому. */
    function matchCatalog(text) {
      applyUserCatalog();
      const norm = s => String(s || '').toUpperCase().replace(/Ё/g, 'Е');
      const tokens = (norm(text).match(/[A-ZA-Я]{1,6}-\d+(?:[.,]\d+)?(?:-\d+)*[A-ZA-Я]?/g) || [])
        .filter(t => t.length >= 4);
      if (!tokens.length) return '';
      const res = tokens.map(t => new RegExp('(^|[^0-9A-ZA-Я])' + t.replace(/[.*+?^${}()|[\]\\]/g, '\\$&') +
    '(?![0-9A-ZA-Я])'));
      let best = '', bestScore = 0, tie = false;
      for (const l of CATALOG_LEAVES) {
        const name = norm(l.name);
        const score = res.filter(re => re.test(name)).length;
        if (score > bestScore) { best = l.id; bestScore = score; tie = false; }
        else if (score && score === bestScore) tie = true;
      }
      return tie ? '' : best;
    }

    /* Регламент ТО позиции: из учебного пособия, а если там его нет —
       пользовательский норматив из модуля учёта (TST.fleet.customMaint).
       Возвращает { so: {...}, to1: {...}, … } или null */
    function effectiveMaint(node) {
      if (!node) return null;
      if (node.maintenance) return node.maintenance;
      return (TST.fleet && TST.fleet.customMaint(node.id)) || null;
    }

    /* Раскрыть ветви дерева до узла, чтобы выбранная позиция была видна */
    function expandPathTo(id, nodes = CATALOG, trail = []) {
      for (const n of nodes) {
        if (n.id === id) { trail.forEach(t => state.expanded.add(t)); return true; }
        if (n.children && expandPathTo(id, n.children, [...trail, n.id])) return true;
      }
      return false;
    }

    /* Маршрут справочника: #catalog/<id>[/<вид ТО>|/summary] */
    TST.routeProviders.catalog = () => {
      if (!state.selectedId) return 'catalog';
      const node = findById(CATALOG, state.selectedId);
      const tab = state.activeSection === 'norms' ? '/norms'
        : state.activeSection === 'maintenance' && effectiveMaint(node)
        ? '/' + (state.activeTab === SUMMARY_TAB ? 'summary' : state.activeTab) : '';
      return 'catalog/' + encodeURIComponent(state.selectedId) + tab;
    };
    TST.routeHandlers.catalog = ([id, tab]) => {
      if (!id) return;
      if (tab === 'norms') TST.catalog.open(id, { section: 'norms' });
      else TST.catalog.open(id, { tab: tab === 'summary' ? SUMMARY_TAB : tab });
    };

    /* Сброс поиска справочника (используется общим поиском) */
    function clearCatalogSearch() {
      if (!state.query && !document.getElementById('searchInput').value) return;
      state.query = '';
      document.getElementById('searchInput').value = '';
      document.getElementById('searchClear').classList.remove('visible');
      renderTree();
      if (state.selectedId) renderDetail();
    }

    TST.catalog = {
      types: MAINT_TYPES,
      clearSearch: clearCatalogSearch,
      /* Позиции каталога, где встречается запрос (по наименованию — выше) */
      search(q, limit) {
        applyUserCatalog();
        const s = String(q || '').trim().toLowerCase();
        if (!s) return { items: [], total: 0 };
        const all = CATALOG_LEAVES.filter(l => nodeFullText(l.node).includes(s));
        all.sort((a, b) => (b.name.toLowerCase().includes(s) - a.name.toLowerCase().includes(s)) ||
    a.name.localeCompare(b.name, 'ru'));
        return { items: all.slice(0, limit || 6), total: all.length };
      },
      leaves: () => { applyUserCatalog(); return CATALOG_LEAVES; },
      get: id => { applyUserCatalog(); return id ? findById(CATALOG, id) : null; },
      path: id => { applyUserCatalog(); return (getBreadcrumb(CATALOG, id) || []).slice(0, -1); },
      /* Разделы, в которые можно добавить позицию: [{id, label}] */
      /* id раздела, в котором лежит позиция */
      parentOf(id) {
        applyUserCatalog();
        let found = '';
        const walk = (nodes, pid) => nodes.forEach(n => { if (n.id === id) found = pid; else if (n.children)
    walk(n.children, n.id); });
        walk(CATALOG, '');
```

```
      return found;
    },
    groups() {
      applyUserCatalog();
      const out = [];
      const walk = (nodes, path) => nodes.forEach(n => {
        if (!n.children) return;
        const p = [...path, n.name];
        out.push({ id: n.id, label: p.join(' › '), name: n.name, depth: path.length, userAdded: !!n.userAdded, userEdited: !!n.userEdited, leaves: catalogLeaves(n.children).length });
        walk(n.children, p);
      });
      walk(CATALOG, []);
      return out;
    },
    refresh() { renderTree(); if (state.selectedId && findById(CATALOG, state.selectedId)) renderDetail(); else { state.selectedId = null; renderDetail(); } },
    match: matchCatalog,
    maintenanceOf: id => { applyUserCatalog(); return effectiveMaint(id ? findById(CATALOG, id) : null); },
    hasBuiltIn: id => { applyUserCatalog(); const n = id ? findById(CATALOG, id) : null; return !!(n && n.maintenance); },
  },

    /* Открыть карточку позиции в справочнике; tab — ключ вида ТО ('to1', …)
       или '__summary__' — сразу на вкладке обслуживания */
    open(id, opts = {}) {
      applyUserCatalog();
      const node = findById(CATALOG, id);
      if (!node || node.children) return;
      TST.setWorkspace('catalog');
      // Поисковый фильтр мог скрыть позицию в дереве — сбрасываем его
      if (state.query) {
        state.query = '';
        const input = document.getElementById('searchInput');
        if (input) input.value = '';
        document.getElementById('searchClear')?.classList.remove('visible');
      }
      state.selectedId = id;
      expandPathTo(id);
      if (opts.section === 'norms') {
        state.activeSection = 'norms';
      } else if (opts.tab && effectiveMaint(node)) {
        state.activeSection = 'maintenance';
        state.activeTab = opts.tab;
      } else if (!opts.keepSection) {
        state.activeSection = 'specs';
      }
      renderTree();
      renderDetail();
      TST.syncRoute();
      document.getElementById('content').scrollTo({ top: 0 });
      document.querySelector(`.tree-row[data-id="${CSS.escape(id)}"]`)?.scrollIntoView({ block: 'center' });
    }
  };

  /* ========================================================================
     ПОСТРОЕНИЕ ДЕРЕВА КАТАЛОГА
     ======================================================================== */
  function buildTree(nodes, parentEl, level = 0) {
    for (const node of nodes) {
      const wrap = document.createElement('div');
      wrap.className = 'tree-node';
      wrap.dataset.id = node.id;
      if (state.expanded.has(node.id)) wrap.classList.add('is-open');

      const row = document.createElement('div');
      row.className = 'tree-row';
      row.dataset.id = node.id;
      if (!node.children) row.classList.add('is-leaf');
      if (state.selectedId === node.id) row.classList.add('is-active');
      // Доступность с клавиатуры: строка дерева — не нативная кнопка,
      // без tabindex/keydown её нельзя было ни получить фокусом, ни активировать Enter/Space
      row.tabIndex = 0;
      if (node.children) {
        row.setAttribute('role', 'button');
        row.setAttribute('aria-expanded', String(state.expanded.has(node.id)));
      } else {
        row.setAttribute('role', 'treeitem');
      }
      row.addEventListener('keydown', e => {
        if (e.key === 'Enter' || e.key === ' ') {
          e.preventDefault();
          onTreeRowClick(node);
        }
      });

      // Шеврон
      const chev = document.createElement('span');
      chev.className = 'tree-chevron';
      chev.innerHTML = ICONS.chevron;
      row.appendChild(chev);
```

```
      // Иконка
      const icn = document.createElement('span');
      icn.className = 'tree-icon';
      icn.innerHTML = isLeaf(node) ? ICONS.item : (level === 0 ? ICONS.folder : ICONS.group);
      row.appendChild(icn);

      // Название с подсветкой совпадения
      const lbl = document.createElement('span');
      lbl.className = 'tree-label';
      lbl.innerHTML = highlight(node.name, state.query);
      lbl.title = node.name;
      row.appendChild(lbl);

      // Связь с учётом: сколько единиц этого типа стоит на учёте. Цвет —
      // худшее состояние ТО среди них (та же шкала --st-*, что и в модуле учёта)
      if (!node.children && TST.fleet) {
        const units = TST.fleet.unitsFor(node.id);
        if (units.length) {
          const worst = TST.fleet.worstLevel(units);
          const cnt = document.createElement('span');
          cnt.className = 'tree-count lvl-' + worst;
          cnt.textContent = units.length;
          cnt.title = `На учёте: ${units.length} ед. · ${TST.fleet.levelLabel(worst)}`;
          row.appendChild(cnt);
        }
      }

      row.addEventListener('click', () => onTreeRowClick(node));
      wrap.appendChild(row);

      // Дочерние элементы
      if (node.children) {
        const childrenWrap = document.createElement('div');
        childrenWrap.className = 'tree-children';
        const inner = document.createElement('div');
        childrenWrap.appendChild(inner);
        buildTree(node.children, inner, level + 1);
        wrap.appendChild(childrenWrap);
      }

      parentEl.appendChild(wrap);
    }
  }

  /* Клик по строке дерева: либо раскрывает группу, либо открывает карточку */
  function onTreeRowClick(node) {
    if (node.children) {
      // Группа — раскрыть/свернуть
      if (state.expanded.has(node.id)) {
        state.expanded.delete(node.id);
      } else {
        state.expanded.add(node.id);
      }
      renderTree();
    } else {
      // Лист — выбрать средство
      state.selectedId = node.id;
      renderTree();
      renderDetail();
      TST.syncRoute();
      // На мобильных закрываем сайдбар
      if (window.matchMedia('(max-width: 900px)').matches) {
        closeSidebar();
      }
      // Прокрутить контент к верху и перевести фокус на заголовок карточки —
      // иначе после выбора клавиатурой/скринридером фокус остаётся на
      // удалённом узле дерева (renderTree его уже пересоздал)
      document.getElementById('content').scrollTo({ top: 0, behavior: 'smooth' });
      document.querySelector('.detail-title')?.focus({ preventScroll: true });
    }
  }

  /* Перерисовать дерево с учётом поиска */
  function renderTree() {
    applyUserCatalog();
    const root = document.getElementById('treeRoot');
    root.innerHTML = '';
    buildTree(CATALOG, root, 0);
    applySearchFilter();
  }

  /* Применение поискового фильтра — скрывает несовпадающие ветви */
  function applySearchFilter() {
    const q = state.query.trim().toLowerCase();
    const empty = document.getElementById('emptySearch');

    if (!q) {
      // Без поиска показываем всё
      document.querySelectorAll('.tree-row').forEach(r => r.classList.remove('is-hidden'));
```

```
      document.querySelectorAll('.tree-node').forEach(n => n.style.display = '');
      empty.style.display = 'none';
      return;
    }

    let anyVisible = false;

    // Рекурсивная функция: возвращает true, если узел сам или его дети содержат совпадение
    function filterNode(node, dom) {
      const ownMatch = nodeFullText(node).includes(q);
      let childMatch = false;

      if (node.children) {
        const childContainer = dom.querySelector(':scope > .tree-children > div');
        if (childContainer) {
          const childDoms = Array.from(childContainer.children);
          node.children.forEach((c, i) => {
            const childDom = childDoms[i];
            if (childDom && filterNode(c, childDom)) childMatch = true;
          });
        }
      }

      const visible = ownMatch || childMatch;
      dom.style.display = visible ? '' : 'none';
      if (visible) {
        anyVisible = true;
        // Раскрываем ветви, в которых есть совпадение
        if (node.children && childMatch) {
          dom.classList.add('is-open');
          state.expanded.add(node.id);
        }
      }
      return visible;
    }

    const rootDoms = Array.from(document.getElementById('treeRoot').children);
    CATALOG.forEach((n, i) => {
      if (rootDoms[i]) filterNode(n, rootDoms[i]);
    });

    empty.style.display = anyVisible ? 'none' : 'block';
  }

  /* ========================================================================
     ОТРИСОВКА ДЕТАЛЬНОЙ КАРТОЧКИ
     ======================================================================== */
  function renderDetail() {
    applyUserCatalog();
    const inner = document.getElementById('contentInner');
    if (!state.selectedId) {
      // Возврат к приветствию
      inner.innerHTML = `
        <div class="welcome">
          <div class="welcome-icon">
            <svg width="42" height="42" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.7" stroke-
linecap="round" stroke-linejoin="round">
              <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/>
              <polyline points="14 2 14 8 20 8"/>
              <line x1="9" y1="13" x2="15" y2="13"/>
              <line x1="9" y1="17" x2="15" y2="17"/>
            </svg>
          </div>
          <h2>Выберите средство</h2>
          <p>Иерархический каталог технических средств служб тыла — слева, а на телефоне открывается кнопкой  в шапке.
Выберите наименование, чтобы открыть карты технического обслуживания.</p>
        </div>`;
      return;
    }

    const node = findById(CATALOG, state.selectedId);
    if (!node || node.children) return;

    const crumbs = getBreadcrumb(CATALOG, state.selectedId) || [];
    // Из крошек убираем последнее (само наименование — оно в заголовке)
    const parents = crumbs.slice(0, -1);

    const q = state.query;
    const breadcrumbHtml = parents.map(p => `<span>${highlight(p, q)}</span>`).join('');
    const maintData = effectiveMaint(node);
    const hasMaint = !!maintData;
    // Нет данных по обслуживанию — показываются только характеристики (разметка
    // раздела «Обслуживание» не строится). Выбранный раздел не сбрасываем:
    // если норматив удалили и тут же вернули кнопкой «Отменить», пользователь
    // должен остаться там же, где был

    inner.innerHTML = `
      <div class="detail-header">
        <div class="detail-breadcrumb">${breadcrumbHtml}</div>
        <h1 class="detail-title" tabindex="-1">${highlight(node.name, q)}</h1>
```

```
        ${node.description ? `<p class="detail-description">${highlight(node.description, q)}</p>` : ''}
        ${renderEditBar(node)}
      </div>

      ${renderFleetLink(node)}
      ${node.maintenance ? '' : renderNormNotice(node, maintData)}

      <div class="detail-sections" role="tablist" aria-label="Раздел карточки">
        <button type="button" class="detail-section-tab ${state.activeSection === 'specs' ? 'is-active' : ''}"
                role="tab" aria-selected="${state.activeSection === 'specs'}" data-section="specs">
          ${ICONS.summary}<span>Характеристики</span>
        </button>
        ${hasMaint ? `<button type="button" class="detail-section-tab ${state.activeSection === 'maintenance' ? 'is-active' : ''}"
                role="tab" aria-selected="${state.activeSection === 'maintenance'}" data-section="maintenance">
          ${ICONS.wrench}<span>Обслуживание</span>
        </button>` : ''}
        <button type="button" class="detail-section-tab ${state.activeSection === 'norms' ? 'is-active' : ''}"
                role="tab" aria-selected="${state.activeSection === 'norms'}" data-section="norms">
          ${ICONS.actions}<span>Нормативная база</span>
        </button>
      </div>

      <div id="detailSectionSpecs" ${state.activeSection !== 'specs' && (hasMaint || state.activeSection === 'norms') ? 'hidden' : ''}>
        ${renderSpecs(node, q)}
      </div>

      <div id="detailSectionNorms" ${state.activeSection !== 'norms' ? 'hidden' : ''}>
        ${renderNormBase(node)}
      </div>

      ${hasMaint ? `
      <div id="detailSectionMaintenance" ${state.activeSection !== 'maintenance' ? 'hidden' : ''}>
        <div class="tabs" role="tablist" id="maintenanceTabs">
          ${MAINT_TYPES.map(t => `
            <button type="button" class="tab ${t.key === state.activeTab ? 'is-active' : ''}" role="tab" data-tab="${t.key}" title="${escapeHtml(t.full)}">
              ${t.short}
            </button>
          `).join('')}
          <button type="button" class="tab tab-summary ${state.activeTab === SUMMARY_TAB ? 'is-active' : ''}" role="tab"
                  data-tab="${SUMMARY_TAB}" title="Итоговая сводка по всем видам обслуживания">
            ${ICONS.box}<span>Сводка</span>
          </button>
        </div>

        <div id="tabContent"></div>
      </div>
      ` : ''}
    `;

    // Переключатель разделов «Характеристики / Обслуживание / Нормативная база»
    document.querySelectorAll('.detail-section-tab').forEach(btn => {
      btn.addEventListener('click', () => {
        state.activeSection = btn.dataset.section;
        document.querySelectorAll('.detail-section-tab').forEach(b => {
          const active = b === btn;
          b.classList.toggle('is-active', active);
          b.setAttribute('aria-selected', String(active));
        });
        document.getElementById('detailSectionSpecs').hidden = state.activeSection !== 'specs';
        document.getElementById('detailSectionNorms').hidden = state.activeSection !== 'norms';
        const m = document.getElementById('detailSectionMaintenance');
        if (m) m.hidden = state.activeSection !== 'maintenance';
        TST.syncRoute();
      });
    });

    if (hasMaint) {

      // Подвешиваем обработчики на табы вида обслуживания
      document.querySelectorAll('.tab').forEach(btn => {
        btn.addEventListener('click', () => {
          state.activeTab = btn.dataset.tab;
          // Обновляем активный класс
          document.querySelectorAll('.tab').forEach(b => b.classList.toggle('is-active', b === btn));
          renderTabContent(node);
          TST.syncRoute();
        });
      });

      renderTabContent(node);
    }
  }

  /* Связь с модулем учёта: единицы техники, привязанные к этой позиции */
  function renderFleetLink(node) {
    if (!TST.fleet) return '';
    const units = TST.fleet.unitsFor(node.id);
```

```
  const truck = '<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M10 17h4V5H2v12h3"/><path d="M20 17h2v-3.34a4 4 0 0 0-1.17-2.83L19 9h-5v8h2"/><circle cx="7.5" cy="17.5" r="2.5"/><circle cx="17.5" cy="17.5" r="2.5"/></svg>';
  return `
    <div class="card link-card">
      <div class="card-header">
        <div class="card-icon">${truck}</div>
        <div style="flex:1;min-width:0">
          <div class="card-title">На учёте в подразделении</div>
          <div class="card-subtitle">${units.length
            ? `${units.length} ед. в модуле «Технические средства тыла»`
            : 'Техника этого типа ещё не поставлена на учёт'}</div>
        </div>
        <button type="button" class="link-btn" data-link-add="${escapeHtml(node.id)}">+ Поставить на учёт</button>
      </div>
      ${units.length ? `
        <ul class="link-units">
          ${units.map(u => `
            <li>
              <button type="button" class="link-unit" data-link-vehicle="${escapeHtml(u.id)}">
                <span class="f-dot f-lvl-${u.level}"></span>
                <span class="link-unit-name">${escapeHtml(u.name)}${u.regNumber ? ` <span class="link-unit-plate">${escapeHtml(u.regNumber)}</span>` : ''}</span>
                <span class="link-unit-meta">${escapeHtml(u.nextText)}</span>
                <span class="f-badge f-badge-${u.level}">${escapeHtml(u.levelLabel)}</span>
              </button>
            </li>`).join('')}
        </ul>` : ''}
    </div>`;
}

/* Панель правки позиции: изменить / сбросить правки / удалить свою позицию */
function renderEditBar(node) {
  if (!TST.fleet || !TST.fleet.editCatalogNode) return '';
  const id = escapeHtml(node.id);
  return `<div class="detail-actions">
    ${node.userAdded ? '<span class="user-badge">Добавлено вами</span>' : node.userEdited ? '<span class="user-badge">Изменено вами</span>' : ''}
    <button type="button" class="link-btn" data-cat-edit="${id}">Изменить</button>
    ${node.userEdited && !node.userAdded ? `<button type="button" class="link-btn" data-cat-reset="${id}" title="Вернуть данные из учебного пособия">Сбросить правки</button>` : ''}
    ${node.userAdded ? `<button type="button" class="link-btn is-danger" data-cat-del="${id}">Удалить позицию</button>`
                     : `<button type="button" class="link-btn is-danger" data-cat-hide="${id}" title="Убрать позицию из справочника (можно вернуть в «Разделы»)">Скрыть позицию</button>`}
  </div>`;
}

/* Позиция без регламента в пособии: предложение ввести свой норматив */

/* ============================================================================
   НОРМАТИВНАЯ БАЗА ПОЗИЦИЙ СПРАВОЧНИКА
   ----------------------------------------------------------------------------
   Базовый документ (шифр РЭ / инструкции / ГОСТ), периодичность ЕО, ТО-1,
   ТО-2, СО и контрольные параметры. Вносятся ТОЛЬКО сведения, найденные в
   источниках (ссылка указывается в sources); всё, что не установлено, так
   и показывается — «не установлено, внести по РЭ образца».
   status: 'verified' — сверено с текстом РЭ;
           'partial'  — найден шифр документа или отдельные нормы;
           'none'     — сведений в открытых источниках не найдено.
   ============================================================================ */
const NORMBASE = {
  'ac-6-40': {
    status: 'partial',
    designation: 'Действующее обозначение образца: АЦ-6,0-40 (5557) — автоцистерна пожарная на шасси Урал-5557 (МЧС России)',
    docs: [
      { title: 'Руководство по эксплуатации насоса центробежного пожарного нормального давления НЦПН-40/100', code: 'КШИН.062223.011РЭ', note: 'Шифр установлен; полный текст раздела ТО не просмотрен' },
      { title: 'Руководство по эксплуатации (паспорт) насоса НЦПН-40/100УВМ.Э и НЦПН-40/100УВМ.01', code: 'ОАО «Ливенский завод противопожарного машиностроения»', note: 'Выдержки по вакуумной системе и маслобачку' },
      { title: 'Руководство по эксплуатации автомобиля Урал-5557 (шасси)', code: 'шифр не установлен', note: 'Перечень работ ТО на сайте завода помечен как неактуальный для современных модификаций' }
    ],
    intervals: {
      eo:  'Ежедневно (ЕТО) — для подразделений ФПС выполняется водителями заступающего и сменяющегося караулов (приказ МЧС России от 18.09.2012 № 555). Для Росгвардии — по руководству по технической службе: не установлено',
      to1: 'По инструкции завода-изготовителя шасси и надстройки (приказ МЧС России от 01.10.2020 № 737); значение не установлено',
      to2: 'По инструкции завода-изготовителя шасси и надстройки (приказ МЧС России от 01.10.2020 № 737); значение не установлено',
      so:  'Не установлено'
    },
    controls: [
      { param: 'Разрежение в насосе при пробном запуске вакуумной системы', norm: 'довести до 0,8 кгс/см²', src: 'РЭ НЦПН-40/100УВМ.01' },
      { param: 'Время заполнения насоса при разрежении 0,8 кгс/см²', norm: 'не более 40 с; иначе проверить герметичность коммуникаций', src: 'РЭ НЦПН-40/100УВМ.01' },
      { param: 'Разрежение при нормальной работе', norm: 'не менее 0,8 кгс/см²; иначе проверить контакты силовых цепей', src: 'РЭ НЦПН-40/100УВМ.Э' },
```

```
      { param: 'Масло в маслобачке', norm: 'проверить наличие, при необходимости долить до уровня; работа без масла запрещена; маслобачок не устанавливать выше дозатора', src: 'РЭ НЦПН-40/100УВМ.01' },
      { param: 'Заполнение рабочей полости насоса водой', norm: 'работа насоса без предварительного заполнения водой запрещена (торцевое уплотнение)', src: 'РЭ НЦПН-40/100УВМ.Э' },
      { param: 'Номинальные подача / напор насоса', norm: '40 л/с / 100 м (по обозначению НЦПН-40/100)', src: 'обозначение насоса' }
    ],
    sources: [
      { t: 'МЧС России (ГУ по Псковской обл.): АЦ-6,0-40 (5557)', url: 'https://60.mchs.gov.ru/glavnoe-upravlenie/sily-i-sredstva/tehnika/avtocisterna-pozharnaya-ac-40-131-137' },
      { t: 'РЭ НЦПН-40/100УВМ.Э (ЛЗПМ)', url: 'http://www.lzpm.ru/wp-content/uploads/2023/11/%D0%9D%D0%A6%D0%9F%D0%9D-40-100%D0%A3%D0%92%D0%9C.%D0%AD.pdf' },
      { t: 'РЭ НЦПН-40/100УВМ.01 (ЛЗПМ)', url: 'https://www.lzpm.ru/wp-content/uploads/2023/11/%D0%9D%D0%A6%D0%9F%D0%9D-40-100%D0%A3%D0%92%D0%9C.01.pdf' },
      { t: 'РЭ НЦПН-40/100, КШИН.062223.011РЭ', url: 'https://mchs.fun/rukovodstvo-po-ekspluataczii-nasosa-czentrobezhnogo-pozharnogo-normalnogo-davleniya-nczpn-40-100-kshin-062223-011re-2006/' },
      { t: 'Приказ МЧС России от 18.09.2012 № 555', url: 'https://www.glavbukh.ru/npd/edoc/99_499015416' },
      { t: 'Приказ МЧС России от 01.10.2020 № 737', url: 'https://rulaws.ru/acts/Prikaz-MCHS-Rossii-ot-01.10.2020-N-737/' },
      { t: 'УЗСТ: перечень работ ТО Урал-5557', url: 'https://uzst.ru/poleznaya-informaciya/poleznaya-informaciya-po-avtomobilyam-ural/instrukcii-po-ekspluatacii-avtomobilej/ural-5557/perechen-rabot-tehnicheskogo-obsluzhivaniya' }
    ]
  },
  'kp-130': {
    status: 'partial',
    docs: [
      { title: 'Кухня прицепная КП-130.11. Техническое описание и инструкция по эксплуатации (1990, 94 с.)', code: 'ТО 1446-77', note: 'Найдено по библиографическому описанию; текст разделов ТО не просмотрен' }
    ],
    sources: [ { t: 'Twirpx: «Кухня прицепная КП 130.11 ТО и ИЭ»', url: 'https://www.twirpx.com/file/2528570/' } ]
  },
  'phb-04': {
    status: 'none',
    designation: 'ПХБ-0,4: оборудование в кузове-фургоне на прицепе 2-ПН-4М; модификация ПХБ-0,4М — на шасси прицепа СМЗ-8326-01',
    sources: [
      { t: 'Техгарант: прицепной хлебопекарный блок ПХБ-0,4М', url: 'http://tehgarantbr.ru/produktsiya/item/pritsepnoj-khlebopekarnyj-blok-pkhb-04m' },
      { t: 'КонсультантПлюс: средства полевого хлебопечения', url: 'https://www.consultant.ru/document/cons_doc_LAW_510538/7839172d53f0f0b9b7ada8dc516ec5fc874b03df/' }
    ]
  },
  'atspt-47': {
    status: 'none',
    designation: 'АЦПТ-4,7 — автоцистерна 4615-01 на шасси Урал-4320-1121-41Ж (каталог ОАО «Завод Старт»). Регламент ТО шасси — по РЭ автомобиля Урал-4320',
    sources: [ { t: 'ibprom.ru: ОАО «Завод Старт»', url: 'https://ibprom.ru/zavod-start' } ]
  },
  'atspt-56': {
    status: 'none',
    designation: 'АЦПТ-5,6 — автоцистерна 562701 на шасси КамАЗ-43114-1029-15 (каталог ОАО «Завод Старт»). Регламент ТО шасси — по РЭ автомобиля КамАЗ-43114',
    sources: [ { t: 'ibprom.ru: ОАО «Завод Старт»', url: 'https://ibprom.ru/zavod-start' } ]
  },
  'pmh': {
    status: 'partial',
    docs: [
      { title: 'Польовий механізований хлібозавод ПМХ: технічний опис, основні технічні характеристики, порядок підготовки до застосування (Одеса, 2020)', code: 'учебно-справочное издание', note: 'Найдено в открытом доступе; текст не просмотрен' }
    ],
    sources: [ { t: 'Справочник по ПМХ (Одесса, 2020)', url: 'https://vaodesa.mil.gov.ua/wp-content/uploads/2024/10/Polovyy-mekhanizovanyy-khlibozavod-PMKH.pdf' } ]
  }
};
const NORM_STATUS = {
  verified: { label: 'Сверено с РЭ', cls: 'ok' },
  partial:  { label: 'Частично: найден документ / отдельные нормы', cls: 'soon' },
  none:     { label: 'Сведения в открытых источниках не найдены', cls: 'none' }
};
const normOf = id => NORMBASE[id] || { status: 'none' };

/* Раздел карточки «Нормативная база» */
function renderNormBase(node) {
  const n = normOf(node.id);
  const st = NORM_STATUS[n.status] || NORM_STATUS.none;
  const NA = '<span class="nb-na">не установлено — внести по РЭ образца</span>';
  const kinds = [['eo', 'ЕО'], ['to1', 'ТО-1'], ['to2', 'ТО-2'], ['so', 'СО']];
  return `
    <div class="card nb-card">
      <div class="card-header">
        <div class="card-icon">${ICONS.summary}</div>
        <div style="flex:1;min-width:0">
          <div class="card-title">Базовый нормативный документ</div>
          <div class="card-subtitle"><span class="nb-status nb-${st.cls}">${escapeHtml(st.label)}</span></div>
        </div>
      </div>
      ${n.designation ? `<p class="nb-note">${escapeHtml(n.designation)}</p>` : ''}
      ${n.docs && n.docs.length ? `<ul class="item-list">${n.docs.map(d => `
        <li><span><b>${escapeHtml(d.title)}</b>${d.note ? `<br><small class="nb-small">${escapeHtml(d.note)}</small>`
```

```
: ''}</span>
              <span class="item-meta nb-code">${escapeHtml(d.code || '—')}</span></li>`).join('')}</ul>`
          : `<p class="nb-note">${NA}</p>`}
      </div>

      <div class="card nb-card">
        <div class="card-header"><div class="card-icon">${ICONS.clock}</div>
          <div class="card-title">Виды и периодичность обслуживания</div></div>
        <ul class="item-list">${kinds.map(([k, label]) => `
          <li><span><b>${label}</b></span><span class="item-meta nb-val">${n.intervals && n.intervals[k] ?
escapeHtml(n.intervals[k]) : NA}</span></li>`).join('')}</ul>
      </div>

      <div class="card nb-card">
        <div class="card-header"><div class="card-icon">${ICONS.actions}</div>
          <div class="card-title">Контрольные параметры и критерии исправности</div></div>
        ${n.controls && n.controls.length ? `<div class="nb-scroll"><table class="nb-table">
          <thead><tr><th>Параметр</th><th>Норма / критерий</th><th>Источник</th></tr></thead>
          <tbody>${n.controls.map(c =>
`<tr><td>${escapeHtml(c.param)}</td><td>${escapeHtml(c.norm)}</td><td>${escapeHtml(c.src ||
'')}</td></tr>`).join('')}</tbody>
        </table></div>` : `<p class="nb-note">${NA}</p>`}
      </div>

      <div class="card nb-card">
        <div class="card-header"><div class="card-icon">${ICONS.box}</div>
          <div class="card-title">Источники</div></div>
        ${n.sources && n.sources.length ? `<ul class="item-list">${n.sources.map(x =>
`<li><span>${escapeHtml(x.t)}</span><span class="item-meta nb-src">${escapeHtml((() => { try { return
decodeURI(x.url); } catch (e) { return x.url; } })())}</span></li>`).join('')}</ul>`
          : '<p class="nb-note">Руководство по эксплуатации образца в открытых источниках не найдено. Внесите данные по
РЭ (формуляру) образца, имеющемуся в подразделении.</p>'}
        <p class="nb-note nb-small">Перечни работ, инструмента и материалов в разделе «Обслуживание» составлены по
учебным материалам и с РЭ не сверены, кроме сведений, указанных здесь со ссылкой на источник.</p>
      </div>`;
    }

    function renderNormNotice(node, maintData) {
      if (!TST.fleet) return '';
      const n = maintData ? Object.keys(maintData).length : 0;
      const warn = '<svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round"><path d="m21.73 18-8-14a2 2 0 0 0-3.48 0l-8 14A2 2 0 0 0 4 21h16a2 2 0
0 0 1.73-3z"/><line x1="12" y1="9" x2="12" y2="13"/><line x1="12" y1="17" x2="12.01" y2="17"/></svg>';
      return `
        <div class="card link-card norm-card">
          <div class="card-header">
            <div class="card-icon">${warn}</div>
            <div style="flex:1;min-width:0">
              <div class="card-title">${n ? 'Регламент ТО — норматив пользователя' : 'Регламент ТО в учебном пособии не
приведён'}</div>
              <div class="card-subtitle">${n
                ? `Описано видов ТО: ${n} из ${MAINT_TYPES.length}. Хранится в данных учёта и попадает в резервную копию.`
                : 'Можно ввести свой норматив (например, по руководству по эксплуатации) — он будет использоваться в
карточках техники, записях ТО и заявках.'}</div>
            </div>
            <button type="button" class="link-btn" data-norm-edit="">+ ${n ? 'Добавить вид ТО' : 'Ввести
норматив'}</button>
          </div>
        </div>`;
    }

    /* Карточки технических характеристик узла (данные из учебного пособия) */
    function renderSpecs(node, q) {
      if (!node.specs || !Array.isArray(node.specs.groups) || !node.specs.groups.length) return '';

      return `
        <div class="card-grid-two" style="margin-bottom: 16px;">
          ${node.specs.groups.map(g => `
            <div class="card" style="grid-column: 1 / -1;">
              <div class="card-header">
                <div class="card-icon">${ICONS.summary}</div>
                <div>
                  <div class="card-title">Основные технические характеристики${g.title ? ' — ' + escapeHtml(g.title) :
''}</div>
                  <div class="card-subtitle">${node.userAdded ? 'Введено пользователем' : node.userEdited ? 'С правками
пользователя' : 'По данным учебного пособия'}</div>
                </div>
              </div>
              <ul class="item-list">
                ${g.params.map(p => `
                  <li>
                    <span>${highlight(p.label, q)}</span>
                    <span class="item-meta">${highlight(p.value, q)}</span>
                  </li>
                `).join('')}
              </ul>
            </div>
          `).join('')}
        </div>
      `;
```

```
  }

  /* Контент текущей вкладки обслуживания: либо один вид ТО, либо сводка по всем сразу */
  function renderTabContent(node) {
    const wrap = document.getElementById('tabContent');
    const q = state.query;

    if (state.activeTab === SUMMARY_TAB) {
      wrap.innerHTML = renderSummary(node, q);
      return;
    }

    const maint = effectiveMaint(node)[state.activeTab];
    const type  = MAINT_TYPES.find(t => t.key === state.activeTab);

    if (!maint) {
      // Для позиций без регламента в пособии недостающий вид ТО можно описать самим
      wrap.innerHTML = `<p style="color: var(--text-secondary)">Данные обслуживания отсутствуют.</p>
        ${!node.maintenance && TST.fleet ? `<button type="button" class="link-btn" style="margin-top:12px" data-norm-edit="${escapeHtml(state.activeTab)}">+ Ввести норматив «${escapeHtml(type.short)}»</button>` : ''}`;
      return;
    }

    // Формируем HTML вкладки
    wrap.innerHTML = `
      <div class="card" style="margin-bottom: 16px;">
        <div class="card-header">
          <div class="card-icon">${ICONS.summary}</div>
          <div style="flex:1;min-width:0">
            <div class="card-title">${escapeHtml(type.full)}</div>
            <div class="card-subtitle">${escapeHtml(type.desc)}${maint.custom ? ' · <b>норматив введён пользователем</b>' : ' · <span class="nb-status nb-none">по учебным материалам, с РЭ не сверено</span>'}</div>
          </div>
          <div class="card-actions">
            ${TST.fleet ? `<button type="button" class="link-btn" data-print-card="${escapeHtml(state.activeTab)}" title="Печать карты ТО с отметками о выполнении">Печать карты</button>` : ''}
            ${maint.custom ? `<button type="button" class="link-btn" data-norm-edit="${escapeHtml(state.activeTab)}">Изменить</button>
              <button type="button" class="link-btn is-danger" data-norm-del="${escapeHtml(state.activeTab)}">Удалить</button>` : ''}
          </div>
        </div>
      </div>

      <div class="card-grid-two">
        <!-- Перечень действий -->
        <div class="card" style="grid-column: 1 / -1;">
          <div class="card-header">
            <div class="card-icon">${ICONS.actions}</div>
            <div class="card-title">Перечень действий</div>
          </div>
          <ol class="action-list">
            ${maint.actions.map(a => `<li><span>${highlight(a, q)}</span></li>`).join('')}
          </ol>
        </div>

        <!-- Трудоёмкость -->
        <div class="card">
          <div class="card-header">
            <div class="card-icon">${ICONS.clock}</div>
            <div class="card-title">Трудоёмкость</div>
          </div>
          <div class="metric">
            <div class="metric-value">${maint.labor.toFixed(1)}</div>
            <div class="metric-unit">человеко-часов</div>
          </div>
        </div>

        <!-- Персонал -->
        <div class="card">
          <div class="card-header">
            <div class="card-icon">${ICONS.people}</div>
            <div>
              <div class="card-title">Состав персонала</div>
              <div class="card-subtitle">Всего: ${maint.personnel.reduce((s,p) => s + p.count, 0)} чел.</div>
            </div>
          </div>
          <ul class="item-list">
            ${maint.personnel.map(p => `
              <li>
                <span>${highlight(p.role, q)}</span>
                <span class="item-meta">${p.count} чел.</span>
              </li>
            `).join('')}
          </ul>
        </div>

        <!-- Инструменты -->
        <div class="card">
          <div class="card-header">
```

```
            <div class="card-icon">${ICONS.wrench}</div>
            <div>
              <div class="card-title">Инструменты и оснастка</div>
              <div class="card-subtitle">${maint.tools.length} наименований</div>
            </div>
          </div>
          <ul class="item-list numbered">
            ${maint.tools.map(t => `<li><span>${highlight(t, q)}</span></li>`).join('')}
          </ul>
        </div>

        <!-- Расходные материалы -->
        <div class="card">
          <div class="card-header">
            <div class="card-icon" style="background: rgba(52,199,89,0.12); color: var(--st-ok-text)">${ICONS.box}</div>
            <div>
              <div class="card-title">Расходные материалы</div>
              <div class="card-subtitle">${maint.materials.length} наименований</div>
            </div>
          </div>
          <ul class="item-list materials numbered">
            ${maint.materials.map(m => `<li><span>${highlight(m, q)}</span></li>`).join('')}
          </ul>
        </div>
      </div>
    `;
  }

  /* Сводный блок: общая трудоёмкость по ВСЕМ видам обслуживания + перечни.
     Отдельная вкладка «Сводка», а не всегда развёрнутый блок под каждым видом ТО */
  function renderSummary(node, q) {
    const allMaint = Object.values(effectiveMaint(node));

    // Общая трудоёмкость
    const totalLabor = allMaint.reduce((s, m) => s + (m.labor || 0), 0);

    // Суммарный персонал (по специальностям)
    const personMap = new Map();
    allMaint.forEach(m => {
      (m.personnel || []).forEach(p => {
        personMap.set(p.role, Math.max(personMap.get(p.role) || 0, p.count));
      });
    });
    const totalPersonnel = Array.from(personMap.values()).reduce((s, c) => s + c, 0);

    // Уникальные инструменты и материалы
    const allTools = [...new Set(allMaint.flatMap(m => m.tools || []))];
    const allMaterials = [...new Set(allMaint.flatMap(m => m.materials || []))];

    return `
      <div class="summary">
        <div class="summary-title">Итоговая сводка по всем видам обслуживания</div>
        <div class="summary-grid">
          <div class="summary-stat">
            <div class="summary-stat-label">Суммарная трудоёмкость</div>
            <div class="summary-stat-value">${totalLabor.toFixed(1)}<small>чел·ч</small></div>
          </div>
          <div class="summary-stat">
            <div class="summary-stat-label">Видов обслуживания</div>
            <div class="summary-stat-value">${allMaint.length}<small>СО / ТО / Н3</small></div>
          </div>
          <div class="summary-stat">
            <div class="summary-stat-label">Привлекаемый персонал</div>
            <div class="summary-stat-value">${totalPersonnel}<small>чел.</small></div>
          </div>
          <div class="summary-stat">
            <div class="summary-stat-label">Уникальных инструментов</div>
            <div class="summary-stat-value">${allTools.length}<small>наим.</small></div>
          </div>
          <div class="summary-stat">
            <div class="summary-stat-label">Расходных материалов</div>
            <div class="summary-stat-value">${allMaterials.length}<small>наим.</small></div>
          </div>
        </div>

        <div class="summary-section">
          <h4>Полный перечень специальностей</h4>
          <ul class="item-list">
            ${Array.from(personMap.entries()).map(([role, n]) => `
              <li>
                <span>${highlight(role, q)}</span>
                <span class="item-meta">${n} чел.</span>
              </li>
            `).join('')}
          </ul>
        </div>

        <div class="summary-section">
          <h4>Полный перечень инструментов и оснастки</h4>
          <ul class="item-list numbered">
```

```
            ${allTools.map(t => `<li><span>${highlight(t, q)}</span></li>`).join('')}
          </ul>
        </div>

        <div class="summary-section">
          <h4>Полный перечень расходных материалов</h4>
          <ul class="item-list materials numbered">
            ${allMaterials.map(m => `<li><span>${highlight(m, q)}</span></li>`).join('')}
          </ul>
        </div>
      </div>
    `;
  }

  /* ============================================================================
     ТЕМА (светлая / тёмная) с сохранением выбора в localStorage
     ============================================================================ */
  /* Выбор пользователя: 'light' | 'dark' | 'system'. В <html data-theme>
     всегда записывается итоговая тема, поэтому стили по-прежнему опираются
     только на [data-theme="dark"] */
  const THEME_PREFS = ['light', 'system', 'dark'];
  const darkQuery = window.matchMedia ? window.matchMedia('(prefers-color-scheme: dark)') : null;

  function applyTheme(pref) {
    if (THEME_PREFS.indexOf(pref) === -1) pref = 'system';
    const theme = pref === 'system' ? (darkQuery && darkQuery.matches ? 'dark' : 'light') : pref;
    state.themePref = pref;
    state.theme = theme;
    document.documentElement.setAttribute('data-theme', theme);
    try { localStorage.setItem('spravochnik-tst-theme', pref); } catch (e) { /* приватный режим */ }

    const sw = document.getElementById('themeToggle');
    if (!sw) return;
    sw.dataset.t = pref;
    sw.querySelectorAll('[data-theme-set]').forEach(b => {
      const on = b.dataset.themeSet === pref;
      b.setAttribute('aria-checked', String(on));
      b.tabIndex = on ? 0 : -1;
    });
  }

  function initTheme() {
    let pref = 'system';
    try {
      const saved = localStorage.getItem('spravochnik-tst-theme');
      if (THEME_PREFS.indexOf(saved) !== -1) pref = saved;
    } catch (e) { /* приватный режим */ }
    applyTheme(pref);
    // «Авто» следует за системной темой и после смены её на лету
    if (darkQuery) {
      const follow = () => { if (state.themePref === 'system') applyTheme('system'); };
      if (darkQuery.addEventListener) darkQuery.addEventListener('change', follow);
      else if (darkQuery.addListener) darkQuery.addListener(follow);
    }
  }

  /* ============================================================================
     САЙДБАР НА МОБИЛЬНЫХ — открытие/закрытие
     ============================================================================ */
  function openSidebar()  {
    document.getElementById('sidebar').classList.add('is-open');
    document.getElementById('scrim').classList.add('is-visible');
    // Переносим фокус в панель — иначе после открытия по клавиатуре
    // фокус остаётся на кнопке-гамбургере под сдвинутым сайдбаром
    document.querySelector('#sidebar .tree-row')?.focus();
  }
  function closeSidebar() {
    const sidebar = document.getElementById('sidebar');
    // Возвращаем фокус на кнопку-гамбургер, только если он реально был
    // внутри сайдбара (Escape/скрим) — при закрытии после выбора пункта
    // дерева фокус уже переведён на карточку, и его не нужно перехватывать
    const shouldRestoreFocus = sidebar.contains(document.activeElement);
    sidebar.classList.remove('is-open');
    document.getElementById('scrim').classList.remove('is-visible');
    if (shouldRestoreFocus) document.getElementById('menuToggle').focus();
  }

  /* ============================================================================
     ИНИЦИАЛИЗАЦИЯ ПРИЛОЖЕНИЯ
     ============================================================================ */
  function init() {
    initTheme();

    // Раскрываем корневые узлы по умолчанию
    CATALOG.forEach(n => state.expanded.add(n.id));
    // Также раскрываем подгруппы первой ветви для дружелюбного старта
    if (CATALOG[0] && CATALOG[0].children) {
      CATALOG[0].children.forEach(c => state.expanded.add(c.id));
    }

```

```
    renderTree();
    renderDetail();

    // Поиск с дебаунсом для плавности
    const searchInput = document.getElementById('searchInput');
    const searchClear = document.getElementById('searchClear');
    const searchWrap = document.getElementById('catalogSearchWrap');
    const isMobileSearch = () => window.matchMedia('(max-width: 560px)').matches;
    let searchTimer = null;
    searchInput.addEventListener('input', () => {
      const value = searchInput.value;
      searchClear.classList.toggle('visible', value.length > 0);
      clearTimeout(searchTimer);
      searchTimer = setTimeout(() => {
        state.query = value;
        renderTree();
        // Перерисуем и детали, чтобы обновить подсветку
        if (state.selectedId) renderDetail();
      }, 120);
    });
    searchClear.addEventListener('click', e => {
      // Клик всплывает до .search-wrap, чей обработчик тут же снова
      // развернул бы поиск — гасим всплытие
      e.stopPropagation();
      searchInput.value = '';
      searchClear.classList.remove('visible');
      state.query = '';
      renderTree();
      if (state.selectedId) renderDetail();
      if (isMobileSearch()) {
        // На телефоне «крестик» одновременно очищает и сворачивает поиск обратно в иконку
        searchWrap.classList.remove('is-mobile-open');
      } else {
        searchInput.focus();
      }
    });

    // На узких экранах поиск свёрнут в иконку — тап по ней разворачивает
    // строку поиска на всю ширину шапки, чтобы было куда печатать
    searchWrap.addEventListener('click', () => {
      if (isMobileSearch() && !searchWrap.classList.contains('is-mobile-open')) {
        searchWrap.classList.add('is-mobile-open');
        searchInput.focus();
      }
    });
    searchInput.addEventListener('blur', () => {
      if (isMobileSearch() && !searchInput.value) {
        searchWrap.classList.remove('is-mobile-open');
      }
    });

    // Переключение темы
    const themeSwitch = document.getElementById('themeToggle');
    themeSwitch.addEventListener('click', e => {
      const b = e.target.closest('[data-theme-set]');
      if (b) applyTheme(b.dataset.themeSet);
    });
    // Радиогруппа: стрелки переключают вариант и переносят фокус
    themeSwitch.addEventListener('keydown', e => {
      const d = e.key === 'ArrowRight' || e.key === 'ArrowDown' ? 1 : e.key === 'ArrowLeft' || e.key === 'ArrowUp' ? -1
  : 0;
      if (!d) return;
      e.preventDefault();
      const next = THEME_PREFS[(THEME_PREFS.indexOf(state.themePref) + d + THEME_PREFS.length) % THEME_PREFS.length];
      applyTheme(next);
      themeSwitch.querySelector('[data-theme-set="' + next + '"]').focus();
    });

    // Мобильное меню
    document.getElementById('menuToggle').addEventListener('click', openSidebar);
    document.getElementById('scrim').addEventListener('click', closeSidebar);

    // Связь с учётом: клики внутри карточки (разметка перерисовывается,
    // поэтому один делегированный обработчик на контейнер)
    document.getElementById('contentInner').addEventListener('click', e => {
      const unit = e.target.closest('[data-link-vehicle]');
      if (unit && TST.fleet) { TST.fleet.openVehicle(unit.dataset.linkVehicle); return; }
      const add = e.target.closest('[data-link-add]');
      if (add && TST.fleet) {
        const node = findById(CATALOG, add.dataset.linkAdd);
        TST.fleet.addVehicle({ catalogId: add.dataset.linkAdd, name: node ? node.name : '' });
        return;
      }
      if (!TST.fleet || !state.selectedId) return;
      const ce = e.target.closest('[data-cat-edit]');
      if (ce) { TST.fleet.editCatalogNode(ce.dataset.catEdit); return; }
      const cr = e.target.closest('[data-cat-reset]');
      if (cr) { TST.fleet.resetCatalogNode(cr.dataset.catReset); return; }
      const ch = e.target.closest('[data-cat-hide]');
      if (ch) { TST.fleet.hideCatalogNode(ch.dataset.catHide); return; }
```

```
      const cd = e.target.closest('[data-cat-del]');
      if (cd) { TST.fleet.deleteCatalogNode(cd.dataset.catDel); return; }
      const edit = e.target.closest('[data-norm-edit]');
      if (edit) { TST.fleet.editNorm(state.selectedId, edit.dataset.normEdit || ''); return; }
      const del = e.target.closest('[data-norm-del]');
      if (del) { TST.fleet.deleteNorm(state.selectedId, del.dataset.normDel); return; }
      const pr = e.target.closest('[data-print-card]');
      if (pr) TST.fleet.printChecklist(state.selectedId, pr.dataset.printCard);
    });

    // Нормативы пользователя и привязки живут в данных учёта — после их
    // изменения карточка справочника должна перерисоваться
    TST.on('fleet-changed', () => {
      if (TST.workspace !== 'catalog') return;
      // Правки справочника меняют дерево (новые / удалённые / переименованные позиции)
      if (applyUserCatalog()) {
        renderTree();
        if (state.selectedId && !findById(CATALOG, state.selectedId)) state.selectedId = null;
        renderDetail();
      } else if (state.selectedId) renderDetail();
    });

    // «+ Добавить» над деревом: новая позиция в раздел выбранной (или первой) ветви
    document.getElementById('catalogAdd')?.addEventListener('click', () => {
      if (!TST.fleet || !TST.fleet.addCatalogNode) return;
      const sel = state.selectedId ? findById(CATALOG, state.selectedId) : null;
      const crumbs = sel ? (TST.catalog.parentOf(sel.id) || '') : '';
      TST.fleet.addCatalogNode(crumbs);
    });

    document.getElementById('catalogManage')?.addEventListener('click', () => { if (TST.fleet &&
  TST.fleet.manageCatalog) TST.fleet.manageCatalog(); });

    // Кнопки переключения разделов в шапке — единый роутер TST
    document.querySelectorAll('[data-ws-btn]').forEach(b =>
      b.addEventListener('click', () => TST.setWorkspace(b.dataset.wsBtn)));

    // При входе в справочник обновляем счётчики техники в дереве и блок
    // «На учёте» — данные учёта могли измениться, пока раздел был скрыт
    TST.on('workspace', ({ ws }) => {
      if (ws === 'catalog') {
        renderTree();
        if (state.selectedId) renderDetail();
      } else {
        document.getElementById('catalogSearchWrap').classList.remove('is-mobile-open');
        closeSidebar();
      }
    });

    // Горячая клавиша «/» — фокус на поиск.
    // Работает только в разделе «Справочник» и не при открытом модальном окне:
    // раньше «/» перехватывался и в учёте (в textarea форм), а Escape из окна
    // модуля учёта заодно сбрасывал поиск справочника
    document.addEventListener('keydown', e => {
      if (!TST.isApp() || document.querySelector('.f-modal-backdrop')) return;
      const typing = e.target.closest && e.target.closest('input, textarea, select, [contenteditable="true"]');
      // «/» — общий поиск в шапке (работает во всех разделах)
      if (e.key === '/' && !typing) {
        e.preventDefault();
        if (isMobileSearch()) searchWrap.classList.add('is-mobile-open');
        searchInput.focus();
      }
      if (TST.workspace !== 'catalog') return;
      if (e.key === 'Escape') {
        if (state.query) {
          searchInput.value = '';
          searchClear.classList.remove('visible');
          state.query = '';
          renderTree();
          if (state.selectedId) renderDetail();
        }
        searchWrap.classList.remove('is-mobile-open');
        closeSidebar();
      }
    });
  }

  // Запуск приложения после загрузки DOM
  if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', init);
  } else {
    init();
  }
```

2. QR-код: собственный генератор (index.html, скрипт 2)

```
/* ========================================================================
   QR-КОД: собственный генератор (ISO/IEC 18004), без внешних библиотек
   ------------------------------------------------------------------------
   Байтовый режим (UTF-8), версии 1–40, уровни коррекции L/M/Q/H,
   автоматический выбор версии и маски. Результат — матрица модулей
   (true — тёмный), которую рисуют qrSvg() / qrCanvas().
   ======================================================================== */
const QR = (() => {
  'use strict';
  const ECL = { L: { ord: 0, fmt: 1 }, M: { ord: 1, fmt: 0 }, Q: { ord: 2, fmt: 3 }, H: { ord: 3, fmt: 2 } };
  const ECC_PER_BLOCK = [
    [-1, 7, 10, 15, 20, 26, 18, 20, 24, 30, 18, 20, 24, 26, 30, 22, 24, 28, 30, 28, 28, 28, 28, 30, 30, 26, 28, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30],
    [-1, 10, 16, 26, 18, 24, 16, 18, 22, 22, 26, 30, 22, 22, 24, 24, 28, 28, 26, 26, 26, 26, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28],
    [-1, 13, 22, 18, 26, 18, 24, 18, 22, 20, 24, 28, 26, 24, 20, 30, 24, 28, 28, 26, 30, 28, 30, 30, 30, 30, 28, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30],
    [-1, 17, 28, 22, 16, 22, 28, 26, 26, 24, 28, 24, 28, 22, 24, 24, 30, 28, 28, 26, 28, 30, 24, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30]
  ];
  const NUM_BLOCKS = [
    [-1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 4, 4, 4, 4, 4, 6, 6, 6, 6, 7, 8, 8, 9, 9, 10, 12, 12, 12, 13, 14, 15, 16, 17, 18, 19, 19, 20, 21, 22, 24, 25],
    [-1, 1, 1, 1, 2, 2, 4, 4, 4, 5, 5, 5, 8, 9, 9, 10, 10, 11, 13, 14, 16, 17, 17, 18, 20, 21, 23, 25, 26, 28, 29, 31, 33, 35, 37, 38, 40, 43, 45, 47, 49],
    [-1, 1, 1, 2, 2, 4, 4, 6, 6, 8, 8, 8, 10, 12, 16, 12, 17, 16, 18, 21, 20, 23, 23, 25, 27, 29, 34, 34, 35, 38, 40, 43, 45, 48, 51, 53, 56, 59, 62, 65, 68],
    [-1, 1, 1, 2, 4, 4, 4, 5, 6, 8, 8, 11, 11, 16, 16, 18, 16, 19, 21, 25, 25, 25, 34, 30, 32, 35, 37, 40, 42, 45, 48, 51, 54, 57, 60, 63, 66, 70, 74, 77, 81]
  ];
  const bit = (x, i) => ((x >>> i) & 1) !== 0;

  function rawDataModules(ver) {
    let r = (16 * ver + 128) * ver + 64;
    if (ver >= 2) {
      const na = Math.floor(ver / 7) + 2;
      r -= (25 * na - 10) * na - 55;
      if (ver >= 7) r -= 36;
    }
    return r;
  }
  const dataCodewords = (ver, e) => Math.floor(rawDataModules(ver) / 8) - ECC_PER_BLOCK[e.ord][ver] * NUM_BLOCKS[e.ord][ver];

  /* GF(256) с полиномом 0x11D */
  function gfMul(x, y) {
    let z = 0;
    for (let i = 7; i >= 0; i--) { z = (z << 1) ^ ((z >>> 7) * 0x11D); z ^= ((y >>> i) & 1) * x; }
    return z & 0xFF;
  }
  function rsDivisor(deg) {
    const r = new Array(deg).fill(0); r[deg - 1] = 1;
    let root = 1;
    for (let i = 0; i < deg; i++) {
      for (let j = 0; j < deg; j++) { r[j] = gfMul(r[j], root); if (j + 1 < deg) r[j] ^= r[j + 1]; }
      root = gfMul(root, 0x02);
    }
    return r;
  }
  function rsRemainder(data, div) {
    const r = div.map(() => 0);
    for (const b of data) {
      const f = b ^ r.shift(); r.push(0);
      div.forEach((c, i) => { r[i] ^= gfMul(c, f); });
    }
    return r;
  }
  function utf8(s) { return Array.from(new TextEncoder().encode(s)); }

  function encode(text, opts) {
    opts = opts || {};
    const bytes = utf8(String(text));
    let e = ECL[opts.ecl || 'M'] || ECL.M;
    const minV = opts.minVersion || 1, maxV = opts.maxVersion || 40;
    let ver = 0, need = 0;
    for (let v = minV; v <= maxV; v++) {
      const cc = v < 10 ? 8 : 16;
      need = 4 + cc + bytes.length * 8;
      if (bytes.length < (1 << cc) && need <= dataCodewords(v, e) * 8) { ver = v; break; }
    }
    if (!ver) throw new Error('QR: объём данных (' + bytes.length + ' байт) превышает допустимый');
    // Повышаем уровень коррекции, если он помещается в ту же версию
    if (opts.boost !== false) for (const k of ['Q', 'H']) if (ECL[k].ord > e.ord && need <= dataCodewords(ver, ECL[k]) * 8) e = ECL[k];

    // Битовый поток: режим 0100, длина, данные, терминатор, дополнение
    const bits = [];
    const put = (val, len) => { for (let i = len - 1; i >= 0; i--) bits.push((val >>> i) & 1); };
    put(4, 4); put(bytes.length, ver < 10 ? 8 : 16); bytes.forEach(b => put(b, 8));
    const cap = dataCodewords(ver, e) * 8;
```

```
        put(0, Math.min(4, cap - bits.length));
        put(0, (8 - bits.length % 8) % 8);
        for (let pad = 0xEC; bits.length < cap; pad ^= 0xEC ^ 0x11) put(pad, 8);
        const data = [];
        for (let i = 0; i < bits.length; i += 8) { let b = 0; for (let j = 0; j < 8; j++) b = (b << 1) | bits[i + j];
    data.push(b); }

        // Блоки Рида — Соломона и перемежение
        const nb = NUM_BLOCKS[e.ord][ver], el = ECC_PER_BLOCK[e.ord][ver];
        const raw = Math.floor(rawDataModules(ver) / 8);
        const nShort = nb - raw % nb, shortLen = Math.floor(raw / nb);
        const div = rsDivisor(el), blocks = [];
        for (let i = 0, k = 0; i < nb; i++) {
          const dat = data.slice(k, k + shortLen - el + (i < nShort ? 0 : 1));
          k += dat.length;
          const ecc = rsRemainder(dat, div);
          if (i < nShort) dat.push(0);
          blocks.push(dat.concat(ecc));
        }
        const cw = [];
        for (let i = 0; i < blocks[0].length; i++) blocks.forEach((b, j) => { if (i !== shortLen - el || j >= nShort)
    cw.push(b[i]); });

        // Матрица
        const size = ver * 4 + 17;
        const mod = Array.from({ length: size }, () => new Array(size).fill(false));
        const fn = Array.from({ length: size }, () => new Array(size).fill(false));
        const setF = (x, y, d) => { mod[y][x] = d; fn[y][x] = true; };
        for (let i = 0; i < size; i++) { setF(6, i, i % 2 === 0); setF(i, 6, i % 2 === 0); }
        const finder = (cx, cy) => {
          for (let dy = -4; dy <= 4; dy++) for (let dx = -4; dx <= 4; dx++) {
            const d = Math.max(Math.abs(dx), Math.abs(dy)), x = cx + dx, y = cy + dy;
            if (x >= 0 && x < size && y >= 0 && y < size) setF(x, y, d !== 2 && d !== 4);
          }
        };
        finder(3, 3); finder(size - 4, 3); finder(3, size - 4);
        const align = [];
        if (ver > 1) {
          const na = Math.floor(ver / 7) + 2;
          const step = ver === 32 ? 26 : Math.ceil((ver * 4 + 4) / (na * 2 - 2)) * 2;
          align.push(6);
          for (let p = size - 7; align.length < na; p -= step) align.splice(1, 0, p);
        }
        const na = align.length;
        for (let i = 0; i < na; i++) for (let j = 0; j < na; j++) {
          if ((i === 0 && j === 0) || (i === 0 && j === na - 1) || (i === na - 1 && j === 0)) continue;
          for (let dy = -2; dy <= 2; dy++) for (let dx = -2; dx <= 2; dx++) setF(align[i] + dx, align[j] + dy,
    Math.max(Math.abs(dx), Math.abs(dy)) !== 1);
        }
        const drawFormat = mask => {
          const d = (e.fmt << 3) | mask;
          let r = d;
          for (let i = 0; i < 10; i++) r = (r << 1) ^ ((r >>> 9) * 0x537);
          const b = ((d << 10) | r) ^ 0x5412;
          for (let i = 0; i <= 5; i++) setF(8, i, bit(b, i));
          setF(8, 7, bit(b, 6)); setF(8, 8, bit(b, 7)); setF(7, 8, bit(b, 8));
          for (let i = 9; i < 15; i++) setF(14 - i, 8, bit(b, i));
          for (let i = 0; i < 8; i++) setF(size - 1 - i, 8, bit(b, i));
          for (let i = 8; i < 15; i++) setF(8, size - 15 + i, bit(b, i));
          setF(8, size - 8, true);
        };
        drawFormat(0);
        if (ver >= 7) {
          let r = ver;
          for (let i = 0; i < 12; i++) r = (r << 1) ^ ((r >>> 11) * 0x1F25);
          const b = (ver << 12) | r;
          for (let i = 0; i < 18; i++) { const d = bit(b, i), a = size - 11 + i % 3, c = Math.floor(i / 3); setF(a, c, d);
     setF(c, a, d); }
        }
        // Данные змейкой
        let i = 0;
        for (let right = size - 1; right >= 1; right -= 2) {
          if (right === 6) right = 5;
          for (let v = 0; v < size; v++) for (let j = 0; j < 2; j++) {
            const x = right - j, up = ((right + 1) & 2) === 0, y = up ? size - 1 - v : v;
            if (!fn[y][x] && i < cw.length * 8) { mod[y][x] = bit(cw[i >>> 3], 7 - (i & 7)); i++; }
          }
        }
        const maskFn = [
          (x, y) => (x + y) % 2 === 0, (x, y) => y % 2 === 0, x => x % 3 === 0, (x, y) => (x + y) % 3 === 0,
          (x, y) => (Math.floor(x / 3) + Math.floor(y / 2)) % 2 === 0, (x, y) => x * y % 2 + x * y % 3 === 0,
          (x, y) => (x * y % 2 + x * y % 3) % 2 === 0, (x, y) => ((x + y) % 2 + x * y % 3) % 2 === 0
        ];
        const applyMask = m => { for (let y = 0; y < size; y++) for (let x = 0; x < size; x++) if (!fn[y][x] &&
    maskFn[m](x, y)) mod[y][x] = !mod[y][x]; };
        const penalty = () => {
          let p = 0;
          const lines = [];
          for (let y = 0; y < size; y++) lines.push(mod[y]);
          for (let x = 0; x < size; x++) lines.push(mod.map(r => r[x]));
```

2. QR-код: собственный генератор (index.html, скрипт 2) (продолжение)

```
        lines.forEach(l => {
          let run = 1;
          for (let k = 1; k <= size; k++) {
            if (k < size && l[k] === l[k - 1]) run++;
            else { if (run >= 5) p += run - 2; run = 1; }
          }
          const s = l.map(b => (b ? 1 : 0)).join('');
          for (const pat of ['10111010000', '00001011101']) { let at = s.indexOf(pat); while (at >= 0) { p += 40; at = s.indexOf(pat, at + 1); } }
        });
        for (let y = 0; y < size - 1; y++) for (let x = 0; x < size - 1; x++) {
          const c = mod[y][x];
          if (c === mod[y][x + 1] && c === mod[y + 1][x] && c === mod[y + 1][x + 1]) p += 3;
        }
        let dark = 0; mod.forEach(r => r.forEach(b => { if (b) dark++; }));
        p += Math.floor(Math.abs(dark * 20 - size * size * 10) / (size * size)) * 10;
        return p;
      };
      let mask = opts.mask, best = Infinity;
      if (mask == null) {
        for (let m = 0; m < 8; m++) {
          applyMask(m); drawFormat(m);
          const pe = penalty();
          if (pe < best) { best = pe; mask = m; }
          applyMask(m);
        }
      }
      applyMask(mask); drawFormat(mask);
      const eclName = Object.keys(ECL).find(k => ECL[k] === e);
      return { size, modules: mod, version: ver, ecl: eclName, mask, bytes: bytes.length };
    }

    /* SVG: чёрные модули на белом поле, с «тихой зоной» quiet модулей */
    function svg(qr, opts) {
      opts = opts || {};
      const q = opts.quiet == null ? 4 : opts.quiet, n = qr.size + q * 2;
      let d = '';
      for (let y = 0; y < qr.size; y++) {
        for (let x = 0; x < qr.size; x++) {
          if (!qr.modules[y][x]) continue;
          let w = 1;
          while (x + w < qr.size && qr.modules[y][x + w]) w++;
          d += 'M' + (x + q) + ' ' + (y + q) + 'h' + w + 'v1h-' + w + 'z';
          x += w - 1;
        }
      }
      const size = opts.size ? ' width="' + opts.size + '" height="' + opts.size + '"' : '';
      return '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 ' + n + ' ' + n + '"' + size +
        ' shape-rendering="crispEdges" role="img" aria-label="' + (opts.label || 'QR-код') + '">' +
        '<rect width="' + n + '" height="' + n + '" fill="#fff"/><path fill="#000" d="' + d + '"/></svg>';
    }
    /* Растровое изображение для скачивания PNG */
    function canvas(qr, scale, quiet) {
      const q = quiet == null ? 4 : quiet, n = qr.size + q * 2, s = Math.max(1, scale || 10);
      const c = document.createElement('canvas');
      c.width = c.height = n * s;
      const g = c.getContext('2d');
      g.fillStyle = '#fff'; g.fillRect(0, 0, c.width, c.height);
      g.fillStyle = '#000';
      for (let y = 0; y < qr.size; y++) for (let x = 0; x < qr.size; x++) if (qr.modules[y][x]) g.fillRect((x + q) * s, (y + q) * s, s, s);
      return c;
    }
    return { encode, svg, canvas, capacity: (ver, ecl) => dataCodewords(ver, ECL[ecl || 'M']) - (ver < 10 ? 2 : 3) };
  })();
```

3. Платформа: модули, целостность, события (index.html, скрипт 3)

```
/* ========================================================================
   ПЛАТФОРМА «РЕГЛАМЕНТ-ТС» — window.TST.platform
   ------------------------------------------------------------------------
   Ядро модульной архитектуры внутри одного офлайн-файла. Ничего не рисует
   и не хранит само — задаёт правила, по которым работают модули:

     • реестр модулей: идентификатор, версия, владение коллекциями данных,
       зависимости, самотесты (register / modules / runTests);
     • единые правила идентификации записей (ids);
     • общая модель данных: описание коллекций, обязательных полей и связей
       между записями (defineCollection / checkIntegrity);
     • единый механизм событий — шина TST.on / TST.emit (общая шина
       приложения); имена событий платформы перечислены в EVENTS.

   Правило совместимости: модуль обращается к чужим данным только через
   публичный API владельца (TST.fleet, TST.catalog …) или события шины,
   поэтому новый модуль не может сломать уже работающие. Проверка
   целостности и самотесты запускаются перед восстановлением из копии,
   после обновления структуры данных и вручную в разделе «Сохранность данных».
   ======================================================================== */
(function () {
  const T = window.TST;
  if (!T) return;

  const PLATFORM_VERSION = '2.0.0';

  /* --- Правила идентификации ---------------------------------------------
     Идентификатор записи: <префикс>_<время base36>_<случайная часть>.
     Префикс определяет вид записи. Идентификатор присваивается один раз
     при создании и больше не меняется (на нём держатся связи, история
     изменений и QR-код техники). Допустимые символы — латиница, цифры,
     «_» и «-», длина 4–64. */
  const ID_RE = /^[A-Za-z0-9_-]{4,64}$/;
  const ID_PREFIX = {
    vehicles: 'v', maint: 'm', hours: 'h', trips: 't', norms: 'n', catalogEdits: 'c',
    trash: 'r', history: 'e', snapshot: 's'
  };
  const ids = {
    RE: ID_RE,
    PREFIX: ID_PREFIX,
    valid: id => typeof id === 'string' && ID_RE.test(id),
    make(prefix, taken) {
      let id;
      do {
        id = (prefix || 'id') + '_' + Date.now().toString(36) + '_' + Math.random().toString(36).slice(2, 9);
      } while (!ID_RE.test(id) || (taken && taken(id)));
      return id;
    }
  };

  /* --- События платформы (передаются через общую шину TST) --- */
  const EVENTS = {
    DATA_CHANGED: 'fleet-changed',          // данные учёта сохранены
    MODULE: 'platform:module',              // модуль зарегистрирован
    INTEGRITY: 'platform:integrity',        // выполнена проверка целостности
    SNAPSHOT: 'platform:snapshot',          // создана точка восстановления
    MIGRATED: 'platform:migrated',          // структура данных обновлена
    TESTS: 'platform:tests'                 // выполнены самотесты
  };

  /* --- Реестр модулей --- */
  const modules = new Map();
  function register(def) {
    if (!def || !def.id || !def.name) throw new Error('platform.register: нужны id и name');
    if (modules.has(def.id)) throw new Error('platform.register: модуль уже зарегистрирован — ' + def.id);
    const m = Object.assign({ version: '1.0.0', state: 'работает', owns: [], depends: [], tests: [], desc: '' }, def);
    (m.depends || []).forEach(d => {
      if (!modules.has(d)) console.warn('[Платформа] модуль ' + m.id + ' зависит от незарегистрированного ' + d);
    });
    modules.set(m.id, m);
    T.emit(EVENTS.MODULE, { id: m.id });
    return m;
  }

  /* --- Модель данных: коллекции, обязательные поля, связи ----------------
     refs: { поле: 'коллекция' } — значение поля должно быть id записи этой
     коллекции (пустое значение допустимо, если поле не обязательно).
     check(rec, ctx, issue) — дополнительные правила владельца коллекции. */
  const collections = new Map();
  function defineCollection(name, spec) {
    collections.set(name, Object.assign({ name, required: [], refs: {}, unique: [] }, spec));
  }

  /* Проверка целостности базы. Ничего не меняет — только находит.
     Уровни: error — нарушена связь или идентификатор (данные могут
     работать неверно); warn — сомнительное значение. Каждой проблеме
     может соответствовать fix — код автоматического исправления */
  function checkIntegrity(db) {
    const issues = [];
    const issue = (level, coll, id, text, fix) => issues.push({ level, coll, id: id || '', text, fix: fix || null });
```

```
    const index = {};
    collections.forEach((spec, name) => {
      const arr = db && db[name];
      if (arr == null) return;
      if (!Array.isArray(arr)) { issue('error', name, '', 'Коллекция повреждена (не список записей)', 'reset-collection'); return; }
      const seen = new Map();
      arr.forEach((r, i) => {
        if (!r || typeof r !== 'object') { issue('error', name, '', `Запись №${i + 1} повреждена`, 'drop-broken'); return; }
        if (!ids.valid(r.id)) issue('error', name, String(r.id || ''), 'Некорректный идентификатор записи', 'new-id');
        else if (seen.has(r.id)) issue('error', name, r.id, 'Идентификатор повторяется', 'new-id');
        else seen.set(r.id, r);
      });
      index[name] = seen;
    });
    collections.forEach((spec, name) => {
      const arr = Array.isArray(db && db[name]) ? db[name] : [];
      const uniq = {};
      arr.forEach(r => {
        if (!r || typeof r !== 'object') return;
        spec.required.forEach(f => {
          if (r[f] == null || r[f] === '') issue('error', name, r.id, `Не заполнено обязательное поле «${spec.labels && spec.labels[f] || f}»`, spec.fixRequired && spec.fixRequired[f]);
        });
        Object.keys(spec.refs).forEach(f => {
          const target = spec.refs[f];
          const val = r[f];
          if (val == null || val === '') return;
          if (!index[target] || !index[target].has(val)) issue('error', name, r.id, `Ссылка «${spec.labels && spec.labels[f] || f}» указывает на несуществующую запись (${target})`, 'drop-orphan');
        });
        spec.unique.forEach(f => {
          const val = r[f];
          if (val == null || String(val).trim() === '') return;
          const k = String(val).trim().toLowerCase();
          uniq[f] = uniq[f] || new Map();
          if (uniq[f].has(k)) issue('warn', name, r.id, `Значение «${spec.labels && spec.labels[f] || f}» = «${val}» повторяется`);
          else uniq[f].set(k, r.id);
        });
        if (spec.check) spec.check(r, { db, index }, (level, text, fix) => issue(level, name, r.id, text, fix));
      });
    });
    const report = {
      at: new Date().toISOString(),
      errors: issues.filter(x => x.level === 'error'),
      warnings: issues.filter(x => x.level === 'warn'),
      fixable: issues.filter(x => x.fix).length,
      counts: {}
    };
    collections.forEach((spec, name) => { report.counts[name] = Array.isArray(db && db[name]) ? db[name].length : 0; });
    T.emit(EVENTS.INTEGRITY, { errors: report.errors.length, warnings: report.warnings.length });
    return report;
  }

  /* Автоматическое исправление того, что исправляется однозначно:
     повреждённые записи и записи-«сироты» удаляются, повторяющиеся и
     некорректные идентификаторы заменяются новыми (кроме техники, на
     которую могут ссылаться записи — там новый id получает только копия).
     Возвращает копию базы и перечень исправлений; исходная не меняется */
  function repair(db) {
    const out = JSON.parse(JSON.stringify(db));
    const fixes = [];
    for (let pass = 0; pass < 3; pass++) {           // удаление сирот может породить новых (часы → ТО)
      const rep = checkIntegrity(out);
      const todo = rep.errors.filter(x => x.fix);
      if (!todo.length) break;
      collections.forEach((spec, name) => {
        if (!Array.isArray(out[name])) {
          if (todo.some(x => x.coll === name && x.fix === 'reset-collection')) { out[name] = []; fixes.push(`${name}: коллекция пересоздана пустой`); }
          return;
        }
        const seen = new Set();
        const prefix = ID_PREFIX[name] || 'id';
        out[name] = out[name].filter(r => {
          if (!r || typeof r !== 'object') { fixes.push(`${name}: удалена повреждённая запись`); return false; }
          if (!ids.valid(r.id) || seen.has(r.id)) {
            const old = r.id;
            r.id = ids.make(prefix, x => seen.has(x));
            fixes.push(`${name}: идентификатор ${old || '(пусто)'} заменён на ${r.id}`);
          }
          seen.add(r.id);
          return true;
        });
      });
      const rep2 = checkIntegrity(out);
      const orphans = new Set(rep2.errors.filter(x => x.fix === 'drop-orphan').map(x => x.coll + '|' + x.id));
```

```
      const reqFix = rep2.errors.filter(x => x.fix && x.fix.startsWith('set:'));
      collections.forEach((spec, name) => {
        if (!Array.isArray(out[name])) return;
        out[name] = out[name].filter(r => {
          if (orphans.has(name + '|' + r.id)) { fixes.push(`${name}: удалена запись ${r.id} без связанной записи`); return false; }
          return true;
        });
        reqFix.filter(x => x.coll === name).forEach(x => {
          const r = out[name].find(z => z.id === x.id);
          const [, field, value] = x.fix.split(':');
          if (r && (r[field] == null || r[field] === '')) { r[field] = value; fixes.push(`${name}: в записи ${r.id} заполнено поле ${field} = «${value}»`); }
        });
      });
    }
    return { db: out, fixes };
  }

  /* --- Самотесты модулей ---------------------------------------------------
     Тест — функция без побочных эффектов: работает на копиях данных, сама
     ничего не сохраняет. Возвращает true / строку-пояснение либо бросает
     исключение. Результат каждого теста изолирован от остальных */
  async function runTests(only) {
    const results = [];
    for (const m of modules.values()) {
      if (only && !only.includes(m.id)) continue;
      for (const t of m.tests || []) {
        const started = performance.now();
        let ok = false, info = '';
        try {
          const r = await t.run();
          ok = r === true || (typeof r === 'string');
          info = typeof r === 'string' ? r : (r === true ? '' : 'тест вернул ' + JSON.stringify(r));
        } catch (e) { ok = false; info = e && e.message ? e.message : String(e); }
        results.push({ module: m.id, moduleName: m.name, version: m.version, name: t.name, ok, info, ms: Math.round(performance.now() - started) });
      }
    }
    T.emit(EVENTS.TESTS, { total: results.length, failed: results.filter(r => !r.ok).length });
    return results;
  }
  const assert = (cond, msg) => { if (!cond) throw new Error(msg || 'условие не выполнено'); };

  T.platform = {
    version: PLATFORM_VERSION,
    ids, EVENTS,
    register,
    modules: () => Array.from(modules.values()),
    module: id => modules.get(id) || null,
    defineCollection,
    collections: () => Array.from(collections.values()),
    checkIntegrity, repair,
    runTests, assert
  };

  /* Модуль ядра и его самотесты */
  let lastSelftestEvent = null;
  T.on('platform:selftest', d => { lastSelftestEvent = d; });
  register({
    id: 'platform', name: 'Ядро платформы', version: PLATFORM_VERSION,
    desc: 'Реестр модулей, правила идентификации, модель данных, проверка целостности, шина событий',
    tests: [
      { name: 'Идентификаторы уникальны и соответствуют правилу', run() {
        const set = new Set();
        for (let i = 0; i < 2000; i++) { const id = ids.make('v', x => set.has(x)); assert(ids.valid(id), 'некорректный id ' + id); assert(!set.has(id), 'повтор ' + id); set.add(id); }
        assert(!ids.valid('v_<script>') && !ids.valid('') && !ids.valid('a b'), 'правило пропускает недопустимые символы');
        return '2000 идентификаторов без повторов';
      } },
      { name: 'Шина событий доставляет сообщения', run() {
        const x = Math.random();
        T.emit('platform:selftest', { x });
        assert(lastSelftestEvent && lastSelftestEvent.x === x, 'событие не доставлено');
        return true;
      } },
      { name: 'Проверка целостности находит нарушенные связи', run() {
        const db = { vehicles: [{ id: 'v_test_0001', name: 'T' }], maint: [{ id: 'm_test_0001', vehicleId: 'v_nope_0001', kind: 'ТО-1', date: '2025-01-01' }, { id: 'm_test_0001', vehicleId: 'v_test_0001', kind: 'ТО-1', date: '2025-01-01' }] };
        const r = checkIntegrity(db);
        assert(r.errors.some(x => /несуществующую/.test(x.text)), 'сирота не найдена');
        assert(r.errors.some(x => /повторяется/.test(x.text)), 'повтор id не найден');
        const fixed = repair(db);
        assert(checkIntegrity(fixed.db).errors.length === 0, 'после исправления остались ошибки');
        assert(db.maint.length === 2, 'исправление изменило исходную базу');
        return 'исправлений: ' + fixed.fixes.length;
      } }
    ]
```

```
      });
    })();
```

4. Модуль учёта: парк, ТО, моточасы, сохранность, пункт управления, сезоны, фото и документы,

```
/* ============================================================================
   ═══════════════════════════════════════════════════════════════════════
   МОДУЛЬ «РЕГИСТРАЦИЯ ТЕХНИЧЕСКИХ СРЕДСТВ ТЫЛА»
   ═══════════════════════════════════════════════════════════════════════
   Автономный модуль учёта техники, технического обслуживания и моточасов.

   Архитектура:
     • Данные      — localStorage (записи) + IndexedDB (файлы вложений)
     • Вычисления  — чистые функции computeStatus / computeNotifications
     • Отрисовка   — функции render*, единая точка входа renderView()
     • Экспорт     — собственный генератор XLSX (без библиотек) и печать в PDF

   Модуль полностью обёрнут в IIFE: глобальная область не засоряется,
   существующие переменные справочника (state, ICONS, …) не затрагиваются.
   ============================================================================ */
(function () {
'use strict';

/* ============================================================================
   1. КОНСТАНТЫ ПРЕДМЕТНОЙ ОБЛАСТИ
   ============================================================================ */

const LS_KEY  = 'spravochnik-tst-fleet-v1';
const LS_PERSIST_ASKED = 'spravochnik-tst-persist-asked';
const IDB_DB  = 'tst-fleet-files';
const IDB_ST  = 'files';
const SCHEMA_VERSION = 2;   // история версий и миграции — MIGRATIONS (раздел 27с)

/* Статусы единицы техники */
const STATUSES = [
  'В эксплуатации',
  'Ремонт',
  'Неисправна',
  'На хранении',
  'Консервация',
  'Списано'
];

/* Виды технического обслуживания */
const MAINT_KINDS = [
  'ЕО (ежедневное обслуживание)',
  'ТО-1',
  'ТО-2',
  'ТО-3',
  'Сезонное обслуживание (СО)',
  'Текущий ремонт',
  'Капитальный ремонт',
  'НЗ (расконсервация)'
];

/* Виды ТО, которые «обнуляют» ресурс до следующего планового ТО.
   ЕО проводится ежедневно и не сбрасывает межсервисный интервал. */
const RESET_KINDS = new Set([
  'ТО-1', 'ТО-2', 'ТО-3', 'Сезонное обслуживание (СО)', 'Капитальный ремонт'
]);
/* Виды, которые начинают отсчёт до следующего ТО-2 (ТО-1 и СО — нет) */
const TO2_RESET_KINDS = new Set(['ТО-2', 'ТО-3', 'Капитальный ремонт']);

/* Соответствие видов ТО журнала учёта и карт обслуживания справочника
   (ключи MAINT_TYPES). ЕО и ремонты в справочнике регламентом не описаны. */
const KIND_TO_CATALOG = {
  'ЕО (ежедневное обслуживание)': 'eo',
  'ТО-1': 'to1', 'ТО-2': 'to2', 'ТО-3': 'to3',
  'Сезонное обслуживание (СО)': 'so', 'НЗ (расконсервация)': 'nz'
};
const CATALOG_TO_KIND = Object.keys(KIND_TO_CATALOG)
  .reduce((o, k) => (o[KIND_TO_CATALOG[k]] = k, o), {});

/* API справочника (window.TST.catalog). Модуль учёта работает и без него —
   тогда просто не показывает блоки связи */
const CAT = () => (window.TST && window.TST.catalog) || null;
const catNodeOf = v => { const c = CAT(); return (c && v && v.catalogId) ? c.get(v.catalogId) : null; };
/* Регламент ТО для техники: из пособия или норматив пользователя */
const catMaintOf = v => { const c = CAT(); return (c && v && v.catalogId) ? c.maintenanceOf(v.catalogId) : null; };

/* Группы эксплуатации техники — по методике планирования эксплуатации и
   ремонта ВВСТ (ВАМТО, 2025): от группы зависит норматив КТГ и годовые
   нормы расхода ресурса. */
const EXP_GROUPS = ['Боевая', 'Строевая', 'Учебно-боевая', 'Учебно-строевая'];

/* Виды работ, для которых в методике приводятся нормативы трудоёмкости
   (используются в расчётах потребности в ТО, фонда времени и экономики). */
const LABOR_KINDS = ['ТО-1', 'ТО-2', 'ТР'];

/* Настройки по умолчанию */
const DEFAULT_SETTINGS = {
  defaultHours:   250,   // межсервисный интервал по моточасам
  defaultDays:    180,   // межсервисный интервал по календарю
  warnPct:        30,    // □ порог «осталось менее 30%»
  urgentPct:      10,    // □ порог «обслужить в ближайшее время»
```

```
    hoursStaleDays: 14,    // напоминание об обновлении моточасов
    notifyOnStart:  true,  // всплывающая сводка при открытии раздела
    backupRemindDays: 14,  // напоминать о резервной копии, если последней больше N дней
    trashDays:      30,    // хранить удалённое в корзине, дней
    seasons: { summer: '04-15', winter: '10-15', leadDays: 30 },   // сезонное обслуживание: даты перевода и срок напоминания

    /* --- Нормативные параметры планирования (методика ВАМТО, 2025) --- */
    ktgByGroup: {           // норматив КТГ по группам эксплуатации
      'Боевая': 0.95, 'Строевая': 0.90, 'Учебно-боевая': 0.85, 'Учебно-строевая': 0.85
    },
    periodTO1: 2000,        // периодичность ТО-1 по умолчанию, мото·ч (км)
    periodTO2: 12000,       // периодичность ТО-2 по умолчанию, мото·ч (км)
    periodTR:  1000,        // периодичность текущего ремонта, мото·ч (км) — формула (7)
    laborNorms: { 'ТО-1': 5, 'ТО-2': 27, 'ТР': 3 },  // нормативная трудоёмкость, чел.-ч
    normHourCost: 350,      // стоимость нормочаса Снч, руб. — формула Эф = Трт·Снч
    repairUnit: {           // ремонтное подразделение — расчёт фонда рабочего времени
      specialists: 4,       // количество производственников, чел.
      workDaysMonth: 20,    // рабочих дней в месяце, отводимых на ТО и ремонт (методика, 3.12)
      hoursPerDay: 7,       // продолжительность рабочего дня, ч
      nonProdPct: 3         // вынужденные потери, % от годового фонда (методика, 3.9)
    }
  };

  /* Уровни состояния ТО — единая таблица соответствий */
  const LEVELS = {
    ok:      { label: 'Норма',            emoji: '□', order: 4 },
    soon:    { label: 'Скоро ТО',         emoji: '□', order: 3 },
    urgent:  { label: 'Требуется ТО',     emoji: '□', order: 2 },
    overdue: { label: 'ТО просрочено',    emoji: '□', order: 1 },
    none:    { label: 'Интервал не задан', emoji: '□', order: 5 },
    retired: { label: 'Снято с учёта',    emoji: '□', order: 6 }
  };

  /* ========================================================================
     2. БАЗОВЫЕ ХЕЛПЕРЫ
     ======================================================================== */

  /* Экранирование HTML (собственная копия — не зависим от справочника) */
  function esc(s) {
    return String(s == null ? '' : s)
      .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;')
      .replace(/"/g, '&quot;').replace(/'/g, '&#39;');
  }

  /* Генератор идентификаторов */
  function uid(prefix) {
    return (prefix || 'id') + '_' +
      Date.now().toString(36) + '_' +
      Math.random().toString(36).slice(2, 9);
  }

  /* --- Работа с датами (везде формат ISO YYYY-MM-DD, локальная полночь) --- */
  const MS_DAY = 86400000;

  function todayISO() {
    const d = new Date();
    return isoOf(d);
  }
  function isoOf(d) {
    const y = d.getFullYear();
    const m = String(d.getMonth() + 1).padStart(2, '0');
    const dd = String(d.getDate()).padStart(2, '0');
    return `${y}-${m}-${dd}`;
  }
  function toDate(iso) {
    if (!iso) return null;
    const [y, m, d] = String(iso).split('-').map(Number);
    if (!y || !m || !d) return null;
    return new Date(y, m - 1, d);
  }
  /* Календарных дней от a до b (b - a). Отрицательное — b в прошлом. */
  function daysBetween(aIso, bIso) {
    const a = toDate(aIso), b = toDate(bIso);
    if (!a || !b) return null;
    return Math.round((b - a) / MS_DAY);
  }
  function addDays(iso, n) {
    const d = toDate(iso);
    if (!d) return null;
    d.setDate(d.getDate() + n);
    return isoOf(d);
  }
  /* dd.mm.yyyy — привычный вид для служебных документов */
  function fmtDate(iso) {
    const d = toDate(iso);
    if (!d) return '—';
    return String(d.getDate()).padStart(2, '0') + '.' +
           String(d.getMonth() + 1).padStart(2, '0') + '.' + d.getFullYear();
  }
```

```
180
181     /* --- Числа --- */
182     function fmtNum(n, digits) {
183       if (n == null || !isFinite(n)) return '—';
184       return Number(n).toLocaleString('ru-RU', {
185         minimumFractionDigits: digits || 0,
186         maximumFractionDigits: digits == null ? 1 : digits
187       });
188     }
189     function fmtMoney(n) {
190       if (!n && n !== 0) return '—';
191       return Number(n).toLocaleString('ru-RU', { maximumFractionDigits: 2 }) + ' \u20BD';
192     }
193     function fmtSize(bytes) {
194       if (bytes == null) return '';
195       if (bytes < 1024) return bytes + ' Б';
196       if (bytes < 1048576) return (bytes / 1024).toFixed(1) + ' КБ';
197       return (bytes / 1048576).toFixed(1) + ' МБ';
198     }
199     function clamp(v, lo, hi) { return Math.max(lo, Math.min(hi, v)); }
200
201     /* Разумный верхний предел показаний счётчика: 10 млн мото·ч (≈1100 лет
202        непрерывной работы). Защищает расчёты и вёрстку таблиц от значений вида
203        1e21, которые браузер принимает в поле type="number". */
204     const MAX_HOURS = 10000000;
205     const validHours = n => isFinite(n) && n >= 0 && n <= MAX_HOURS;
206
207     /* Склонение существительных: plural(5,'день','дня','дней') → 'дней' */
208     function plural(n, one, few, many) {
209       const a = Math.abs(n) % 100, b = a % 10;
210       if (a > 10 && a < 20) return many;
211       if (b > 1 && b < 5) return few;
212       if (b === 1) return one;
213       return many;
214     }
215
216     /* Скачивание Blob как файла */
217     function downloadBlob(blob, filename) {
218       const url = URL.createObjectURL(blob);
219       const a = document.createElement('a');
220       a.href = url;
221       a.download = filename;
222       document.body.appendChild(a);
223       a.click();
224       document.body.removeChild(a);
225       setTimeout(() => URL.revokeObjectURL(url), 4000);
226     }
227
228     /* ========================================================================
229        3. ХРАНИЛИЩЕ: localStorage (записи) + IndexedDB (файлы)
230        ======================================================================== */
231
232     let DB = null;
233     let storageOK = true;
234
235     /* Коллекции записей: у каждой записи есть уникальный id. По ним работают
236        нормализация, слияние изменений из других вкладок и отмена удаления */
237     const COLLECTIONS = ['vehicles', 'maint', 'hours', 'trips', 'norms', 'catalogEdits'];
238     /* Плюс служебные коллекции подсистемы сохранности: тоже записи с id,
239        сливаются между вкладками и попадают в резервную копию, но удаление
240        с отменой их не касается */
241     const ALL_COLLECTIONS = COLLECTIONS.concat(['trash', 'history']);
242
243     /* Синхронизация вкладок.
244        lastRaw  — строка, которую эта вкладка последней прочитала/записала;
245        baseJSON — снимок базы на тот же момент (общий предок для слияния).
246        Если перед записью в хранилище лежит не lastRaw — значит, другая вкладка
247        успела сохранить свои изменения, и вместо перезаписи выполняется слияние */
248     let lastRaw = null;
249     let baseJSON = null;
250     let pendingExternal = false;
251     let lastSyncToast = 0;
252
253     function blankDB() {
254       return {
255         version: SCHEMA_VERSION,
256         vehicles: [],
257         maint: [],
258         hours: [],
259         trips: [],
260         norms: [],            // пользовательские нормативы ТО для позиций справочника без регламента
261         catalogEdits: [],     // правки и добавленные позиции справочника (см. applyUserCatalog)
262         trash: [],            // корзина: удалённые записи со связанными (раздел 27c)
263         history: [],          // журнал изменений важных полей карточек
264         migrations: [],       // выполненные обновления структуры данных
265         settings: Object.assign({}, DEFAULT_SETTINGS),
266         seeded: false,
267         lastBackupAt: ''      // ISO-время последней резервной копии
268       };
269     }
```

```
270
271     /* Приведение произвольного JSON (хранилище, резервная копия, другая вкладка)
272        к корректной структуре базы: недостающие настройки — из значений по
273        умолчанию, повреждённые коллекции — пустые массивы */
274     function normalizeDB(parsed) {
275       const d = Object.assign(blankDB(), parsed || {});
276       const ps = (parsed && parsed.settings) || {};
277       d.settings = Object.assign({}, DEFAULT_SETTINGS, ps);
278       d.settings.ktgByGroup = Object.assign({}, DEFAULT_SETTINGS.ktgByGroup, ps.ktgByGroup || {});
279       d.settings.laborNorms = Object.assign({}, DEFAULT_SETTINGS.laborNorms, ps.laborNorms || {});
280       d.settings.repairUnit = Object.assign({}, DEFAULT_SETTINGS.repairUnit, ps.repairUnit || {});
281       d.settings.seasons = Object.assign({}, DEFAULT_SETTINGS.seasons, ps.seasons || {});
282       ALL_COLLECTIONS.forEach(k => { if (!Array.isArray(d[k])) d[k] = []; });
283       if (!Array.isArray(d.migrations)) d.migrations = [];
284       return d;
285     }
286
287     function loadDB() {
288       try {
289         const raw = NATIVE() ? NATIVE().initialDb : localStorage.getItem(LS_KEY);   // в приложении основной источник —
    файл db.json
290         const parsedRaw = raw ? JSON.parse(raw) : null;
291         // Старая структура переводится на текущую (перед этим — точка восстановления)
292         DB = parsedRaw ? migrateDB(normalizeDB(parsedRaw), parsedRaw.version, raw) : blankDB();
293         lastRaw = raw;
294         migrateCatalogLinks();
295       } catch (e) {
296         console.warn('[ТСТ] Не удалось прочитать данные, создаётся пустая база', e);
297         DB = blankDB();
298       }
299       baseJSON = JSON.stringify(DB);
300     }
301
302     /* Трёхстороннее слияние по записям: для каждой записи (по id) берётся наша
303        версия, если МЫ её меняли относительно общего предка, иначе — версия
304        другой вкладки. Удаление — тоже изменение. Производные поля (приросты и
305        текущие моточасы) в сравнении не участвуют и пересчитываются после */
306     const DERIVED_FIELDS = { vehicles: ['currentHours', 'hoursUpdatedAt'], hours: ['delta'] };
307     function recordKey(k, r) {
308       if (!r) return '';
309       const drop = DERIVED_FIELDS[k];
310       if (!drop) return JSON.stringify(r);
311       const c = Object.assign({}, r);
312       drop.forEach(f => { delete c[f]; });
313       return JSON.stringify(c);
314     }
315     function mergeDB(base, ours, theirs) {
316       const out = normalizeDB(theirs);
317       const index = arr => new Map((arr || []).map(r => [r && r.id, r]));
318       ALL_COLLECTIONS.forEach(k => {
319         const b = index(base[k]), o = index(ours[k]), t = index(theirs[k]);
320         const ids = new Set([...t.keys(), ...o.keys(), ...b.keys()]);
321         out[k] = [];
322         ids.forEach(id => {
323           const oursChanged = recordKey(k, o.get(id)) !== recordKey(k, b.get(id));
324           const pick = oursChanged ? o.get(id) : t.get(id);
325           if (pick) out[k].push(pick);
326         });
327       });
328       ['settings', 'seeded', 'lastBackupAt', 'migrations', 'version'].forEach(f => {
329         if (JSON.stringify(ours[f]) !== JSON.stringify(base[f])) out[f] = ours[f];
330       });
331       out.vehicles.forEach(v => recomputeHours(v.id, out));
332       return out;
333     }
334
335     /* Применить изменения, сохранённые другой вкладкой */
336     function applyExternal() {
337       pendingExternal = false;
338       let stored;
339       try { stored = localStorage.getItem(LS_KEY); } catch (e) { return; }
340       if (stored === lastRaw) return;
341       try {
342         const theirs = stored ? normalizeDB(JSON.parse(stored)) : blankDB();
343         DB = mergeDB(JSON.parse(baseJSON || 'null') || blankDB(), DB, theirs);
344         lastRaw = stored;
345         baseJSON = stored || JSON.stringify(blankDB());
346       } catch (e) {
347         console.warn('[ТСТ] Не удалось применить изменения из другой вкладки', e);
348         return;
349       }
350       ui.park.selected.forEach(id => { if (!getVehicle(id)) ui.park.selected.delete(id); });
351       if (window.TST) window.TST.emit('fleet-changed');
352       if (window.TST && window.TST.workspace === 'fleet') rerender(); else renderNotifPanel();
353       if (Date.now() - lastSyncToast > 10000) {
354         lastSyncToast = Date.now();
355         toast('info', 'Данные обновлены', 'Загружены изменения, сделанные в другой вкладке программы.');
356       }
357     }
358
```

```
  /* Карточки, созданные до появления связи со справочником, получают
     catalogId автоматически — по обозначению в наименовании/марке.
     Признак миграции — наличие поля: пустая строка = «проверено, не найдено»,
     и пользовательский выбор «не привязано» не перезаписывается */
  function migrateCatalogLinks() {
    const c = CAT();
    DB.vehicles.forEach(v => {
      if (v && !('catalogId' in v)) v.catalogId = c ? c.match(`${v.name || ''} ${v.brand || ''}`) : '';
    });
  }

  /* Сохранение. opts.replace — полная замена базы (восстановление из копии,
     очистка): слияние с чужими изменениями в этом случае не нужно */
  function saveDB(opts) {
    if (testDepth) return;          // самотест работает на временной базе — в хранилище не пишем
    if (!(opts && opts.replace)) recordHistory();
    let merged = false;
    try {
      if (!(opts && opts.replace) && !NATIVE()) {      // слияние с другой вкладкой нужно только в браузере
        const stored = localStorage.getItem(LS_KEY);
        if (stored && stored !== lastRaw) {
          DB = mergeDB(JSON.parse(baseJSON || 'null') || blankDB(), DB, normalizeDB(JSON.parse(stored)));
          merged = true;
        }
      }
      const raw = JSON.stringify(DB);
      if (NATIVE()) nativeSaveDb(raw); else localStorage.setItem(LS_KEY, raw);
      lastRaw = raw;
      baseJSON = raw;
      pendingExternal = false;
      storageOK = true;
    } catch (e) {
      if (storageOK) {
        storageOK = false;
        toast('danger', 'Данные не сохранены',
          'Браузер заблокировал локальное хранилище (приватный режим или переполнение). Изменения будут потеряны при закрытии вкладки.');
      }
    }
    if (window.TST) window.TST.emit('fleet-changed');
    if (merged) toast('info', 'Изменения объединены', 'В другой вкладке тоже вносились правки — сохранены и они, и ваши.');
  }

  /* --- IndexedDB для вложений (PDF, Word, изображения) --- */
  let idbPromise = null;
  const memFiles = new Map();   // аварийный резерв, если IndexedDB недоступна
  let idbAvailable = true;

  function idb() {
    if (idbPromise) return idbPromise;
    idbPromise = new Promise((resolve, reject) => {
      if (!('indexedDB' in window)) { reject(new Error('no idb')); return; }
      const req = indexedDB.open(IDB_DB, 1);
      req.onupgradeneeded = () => {
        const d = req.result;
        if (!d.objectStoreNames.contains(IDB_ST)) d.createObjectStore(IDB_ST, { keyPath: 'id' });
      };
      req.onsuccess = () => resolve(req.result);
      req.onerror = () => reject(req.error);
    }).catch(err => { idbAvailable = false; throw err; });
    return idbPromise;
  }

  function idbRun(mode, fn) {
    return idb().then(d => new Promise((resolve, reject) => {
      const tx = d.transaction(IDB_ST, mode);
      const store = tx.objectStore(IDB_ST);
      const req = fn(store);
      tx.oncomplete = () => resolve(req && req.result);
      tx.onerror = () => reject(tx.error);
    }));
  }

  function filePut(rec) {
    nativeFilePut(rec);
    return idbRun('readwrite', st => st.put(rec))
      .catch(() => { memFiles.set(rec.id, rec); });
  }
  function fileGet(id) {
    return idbRun('readonly', st => st.get(id))
      .then(r => r || memFiles.get(id) || nativeFileGet(id) || null)
      .catch(() => memFiles.get(id) || nativeFileGet(id) || null);
  }
  function fileDel(id) {
    nativeFileDel(id);
    return idbRun('readwrite', st => st.delete(id))
      .catch(() => { memFiles.delete(id); });
  }

```

```
  /* ========================================================================
     4. ДОСТУП К ДАННЫМ
     ======================================================================== */

  const getVehicle = id => DB.vehicles.find(v => v.id === id) || null;

  /* Постоянный идентификатор новой единицы техники (он же — содержимое
     QR-кода). Совпадение с существующей записью исключается проверкой */
  function uniqueVehicleId() {
    let id;
    do { id = uid('v'); } while (DB.vehicles.some(v => v.id === id) || !/^[A-Za-z0-9_-]{4,64}$/.test(id));
    return id;
  }

  /* Записи ТО по единице техники, от новых к старым */
  function maintOf(vehicleId) {
    return DB.maint
      .filter(m => m.vehicleId === vehicleId)
      .sort((a, b) => (b.date || '').localeCompare(a.date || '') ||
                      (b.createdAt || '').localeCompare(a.createdAt || ''));
  }
  /* История моточасов, от новых к старым */
  function hoursOf(vehicleId) {
    return DB.hours
      .filter(h => h.vehicleId === vehicleId)
      .sort((a, b) => (b.date || '').localeCompare(a.date || '') ||
                      (b.createdAt || '').localeCompare(a.createdAt || ''));
  }

  /* Путевые листы по единице техники, от новых к старым */
  function tripsOf(vehicleId) {
    return DB.trips
      .filter(t => t.vehicleId === vehicleId)
      .sort((a, b) => (b.date || '').localeCompare(a.date || '') ||
                      (b.createdAt || '').localeCompare(a.createdAt || ''));
  }

  /* Годовая наработка техники: сумма приростов показаний счётчика за календарный
     год. Прирост считается относительно предыдущего показания (в том числе
     сделанного в прошлом году), поэтому самое первое показание единицы техники
     в расход не попадает — это база отсчёта, а не наработка. */
  function hoursInYear(vehicleId, year) {
    const asc = hoursOf(vehicleId).slice().reverse();   // hoursOf отдаёт от новых к старым
    let prev = null, sum = 0;
    asc.forEach(h => {
      const val = Number(h.value);
      if (!isFinite(val)) return;
      if (prev != null && h.date && h.date.slice(0, 4) === String(year)) {
        sum += Math.max(0, val - prev);                 // отрицательные скачки (замена счётчика) не учитываем
      }
      prev = val;
    });
    return sum;
  }

  /* Последнее ТО, сбрасывающее межсервисный интервал (ЕО и ремонты не в счёт) */
  function lastResetMaint(vehicleId) {
    return maintOf(vehicleId).find(m => RESET_KINDS.has(m.kind)) || null;
  }

  /* Интервал ТО-2 по умолчанию — кратный интервалу ТО-1:
     N = периодичность ТО-2 / периодичность ТО-1 из раздела «Планирование»
     (по умолчанию 12000 / 2000 = 6). В карточке техники можно задать свой */
  function to2Every() {
    const s = DB.settings;
    const n = Math.round(Number(s.periodTO2) / Number(s.periodTO1));
    return isFinite(n) && n >= 2 ? n : 6;
  }
  function to2Intervals(iv) {
    const n = to2Every();
    return {
      hours: Number(iv.to2Hours) > 0 ? Number(iv.to2Hours) : (Number(iv.hours) > 0 ? Number(iv.hours) * n : 0),
      days:  Number(iv.to2Days)  > 0 ? Number(iv.to2Days)  : (Number(iv.days)  > 0 ? Number(iv.days)  * n : 0)
    };
  }

  /* Вид очередного планового ТО — из расчёта сроков по видам (computeStatus).
     Если интервалы не заданы — по циклу: после N−1 ТО-1 идёт ТО-2 */
  function nextMaintKind(v) {
    const st = computeStatus(v);
    if (st.nextKind) return st.nextKind;
    let to1 = 0;
    for (const m of maintOf(v.id)) {
      if (m.kind === 'ТО-1') to1++;
      else if (TO2_RESET_KINDS.has(m.kind)) break;
    }
    return to1 >= to2Every() - 1 ? 'ТО-2' : 'ТО-1';
  }

  /* «через 140 мото·ч · через 12 дней» / «перерасход 30 мото·ч · просрочено на 3 дня» */
```

```
  function dueText(k) {
    if (!k) return '—';
    const bits = [];
    if (k.remainingHours != null) bits.push(k.remainingHours > 0
      ? `через ${fmtNum(k.remainingHours, 0)} мото·ч` : `перерасход ${fmtNum(Math.abs(k.remainingHours), 0)} мото·ч`);
    if (k.remainingDays != null) bits.push(k.remainingDays > 0
      ? `через ${k.remainingDays} ${plural(k.remainingDays, 'день', 'дня', 'дней')}`
      : `просрочено на ${Math.abs(k.remainingDays)} ${plural(Math.abs(k.remainingDays), 'день', 'дня', 'дней')}`);
    return bits.join(' · ') || '—';
  }

  /* Уникальные значения поля — для выпадающих фильтров */
  function uniqueValues(field) {
    const s = new Set();
    DB.vehicles.forEach(v => { if (v[field]) s.add(v[field]); });
    return Array.from(s).sort((a, b) => a.localeCompare(b, 'ru'));
  }

  /* ============================================================================
     5. ВЫЧИСЛИТЕЛЬНОЕ ЯДРО: СОСТОЯНИЕ ТО
     ----------------------------------------------------------------------------
     Чистая функция: по единице техники возвращает полный расчёт ресурса
     до следующего обслуживания — по моточасам и/или по календарю.
     Итоговый уровень определяется наиболее «тревожным» из включённых критериев.
     ============================================================================ */
  function computeStatus(v) {
    const s = DB.settings;
    const res = {
      level: 'none', pct: null,
      lastMaint: null, baseDate: null, baseHours: 0,
      hoursSince: 0,
      remainingHours: null, remainingDays: null,
      nextHours: null, nextDate: null,
      kinds: [],          // сроки по видам: [{ kind: 'ТО-1', … }, { kind: 'ТО-2', … }]
      nextKind: null      // вид очередного ТО
    };

    if (!v) return res;

    if (v.status === 'Списано') {
      res.level = 'retired';
      return res;
    }

    const cur = Number(v.currentHours || 0);
    const regHours = registrationHours(v);
    const iv = v.interval || {};

    /* Срок одного вида ТО от его точки отсчёта. pct — остаток ресурса в %
       (наихудший из критериев «моточасы» и «календарь») */
    const crit = (kind, lm, hoursInt, daysInt) => {
      const k = { kind, lastMaint: lm,
        baseDate: lm ? lm.date : (v.regDate || null),
        baseHours: lm ? Number(lm.hours || 0) : regHours,
        nextHours: null, remainingHours: null, nextDate: null, remainingDays: null, pct: null };
      const pcts = [];
      if (iv.byHours && hoursInt > 0) {
        k.nextHours = k.baseHours + hoursInt;
        k.remainingHours = k.nextHours - cur;
        pcts.push((k.remainingHours / hoursInt) * 100);
      }
      if (iv.byDate && daysInt > 0 && k.baseDate) {
        k.nextDate = addDays(k.baseDate, daysInt);
        k.remainingDays = daysBetween(todayISO(), k.nextDate);
        pcts.push((k.remainingDays / daysInt) * 100);
      }
      k.pct = pcts.length ? Math.min.apply(null, pcts) : null;
      return k;
    };

    // ТО-1: отсчёт от любого планового ТО; ТО-2: только от ТО-2, ТО-3, КР
    const lm = lastResetMaint(v.id);
    const lm2 = maintOf(v.id).find(m => TO2_RESET_KINDS.has(m.kind)) || null;
    const t2 = to2Intervals(iv);
    const k1 = crit('ТО-1', lm, Number(iv.hours), Number(iv.days));
    const k2 = crit('ТО-2', lm2, t2.hours, t2.days);

    /* Нет записи о ТО-2, а с постановки на учёт прошло больше интервала ТО-2:
       данных нет (ТО-2 могли проводить до ведения журнала). Такой срок не
       ухудшает состояние — карточка просит внести дату последнего ТО-2 */
    if (!lm2 && k2.pct != null) {
      const sinceRegDays = v.regDate ? daysBetween(v.regDate, todayISO()) : null;
      k2.unknown = (iv.byDate && t2.days > 0 && sinceRegDays != null && sinceRegDays >= t2.days) ||
                   (iv.byHours && t2.hours > 0 && cur - regHours >= t2.hours);
    }

    res.lastMaint = lm;
    res.baseDate  = k1.baseDate;
    res.baseHours = k1.baseHours;
    res.hoursSince = Math.max(0, cur - res.baseHours);
```

```
    res.kinds = [k1, k2].filter(k => k.pct != null);
    const known = res.kinds.filter(k => !k.unknown);
    if (!known.length) { res.level = 'none'; return res; }

    // ТО-2 включает объём ТО-1, поэтому оно очередное, если наступает не позже ТО-1
    const earlier = (a, b) =>
      (a.remainingHours != null && b.remainingHours != null && a.remainingHours <= b.remainingHours) ||
      (a.remainingDays  != null && b.remainingDays  != null && a.remainingDays  <= b.remainingDays);
    const k2ok = k2.pct != null && !k2.unknown;
    const next = (k2ok && (k1.pct == null || earlier(k2, k1))) ? k2 : k1;
    // Срок ТО-2 неизвестен — вид очередного ТО определит цикл по журналу
    res.nextKind = next.kind === 'ТО-1' && k2.unknown ? null : next.kind;
    res.nextHours = next.nextHours;       res.remainingHours = next.remainingHours;
    res.nextDate  = next.nextDate;        res.remainingDays  = next.remainingDays;

    // Состояние — по наихудшему остатку среди всех видов
    res.pct = Math.min.apply(null, known.map(k => k.pct));

    if (res.pct <= 0)            res.level = 'overdue';
    else if (res.pct <= s.urgentPct) res.level = 'urgent';
    else if (res.pct <  s.warnPct)   res.level = 'soon';
    else                         res.level = 'ok';

    return res;
  }

  /* Показание счётчика при постановке на учёт — самое раннее показание не
     позже даты постановки (в тот же день могут быть и более поздние записи ТО). Раньше отсчёт без записей ТО всегда шёл от нуля, и
     техника, поставленная на учёт с наработкой, сразу считалась просроченной */
  function registrationHours(v) {
    if (!v.regDate) return 0;
    const early = hoursOf(v.id).filter(h => h.date && h.date <= v.regDate);
    return early.length ? Number(early[early.length - 1].value) || 0 : 0;   // hoursOf — от новых к старым
  }

  /* Признак «моточасы давно не обновлялись» */
  function hoursStale(v) {
    if (!v || v.status !== 'В эксплуатации') return false;
    const last = v.hoursUpdatedAt || v.regDate;
    if (!last) return false;
    const d = daysBetween(last, todayISO());
    return d != null && d >= DB.settings.hoursStaleDays;
  }

  /* Полный список уведомлений */
  function computeNotifications() {
    const out = [];
    DB.vehicles.forEach(v => {
      if (v.status === 'Списано') return;
      const st = computeStatus(v);
      const who = `${v.name}${v.regNumber ? ' · ' + v.regNumber : ''}`;

      if (st.level === 'overdue') {
        const parts = [];
        if (st.remainingHours != null && st.remainingHours <= 0)
          parts.push(`перерасход ${fmtNum(Math.abs(st.remainingHours))} мото·ч`);
        if (st.remainingDays != null && st.remainingDays <= 0)
          parts.push(`просрочено на ${Math.abs(st.remainingDays)} ${plural(Math.abs(st.remainingDays), 'день', 'дня', 'дней')}`);
        out.push({ level: 'overdue', vehicleId: v.id, title: `${st.nextKind || nextMaintKind(v)} просрочено`,
                   text: `${who}${parts.length ? ' — ' + parts.join(', ') : ''}` });
      } else if (st.level === 'urgent') {
        out.push({ level: 'urgent', vehicleId: v.id, title: 'Требуется ТО в ближайшее время',
                   text: `${who} — остаток ресурса ${Math.round(st.pct)}%` });
      } else if (st.level === 'soon') {
        out.push({ level: 'soon', vehicleId: v.id, title: 'Приближается срок ТО',
                   text: `${who} — остаток ресурса ${Math.round(st.pct)}%` });
      }

      if (hoursStale(v)) {
        const d = daysBetween(v.hoursUpdatedAt || v.regDate, todayISO());
        out.push({ level: 'info', vehicleId: v.id, title: 'Необходимо обновить моточасы',
                   text: `${who} — нет данных ${d} ${plural(d, 'день', 'дня', 'дней')}` });
      }
    });

    const sn = seasonNotification();
    if (sn) out.push(sn);
    const rank = { overdue: 0, urgent: 1, soon: 2, info: 3 };
    return out.sort((a, b) => rank[a.level] - rank[b.level]);
  }

  /* ========================================================================
     6. ЭКСПОРТ В XLSX — собственный генератор (ZIP «store» + OOXML)
     ------------------------------------------------------------------------
     Полностью автономен: без внешних библиотек и без обращения к сети.
     ======================================================================== */

  const CRC_TABLE = (() => {
```

```
    const t = new Uint32Array(256);
    for (let i = 0; i < 256; i++) {
      let c = i;
      for (let k = 0; k < 8; k++) c = (c & 1) ? (0xEDB88320 ^ (c >>> 1)) : (c >>> 1);
      t[i] = c >>> 0;
    }
    return t;
  })();

  function crc32(bytes) {
    let c = 0xFFFFFFFF;
    for (let i = 0; i < bytes.length; i++) c = CRC_TABLE[(c ^ bytes[i]) & 0xFF] ^ (c >>> 8);
    return (c ^ 0xFFFFFFFF) >>> 0;
  }

  const utf8 = s => new TextEncoder().encode(s);

  function zipStore(files) {
    const now = new Date();
    const time = ((now.getHours() << 11) | (now.getMinutes() << 5) | (now.getSeconds() >> 1)) & 0xFFFF;
    const date = (((now.getFullYear() - 1980) << 9) | ((now.getMonth() + 1) << 5) | now.getDate()) & 0xFFFF;

    const local = [], central = [];
    let offset = 0;

    files.forEach(f => {
      const nameBytes = utf8(f.name);
      const data = f.data;
      const crc = crc32(data);
      const size = data.length;

      const lh = new Uint8Array(30 + nameBytes.length);
      const lv = new DataView(lh.buffer);
      lv.setUint32(0, 0x04034b50, true);
      lv.setUint16(4, 20, true);
      lv.setUint16(6, 0x0800, true);      // флаг UTF-8 в именах
      lv.setUint16(8, 0, true);           // метод 0 — без сжатия
      lv.setUint16(10, time, true);
      lv.setUint16(12, date, true);
      lv.setUint32(14, crc, true);
      lv.setUint32(18, size, true);
      lv.setUint32(22, size, true);
      lv.setUint16(26, nameBytes.length, true);
      lv.setUint16(28, 0, true);
      lh.set(nameBytes, 30);
      local.push(lh, data);

      const ch = new Uint8Array(46 + nameBytes.length);
      const cv = new DataView(ch.buffer);
      cv.setUint32(0, 0x02014b50, true);
      cv.setUint16(4, 20, true);
      cv.setUint16(6, 20, true);
      cv.setUint16(8, 0x0800, true);
      cv.setUint16(10, 0, true);
      cv.setUint16(12, time, true);
      cv.setUint16(14, date, true);
      cv.setUint32(16, crc, true);
      cv.setUint32(20, size, true);
      cv.setUint32(24, size, true);
      cv.setUint16(28, nameBytes.length, true);
      cv.setUint32(42, offset, true);
      ch.set(nameBytes, 46);
      central.push(ch);

      offset += lh.length + size;
    });

    const cdSize = central.reduce((s, c) => s + c.length, 0);
    const eocd = new Uint8Array(22);
    const ev = new DataView(eocd.buffer);
    ev.setUint32(0, 0x06054b50, true);
    ev.setUint16(8, files.length, true);
    ev.setUint16(10, files.length, true);
    ev.setUint32(12, cdSize, true);
    ev.setUint32(16, offset, true);

    const all = local.concat(central, [eocd]);
    const total = all.reduce((s, c) => s + c.length, 0);
    const out = new Uint8Array(total);
    let p = 0;
    all.forEach(c => { out.set(c, p); p += c.length; });
    return out;
  }

  const xmlEsc = s => String(s == null ? '' : s)
    .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;').replace(/'/g, '&apos;')
    .replace(/[\x00-\x08\x0B\x0C\x0E-\x1F]/g, '');

  function colLetter(n) {
```

```
    let s = '';
    while (n > 0) { const m = (n - 1) % 26; s = String.fromCharCode(65 + m) + s; n = ((n - 1) / 26) | 0; }
    return s;
  }

  function sheetXml(rows, widths) {
    const cols = (widths && widths.length)
      ? `<cols>${widths.map((w, i) => `<col min="${i + 1}" max="${i + 1}" width="${w}" customWidth="1"/>`).join('')}</cols>`
      : '';
    const body = rows.map((row, r) => {
      const cells = row.map((val, c) => {
        const ref = colLetter(c + 1) + (r + 1);
        const st = r === 0 ? ' s="1"' : '';
        if (val === null || val === undefined || val === '') return `<c r="${ref}"${st}/>`;
        if (typeof val === 'number' && isFinite(val)) return `<c r="${ref}"${st}><v>${val}</v></c>`;
        return `<c r="${ref}"${st} t="inlineStr"><is><t xml:space="preserve">${xmlEsc(val)}</t></is></c>`;
      }).join('');
      return `<row r="${r + 1}">${cells}</row>`;
    }).join('');
    const last = rows.length ? colLetter(Math.max(1, rows[0].length)) + rows.length : 'A1';
    return `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<worksheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main"><dimension ref="A1:${last}"/><sheetViews><sheetView workbookViewId="0"><pane ySplit="1" topLeftCell="A2" activePane="bottomLeft" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="15"/>${cols}<sheetData>${body}</sheetData><autoFilter ref="A1:${last}"/></worksheet>`;
  }

  const STYLES_XML = `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<styleSheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main"><fonts count="2"><font><sz val="11"/><name val="Calibri"/></font><font><b/><sz val="11"/><color rgb="FFFFFFFF"/><name val="Calibri"/></font></fonts><fills count="3"><fill><patternFill patternType="none"/></fill><fill><patternFill patternType="gray125"/></fill><fill><patternFill patternType="solid"><fgColor rgb="FF1F4E78"/><bgColor indexed="64"/></patternFill></fill></fills><borders count="1"><border><left/><right/><top/><bottom/><diagonal/></border></borders><cellStyleXfs count="1"><xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs count="2"><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1" fillId="2" borderId="0" xfId="0" applyFont="1" applyFill="1" applyAlignment="1"><alignment vertical="center" wrapText="1"/></xf></cellXfs><cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>`;

  /* sheets: [{ name, rows, widths }] → Uint8Array (.xlsx) */
  function buildXlsx(sheets) {
    const safe = (n, i) => (String(n).replace(/[\\/\*\?\[\]:]/g, ' ').slice(0, 31) || ('Лист' + (i + 1)));

    const ct = `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types"><Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/><Default Extension="xml" ContentType="application/xml"/><Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>${sheets.map((s, i) => `<Override PartName="/xl/worksheets/sheet${i + 1}.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>`).join('')}<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/></Types>`;

    const rels = `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships"><Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument" Target="xl/workbook.xml"/></Relationships>`;

    const wb = `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<workbook xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main" xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships"><sheets>${sheets.map((s, i) => `<sheet name="${xmlEsc(safe(s.name, i))}" sheetId="${i + 1}" r:id="rId${i + 1}"/>`).join('')}</sheets></workbook>`;

    const wbRels = `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">${sheets.map((s, i) => `<Relationship Id="rId${i + 1}" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/worksheet" Target="worksheets/sheet${i + 1}.xml"/>`).join('')}<Relationship Id="rId${sheets.length + 1}" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/styles" Target="styles.xml"/></Relationships>`;

    return zipStore([
      { name: '[Content_Types].xml',       data: utf8(ct) },
      { name: '_rels/.rels',               data: utf8(rels) },
      { name: 'xl/workbook.xml',           data: utf8(wb) },
      { name: 'xl/_rels/workbook.xml.rels',data: utf8(wbRels) },
      { name: 'xl/styles.xml',             data: utf8(STYLES_XML) },
    ].concat(sheets.map((s, i) => ({
      name: `xl/worksheets/sheet${i + 1}.xml`,
      data: utf8(sheetXml(s.rows, s.widths))
    }))));
  }

  /* Экспорт отчёта в Excel */
  function exportXlsx(report) {
    const data = report.rows();
    if (!data.length) { toast('warning', 'Нет данных', 'Отчёт пуст — экспорт не выполнен.'); return; }

    const header = report.columns.map(c => c.label);
    const rows = [header].concat(data.map(r => report.columns.map(c => {
      const v = r[c.key];
```

```
      return (c.num && typeof v === 'number') ? v : (v == null ? '' : String(v));
    })));

    const bytes = buildXlsx([{ name: report.sheet || report.title, rows, widths: report.columns.map(c => c.width || 18) }]);
    const blob = new Blob([bytes], { type: 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' });
    downloadBlob(blob, `${report.file || 'otchet'}_${todayISO()}.xlsx`);
    toast('success', 'Экспорт выполнен', `Файл Excel сформирован (${data.length} ${plural(data.length, 'запись', 'записи', 'записей')}).`);
  }

  /* ========================================================================
     7. ЭКСПОРТ В PDF — через системный диалог печати
     ------------------------------------------------------------------------
     Оффлайн-приложение не может встроить кириллический шрифт в PDF без
     внешних зависимостей, поэтому используется штатный механизм браузера
     «Печать → Сохранить как PDF». Разметка готовится под формат А4.
     ======================================================================== */
  function exportPdf(report) {
    const data = report.rows();
    if (!data.length) { toast('warning', 'Нет данных', 'Отчёт пуст — печать не выполнена.'); return; }

    const head = report.columns.map(c => `<th${c.num ? ' class="p-num"' : ''}>${esc(c.label)}</th>`).join('');
    const body = data.map(r => '<tr>' + report.columns.map(c => {
      const v = r[c.key];
      const cls = c.num ? ' class="p-num"' : '';
      return `<td${cls}>${esc(v == null || v === '' ? '—' : (c.num && typeof v === 'number' ? fmtNum(v) : v))}</td>`;
    }).join('') + '</tr>').join('');

    emitPrint({
      key: 'report-' + report.key, file: report.file, sheet: report.sheet,
      title: report.title,
      subtitle: 'АСУК «Регламент-ТС» · Учёт технических средств тыла',
      meta: 'Всего записей: ' + data.length,
      bodyHtml: `<table${report.columns.length > 9 ? ' class="dense"' : ''}><thead><tr>${head}</tr></thead><tbody>${body}</tbody></table>`,
      footHtml: null
    });
  }

  /* ========================================================================
     7а. ДОКУМЕНТЫ ДЛЯ ОКНА ПЕЧАТИ
     ------------------------------------------------------------------------
     Все печатные формы учёта собираются в «спецификацию» { title, subtitle,
     bodyHtml, footHtml } и открываются в окне предпросмотра (TSTPrint).
     Реестр PRINT_DOCS позволяет сменить документ прямо в окне: функция
     документа вызывается в режиме захвата (printSink) и окно не открывает
     ======================================================================== */
  let printKey = null;    // ключ документа, который сейчас формируется
  let printSink = null;   // захват спецификации вместо открытия окна

  function emitPrint(spec) {
    if (!spec.key) spec.key = printKey || 'custom';
    if (printSink) { printSink(spec); return; }
    if (PRINT_DOCS.some(d => d.key === spec.key)) lastPrintKey = spec.key;
    if (window.TSTPrint) {
      window.TSTPrint.open({ spec, docs: PRINT_DOCS.map(d => ({ key: d.key, label: d.label, group: d.group })), capture: captureDoc });
      return;
    }
    toast('warning', 'Печать недоступна', 'Окно предпросмотра не загружено.');
  }

  const PRINT_DOCS = [
    { key: 'annual',         group: 'Документы', label: 'Годовой план эксплуатации и ремонта',              run: () => printAnnualPlan() },
    { key: 'month',          group: 'Документы', label: 'План-график ТО и ремонта на месяц',                run: () => printMonthPlan() },
    { key: 'need',           group: 'Документы', label: 'Заявка на материалы, инструмент и специалистов',   run: () => printNeed() },
    { key: 'act-writeoff',   group: 'Акты',      label: 'Акт списания моторесурса',                         run: () => printActWriteoff() },
    { key: 'act-inspection', group: 'Акты',      label: 'Акт внезапной проверки наличия техники',           run: () => printActInspection() },
    { key: 'act-usage',      group: 'Акты',      label: 'Акт проверки использования техники и расхода ГСМ', run: () => printActUsage(ui.docs.actFrom, ui.docs.actTo) }
  ].concat(REPORTS_FOR_PRINT());

  /* Отчёты объявлены ниже (раздел 18) — реестр читает их при вызове */
  function REPORTS_FOR_PRINT() {
    return ['vehicles', 'maint', 'overdue', 'due', 'byUnit', 'byHours', 'trips', 'tripsMonthly'].map(k => ({
      key: 'report-' + k, group: 'Отчёты',
      get label() { const r = REPORTS.find(x => x.key === k); return r ? r.title : k; },
      run: () => exportPdf(getReport(k))
    }));
  }

  function runDoc(key) {
    const d = PRINT_DOCS.find(x => x.key === key);
    if (!d) return;
```

```
    printKey = key;
    try { d.run(); } finally { printKey = null; }
  }

  /* Построить документ, не открывая окно; null — данных нет (сообщение уже показано) */
  function captureDoc(key) {
    let got = null;
    printSink = spec => { got = spec; };
    try { runDoc(key); } finally { printSink = null; }
    return got;
  }

  /* Окно печати без заранее выбранного документа (Ctrl+P, кнопка «Печать»):
     открывается последний напечатанный документ или отчёт текущего раздела */
  function openPrintWindow() {
    const pref = ui.view === 'reports' ? 'report-' + ui.report.key
      : ui.view === 'documents' ? 'annual'
      : (lastPrintKey && PRINT_DOCS.some(d => d.key === lastPrintKey) ? lastPrintKey : 'report-vehicles');
    const order = [pref].concat(PRINT_DOCS.map(d => d.key).filter(k => k !== pref));
    for (const k of order) {
      const spec = captureDocQuiet(k);
      if (spec) { emitPrint(spec); return; }
    }
    toast('info', 'Нет данных для печати', 'Добавьте технику в учёт, чтобы сформировать документы.');
  }
  let lastPrintKey = null;
  /* Перебор документов без всплывающих «Нет данных» на каждом пустом */
  function captureDocQuiet(key) {
    const host = el('fleetToasts');
    const mute = host ? host.style.display : '';
    if (host) host.style.display = 'none';
    const n = host ? host.children.length : 0;
    try { return captureDoc(key); }
    finally {
      if (host) { while (host.children.length > n) host.lastChild.remove(); host.style.display = mute; }
    }
  }

  /* ========================================================================
     8. ИКОНКИ МОДУЛЯ (встроенные SVG, без внешних ресурсов)
     ======================================================================== */
  const S = (p, w) => `<svg width="${w || 16}" height="${w || 16}" viewBox="0 0 24 24" fill="none" stroke="currentColor"
   stroke-width="2" stroke-linecap="round" stroke-linejoin="round">${p}</svg>`;
  const FI = {
    truck:   p => S('<path d="M10 17h4V5H2v12h3"/><path d="M20 17h2v-3.34a4 4 0 0 0-1.17-2.83L19 9h-5v8h2"/><circle
  cx="7.5" cy="17.5" r="2.5"/><circle cx="17.5" cy="17.5" r="2.5"/>', p),
    gauge:   p => S('<path d="M12 22a10 10 0 1 0-10-10"/><path d="m12 12 4-4"/><circle cx="12" cy="12" r="1.5"
  fill="currentColor"/>', p),
    wrench:  p => S('<path d="M14.7 6.3a1 1 0 0 0 0 1.4l1.6 1.6a1 1 0 0 0 1.4 0l3.77-3.77a6 6 0 0 1-7.94 7.94l-6.91
  6.91a2.12 2.12 0 0 1-3-3l6.91-6.91a6 6 0 0 1 7.94-7.94l-3.76 3.76z"/>', p),
    alert:   p => S('<path d="m21.73 18-8-14a2 2 0 0 0-3.48 0l-8 14A2 2 0 0 0 4 21h16a2 2 0 0 0 1.73-3z"/><line x1="12"
  y1="9" x2="12" y2="13"/><line x1="12" y1="17" x2="12.01" y2="17"/>', p),
    chart:   p => S('<line x1="18" y1="20" x2="18" y2="10"/><line x1="12" y1="20" x2="12" y2="4"/><line x1="6" y1="20"
  x2="6" y2="14"/>', p),
    gear:    p => S('<circle cx="12" cy="12" r="3"/><path d="M19.4 15a1.65 1.65 0 0 0 .33 1.82l.06.06a2 2 0 0 1-2.83
  2.83l-.06-.06a1.65 1.65 0 0 0-1.82-.33 1.65 1.65 0 0 0-1 1.51V21a2 2 0 0 1-4 0v-.09a1.65 1.65 0 0 0-1-1.51 1.65 1.65 0
   0 0-1.82.33l-.06.06a2 2 0 0 1-2.83-2.83l.06-.06a1.65 1.65 0 0 0 .33-1.82 1.65 1.65 0 0 0-1.51-1H3a2 2 0 0 1
  0-4h.09a1.65 1.65 0 0 0 1.51-1 1.65 1.65 0 0 0-.33-1.82l-.06-.06a2 2 0 0 1 2.83-2.83l.06.06a1.65 1.65 0 0 0
  1.82.33h0a1.65 1.65 0 0 0 1-1.51V3a2 2 0 0 1 4 0v.09a1.65 1.65 0 0 0 1 1.51h0a1.65 1.65 0 0 0 1.82-.33l.06-.06a2 2 0 0
   1 2.83 2.83l-.06.06a1.65 1.65 0 0 0-.33 1.82v0a1.65 1.65 0 0 0 1.51 1H21a2 2 0 0 1 0 4h-.09a1.65 1.65 0 0 0-1.51
  1z"/>', p),
    plus:    p => S('<line x1="12" y1="5" x2="12" y2="19"/><line x1="5" y1="12" x2="19" y2="12"/>', p),
    edit:    p => S('<path d="M11 4H4a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h14a2 2 0 0 0 2-2v-7"/><path d="M18.5 2.5a2.12 2.12
  0 0 1 3 3L12 15l-4 1 1-4z"/>', p),
    trash:   p => S('<polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1
  2-2h4a2 2 0 0 1 2 2v2"/>', p),
    back:    p => S('<line x1="19" y1="12" x2="5" y2="12"/><polyline points="12 19 5 12 12 5"/>', p),
    x:       p => S('<line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18" y2="18"/>', p),
    excel:   p => S('<path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8
  20 8"/><line x1="9" y1="13" x2="15" y2="19"/><line x1="15" y1="13" x2="9" y2="19"/>', p),
    pdf:     p => S('<path d="M6 2h9l5 5v13a2 2 0 0 1-2 2H6a2 2 0 0 1-2-2V4a2 2 0 0 1 2-2z"/><polyline points="15 2 15 8
   20 8"/><path d="M8 15h1.5a1.5 1.5 0 0 0 0-3H8v6"/><path d="M14 18v-6h1.5a2 2 0 0 1 0 6z"/>', p),
    doc:     p => S('<path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8
  20 8"/>', p),
    download:p => S('<path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line
  x1="12" y1="15" x2="12" y2="3"/>', p),
    clock:   p => S('<circle cx="12" cy="12" r="10"/><polyline points="12 6 12 12 16 14"/>', p),
    list:    p => S('<line x1="8" y1="6" x2="21" y2="6"/><line x1="8" y1="12" x2="21" y2="12"/><line x1="8" y1="18"
  x2="21" y2="18"/><line x1="3" y1="6" x2="3.01" y2="6"/><line x1="3" y1="12" x2="3.01" y2="12"/><line x1="3" y1="18"
  x2="3.01" y2="18"/>', p),
    grid:    p => S('<rect x="3" y="3" width="7" height="7" rx="1"/><rect x="14" y="3" width="7" height="7"
  rx="1"/><rect x="3" y="14" width="7" height="7" rx="1"/><rect x="14" y="14" width="7" height="7" rx="1"/>', p),
    save:    p => S('<path d="M19 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11l5 5v11a2 2 0 0 1-2 2z"/><polyline points="17 21
  17 13 7 13 7 21"/><polyline points="7 3 7 8 15 8"/>', p),
    upload:  p => S('<path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line
  x1="12" y1="3" x2="12" y2="15"/>', p),
    building:p => S('<rect x="4" y="2" width="16" height="20" rx="2"/><line x1="9" y1="7" x2="15" y2="7"/><line x1="9"
  y1="11" x2="15" y2="11"/><line x1="9" y1="15" x2="15" y2="15"/>', p),
    route:   p => S('<circle cx="6" cy="19" r="2"/><circle cx="18" cy="5" r="2"/><path d="M8 19h7a4 4 0 0 0 4-4v-1a4 4 0
   0 0-4-4H9a4 4 0 0 1-4-4V5"/>', p),
```

```
    calc:    p => S('<rect x="4" y="2" width="16" height="20" rx="2"/><line x1="8" y1="6" x2="16" y2="6"/><line x1="8" y1="11" x2="8.01" y2="11"/><line x1="12" y1="11" x2="12.01" y2="11"/><line x1="16" y1="11" x2="16.01" y2="11"/><line x1="8" y1="15" x2="8.01" y2="15"/><line x1="12" y1="15" x2="12.01" y2="15"/><line x1="16" y1="15" x2="16.01" y2="15"/><line x1="8" y1="19" x2="8.01" y2="19"/><line x1="12" y1="19" x2="12.01" y2="19"/><line x1="16" y1="19" x2="16.01" y2="19"/>', p),
    file:    p => S('<path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="9" y1="13" x2="15" y2="13"/><line x1="9" y1="17" x2="13" y2="17"/>', p)
  };

  /* ========================================================================
     9. СОСТОЯНИЕ ИНТЕРФЕЙСА МОДУЛЯ
     ======================================================================== */
  const ui = {
    view: 'command',     // стартовый экран — пункт управления
    vehicleId: null,
    vehicleTab: 'info',
    park: {
      q: '', unit: '', responsible: '', status: '', level: '',
      from: '', to: '', mode: 'table',
      sort: { key: 'name', dir: 'asc' },
      selected: new Set()   // id-ы отмеченных чекбоксами единиц техники (массовые операции)
    },
    dash:   { sort: { key: 'pct', dir: 'asc' } },
    report: { key: 'vehicles', from: '', to: '', unit: '' },
    trips:  { vehicle: '', from: '', to: '' },
    qr:     { q: '', filter: '', selected: new Set() },
    cp:     { range: 'today' },
    safety: { report: null, tests: null, fixes: null, hq: '' },
    planTab: 'ktg',
    docs:    { actFrom: addDays(todayISO(), -30), actTo: todayISO(), needDays: 30 },
    econ:    { from: '', to: '' }
  };

  /* Короткие ссылки на DOM */
  const $  = sel => document.querySelector(sel);
  const el = id => document.getElementById(id);

  /* ========================================================================
     10. ВСПЛЫВАЮЩИЕ УВЕДОМЛЕНИЯ (TOAST)
     ======================================================================== */
  /* opts.action / opts.onAction — кнопка в уведомлении (например, «Отменить»);
     opts.onExpire — вызывается, если уведомление закрыто без нажатия кнопки */
  function toast(type, title, text, timeout, opts) {
    opts = opts || {};
    const host = el('fleetToasts');
    if (!host) { if (opts.onExpire) opts.onExpire(); return; }
    const ms = timeout || 5200;
    const node = document.createElement('div');
    node.className = 'f-toast f-toast-' + (type || 'info');
    node.innerHTML = `
      <div style="flex:1;min-width:0">
        <div class="f-toast-title">${esc(title)}</div>
        ${text ? `<div class="f-toast-text">${esc(text)}</div>` : ''}
      </div>
      ${opts.action ? `<button type="button" class="f-toast-action">${esc(opts.action)}</button>` : ''}
      <button type="button" class="f-toast-close" aria-label="Закрыть">${FI.x(14)}</button>
      ${opts.action ? `<span class="f-toast-timer" style="animation-duration:${ms}ms"></span>` : ''}`;
    host.appendChild(node);

    let done = false;
    const kill = reason => {
      if (done) return;
      done = true;
      node.classList.add('is-out');
      setTimeout(() => node.remove(), 260);
      if (reason !== 'action' && opts.onExpire) opts.onExpire();
    };
    node.querySelector('.f-toast-close').addEventListener('click', () => kill('close'));
    const act = node.querySelector('.f-toast-action');
    if (act) act.addEventListener('click', () => { kill('action'); if (opts.onAction) opts.onAction(); });
    setTimeout(() => kill('timeout'), ms);
  }

  /* ========================================================================
     10а. УДАЛЕНИЕ С ОТМЕНОЙ
     ------------------------------------------------------------------------
     Вместо окна «Действие необратимо» запись удаляется сразу, а в уведомлении
     есть кнопка «Отменить» (8 с). Отмена возвращает удалённые записи по id —
     изменения, сделанные за эти секунды, не теряются. Вложения физически
     стираются из IndexedDB только после истечения срока отмены
     ======================================================================== */
  const UNDO_MS = 8000;
  const pendingFileDeletes = new Set();

  function removeWithUndo(title, text, mutate) {
    const before = {};
    COLLECTIONS.forEach(k => { before[k] = DB[k]; });
    const docIds = [];
    mutate(docIds);
    const removed = {};
```

```
    COLLECTIONS.forEach(k => {
      const keep = new Set(DB[k].map(r => r.id));
      removed[k] = before[k].filter(r => !keep.has(r.id));
    });
    docIds.forEach(id => pendingFileDeletes.add(id));
    const trashId = trashPush(title, text, removed);
    saveDB();
    rerender();

    toast('success', title, text, UNDO_MS, {
      action: 'Отменить',
      onAction: () => {
        COLLECTIONS.forEach(k => {
          const have = new Set(DB[k].map(r => r.id));
          DB[k] = DB[k].concat(removed[k].filter(r => !have.has(r.id)));
        });
        docIds.forEach(id => pendingFileDeletes.delete(id));
        if (trashId) DB.trash = DB.trash.filter(t => t.id !== trashId);
        const touched = new Set([].concat(
          removed.vehicles.map(v => v.id), removed.maint.map(m => m.vehicleId),
          removed.hours.map(h => h.vehicleId), removed.trips.map(t => t.vehicleId)));
        touched.forEach(id => recomputeHours(id));
        saveDB();
        rerender();
        toast('info', 'Удаление отменено', '');
      },
      // Файлы остаются: запись лежит в корзине; их уберёт cleanupOrphanFiles, когда ссылок не останется
      onExpire: () => docIds.forEach(id => pendingFileDeletes.delete(id))
    });
  }

  /* Вложения, на которые не ссылается ни одна запись ТО (например, вкладка
     закрыта до истечения срока отмены). Запускается с задержкой, заведомо
     большей срока отмены, чтобы отмена в другой вкладке успела синхронизироваться */
  async function cleanupOrphanFiles() {
    let used;
    // Ссылки — из текущих записей, корзины и точек восстановления. Не удалось
    // прочитать точки — не удаляем ничего (лучше лишний файл, чем потерянный)
    try { used = await referencedFileIds(); } catch (e) { return; }
    idbRun('readonly', st => st.getAllKeys())
      .then(keys => (keys || []).forEach(k => { if (!used.has(k)) fileDel(k); }))
      .catch(() => {});
    nativeFileList().then(ids => ids.forEach(k => { if (!used.has(k)) nativeFileDel(k); }));
  }

  /* ========================================================================
     11. МОДАЛЬНЫЕ ОКНА
     ======================================================================== */
  let modalEsc = null;

  function openModal(html, opts) {
    const host = el('fleetModalHost');
    const back = document.createElement('div');
    back.className = 'f-modal-backdrop';
    back.innerHTML = `<div class="f-modal ${opts && opts.narrow ? 'is-narrow' : ''}" role="dialog" aria-
  modal="true">${html}</div>`;
    host.appendChild(back);

    // Доступность: заголовок окна — его имя для скринридера; фокус после
    // закрытия возвращается туда, откуда окно открыли
    const dialog = back.firstElementChild;
    const titleEl = dialog.querySelector('.f-modal-title');
    if (titleEl) { titleEl.id = uid('mt'); dialog.setAttribute('aria-labelledby', titleEl.id); }
    const opener = document.activeElement;

    // Фокус не уходит за пределы окна: Tab с последнего элемента — на первый,
    // Shift+Tab с первого — на последний (раньше фокус «проваливался» под окно)
    const focusables = () => Array.from(dialog.querySelectorAll(
      'button:not([disabled]), [href], input:not([disabled]):not([type="hidden"]), select:not([disabled]),
  textarea:not([disabled]), [tabindex]:not([tabindex="-1"])'))
      .filter(n => n.offsetParent !== null);
    // Слушаем документ, а не окно: фокус может оказаться снаружи (например,
    // сразу после открытия), и Tab с него тоже должен вернуться в окно
    const trapTab = e => {
      if (e.key !== 'Tab' || !back.isConnected) return;
      const list = focusables();
      if (!list.length) { e.preventDefault(); return; }
      const first = list[0], last = list[list.length - 1];
      if (e.shiftKey && (document.activeElement === first || !dialog.contains(document.activeElement))) {
  e.preventDefault(); last.focus(); }
      else if (!e.shiftKey && (document.activeElement === last || !dialog.contains(document.activeElement))) {
  e.preventDefault(); first.focus(); }
    };
    document.addEventListener('keydown', trapTab, true);

    let closed = false;
    const close = () => {
      if (closed) return;
      closed = true;
      back.remove();
```

```
      document.removeEventListener('keydown', trapTab, true);
      if (opener && opener.focus && document.contains(opener)) opener.focus({ preventScroll: true });
      if (modalEsc) { document.removeEventListener('keydown', modalEsc); modalEsc = null; }
      if (opts && opts.onClose) opts.onClose();
      // Пока окно было открыто, другая вкладка могла сохранить данные —
      // применяем их после закрытия (объекты формы больше не используются)
      if (pendingExternal) setTimeout(applyExternal, 0);
    };
    back.addEventListener('mousedown', e => { if (e.target === back) close(); });
    // Закрывают окно ВСЕ элементы с data-close (крестик и кнопка «Отмена»)
    back.querySelectorAll('[data-close]').forEach(b => b.addEventListener('click', close));

    modalEsc = e => { if (e.key === 'Escape') { e.stopPropagation(); close(); } };
    document.addEventListener('keydown', modalEsc);

    const first = back.querySelector('input:not([type="hidden"]),select,textarea') ||
                  back.querySelector('[data-yes], .f-btn-primary') || back.querySelector('[data-close]');
    if (first) first.focus({ preventScroll: true });

    return { root: back, close };
  }

  /* Диалог подтверждения — возвращает Promise<boolean> */
  function confirmDialog(title, text, confirmLabel) {
    return new Promise(resolve => {
      const m = openModal(`
        <div class="f-modal-head">
          <div>
            <div class="f-modal-title">${esc(title)}</div>
            <div class="f-modal-sub">${esc(text)}</div>
          </div>
          <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
        </div>
        <div class="f-modal-foot">
          <button type="button" class="f-btn" data-no>Отмена</button>
          <button type="button" class="f-btn f-btn-danger" data-yes>${esc(confirmLabel || 'Удалить')}</button>
        </div>`, { narrow: true, onClose: () => resolve(false) });

      // Любое закрытие (Escape, крестик, клик по фону, «Отмена») — это «нет»:
      // раньше Escape закрывал окно, но Promise так и не завершался.
      // Повторный resolve игнорируется, поэтому «Удалить» даёт true
      m.root.querySelector('[data-no]').addEventListener('click', () => m.close());
      m.root.querySelector('[data-yes]').addEventListener('click', () => { resolve(true); m.close(); });
    });
  }

  /* ============================================================================
     12. ПЕРЕИСПОЛЬЗУЕМЫЕ ЭЛЕМЕНТЫ ОТРИСОВКИ
     ============================================================================ */

  /* Бейдж состояния ТО с цветовой индикацией */
  function levelBadge(level) {
    const meta = LEVELS[level] || LEVELS.none;
    return `<span class="f-badge f-badge-${level}"><span class="f-dot f-lvl-${level}"></span>${esc(meta.label)}</span>`;
  }

  /* Ячейка «остаток ресурса»: прогресс-бар + подпись */
  function resourceCell(st) {
    if (st.level === 'retired' || st.level === 'none') {
      return `<div class="f-bar-cell">
        <div class="f-bar"><div class="f-bar-fill f-lvl-${st.level}" style="width:0%"></div></div>
        <div class="f-bar-caption">${st.level === 'retired' ? 'снято с учёта' : 'интервал не задан'}</div>
      </div>`;
    }

    // При просрочке шкала заполняется полностью красным — сигнал перерасхода
    const width = st.level === 'overdue' ? 100 : clamp(st.pct, 2, 100);

    const bits = [];
    if (st.remainingHours != null) {
      bits.push(st.remainingHours >= 0
        ? `${fmtNum(st.remainingHours)} мото·ч`
        : `−${fmtNum(Math.abs(st.remainingHours))} мото·ч`);
    }
    if (st.remainingDays != null) {
      bits.push(st.remainingDays >= 0
        ? `${st.remainingDays} ${plural(st.remainingDays, 'день', 'дня', 'дней')}`
        : `−${Math.abs(st.remainingDays)} ${plural(Math.abs(st.remainingDays), 'день', 'дня', 'дней')}`);
    }

    return `<div class="f-bar-cell">
      <div class="f-bar"><div class="f-bar-fill f-lvl-${st.level}" style="width:${width}%"></div></div>
      <div class="f-bar-caption">${st.level === 'overdue' ? 'просрочено · ' : Math.round(st.pct) + '% ·
'}${esc(bits.join(' · '))}</div>
    </div>`;
  }

  /* Учётная карточка единицы техники (режим «Карточки» в «Парке техники»).
     Оформление — как у штабного документа: шапка с номером карточки и
     статусом, нумерованные разделы, реквизиты моноширинным шрифтом,
```

```
      даты ДД.ММ.ГГГГ, единицы измерения у каждого значения */
  function staffCard(v, st, i, selected) {
    const dash = '—';
    const row = (k, val, cls) => `<div class="sc-row"><span class="sc-k">${k}</span><span class="sc-v ${cls || ''}">${val}</span></div>`;
    const node = catNodeOf(v);
    const mc = maintOf(v.id);
    const live = st.level !== 'retired';
    const kind = live && st.level !== 'none' ? nextMaintKind(v) : '';
    const lastM = st.lastMaint || mc[0] || null;
    const since = v.regDate ? daysBetween(v.regDate, todayISO()) : null;
    const yrs = since != null ? Math.floor(since / 365) : null;
    const stale = hoursStale(v);
    const upd = v.hoursUpdatedAt || v.regDate;
    const red = n => n != null && n <= 0 ? 'is-bad' : '';
    const nextVal = !live ? 'снято с учёта'
      : st.level === 'none' ? 'интервал не задан'
      : `${esc(kind)} · ${st.nextDate ? fmtDate(st.nextDate) : fmtNum(st.nextHours, 0) + ' мото·ч'}`;
    const remain = [];
    if (st.remainingHours != null) remain.push(`${st.remainingHours < 0 ? '−' : ''}${fmtNum(Math.abs(st.remainingHours), 0)} мото·ч`);
    if (st.remainingDays != null) remain.push(`${st.remainingDays < 0 ? '−' : ''}${Math.abs(st.remainingDays)} дн.`);
    const iv = v.interval || {};
    const ivText = [iv.byHours && iv.hours ? fmtNum(iv.hours, 0) + ' мото·ч' : '', iv.byDate && iv.days ? iv.days + ' дн.' : ''].filter(Boolean).join(' / ') || dash;
    return `
      <article class="f-vcard sc lvl-${st.level} ${selected ? 'is-selected' : ''}" data-act="open-vehicle" data-id="${v.id}" aria-label="Учётная карточка: ${esc(v.name)}">
        <input type="checkbox" class="f-vcard-check" data-act="row-select" data-id="${v.id}" title="Выбрать" aria-label="Выбрать: ${esc(v.name)}" ${selected ? 'checked' : ''}>
        <header class="sc-top">
          <span class="sc-doc">Учётная карточка № ${String(i + 1).padStart(3, '0')}</span>
          <span class="sc-status">${esc(v.status)}</span>
        </header>
        <div class="sc-title">
          ${vfThumbHtml(v, 'vf-card-ph')}
          <div class="sc-name">${esc(v.name)}</div>
          <div class="sc-sub">${esc([v.brand, v.year ? v.year + ' г. в.' : ''].filter(Boolean).join(' · ') || 'Марка не указана')}</div>
          ${node ? `<div class="sc-cat" title="${esc(node.name)}">${FI.file(12)}<span>${esc(node.name)}</span></div>` : ''}
        </div>

        <section class="sc-sec">
          <h4 class="sc-h"><b>1</b>Идентификация</h4>
          <div class="sc-ids">
            <div><span>Гос. №</span><b>${esc(v.regNumber || dash)}</b></div>
            <div><span>Инв. №</span><b>${esc(v.invNumber || dash)}</b></div>
            <div><span>Зав. №</span><b>${esc(v.serialNumber || dash)}</b></div>
          </div>
        </section>

        <section class="sc-sec">
          <h4 class="sc-h"><b>2</b>Закрепление</h4>
          ${row('Подразделение', esc(v.unit || dash))}
          ${row('Ответственный', esc(v.responsible || dash))}
          ${row('Группа эксплуатации', esc(v.expGroup || dash))}
          ${row('На учёте', v.regDate ? fmtDate(v.regDate) + (yrs != null ? ` <small>(${yrs ? yrs + ' ' + plural(yrs, 'год', 'года', 'лет') : 'менее года'})</small>` : '') : dash)}
        </section>

        <section class="sc-sec">
          <h4 class="sc-h"><b>3</b>Техническое состояние</h4>
          <div class="sc-state">
            ${resourceRing(st)}
            <div class="sc-state-rows">
              ${row('Наработка', fmtNum(v.currentHours, 0) + ' <small>мото·ч</small>')}
              ${row('С последнего ТО', live ? fmtNum(st.hoursSince, 0) + ' <small>мото·ч</small>' : dash)}
              ${row('Интервал ТО-1', ivText)}
            </div>
          </div>
          ${row('Последнее ТО', lastM ? `${esc(lastM.kind.replace(/ \(.*\)$/, ''))} · ${fmtDate(lastM.date)}` : 'записей нет')}
          ${row('Очередное ТО', nextVal, st.level === 'overdue' ? 'is-bad' : '')}
          ${remain.length ? row('Остаток', remain.join(' · '), red(Math.min(st.remainingHours == null ? 1 : st.remainingHours, st.remainingDays == null ? 1 : st.remainingDays))) : ''}
        </section>

        <footer class="sc-foot">
          ${levelBadge(st.level)}
          <span class="sc-meta">Записей ТО: ${mc.length}</span>
          <span class="sc-meta ${stale ? 'is-warn' : ''}" title="Дата последнего показания счётчика">${stale ? '⚠ ' : ''}Счётчик: ${upd ? fmtDate(upd) : dash}</span>
        </footer>
      </article>`;
  }

  /* Круговая диаграмма остатка ресурса */
  function resourceRing(st) {
    const R = 19, C = 2 * Math.PI * R;
```

```
  const pct = (st.level === 'none' || st.level === 'retired') ? 0 : clamp(st.pct, 0, 100);
  const offset = C * (1 - pct / 100);
  const label = (st.level === 'none') ? '—'
              : (st.level === 'retired') ? '—'
              : (st.level === 'overdue') ? '0%' : Math.round(st.pct) + '%';
  return `
    <div class="f-ring-wrap">
      <svg class="f-ring" viewBox="0 0 44 44">
        <circle class="f-ring-bg" cx="22" cy="22" r="${R}"/>
        <circle class="f-ring-fg f-lvl-${st.level}" cx="22" cy="22" r="${R}"
                stroke-dasharray="${C.toFixed(2)}" stroke-dashoffset="${offset.toFixed(2)}"/>
      </svg>
      <div class="f-ring-label">
        <div class="f-ring-pct">${label}</div>
        <div class="f-ring-cap">${esc(LEVELS[st.level].label)}</div>
      </div>
    </div>`;
}

/* Заголовок сортируемой таблицы */
function th(label, key, table, opts) {
  const o = opts || {};
  const cur = table === 'park' ? ui.park.sort : ui.dash.sort;
  const active = cur.key === key;
  const ind = active ? (cur.dir === 'asc' ? '▲' : '▼') : '';
  return `<th class="is-sortable ${o.num ? 'f-td-num' : ''}" data-act="sort" data-table="${table}" data-key="${key}">
    ${esc(label)}<span class="f-sort-ind">${ind}</span>
  </th>`;
}

/* Универсальный компаратор */
function cmp(a, b, dir) {
  const k = dir === 'asc' ? 1 : -1;
  if (a == null && b == null) return 0;
  if (a == null) return 1;        // пустые значения всегда в конце
  if (b == null) return -1;
  if (typeof a === 'number' && typeof b === 'number') return (a - b) * k;
  return String(a).localeCompare(String(b), 'ru') * k;
}

/* Пустое состояние */
function emptyState(icon, title, text, actionHtml) {
  return `<div class="f-empty">
    <div class="f-empty-icon">${icon}</div>
    <h3>${esc(title)}</h3>
    <p>${esc(text)}</p>
    ${actionHtml || ''}
  </div>`;
}

/* Опции <select> */
function options(list, selected, placeholder) {
  return (placeholder ? `<option value="">${esc(placeholder)}</option>` : '') +
    list.map(v => `<option value="${esc(v)}"${v === selected ? ' selected' : ''}>${esc(v)}</option>`).join('');
}

/* ============================================================================
   13. НАВИГАЦИЯ МОДУЛЯ
   ============================================================================ */
const NAV = [
  { key: 'command',   label: 'Пункт управления',         icon: FI.gauge },
  { key: 'park',      label: 'Парк техники',             icon: FI.truck },
  { key: 'dashboard', label: 'Ближайшие ТО',             icon: FI.alert },
  { key: 'trips',     label: 'Путевые листы',            icon: FI.route },
  { key: 'planning',  label: 'Планирование',             icon: FI.calc  },
  { key: 'documents', label: 'Документы',                icon: FI.file  },
  { key: 'reports',   label: 'Отчёты',                   icon: FI.chart },
  { key: 'qr',        label: 'QR-маркировка',            icon: FI.grid  },
  { key: 'safety',    label: 'Сохранность данных',       icon: FI.save  },
  { key: 'settings',  label: 'Настройки',                icon: FI.gear  }
];

function renderNav() {
  const alerts = computeNotifications().filter(n => !n.act && (n.level === 'overdue' || n.level === 'urgent')).length;
  el('fleetNavbar').innerHTML = NAV.map(n => `
    <button type="button" class="f-nav-btn ${ui.view === n.key || (n.key === 'park' && ui.view === 'vehicle') ? 'is-active' : ''}"
            role="tab" data-act="nav" data-key="${n.key}">
      ${n.icon(15)}<span>${esc(n.label)}</span>
      ${n.key === 'dashboard' && alerts ? `<span class="f-nav-count">${alerts}</span>` : ''}
    </button>`).join('') + `
    <button type="button" class="f-nav-btn f-nav-help" data-act="open-print" title="Предпросмотр и печать документов (Ctrl+P)">${FI.pdf(15)}<span>Печать</span></button>
    <button type="button" class="f-nav-btn" data-act="help" title="Справка по учёту (F1)">${FI.doc(15)}<span>Справка</span></button>`;
}

function go(view, vehicleId) {
  if (view !== 'qr') qrStopCamera();
  ui.view = view;
```

```
    if (vehicleId !== undefined) ui.vehicleId = vehicleId;
    if (view === 'vehicle') ui.vehicleTab = 'info';
    renderNav();
    renderView();
    if (window.TST) window.TST.syncRoute();
    el('fleetScroll').scrollTo({ top: 0, behavior: 'smooth' });
  }

  function renderView() {
    const v = el('fleetView');
    switch (ui.view) {
      case 'park':      v.innerHTML = renderPark();      bindParkFilters(); break;
      case 'vehicle':
        // Карточка могла быть удалена — корректно откатываемся к списку
        if (!getVehicle(ui.vehicleId)) {
          ui.view = 'park';
          v.innerHTML = renderPark();
          bindParkFilters();
        } else {
          v.innerHTML = renderVehicle();
        }
        break;
      case 'command':   v.innerHTML = renderCommand();   break;
      case 'safety':    v.innerHTML = renderSafety();    bindSafety(); break;
      case 'dashboard': v.innerHTML = renderDashboard(); break;
      case 'trips':     v.innerHTML = renderTrips();     bindTripFilters(); break;
      case 'planning':  v.innerHTML = renderPlanning();  bindPlanningInputs(); break;
      case 'documents': v.innerHTML = renderDocuments(); bindDocuments(); break;
      case 'reports':   v.innerHTML = renderReports();   bindReportFilters(); break;
      case 'qr':        v.innerHTML = renderQr();        bindQr(); break;
      case 'settings':  v.innerHTML = renderSettings();  bindSettings(); break;
    }
  }

  /* ========================================================================
     14. РАЗДЕЛ «ПАРК ТЕХНИКИ» — поиск, фильтрация, таблица/карточки
     ======================================================================== */

  /* Применение всех фильтров и сортировки */
  function parkRows() {
    const f = ui.park;
    const q = f.q.trim().toLowerCase();

    let rows = DB.vehicles.map(v => ({ v, st: computeStatus(v) }));

    rows = rows.filter(({ v, st }) => {
      // Полнотекстовый поиск: гос. номер, модель, наименование, инв./зав. номера
      if (q) {
        const hay = [v.name, v.brand, v.regNumber, v.invNumber, v.serialNumber, v.unit, v.responsible]
          .filter(Boolean).join(' ').toLowerCase();
        if (hay.indexOf(q) === -1) return false;
      }
      if (f.unit        && v.unit        !== f.unit) return false;
      if (f.responsible && v.responsible !== f.responsible) return false;
      if (f.status      && v.status      !== f.status) return false;
      if (f.level       && st.level      !== f.level) return false;

      // Диапазон дат ТО: техника, у которой есть хотя бы одна запись ТО в периоде
      if (f.from || f.to) {
        const hit = DB.maint.some(m => m.vehicleId === v.id &&
          (!f.from || m.date >= f.from) && (!f.to || m.date <= f.to));
        if (!hit) return false;
      }
      return true;
    });

    const { key, dir } = f.sort;
    const pick = ({ v, st }) => {
      switch (key) {
        case 'name':         return v.name;
        case 'regNumber':    return v.regNumber || null;
        case 'unit':         return v.unit || null;
        case 'responsible':  return v.responsible || null;
        case 'status':       return v.status;
        case 'currentHours': return Number(v.currentHours || 0);
        case 'pct':          return st.pct == null ? null : st.pct;
        case 'nextDate':     return st.nextDate || null;
        default:             return v.name;
      }
    };
    return rows.sort((a, b) => cmp(pick(a), pick(b), dir));
  }

  function renderPark() {
    const f = ui.park;
    const rows = parkRows();
    const all = DB.vehicles;

    const counts = { ok: 0, soon: 0, urgent: 0, overdue: 0, none: 0, retired: 0 };
    all.forEach(v => counts[computeStatus(v).level]++);
```

```
  const totalHours = all.reduce((s, v) => s + Number(v.currentHours || 0), 0);

  const demo = all.some(v => v.demo);

  return `
    ${demo ? `<div class="f-demo-banner">
      ${FI.alert(16)}
      <div>В базе присутствуют <b>демонстрационные записи</b> — они добавлены для наглядности индикации.</div>
      <button type="button" class="f-btn f-btn-sm" data-act="seed-remove">Удалить демо-данные</button>
    </div>` : ''}

    <div class="f-page-head">
      <div>
        <div class="f-page-title" role="heading" aria-level="1">Парк техники</div>
        <div class="f-page-sub">Электронные карточки технических средств тыла · всего ${all.length} ${plural(all.length, 'единица', 'единицы', 'единиц')}</div>
      </div>
      <div class="f-page-actions">
        <button type="button" class="f-btn" data-act="mode" data-mode="${f.mode === 'table' ? 'cards' : 'table'}">
          ${f.mode === 'table' ? FI.grid(15) + '<span>Карточки</span>' : FI.list(15) + '<span>Таблица</span>'}
        </button>
        <button type="button" class="f-btn f-btn-primary" data-act="add-vehicle">${FI.plus(15)}<span>Добавить технику</span></button>
      </div>
    </div>

    <div class="f-stats">
      <div class="f-stat"><div class="f-stat-label">Всего единиц</div><div class="f-stat-value">${all.length}</div></div>
      <div class="f-stat is-clickable" data-act="filter-level" data-level="ok">
        <div class="f-stat-label"><span class="f-dot f-lvl-ok"></span>Норма</div>
        <div class="f-stat-value" style="color:var(--st-ok-text)">${counts.ok}</div></div>
      <div class="f-stat is-clickable" data-act="filter-level" data-level="soon">
        <div class="f-stat-label"><span class="f-dot f-lvl-soon"></span>Скоро ТО</div>
        <div class="f-stat-value" style="color:var(--st-soon-text)">${counts.soon}</div></div>
      <div class="f-stat is-clickable" data-act="filter-level" data-level="urgent">
        <div class="f-stat-label"><span class="f-dot f-lvl-urgent"></span>Требуется ТО</div>
        <div class="f-stat-value" style="color:var(--st-urgent-text)">${counts.urgent}</div></div>
      <div class="f-stat is-clickable" data-act="filter-level" data-level="overdue">
        <div class="f-stat-label"><span class="f-dot f-lvl-overdue"></span>Просрочено</div>
        <div class="f-stat-value" style="color:var(--st-overdue-text)">${counts.overdue}</div></div>
      <div class="f-stat"><div class="f-stat-label">Суммарно моточасов</div>
        <div class="f-stat-value">${fmtNum(totalHours, 0)}<small>мото·ч</small></div></div>
    </div>

    <div class="f-toolbar">
      <div class="f-fld f-search">
        <label class="f-fld-label" for="parkQ">Поиск</label>
        <input id="parkQ" class="f-input" type="search" placeholder="Гос. номер, модель, наименование, инв. / зав. номер…" value="${esc(f.q)}">
      </div>
      <div class="f-fld"><label class="f-fld-label" for="parkUnit">Подразделение</label>
        <select id="parkUnit" class="f-select">${options(uniqueValues('unit'), f.unit, 'Все')}</select></div>
      <div class="f-fld"><label class="f-fld-label" for="parkResp">Ответственный</label>
        <select id="parkResp" class="f-select">${options(uniqueValues('responsible'), f.responsible, 'Все')}</select></div>
      <div class="f-fld"><label class="f-fld-label" for="parkStatus">Статус</label>
        <select id="parkStatus" class="f-select">${options(STATUSES, f.status, 'Любой')}</select></div>
      <div class="f-fld"><label class="f-fld-label" for="parkLevel">Состояние ТО</label>
        <select id="parkLevel" class="f-select">
          <option value="">Любое</option>
          ${Object.keys(LEVELS).map(k => `<option value="${k}"${f.level === k ? ' selected' : ''}>${LEVELS[k].emoji} ${esc(LEVELS[k].label)}</option>`).join('')}
        </select></div>
      <div class="f-fld"><label class="f-fld-label" for="parkFrom">ТО с даты</label>
        <input id="parkFrom" class="f-input" type="date" value="${esc(f.from)}"></div>
      <div class="f-fld"><label class="f-fld-label" for="parkTo">ТО по дату</label>
        <input id="parkTo" class="f-input" type="date" value="${esc(f.to)}"></div>
      <div class="f-toolbar-end">
        <button type="button" class="f-btn f-btn-ghost" data-act="reset-filters">Сбросить</button>
      </div>
    </div>

    <div id="parkResults">${renderParkResults(rows)}</div>`;
}

function renderParkResults(rows) {
  rows = rows || parkRows();
  const sel = ui.park.selected;

  if (!DB.vehicles.length) {
    return emptyState(FI.truck(28), 'Техника не зарегистрирована',
      'Добавьте первую единицу техники, чтобы начать вести учёт обслуживания и моточасов.',
      '<button type="button" class="f-btn f-btn-primary" data-act="add-vehicle">Добавить технику</button>');
  }
  if (!rows.length) {
    return emptyState(FI.truck(28), 'Ничего не найдено',
      'Измените условия поиска или сбросьте фильтры.',
      '<button type="button" class="f-btn" data-act="reset-filters">Сбросить фильтры</button>');
  }
```

```
    // Убираем из выбора id, которых больше нет среди видимых по фильтрам записей — только визуально не мешает,
    // сам Set чистится по факту удаления в removeVehicle()
    const bulkBar = sel.size ? `
      <div class="f-bulkbar">
        <span class="f-bulkbar-count">Выбрано: ${sel.size}</span>
        <div class="f-bulkbar-actions">
          <button type="button" class="f-btn f-btn-sm" data-act="bulk-status">${FI.edit(14)}<span>Статус</span></button>
          <button type="button" class="f-btn f-btn-sm" data-act="bulk-interval">${FI.gauge(14)}<span>Интервал ТО</span></button>
          <button type="button" class="f-btn f-btn-sm f-btn-danger" data-act="bulk-delete">${FI.trash(14)}<span>Удалить</span></button>
          <button type="button" class="f-btn f-btn-ghost f-btn-sm" data-act="clear-selection">Снять выделение</button>
        </div>
      </div>` : '';

    if (ui.park.mode === 'cards') {
      return bulkBar + `<div class="f-cards f-cards-staff">${rows.map(({ v, st }, i) => staffCard(v, st, i, sel.has(v.id))).join('')}</div>`;
    }

    const allSelected = rows.length > 0 && rows.every(({ v }) => sel.has(v.id));

    return bulkBar + `
      <div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table">
          <thead><tr>
            <th style="width:34px"><input type="checkbox" data-act="select-all" ${allSelected ? 'checked' : ''} title="Выбрать всё показанное" aria-label="Выбрать всё показанное"></th>
            <th style="width:34px"></th>
            ${th('Наименование', 'name', 'park')}
            ${th('Гос. номер', 'regNumber', 'park')}
            ${th('Подразделение', 'unit', 'park')}
            ${th('Ответственный', 'responsible', 'park')}
            ${th('Статус', 'status', 'park')}
            ${th('Моточасы', 'currentHours', 'park', { num: true })}
            ${th('Остаток ресурса', 'pct', 'park')}
            ${th('Следующее ТО', 'nextDate', 'park')}
            <th style="width:80px"></th>
          </tr></thead>
          <tbody>
            ${rows.map(({ v, st }) => `
              <tr class="is-clickable ${sel.has(v.id) ? 'is-selected' : ''}" data-act="open-vehicle" data-id="${v.id}">
                <td><input type="checkbox" data-act="row-select" data-id="${v.id}" aria-label="Выбрать: ${esc(v.name)}" ${sel.has(v.id) ? 'checked' : ''}></td>
                <td><span class="f-dot f-lvl-${st.level}" title="${esc(LEVELS[st.level].label)}"></span></td>
                <td class="f-td-strong">${vfThumbHtml(v, 'vf-mini')}${esc(v.name)}<div class="f-td-muted" style="font-size:12px;font-weight:400">${esc(v.brand || '')}</div></td>
                <td>${v.regNumber ? `<span class="f-plate">${esc(v.regNumber)}</span>` : '<span class="f-td-muted">—</span>'}</td>
                <td>${esc(v.unit || '—')}</td>
                <td>${esc(v.responsible || '—')}</td>
                <td><span class="f-badge f-badge-neutral">${esc(v.status)}</span></td>
                <td class="f-td-num">${fmtNum(v.currentHours, 0)}</td>
                <td>${resourceCell(st)}</td>
                <td class="f-td-nowrap">${st.nextDate ? fmtDate(st.nextDate) : (st.nextHours != null ? fmtNum(st.nextHours, 0) + ' м·ч' : '—')}</td>
                <td>
                  <div class="f-row-actions">
                    <button type="button" class="f-icon-btn" data-act="edit-vehicle" data-id="${v.id}" title="Редактировать">${FI.edit(15)}</button>
                    <button type="button" class="f-icon-btn is-danger" data-act="del-vehicle" data-id="${v.id}" title="Удалить">${FI.trash(15)}</button>
                  </div>
                </td>
              </tr>`).join('')}
          </tbody>
        </table>
      </div></div>
      <div style="margin-top:10px;font-size:12.5px;color:var(--text-secondary)">
        Показано ${rows.length} из ${DB.vehicles.length}
      </div>`;
  }

  /* Обновление только области результатов — фокус в поле поиска не теряется */
  function refreshPark() {
    const box = el('parkResults');
    if (box) box.innerHTML = renderParkResults();
  }

  function bindParkFilters() {
    const f = ui.park;
    const bind = (id, prop, ev) => {
      const node = el(id);
      if (!node) return;
      node.addEventListener(ev || 'change', () => { f[prop] = node.value; refreshPark(); });
    };
    let t = null;
    const q = el('parkQ');
    if (q) q.addEventListener('input', () => {
```

```
      clearTimeout(t);
      t = setTimeout(() => { f.q = q.value; refreshPark(); }, 140);
    });
    bind('parkUnit', 'unit');
    bind('parkResp', 'responsible');
    bind('parkStatus', 'status');
    bind('parkLevel', 'level');
    bind('parkFrom', 'from');
    bind('parkTo', 'to');
  }

  /* ========================================================================
     15. КАРТОЧКА ТЕХНИКИ
     ======================================================================== */
  function renderVehicle() {
    const v = getVehicle(ui.vehicleId);
    if (!v) { ui.view = 'park'; return renderPark(); }
    const st = computeStatus(v);
    const ms = maintOf(v.id);
    const hs = hoursOf(v.id);

    const nextText = st.nextDate
      ? fmtDate(st.nextDate)
      : (st.nextHours != null ? fmtNum(st.nextHours, 0) + ' мото·ч' : '—');

    return `
      <a class="f-back" data-act="back-park">${FI.back(15)} К списку техники</a>

      <div class="f-detail-head">
        <div style="min-width:0;display:flex;gap:14px;align-items:center">
        ${vfThumbId(v) ? `<button type="button" class="vf-cover" data-act="vf" data-do="open-files" title="Фото и документы">${vfThumbHtml(v, '')}</button>` : ''}
        <div style="min-width:0">
          <div class="f-detail-title">${esc(v.name)}</div>
          <div class="f-detail-sub">
            <span>${esc(v.brand || 'Марка не указана')}</span>
            ${v.regNumber ? `<span class="f-plate">${esc(v.regNumber)}</span>` : ''}
            ${catNodeOf(v) ? `<button type="button" class="f-link-chip" data-act="open-catalog" data-id="${esc(v.catalogId)}" title="${esc(catNodeOf(v).name)}">${FI.file(13)}<span>Регламент в справочнике</span></button>` : ''}
            <span class="f-badge f-badge-neutral">${esc(v.status)}</span>
            ${levelBadge(st.level)}
          </div>
        </div>
        </div>
        <div class="f-page-actions">
          <button type="button" class="f-btn" data-act="add-hours" data-id="${v.id}">${FI.gauge(15)}<span>Моточасы</span></button>
          <button type="button" class="f-btn" data-act="edit-vehicle" data-id="${v.id}">${FI.edit(15)}<span>Изменить</span></button>
          <button type="button" class="f-btn f-btn-primary" data-act="add-maint" data-id="${v.id}">${FI.plus(15)}<span>Запись ТО</span></button>
        </div>
      </div>

      <div class="f-hero">
        ${resourceRing(st)}
        <div class="f-hero-metrics">
          <div><div class="f-metric-label">Текущие моточасы</div>
            <div class="f-metric-value">${fmtNum(v.currentHours, 0)}<small>мото·ч</small></div></div>
          <div><div class="f-metric-label">С последнего ТО</div>
            <div class="f-metric-value">${fmtNum(st.hoursSince, 0)}<small>мото·ч</small></div></div>
          <div><div class="f-metric-label">Последнее ТО</div>
            <div class="f-metric-value" style="font-size:16px">${st.lastMaint ? fmtDate(st.lastMaint.date) : '—'}</div></div>
          <div><div class="f-metric-label">Следующее ТО</div>
            <div class="f-metric-value" style="font-size:16px">${st.level === 'retired' || st.level === 'none' ? esc(nextText) : esc(nextMaintKind(v) + ' ' + dueText(st))}</div></div>
          <div><div class="f-metric-label">Осталось моточасов</div>
            <div class="f-metric-value" ${st.remainingHours != null && st.remainingHours <= 0 ? 'style="color:var(--st-overdue-text)"' : ''}>
              ${st.remainingHours != null ? fmtNum(st.remainingHours, 0) : '—'}</div></div>
          <div><div class="f-metric-label">Осталось дней</div>
            <div class="f-metric-value" ${st.remainingDays != null && st.remainingDays <= 0 ? 'style="color:var(--st-overdue-text)"' : ''}>
              ${st.remainingDays != null ? st.remainingDays : '—'}</div></div>
        </div>
      </div>

      <div class="f-tabs">
        <button type="button" class="f-tab ${ui.vehicleTab === 'info'  ? 'is-active' : ''}" data-act="vtab" data-tab="info">Основные сведения</button>
        <button type="button" class="f-tab ${ui.vehicleTab === 'maint' ? 'is-active' : ''}" data-act="vtab" data-tab="maint">Журнал ТО (${ms.length})</button>
        <button type="button" class="f-tab ${ui.vehicleTab === 'hours' ? 'is-active' : ''}" data-act="vtab" data-tab="hours">Моточасы (${hs.length})</button>
        <button type="button" class="f-tab ${ui.vehicleTab === 'files' ? 'is-active' : ''}" data-act="vtab" data-tab="files">Фото и документы (${vfPhotos(v).length + vfDocs(v).length})</button>
      </div>

```

```
      <div id="vehTabBody">${renderVehicleTab(v, st, ms, hs)}</div>`;
  }

  function renderVehicleTab(v, st, ms, hs) {
    if (ui.vehicleTab === 'info')  return renderVehicleInfo(v, st);
    if (ui.vehicleTab === 'files') return renderVehicleFiles(v);
    if (ui.vehicleTab === 'maint') return renderVehicleMaint(v, ms);
    return renderVehicleHours(v, hs);
  }

  function refreshVehicleTab() {
    const v = getVehicle(ui.vehicleId);
    if (!v) return;
    const box = el('vehTabBody');
    if (box) box.innerHTML = renderVehicleTab(v, computeStatus(v), maintOf(v.id), hoursOf(v.id));
  }

  function renderVehicleInfo(v, st) {
    const iv = v.interval || {};
    const rules = [];
    if (iv.byHours && iv.hours) rules.push(`каждые ${fmtNum(iv.hours, 0)} мото·ч`);
    if (iv.byDate && iv.days)   rules.push(`каждые ${iv.days} ${plural(iv.days, 'день', 'дня', 'дней')}`);
    const t2 = to2Intervals(iv);
    if (rules.length) rules.push(`ТО-2: ${[iv.byHours && t2.hours ? fmtNum(t2.hours, 0) + ' мото·ч' : '', iv.byDate &&
  t2.days ? t2.days + ' дн.' : ''].filter(Boolean).join(' / ')}`);

    const F = (k, val) => `<div><div class="f-info-k">${esc(k)}</div><div class="f-info-v">${esc(val ||
  '—')}</div></div>`;

    return `
      <div class="f-panel">
        <div class="f-panel-title">Основные сведения</div>
        <div class="f-info-grid">
          ${F('Наименование техники', v.name)}
          ${F('Марка и модель', v.brand)}
          ${F('Государственный номер', v.regNumber)}
          ${F('Инвентарный номер', v.invNumber)}
          ${F('Заводской номер', v.serialNumber)}
          ${F('Подразделение', v.unit)}
          ${F('Ответственное лицо', v.responsible)}
          ${F('Год выпуска', v.year)}
          ${F('Дата постановки на учёт', v.regDate ? fmtDate(v.regDate) : '')}
          ${F('Статус', v.status)}
          ${F('Группа эксплуатации', v.expGroup)}
          ${F('Плановая годовая наработка', v.annualPlanHours !== '' && v.annualPlanHours != null ?
  fmtNum(v.annualPlanHours, 0) + ' мото·ч' : '')}
          ${F('Годовая норма расхода ресурса', v.annualNormHours !== '' && v.annualNormHours != null ?
  fmtNum(v.annualNormHours, 0) + ' мото·ч' : '')}
          ${F('Периодичность ТО', rules.length ? rules.slice(0, -1).join(' или ') + '; ' + rules[rules.length - 1] : 'не
   задана')}
          ${F('Моточасы обновлены', v.hoursUpdatedAt ? fmtDate(v.hoursUpdatedAt) : '')}
        </div>
        ${v.note ? `<div style="margin-top:18px"><div class="f-info-k">Примечание</div>
          <div class="f-info-v" style="white-space:pre-wrap">${esc(v.note)}</div></div>` : ''}
      </div>

      ${hoursStale(v) ? `<div class="f-demo-banner">${FI.gauge(16)}
        <div>Моточасы не обновлялись более ${DB.settings.hoursStaleDays} ${plural(DB.settings.hoursStaleDays, 'дня',
  'дней', 'дней')}. Расчёт ресурса может быть неточным.</div>
        <button type="button" class="f-btn f-btn-sm" data-act="add-hours" data-id="${v.id}">Обновить</button></div>` :
  ''}

      ${renderVehicleQr(v)}

      ${renderVehicleCatalog(v)}

      ${renderVehicleHistory(v)}

      <div class="f-panel">
        <div class="f-panel-title">Расчёт ресурса до следующего ТО</div>
        <div class="f-info-grid">
          ${st.kinds.map(k => F(`${k.kind}${k.kind === st.nextKind ? ' — очередное' : ''}`,
              k.unknown ? 'нет записи о последнем ТО-2 — внесите её в журнал (вид «ТО-2»), чтобы рассчитать срок' :
              dueText(k) + (k.nextHours != null ? ` · при ${fmtNum(k.nextHours, 0)} мото·ч` : '') + (k.nextDate ? ` · до
   ${fmtDate(k.nextDate)}` : '') +
              ` (отсчёт: ${k.lastMaint ? `${k.lastMaint.kind} от ${fmtDate(k.lastMaint.date)}` : 'постановка на
  учёт'})`)).join('')}
          ${F('Точка отсчёта', st.lastMaint ? `ТО от ${fmtDate(st.lastMaint.date)} (${st.lastMaint.kind})` : 'постановка
   на учёт')}
          ${F('Моточасы на момент отсчёта', fmtNum(st.baseHours, 0))}
          ${F('Наработка с последнего ТО', fmtNum(st.hoursSince, 0) + ' мото·ч')}
          ${F('Состояние', LEVELS[st.level].emoji + ' ' + LEVELS[st.level].label)}
        </div>
      </div>`;
  }

  /* Связь со справочником: нормативы ТО для этого типа техники + когда
     каждый вид ТО последний раз проводился по журналу этой единицы */
  function renderVehicleCatalog(v) {
    const c = CAT();
```

```
  if (!c) return '';
  const node = catNodeOf(v);
  if (!node) {
    return `<div class="f-panel">
      <div class="f-panel-title">Регламент ТО по справочнику</div>
      <p class="f-hint" style="margin-bottom:12px">Карточка не привязана к позиции справочника. После привязки здесь появятся нормативы по каждому виду ТО, а запись обслуживания можно будет заполнить перечнем работ и материалов из регламента.</p>
      <button type="button" class="f-btn f-btn-sm" data-act="edit-vehicle" data-id="${v.id}">${FI.edit(14)}<span>Привязать к справочнику</span></button>
    </div>`;
  }

  const maint = catMaintOf(v) || {};
  const custom = !c.hasBuiltIn(node.id);
  const ms = maintOf(v.id);
  const next = v.status === 'Списано' ? '' : nextMaintKind(v);
  const rows = c.types.filter(t => maint[t.key]).map(t => {
    const m = maint[t.key];
    const kind = CATALOG_TO_KIND[t.key];
    const last = kind ? ms.find(x => x.kind === kind) : null;
    const people = (m.personnel || []).reduce((s, p) => s + (Number(p.count) || 0), 0);
    return `<tr class="${kind === next ? 'is-next' : ''}">
      <td class="f-td-strong f-td-nowrap" title="${esc(t.full)}">${esc(t.short)}${kind === next ? ' <span class="f-badge f-badge-neutral">следующее</span>' : ''}</td>
      <td>${esc(t.full)}</td>
      <td class="f-td-num">${(m.actions || []).length}</td>
      <td class="f-td-num">${fmtNum(m.labor, 1)}</td>
      <td class="f-td-num">${people}</td>
      <td class="f-td-nowrap">${last ? fmtDate(last.date) : '<span class="f-td-muted">не проводилось</span>'}</td>
      <td><div class="f-row-actions">
        <button type="button" class="f-btn f-btn-sm" data-act="open-catalog" data-id="${esc(node.id)}" data-tab="${t.key}">Карта ТО</button>
        <button type="button" class="f-icon-btn" data-act="print-checklist" data-id="${v.id}" data-cat="${esc(node.id)}" data-tab="${t.key}" title="Печать карты ТО с отметками о выполнении">${FI.pdf(15)}</button>
        ${kind ? `<button type="button" class="f-icon-btn" data-act="maint-from-norm" data-id="${v.id}" data-tab="${t.key}" title="Внести в журнал: запись ${esc(kind)} с перечнем работ и материалов">${FI.plus(15)}</button>` : ''}
      </div></td>
    </tr>`;
  }).join('');

  return `<div class="f-panel">
    <div class="f-reg-head">
      <div style="min-width:0">
        <div class="f-panel-title" style="margin-bottom:0">Регламент ТО по справочнику</div>
        <div class="f-reg-path">${esc(c.path(node.id).concat(node.name).join(' › '))}${custom && rows ? ' · <b>норматив пользователя</b>' : ''}</div>
      </div>
      <button type="button" class="f-btn f-btn-sm" data-act="open-catalog" data-id="${esc(node.id)}">${FI.file(14)}<span>Открыть в справочнике</span></button>
    </div>
    ${rows ? `<div class="f-table-wrap"><div class="f-table-scroll">
      <table class="f-table f-reg-table">
        <thead><tr>
          <th>Вид</th><th>Наименование</th><th class="f-td-num">Операций</th>
          <th class="f-td-num">Чел·ч</th><th class="f-td-num">Персонал</th><th>Последнее по журналу</th><th></th>
        </tr></thead>
        <tbody>${rows}</tbody>
      </table>
    </div></div>
    <div class="f-hint" style="margin-top:8px">Следующее плановое ТО — ${esc(next || '—')}${next && computeStatus(v).pct != null ? ' (' + esc(dueText(computeStatus(v))) + ')' : ''}. Сроки ТО-1 и ТО-2 считаются отдельно. Кнопки строки: карта в справочнике, печать чек-листа, запись в журнал с перечнем из норматива.</div>`
      : `<p class="f-hint" style="margin-bottom:12px">Регламент ТО для этой позиции в учебном пособии не приведён. Можно ввести свой норматив — по руководству по эксплуатации.</p>
        <button type="button" class="f-btn f-btn-sm" data-act="edit-norm" data-id="${esc(node.id)}">${FI.plus(14)}<span>Ввести норматив</span></button>`}
  </div>`;
}

function renderVehicleMaint(v, ms) {
  if (!ms.length) {
    return `<div class="f-panel">` + emptyState(FI.wrench(28), 'Записей ТО пока нет',
      'Добавьте первую запись технического обслуживания — программа рассчитает срок следующего ТО.',
      `<button type="button" class="f-btn f-btn-primary" data-act="add-maint" data-id="${v.id}">Добавить запись ТО</button>`) + `</div>`;
  }

  return `
    <div class="f-table-wrap"><div class="f-table-scroll">
      <table class="f-table">
        <thead><tr>
          <th>Вид ТО</th><th>Дата</th><th class="f-td-num">Моточасы</th>
          <th>Исполнитель</th><th>Подразделение</th><th>Работы</th>
          <th>Запчасти</th><th class="f-td-num">Стоимость</th><th>Документы</th><th style="width:80px"></th>
        </tr></thead>
        <tbody>
          ${ms.map(m => `
            <tr>
```

```
                <td class="f-td-strong f-td-nowrap">
                  ${RESET_KINDS.has(m.kind) ? '<span class="f-dot f-lvl-ok" title="Сбрасывает межсервисный интервал"></span> ' : ''}
                  ${esc(m.kind)}${m.season && SEASONS[m.season] ? ' · ' + esc(SEASONS[m.season].label) : ''}
                </td>
                <td class="f-td-nowrap">${fmtDate(m.date)}</td>
                <td class="f-td-num">${fmtNum(m.hours, 0)}</td>
                <td>${esc(m.executor || '—')}</td>
                <td>${esc(m.unit || '—')}</td>
                <td style="max-width:240px">${m.works ? esc(m.works) : '<span class="f-td-muted">—</span>'}${(() => { const s = checklistSummary(m.checklist); return s.total ? `<div class="mc-badge ${s.done === s.total ? 'is-full' : ''}" title="${esc(m.checklist.items.filter(x => !x.done).map(x => x.t).join('; ') || 'Все операции выполнены')}">По регламенту: ${s.done} из ${s.total}</div>` : ''; })()}</td>
                <td style="max-width:200px">${m.parts ? esc(m.parts) : '<span class="f-td-muted">—</span>'}</td>
                <td class="f-td-num">${m.cost ? fmtMoney(m.cost) : '—'}</td>
                <td>${(m.docs && m.docs.length)
                      ? m.docs.map(d => `<button type="button" class="f-icon-btn" data-act="download-file" data-id="${d.id}" title="${esc(d.name)}">${FI.doc(15)}</button>`).join('')
                      : '<span class="f-td-muted">—</span>'}</td>
                <td><div class="f-row-actions">
                  <button type="button" class="f-icon-btn" data-act="edit-maint" data-id="${m.id}" title="Редактировать">${FI.edit(15)}</button>
                  <button type="button" class="f-icon-btn is-danger" data-act="del-maint" data-id="${m.id}" title="Удалить">${FI.trash(15)}</button>
                </div></td>
              </tr>
              ${m.note ? `<tr><td colspan="10" style="padding-top:0;color:var(--text-secondary);font-size:12.5px">
                <b>Примечание:</b> ${esc(m.note)}</td></tr>` : ''}
            `).join('')}
          </tbody>
        </table>
      </div></div>
      <div style="margin-top:12px;display:flex;gap:8px">
        <button type="button" class="f-btn f-btn-primary" data-act="add-maint" data-id="${v.id}">${FI.plus(15)}<span>Добавить запись ТО</span></button>
      </div>`;
  }

  function renderVehicleHours(v, hs) {
    if (!hs.length) {
      return `<div class="f-panel">` + emptyState(FI.gauge(28), 'История моточасов пуста',
        'Внесите текущие показания счётчика моточасов — программа сохранит историю изменений.',
        `<button type="button" class="f-btn f-btn-primary" data-act="add-hours" data-id="${v.id}">Внести моточасы</button>`) + `</div>`;
    }

    return `
      <div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table">
          <thead><tr>
            <th>Дата</th><th class="f-td-num">Показание</th><th class="f-td-num">Прирост</th>
            <th>Источник</th><th>Примечание</th><th style="width:44px"></th>
          </tr></thead>
          <tbody>
            ${hs.map(h => `
              <tr>
                <td class="f-td-nowrap">${fmtDate(h.date)}</td>
                <td class="f-td-num f-td-strong">${fmtNum(h.value, 0)}</td>
                <td class="f-td-num" style="color:${h.delta > 0 ? 'var(--st-ok-text)' : 'var(--text-secondary)'}">
                  ${h.delta > 0 ? '+' + fmtNum(h.delta, 0) : (h.delta === 0 ? '0' : fmtNum(h.delta, 0))}</td>
                <td><span class="f-badge f-badge-neutral">${h.source === 'maint' ? 'запись ТО' : (h.source === 'trip' ? 'путевой лист' : 'вручную')}</span></td>
                <td>${h.note ? esc(h.note) : '<span class="f-td-muted">—</span>'}</td>
                <td><button type="button" class="f-icon-btn is-danger" data-act="del-hours" data-id="${h.id}" title="Удалить">${FI.trash(15)}</button></td>
              </tr>`).join('')}
          </tbody>
        </table>
      </div></div>
      <div style="margin-top:12px">
        <button type="button" class="f-btn f-btn-primary" data-act="add-hours" data-id="${v.id}">${FI.plus(15)}<span>Внести моточасы</span></button>
      </div>`;
  }

  /* ============================================================================
     16. МУТАЦИИ ДАННЫХ (единая точка изменения состояния)
     ============================================================================ */

  /* Пересчёт приростов моточасов по всей истории единицы техники */
  function recomputeHours(vehicleId, db) {
    db = db || DB;
    const asc = db.hours
      .filter(h => h.vehicleId === vehicleId)
      .sort((a, b) => (a.date || '').localeCompare(b.date || '') ||
                      (a.createdAt || '').localeCompare(b.createdAt || ''));

    let prev = 0;
    asc.forEach(h => {
      h.delta = Number(h.value) - prev;
```

```
        prev = Number(h.value);
      });

      // Текущие моточасы = последнее по дате показание
      const v = db.vehicles.find(x => x.id === vehicleId);
      if (v && asc.length) {
        const last = asc[asc.length - 1];
        v.currentHours = Number(last.value);
        v.hoursUpdatedAt = last.date;
      }
    }

    /* Запись ТО фиксирует показание счётчика — синхронизируем историю моточасов */
    function syncMaintHours(m) {
      let entry = DB.hours.find(h => h.maintId === m.id);
      if (!entry) {
        entry = { id: uid('h'), vehicleId: m.vehicleId, maintId: m.id,
                  source: 'maint', createdAt: new Date().toISOString() };
        DB.hours.push(entry);
      }
      entry.date  = m.date;
      entry.value = Number(m.hours || 0);
      entry.note  = 'Показание при проведении: ' + m.kind;
    }

    /* docIds — если передан массив, id вложений собираются в него (удаление
       файлов откладывается до истечения срока отмены), иначе стираются сразу */
    const dropFiles = (docs, docIds) => (docs || []).forEach(d => (docIds ? docIds.push(d.id) : fileDel(d.id)));

    function removeMaint(m, docIds) {
      DB.maint = DB.maint.filter(x => x.id !== m.id);
      DB.hours = DB.hours.filter(h => h.maintId !== m.id);
      dropFiles(m.docs, docIds);
      recomputeHours(m.vehicleId);
      saveDB();
    }

    /* Путевой лист фиксирует показание счётчика на конец выхода — синхронизируем
       историю моточасов, как и при записи ТО (syncMaintHours). */
    function syncTripHours(t) {
      let entry = DB.hours.find(h => h.tripId === t.id);
      if (!entry) {
        entry = { id: uid('h'), vehicleId: t.vehicleId, tripId: t.id,
                  source: 'trip', createdAt: new Date().toISOString() };
        DB.hours.push(entry);
      }
      // При смене техники в путевом листе показание должно уйти вместе с ним
      entry.vehicleId = t.vehicleId;
      entry.date  = t.date;
      entry.value = Number(t.endHours || 0);
      entry.note  = 'Путевой лист' + (t.driver ? ' · ' + t.driver : '') + (t.purpose ? ' — ' + t.purpose : '');
    }

    function removeTrip(t) {
      DB.trips = DB.trips.filter(x => x.id !== t.id);
      DB.hours = DB.hours.filter(h => h.tripId !== t.id);
      recomputeHours(t.vehicleId);
      saveDB();
    }

    function removeVehicle(v, docIds) {
      DB.maint.filter(m => m.vehicleId === v.id)
              .forEach(m => dropFiles(m.docs, docIds));
      vfFileIds(v).forEach(id => (docIds ? docIds.push(id) : fileDel(id)));
      DB.maint = DB.maint.filter(m => m.vehicleId !== v.id);
      DB.hours = DB.hours.filter(h => h.vehicleId !== v.id);
      DB.trips = DB.trips.filter(t => t.vehicleId !== v.id);
      DB.vehicles = DB.vehicles.filter(x => x.id !== v.id);
      ui.park.selected.delete(v.id);
      // Фильтр путевых листов не должен указывать на удалённую технику
      if (ui.trips.vehicle === v.id) ui.trips.vehicle = '';
      saveDB();
    }

    /* ========================================================================
       17. РАЗДЕЛ «БЛИЖАЙШИЕ ТЕХНИЧЕСКИЕ ОБСЛУЖИВАНИЯ» (панель контроля)
       ======================================================================== */
    function dashRows() {
      let rows = DB.vehicles
        .filter(v => v.status !== 'Списано')
        .map(v => ({ v, st: computeStatus(v) }))
        .filter(({ st }) => st.level === 'overdue' || st.level === 'urgent' || st.level === 'soon');

      const { key, dir } = ui.dash.sort;
      const pick = ({ v, st }) => {
        switch (key) {
          case 'unit':        return v.unit || null;
          case 'responsible': return v.responsible || null;
          case 'nextDate':    return st.nextDate || null;
          case 'hours':       return st.remainingHours == null ? null : st.remainingHours;
```

```
        case 'days':        return st.remainingDays == null ? null : st.remainingDays;
        case 'name':        return v.name;
        default:            return st.pct == null ? null : st.pct;
      }
    };
    return rows.sort((a, b) => cmp(pick(a), pick(b), dir));
  }

  function renderDashboard() {
    const rows = dashRows();
    const active = DB.vehicles.filter(v => v.status !== 'Списано');
    const c = { overdue: 0, urgent: 0, soon: 0 };
    active.forEach(v => { const l = computeStatus(v).level; if (c[l] != null) c[l]++; });
    const stale = active.filter(hoursStale).length;

    return `
      <div class="f-page-head">
        <div>
          <div class="f-page-title" role="heading" aria-level="1">Ближайшие технические обслуживания</div>
          <div class="f-page-sub">Техника, требующая внимания: просроченные и приближающиеся сроки ТО</div>
        </div>
      </div>

      <div class="f-stats">
        <div class="f-stat"><div class="f-stat-label"><span class="f-dot f-lvl-overdue"></span>Просрочено</div>
          <div class="f-stat-value" style="color:var(--st-overdue-text)">${c.overdue}</div></div>
        <div class="f-stat"><div class="f-stat-label"><span class="f-dot f-lvl-urgent"></span>Требуется ТО</div>
          <div class="f-stat-value" style="color:var(--st-urgent-text)">${c.urgent}</div></div>
        <div class="f-stat"><div class="f-stat-label"><span class="f-dot f-lvl-soon"></span>Скоро ТО</div>
          <div class="f-stat-value" style="color:var(--st-soon-text)">${c.soon}</div></div>
        <div class="f-stat"><div class="f-stat-label">Обновить моточасы</div>
          <div class="f-stat-value">${stale}</div></div>
      </div>

      <div id="dashResults">${renderDashResults(rows)}</div>`;
  }

  function renderDashResults(rows) {
    rows = rows || dashRows();
    if (!rows.length) {
      return emptyState(FI.wrench(28), 'Всё обслужено в срок',
        'Нет техники с приближающимся или просроченным сроком технического обслуживания.');
    }

    return `
      <div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table f-table-fit">
          <thead><tr>
            <th style="width:30px"></th>
            ${th('Техника', 'name', 'dash')}
            ${th('Подразделение / ответственный', 'unit', 'dash')}
            ${th('Плановая дата ТО', 'nextDate', 'dash')}
            ${th('Состояние и остаток ресурса', 'hours', 'dash')}
            <th style="width:44px"><span class="sr-only">Действия</span></th>
          </tr></thead>
          <tbody>
            ${rows.map(({ v, st }) => `
              <tr class="is-clickable" data-act="open-vehicle" data-id="${v.id}">
                <td><span class="f-dot f-lvl-${st.level}"></span></td>
                <td><div class="f-td-strong">${esc(v.name)}</div>
                  ${v.regNumber ? `<span class="f-plate f-fit-sub">${esc(v.regNumber)}</span>` : ''}</td>
                <td><div>${esc(v.unit || '—')}</div>
                  <div class="f-td-muted f-fit-sub">${esc(v.responsible || '—')}</div></td>
                <td class="f-td-nowrap">${st.nextDate ? fmtDate(st.nextDate) : '—'}</td>
                <td><div class="f-fit-state">${levelBadge(st.level)}${resourceCell(st)}</div></td>
                <td><button type="button" class="f-icon-btn" data-act="add-maint" data-id="${v.id}" title="Провести ТО"
aria-label="Провести ТО">${FI.wrench(15)}</button></td>
              </tr>`).join('')}
          </tbody>
        </table>
      </div></div>`;
  }

  function refreshDash() {
    const box = el('dashResults');
    if (box) box.innerHTML = renderDashResults();
  }

  /* ========================================================================
     17а. РАЗДЕЛ «ПУТЕВЫЕ ЛИСТЫ»
     ------------------------------------------------------------------------
     Ежедневный учёт выхода техники (кто, куда, показание счётчика на начало
     и конец выхода). Методика ВАМТО (2025), гл. 3: наряд на использование
     машин и путевой лист — ежедневные документы, на основании которых
     ведётся карточка учёта работы машины и сводная ведомость за месяц.
     Каждый путевой лист синхронизирован с историей моточасов (syncTripHours),
     поэтому расход ресурса из путевых листов автоматически учитывается в
     расчёте срока ТО, наличия моторесурса и потребности в ТО.
     ======================================================================== */
```

```
function tripRows() {
  const f = ui.trips;
  return DB.trips
    .filter(t => !f.vehicle || t.vehicleId === f.vehicle)
    .filter(t => (!f.from || t.date >= f.from) && (!f.to || t.date <= f.to))
    .sort((a, b) => (b.date || '').localeCompare(a.date || '') || (b.createdAt || '').localeCompare(a.createdAt || ''));
}

function renderTrips() {
  const rows = tripRows();

  return `
    <div class="f-page-head">
      <div>
        <div class="f-page-title" role="heading" aria-level="1">Путевые листы</div>
        <div class="f-page-sub">Ежедневный учёт выхода техники — основа карточки учёта работы машины</div>
      </div>
      <div class="f-page-actions">
        <button type="button" class="f-btn f-btn-primary" data-act="add-trip">${FI.plus(15)}<span>Путевой лист</span></button>
      </div>
    </div>

    <div class="f-toolbar">
      <div class="f-fld"><label class="f-fld-label" for="tripVehicle">Техника</label>
        <select id="tripVehicle" class="f-select"></select>
      </div>
      <div class="f-fld"><label class="f-fld-label" for="tripFrom">Период с</label>
        <input id="tripFrom" class="f-input" type="date" value="${esc(ui.trips.from)}"></div>
      <div class="f-fld"><label class="f-fld-label" for="tripTo">Период по</label>
        <input id="tripTo" class="f-input" type="date" value="${esc(ui.trips.to)}"></div>
      <div class="f-toolbar-end">
        <button type="button" class="f-btn f-btn-ghost" data-act="reset-trips">Сбросить</button>
      </div>
    </div>

    <div id="tripResults">${renderTripResults(rows)}</div>`;
}

function renderTripResults(rows) {
  rows = rows || tripRows();
  if (!rows.length) {
    return `<div class="f-panel">` + emptyState(FI.route(28), 'Путевых листов пока нет',
      'Добавьте первый путевой лист — показания на конец выхода будут учтены в истории моточасов.',
      `<button type="button" class="f-btn f-btn-primary" data-act="add-trip">Добавить путевой лист</button>`) + `</div>`;
  }

  // Журнал может накопить тысячи записей: показываем страницу и подсказываем
  // сузить период, иначе отрисовка всей таблицы заметно тормозит интерфейс
  const LIMIT = 200;
  const shown = rows.slice(0, LIMIT);
  // Поиск техники по id через Map — линейный find на каждую строку давал
  // квадратичную сложность на больших журналах
  const vmap = new Map(DB.vehicles.map(v => [v.id, v]));

  return `
    ${rows.length > LIMIT ? `<div class="f-hint" style="margin-bottom:10px">Найдено записей: ${rows.length}. Показаны первые ${LIMIT} — уточните технику или период, чтобы увидеть остальные.</div>` : ''}
    <div class="f-table-wrap"><div class="f-table-scroll">
      <table class="f-table">
        <thead><tr>
          <th>Дата</th><th>Техника</th><th>Водитель</th><th>Маршрут / задача</th>
          <th class="f-td-num">Показание на начало</th><th class="f-td-num">Показание на конец</th>
          <th class="f-td-num">Наработка за выход</th><th style="width:80px"></th>
        </tr></thead>
        <tbody>
          ${shown.map(t => {
            const v = vmap.get(t.vehicleId);
            const dist = (t.endHours != null && t.startHours != null) ? Number(t.endHours) - Number(t.startHours) : null;
            return `
            <tr>
              <td class="f-td-nowrap">${fmtDate(t.date)}</td>
              <td class="f-td-strong">${v ? esc(v.name) : '<span class="f-td-muted">техника удалена</span>'}${v && v.regNumber ? ' <span class="f-plate" style="margin-left:6px">' + esc(v.regNumber) + '</span>' : ''}</td>
              <td>${esc(t.driver || '—')}</td>
              <td style="max-width:260px">${t.purpose ? esc(t.purpose) : '<span class="f-td-muted">—</span>'}</td>
              <td class="f-td-num">${fmtNum(t.startHours, 0)}</td>
              <td class="f-td-num">${fmtNum(t.endHours, 0)}</td>
              <td class="f-td-num f-td-strong">${dist != null ? fmtNum(dist, 0) : '—'}</td>
              <td><div class="f-row-actions">
                <button type="button" class="f-icon-btn" data-act="edit-trip" data-id="${t.id}" title="Редактировать">${FI.edit(15)}</button>
                <button type="button" class="f-icon-btn is-danger" data-act="del-trip" data-id="${t.id}" title="Удалить">${FI.trash(15)}</button>
              </div></td>
            </tr>`;
          }).join('')}
```

```
          </tbody>
        </table>
      </div></div>`;
  }

  function refreshTrips() {
    const box = el('tripResults');
    if (box) box.innerHTML = renderTripResults();
  }

  function bindTripFilters() {
    // Фильтр мог остаться от техники, удалённой или отсутствующей в копии
    if (ui.trips.vehicle && !getVehicle(ui.trips.vehicle)) ui.trips.vehicle = '';
    const sel = el('tripVehicle');
    if (sel) {
      sel.innerHTML = `<option value="">Все</option>` +
        DB.vehicles.map(v => `<option value="${esc(v.id)}"${v.id === ui.trips.vehicle ? ' selected' :
  ''}>${esc(v.name)}${v.regNumber ? ' · ' + esc(v.regNumber) : ''}</option>`).join('');
      sel.addEventListener('change', () => { ui.trips.vehicle = sel.value; refreshTrips(); });
    }
    const bind = (id, prop) => {
      const n = el(id);
      if (n) n.addEventListener('change', () => { ui.trips[prop] = n.value; refreshTrips(); });
    };
    bind('tripFrom', 'from');
    bind('tripTo', 'to');
  }

  /* Форма путевого листа */
  function tripModal(trip) {
    const isNew = !trip;
    const active = DB.vehicles.filter(v => v.status !== 'Списано');
    const t = trip || {
      vehicleId: active[0] ? active[0].id : '', date: todayISO(), driver: '', purpose: '',
      startHours: '', endHours: '', note: ''
    };
    const vForStart = getVehicle(t.vehicleId);

    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">${isNew ? 'Новый путевой лист' : 'Редактирование путевого листа'}</div>
          <div class="f-modal-sub">Ежедневный учёт выхода техники</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <div class="f-form-grid">
          <div><label class="f-fld-label" for="tVeh">Техника *</label>
            <select id="tVeh" class="f-select">${active.map(v => `<option value="${esc(v.id)}"${v.id === t.vehicleId ? '
   selected' : ''}>${esc(v.name)}${v.regNumber ? ' · ' + esc(v.regNumber) : ''}</option>`).join('')}</select></div>
          <div><label class="f-fld-label" for="tDate">Дата выхода *</label>
            <input id="tDate" class="f-input" type="date" value="${esc(t.date)}" max="${todayISO()}">
            <div class="f-error" id="errTDate">Укажите корректную дату (не позднее сегодняшней)</div></div>
          <div><label class="f-fld-label" for="tDriver">Водитель (расчёт)</label>
            <input id="tDriver" class="f-input" value="${esc(t.driver)}" placeholder="ефрейтор Сидоров С.С."></div>
          <div><label class="f-fld-label" for="tPurpose">Маршрут / задача</label>
            <input id="tPurpose" class="f-input" value="${esc(t.purpose)}" placeholder="Подвоз горячей пищи на
  полигон"></div>
          <div><label class="f-fld-label" for="tStart">Показание на начало выхода *</label>
            <input id="tStart" class="f-input" type="number" min="0" step="0.1" value="${t.startHours !== '' ?
  t.startHours : (vForStart ? Number(vForStart.currentHours || 0) : 0)}">
            <div class="f-error" id="errStart">Укажите значение от 0 до 10 000 000 мото·ч</div></div>
          <div><label class="f-fld-label" for="tEnd">Показание на конец выхода *</label>
            <input id="tEnd" class="f-input" type="number" min="0" step="0.1" value="${t.endHours}">
            <div class="f-error" id="errEnd">Должно быть не меньше показания на начало</div></div>
          <div class="f-span2"><label class="f-fld-label" for="tNote">Примечание</label>
            <input id="tNote" class="f-input" value="${esc(t.note)}"></div>
        </div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary" id="tSave">${FI.save(15)}<span>Сохранить</span></button>
      </div>`);

    const R = m.root;
    const q = id => R.querySelector('#' + id);

    q('tVeh').addEventListener('change', () => {
      if (!isNew) return;
      const veh = getVehicle(q('tVeh').value);
      if (veh) q('tStart').value = Number(veh.currentHours || 0);
    });

    q('tSave').addEventListener('click', () => {
      const vehicleId = q('tVeh').value;
      const date = q('tDate').value;
      const start = parseFloat(String(q('tStart').value).replace(',', '.'));
      const end   = parseFloat(String(q('tEnd').value).replace(',', '.'));
```

```
      let bad = false;
      const setErr = (inputId, errId, cond) => {
        q(inputId).classList.toggle('is-invalid', cond);
        q(errId).classList.toggle('is-visible', cond);
        if (cond) bad = true;
      };
      setErr('tStart', 'errStart', !validHours(start));
      setErr('tEnd', 'errEnd', !validHours(end) || end < start);
      setErr('tDate', 'errTDate', !date || date > todayISO());
      if (!vehicleId) { toast('warning', 'Не выбрана техника', 'Укажите единицу техники для путевого листа.'); bad = true; }
      if (bad) return;

      const payload = {
        vehicleId, date, driver: q('tDriver').value.trim(), purpose: q('tPurpose').value.trim(),
        startHours: start, endHours: end, note: q('tNote').value.trim()
      };

      // При редактировании технику могли сменить — пересчитать нужно обе карточки
      const prevVehicleId = isNew ? null : trip.vehicleId;

      if (isNew) {
        const nt = Object.assign({ id: uid('t'), createdAt: new Date().toISOString() }, payload);
        DB.trips.push(nt);
        syncTripHours(nt);
      } else {
        Object.assign(trip, payload, { updatedAt: new Date().toISOString() });
        syncTripHours(trip);
      }

      if (prevVehicleId && prevVehicleId !== vehicleId) recomputeHours(prevVehicleId);
      recomputeHours(vehicleId);
      saveDB();
      m.close();
      toast('success', isNew ? 'Путевой лист добавлен' : 'Путевой лист обновлён', `Наработка за выход: ${fmtNum(end - start, 0)} мото·ч`);
      rerender();
    });
  }

  /* ========================================================================
     17b. РАЗДЕЛ «ПЛАНИРОВАНИЕ»
     ------------------------------------------------------------------------
     Расчётные методики из учебного пособия «Методика планирования
     эксплуатации и ремонта ВВСТ» (ВАМТО, 2025), гл. 3: группы эксплуатации
     и КТГ, наличие моторесурса, потребность в ТО и трудозатратах на год,
     фонд рабочего времени ремонтного подразделения, экономическая
     эффективность и загрузка производственных возможностей.
     Расчёты приближённые и опираются на нормативы, заданные в настройках —
     они предназначены для оценки, а не заменяют официальные формы приказов.
     ======================================================================== */

  /* --- A. Группы эксплуатации и КТГ (формула 4) --- */
  function ktgSummary() {
    return EXP_GROUPS.map(group => {
      const list = DB.vehicles.filter(v => v.expGroup === group && v.status !== 'Списано');
      const total = list.length;
      const inRepair = list.filter(v => v.status === 'Ремонт').length;
      const ktg = Number(DB.settings.ktgByGroup[group] != null ? DB.settings.ktgByGroup[group] : 0.9);
      const allowed = total > 0 ? Math.max(0, Math.floor(total * (1 - ktg)) - 1) : 0;
      return { group, ktg, total, inRepair, allowed, exceeded: total > 0 && inRepair > allowed };
    });
  }

  /* --- B. Наличие моторесурса: годовые нормы, 5% резерв, факт/остаток (3.7) --- */
  function resourceSummary() {
    const year = new Date().getFullYear();
    return DB.vehicles.filter(v => v.status !== 'Списано').map(v => {
      const norm = Number(v.annualNormHours || 0);
      const used = hoursInYear(v.id, year);
      const reserve = norm * 0.05;
      const allowed = norm - reserve;
      const remaining = norm > 0 ? allowed - used : null;
      return { v, norm, used, reserve, allowed, remaining, normSet: norm > 0, exceeded: norm > 0 && used > allowed };
    });
  }

  /* --- C. Потребность в ТО-1/ТО-2/ТР и трудозатратах на год (формулы 5-10) ---
     Считается по группам «марка × группа эксплуатации» — так же, как в
     методике (LГij — суммарный годовой пробег/наработка группы машин). */
  function toNeedByGroup() {
    const s = DB.settings;
    const map = new Map();
    DB.vehicles.filter(v => v.status !== 'Списано' && Number(v.annualPlanHours) > 0).forEach(v => {
      const key = (v.brand || v.name) + '||' + (v.expGroup || '—');
      if (!map.has(key)) map.set(key, { brand: v.brand || v.name, group: v.expGroup || '—', count: 0, Lgod: 0 });
      const r = map.get(key);
      r.count++;
      r.Lgod += Number(v.annualPlanHours || 0);
    });
```

```
    // Периодичность — делитель в формулах (5)-(7): защищаемся от нуля и мусора
    // в настройках (например, после восстановления из повреждённой копии).
    const period = (val, fallback) => {
      const n = Number(val);
      return (isFinite(n) && n > 0) ? n : fallback;
    };
    const pTO1 = period(s.periodTO1, DEFAULT_SETTINGS.periodTO1);
    const pTO2 = period(s.periodTO2, DEFAULT_SETTINGS.periodTO2);
    const pTR  = period(s.periodTR,  DEFAULT_SETTINGS.periodTR);

    return Array.from(map.values()).map(r => {
      const nTO2 = Math.floor(r.Lgod / pTO2);
      const nTO1 = Math.max(0, Math.floor(r.Lgod / pTO1) - nTO2);
      const nTR  = Math.floor(r.Lgod / pTR);
      const tTO1 = nTO1 * Number(s.laborNorms['TO-1'] || 0);
      const tTO2 = nTO2 * Number(s.laborNorms['TO-2'] || 0);
      const tTR  = nTR  * Number(s.laborNorms['TP']   || 0);
      return Object.assign(r, { nTO1, nTO2, nTR, tTO1, tTO2, tTR, tTotal: tTO1 + tTO2 + tTR });
    }).sort((a, b) => a.brand.localeCompare(b.brand, 'ru'));
  }

  /* --- E. Фонд рабочего времени ремонтного подразделения (формулы 21-24, 3.12) --- */
  function repairFund() {
    const r = DB.settings.repairUnit;
    const monthlyPerSpecialist = Number(r.workDaysMonth) * Number(r.hoursPerDay);
    const annualPerSpecialist  = monthlyPerSpecialist * 12 * (1 - Number(r.nonProdPct) / 100);
    return {
      specialists: Number(r.specialists) || 0,
      monthlyPerSpecialist,
      annualPerSpecialist,
      monthlyTotal: monthlyPerSpecialist * (Number(r.specialists) || 0),
      annualTotal:  annualPerSpecialist  * (Number(r.specialists) || 0)
    };
  }

  /* --- H. Экономическая эффективность и загрузка мощностей (3.12) --- */
  function economicsSummary(from, to) {
    const s = DB.settings;
    const kindMap = { 'TO-1': 'TO-1', 'TO-2': 'TO-2', 'Текущий ремонт': 'TP' };
    const records = DB.maint.filter(m => (!from || m.date >= from) && (!to || m.date <= to) && kindMap[m.kind]);

    const byKind = { 'TO-1': { count: 0, hours: 0 }, 'TO-2': { count: 0, hours: 0 }, 'TP': { count: 0, hours: 0 } };
    const months = new Set();
    records.forEach(m => {
      const key = kindMap[m.kind];
      const t = Number(s.laborNorms[key] || 0);
      byKind[key].count++;
      byKind[key].hours += t;
      if (m.date) months.add(m.date.slice(0, 7));
    });
    const totalHours = byKind['TO-1'].hours + byKind['TO-2'].hours + byKind['TP'].hours;
    const ef = totalHours * Number(s.normHourCost || 0);
    const fund = repairFund();
    const monthsCount = Math.max(1, months.size);
    const periodFund = fund.monthlyTotal * monthsCount;
    const loadPct = periodFund > 0 ? (totalHours / periodFund) * 100 : null;
    return { records: records.length, byKind, totalHours, ef, fund, monthsCount, periodFund, loadPct };
  }

  const PLAN_TABS = [
    { key: 'ktg',      label: 'Группы и КТГ' },
    { key: 'resource', label: 'Моторесурс' },
    { key: 'labor',    label: 'Потребность в ТО' },
    { key: 'repair',   label: 'Ремонтное подразделение' },
    { key: 'economy',  label: 'Экономика' }
  ];

  function renderPlanning() {
    return `
      <div class="f-page-head">
        <div>
          <div class="f-page-title" role="heading" aria-level="1">Планирование</div>
          <div class="f-page-sub">Расчёты по методике планирования эксплуатации и ремонта ВВСТ (ВАМТО, 2025)</div>
        </div>
      </div>
      <div class="f-tabs">
        ${PLAN_TABS.map(t => `<button type="button" class="f-tab ${ui.planTab === t.key ? 'is-active' : ''}" data-act="plantab" data-tab="${t.key}">${esc(t.label)}</button>`).join('')}
      </div>
      <div id="planBody">${renderPlanTab()}</div>`;
  }

  function renderPlanTab() {
    switch (ui.planTab) {
      case 'ktg':      return renderPlanKtg();
      case 'resource': return renderPlanResource();
      case 'labor':    return renderPlanLabor();
      case 'repair':   return renderPlanRepair();
      case 'economy':  return renderPlanEconomy();
      default:         return '';
```

```
  }
}

function refreshPlanTab() {
  const box = el('planBody');
  if (box) box.innerHTML = renderPlanTab();
}

function renderPlanKtg() {
  const rows = ktgSummary();
  return `
    <div class="f-panel">
      <div class="f-panel-title">Допустимое количество машин одновременно в ремонте</div>
      <div class="f-hint" style="margin-bottom:12px">Формула методики: M<sub>план.рем</sub> = M<sub>списочн</sub>·(1 − КТГ) − 1. Норматив КТГ по группам задаётся в настройках.</div>
      <div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table">
          <thead><tr>
            <th>Группа эксплуатации</th><th class="f-td-num" style="width:120px">Норматив КТГ</th>
            <th class="f-td-num">Списочный состав</th><th class="f-td-num">Допустимо в ремонте</th>
            <th class="f-td-num">Фактически в ремонте</th><th>Состояние</th>
          </tr></thead>
          <tbody>
            ${rows.map(r => `
              <tr>
                <td class="f-td-strong">${esc(r.group)}</td>
                <td class="f-td-num"><input class="f-input" data-ktg-group="${esc(r.group)}" type="number" min="0.01" max="0.99" step="0.01" value="${r.ktg}" style="width:80px;text-align:right"></td>
                <td class="f-td-num">${r.total}</td>
                <td class="f-td-num">${r.allowed}</td>
                <td class="f-td-num" style="${r.exceeded ? 'color:var(--st-overdue-text);font-weight:600' : ''}">${r.inRepair}</td>
                <td>${r.total === 0 ? '<span class="f-td-muted">нет техники</span>' : (r.exceeded ? levelBadge('overdue') : levelBadge('ok'))}</td>
              </tr>`).join('')}
          </tbody>
        </table>
      </div></div>
      <div class="f-hint" style="margin-top:10px">Группа эксплуатации задаётся в карточке техники (вкладка «Основные сведения» → раздел «Планирование эксплуатации»).</div>
    </div>`;
}

function renderPlanResource() {
  const all = resourceSummary();
  const rows = all.filter(r => r.normSet);
  const skipped = all.length - rows.length;
  return `
    <div class="f-panel">
      <div class="f-panel-title">Наличие моторесурса на ${new Date().getFullYear()} год</div>
      <div class="f-hint" style="margin-bottom:12px">Резерв — 5% от годовой нормы (на инспектирование и итоговые проверки). Разрешаемый расход = норма − резерв. Все значения в таблице — в мото·ч.</div>
      ${!rows.length ? emptyState(FI.gauge(28), 'Нет данных для расчёта',
        'Укажите годовую норму расхода ресурса в карточках техники, чтобы увидеть расчёт наличия моторесурса.') : `
      <div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table">
          <thead><tr>
            <th>Техника</th><th>Группа</th><th class="f-td-num">Годовая норма</th>
            <th class="f-td-num">Резерв 5%</th><th class="f-td-num">Разрешено</th>
            <th class="f-td-num">Израсходовано за год</th><th class="f-td-num">Остаток</th><th></th>
          </tr></thead>
          <tbody>
            ${rows.map(r => `
              <tr>
                <td class="f-td-strong">${esc(r.v.name)}</td>
                <td>${esc(r.v.expGroup || '—')}</td>
                <td class="f-td-num">${fmtNum(r.norm, 0)}</td>
                <td class="f-td-num">${fmtNum(r.reserve, 0)}</td>
                <td class="f-td-num">${fmtNum(r.allowed, 0)}</td>
                <td class="f-td-num">${fmtNum(r.used, 0)}</td>
                <td class="f-td-num" style="${r.exceeded ? 'color:var(--st-overdue-text);font-weight:600' : ''}">${fmtNum(r.remaining, 0)}</td>
                <td>${r.exceeded ? levelBadge('overdue') : levelBadge('ok')}</td>
              </tr>`).join('')}
          </tbody>
        </table>
      </div></div>`}
      ${skipped ? `<div class="f-hint" style="margin-top:10px">Без заданной годовой нормы: ${skipped} ед. техники — не включены в расчёт.</div>` : ''}
    </div>`;
}

function renderPlanLabor() {
  const rows = toNeedByGroup();
  const s = DB.settings;
  const totalT = rows.reduce((sum, r) => sum + r.tTotal, 0);
  return `
    <div class="f-panel">
      <div class="f-panel-title">Нормативы для расчёта (периодичность и трудоёмкость)</div>
      <div class="f-form-grid">
```

```
          <div><label class="f-fld-label" for="perTO1">Периодичность ТО-1, мото·ч (км)</label>
            <input id="perTO1" class="f-input" type="number" min="1" step="1" value="${s.periodTO1}"></div>
          <div><label class="f-fld-label" for="perTO2">Периодичность ТО-2, мото·ч (км)</label>
            <input id="perTO2" class="f-input" type="number" min="1" step="1" value="${s.periodTO2}"></div>
          <div><label class="f-fld-label" for="perTR">Периодичность ТР, мото·ч (км)</label>
            <input id="perTR" class="f-input" type="number" min="1" step="1" value="${s.periodTR}"></div>
          <div><label class="f-fld-label" for="laborTO1">Трудоёмкость ТО-1, чел.-ч</label>
            <input id="laborTO1" class="f-input" type="number" min="0" step="0.1" value="${s.laborNorms['TO-1']}"></div>
          <div><label class="f-fld-label" for="laborTO2">Трудоёмкость ТО-2, чел.-ч</label>
            <input id="laborTO2" class="f-input" type="number" min="0" step="0.1" value="${s.laborNorms['TO-2']}"></div>
          <div><label class="f-fld-label" for="laborTR">Трудоёмкость ТР, чел.-ч</label>
            <input id="laborTR" class="f-input" type="number" min="0" step="0.1" value="${s.laborNorms['TP']}"></div>
        </div>
      </div>
      <div class="f-panel">
        <div class="f-panel-title">Потребность в ТО-1 / ТО-2 / ТР и трудозатраты на год</div>
        <div class="f-hint" style="margin-bottom:12px">
          N(ТО-2) = L<sub>год</sub>/${fmtNum(s.periodTO2,0)}; N(ТО-1) = L<sub>год</sub>/${fmtNum(s.periodTO1,0)} − N(ТО-2); N(ТР) = L<sub>год</sub>/${fmtNum(s.periodTR,0)}.
          Трудозатраты — по нормативам из настроек (ТО-1: ${s.laborNorms['TO-1']} чел.-ч, ТО-2: ${s.laborNorms['TO-2']} чел.-ч, ТР: ${s.laborNorms['TP']} чел.-ч).
        </div>
        ${!rows.length ? emptyState(FI.calc(28), 'Нет данных для расчёта',
          'Укажите плановую годовую наработку в карточках техники, чтобы рассчитать потребность в ТО и трудозатраты.') : `
        <div class="f-table-wrap"><div class="f-table-scroll">
          <table class="f-table">
            <thead><tr>
              <th>Марка</th><th>Группа</th><th class="f-td-num">Единиц</th><th class="f-td-num">L<sub>год</sub>, мото·ч</th>
              <th class="f-td-num">ТО-1, шт.</th><th class="f-td-num">ТО-2, шт.</th><th class="f-td-num">ТР, шт.</th>
              <th class="f-td-num">Трудозатраты, чел.-ч</th>
            </tr></thead>
            <tbody>
              ${rows.map(r => `
                <tr>
                  <td class="f-td-strong">${esc(r.brand)}</td>
                  <td>${esc(r.group)}</td>
                  <td class="f-td-num">${r.count}</td>
                  <td class="f-td-num">${fmtNum(r.Lgod, 0)}</td>
                  <td class="f-td-num">${r.nTO1}</td>
                  <td class="f-td-num">${r.nTO2}</td>
                  <td class="f-td-num">${r.nTR}</td>
                  <td class="f-td-num f-td-strong">${fmtNum(r.tTotal, 0)}</td>
                </tr>`).join('')}
            </tbody>
            <tfoot><tr>
              <td colspan="7" class="f-td-strong" style="text-align:right">Итого трудозатрат на год</td>
              <td class="f-td-num f-td-strong">${fmtNum(totalT, 0)}</td>
            </tr></tfoot>
          </table>
        </div></div>`}
      </div>
      ${rows.length ? `
      <div class="f-panel">
        <div class="f-panel-title">Хватает ли сил ремонтного подразделения</div>
        ${(() => {
          const fund = repairFund();
          const enough = fund.annualTotal >= totalT;
          return `
          <div class="f-info-grid">
            <div><div class="f-info-k">Требуется трудозатрат на год</div><div class="f-info-v">${fmtNum(totalT, 0)} чел.-ч</div></div>
            <div><div class="f-info-k">Годовой фонд рабочего времени подразделения</div><div class="f-info-v">${fmtNum(fund.annualTotal, 0)} чел.-ч</div></div>
            <div><div class="f-info-k">Вывод</div><div class="f-info-v" style="color:${enough ? 'var(--st-ok-text)' : 'var(--st-overdue-text)'}">${enough ? 'Сил и средств достаточно' : 'Требуется усиление подразделения'}</div></div>
          </div>`;
        })()}
      </div>` : ''}`;
    }

    function renderPlanRepair() {
      const r = DB.settings.repairUnit;
      const fund = repairFund();
      return `
        <div class="f-panel">
          <div class="f-panel-title">Параметры ремонтного подразделения</div>
          <div class="f-form-grid">
            <div><label class="f-fld-label" for="ruSpecialists">Количество специалистов, чел.</label>
              <input id="ruSpecialists" class="f-input" type="number" min="0" step="1" value="${r.specialists}"></div>
            <div><label class="f-fld-label" for="ruWorkDays">Рабочих дней в месяце</label>
              <input id="ruWorkDays" class="f-input" type="number" min="1" max="31" step="1" value="${r.workDaysMonth}"></div>
            <div><label class="f-fld-label" for="ruHoursDay">Продолжительность рабочего дня, ч</label>
              <input id="ruHoursDay" class="f-input" type="number" min="1" max="24" step="0.5" value="${r.hoursPerDay}"></div>
            <div><label class="f-fld-label" for="ruNonProd">Вынужденные потери, % от годового фонда</label>
              <input id="ruNonProd" class="f-input" type="number" min="0" max="100" step="1" value="${r.nonProdPct}"></div>
```

```
        </div>
      </div>
      <div class="f-panel">
        <div class="f-panel-title">Расчётный фонд рабочего времени</div>
        <div class="f-info-grid">
          <div><div class="f-info-k">Фонд одного специалиста в месяц</div><div class="f-info-v">${fmtNum(fund.monthlyPerSpecialist, 0)} чел.-ч</div></div>
          <div><div class="f-info-k">Фонд одного специалиста в год</div><div class="f-info-v">${fmtNum(fund.annualPerSpecialist, 0)} чел.-ч</div></div>
          <div><div class="f-info-k">Фонд подразделения в месяц</div><div class="f-info-v">${fmtNum(fund.monthlyTotal, 0)} чел.-ч</div></div>
          <div><div class="f-info-k">Фонд подразделения в год</div><div class="f-info-v">${fmtNum(fund.annualTotal, 0)} чел.-ч</div></div>
        </div>
      </div>`;
  }

  function renderPlanEconomy() {
    const from = ui.econ.from;
    const to   = ui.econ.to;
    const e = economicsSummary(from, to);
    const s = DB.settings;
    return `
      <div class="f-panel">
        <div class="f-panel-title">Стоимость нормочаса</div>
        <div class="f-form-grid">
          <div><label class="f-fld-label" for="econCost">Стоимость нормочаса, ₽</label>
            <input id="econCost" class="f-input" type="number" min="0" step="1" value="${s.normHourCost}"></div>
          <div><label class="f-fld-label" for="econFrom">Период с</label>
            <input id="econFrom" class="f-input" type="date" value="${esc(from)}"></div>
          <div><label class="f-fld-label" for="econTo">Период по</label>
            <input id="econTo" class="f-input" type="date" value="${esc(to)}"></div>
        </div>
      </div>
      <div class="f-panel">
        <div class="f-panel-title">Экономическая эффективность ТО и ремонта</div>
        <div class="f-hint" style="margin-bottom:12px">Э<sub>ф</sub> = Т<sub>рт</sub> · С<sub>нч</sub> − трудоёмкость выполненных работ × стоимость нормочаса.</div>
        <div class="f-info-grid">
          <div><div class="f-info-k">Выполнено работ (ТО-1/ТО-2/ТР)</div><div class="f-info-v">${e.records}</div></div>
          <div><div class="f-info-k">Суммарная трудоёмкость</div><div class="f-info-v">${fmtNum(e.totalHours, 0)} чел.-ч</div></div>
          <div><div class="f-info-k">Экономический эффект</div><div class="f-info-v">${fmtMoney(e.ef)}</div></div>
        </div>
        <div class="f-table-wrap" style="margin-top:14px"><div class="f-table-scroll">
          <table class="f-table">
            <thead><tr><th>Вид работ</th><th class="f-td-num">Количество</th><th class="f-td-num">Трудоёмкость, чел.-ч</th></tr></thead>
            <tbody>
              ${LABOR_KINDS.map(k => `<tr><td>${esc(k)}</td><td class="f-td-num">${e.byKind[k].count}</td><td class="f-td-num">${fmtNum(e.byKind[k].hours, 0)}</td></tr>`).join('')}
            </tbody>
          </table>
        </div></div>
      </div>
      <div class="f-panel">
        <div class="f-panel-title">Загрузка производственных возможностей</div>
        <div class="f-hint" style="margin-bottom:12px">З<sub>пв</sub> = Т / (Ф<sub>мес</sub>·К) · 100% — за расчётный период учтено ${e.monthsCount} ${plural(e.monthsCount, 'месяц', 'месяца', 'месяцев')} работы подразделения (${fmtNum(e.fund.specialists,0)} чел., фонд ${fmtNum(e.periodFund,0)} чел.-ч).</div>
        <div class="f-info-grid">
          <div><div class="f-info-k">Загрузка мощностей</div><div class="f-info-v" style="color:${e.loadPct != null && e.loadPct > 100 ? 'var(--st-overdue-text)' : 'var(--text-primary)'}">${e.loadPct != null ? e.loadPct.toFixed(1) + '%' : '—'}</div></div>
        </div>
      </div>`;
  }

  function bindPlanningInputs() {
    /* min/max — те же границы, что объявлены у полей в разметке. Периодичность
       не может быть нулевой: на неё выполняется деление в формулах (5)-(7). */
    const num = (id, apply, min, max) => {
      const n = el(id);
      if (!n) return;
      const lo = min == null ? 0 : min;
      const hi = max == null ? 1e9 : max;
      n.addEventListener('change', () => {
        const v = parseFloat(String(n.value).replace(',', '.'));
        if (isNaN(v) || v < lo || v > hi) {
          toast('warning', 'Некорректное значение', `Допустимый диапазон: от ${fmtNum(lo)} до ${fmtNum(hi)}.`);
          refreshPlanTab();
          bindPlanningInputs();
          return;
        }
        apply(v);
        saveDB();
        toast('success', 'Параметр сохранён', '');
        refreshPlanTab();
        bindPlanningInputs();
      });
```

```
    };
    num('ruSpecialists', v => DB.settings.repairUnit.specialists = v, 0, 10000);
    num('ruWorkDays',    v => DB.settings.repairUnit.workDaysMonth = v, 1, 31);
    num('ruHoursDay',    v => DB.settings.repairUnit.hoursPerDay = v, 1, 24);
    num('ruNonProd',     v => DB.settings.repairUnit.nonProdPct = v, 0, 100);
    num('econCost',      v => DB.settings.normHourCost = v, 0, 1e7);
    num('perTO1',        v => DB.settings.periodTO1 = v, 1);
    num('perTO2',        v => DB.settings.periodTO2 = v, 1);
    num('perTR',         v => DB.settings.periodTR = v, 1);
    num('laborTO1',      v => DB.settings.laborNorms['TO-1'] = v, 0, 100000);
    num('laborTO2',      v => DB.settings.laborNorms['TO-2'] = v, 0, 100000);
    num('laborTR',       v => DB.settings.laborNorms['TP'] = v, 0, 100000);

    document.querySelectorAll('[data-ktg-group]').forEach(n => {
      n.addEventListener('change', () => {
        const v = parseFloat(String(n.value).replace(',', '.'));
        if (isNaN(v) || v <= 0 || v >= 1) {
          toast('warning', 'Некорректное значение', 'Норматив КТГ задаётся долей от 0,01 до 0,99.');
          refreshPlanTab();
          bindPlanningInputs();
          return;
        }
        DB.settings.ktgByGroup[n.dataset.ktgGroup] = v;
        saveDB();
        toast('success', 'Норматив КТГ сохранён', '');
        refreshPlanTab();
        bindPlanningInputs();
      });
    });

    const econDate = (id, prop) => {
      const n = el(id);
      if (!n) return;
      n.addEventListener('change', () => {
        ui.econ[prop] = n.value;
        refreshPlanTab();
        bindPlanningInputs();
      });
    };
    econDate('econFrom', 'from');
    econDate('econTo', 'to');
  }

  /* ============================================================================
     17c. РАЗДЕЛ «ДОКУМЕНТЫ»
     ----------------------------------------------------------------------------
     Формирование печатных документов по методике планирования эксплуатации
     и ремонта ВВСТ (ВАМТО, 2025): годовой план, план-график ТО на месяц и
     типовые акты. Печать — через системный диалог браузера (как и отчёты),
     разметка использует те же классы .p-title/.p-sub/.p-meta/.p-foot.
     ============================================================================ */

  function printCustom(title, subtitle, bodyHtml, footHtml) {
    emitPrint({ title, subtitle, bodyHtml, footHtml: footHtml || null });
  }

  /* --- F. Годовой план эксплуатации и ремонта --- */
  function printAnnualPlan() {
    const rows = toNeedByGroup();
    if (!rows.length) {
      toast('warning', 'Нет данных', 'Укажите плановую годовую наработку в карточках техники (вкладка «Планирование эксплуатации»).');
      return;
    }
    const totalT = rows.reduce((s, r) => s + r.tTotal, 0);
    const body = `<table>
      <thead><tr><th>Марка</th><th>Группа эксплуатации</th><th class="p-num">Единиц</th>
      <th class="p-num">Годовая наработка, мото·ч</th><th class="p-num">ТО-1</th><th class="p-num">ТО-2</th>
      <th class="p-num">ТР</th><th class="p-num">Трудозатраты, чел.-ч</th></tr></thead>
      <tbody>
        ${rows.map(r => `<tr><td>${esc(r.brand)}</td><td>${esc(r.group)}</td><td class="p-num">${r.count}</td>
          <td class="p-num">${fmtNum(r.Lgod, 0)}</td><td class="p-num">${r.nTO1}</td><td class="p-num">${r.nTO2}</td>
          <td class="p-num">${r.nTR}</td><td class="p-num">${fmtNum(r.tTotal, 0)}</td></tr>`).join('')}
        <tr><td colspan="7"><b>Итого трудозатрат на год</b></td><td class="p-num"><b>${fmtNum(totalT, 0)}</b></td></tr>
      </tbody></table>`;
    printCustom('ГОДОВОЙ ПЛАН ЭКСПЛУАТАЦИИ И РЕМОНТА ТЕХНИЧЕСКИХ СРЕДСТВ ТЫЛА',
      `на ${new Date().getFullYear() + 1} год · рассчитано по методике планирования эксплуатации и ремонта ВВСТ`, body);
  }

  /* --- F. План-график ТО на месяц (формула 18: Дто = (Lто-1 − Lф)/Lсс) --- */
  function monthPlanRows() {
    const s = DB.settings;
    const basePeriod = (isFinite(Number(s.periodTO1)) && Number(s.periodTO1) > 0)
      ? Number(s.periodTO1) : DEFAULT_SETTINGS.periodTO1;
    return DB.vehicles.filter(v => v.status !== 'Списано').map(v => {
      const st = computeStatus(v);
      const periodTO1 = (v.interval && v.interval.byHours && Number(v.interval.hours) > 0)
        ? Number(v.interval.hours) : basePeriod;
      const Lf = st.hoursSince;
      const dailyRun = Number(v.annualPlanHours || 0) > 0 ? Number(v.annualPlanHours) / 365 : null;
```

```
      let daysLeft = null, plannedDate = null;
      if (dailyRun && dailyRun > 0) {
        daysLeft = Math.round((periodTO1 - Lf) / dailyRun);
        plannedDate = addDays(todayISO(), Math.max(0, daysLeft));
      }
      return { v, periodTO1, Lf, dailyRun, daysLeft, plannedDate };
    }).sort((a, b) => (a.daysLeft == null ? Infinity : a.daysLeft) - (b.daysLeft == null ? Infinity : b.daysLeft));
  }

  function renderMonthPlanTable(rows) {
    return `
      <div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table">
          <thead><tr>
            <th>Техника</th><th class="f-td-num">Периодичность ТО-1</th><th class="f-td-num">Наработка с ТО</th>
            <th class="f-td-num">Среднесут. наработка</th><th class="f-td-num">Дней до ТО</th><th>Расчётная дата ТО</th>
          </tr></thead>
          <tbody>
            ${rows.map(r => `
              <tr>
                <td class="f-td-strong">${esc(r.v.name)}</td>
                <td class="f-td-num">${fmtNum(r.periodTO1, 0)}</td>
                <td class="f-td-num">${fmtNum(r.Lf, 0)}</td>
                <td class="f-td-num">${r.dailyRun != null ? fmtNum(r.dailyRun, 1) : '—'}</td>
                <td class="f-td-num" style="${r.daysLeft != null && r.daysLeft <= 0 ? 'color:var(--st-overdue-text);font-weight:600' : ''}">${r.daysLeft != null ? r.daysLeft : '—'}</td>
                <td>${r.plannedDate ? fmtDate(r.plannedDate) : '<span class="f-td-muted">нет плановой наработки</span>'}</td>
              </tr>`).join('')}
          </tbody>
        </table>
      </div></div>`;
  }

  function printMonthPlan() {
    const rows = monthPlanRows();
    const body = `<table>
      <thead><tr><th>Техника</th><th class="p-num">Периодичность ТО-1</th><th class="p-num">Наработка с ТО</th>
      <th class="p-num">Среднесут. наработка</th><th class="p-num">Дней до ТО</th><th>Расчётная дата ТО</th></tr></thead>
      <tbody>${rows.map(r => `<tr><td>${esc(r.v.name)}</td><td class="p-num">${fmtNum(r.periodTO1,0)}</td>
        <td class="p-num">${fmtNum(r.Lf,0)}</td><td class="p-num">${r.dailyRun != null ? fmtNum(r.dailyRun,1) : '—'}</td>
        <td class="p-num">${r.daysLeft != null ? r.daysLeft : '—'}</td><td>${r.plannedDate ? fmtDate(r.plannedDate) : '—'}</td></tr>`).join('')}
      </tbody></table>`;
    printCustom('ПЛАН-ГРАФИК ТЕХНИЧЕСКОГО ОБСЛУЖИВАНИЯ И РЕМОНТА',
      `на ${fmtDate(todayISO())}, расчёт по среднесуточному пробегу`, body);
  }

  /* ============================================================================
     НОРМАТИВЫ ПОЛЬЗОВАТЕЛЯ
     ----------------------------------------------------------------------------
     Для позиций справочника, у которых в учебном пособии нет регламента ТО.
     Хранятся в DB.norms (попадают в резервную копию и синхронизацию вкладок),
     справочник подхватывает их через TST.fleet.customMaint()
     ============================================================================ */
  function customMaint(catalogId) {
    const c = CAT();
    const list = DB ? DB.norms.filter(n => n.catalogId === catalogId) : [];
    if (!list.length || !c) return null;
    const out = {};
    c.types.forEach(t => {
      const n = list.find(x => x.type === t.key);
      if (n) out[t.key] = {
        actions: n.actions || [], labor: Number(n.labor) || 0, personnel: n.personnel || [],
        tools: n.tools || [], materials: n.materials || [], custom: true, normId: n.id
      };
    });
    return Object.keys(out).length ? out : null;
  }

  /* «Должность — 2» / «Должность: 2 чел.» / «Должность» (= 1) */
  function parsePersonnel(text) {
    return String(text || '').split('\n').map(s => s.trim()).filter(Boolean).map(line => {
      const m = /^(.*?)[\s—:\-]*(\d+)\s*(?:чел\.?)?\s*$/.exec(line);
      return (m && m[1].trim()) ? { role: m[1].trim(), count: Number(m[2]) } : { role: line, count: 1 };
    });
  }
  const splitLines = text => String(text || '').split('\n').map(s => s.replace(/^\s*\d+[.)]\s*/, '').trim()).filter(Boolean);

  function normModal(catalogId, type) {
    const c = CAT();
    const node = c && c.get(catalogId);
    if (!node) return;
    if (c.hasBuiltIn(catalogId)) { toast('info', 'Регламент уже есть', 'Для этой позиции действует регламент из учебного пособия.'); return; }
    const mine = () => DB.norms.filter(n => n.catalogId === catalogId);
    const firstFree = (c.types.find(t => !mine().some(n => n.type === t.key)) || c.types[0]).key;
```

```
  const startType = type || firstFree;
  const lines = arr => (arr || []).join('\n');

  const m = openModal(`
    <div class="f-modal-head">
      <div>
        <div class="f-modal-title">Норматив технического обслуживания</div>
        <div class="f-modal-sub">${esc(node.name)} · вводится пользователем</div>
      </div>
      <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
    </div>
    <div class="f-modal-body">
      <div class="f-form-grid">
        <div><label class="f-fld-label" for="nType">Вид обслуживания</label>
          <select id="nType" class="f-select">${c.types.map(t =>
            `<option value="${t.key}"${t.key === startType ? ' selected' : ''}>${esc(t.short)} — ${esc(t.full)}</option>`).join('')}</select>
          <div class="f-hint" id="nTypeHint"></div></div>
        <div><label class="f-fld-label" for="nLabor">Трудоёмкость, чел·ч *</label>
          <input id="nLabor" class="f-input" type="number" min="0" step="0.1">
          <div class="f-error" id="errNLabor">Укажите трудоёмкость (число от 0)</div></div>
        <div class="f-span2"><label class="f-fld-label" for="nActions">Перечень операций * <span class="f-hint" style="display:inline">— по одной в строке</span></label>
          <textarea id="nActions" class="f-textarea" rows="7" placeholder="Внешний осмотр, очистка от пыли и грязи&#10;Проверка креплений узлов"></textarea>
          <div class="f-error" id="errNActions">Добавьте хотя бы одну операцию</div></div>
        <div class="f-span2"><label class="f-fld-label" for="nPersonnel">Личный состав <span class="f-hint" style="display:inline">— «должность — количество», по одной в строке</span></label>
          <textarea id="nPersonnel" class="f-textarea" rows="3" placeholder="Механик-водитель — 1&#10;Слесарь — 2"></textarea></div>
        <div><label class="f-fld-label" for="nTools">Инструмент и оснастка</label>
          <textarea id="nTools" class="f-textarea" rows="4"></textarea></div>
        <div><label class="f-fld-label" for="nMaterials">Расходные материалы</label>
          <textarea id="nMaterials" class="f-textarea" rows="4"></textarea></div>
      </div>
    </div>
    <div class="f-modal-foot">
      <button type="button" class="f-btn" data-close>Отмена</button>
      <button type="button" class="f-btn f-btn-primary" id="nSave">${FI.save(15)}<span>Сохранить</span></button>
    </div>`);

  const R = m.root;
  const q = id => R.querySelector('#' + id);
  // Смена вида подгружает уже введённый норматив этого вида (редактирование)
  const load = () => {
    const n = mine().find(x => x.type === q('nType').value);
    q('nTypeHint').textContent = n ? 'Норматив этого вида уже введён — сохранение заменит его.' : '';
    q('nLabor').value = n ? n.labor : '';
    q('nActions').value = n ? lines(n.actions) : '';
    q('nPersonnel').value = n ? (n.personnel || []).map(p => `${p.role} — ${p.count}`).join('\n') : '';
    q('nTools').value = n ? lines(n.tools) : '';
    q('nMaterials').value = n ? lines(n.materials) : '';
  };
  q('nType').addEventListener('change', load);
  load();

  q('nSave').addEventListener('click', () => {
    const actions = splitLines(q('nActions').value);
    const labor = parseFloat(String(q('nLabor').value).replace(',', '.'));
    const badLabor = !isFinite(labor) || labor < 0 || labor > 100000;
    q('nActions').classList.toggle('is-invalid', !actions.length); q('errNActions').classList.toggle('is-visible', !actions.length);
    q('nLabor').classList.toggle('is-invalid', badLabor);         q('errNLabor').classList.toggle('is-visible', badLabor);
    if (!actions.length || badLabor) return;

    const t = q('nType').value;
    const data = { actions, labor, personnel: parsePersonnel(q('nPersonnel').value),
                   tools: splitLines(q('nTools').value), materials: splitLines(q('nMaterials').value) };
    const cur = DB.norms.find(n => n.catalogId === catalogId && n.type === t);
    if (cur) Object.assign(cur, data, { updatedAt: new Date().toISOString() });
    else DB.norms.push(Object.assign({ id: uid('n'), catalogId, type: t, createdAt: new Date().toISOString() }, data));
    saveDB();
    m.close();
    const short = (c.types.find(x => x.key === t) || {}).short || t;
    toast('success', 'Норматив сохранён', `${short} · ${node.name}`);
    if (window.TST.workspace === 'catalog') c.open(catalogId, { tab: t });
    else rerender();
  });
}

function deleteNorm(catalogId, type) {
  const c = CAT();
  const node = c && c.get(catalogId);
  const short = ((c && c.types.find(x => x.key === type)) || {}).short || type;
  removeWithUndo('Норматив удалён', `${short}${node ? ' · ' + node.name : ''}`, () => {
    DB.norms = DB.norms.filter(n => !(n.catalogId === catalogId && n.type === type));
  });
}
```

```
3020
3021    /* ========================================================================
3022       РЕДАКТОР СПРАВОЧНИКА: правка и добавление позиций
3023       ------------------------------------------------------------------------
3024       Исходный каталог не меняется — правки лежат в DB.catalogEdits и
3025       накладываются поверх (см. applyUserCatalog в справочнике)
3026       ======================================================================== */
3027    const cloneJSON = o => JSON.parse(JSON.stringify(o));
3028
3029    function catalogModal({ id, parentId }) {
3030      const c = CAT();
3031      if (!c) return;
3032      const node = id ? c.get(id) : null;
3033      if (id && (!node || node.children)) return;
3034      const isAdd = !id;
3035      const addRec = node && node.userAdded ? DB.catalogEdits.find(e => e.kind === 'add' && e.node && e.node.id === id) :
    null;
3036      const editRec = node && !node.userAdded ? DB.catalogEdits.find(e => e.kind === 'edit' && e.catalogId === id) : null;
3037      if (node && node.userAdded && !addRec) return;
3038      const groups = c.groups();
3039      if (isAdd && !groups.length) return;
3040      const types = c.types;
3041
3042      /* --- рабочая копия --- */
3043      const work = {
3044        specs: node && node.specs ? cloneJSON(node.specs.groups || []) : [],
3045        maint: {}
3046      };
3047      const eff = node ? c.maintenanceOf(id) : null;
3048      if (eff) Object.keys(eff).forEach(k => {
3049        const m = eff[k];
3050        work.maint[k] = { actions: (m.actions || []).slice(), laborRaw: String(m.labor == null ? '' : m.labor),
3051          personnel: (m.personnel || []).map(x => ({ role: x.role, count: x.count })),
3052          tools: (m.tools || []).slice(), materials: (m.materials || []).slice() };
3053      });
3054      const cleanSpecs = () => work.specs
3055        .map(g => ({ title: String(g.title || '').trim(),
3056                     params: (g.params || []).map(x => ({ label: String(x.label || '').trim(), value: String(x.value ||
    '').trim() }))
3057                                            .filter(x => x.label || x.value) }))
3058        .filter(g => g.params.length);
3059      const cleanMaint = () => {
3060        const out = {};
3061        types.forEach(t => {
3062          const m = work.maint[t.key];
3063          if (!m) return;
3064          out[t.key] = { actions: m.actions, labor: parseFloat(String(m.laborRaw).replace(',', '.')) || 0,
3065                         personnel: m.personnel, tools: m.tools, materials: m.materials };
3066        });
3067        return out;
3068      };
3069      const initial = {
3070        name: node ? node.name : '', description: node ? (node.description || '') : '',
3071        specs: JSON.stringify(cleanSpecs()), maintenance: JSON.stringify(cleanMaint())
3072      };
3073      const lines = arr => (arr || []).join('\n');
3074      const startType = (types.find(t => work.maint[t.key]) || types[0]).key;
3075      const defaultParent = (groups.find(g => g.id === parentId) || groups.find(g => g.depth === 1) || groups[0] ||
    {}).id;
3076
3077      const m = openModal(`
3078        <div class="f-modal-head">
3079          <div>
3080            <div class="f-modal-title">${isAdd ? 'Новая позиция справочника' : 'Редактирование позиции'}</div>
3081            <div class="f-modal-sub">${isAdd ? 'Добавляется только в этой программе, исходный справочник не меняется' :
    esc(node.name)}</div>
3082          </div>
3083          <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
3084        </div>
3085        <div class="f-modal-body">
3086          <div class="f-form-grid">
3087            ${isAdd ? `<div class="f-span2"><label class="f-fld-label" for="cParent">Раздел *</label>
3088              <select id="cParent" class="f-select">${groups.map(g =>
3089                `<option value="${esc(g.id)}"${g.id === defaultParent ? ' selected' :
    ''}>${'\u00A0\u00A0'.repeat(g.depth)}${esc(g.label.split(' › ').pop())}</option>`).join('')}</select></div>` : ''}
3090            <div class="f-span2"><label class="f-fld-label" for="cName">Наименование *</label>
3091              <input id="cName" class="f-input" maxlength="200" value="${esc(initial.name)}">
3092              <div class="f-error" id="errCName">Укажите наименование</div></div>
3093            <div class="f-span2"><label class="f-fld-label" for="cDesc">Описание</label>
3094              <textarea id="cDesc" class="f-textarea" rows="3">${esc(initial.description)}</textarea></div>
3095          </div>
3096
3097          <h3 class="ce-h">Технические характеристики</h3>
3098          <div id="cSpecs"></div>
3099          <button type="button" class="link-btn" data-act="add-group">+ Группа параметров</button>
3100
3101          <h3 class="ce-h">Регламент обслуживания</h3>
3102          <div class="f-form-grid">
3103            <div><label class="f-fld-label" for="cType">Вид обслуживания</label>
3104              <select id="cType" class="f-select"></select>
```

```
            <div class="f-hint">Виды, у которых что-то заполнено, сохраняются. Пустой вид не создаётся.</div></div>
          <div><label class="f-fld-label" for="cLabor">Трудоёмкость, чел·ч</label>
            <input id="cLabor" class="f-input" inputmode="decimal">
            <div class="f-error" id="errCLabor">Число от 0 до 100000</div></div>
          <div class="f-span2"><label class="f-fld-label" for="cActions">Перечень операций <span class="f-hint" style="display:inline">— по одной в строке</span></label>
            <textarea id="cActions" class="f-textarea" rows="6"></textarea>
            <div class="f-error" id="errCActions">Добавьте хотя бы одну операцию или очистите вид</div></div>
          <div class="f-span2"><label class="f-fld-label" for="cPersonnel">Личный состав <span class="f-hint" style="display:inline">— «должность — количество», по одной в строке</span></label>
            <textarea id="cPersonnel" class="f-textarea" rows="3" placeholder="Механик-водитель — 1&#10;Слесарь — 2"></textarea></div>
          <div><label class="f-fld-label" for="cTools">Инструмент и оснастка</label>
            <textarea id="cTools" class="f-textarea" rows="4"></textarea></div>
          <div><label class="f-fld-label" for="cMaterials">Расходные материалы</label>
            <textarea id="cMaterials" class="f-textarea" rows="4"></textarea></div>
        </div>
        <button type="button" class="link-btn is-danger" data-act="clear-type">Очистить этот вид обслуживания</button>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary" id="cSave">${FI.save(15)}<span>Сохранить</span></button>
      </div>`);

    const R = m.root;
    const q = i => R.querySelector('#' + i);

    /* --- характеристики --- */
    const drawSpecs = () => {
      q('cSpecs').innerHTML = work.specs.length ? work.specs.map((g, gi) => `
        <div class="ce-group" data-g="${gi}">
          <div class="ce-group-head">
            <input class="f-input" data-f="title" value="${esc(g.title || '')}" placeholder="Название группы (необязательно), например КА-220" aria-label="Название группы параметров">
            <button type="button" class="link-btn is-danger" data-act="del-group">Удалить группу</button>
          </div>
          ${(g.params || []).map((x, pi) => `
            <div class="ce-row" data-p="${pi}">
              <input class="f-input" data-f="label" value="${esc(x.label || '')}" placeholder="Параметр" aria-label="Параметр">
              <input class="f-input" data-f="value" value="${esc(x.value || '')}" placeholder="Значение" aria-label="Значение">
              <button type="button" class="ce-x" data-act="del-param" aria-label="Удалить параметр">${FI.x(14)}</button>
            </div>`).join('')}
          <button type="button" class="link-btn" data-act="add-param">+ Параметр</button>
        </div>`).join('') : '<div class="f-hint">Характеристики не заданы. Добавьте группу параметров.</div>';
    };
    drawSpecs();
    R.addEventListener('input', e => {
      const f = e.target.dataset && e.target.dataset.f;
      if (!f) return;
      const gEl = e.target.closest('.ce-group');
      if (!gEl) return;
      const g = work.specs[+gEl.dataset.g];
      if (f === 'title') g.title = e.target.value;
      else { const row = e.target.closest('.ce-row'); if (row) g.params[+row.dataset.p][f] = e.target.value; }
    });

    /* --- регламент --- */
    let cur = startType;
    const optLabel = t => `${work.maint[t.key] ? '✓ ' : ''}${t.short} — ${t.full}`;
    const drawTypes = () => {
      q('cType').innerHTML = types.map(t => `<option value="${t.key}"${t.key === cur ? ' selected' : ''}>${esc(optLabel(t))}</option>`).join('');
    };
    const loadType = () => {
      const x = work.maint[cur];
      q('cLabor').value = x ? x.laborRaw : '';
      q('cActions').value = x ? lines(x.actions) : '';
      q('cPersonnel').value = x ? x.personnel.map(pp => `${pp.role} — ${pp.count}`).join('\n') : '';
      q('cTools').value = x ? lines(x.tools) : '';
      q('cMaterials').value = x ? lines(x.materials) : '';
    };
    const storeType = () => {
      const x = { actions: splitLines(q('cActions').value), laborRaw: q('cLabor').value.trim(),
        personnel: parsePersonnel(q('cPersonnel').value), tools: splitLines(q('cTools').value), materials: splitLines(q('cMaterials').value) };
      const empty = !x.actions.length && !x.laborRaw && !x.personnel.length && !x.tools.length && !x.materials.length;
      if (empty) delete work.maint[cur]; else work.maint[cur] = x;
    };
    drawTypes(); loadType();
    q('cType').addEventListener('change', () => { storeType(); cur = q('cType').value; drawTypes(); loadType(); });

    R.addEventListener('click', e => {
      const b = e.target.closest('[data-act]');
      if (!b) return;
      const gEl = b.closest('.ce-group');
      const g = gEl ? work.specs[+gEl.dataset.g] : null;
      switch (b.dataset.act) {
        case 'add-group': work.specs.push({ title: '', params: [{ label: '', value: '' }] }); drawSpecs();
```

```
3187          { const last = q('cSpecs').querySelector('.ce-group:last-child [data-f="label"]'); if (last) last.focus(); }
      break;
3188        case 'del-group': work.specs.splice(+gEl.dataset.g, 1); drawSpecs(); break;
3189        case 'add-param': g.params.push({ label: '', value: '' }); drawSpecs();
3190          { const rows = q('cSpecs').querySelectorAll(`.ce-group[data-g="${gEl.dataset.g}"] .ce-row`); const l =
      rows[rows.length - 1]; if (l) l.querySelector('input').focus(); } break;
3191        case 'del-param': g.params.splice(+b.closest('.ce-row').dataset.p, 1); drawSpecs(); break;
3192        case 'clear-type': delete work.maint[cur]; drawTypes(); loadType(); break;
3193      }
3194    });
3195
3196    /* --- сохранение --- */
3197    q('cSave').addEventListener('click', () => {
3198      storeType();
3199      const name = q('cName').value.trim();
3200      const description = q('cDesc').value.trim();
3201      const badName = !name;
3202      q('cName').classList.toggle('is-invalid', badName); q('errCName').classList.toggle('is-visible', badName);
3203      if (badName) { q('cName').focus(); return; }
3204
3205      for (const t of types) {
3206        const x = work.maint[t.key];
3207        if (!x) continue;
3208        const labor = x.laborRaw === '' ? 0 : parseFloat(x.laborRaw.replace(',', '.'));
3209        const badLabor = !isFinite(labor) || labor < 0 || labor > 100000;
3210        const noActions = !x.actions.length;
3211        if (badLabor || noActions) {
3212          cur = t.key; drawTypes(); loadType();
3213          q('cLabor').classList.toggle('is-invalid', badLabor); q('errCLabor').classList.toggle('is-visible', badLabor);
3214          q('cActions').classList.toggle('is-invalid', noActions); q('errCActions').classList.toggle('is-visible',
      noActions);
3215          (noActions ? q('cActions') : q('cLabor')).focus();
3216          toast('warning', 'Проверьте вид обслуживания', `${t.short}: ${noActions ? 'нет ни одной операции' : 'неверная
      трудоёмкость'}.`);
3217          return;
3218        }
3219      }
3220
3221      const groupsOut = cleanSpecs();
3222      const maintOut = cleanMaint();
3223      const now = new Date().toISOString();
3224
3225      if (isAdd) {
3226        const node2 = { id: uid('user'), name, description };
3227        if (groupsOut.length) node2.specs = { groups: groupsOut };
3228        if (Object.keys(maintOut).length) node2.maintenance = maintOut;
3229        DB.catalogEdits.push({ id: uid('ce'), kind: 'add', parentId: q('cParent').value, node: node2, createdAt: now });
3230        saveDB(); m.close();
3231        toast('success', 'Позиция добавлена', name);
3232        c.open(node2.id);
3233      } else if (addRec) {
3234        addRec.node.name = name; addRec.node.description = description;
3235        if (groupsOut.length) addRec.node.specs = { groups: groupsOut }; else delete addRec.node.specs;
3236        if (Object.keys(maintOut).length) addRec.node.maintenance = maintOut; else delete addRec.node.maintenance;
3237        addRec.updatedAt = now;
3238        saveDB(); m.close();
3239        toast('success', 'Изменения сохранены', name);
3240        c.open(id, { keepSection: true });
3241      } else {
3242        const patch = Object.assign({}, editRec ? editRec.patch : {});
3243        let changed = false;
3244        if (name !== initial.name) { patch.name = name; changed = true; }
3245        if (description !== initial.description) { patch.description = description; changed = true; }
3246        if (JSON.stringify(groupsOut) !== initial.specs) { patch.specs = { groups: groupsOut }; changed = true; }
3247        if (JSON.stringify(maintOut) !== initial.maintenance) { patch.maintenance = maintOut; changed = true; }
3248        if (!changed) { m.close(); toast('info', 'Изменений нет', ''); return; }
3249        if (editRec) { editRec.patch = patch; editRec.updatedAt = now; }
3250        else DB.catalogEdits.push({ id: uid('ce'), kind: 'edit', catalogId: id, patch, createdAt: now });
3251        saveDB(); m.close();
3252        toast('success', 'Изменения сохранены', 'Исходные данные пособия можно вернуть кнопкой «Сбросить правки».');
3253        c.open(id, { keepSection: true });
3254      }
3255    });
3256    setTimeout(() => { const f = q(isAdd ? 'cName' : 'cName'); if (f) f.focus(); }, 30);
3257  }
3258
3259  function deleteCatalogNode(id) {
3260    const c = CAT();
3261    const node = c && c.get(id);
3262    if (!node || !node.userAdded) return;
3263    const linked = DB.vehicles.filter(v => v.catalogId === id).length;
3264    removeWithUndo('Позиция удалена', node.name + (linked ? ` · привязанной техники: ${linked} (привязка вернётся при
      отмене)` : ''), () => {
3265      DB.catalogEdits = DB.catalogEdits.filter(e => !(e.kind === 'add' && e.node && e.node.id === id));
3266    });
3267  }
3268
3269  function resetCatalogNode(id) {
3270    const c = CAT();
3271    const node = c && c.get(id);
```

```
    if (!node) return;
    removeWithUndo('Правки сброшены', node.name, () => {
      DB.catalogEdits = DB.catalogEdits.filter(e => !(e.kind === 'edit' && e.catalogId === id));
    });
  }

  function hideCatalogNode(id) {
    const c = CAT();
    const node = c && c.get(id);
    if (!node || node.userAdded) return;
    const rec = { id: uid('ce'), kind: 'hide', catalogId: id, name: node.name,
                  path: c.path(id).join(' › '), isGroup: !!node.children, createdAt: new Date().toISOString() };
    const linked = DB.vehicles.filter(v => v.catalogId === id).length;
    DB.catalogEdits.push(rec);
    saveDB();
    toast('success', node.children ? 'Раздел скрыт' : 'Позиция скрыта',
      node.name + (linked ? ` · техника на учёте остаётся, привязка к справочнику временно не видна` : ''), UNDO_MS, {
      action: 'Отменить',
      onAction: () => { DB.catalogEdits = DB.catalogEdits.filter(r => r.id !== rec.id); saveDB(); }
    });
  }

  /* Небольшое окно с одним полем — наименование раздела */
  function nameModal(title, sub, value, onSave) {
    const m = openModal(`
      <div class="f-modal-head"><div><div class="f-modal-title">${esc(title)}</div>
        ${sub ? `<div class="f-modal-sub">${esc(sub)}</div>` : ''}</div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button></div>
      <div class="f-modal-body">
        <label class="f-fld-label" for="nmName">Наименование *</label>
        <input id="nmName" class="f-input" maxlength="200" value="${esc(value || '')}">
        <div class="f-error" id="errNmName">Укажите наименование</div>
      </div>
      <div class="f-modal-foot"><button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary"
  id="nmSave">${FI.save(15)}<span>Сохранить</span></button></div>`, { narrow: true });
    const input = m.root.querySelector('#nmName');
    const go = () => {
      const v = input.value.trim();
      input.classList.toggle('is-invalid', !v); m.root.querySelector('#errNmName').classList.toggle('is-visible', !v);
      if (!v) return;
      onSave(v); m.close();
    };
    m.root.querySelector('#nmSave').addEventListener('click', go);
    input.addEventListener('keydown', e => { if (e.key === 'Enter') { e.preventDefault(); go(); } });
    setTimeout(() => { input.focus(); input.select(); }, 30);
  }

  function addCatalogSection(parentId) {
    const c = CAT();
    const parent = parentId ? c.get(parentId) : null;
    nameModal('Новый раздел', parent ? 'В разделе: ' + parent.name : 'Корневой раздел (новая служба)', '', name => {
      DB.catalogEdits.push({ id: uid('ce'), kind: 'add', parentId: parentId || '', node: { id: uid('user'), name,
  isGroup: true },
                             createdAt: new Date().toISOString() });
      saveDB();
      toast('success', 'Раздел добавлен', name);
    });
  }

  function renameCatalogSection(id) {
    const c = CAT();
    const node = c && c.get(id);
    if (!node || !node.children) return;
    nameModal('Переименовать раздел', '', node.name, name => {
      const addRec = DB.catalogEdits.find(e => e.kind === 'add' && e.node && e.node.id === id);
      if (addRec) { addRec.node.name = name; addRec.updatedAt = new Date().toISOString(); }
      else {
        const ed = DB.catalogEdits.find(e => e.kind === 'edit' && e.catalogId === id);
        if (ed) { ed.patch = Object.assign({}, ed.patch, { name }); ed.updatedAt = new Date().toISOString(); }
        else DB.catalogEdits.push({ id: uid('ce'), kind: 'edit', catalogId: id, patch: { name }, createdAt: new
  Date().toISOString() });
      }
      saveDB();
      toast('success', 'Раздел переименован', name);
    });
  }

  /* Удаление добавленного раздела вместе со всем, что в него добавлено */
  function deleteCatalogSection(id) {
    const c = CAT();
    const node = c && c.get(id);
    if (!node || !node.userAdded || !node.children) return;
    const gone = new Set([id]);
    let grew = true;
    while (grew) {
      grew = false;
      DB.catalogEdits.forEach(e => {
        if (e.kind === 'add' && e.node && gone.has(e.parentId) && !gone.has(e.node.id)) { gone.add(e.node.id); grew =
  true; }
```

```
      });
    }
    const n = gone.size - 1;
    removeWithUndo('Раздел удалён', node.name + (n ? ` · вместе с добавленным внутри: ${n}` : ''), () => {
      DB.catalogEdits = DB.catalogEdits.filter(e => !(e.kind === 'add' && e.node && gone.has(e.node.id)));
    });
  }

  /* Окно «Разделы»: структура справочника, добавление и скрытие разделов,
     возврат скрытого. Позиции правятся в их собственных карточках */
  function catalogManager() {
    const c = CAT();
    if (!c) return;
    const m = openModal(`
      <div class="f-modal-head"><div><div class="f-modal-title">Разделы справочника</div>
        <div class="f-modal-sub">Исходный справочник не меняется: скрытое и добавленное можно вернуть или удалить</div></div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button></div>
      <div class="f-modal-body" id="cmBody"></div>
      <div class="f-modal-foot"><button type="button" class="f-btn" data-close>Закрыть</button></div>`);
    const R = m.root;
    const body = R.querySelector('#cmBody');
    const draw = () => {
      const groups = c.groups();
      const hidden = DB.catalogEdits.filter(e => e.kind === 'hide');
      body.innerHTML = `
        <div class="cm-top"><button type="button" class="link-btn" data-cm="add-root">+ Новая служба (корневой раздел)</button></div>
        <ul class="cm-list">${groups.map(g => `
          <li class="cm-row" style="--d:${g.depth}">
            <div class="cm-main">
              <span class="cm-name">${esc(g.name)}</span>
              <span class="cm-meta">${g.leaves} поз.</span>
              ${g.userAdded ? '<span class="user-badge">Добавлено вами</span>' : g.userEdited ? '<span class="user-badge">Переименовано</span>' : ''}
            </div>
            <div class="cm-actions">
              <button type="button" class="link-btn" data-cm="add-section" data-id="${esc(g.id)}">+ Раздел</button>
              <button type="button" class="link-btn" data-cm="add-item" data-id="${esc(g.id)}">+ Позиция</button>
              <button type="button" class="link-btn" data-cm="rename" data-id="${esc(g.id)}">Переименовать</button>
              ${g.userAdded
                ? `<button type="button" class="link-btn is-danger" data-cm="delete" data-id="${esc(g.id)}">Удалить</button>`
                : `<button type="button" class="link-btn is-danger" data-cm="hide" data-id="${esc(g.id)}">Скрыть</button>`}
            </div>
          </li>`).join('')}
        </ul>
        <h3 class="ce-h">Скрытые разделы и позиции</h3>
        ${hidden.length ? `<ul class="cm-list">${hidden.map(h => `
          <li class="cm-row" style="--d:0">
            <div class="cm-main"><span class="cm-name">${esc(h.name)}</span>
              <span class="cm-meta">${h.isGroup ? 'раздел · ' : ''}${esc(h.path || '')}</span></div>
            <div class="cm-actions"><button type="button" class="link-btn" data-cm="unhide" data-rec="${esc(h.id)}">Вернуть</button></div>
          </li>`).join('')}</ul>` : '<div class="f-hint">Скрытых нет.</div>'}`;
    };
    draw();
    const onChange = () => { if (R.isConnected) draw(); };
    if (window.TST) window.TST.on('fleet-changed', onChange);
    R.addEventListener('click', e => {
      const b = e.target.closest('[data-cm]');
      if (!b) return;
      const id = b.dataset.id;
      switch (b.dataset.cm) {
        case 'add-root': addCatalogSection(''); break;
        case 'add-section': addCatalogSection(id); break;
        case 'add-item': catalogModal({ parentId: id }); break;
        case 'rename': renameCatalogSection(id); break;
        case 'hide': hideCatalogNode(id); break;
        case 'delete': deleteCatalogSection(id); break;
        case 'unhide':
          DB.catalogEdits = DB.catalogEdits.filter(r => r.id !== b.dataset.rec);
          saveDB();
          toast('success', 'Возвращено в справочник', '');
          break;
      }
    });
  }

  /* ========================================================================
     КАРТА ТО ДЛЯ ПЕЧАТИ (чек-лист с отметками о выполнении)
     ======================================================================== */
  function printChecklist(catalogId, typeKey, vehicleId) {
    const c = CAT();
    const node = c && c.get(catalogId);
    const maint = c && c.maintenanceOf(catalogId);
    const reg = maint && maint[typeKey];
    const type = c && c.types.find(t => t.key === typeKey);
    if (!node || !reg || !type) { toast('warning', 'Нет норматива', 'Для этого вида ТО регламент не задан.'); return; }
```

```
  const v = vehicleId ? getVehicle(vehicleId) : null;
  const blank = '_____';
  const info = [
    ['Техническое средство', v ? v.name : blank],
    ['Позиция справочника', node.name],
    ['Гос. номер', v ? (v.regNumber || '—') : blank],
    ['Инвентарный / заводской номер', v ? `${v.invNumber || '—'} / ${v.serialNumber || '—'}` : blank],
    ['Подразделение', v ? (v.unit || '—') : blank],
    ['Показание счётчика при ТО, мото·ч', blank]
  ];
  const people = (reg.personnel || []);
  const body = `
    <table>${info.map(([k, val]) => `<tr><th style="width:34%">${esc(k)}</th><td>${esc(val)}</td></tr>`).join('')}</table>
    <h3 class="p-h">Перечень работ</h3>
    <table><thead><tr><th style="width:24pt">№</th><th>Операция</th><th style="width:70pt">Выполнено</th><th style="width:130pt">Примечание</th></tr></thead>
      <tbody>${(reg.actions || []).map((a, i) => `<tr><td>${i + 1}</td><td>${esc(a)}</td><td></td><td></td><td></td></tr>`).join('')}</tbody></table>
    ${people.length ? `<h3 class="p-h">Личный состав</h3>
    <table><thead><tr><th>Должность</th><th class="p-num" style="width:50pt">Кол-во</th><th>Воинское звание, фамилия</th><th style="width:80pt">Подпись</th></tr></thead>
      <tbody>${people.map(p => `<tr><td>${esc(p.role)}</td><td class="p-num">${esc(p.count)}</td><td></td><td></td><td></td></tr>`).join('')}</tbody></table>` : ''}
    ${(reg.tools || []).length ? `<h3 class="p-h">Инструмент и оснастка</h3>
    <p>${reg.tools.map((t, i) => `${i + 1}. ${esc(t)}`).join('; ')}</p>` : ''}
    ${(reg.materials || []).length ? `<h3 class="p-h">Расходные материалы</h3>
    <table><thead><tr><th style="width:24pt">№</th><th>Наименование</th><th style="width:100pt">Израсходовано</th></tr></thead>
      <tbody>${reg.materials.map((m, i) => `<tr><td>${i + 1}</td><td>${esc(m)}</td><td></td><td></td></tr>`).join('')}</tbody></table>` : ''}
    <p style="margin-top:8pt">Нормативная трудоёмкость: <b>${fmtNum(reg.labor, 1)} чел·ч</b>. ${reg.custom ? 'Норматив введён пользователем.' : 'Норматив — по учебному пособию.'}</p>`;
  printCustom(`КАРТА ТЕХНИЧЕСКОГО ОБСЛУЖИВАНИЯ · ${type.short}`, `${type.full} — ${node.name}`, body,
    '<span>Работы выполнил: _____</span><span>Принял: _____ «____» _____ 20____ г.</span>');
}

/* ============================================================================
   ЗАЯВКА НА МАТЕРИАЛЫ, ИНСТРУМЕНТ И СПЕЦИАЛИСТОВ
   ----------------------------------------------------------------------------
   Техника, у которой очередное ТО наступает в ближайшие N дней (или уже
   просрочено) → вид ТО по циклу → норматив справочника → сводная потребность
   ============================================================================ */
function needRows(days) {
  const end = addDays(todayISO(), days);
  return DB.vehicles.filter(v => v.status !== 'Списано').map(v => {
    const st = computeStatus(v);
    if (st.level === 'none' || st.level === 'retired') return null;
    let due = st.nextDate || null;
    // По моточасам: дата наступления — по среднесуточной наработке из плана
    const daily = Number(v.annualPlanHours || 0) > 0 ? Number(v.annualPlanHours) / 365 : null;
    if (st.remainingHours != null && daily) {
      const d = addDays(todayISO(), Math.max(0, Math.floor(st.remainingHours / daily)));
      if (!due || d < due) due = d;
    }
    if (st.level === 'overdue' || (st.remainingHours != null && st.remainingHours <= 0)) due = todayISO();
    if (!due || due > end) return null;
    const kind = nextMaintKind(v);
    const key = KIND_TO_CATALOG[kind];
    const maint = catMaintOf(v);
    return { v, st, due: due < todayISO() ? todayISO() : due, kind, reg: (maint && key) ? maint[key] || null : null };
  }).filter(Boolean).sort((a, b) => a.due.localeCompare(b.due));
}

function needSummary(rows) {
  const mats = new Map(), tools = new Map(), roles = new Map();
  let labor = 0;
  const add = (map, name) => { const k = String(name || '').trim(); if (k) map.set(k, (map.get(k) || 0) + 1); };
  rows.forEach(r => {
    if (!r.reg) return;
    labor += Number(r.reg.labor) || 0;
    (r.reg.materials || []).forEach(x => add(mats, x));
    (r.reg.tools || []).forEach(x => add(tools, x));
    (r.reg.personnel || []).forEach(p => {
      const e = roles.get(p.role) || { max: 0, total: 0, jobs: 0 };
      e.max = Math.max(e.max, Number(p.count) || 0); e.total += Number(p.count) || 0; e.jobs++;
      roles.set(p.role, e);
    });
  });
  const sorted = map => Array.from(map).sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0], 'ru'));
  return { labor, mats: sorted(mats), tools: sorted(tools),
           roles: Array.from(roles).sort((a, b) => b[1].max - a[1].max || a[0].localeCompare(b[0], 'ru')),
           noNorm: rows.filter(r => !r.reg) };
}

function renderNeedPanel() {
  const days = ui.docs.needDays || 30;
  const rows = needRows(days);
  const sum = needSummary(rows);
```

```
  return `
    <div class="f-panel">
      <div class="f-panel-title">Заявка на материалы, инструмент и специалистов</div>
      <p class="f-hint">Техника, у которой очередное ТО наступает в выбранный период или уже просрочено. Вид ТО — по отдельным срокам ТО-1 и ТО-2, нормативы — из справочника.</p>
      <div class="f-form-grid" style="margin:12px 0">
        <div><label class="f-fld-label" for="needDays">Период планирования, дней</label>
          <input id="needDays" class="f-input" type="number" min="1" max="366" value="${days}"></div>
        <div class="f-need-stats">
          <span><b>${rows.length}</b> ТО</span><span><b>${fmtNum(sum.labor, 1)}</b> чел·ч</span>
          <span><b>${sum.mats.length}</b> материалов</span><span><b>${sum.roles.length}</b> специальностей</span>
        </div>
      </div>
      ${rows.length ? `<div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table f-reg-table">
          <thead><tr><th>Техника</th><th>Вид ТО</th><th>Срок</th><th class="f-td-num">Чел·ч</th><th>Норматив</th></tr></thead>
          <tbody>${rows.map(r => `<tr>
            <td class="f-td-strong">${esc(r.v.name)}${r.v.regNumber ? ` <span class="f-td-muted">${esc(r.v.regNumber)}</span>` : ''}</td>
            <td class="f-td-nowrap">${esc(r.kind)}</td>
            <td class="f-td-nowrap"${r.st.level === 'overdue' ? ' style="color:var(--st-overdue-text);font-weight:600"' : ''}>${r.st.level === 'overdue' ? 'просрочено' : fmtDate(r.due)}</td>
            <td class="f-td-num">${r.reg ? fmtNum(r.reg.labor, 1) : '—'}</td>
            <td>${r.reg ? (r.reg.custom ? 'пользователя' : 'справочник')
              : `<button type="button" class="f-btn f-btn-sm" data-act="${r.v.catalogId ? 'edit-norm' : 'edit-vehicle'}" data-id="${esc(r.v.catalogId || r.v.id)}">${r.v.catalogId ? 'Ввести норматив' : 'Привязать'}</button>`}</td>
          </tr>`).join('')}</tbody>
        </table>
      </div></div>
      <div style="margin-top:12px">
        <button type="button" class="f-btn f-btn-primary" data-act="print-need">${FI.pdf(15)}<span>Сформировать заявку</span></button>
      </div>` : `<p class="f-hint">В выбранный период плановых ТО нет.</p>`}
    </div>`;
}

function printNeed() {
  const days = ui.docs.needDays || 30;
  const rows = needRows(days);
  if (!rows.length) { toast('info', 'Нет данных', 'В выбранный период плановых ТО нет.'); return; }
  const sum = needSummary(rows);
  const body = `
    <h3 class="p-h">1. График технического обслуживания</h3>
    <table><thead><tr><th style="width:24pt">№</th><th>Техника</th><th>Гос. номер</th><th>Вид ТО</th><th>Срок</th><th class="p-num">Трудоёмкость, чел·ч</th></tr></thead>
      <tbody>${rows.map((r, i) => `<tr><td>${i + 1}</td><td>${esc(r.v.name)}</td><td>${esc(r.v.regNumber || '—')}</td>
        <td>${esc(r.kind)}</td><td>${r.st.level === 'overdue' ? 'просрочено' : fmtDate(r.due)}</td>
        <td class="p-num">${r.reg ? fmtNum(r.reg.labor, 1) : 'нет норматива'}</td></tr>`).join('')}
        <tr><td colspan="5"><b>Итого</b></td><td class="p-num"><b>${fmtNum(sum.labor, 1)}</b></td></tr></tbody></table>
    ${sum.mats.length ? `<h3 class="p-h">2. Расходные материалы</h3>
    <table><thead><tr><th style="width:24pt">№</th><th>Наименование</th><th class="p-num" style="width:90pt">Для ТО, ед. техники</th><th style="width:110pt">Потребное количество</th></tr></thead>
      <tbody>${sum.mats.map(([n, k], i) => `<tr><td>${i + 1}</td><td>${esc(n)}</td><td class="p-num">${k}</td><td></td></tr>`).join('')}</tbody></table>` : ''}
    ${sum.roles.length ? `<h3 class="p-h">3. Специалисты</h3>
    <table><thead><tr><th>Специальность</th><th class="p-num">Требуется одновременно, чел.</th><th class="p-num">Участие в ТО, чел.×ТО</th></tr></thead>
      <tbody>${sum.roles.map(([r, e]) => `<tr><td>${esc(r)}</td><td class="p-num">${e.max}</td><td class="p-num">${e.total}</td></tr>`).join('')}</tbody></table>` : ''}
    ${sum.tools.length ? `<h3 class="p-h">4. Инструмент и оснастка</h3>
    <table><thead><tr><th style="width:24pt">№</th><th>Наименование</th><th class="p-num" style="width:90pt">Для ТО, ед. техники</th></tr></thead>
      <tbody>${sum.tools.map(([n, k], i) => `<tr><td>${i + 1}</td><td>${esc(n)}</td><td class="p-num">${k}</td></tr>`).join('')}</tbody></table>` : ''}
    <p style="margin-top:8pt;font-size:9pt">Нормативы справочника не содержат норм расхода материалов — графа «Потребное количество» заполняется ответственным лицом.${
      sum.noNorm.length ? ` Без норматива ТО (в потребность не вошли): ${sum.noNorm.map(r => esc(r.v.name)).join(', ')}.` : ''}</p>`;
  printCustom('ЗАЯВКА', `на обеспечение технического обслуживания техники в период с ${fmtDate(todayISO())} по ${fmtDate(addDays(todayISO(), days))}`,
    body, '<span>Начальник службы: _____</span><span>«____» _____ 20____ г.</span>');
}

/* --- G. Акты --- */
function printActWriteoff() {
  const rows = resourceSummary().filter(r => r.normSet && r.exceeded);
  const body = rows.length ? `<table>
    <thead><tr><th>Техника</th><th>Группа</th><th class="p-num">Годовая норма</th>
    <th class="p-num">Разрешено (за вычетом 5%)</th><th class="p-num">Израсходовано</th><th class="p-num">Перерасход</th></tr></thead>
    <tbody>${rows.map(r => `<tr><td>${esc(r.v.name)}</td><td>${esc(r.v.expGroup || '—')}</td>
      <td class="p-num">${fmtNum(r.norm,0)}</td><td class="p-num">${fmtNum(r.allowed,0)}</td>
      <td class="p-num">${fmtNum(r.used,0)}</td><td class="p-num">${fmtNum(r.used - r.allowed,0)}</td></tr>`).join('')}
    </tbody></table>` : `<p>Техники с перерасходом годовой нормы моторесурса не выявлено.</p>`;
  printCustom('АКТ',
    'о списании моторесурса, израсходованного сверх годовой нормы',
```

```
      body + `<p style="margin-top:14pt">Комиссия в составе: председатель — _____, члены комиссии — _____, _____, рассмотрев данные учёта моторесурса, составила настоящий акт.</p>`,
      `<span>Председатель комиссии: _____</span><span>«____» _____ 20____ г.</span>`);
  }

  function printActInspection() {
    const rows = DB.vehicles.filter(v => v.status !== 'Списано');
    const body = `<table>
      <thead><tr><th style="width:26pt">№</th><th>Техника</th><th>Гос. номер</th><th>Подразделение</th>
      <th class="p-num">Моточасы</th><th>Статус по учёту</th><th>В наличии (отметка)</th></tr></thead>
      <tbody>${rows.map((v, i) => `<tr><td>${i + 1}</td><td>${esc(v.name)}</td><td>${esc(v.regNumber || '—')}</td>
        <td>${esc(v.unit || '—')}</td><td class="p-num">${fmtNum(v.currentHours,0)}</td><td>${esc(v.status)}</td><td></td></tr>`).join('')}
      </tbody></table>`;
    printCustom('АКТ', 'внезапной проверки наличия технических средств тыла', body,
      `<span>Проверку провёл: _____</span><span>«____» _____ 20____ г.</span>`);
  }

  function printActUsage(from, to) {
    const map = new Map();
    DB.trips.filter(t => (!from || t.date >= from) && (!to || t.date <= to)).forEach(t => {
      if (!map.has(t.vehicleId)) map.set(t.vehicleId, { trips: 0, run: 0 });
      const r = map.get(t.vehicleId);
      r.trips++;
      r.run += Number(t.endHours || 0) - Number(t.startHours || 0);
    });
    const rows = DB.vehicles.filter(v => map.has(v.id));
    const body = rows.length ? `<table>
      <thead><tr><th>Техника</th><th class="p-num">Выходов за период</th><th class="p-num">Наработка, мото·ч</th>
      <th class="p-num">Расход ГСМ по норме</th><th class="p-num">Расход ГСМ фактически</th></tr></thead>
      <tbody>${rows.map(v => `<tr><td>${esc(v.name)}</td><td class="p-num">${map.get(v.id).trips}</td>
        <td class="p-num">${fmtNum(map.get(v.id).run,0)}</td><td class="p-num"></td><td class="p-num"></td></tr>`).join('')}
      </tbody></table>` : `<p>За выбранный период путевые листы не найдены.</p>`;
    printCustom('АКТ', `проверки использования техники и расхода ГСМ за период с ${fmtDate(from)} по ${fmtDate(to)}`,
      body + `<p style="margin-top:10pt;font-size:9pt">Графы «Расход ГСМ» заполняются вручную — учёт горюче-смазочных материалов программой не ведётся.</p>`,
      `<span>Председатель комиссии: _____</span><span>«____» _____ 20____ г.</span>`);
  }

  function renderDocuments() {
    const monthRows = monthPlanRows();
    return `
      <div class="f-page-head">
        <div>
          <div class="f-page-title" role="heading" aria-level="1">Документы</div>
          <div class="f-page-sub">Печатные формы по методике планирования эксплуатации и ремонта ВВСТ (ВАМТО, 2025)</div>
        </div>
      </div>

      <div class="f-panel">
        <div class="f-panel-title">Годовой план эксплуатации и ремонта</div>
        <p class="f-hint">Формируется на основе плановой годовой наработки и нормативов из раздела «Планирование → Потребность в ТО».</p>
        <button type="button" class="f-btn f-btn-primary" data-act="print-annual-plan">${FI.pdf(15)}<span>Сформировать и распечатать</span></button>
      </div>

      <div class="f-panel">
        <div class="f-panel-title">План-график технического обслуживания и ремонта на месяц</div>
        <p class="f-hint">Расчётная дата очередного ТО-1 — по формуле Д<sub>ТО</sub> = (L<sub>ТО-1</sub> − L<sub>ф</sub>)/L<sub>сс</sub>, где L<sub>сс</sub> — среднесуточная наработка (плановая годовая наработка / 365). Показания — в мото·ч.</p>
        ${renderMonthPlanTable(monthRows)}
        <div style="margin-top:12px">
          <button type="button" class="f-btn f-btn-primary" data-act="print-month-plan">${FI.pdf(15)}<span>Сформировать и распечатать</span></button>
        </div>
      </div>

      ${renderNeedPanel()}

      <div class="f-panel">
        <div class="f-panel-title">Акты</div>
        <div class="f-form-grid" style="margin-bottom:12px">
          <div><label class="f-fld-label" for="actFrom">Период с</label>
            <input id="actFrom" class="f-input" type="date" value="${esc(ui.docs.actFrom)}"></div>
          <div><label class="f-fld-label" for="actTo">Период по</label>
            <input id="actTo" class="f-input" type="date" value="${esc(ui.docs.actTo)}"></div>
        </div>
        <div style="display:flex;gap:8px;flex-wrap:wrap">
          <button type="button" class="f-btn" data-act="print-act-writeoff">${FI.pdf(15)}<span>Акт списания моторесурса</span></button>
          <button type="button" class="f-btn" data-act="print-act-inspection">${FI.pdf(15)}<span>Акт внезапной проверки наличия техники</span></button>
          <button type="button" class="f-btn" data-act="print-act-usage">${FI.pdf(15)}<span>Акт проверки использования техники и расхода ГСМ</span></button>
        </div>
```

```
      </div>`;
  }

  function bindDocuments() {
    const bind = (id, prop) => {
      const n = el(id);
      if (n) n.addEventListener('change', () => { ui.docs[prop] = n.value; });
    };
    bind('actFrom', 'actFrom');
    bind('actTo', 'actTo');
    const nd = el('needDays');
    if (nd) nd.addEventListener('change', () => {
      const n = parseInt(nd.value, 10);
      ui.docs.needDays = isFinite(n) ? clamp(n, 1, 366) : 30;
      renderView();
    });
  }

  /* ============================================================================
     18. ОТЧЁТЫ
     ----------------------------------------------------------------------------
     Единый реестр: описание колонок + функция строк. Из него формируются
     и предпросмотр, и выгрузка в Excel, и печать в PDF.
     ============================================================================ */
  const levelText = lvl => LEVELS[lvl].emoji + ' ' + LEVELS[lvl].label;

  const REPORTS = [
    {
      key: 'vehicles', title: 'Список техники', file: 'spisok_tehniki', sheet: 'Список техники',
      desc: 'Полный реестр зарегистрированных технических средств тыла.', icon: FI.truck,
      columns: [
        { key: 'name', label: 'Наименование', width: 34 },
        { key: 'brand', label: 'Марка и модель', width: 24 },
        { key: 'regNumber', label: 'Гос. номер', width: 14 },
        { key: 'invNumber', label: 'Инв. №', width: 13 },
        { key: 'serialNumber', label: 'Зав. №', width: 15 },
        { key: 'unit', label: 'Подразделение', width: 20 },
        { key: 'responsible', label: 'Ответственный', width: 22 },
        { key: 'expGroup', label: 'Группа эксплуатации', width: 18 },
        { key: 'year', label: 'Год выпуска', width: 11, num: true },
        { key: 'regDate', label: 'Дата учёта', width: 13 },
        { key: 'status', label: 'Статус', width: 16 },
        { key: 'hours', label: 'Моточасы', width: 11, num: true },
        { key: 'level', label: 'Состояние ТО', width: 19 },
        { key: 'next', label: 'Следующее ТО', width: 15 },
        { key: 'note', label: 'Примечание', width: 30 }
      ],
      rows() {
        return DB.vehicles.map(v => {
          const st = computeStatus(v);
          return {
            name: v.name, brand: v.brand, regNumber: v.regNumber, invNumber: v.invNumber,
            serialNumber: v.serialNumber, unit: v.unit, responsible: v.responsible,
            expGroup: v.expGroup || '', year: v.year ? Number(v.year) : '', regDate: fmtDate(v.regDate), status:
  v.status,
            hours: Number(v.currentHours || 0), level: levelText(st.level),
            next: st.nextDate ? fmtDate(st.nextDate) : (st.nextHours != null ? fmtNum(st.nextHours, 0) + ' мото·ч' :
  '—'),
            note: v.note
          };
        });
      }
    },
    {
      key: 'maint', title: 'История технического обслуживания', file: 'istoriya_to', sheet: 'История ТО',
      desc: 'Журнал всех проведённых обслуживаний с фильтром по периоду и подразделению.',
      icon: FI.wrench, supportsRange: true, supportsUnit: true,
      columns: [
        { key: 'vehicle', label: 'Техника', width: 32 },
        { key: 'regNumber', label: 'Гос. номер', width: 14 },
        { key: 'kind', label: 'Вид ТО', width: 24 },
        { key: 'date', label: 'Дата проведения', width: 15 },
        { key: 'hours', label: 'Моточасы', width: 11, num: true },
        { key: 'executor', label: 'Исполнитель', width: 22 },
        { key: 'unit', label: 'Подразделение', width: 20 },
        { key: 'works', label: 'Выполненные работы', width: 42 },
        { key: 'parts', label: 'Запасные части', width: 32 },
        { key: 'cost', label: 'Стоимость, ₽', width: 14, num: true },
        { key: 'docs', label: 'Документы', width: 26 },
        { key: 'note', label: 'Примечание', width: 28 }
      ],
      rows() {
        const f = ui.report;
        return DB.maint
          .filter(m => (!f.from || m.date >= f.from) && (!f.to || m.date <= f.to))
          .filter(m => !f.unit || m.unit === f.unit)
          .sort((a, b) => (b.date || '').localeCompare(a.date || ''))
          .map(m => {
            const v = getVehicle(m.vehicleId);
            return {
```

```
              vehicle: v ? v.name : '—', regNumber: v ? v.regNumber : '',
              kind: m.kind, date: fmtDate(m.date), hours: Number(m.hours || 0),
              executor: m.executor, unit: m.unit, works: m.works, parts: m.parts,
              cost: m.cost ? Number(m.cost) : '',
              docs: (m.docs || []).map(d => d.name).join('; '),
              note: m.note
            };
          });
      }
    },
    {
      key: 'overdue', title: 'Просроченные ТО', file: 'prosrochennye_to', sheet: 'Просроченные ТО',
      desc: 'Техника, у которой нарушен установленный срок обслуживания.', icon: FI.alert,
      columns: [
        { key: 'name', label: 'Наименование', width: 34 },
        { key: 'regNumber', label: 'Гос. номер', width: 14 },
        { key: 'unit', label: 'Подразделение', width: 20 },
        { key: 'responsible', label: 'Ответственный', width: 22 },
        { key: 'last', label: 'Последнее ТО', width: 15 },
        { key: 'planned', label: 'Плановая дата ТО', width: 16 },
        { key: 'overdueDays', label: 'Просрочка, дней', width: 15, num: true },
        { key: 'overHours', label: 'Перерасход, мото·ч', width: 18, num: true }
      ],
      rows() {
        return DB.vehicles
          .map(v => ({ v, st: computeStatus(v) }))
          .filter(({ st }) => st.level === 'overdue')
          .map(({ v, st }) => ({
            name: v.name, regNumber: v.regNumber, unit: v.unit, responsible: v.responsible,
            last: st.lastMaint ? fmtDate(st.lastMaint.date) : '—',
            planned: st.nextDate ? fmtDate(st.nextDate) : '—',
            overdueDays: (st.remainingDays != null && st.remainingDays < 0) ? Math.abs(st.remainingDays) : '',
            overHours: (st.remainingHours != null && st.remainingHours < 0) ? Math.abs(st.remainingHours) : ''
          }));
      }
    },
    {
      key: 'due', title: 'Техника, требующая обслуживания', file: 'trebuet_to', sheet: 'Требует ТО',
      desc: 'Единицы с остатком ресурса менее установленного порога.', icon: FI.clock,
      columns: [
        { key: 'name', label: 'Наименование', width: 34 },
        { key: 'regNumber', label: 'Гос. номер', width: 14 },
        { key: 'unit', label: 'Подразделение', width: 20 },
        { key: 'responsible', label: 'Ответственный', width: 22 },
        { key: 'level', label: 'Состояние', width: 19 },
        { key: 'pct', label: 'Остаток ресурса, %', width: 18, num: true },
        { key: 'hours', label: 'Осталось мото·ч', width: 16, num: true },
        { key: 'days', label: 'Осталось дней', width: 14, num: true },
        { key: 'planned', label: 'Плановая дата ТО', width: 16 }
      ],
      rows() {
        return DB.vehicles
          .filter(v => v.status !== 'Списано')
          .map(v => ({ v, st: computeStatus(v) }))
          .filter(({ st }) => ['overdue', 'urgent', 'soon'].indexOf(st.level) !== -1)
          .sort((a, b) => (a.st.pct || 0) - (b.st.pct || 0))
          .map(({ v, st }) => ({
            name: v.name, regNumber: v.regNumber, unit: v.unit, responsible: v.responsible,
            level: levelText(st.level),
            pct: Math.round(Math.max(0, st.pct)),
            hours: st.remainingHours != null ? st.remainingHours : '',
            days: st.remainingDays != null ? st.remainingDays : '',
            planned: st.nextDate ? fmtDate(st.nextDate) : '—'
          }));
      }
    },
    {
      key: 'byUnit', title: 'Статистика по подразделениям', file: 'statistika_podrazdeleniya', sheet: 'По подразделениям',
      desc: 'Сводные показатели парка и обслуживания в разрезе подразделений.', icon: FI.building,
      columns: [
        { key: 'unit', label: 'Подразделение', width: 30 },
        { key: 'count', label: 'Единиц техники', width: 15, num: true },
        { key: 'active', label: 'В эксплуатации', width: 15, num: true },
        { key: 'overdue', label: 'Просрочено ТО', width: 15, num: true },
        { key: 'maintCount', label: 'Записей ТО', width: 13, num: true },
        { key: 'hours', label: 'Суммарно мото·ч', width: 17, num: true },
        { key: 'cost', label: 'Затраты на ТО, ₽', width: 17, num: true }
      ],
      rows() {
        const map = new Map();
        DB.vehicles.forEach(v => {
          const key = v.unit || 'Не указано';
          if (!map.has(key)) map.set(key, { unit: key, count: 0, active: 0, overdue: 0, maintCount: 0, hours: 0, cost: 0 });
          const r = map.get(key);
          r.count++;
          if (v.status === 'В эксплуатации') r.active++;
          if (computeStatus(v).level === 'overdue') r.overdue++;
          r.hours += Number(v.currentHours || 0);
```

```
          const ms = DB.maint.filter(m => m.vehicleId === v.id);
          r.maintCount += ms.length;
          r.cost += ms.reduce((s, m) => s + Number(m.cost || 0), 0);
        });
        return Array.from(map.values()).sort((a, b) => a.unit.localeCompare(b.unit, 'ru'));
      }
    },
    {
      key: 'byHours', title: 'Статистика по моточасам', file: 'statistika_motochasy', sheet: 'По моточасам',
      desc: 'Наработка техники и остаточный ресурс до следующего обслуживания.', icon: FI.gauge,
      columns: [
        { key: 'name', label: 'Наименование', width: 34 },
        { key: 'regNumber', label: 'Гос. номер', width: 14 },
        { key: 'unit', label: 'Подразделение', width: 20 },
        { key: 'current', label: 'Текущие мото·ч', width: 15, num: true },
        { key: 'since', label: 'С последнего ТО', width: 16, num: true },
        { key: 'remaining', label: 'Остаток до ТО', width: 15, num: true },
        { key: 'interval', label: 'Интервал, мото·ч', width: 17, num: true },
        { key: 'maintCount', label: 'Записей ТО', width: 13, num: true },
        { key: 'updated', label: 'Моточасы обновлены', width: 19 }
      ],
      rows() {
        return DB.vehicles.map(v => {
          const st = computeStatus(v);
          return {
            name: v.name, regNumber: v.regNumber, unit: v.unit,
            current: Number(v.currentHours || 0),
            since: st.hoursSince,
            remaining: st.remainingHours != null ? st.remainingHours : '',
            interval: (v.interval && v.interval.byHours) ? Number(v.interval.hours) : '',
            maintCount: DB.maint.filter(m => m.vehicleId === v.id).length,
            updated: v.hoursUpdatedAt ? fmtDate(v.hoursUpdatedAt) : '—'
          };
        }).sort((a, b) => b.current - a.current);
      }
    },
    {
      key: 'trips', title: 'Путевые листы', file: 'putevye_listy', sheet: 'Путевые листы',
      desc: 'Журнал ежедневных выходов техники с показаниями счётчика.',
      icon: FI.route, supportsRange: true,
      columns: [
        { key: 'date', label: 'Дата', width: 12 },
        { key: 'vehicle', label: 'Техника', width: 32 },
        { key: 'regNumber', label: 'Гос. номер', width: 14 },
        { key: 'driver', label: 'Водитель', width: 24 },
        { key: 'purpose', label: 'Маршрут / задача', width: 34 },
        { key: 'start', label: 'Показание на начало', width: 18, num: true },
        { key: 'end', label: 'Показание на конец', width: 18, num: true },
        { key: 'run', label: 'Наработка за выход', width: 17, num: true },
        { key: 'note', label: 'Примечание', width: 24 }
      ],
      rows() {
        const f = ui.report;
        return DB.trips
          .filter(t => (!f.from || t.date >= f.from) && (!f.to || t.date <= f.to))
          .sort((a, b) => (b.date || '').localeCompare(a.date || ''))
          .map(t => {
            const v = getVehicle(t.vehicleId);
            return {
              date: fmtDate(t.date), vehicle: v ? v.name : '—', regNumber: v ? v.regNumber : '',
              driver: t.driver, purpose: t.purpose,
              start: Number(t.startHours || 0), end: Number(t.endHours || 0),
              run: Number(t.endHours || 0) - Number(t.startHours || 0), note: t.note
            };
          });
      }
    },
    {
      key: 'tripsMonthly', title: 'Карточка учёта работы машины', file: 'kartochka_ucheta_raboty', sheet: 'Учёт работы машин',
      desc: 'Помесячная сводка наработки и количества выходов по каждой единице техники (по путевым листам).',
      icon: FI.gauge, supportsRange: true, supportsUnit: true,
      columns: [
        { key: 'month', label: 'Месяц', width: 10 },
        { key: 'vehicle', label: 'Техника', width: 32 },
        { key: 'regNumber', label: 'Гос. номер', width: 14 },
        { key: 'unit', label: 'Подразделение', width: 20 },
        { key: 'trips', label: 'Количество выходов', width: 16, num: true },
        { key: 'run', label: 'Наработка за месяц, мото·ч', width: 20, num: true }
      ],
      rows() {
        const f = ui.report;
        const map = new Map();
        DB.trips
          .filter(t => (!f.from || t.date >= f.from) && (!f.to || t.date <= f.to))
          .forEach(t => {
            const v = getVehicle(t.vehicleId);
            if (f.unit && (!v || v.unit !== f.unit)) return;
            const month = (t.date || '').slice(0, 7);
            const key = month + '|' + t.vehicleId;
```

```
            if (!map.has(key)) map.set(key, {
              month, vehicle: v ? v.name : '—', regNumber: v ? v.regNumber : '',
              unit: v ? v.unit : '', trips: 0, run: 0
            });
            const r = map.get(key);
            r.trips++;
            r.run += Number(t.endHours || 0) - Number(t.startHours || 0);
          });
        return Array.from(map.values()).sort((a, b) => b.month.localeCompare(a.month) || a.vehicle.localeCompare(b.vehicle, 'ru'));
      }
    }
  ];

  const getReport = key => REPORTS.find(r => r.key === key) || REPORTS[0];

  function renderReports() {
    const rep = getReport(ui.report.key);
    const rows = rep.rows();
    const preview = rows.slice(0, 50);

    return `
      <div class="f-page-head">
        <div>
          <div class="f-page-title" role="heading" aria-level="1">Отчёты</div>
          <div class="f-page-sub">Формирование и выгрузка отчётных документов в Excel и PDF</div>
        </div>
      </div>

      <div class="f-report-list">
        ${REPORTS.map(r => `
          <div class="f-report-card ${r.key === ui.report.key ? 'is-active' : ''}" data-act="report" data-key="${r.key}">
            <div class="f-report-ico">${r.icon(18)}</div>
            <div><div class="f-report-name">${esc(r.title)}</div>
                 <div class="f-report-desc">${esc(r.desc)}</div></div>
          </div>`).join('')}
      </div>

      ${(rep.supportsRange || rep.supportsUnit) ? `
        <div class="f-toolbar">
          ${rep.supportsRange ? `
            <div class="f-fld"><label class="f-fld-label" for="repFrom">Период с</label>
              <input id="repFrom" class="f-input" type="date" value="${esc(ui.report.from)}"></div>
            <div class="f-fld"><label class="f-fld-label" for="repTo">Период по</label>
              <input id="repTo" class="f-input" type="date" value="${esc(ui.report.to)}"></div>` : ''}
          ${rep.supportsUnit ? `
            <div class="f-fld"><label class="f-fld-label" for="repUnit">Подразделение</label>
              <select id="repUnit" class="f-select">${options(uniqueValues('unit'), ui.report.unit, 'Все')}</select></div>` : ''}
          <div class="f-toolbar-end">
            <button type="button" class="f-btn f-btn-ghost" data-act="reset-report">Сбросить</button>
          </div>
        </div>` : ''}

      <div class="f-page-head" style="margin-bottom:14px">
        <div>
          <div style="font-size:17px;font-weight:620">${esc(rep.title)}</div>
          <div class="f-page-sub">Записей: ${rows.length}${rows.length > 50 ? ' · в предпросмотре показаны первые 50' : ''}</div>
        </div>
        <div class="f-page-actions">
          <button type="button" class="f-btn" data-act="export-xlsx" ${!rows.length ? 'disabled' : ''}>${FI.excel(15)}<span>Excel</span></button>
          <button type="button" class="f-btn" data-act="export-pdf"  ${!rows.length ? 'disabled' : ''}>${FI.pdf(15)}<span>PDF</span></button>
        </div>
      </div>

      <div id="reportPreview">
        ${!rows.length
          ? emptyState(FI.chart(28), 'Нет данных', 'По выбранным условиям записей не найдено.')
          : `<div class="f-table-wrap"><div class="f-table-scroll">
              <table class="f-table f-table-rep${rep.columns.length > 9 ? ' is-wide' : ''}">
                <thead><tr>${rep.columns.map(c => `<th class="${c.num ? 'f-td-num' : ''}">${esc(c.label)}</th>`).join('')}</tr></thead>
                <tbody>${preview.map(r => `<tr>${rep.columns.map(c => {
                  const v = r[c.key];
                  const disp = (v === '' || v == null) ? '–' : (c.num && typeof v === 'number' ? fmtNum(v) : v);
                  return `<td class="${c.num ? 'f-td-num' : ''}" data-label="${esc(c.label)}">${esc(disp)}</td>`;
                }).join('')}</tr>`).join('')}</tbody>
              </table>
            </div></div>`}
      </div>`;
  }

  function bindReportFilters() {
    const bind = (id, prop) => {
      const n = el(id);
      if (n) n.addEventListener('change', () => { ui.report[prop] = n.value; renderView(); });
```

```
  };
  bind('repFrom', 'from');
  bind('repTo', 'to');
  bind('repUnit', 'unit');
}

/* ========================================================================
   19. НАСТРОЙКИ
   ======================================================================== */
function renderSettings() {
  const s = DB.settings;
  const demo = DB.vehicles.some(v => v.demo);
  return `
    <div class="f-page-head">
      <div>
        <div class="f-page-title" role="heading" aria-level="1">Настройки</div>
        <div class="f-page-sub">Интервалы обслуживания, пороги индикации и управление данными</div>
      </div>
    </div>

    <div class="f-panel">
      <div class="f-panel-title">Интервалы технического обслуживания по умолчанию</div>
      <div class="f-form-grid">
        <div><label class="f-fld-label" for="setHours">Периодичность по моточасам</label>
          <input id="setHours" class="f-input" type="number" min="1" step="1" value="${s.defaultHours}">
          <div class="f-hint">Например: 250 или 500 мото·ч</div></div>
        <div><label class="f-fld-label" for="setDays">Периодичность по календарю, дней</label>
          <input id="setDays" class="f-input" type="number" min="1" step="1" value="${s.defaultDays}">
          <div class="f-hint">Например: 180 дней (полугодие)</div></div>
      </div>
      <div class="f-hint" style="margin-top:8px">Это интервалы ТО-1. Срок ТО-2 считается отдельно: по умолчанию каждые ${to2Every()} интервала ТО-1 (соотношение периодичностей ТО-2 / ТО-1 в «Планировании»); в карточке техники можно задать свой.</div>
      <div style="margin-top:14px">
        <button type="button" class="f-btn" data-act="apply-intervals">Применить ко всей технике</button>
      </div>
    </div>

    ${renderSeasonSettings()}

    <div class="f-panel">
      <div class="f-panel-title">Пороги цветовой индикации</div>
      <div class="f-form-grid">
        <div><label class="f-fld-label" for="setWarn"><span class="f-dot f-lvl-soon"></span> Жёлтый — остаток менее, %</label>
          <input id="setWarn" class="f-input" type="number" min="1" max="99" value="${s.warnPct}"></div>
        <div><label class="f-fld-label" for="setUrgent"><span class="f-dot f-lvl-urgent"></span> Оранжевый — остаток менее, %</label>
          <input id="setUrgent" class="f-input" type="number" min="1" max="99" value="${s.urgentPct}"></div>
        <div><label class="f-fld-label" for="setStale">Напоминать об обновлении моточасов через, дней</label>
          <input id="setStale" class="f-input" type="number" min="1" value="${s.hoursStaleDays}"></div>
        <div style="display:flex;align-items:flex-end">
          <label class="f-check"><input id="setNotify" type="checkbox" ${s.notifyOnStart ? 'checked' : ''}>
            Показывать сводку уведомлений при открытии раздела</label>
        </div>
      </div>
      <div class="f-hint" style="margin-top:10px">□ Красный устанавливается автоматически при исчерпании ресурса (остаток ≤ 0%).</div>
    </div>

    <div class="f-panel">
      <div class="f-panel-title">Сохранность данных</div>
      ${(() => {
        const b = backupInfo();
        return `<div class="f-backup-state ${b.due ? 'is-due' : ''}">
          ${FI.save(18)}
          <div><b>${esc(backupAgoText(b).replace(/^./, c => c.toUpperCase()))}</b>
            <div class="f-hint" style="margin:2px 0 0">Копия — ZIP-архив: все записи, нормативы, настройки и прикреплённые файлы. Храните её вне этого компьютера.</div></div>
        </div>`;
      })()}
      <div class="f-info-grid" id="setStorageStatus" style="margin:14px 0"></div>
      <div style="display:flex;gap:8px;flex-wrap:wrap">
        <button type="button" class="f-btn f-btn-primary" data-act="backup-export">${FI.download(15)}<span>Сохранить копию (ZIP)</span></button>
        <button type="button" class="f-btn" data-act="backup-import">${FI.upload(15)}<span>Восстановить из копии</span></button>
        <button type="button" class="f-btn" data-act="persist-request" id="setPersistBtn" hidden>Включить постоянное хранение</button>
      </div>
      <div class="f-form-grid" style="margin-top:14px">
        <div><label class="f-fld-label" for="setBackupDays">Напоминать о копии, если последней больше, дней</label>
          <input id="setBackupDays" class="f-input" type="number" min="1" max="365" value="${s.backupRemindDays}"></div>
        <div><label class="f-fld-label" for="setTrashDays">Хранить удалённое в корзине, дней</label>
          <input id="setTrashDays" class="f-input" type="number" min="1" max="365" value="${s.trashDays}"></div>
      </div>
      <div class="f-hint" style="margin-top:10px">Точки восстановления, корзина, журнал изменений и проверка целостности — в разделе
        <button type="button" class="link-btn" data-act="nav" data-key="safety">«Сохранность данных»</button>.</div>
```

```
        <div class="f-hint" style="margin-top:10px">
          ${NATIVE() ? 'Данные хранятся в файлах в папке данных приложения на этом компьютере (Сохранность данных → Файлы данных приложения).' : 'Данные хранятся только в этом браузере на этом компьютере. Программу можно открывать в нескольких вкладках — изменения синхронизируются между ними автоматически.'}
        </div>
      </div>

      <div class="f-panel">
        <div class="f-panel-title">Управление данными</div>
        <div style="display:flex;gap:8px;flex-wrap:wrap">
          ${demo ? `<button type="button" class="f-btn" data-act="seed-remove">Удалить демо-данные</button>` : ''}
          <button type="button" class="f-btn f-btn-danger" data-act="wipe">${FI.trash(15)}<span>Очистить все данные</span></button>
        </div>
        <div class="f-hint" style="margin-top:10px">Единиц техники: ${DB.vehicles.length} · Записей ТО: ${DB.maint.length} · Записей моточасов: ${DB.hours.length} · Своих нормативов ТО: ${DB.norms.length}</div>
      </div>`;
  }

  function bindSettings() {
    const num = (id, prop, lo, hi) => {
      const n = el(id);
      if (!n) return;
      n.addEventListener('change', () => {
        const v = parseInt(n.value, 10);
        if (isNaN(v) || v < lo || (hi && v > hi)) { n.value = DB.settings[prop]; return; }
        // Оранжевый порог должен быть строже жёлтого, иначе расчёт состояния ТО становится некорректным
        if (prop === 'warnPct'   && v <= DB.settings.urgentPct) { n.value = DB.settings[prop]; toast('warning', 'Некорректное значение', 'Жёлтый порог должен быть больше оранжевого.'); return; }
        if (prop === 'urgentPct' && v >= DB.settings.warnPct)   { n.value = DB.settings[prop]; toast('warning', 'Некорректное значение', 'Оранжевый порог должен быть меньше жёлтого.'); return; }
        DB.settings[prop] = v;
        saveDB(); renderNav();
        toast('success', 'Настройки сохранены', '');
      });
    };
    num('setHours', 'defaultHours', 1);
    num('setDays', 'defaultDays', 1);
    num('setWarn', 'warnPct', 1, 99);
    num('setUrgent', 'urgentPct', 1, 99);
    num('setStale', 'hoursStaleDays', 1);
    num('setBackupDays', 'backupRemindDays', 1, 365);
    num('setTrashDays', 'trashDays', 1, 365);
    bindSeasonSettings();
    fillStorageStatus();
    refreshStorageInfo();

    const chk = el('setNotify');
    if (chk) chk.addEventListener('change', () => { DB.settings.notifyOnStart = chk.checked; saveDB(); });
  }

  /* ============================================================================
     20. ФОРМА КАРТОЧКИ ТЕХНИКИ
     ============================================================================ */
  /* <option> для выбора позиции справочника, сгруппированные по службам */
  function catalogOptions(selected) {
    const c = CAT();
    const groups = new Map();
    (c ? c.leaves() : []).forEach(l => {
      if (!groups.has(l.service)) groups.set(l.service, []);
      groups.get(l.service).push(l);
    });
    return '<option value="">— не привязано —</option>' +
      Array.from(groups).map(([svc, list]) => `<optgroup label="${esc(svc)}">${list.map(l =>
        `<option value="${esc(l.id)}"${l.id === selected ? ' selected' : ''}>${esc(l.name)}${c.maintenanceOf(l.id) ? '' : ' (без регламента ТО)'}</option>`
      ).join('')}</optgroup>`).join('');
  }

  function vehicleModal(vehicle, prefill) {
    const isNew = !vehicle;
    const v = vehicle || Object.assign({
      catalogId: '',
      name: '', brand: '', regNumber: '', invNumber: '', serialNumber: '',
      unit: '', responsible: '', year: '', regDate: todayISO(),
      status: 'В эксплуатации', note: '', currentHours: 0,
      expGroup: '', annualPlanHours: '', annualNormHours: '',
      interval: { byHours: true, hours: DB.settings.defaultHours, byDate: true, days: DB.settings.defaultDays }
    }, prefill || {});
    const iv = v.interval || {};

    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">${isNew ? 'Новая единица техники' : 'Редактирование карточки'}</div>
          <div class="f-modal-sub">Электронная карточка технического средства тыла</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
```

```
        <div class="f-form-grid">
          ${CAT() ? `<div class="f-span2">
            <label class="f-fld-label" for="vCatalog">Тип по справочнику</label>
            <select id="vCatalog" class="f-select">${catalogOptions(v.catalogId || '')}</select>
            <div class="f-hint">Связывает карточку с регламентом ТО: нормативы, перечни работ и материалов. Определяется автоматически по наименованию или марке.</div>
          </div>` : ''}
          <div class="f-span2">
            <label class="f-fld-label" for="vName">Наименование техники *</label>
            <input id="vName" class="f-input" value="${esc(v.name)}" placeholder="Кухня автомобильная КА-220">
            <div class="f-error" id="errName">Укажите наименование техники</div>
          </div>
          <div><label class="f-fld-label" for="vBrand">Марка и модель</label>
            <input id="vBrand" class="f-input" value="${esc(v.brand)}" placeholder="КамАЗ-4310 / КА-220"></div>
          <div><label class="f-fld-label" for="vReg">Государственный регистрационный номер</label>
            <input id="vReg" class="f-input" value="${esc(v.regNumber)}" placeholder="АА 1234 77"></div>
          <div><label class="f-fld-label" for="vInv">Инвентарный номер</label>
            <input id="vInv" class="f-input" value="${esc(v.invNumber)}"></div>
          <div><label class="f-fld-label" for="vSer">Заводской номер</label>
            <input id="vSer" class="f-input" value="${esc(v.serialNumber)}"></div>
          <div><label class="f-fld-label" for="vUnit">Подразделение</label>
            <input id="vUnit" class="f-input" value="${esc(v.unit)}" list="unitsList" placeholder="1-я рота материального обеспечения">
            <datalist id="unitsList">${uniqueValues('unit').map(u => `<option value="${esc(u)}"></option>`).join('')}</datalist></div>
          <div><label class="f-fld-label" for="vResp">Ответственное лицо</label>
            <input id="vResp" class="f-input" value="${esc(v.responsible)}" list="respList" placeholder="ст. прапорщик Иванов И.И.">
            <datalist id="respList">${uniqueValues('responsible').map(u => `<option value="${esc(u)}"></option>`).join('')}</datalist></div>
          <div><label class="f-fld-label" for="vYear">Год выпуска</label>
            <input id="vYear" class="f-input" type="number" min="1900" max="${new Date().getFullYear() + 1}" value="${esc(v.year)}">
            <div class="f-error" id="errYear">Некорректный год выпуска</div></div>
          <div><label class="f-fld-label" for="vRegDate">Дата постановки на учёт</label>
            <input id="vRegDate" class="f-input" type="date" value="${esc(v.regDate || '')}"></div>
          <div><label class="f-fld-label" for="vStatus">Статус</label>
            <select id="vStatus" class="f-select">${options(STATUSES, v.status)}</select></div>
          <div><label class="f-fld-label" for="vHours">Текущие моточасы</label>
            <input id="vHours" class="f-input" type="number" min="0" step="0.1" value="${Number(v.currentHours || 0)}">
            <div class="f-hint">${isNew ? 'Будет создана первая запись в истории моточасов' : 'Изменение добавит запись в историю'}</div>
            <div class="f-error" id="errHours">Укажите значение от 0 до 10 000 000 мото·ч</div></div>

          <fieldset class="f-fieldset f-span2">
            <legend>Планирование эксплуатации (методика ВАМТО, 2025)</legend>
            <div class="f-form-grid">
              <div><label class="f-fld-label" for="vExpGroup">Группа эксплуатации</label>
                <select id="vExpGroup" class="f-select">${options(EXP_GROUPS, v.expGroup, 'Не указана')}</select>
                <div class="f-hint">Определяет норматив КТГ и годовые нормы расхода ресурса</div></div>
              <div><label class="f-fld-label" for="vAnnualPlan">Плановая годовая наработка, мото·ч</label>
                <input id="vAnnualPlan" class="f-input" type="number" min="0" step="1" value="${esc(v.annualPlanHours)}">
                <div class="f-hint">Для расчёта потребности в ТО и трудозатрат на год</div></div>
              <div><label class="f-fld-label" for="vAnnualNorm">Годовая норма расхода ресурса, мото·ч</label>
                <input id="vAnnualNorm" class="f-input" type="number" min="0" step="1" value="${esc(v.annualNormHours)}">
                <div class="f-hint">Лимит на год (приказ Росгвардии); используется в расчёте наличия моторесурса</div></div>
            </div>
          </fieldset>

          <fieldset class="f-fieldset f-span2">
            <legend>Планирование следующего ТО</legend>
            <div class="f-form-grid">
              <div>
                <label class="f-check" style="margin-bottom:8px">
                  <input type="checkbox" id="vByHours" ${iv.byHours ? 'checked' : ''}> По моточасам
                </label>
                <input id="vHoursInt" class="f-input" type="number" min="1" step="1"
                       value="${iv.hours || DB.settings.defaultHours}" ${iv.byHours ? '' : 'disabled'}>
                <div class="f-hint">Например, каждые 250 или 500 мото·ч</div>
              </div>
              <div>
                <label class="f-check" style="margin-bottom:8px">
                  <input type="checkbox" id="vByDate" ${iv.byDate ? 'checked' : ''}> По календарной дате
                </label>
                <input id="vDaysInt" class="f-input" type="number" min="1" step="1"
                       value="${iv.days || DB.settings.defaultDays}" ${iv.byDate ? '' : 'disabled'}>
                <div class="f-hint">Интервал в днях между обслуживаниями</div>
              </div>
              <div><label class="f-fld-label" for="vTo2Hours">ТО-2: каждые, мото·ч</label>
                <input id="vTo2Hours" class="f-input" type="number" min="1" step="1" value="${iv.to2Hours || ''}"
                       placeholder="по умолчанию ×${to2Every()} = ${fmtNum((iv.hours || DB.settings.defaultHours) * to2Every(), 0)}"></div>
              <div><label class="f-fld-label" for="vTo2Days">ТО-2: каждые, дней</label>
                <input id="vTo2Days" class="f-input" type="number" min="1" step="1" value="${iv.to2Days || ''}"
                       placeholder="по умолчанию ×${to2Every()} = ${(iv.days || DB.settings.defaultDays) * to2Every()}"></div>
            </div>
```

```
            <div class="f-hint" style="margin-top:8px">Если включены оба критерия, ТО назначается по тому, который наступит раньше. ТО-1 и ТО-2 считаются отдельно: ТО-2 — от последнего ТО-2 (ТО-3, КР); если ТО-2 наступает не позже ТО-1, очередным будет ТО-2. Пустые поля ТО-2 — интервал ТО-1 × ${to2Every()} (соотношение периодичностей в «Планировании»).</div>
          </fieldset>

          <div class="f-span2"><label class="f-fld-label" for="vNote">Примечание</label>
            <textarea id="vNote" class="f-textarea">${esc(v.note)}</textarea></div>
        </div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary" id="vSave">${FI.save(15)}<span>Сохранить</span></button>
      </div>`);

    const R = m.root;
    const q = id => R.querySelector('#' + id);

    // Привязка к справочнику: выбор позиции подставляет наименование (если
    // оно ещё пустое), а ввод наименования/марки — подбирает позицию (если
    // пользователь её ещё не выбрал вручную)
    const catSel = q('vCatalog');
    if (catSel) {
      let catTouched = !!catSel.value;
      catSel.addEventListener('change', () => {
        catTouched = true;
        const node = CAT().get(catSel.value);
        if (node && !q('vName').value.trim()) q('vName').value = node.name;
      });
      const guess = () => {
        if (catTouched) return;
        const id = CAT().match(`${q('vName').value} ${q('vBrand').value}`);
        if (id) catSel.value = id;
      };
      q('vName').addEventListener('change', guess);
      q('vBrand').addEventListener('change', guess);
    }

    // Чекбоксы включают/выключают соответствующие поля интервалов
    q('vByHours').addEventListener('change', e => { q('vHoursInt').disabled = !e.target.checked; });
    q('vByDate').addEventListener('change',  e => { q('vDaysInt').disabled  = !e.target.checked; });

    q('vSave').addEventListener('click', () => {
      const name = q('vName').value.trim();
      const year = q('vYear').value.trim();
      const hours = parseFloat(String(q('vHours').value).replace(',', '.'));

      let bad = false;
      const setErr = (inputId, errId, cond) => {
        q(inputId).classList.toggle('is-invalid', cond);
        q(errId).classList.toggle('is-visible', cond);
        if (cond) bad = true;
      };
      setErr('vName', 'errName', !name);
      setErr('vYear', 'errYear', !!year && (Number(year) < 1900 || Number(year) > new Date().getFullYear() + 1));
      setErr('vHours', 'errHours', !validHours(hours));
      if (bad) return;

      const byHours = q('vByHours').checked;
      const byDate  = q('vByDate').checked;
      const payload = {
        catalogId:    catSel ? catSel.value : (v.catalogId || ''),
        name,
        brand:        q('vBrand').value.trim(),
        regNumber:    q('vReg').value.trim(),
        invNumber:    q('vInv').value.trim(),
        serialNumber: q('vSer').value.trim(),
        unit:         q('vUnit').value.trim(),
        responsible:  q('vResp').value.trim(),
        year:         year,
        regDate:      q('vRegDate').value || todayISO(),
        status:       q('vStatus').value,
        note:         q('vNote').value.trim(),
        expGroup:     q('vExpGroup').value,
        annualPlanHours: (() => { const n = parseFloat(q('vAnnualPlan').value); return (q('vAnnualPlan').value !== '' && !isNaN(n) && n >= 0) ? n : ''; })(),
        annualNormHours: (() => { const n = parseFloat(q('vAnnualNorm').value); return (q('vAnnualNorm').value !== '' && !isNaN(n) && n >= 0) ? n : ''; })(),
        interval: {
          byHours, hours: Math.max(1, parseInt(q('vHoursInt').value, 10) || DB.settings.defaultHours),
          byDate,  days:  Math.max(1, parseInt(q('vDaysInt').value, 10)  || DB.settings.defaultDays),
          // Свой интервал ТО-2 (пусто — кратный интервалу ТО-1)
          to2Hours: Math.max(0, parseInt(q('vTo2Hours').value, 10) || 0) || '',
          to2Days:  Math.max(0, parseInt(q('vTo2Days').value, 10)  || 0) || ''
        }
      };

      if (isNew) {
        const nv = Object.assign({ id: uniqueVehicleId(), createdAt: new Date().toISOString() }, payload);
        nv.currentHours = hours;
```

```
        nv.hoursUpdatedAt = nv.regDate;
        DB.vehicles.push(nv);
        // Первичное показание счётчика — в историю моточасов
        DB.hours.push({ id: uid('h'), vehicleId: nv.id, date: nv.regDate, value: hours,
                        source: 'manual', note: 'Первичное показание при постановке на учёт',
                        createdAt: new Date().toISOString() });
        recomputeHours(nv.id);
        saveDB();
        m.close();
        toast('success', 'Техника добавлена', nv.name);
        go('vehicle', nv.id);
      } else {
        const prevHours = Number(vehicle.currentHours || 0);
        Object.assign(vehicle, payload, { updatedAt: new Date().toISOString() });
        if (hours !== prevHours) {
          DB.hours.push({ id: uid('h'), vehicleId: vehicle.id, date: todayISO(), value: hours,
                          source: 'manual', note: 'Корректировка через карточку техники',
                          createdAt: new Date().toISOString() });
        }
        recomputeHours(vehicle.id);
        saveDB();
        m.close();
        toast('success', 'Изменения сохранены', vehicle.name);
        rerender();
      }
    });
  }

  /* ============================================================================
     20а. МАССОВЫЕ ОПЕРАЦИИ НАД ВЫБРАННОЙ ТЕХНИКОЙ
     ============================================================================ */

  /* Массовая смена статуса */
  function bulkStatusModal(ids) {
    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">Изменить статус</div>
          <div class="f-modal-sub">Будет применено к ${ids.length} ед. техники</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <div class="f-form-grid">
          <div class="f-span2"><label class="f-fld-label" for="bulkStatus">Новый статус</label>
            <select id="bulkStatus" class="f-select">${options(STATUSES, STATUSES[0])}</select></div>
        </div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary"
  id="bulkStatusSave">${FI.save(15)}<span>Применить</span></button>
      </div>`, { narrow: true });

    m.root.querySelector('#bulkStatusSave').addEventListener('click', () => {
      const status = m.root.querySelector('#bulkStatus').value;
      let count = 0;
      ids.forEach(id => {
        const v = getVehicle(id);
        if (v) { v.status = status; v.updatedAt = new Date().toISOString(); count++; }
      });
      saveDB();
      m.close();
      ui.park.selected.clear();
      toast('success', 'Статус изменён', `Обновлено ${count} ед. техники.`);
      rerender();
    });
  }

  /* Массовая смена интервала планового ТО */
  function bulkIntervalModal(ids) {
    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">Изменить интервал ТО</div>
          <div class="f-modal-sub">Будет применено к ${ids.length} ед. техники</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <div class="f-form-grid">
          <div>
            <label class="f-check" style="margin-bottom:8px">
              <input type="checkbox" id="bulkByHours" checked> По моточасам
            </label>
            <input id="bulkHoursInt" class="f-input" type="number" min="1" step="1" value="${DB.settings.defaultHours}">
          </div>
          <div>
            <label class="f-check" style="margin-bottom:8px">
              <input type="checkbox" id="bulkByDate" checked> По календарной дате
```

```
          </label>
          <input id="bulkDaysInt" class="f-input" type="number" min="1" step="1" value="${DB.settings.defaultDays}">
        </div>
      </div>
      <div class="f-hint" style="margin-top:8px">Если включены оба критерия, ТО назначается по тому, который наступит раньше.</div>
    </div>
    <div class="f-modal-foot">
      <button type="button" class="f-btn" data-close>Отмена</button>
      <button type="button" class="f-btn f-btn-primary" id="bulkIntervalSave">${FI.save(15)}<span>Применить</span></button>
    </div>`, { narrow: true });

  const R = m.root;
  const q = id => R.querySelector('#' + id);
  q('bulkByHours').addEventListener('change', e => { q('bulkHoursInt').disabled = !e.target.checked; });
  q('bulkByDate').addEventListener('change',  e => { q('bulkDaysInt').disabled  = !e.target.checked; });

  q('bulkIntervalSave').addEventListener('click', () => {
    const byHours = q('bulkByHours').checked;
    const byDate  = q('bulkByDate').checked;
    const hours = Math.max(1, parseInt(q('bulkHoursInt').value, 10) || DB.settings.defaultHours);
    const days  = Math.max(1, parseInt(q('bulkDaysInt').value, 10)  || DB.settings.defaultDays);
    let count = 0;
    ids.forEach(id => {
      const v = getVehicle(id);
      if (v) { v.interval = Object.assign({}, v.interval, { byHours, hours, byDate, days }); count++; }
    });
    saveDB();
    m.close();
    ui.park.selected.clear();
    toast('success', 'Интервал обновлён', `Обновлено ${count} ед. техники.`);
    rerender();
  });
}

/* ============================================================================
   21. ФОРМА ЗАПИСИ ТЕХНИЧЕСКОГО ОБСЛУЖИВАНИЯ (с вложениями)
   ============================================================================ */
/* preset: { kind, fill } — вид ТО и автозаполнение из норматива справочника */
function maintModal(vehicleId, maint, preset) {
  const v = getVehicle(vehicleId);
  if (!v) return;
  const isNew = !maint;
  const rec = maint || {
    kind: (preset && preset.kind) || nextMaintKind(v), date: todayISO(), hours: Number(v.currentHours || 0),
    executor: v.responsible || '', unit: v.unit || '',
    works: '', parts: '', cost: '', note: '', docs: []
  };

  // docs — метаданные; newBlobs — файлы, ещё не записанные в IndexedDB
  let docs = (rec.docs || []).slice();
  const newBlobs = new Map();

  const fileRow = d => `
    <div class="f-file" data-file="${d.id}">
      <div class="f-file-ico">${FI.doc(14)}</div>
      <div class="f-file-name" title="${esc(d.name)}">${esc(d.name)}</div>
      <div class="f-file-size">${fmtSize(d.size)}</div>
      <button type="button" class="f-icon-btn is-danger" data-rm="${d.id}" title="Убрать">${FI.x(14)}</button>
    </div>`;

  const m = openModal(`
    <div class="f-modal-head">
      <div>
        <div class="f-modal-title">${isNew ? 'Новая запись технического обслуживания' : 'Редактирование записи ТО'}</div>
        <div class="f-modal-sub">${esc(v.name)}${v.regNumber ? ' · ' + esc(v.regNumber) : ''}</div>
      </div>
      <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
    </div>
    <div class="f-modal-body">
      <div class="f-form-grid">
        <div><label class="f-fld-label" for="mKind">Вид ТО *</label>
          <select id="mKind" class="f-select">${options(MAINT_KINDS, rec.kind)}</select>
          <div class="f-hint">ЕО и ремонты не сбрасывают межсервисный интервал</div></div>
        <div id="mSeasonBox" hidden><label class="f-fld-label" for="mSeason">Режим (сезон)</label>
          <select id="mSeason" class="f-select"><option value="summer">Летний</option><option value="winter">Зимний</option></select>
          <div class="f-hint">Засчитывается в сезонное обслуживание «Пункта управления»</div></div>
        <div><label class="f-fld-label" for="mDate">Дата проведения *</label>
          <input id="mDate" class="f-input" type="date" value="${esc(rec.date)}" max="${todayISO()}">
          <div class="f-error" id="errDate">Укажите корректную дату (не позднее сегодняшней)</div></div>
        <div><label class="f-fld-label" for="mHours">Текущие моточасы *</label>
          <input id="mHours" class="f-input" type="number" min="0" step="0.1" value="${Number(rec.hours || 0)}">
          <div class="f-error" id="errMHours">Укажите значение от 0 до 10 000 000 мото·ч</div></div>
        <div><label class="f-fld-label" for="mExec">Ответственный исполнитель</label>
          <input id="mExec" class="f-input" value="${esc(rec.executor)}"></div>
        <div><label class="f-fld-label" for="mUnit">Подразделение</label>
          <input id="mUnit" class="f-input" value="${esc(rec.unit)}"></div>
```

```
          <div><label class="f-fld-label" for="mCost">Стоимость, ₽</label>
            <input id="mCost" class="f-input" type="number" min="0" step="0.01" value="${rec.cost || ''}"></div>

          <div class="f-span2 f-reg-hint" id="mCatHint" hidden></div>
          <div class="f-span2 mc-box" id="mCheck" hidden></div>
          <div class="f-span2"><label class="f-fld-label" for="mWorks">Выполненные работы</label>
            <textarea id="mWorks" class="f-textarea" placeholder="Замена масла и фильтров, регулировка топливной аппаратуры…">${esc(rec.works)}</textarea></div>
          <div class="f-span2"><label class="f-fld-label" for="mParts">Использованные запасные части</label>
            <textarea id="mParts" class="f-textarea" placeholder="Фильтр масляный — 1 шт.; прокладка ГБЦ — 1 шт.">${esc(rec.parts)}</textarea></div>

          <div class="f-span2">
            <label class="f-fld-label">Прикреплённые документы</label>
            <div class="f-drop" id="mDrop">
              ${FI.upload(18)} Перетащите файлы сюда или нажмите для выбора<br>
              <span style="font-size:11.5px">PDF, Word, Excel, изображения</span>
            </div>
            <input type="file" id="mFiles" multiple hidden
                   accept=".pdf,.doc,.docx,.xls,.xlsx,.png,.jpg,.jpeg,.webp,.gif,.txt">
            <div class="f-files" id="mFileList">${docs.map(fileRow).join('')}</div>
          </div>

          <div class="f-span2"><label class="f-fld-label" for="mNote">Примечание</label>
            <textarea id="mNote" class="f-textarea">${esc(rec.note)}</textarea></div>
        </div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary" id="mSave">${FI.save(15)}<span>Сохранить</span></button>
      </div>`);

    const R = m.root;
    const q = id => R.querySelector('#' + id);
    const list = q('mFileList');
    const drop = q('mDrop');
    const input = q('mFiles');

    // Норматив по справочнику для выбранного вида ТО + заполнение формы из него
    const catMaint = catMaintOf(v);
    const addText = (id, text) => {
      const t = q(id), cur = t.value.trim();
      if (!cur) t.value = text;
      else if (cur.indexOf(text) === -1) t.value = cur + '\n' + text;
    };
    const updCatHint = () => {
      const box = q('mCatHint');
      const key = KIND_TO_CATALOG[q('mKind').value];
      const reg = catMaint && key ? catMaint[key] : null;
      box.hidden = !reg;
      if (!reg) { box.innerHTML = ''; return; }
      const ops = (reg.actions || []).length;
      const people = (reg.personnel || []).reduce((s, p) => s + (Number(p.count) || 0), 0);
      box.innerHTML = `${FI.file(15)}<span>Норматив по справочнику: <b>${ops}</b> ${plural(ops, 'операция', 'операции', 'операций')} · <b>${fmtNum(reg.labor, 1)}</b> чел·ч · <b>${people}</b> чел.</span>
        <button type="button" class="f-btn f-btn-sm" data-fill-cat>Подставить материалы</button>`;
      box.querySelector('[data-fill-cat]').addEventListener('click', () => {
        addText('mParts', (reg.materials || []).join('; '));
        toast('info', 'Материалы подставлены', 'Проверьте перечень и скорректируйте под фактически использованное.');
      });
    };
    // Перечень работ регламента с галочками (шаблон работ); снимок хранится в записи
    let curChecklist = null;
    const memoChecks = new Map();   // отметки по видам ТО в пределах открытой формы
    const savedChecklist = rec.checklist && Array.isArray(rec.checklist.items) ? rec.checklist : null;
    const renderCheck = () => {
      const box = q('mCheck');
      const key = KIND_TO_CATALOG[q('mKind').value];
      const reg = catMaint && key ? catMaint[key] : null;
      const actions = reg ? (reg.actions || []) : [];
      if (!actions.length) { box.hidden = true; box.innerHTML = ''; curChecklist = null; return; }
      const kindNow = q('mKind').value;
      const prev = memoChecks.get(kindNow) || (savedChecklist && savedChecklist.kind === kindNow ? savedChecklist.items : []);
      curChecklist = buildChecklist(actions, prev);
      memoChecks.set(kindNow, curChecklist);
      box.hidden = false;
      box.innerHTML = `<div class="mc-head"><label class="f-fld-label">Перечень работ по регламенту</label><span class="mc-count" id="mcCount"></span>
          <button type="button" class="f-btn f-btn-sm" data-mc-all>Отметить все</button><button type="button" class="f-btn f-btn-sm" data-mc-none>Снять отметки</button></div>
        <div class="mc-list">${curChecklist.map((x, i) => `<label class="mc-row"><input type="checkbox" data-mc="${i}" ${x.done ? 'checked' : ''}><span>${i + 1}. ${esc(x.t)}</span></label>`).join('')}</div>`;
      const upd = () => { const s = checklistSummary({ items: curChecklist }); q('mcCount').textContent = `выполнено ${s.done} из ${s.total}`; };
      box.querySelectorAll('[data-mc]').forEach(cb => cb.addEventListener('change', () => { curChecklist[Number(cb.dataset.mc)].done = cb.checked; upd(); }));
      const setAll = on => { curChecklist.forEach(x => { x.done = on; }); box.querySelectorAll('[data-mc]').forEach(cb => { cb.checked = on; }); upd(); };
      box.querySelector('[data-mc-all]').addEventListener('click', () => setAll(true));
```

```
      box.querySelector('[data-mc-none]').addEventListener('click', () => setAll(false));
      upd();
    };
    const updSeason = () => { q('mSeasonBox').hidden = q('mKind').value !== SEASON_KIND; };
    { const ev = seasonEvent(); q('mSeason').value = rec.season || (ev ? ev.season : 'summer'); }
    q('mKind').addEventListener('change', () => { updCatHint(); renderCheck(); updSeason(); });
    updCatHint(); renderCheck(); updSeason();
    if (preset && preset.fill) q('mCatHint').querySelector('[data-fill-cat]')?.click();

    const redrawFiles = () => {
      list.innerHTML = docs.map(fileRow).join('');
      list.querySelectorAll('[data-rm]').forEach(b => b.addEventListener('click', () => {
        const id = b.dataset.rm;
        docs = docs.filter(d => d.id !== id);
        if (newBlobs.has(id)) newBlobs.delete(id);
        redrawFiles();
      }));
    };
    redrawFiles();

    const addFiles = files => {
      Array.from(files).forEach(f => {
        if (f.size > 12 * 1024 * 1024) {
          toast('warning', 'Файл слишком большой', `${f.name} — более 12 МБ, пропущен.`);
          return;
        }
        const id = uid('file');
        docs.push({ id, name: f.name, type: f.type, size: f.size });
        newBlobs.set(id, f);
      });
      redrawFiles();
    };

    drop.addEventListener('click', () => input.click());
    input.addEventListener('change', () => { addFiles(input.files); input.value = ''; });
    ['dragenter', 'dragover'].forEach(ev => drop.addEventListener(ev, e => {
      e.preventDefault(); drop.classList.add('is-over');
    }));
    ['dragleave', 'drop'].forEach(ev => drop.addEventListener(ev, e => {
      e.preventDefault(); drop.classList.remove('is-over');
    }));
    drop.addEventListener('drop', e => { if (e.dataTransfer && e.dataTransfer.files) addFiles(e.dataTransfer.files); });

    // Сохранение асинхронное (запись вложений), поэтому повторные нажатия до его
    // завершения блокируются — иначе одна и та же запись ТО создаётся несколько раз
    let saving = false;

    q('mSave').addEventListener('click', async () => {
      if (saving) return;

      const date = q('mDate').value;
      const hours = parseFloat(String(q('mHours').value).replace(',', '.'));

      let bad = false;
      const setErr = (inputId, errId, cond) => {
        q(inputId).classList.toggle('is-invalid', cond);
        q(errId).classList.toggle('is-visible', cond);
        if (cond) bad = true;
      };
      setErr('mDate', 'errDate', !date || date > todayISO());
      setErr('mHours', 'errMHours', !validHours(hours));
      if (bad) return;

      saving = true;
      q('mSave').disabled = true;

      // Сохраняем новые файлы в IndexedDB
      try {
        await Promise.all(Array.from(newBlobs.entries()).map(([id, file]) => {
          const meta = docs.find(d => d.id === id);
          return filePut({ id, name: meta.name, type: meta.type, size: meta.size, blob: file });
        }));
      } catch (e) {
        toast('warning', 'Вложения', 'Не удалось сохранить часть файлов в локальном хранилище.');
      }

      try {
        const payload = {
          vehicleId, kind: q('mKind').value, date, hours,
          executor: q('mExec').value.trim(),
          unit:     q('mUnit').value.trim(),
          works:    (() => { const w = q('mWorks').value.trim(); if (w || !curChecklist) return w; return
curChecklist.filter(x => x.done).map((x, i) => `${i + 1}. ${x.t}`).join('\n'); })(),
          season:   q('mKind').value === SEASON_KIND ? q('mSeason').value : '',
          checklist: curChecklist ? { kind: q('mKind').value, items: curChecklist.map(x => ({ t: x.t, done: x.done })) }
                                  : (savedChecklist && savedChecklist.kind === q('mKind').value ? savedChecklist :
undefined),
          parts:    q('mParts').value.trim(),
          cost:     (() => { const c = parseFloat(q('mCost').value); return (q('mCost').value !== '' && !isNaN(c) && c
>= 0) ? c : ''; })(),
```

```
          note:     q('mNote').value.trim(),
          docs
        };

        if (isNew) {
          const nm = Object.assign({ id: uid('m'), createdAt: new Date().toISOString() }, payload);
          DB.maint.push(nm);
          syncMaintHours(nm);
        } else {
          // Файлы, удалённые пользователем, стираем из IndexedDB
          (maint.docs || []).forEach(old => { if (!docs.find(d => d.id === old.id)) fileDel(old.id); });
          Object.assign(maint, payload, { updatedAt: new Date().toISOString() });
          syncMaintHours(maint);
        }

        recomputeHours(vehicleId);
        saveDB();
        m.close();

        const st = computeStatus(getVehicle(vehicleId));
        const next = st.nextDate ? fmtDate(st.nextDate)
                   : (st.nextHours != null ? fmtNum(st.nextHours, 0) + ' мото·ч' : '—');
        toast('success', isNew ? 'Запись ТО добавлена' : 'Запись ТО обновлена',
              `Следующее обслуживание: ${next}`);
        rerender();
      } catch (err) {
        // Форма остаётся открытой и доступной для повторного сохранения
        saving = false;
        q('mSave').disabled = false;
        toast('danger', 'Не удалось сохранить запись', 'Проверьте данные и повторите попытку.');
      }
    });
  }

  /* ============================================================================
     22. ФОРМА ВВОДА МОТОЧАСОВ
     ============================================================================ */
  function hoursModal(vehicleId) {
    const v = getVehicle(vehicleId);
    if (!v) return;
    const cur = Number(v.currentHours || 0);

    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">Ввод текущих моточасов</div>
          <div class="f-modal-sub">${esc(v.name)} · текущее показание: ${fmtNum(cur, 0)} мото·ч</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <div class="f-form-grid">
          <div><label class="f-fld-label" for="hDate">Дата снятия показаний *</label>
            <input id="hDate" class="f-input" type="date" value="${todayISO()}" max="${todayISO()}"></div>
          <div><label class="f-fld-label" for="hVal">Показание счётчика, мото·ч *</label>
            <input id="hVal" class="f-input" type="number" min="0" step="0.1" value="${cur}">
            <div class="f-error" id="errVal">Укажите значение от 0 до 10 000 000 мото·ч</div></div>
          <div class="f-span2"><label class="f-fld-label" for="hNote">Примечание</label>
            <input id="hNote" class="f-input" placeholder="Например: снято при возвращении с выхода"></div>
          <div class="f-span2" id="hWarn" style="display:none">
            <div class="f-demo-banner" style="margin:0">${FI.alert(16)}
              <div>Введённое значение меньше текущего. Это допустимо при замене счётчика или двигателя.</div></div>
          </div>
        </div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary" id="hSave">${FI.save(15)}<span>Сохранить</span></button>
      </div>`, { narrow: false });

    const R = m.root;
    const q = id => R.querySelector('#' + id);

    q('hVal').addEventListener('input', () => {
      const val = parseFloat(String(q('hVal').value).replace(',', '.'));
      q('hWarn').style.display = (!isNaN(val) && val < cur) ? 'block' : 'none';
    });

    q('hSave').addEventListener('click', () => {
      const val = parseFloat(String(q('hVal').value).replace(',', '.'));
      let date = q('hDate').value || todayISO();
      if (date > todayISO()) date = todayISO();   // показание не может быть снято будущей датой
      const bad = !validHours(val);
      q('hVal').classList.toggle('is-invalid', bad);
      q('errVal').classList.toggle('is-visible', bad);
      if (bad) return;

      DB.hours.push({ id: uid('h'), vehicleId, date, value: val, source: 'manual',
                      note: q('hNote').value.trim(), createdAt: new Date().toISOString() });
      recomputeHours(vehicleId);
```

```
      saveDB();
      m.close();

      const st = computeStatus(getVehicle(vehicleId));
      toast('success', 'Моточасы обновлены',
        st.remainingHours != null
          ? `До следующего ТО: ${fmtNum(st.remainingHours, 0)} мото·ч`
          : `Наработка с последнего ТО: ${fmtNum(st.hoursSince, 0)} мото·ч`);
      rerender();
    });
  }

  /* ============================================================================
     23. ПАНЕЛЬ УВЕДОМЛЕНИЙ
     ============================================================================ */
  function renderNotifPanel() {
    const list = computeNotifications();
    const badge = el('fleetBellBadge');
    const alerts = list.filter(n => n.level === 'overdue' || n.level === 'urgent').length;
    badge.textContent = list.length > 99 ? '99+' : list.length;
    badge.classList.toggle('is-visible', list.length > 0);
    badge.style.background = alerts ? 'var(--danger)' : 'var(--warning)';

    const dot = lvl => lvl === 'info' ? 'var(--accent)'
      : lvl === 'soon' ? 'var(--st-soon)'
      : lvl === 'urgent' ? 'var(--st-urgent)' : 'var(--st-overdue-text)';

    el('fleetNotifPanel').innerHTML = `
      <div class="f-notif-head">
        <span>Уведомления${list.length ? ` · ${list.length}` : ''}</span>
        ${list.length ? `<button type="button" class="f-btn f-btn-ghost f-btn-sm" data-act="goto-dash">Панель
контроля</button>` : ''}
      </div>
      ${list.length
        ? list.map(n => `
          <div class="f-notif" data-act="${n.act || 'open-vehicle'}" data-id="${n.vehicleId}">
            <span class="f-notif-dot" style="background:${dot(n.level)}"></span>
            <div style="min-width:0">
              <div class="f-notif-title">${esc(n.title)}</div>
              <div class="f-notif-text">${esc(n.text)}</div>
            </div>
          </div>`).join('')
        : `<div class="f-notif-empty">Активных уведомлений нет.<br>Вся техника обслужена в срок.</div>`}`;
  }

  /* Сводка при открытии раздела */
  function startupNotify() {
    if (!DB.settings.notifyOnStart) return;
    try { if (sessionStorage.getItem('tst-fleet-notified')) return; } catch (e) { /* нет доступа */ }

    const list = computeNotifications();
    const seasonN = list.find(n => n.act === 'season');
    const overdue = list.filter(n => n.level === 'overdue' && !n.act).length;
    const urgent  = list.filter(n => n.level === 'urgent' && !n.act).length;
    const stale   = list.filter(n => n.level === 'info').length;

    if (overdue) toast('danger', 'Просроченные ТО', `${overdue} ${plural(overdue, 'единица техники', 'единицы техники',
'единиц техники')} с истёкшим сроком обслуживания.`, 8000);
    if (urgent)  toast('warning', 'Требуется ТО', `${urgent} ${plural(urgent, 'единица', 'единицы', 'единиц')} техники
нуждается в обслуживании в ближайшее время.`, 8000);
    if (stale)   toast('info', 'Обновите моточасы', `У ${stale} ${plural(stale, 'единицы', 'единиц', 'единиц')} техники
давно не обновлялись показания счётчика.`, 8000);
    if (seasonN) toast(seasonN.level === 'overdue' ? 'danger' : 'warning', seasonN.title, seasonN.text, 9000, { action:
'Список', onAction: () => commandListModal('season') });
    if (!overdue && !urgent && !stale && !seasonN && DB.vehicles.length) toast('success', 'Все ТО в срок', 'Просроченных
 обслуживаний не обнаружено.');
    const bk = backupInfo();
    if (bk.due) toast('warning', 'Сохраните резервную копию', backupAgoText(bk).replace(/^./, c => c.toUpperCase()) +
'.', 10000,
      { action: 'Сохранить', onAction: backupExport });

    try { sessionStorage.setItem('tst-fleet-notified', '1'); } catch (e) { /* нет доступа */ }
  }

  /* ============================================================================
     24. ДЕМОНСТРАЦИОННЫЕ ДАННЫЕ
     ----------------------------------------------------------------------------
     Создаются относительно текущей даты, чтобы цветовая индикация всегда
     показывала полный спектр состояний. Удаляются одной кнопкой.
     ============================================================================ */
  function seedDemo() {
    const T = todayISO();
    const ago = n => addDays(T, -n);
    const now = new Date().toISOString();

    const mk = (o) => {
      const v = Object.assign({
        id: uid('v'), demo: true, createdAt: now, status: 'В эксплуатации',
        interval: { byHours: true, hours: 250, byDate: true, days: 180 }
      }, o);
```

```
      v.catalogId = CAT() ? CAT().match(`${v.name} ${v.brand}`) : '';
      return v;
    };

    const vs = [
      mk({ name: 'Кухня автомобильная КА-220', brand: 'КамАЗ-4310 / КА-220', regNumber: 'АА 1234 77',
           invNumber: 'ИН-001042', serialNumber: '4310-889201', unit: '1-я рота материального обеспечения',
           responsible: 'ст. прапорщик Иванов И.И.', year: '2016', regDate: ago(1400) }),
      mk({ name: 'Автоцистерна АЦ-4,2', brand: 'Урал-4320 / АЦ-4,2', regNumber: 'ВВ 5678 50',
           invNumber: 'ИН-001077', serialNumber: '4320-112455', unit: '2-я рота материального обеспечения',
           responsible: 'прапорщик Петров П.П.', year: '2014', regDate: ago(2100) }),
      mk({ name: 'Полевая кухня ПАК-200М', brand: 'ЗИЛ-131 / ПАК-200М', regNumber: 'СС 9012 77',
           invNumber: 'ИН-001103', serialNumber: '131-337890', unit: '1-я рота материального обеспечения',
           responsible: 'ст. сержант Сидоров С.С.', year: '2011', regDate: ago(3200) }),
      mk({ name: 'Дизельная электростанция АД-30', brand: 'АД-30С-Т400-1РМ2', regNumber: '',
           invNumber: 'ИН-002011', serialNumber: 'АД30-556712', unit: 'Взвод обеспечения',
           responsible: 'сержант Кузнецов К.К.', year: '2009', regDate: ago(4200),
           interval: { byHours: true, hours: 250, byDate: true, days: 180 } }),
      mk({ name: 'Хлебопекарня полевая ПХ-1', brand: 'КамАЗ-5350 / ПХ-1', regNumber: 'ЕЕ 3344 90',
           invNumber: 'ИН-003005', serialNumber: '5350-991204', unit: '3-я рота материального обеспечения',
           responsible: 'прапорщик Смирнов А.В.', year: '2021', regDate: ago(40),
           interval: { byHours: true, hours: 500, byDate: false, days: 180 } }),
      mk({ name: 'Автомобиль-рефрижератор АР-3', brand: 'ГАЗ-3307 / АР-3', regNumber: 'КК 7788 33',
           invNumber: 'ИН-004009', serialNumber: '3307-220145', unit: '2-я рота материального обеспечения',
           responsible: 'ст. сержант Волков Д.М.', year: '2003', regDate: ago(6000),
           status: 'Списано' })
    ];
    DB.vehicles.push.apply(DB.vehicles, vs);

    /* Записи ТО подобраны так, чтобы получить все уровни индикации */
    const M = (v, kind, dateIso, hours, works, parts, cost, exec) => DB.maint.push({
      id: uid('m'), demo: true, vehicleId: v.id, kind, date: dateIso, hours,
      executor: exec || v.responsible, unit: v.unit, works: works || '', parts: parts || '',
      cost: cost || '', note: '', docs: [], createdAt: now
    });

    // □ запас 88% по моточасам
    M(vs[0], 'ТО-2', ago(160), 780, 'Углублённое обслуживание кухонного блока, промывка котлов', 'Прокладки котла — компл.', 18400);
    M(vs[0], 'ТО-1', ago(20), 1000, 'Очистка форсунок ГК-5, контроль уровня масла в редукторе', 'Масло МГЕ-10А — 4 л', 4200);

    // □ остаток ~26%
    M(vs[1], 'ТО-1', ago(30), 800, 'Проверка герметичности цистерны, обслуживание насоса', 'Сальник насоса — 1 шт.', 6100);

    // □ остаток ~8%
    M(vs[2], 'ТО-2', ago(60), 2000, 'Регулировка топливной аппаратуры, замена фильтров', 'Фильтр топливный — 2 шт.', 9700);

    // □ просрочено и по моточасам, и по дате
    M(vs[3], 'ТО-1', ago(200), 4000, 'Замена масла, проверка щёточного узла генератора', 'Масло М-10Г2к — 12 л', 7300);

    // Хлебопекарня — ТО не проводилось, отсчёт от постановки на учёт
    // Рефрижератор — списан
    M(vs[5], 'Капитальный ремонт', ago(900), 12500, 'Капитальный ремонт холодильной установки', 'Компрессор — 1 шт.', 154000);

    /* Текущие показания моточасов */
    const H = (v, dateIso, value, note) => DB.hours.push({
      id: uid('h'), demo: true, vehicleId: v.id, date: dateIso, value,
      source: 'manual', note: note || '', createdAt: now
    });

    H(vs[0], ago(2), 1030);              // 1030 − 1000 = 30 → остаток 220/250 = 88%  □
    H(vs[1], ago(3), 985);               // 985 − 800 = 185 → остаток 65/250 = 26%    □
    H(vs[2], ago(1), 2230);              // 2230 − 2000 = 230 → остаток 20/250 = 8%   □
    H(vs[3], ago(30), 4300, 'Показание снято при постановке на консервацию'); // перерасход □ + моточасы устарели
    H(vs[4], ago(5), 120);               // 120/500 → остаток 76%                     □
    H(vs[5], ago(400), 12800);

    // Синхронизируем показания, записанные вместе с ТО
    DB.maint.forEach(syncMaintHours);
    DB.vehicles.forEach(v => recomputeHours(v.id));

    DB.seeded = true;
    saveDB();
  }

  function removeDemo() {
    const ids = DB.vehicles.filter(v => v.demo).map(v => v.id);
    DB.maint.filter(m => ids.indexOf(m.vehicleId) !== -1)
            .forEach(m => (m.docs || []).forEach(d => fileDel(d.id)));
    DB.vehicles = DB.vehicles.filter(v => !v.demo);
    DB.maint    = DB.maint.filter(m => ids.indexOf(m.vehicleId) === -1);
    DB.hours    = DB.hours.filter(h => ids.indexOf(h.vehicleId) === -1);
    DB.trips    = DB.trips.filter(t => ids.indexOf(t.vehicleId) === -1);
    ids.forEach(id => { if (ui.trips.vehicle === id) ui.trips.vehicle = ''; });
    saveDB();
    toast('success', 'Демо-данные удалены', 'База очищена от демонстрационных записей.');
```

```
    rerender();
  }

  /* Сброс ссылок интерфейса на конкретные записи — после полной замены базы
     (восстановление из копии, очистка) идентификаторы больше не существуют */
  function resetUiSelection() {
    ui.park.selected.clear();
    ui.vehicleId = null;
    ui.trips.vehicle = '';
    ui.report.unit = '';
  }

  /* ========================================================================
     25. РЕЗЕРВНОЕ КОПИРОВАНИЕ И ХРАНИЛИЩЕ
     ------------------------------------------------------------------------
     Копия — ZIP-архив (собственный генератор zipStore, без библиотек):
       backup.json          — все записи + список вложений;
       files/<id>.<ext>     — сами файлы из IndexedDB;
       ПРОЧТИТЕ.txt         — как восстановить.
     Старые копии в формате JSON тоже принимаются (вложений в них нет).
     ======================================================================== */
  const BACKUP_FORMAT = 'tst-fleet-backup';

  function backupFileExt(name) {
    const m = /\.([A-Za-z0-9]{1,8})$/.exec(String(name || ''));
    return m ? '.' + m[1].toLowerCase() : '';
  }

  async function backupExport() {
    try {
      const docs = [];
      DB.maint.forEach(m => (m.docs || []).forEach(d => docs.push(d)));
      DB.vehicles.forEach(v => vfFileIds(v).forEach(id => docs.push({ id })));
      const files = [], manifest = [];
      let missing = 0;
      for (const d of docs) {
        const rec = await fileGet(d.id);
        if (!rec || !rec.blob) { missing++; continue; }
        // Путь только из латиницы и цифр: имена с кириллицей по-разному
        // кодируются архиваторами, а исходное имя хранится в backup.json
        const path = 'files/' + d.id + backupFileExt(rec.name || d.name);
        files.push({ name: path, data: new Uint8Array(await rec.blob.arrayBuffer()) });
        manifest.push({ id: d.id, name: rec.name || d.name, type: rec.type || d.type || '', size: rec.size || d.size ||
0, path });
      }

      const createdAt = new Date().toISOString();
      DB.lastBackupAt = createdAt;
      const pkg = { format: BACKUP_FORMAT, version: 2, createdAt, files: manifest, db: DB };
      files.unshift({ name: 'backup.json', data: utf8(JSON.stringify(pkg, null, 1)) });
      files.push({ name: 'ПРОЧТИТЕ.txt', data: utf8(
        'Резервная копия программы «Регламент-ТС» (модуль учёта техники).\r\n' +
        'Создана: ' + fmtDate(todayISO()) + '\r\n\r\n' +
        'Восстановление: Технические средства тыла → Настройки → «Восстановить из копии» — выберите этот ZIP-файл
целиком, распаковывать его не нужно.\r\n' +
        'Содержимое: backup.json — карточки, журнал ТО, моточасы, путевые листы, нормативы и настройки; папка files —
прикреплённые документы.\r\n') });

      const zip = zipStore(files);
      downloadBlob(new Blob([zip], { type: 'application/zip' }), `tst_backup_${todayISO()}.zip`);
      saveDB();
      toast('success', 'Резервная копия создана',
        `Записи и ${manifest.length} ${plural(manifest.length, 'вложение', 'вложения', 'вложений')} ·
${fmtSize(zip.length)}` +
        (missing ? `. Не найдено в хранилище: ${missing}` : '') + '. Сохраните файл вне компьютера — на флешку или
сетевой диск.', 8000);
      if (window.TST && window.TST.workspace === 'fleet' && ui.view === 'settings') renderView();
    } catch (e) {
      console.warn('[ТСТ] Ошибка резервного копирования', e);
      toast('danger', 'Не удалось создать копию', 'Недостаточно памяти или хранилище недоступно. Попробуйте ещё раз.');
    }
  }

  /* Чтение ZIP: центральный каталог + локальные заголовки. Кроме собственного
     формата «без сжатия» поддерживается deflate (если архив пересохранён
     сторонним архиватором) — через встроенный DecompressionStream */
  async function unzip(bytes) {
    const dv = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
    let e = bytes.length - 22;
    while (e >= 0 && dv.getUint32(e, true) !== 0x06054b50) e--;
    if (e < 0) throw new Error('not a zip');
    const count = dv.getUint16(e + 10, true);
    let p = dv.getUint32(e + 16, true);
    const dec = new TextDecoder();
    const out = new Map();
    for (let i = 0; i < count; i++) {
      if (dv.getUint32(p, true) !== 0x02014b50) throw new Error('bad central directory');
      const method = dv.getUint16(p + 10, true);
      const csize  = dv.getUint32(p + 20, true);
      const nlen = dv.getUint16(p + 28, true), xlen = dv.getUint16(p + 30, true), clen = dv.getUint16(p + 32, true);
```

```
      const lho  = dv.getUint32(p + 42, true);
      const name = dec.decode(bytes.subarray(p + 46, p + 46 + nlen));
      const start = lho + 30 + dv.getUint16(lho + 26, true) + dv.getUint16(lho + 28, true);
      let data = bytes.subarray(start, start + csize);
      if (method === 8) {
        if (typeof DecompressionStream === 'undefined') throw new Error('deflate unsupported');
        const stream = new Blob([data]).stream().pipeThrough(new DecompressionStream('deflate-raw'));
        data = new Uint8Array(await new Response(stream).arrayBuffer());
      } else if (method !== 0) {
        throw new Error('unsupported method ' + method);
      }
      out.set(name, data);
      p += 46 + nlen + xlen + clen;
    }
    return out;
  }

  function backupImport() {
    const inp = document.createElement('input');
    inp.type = 'file';
    inp.accept = '.zip,.json,application/zip,application/json';
    inp.addEventListener('change', async () => {
      const file = inp.files && inp.files[0];
      if (!file) return;
      let parsed, files = null;
      try {
        const bytes = new Uint8Array(await file.arrayBuffer());
        if (bytes[0] === 0x50 && bytes[1] === 0x4B) {          // «PK» — ZIP-архив
          const entries = await unzip(bytes);
          const json = entries.get('backup.json');
          if (!json) throw new Error('no backup.json');
          const pkg = JSON.parse(new TextDecoder().decode(json));
          if (!pkg || pkg.format !== BACKUP_FORMAT || !pkg.db) throw new Error('bad package');
          parsed = pkg.db;
          files = (pkg.files || []).map(f => ({ meta: f, data: entries.get(f.path) })).filter(f => f.data);
        } else {                                              // старый формат — JSON
          parsed = JSON.parse(new TextDecoder().decode(bytes));
        }
        if (!parsed || !Array.isArray(parsed.vehicles)) throw new Error('bad format');
      } catch (e) {
        toast('danger', 'Ошибка восстановления', 'Файл повреждён или не является резервной копией программы.');
        return;
      }

      const decision = await importReview(parsed, files, file.name);
      if (!decision) return;
      const srcVersion = parsed.version;
      if (decision === 'repair') parsed = window.TST.platform.repair(normalizeDB(parsed)).db;
      await snapCreate('import', 'Перед восстановлением из файла ' + file.name).catch(() => null);

      try {
        if (files) {
          // Существующие вложения не стираются: на них могут ссылаться точки
          // восстановления; лишние уберёт cleanupOrphanFiles
          for (const f of files) {
            const type = f.meta.type || '';
            await filePut({ id: f.meta.id, name: f.meta.name, type, size: f.data.length,
                            blob: new Blob([f.data], { type }) });
          }
        }
        DB = migrateDB(normalizeDB(parsed), srcVersion);
        migrateCatalogLinks();
        resetUiSelection();
        saveDB({ replace: true });
        ui.safety.report = null;
        toast('success', 'Данные восстановлены',
          `Загружено ${DB.vehicles.length} ед. техники` + (files ? ` и ${files.length} ${plural(files.length,
'вложение', 'вложения', 'вложений')}.` : '.'));
        go('park');
      } catch (e) {
        console.warn('[ТСТ] Ошибка восстановления', e);
        toast('danger', 'Ошибка восстановления', 'Не удалось записать данные в хранилище браузера.');
      }
    });
    inp.click();
  }

  /* --- Постоянное хранилище (navigator.storage.persist) --- */
  const storageInfo = { supported: false, persisted: null, usage: null, quota: null };

  async function refreshStorageInfo() {
    const sm = navigator.storage;
    storageInfo.supported = !!(sm && sm.persist);
    try {
      if (sm && sm.persisted) storageInfo.persisted = await sm.persisted();
      if (sm && sm.estimate) {
        const est = await sm.estimate();
        storageInfo.usage = est.usage; storageInfo.quota = est.quota;
      }
    } catch (e) { /* file:// и приватный режим могут не поддерживать API */ }
```

```
    fillStorageStatus();
  }

  async function requestPersist(manual) {
    if (!(navigator.storage && navigator.storage.persist)) {
      if (manual) toast('warning', 'Недоступно', 'Этот браузер не поддерживает постоянное хранение. Регулярно сохраняйте резервную копию.');
      return false;
    }
    let ok = false;
    try { ok = await navigator.storage.persist(); } catch (e) { ok = false; }
    try { localStorage.setItem(LS_PERSIST_ASKED, '1'); } catch (e) { /* приватный режим */ }
    storageInfo.persisted = ok;
    if (manual) {
      toast(ok ? 'success' : 'warning',
        ok ? 'Постоянное хранение включено' : 'Браузер отклонил запрос',
        ok ? 'Браузер не будет автоматически удалять данные программы при нехватке места.'
           : 'Браузеры разрешают это часто используемым страницам (в закладках, на рабочем столе). Пока не разрешено — чаще сохраняйте резервную копию.', 8000);
    }
    refreshStorageInfo();
    return ok;
  }

  /* Сведения о резервной копии — для настроек, обзора и стартовой сводки */
  function backupInfo() {
    const last = DB.lastBackupAt ? DB.lastBackupAt.slice(0, 10) : '';
    const daysAgo = last ? daysBetween(last, todayISO()) : null;
    const remind = Number(DB.settings.backupRemindDays) || DEFAULT_SETTINGS.backupRemindDays;
    const hasData = DB.vehicles.some(v => !v.demo) || DB.norms.length > 0 || DB.catalogEdits.length > 0;
    return { last, daysAgo, remind, hasData, due: hasData && (daysAgo == null || daysAgo >= remind),
             persisted: storageInfo.persisted, persistSupported: storageInfo.supported };
  }

  function backupAgoText(info) {
    if (!info.last) return 'резервная копия ещё не создавалась';
    if (info.daysAgo <= 0) return 'последняя копия — сегодня';
    return `последняя копия — ${info.daysAgo} ${plural(info.daysAgo, 'день', 'дня', 'дней')} назад (${fmtDate(info.last)})`;
  }

  /* Строка состояния хранилища в настройках (заполняется асинхронно) */
  function fillStorageStatus() {
    const box = el('setStorageStatus');
    if (!box) return;
    const i = storageInfo;
    const persist = !i.supported ? '□ не поддерживается браузером'
      : i.persisted ? '□ включено — браузер не удалит данные сам'
      : '□ не включено — при нехватке места браузер может очистить данные';
    const used = i.usage != null
      ? `${fmtSize(i.usage)}${i.quota ? ' из ~' + fmtSize(i.quota) : ''}` : 'нет данных';
    box.innerHTML = `
      <div><div class="f-info-k">Постоянное хранение</div><div class="f-info-v">${esc(persist)}</div></div>
      <div><div class="f-info-k">Занято в браузере</div><div class="f-info-v">${esc(used)}</div></div>`;
    const btn = el('setPersistBtn');
    if (btn) btn.hidden = !i.supported || !!i.persisted;
  }

  async function wipeAll() {
    const ok = await confirmDialog('Очистить все данные?',
      'Будут удалены все карточки техники, записи ТО, история моточасов и вложения. Перед очисткой создаётся точка восстановления (раздел «Сохранность данных»), но надёжнее сначала сохранить резервную копию.', 'Очистить');
    if (!ok) return;
    await snapCreate('wipe').catch(() => null);
    DB = blankDB();
    resetUiSelection();
    saveDB({ replace: true });
    setTimeout(cleanupOrphanFiles, 1500);
    toast('success', 'Данные очищены', 'База приведена к исходному состоянию. Вернуть данные можно в разделе «Сохранность данных» → «Точки восстановления».');
    go('park');
  }

  /* ============================================================================
     26. ПОЛНАЯ ПЕРЕРИСОВКА
     ============================================================================ */
  function rerender() {
    renderNav();
    renderView();
    renderNotifPanel();
  }

  /* ============================================================================
     27. ДЕЛЕГИРОВАНИЕ СОБЫТИЙ
     ----------------------------------------------------------------------------
     Один обработчик на весь модуль: атрибут data-act определяет действие.
     Ближайший к цели элемент с data-act «выигрывает», поэтому кнопки внутри
     кликабельных строк таблицы не запускают открытие карточки.
     ============================================================================ */
  function onFleetClick(e) {
```

```
    const node = e.target.closest('[data-act]');
    if (!node) return;
    const act = node.dataset.act;
    const id  = node.dataset.id;

    switch (act) {
      case 'nav':          go(node.dataset.key); break;
      case 'cp-list':      commandListModal(node.dataset.list); break;
      case 'cp-task':      commandTask(node.dataset.list); break;
      case 'cp-range':     ui.cp.range = node.dataset.range; renderView(); break;
      case 'cp-report':    commandReport(); break;
      case 'native-open':  if (NATIVE()) NATIVE().openDataDir(); break;
      case 'native-choose': nativeChooseDir(); break;
      case 'season':       commandListModal('season'); break;
      case 'snap-create':  snapCreate('manual').then(() => { toast('success', 'Точка восстановления создана', fmtStamp(new Date().toISOString())); fillSafetyAsync(); })
                           .catch(() => toast('danger', 'Не удалось создать точку', 'Хранилище браузера недоступно или переполнено.')); break;
      case 'snap-restore': snapRestore(id); break;
      case 'snap-download': snapDownload(id); break;
      case 'snap-delete':  snapDelete(id); break;
      case 'trash-restore': trashRestore(id); break;
      case 'trash-delete': trashDelete(id); break;
      case 'trash-empty':  trashEmpty(); break;
      case 'integrity-run': ui.safety.report = window.TST.platform.checkIntegrity(DB); renderView();
                            toast(ui.safety.report.errors.length ? 'warning' : 'success', 'Проверка целостности',
                              `Ошибок: ${ui.safety.report.errors.length}, замечаний: ${ui.safety.report.warnings.length}.`); break;
      case 'integrity-fix': integrityFix(); break;
      case 'tests-run':    window.TST.platform.runTests().then(r => {
                             ui.safety.tests = r; if (ui.view === 'safety') renderView();
                             const bad = r.filter(t => !t.ok).length;
                             toast(bad ? 'danger' : 'success', 'Самотесты модулей', bad ? `Не пройдено: ${bad} из ${r.length}.` : `Пройдено: ${r.length} из ${r.length}.`);
                           }); break;
      case 'help-safety':  if (window.TSTHelp) window.TSTHelp.open('s12'); break;
      case 'qr-show':      qrShowModal(id); break;
      case 'qr-print':     qrPrintModal([id], getVehicle(id) && getVehicle(id).qr ? { level: getVehicle(id).qr.level, format: getVehicle(id).qr.format } : {}); break;
      case 'qr-download': { const v = getVehicle(id); if (v) qrDownload(v, null, 'png'); break; }
      case 'qr-check': {
        const v = getVehicle(id);
        if (!v) break;
        const fit = qrFit(v, (v.qr && v.qr.level) || 'basic', (v.qr && v.qr.format) || '50x50');
        qrVerify(v, fit).then(msgs => {
          const fail = msgs.some(m => m[0] === '✗');
          toast(fail ? 'warning' : 'success', fail ? 'Проверка QR-кода: есть замечания' : 'QR-код корректен', msgs.join(' · '), 9000);
        });
        break;
      }
      case 'qr-print-sel': qrPrintModal(Array.from(ui.qr.selected)); break;
      case 'qr-sel':
        if (node.checked) ui.qr.selected.add(id); else ui.qr.selected.delete(id);
        renderView();
        break;
      case 'qr-sel-all': {
        const rows = qrRows();
        if (node.checked) rows.forEach(({ v }) => ui.qr.selected.add(v.id)); else rows.forEach(({ v }) => ui.qr.selected.delete(v.id));
        renderView();
        break;
      }
      case 'qr-filter':    ui.qr.filter = ui.qr.filter === node.dataset.f ? '' : node.dataset.f; renderView(); break;
      case 'qr-scan-text': { const inp = el('qrScanInput'); if (inp) qrOpenScanned(inp.value); break; }
      case 'qr-cam':       qrStartCamera(); break;
      case 'qr-cam-stop':  qrStopCamera(); break;
      case 'help-qr':      if (window.TSTHelp) window.TSTHelp.open('s11'); break;
      case 'goto-dash':    closeNotif(); go('dashboard'); break;
      case 'open-vehicle': closeNotif(); go('vehicle', id); break;
      case 'back-park':    go('park'); break;

      case 'add-vehicle':  vehicleModal(null); break;
      case 'edit-vehicle': vehicleModal(getVehicle(id)); break;
      case 'del-vehicle': {
        const v = getVehicle(id);
        if (!v) break;
        if (ui.view === 'vehicle') { ui.view = 'park'; renderNav(); }
        removeWithUndo('Техника удалена', `${v.name} — вместе с журналом ТО, моточасами и вложениями`,
          docs => removeVehicle(v, docs));
        break;
      }

      case 'vtab':
        ui.vehicleTab = node.dataset.tab;
        document.querySelectorAll('.f-tab').forEach(t => t.classList.toggle('is-active', t === node));
        refreshVehicleTab();
        break;

      case 'plantab':
```

```
        ui.planTab = node.dataset.tab;
        document.querySelectorAll('.f-tab').forEach(t => t.classList.toggle('is-active', t === node));
        refreshPlanTab();
        bindPlanningInputs();
        break;

      case 'add-maint':  maintModal(id); break;
      case 'edit-maint': {
        const m = DB.maint.find(x => x.id === id);
        if (m) maintModal(m.vehicleId, m);
        break;
      }
      case 'del-maint': {
        const m = DB.maint.find(x => x.id === id);
        if (!m) break;
        removeWithUndo('Запись ТО удалена', `${m.kind} от ${fmtDate(m.date)}`, docs => removeMaint(m, docs));
        break;
      }

      case 'add-trip':  tripModal(null); break;
      case 'edit-trip': {
        const t = DB.trips.find(x => x.id === id);
        if (t) tripModal(t);
        break;
      }
      case 'del-trip': {
        const t = DB.trips.find(x => x.id === id);
        if (!t) break;
        removeWithUndo('Путевой лист удалён', fmtDate(t.date), () => removeTrip(t));
        break;
      }
      case 'reset-trips':
        Object.assign(ui.trips, { vehicle: '', from: '', to: '' });
        renderView();
        break;

      case 'add-hours': hoursModal(id); break;

      case 'open-catalog':
        if (CAT()) CAT().open(id, { tab: node.dataset.tab });
        break;
      case 'print-checklist':
        printChecklist(node.dataset.cat, node.dataset.tab, id);
        break;
      case 'maint-from-norm':
        maintModal(id, null, { kind: CATALOG_TO_KIND[node.dataset.tab], fill: true });
        break;
      case 'edit-norm':
        normModal(id, node.dataset.tab || '');
        break;
      case 'print-need':
        runDoc('need');
        break;

      case 'del-hours': {
        const h = DB.hours.find(x => x.id === id);
        if (!h) break;
        if (h.maintId) { toast('warning', 'Запись связана с ТО', 'Удалите соответствующую запись технического обслуживания.'); break; }
        if (h.tripId)  { toast('warning', 'Запись связана с путевым листом', 'Удалите соответствующий путевой лист.'); break; }
        removeWithUndo('Показание удалено', `${fmtDate(h.date)} — ${fmtNum(h.value, 0)} мото·ч`, () => {
          DB.hours = DB.hours.filter(x => x.id !== h.id);
          recomputeHours(h.vehicleId);
        });
        break;
      }

      case 'vf':
        vfAction(node);
        break;

      case 'download-file':
        fileGet(id).then(rec => {
          if (!rec || !rec.blob) { toast('warning', 'Файл недоступен', 'Вложение не найдено в локальном хранилище.'); return; }
          downloadBlob(rec.blob, rec.name);
        });
        break;

      case 'sort': {
        const table = node.dataset.table;
        const key = node.dataset.key;
        const s = table === 'park' ? ui.park.sort : ui.dash.sort;
        if (s.key === key) s.dir = s.dir === 'asc' ? 'desc' : 'asc';
        else { s.key = key; s.dir = 'asc'; }
        if (table === 'park') refreshPark(); else refreshDash();
        break;
      }

```

```
      case 'mode':
        ui.park.mode = node.dataset.mode;
        renderView();
        break;

      case 'filter-level':
        ui.park.level = ui.park.level === node.dataset.level ? '' : node.dataset.level;
        renderView();
        break;

      case 'reset-filters':
        Object.assign(ui.park, { q: '', unit: '', responsible: '', status: '', level: '', from: '', to: '' });
        renderView();
        break;

      case 'row-select':
        if (node.checked) ui.park.selected.add(id); else ui.park.selected.delete(id);
        refreshPark();
        break;

      case 'select-all': {
        const visible = parkRows();
        if (node.checked) visible.forEach(({ v }) => ui.park.selected.add(v.id));
        else visible.forEach(({ v }) => ui.park.selected.delete(v.id));
        refreshPark();
        break;
      }

      case 'clear-selection':
        ui.park.selected.clear();
        refreshPark();
        break;

      case 'bulk-status':
        if (ui.park.selected.size) bulkStatusModal(Array.from(ui.park.selected));
        break;

      case 'bulk-interval':
        if (ui.park.selected.size) bulkIntervalModal(Array.from(ui.park.selected));
        break;

      case 'bulk-delete': {
        const ids = Array.from(ui.park.selected);
        if (!ids.length) break;
        removeWithUndo('Техника удалена', `Удалено ${ids.length} ед. техники со всеми записями`, docs =>
          ids.forEach(vid => { const v = getVehicle(vid); if (v) removeVehicle(v, docs); }));
        break;
      }

      case 'report':
        ui.report.key = node.dataset.key;
        renderView();
        break;
      case 'reset-report':
        Object.assign(ui.report, { from: '', to: '', unit: '' });
        renderView();
        break;
      case 'export-xlsx': exportXlsx(getReport(ui.report.key)); break;
      case 'export-pdf':  runDoc('report-' + getReport(ui.report.key).key); break;

      case 'print-annual-plan':    runDoc('annual'); break;
      case 'print-month-plan':     runDoc('month'); break;
      case 'print-act-writeoff':   runDoc('act-writeoff'); break;
      case 'print-act-inspection': runDoc('act-inspection'); break;
      case 'print-act-usage':      runDoc('act-usage'); break;
      case 'open-print':           openPrintWindow(); break;
      case 'help':                 if (window.TSTHelp) window.TSTHelp.open(); break;

      case 'apply-intervals': {
        const s = DB.settings;
        DB.vehicles.forEach(v => {
          v.interval = Object.assign({}, v.interval, { byHours: true, hours: s.defaultHours, byDate: true, days:
s.defaultDays });
        });
        saveDB();
        toast('success', 'Интервалы применены', `Обновлено ${DB.vehicles.length} ед. техники.`);
        rerender();
        break;
      }

      case 'seed-remove':    removeDemo(); break;
      case 'backup-export':  backupExport(); break;
      case 'persist-request': requestPersist(true); break;
      case 'backup-import':  backupImport(); break;
      case 'wipe':           wipeAll(); break;
    }
  }

  /* ========================================================================
     28. ПЕРЕКЛЮЧЕНИЕ РАБОЧИХ ПРОСТРАНСТВ
```

```
     ========================================================================== */
  function closeNotif() {
    el('fleetNotifPanel').classList.remove('is-open');
    el('fleetBell').setAttribute('aria-expanded', 'false');
  }

  /* Переключение разделов выполняет общий роутер TST.setWorkspace();
     модуль учёта только реагирует на вход в свой раздел и выход из него */
  function onWorkspace({ ws }) {
    if (ws === 'fleet') {
      rerender();
      startupNotify();
    } else {
      closeNotif();
    }
  }

  /* ========================================================================
     27b. QR-МАРКИРОВКА ТЕХНИКИ
     ------------------------------------------------------------------------
     Идентификатор QR-кода — постоянный id записи техники (v.id), который
     присваивается при постановке на учёт и не меняется при правке карточки.
     Формат содержимого (версия схемы 1):
       <сведения о технике>    — строки «Поле: значение» для чтения камерой
       TST:1:<id>              — последняя строка, машиночитаемая
     Вид кода по умолчанию — карточка контакта vCard (см. QR_ENCS): сведения
     в FN и NOTE, служебная строка — последней строкой NOTE.
     Служебная строка стоит в конце намеренно: текст, начинающийся с «TST:»,
     камера iPhone принимает за ссылку с неизвестной схемой и сообщает
     «Нет пригодных данных». Поэтому код всегда начинается с обычного текста.
     Уровни состава: id — только идентификатор; basic — основные сведения;
     full — расширенные (только для крупной наклейки). Сведения в коде — это
     снимок на дату печати: при изменении карточки наклейку нужно перепечатать
     (статус «устарела» отслеживается по контрольной сумме v.qr.digest).
     Ответственное лицо в код не включается: наклейку видит любой человек.
     ========================================================================== */
  const QR_SCHEME = 1;
  // Раскладка содержимого: 3 — карточка контакта и коррекция L при нехватке места;
  // 2 — текст со служебной строкой в конце; 1 — служебная строка первой. Наклейки
  // раскладок 1–2 камера iPhone не показывает полностью — их нужно перепечатать
  const QR_LAYOUT = 3;
  const QR_PREFIX = 'TST';
  const QR_ID_RE = /^[A-Za-z0-9_-]{4,64}$/;
  const QR_LEVELS = {
    id:    { label: 'Только идентификатор', hint: 'самый надёжный код; сведения открываются в программе с базой' },
    basic: { label: 'Основные сведения',    hint: 'наименование, номера, очередное ТО — читаются любой камерой' },
    full:  { label: 'Расширенные сведения', hint: 'плюс марка, подразделение, наработка; только для наклейки 70×70 мм' }
  };
  /* Форматы наклеек: размер, размер QR-кода (вместе с «тихой зоной»), раскладка */
  const QR_FORMATS = {
    '50x50': { w: 50, h: 50, qr: 37, layout: 'col', label: '50 × 50 мм' },
    '60x40': { w: 60, h: 40, qr: 36, layout: 'row', label: '60 × 40 мм' },
    '70x70': { w: 70, h: 70, qr: 52, layout: 'col', label: '70 × 70 мм (расширенные сведения)' }
  };
  const QR_MIN_MODULE_MM = 0.5;   // меньше — ненадёжное считывание камерой (проверено испытаниями)

  const qrOneLine = (s, max) => String(s == null ? '' : s).replace(/[\r\n\t]+/g, ' ').replace(/\s{2,}/g, '
').trim().slice(0, max || 60);

  /* Строки сведений без даты формирования — по ним считается актуальность */
  function qrDataLines(v, level) {
    if (level === 'id') return [];
    const st = computeStatus(v);
    const lines = [];
    const add = (k, val, max) => { const t = qrOneLine(val, max); if (t) lines.push(k + ': ' + t); };
    add('Наименование', v.name, 70);
    if (level === 'full') add('Марка, модель', v.brand, 50);
    add('Инв. №', v.invNumber, 30);
    add('Гос. №', v.regNumber, 20);
    add('Зав. №', v.serialNumber, 30);
    if (level === 'full') {
      add('Подразделение', v.unit, 60);
      add('Год выпуска', v.year, 6);
      add('На учёте с', v.regDate ? fmtDate(v.regDate) : '', 12);
      add('Моточасы', v.currentHours != null && v.currentHours !== '' ? fmtNum(v.currentHours, 0) : '', 12);
      if (st.lastMaint) add('Последнее ТО', st.lastMaint.kind.replace(/ \(.*\)$/, '') + ' ' +
fmtDate(st.lastMaint.date), 40);
    }
    if (st.level === 'retired') add('Состояние', 'снято с учёта', 20);
    else if (st.level === 'none') add('Очередное ТО', 'срок не определён (интервал не задан)', 50);
    else {
      const kind = nextMaintKind(v);
      add('Очередное ТО', kind + (st.nextDate ? ' до ' + fmtDate(st.nextDate) : '') + (st.nextHours != null ?
(st.nextDate ? ' / ' : ' при ') + fmtNum(st.nextHours, 0) + ' мото·ч' : ''), 60);
      if (level === 'full') add('Состояние ТО', LEVELS[st.level].label, 30);
    }
    return lines;
  }
  /* Вид кода. «Карточка контакта» (vCard) — камера iPhone показывает
     обычный текст в QR-коде лишь фрагментом (например, номер, принятый
```

```
      за телефон), а карточку контакта открывает целиком и без интернета;
      Android показывает её так же. «Обычный текст» — для сканеров и Android. */
    const QR_ENCS = {
      vcard: { label: 'Карточка контакта (iPhone и Android)' },
      text:  { label: 'Обычный текст (Android, сканеры)' }
    };
    const QR_ENC_KEY = 'tst-qr-enc';
    function qrEnc() { try { const e = localStorage.getItem(QR_ENC_KEY); return QR_ENCS[e] ? e : 'vcard'; } catch (e) {
    return 'vcard'; } }
    function qrSetEnc(e) { if (!QR_ENCS[e]) return; try { localStorage.setItem(QR_ENC_KEY, e); } catch (x) {} }
    const qrVEsc = t => String(t).replace(/\\/g, '\\\\').replace(/\n/g, '\\n').replace(/,/g, '\\,').replace(/;/g, '\\;');
    function qrPayload(v, level, date, enc) {
      const tag = QR_PREFIX + ':' + QR_SCHEME + ':' + v.id;
      const lines = level === 'id' ? [] : qrDataLines(v, level).concat(['Сведения на ' + fmtDate(date || todayISO())]);
      if ((enc || qrEnc()) === 'vcard') {
        const name = level === 'id' ? 'Регламент-ТС' : qrOneLine(v.name, 70);
        const note = lines.filter(l => !/^Наименование: /.test(l)).concat([tag]).join('\n');
        // Только N (без дублирующего FN) — иначе основные сведения не помещаются в наклейку 50 × 50
        return ['BEGIN:VCARD', 'VERSION:3.0', 'N:' + qrVEsc(name), 'NOTE:' + qrVEsc(note), 'END:VCARD'].join('\r\n');
      }
      if (level === 'id') return 'Регламент-ТС\n' + tag;
      return lines.concat([tag]).join('\n');
    }
    /* vCard → текст «Поле: значение» (только N/FN и NOTE; остальное не используется) */
    function qrFromVCard(raw) {
      const props = {};
      raw.replace(/\n[ \t]/g, '').split('\n').forEach(l => {
        const m = /^(N|FN|NOTE)(?:;[^:]*)?:(.*)$/i.exec(l);
        if (m && !(m[1].toUpperCase() in props)) props[m[1].toUpperCase()] = m[2];
      });
      const unesc = t => t.replace(/\\([\\,;nN])/g, (x, c) => (c === 'n' || c === 'N') ? '\n' : c);
      const note = unesc(props.NOTE || '');
      const name = props.FN != null ? unesc(props.FN) : (props.N ||
    '').split(/(?<!\\);/).map(unesc).filter(Boolean).join(' ');
      const named = name && /^[^:\n]+: /m.test(note);   // у наклейки «только идентификатор» в имени — название программы
      return (named ? 'Наименование: ' + name + '\n' : '') + note;
    }
    /* Контрольная сумма сведений (FNV-1a) — для признака «наклейка устарела» */
    function qrDigest(v, level) {
      const s = level + '|' + v.id + '|' + qrDataLines(v, level).join('\n');
      let h = 0x811c9dc5;
      for (let i = 0; i < s.length; i++) { h ^= s.charCodeAt(i); h = Math.imul(h, 0x01000193) >>> 0; }
      return h.toString(36);
    }
    /* Состояние маркировки единицы техники */
    function qrState(v) {
      if (!QR_ID_RE.test(String(v.id || ''))) return { code: 'bad', label: 'Некорректный идентификатор' };
      if (DB.vehicles.filter(x => x.id === v.id).length > 1) return { code: 'dup', label: 'Идентификатор повторяется' };
      const q = v.qr;
      if (!q || !q.issuedAt) return { code: 'none', label: 'Наклейка не печаталась' };
      if (q.layout !== QR_LAYOUT) return { code: 'stale', label: 'Старый формат кода (не читается iPhone) —
    перепечатайте', date: q.issuedAt };
      if (q.level !== 'id' && q.digest !== qrDigest(v, q.level)) return { code: 'stale', label: 'Сведения изменились —
    перепечатайте', date: q.issuedAt };
      return { code: 'ok', label: 'Наклейка актуальна', date: q.issuedAt };
    }
    function qrMark(v, level, format) {
      v.qr = { scheme: QR_SCHEME, layout: QR_LAYOUT, enc: qrEnc(), level, format: format || '', digest: qrDigest(v,
    level), issuedAt: new Date().toISOString() };
    }
    /* Наибольший состав данных, при котором модуль на наклейке не меньше нормы */
    function qrFit(v, level, format) {
      const f = QR_FORMATS[format] || QR_FORMATS['50x50'];
      const order = ['full', 'basic', 'id'];
      let i = order.indexOf(level);
      if (i < 0) i = 1;
      if (format !== '70x70' && i === 0) i = 1;
      for (; i < order.length; i++) {
        const text = qrPayload(v, order[i]);
        // Сначала коррекция M (15 %); если сведения не помещаются — L (7 %), и лишь затем уменьшается состав
        for (const ecl of order[i] === 'id' ? ['M'] : ['M', 'L']) {
          const code = QR.encode(text, { ecl });
          const mm = f.qr / (code.size + 8);
          if (mm >= QR_MIN_MODULE_MM || order[i] === 'id') return { level: order[i], text, code, moduleMm: mm, reduced:
    order[i] !== level };
        }
      }
    }

    /* Разбор считанного содержимого. Ничего не выполняется и не вставляется
       как HTML: текст только сравнивается с шаблоном и выводится через esc() */
    function qrParse(raw) {
      const text = String(raw == null ? '' : raw).replace(/^ /, '').replace(/\r\n?/g, '\n').trim();
      if (!text) return { ok: false, error: 'Пустое содержимое' };
      if (text.length > 3000) return { ok: false, error: 'Слишком длинное содержимое — это не QR-код программы' };
      if (/^BEGIN:VCARD/i.test(text)) {
        const conv = qrFromVCard(text).trim();
        // Карточка склеена в одну строку (вставка в однострочное поле) — берётся только идентификатор
        return /(?:^|\s)TST:/.test(conv) ? qrParseText(conv) : qrParseText(text.replace(/\\n/g, '\n'), true);
      }
```

```
    return qrParseText(text);
  }
  function qrParseText(text, noFields) {
    // Служебная метка TST:<версия>:<идентификатор> — отдельной строкой (в новых
    // наклейках последней, в первых выпусках — первой). Если строки склеены
    // (вставка в однострочное поле), идентификатор заканчивается на пробеле.
    // Две разные метки в одном коде — признак подделки: такой код отклоняется.
    const tags = [...text.matchAll(/(?:^|\s)TST:(\d{1,3}):(\S*)/g)];
    if (!tags.length) return { ok: false, error: 'Это не QR-код программы «Регламент-ТС»: нет служебной строки TST:', text };
    if (new Set(tags.map(t => t[1] + ':' + t[2])).size > 1) return { ok: false, error: 'В коде несколько разных идентификаторов — код отклонён', text };
    const m = tags[0];
    if (Number(m[1]) !== QR_SCHEME) return { ok: false, error: 'Неподдерживаемая версия формата QR-кода: ' + m[1], text };
    if (!QR_ID_RE.test(m[2])) return { ok: false, error: 'Повреждённый идентификатор в QR-коде', text };
    const fields = {};
    // Строки склеены — границы полей утрачены: сведения не сравниваются, только идентификатор
    if (!noFields && text.includes('\n')) text.split('\n').filter(l => !/^TST:/.test(l.trim())).forEach(l => { const i = l.indexOf(': '); if (i > 0) fields[l.slice(0, i).trim()] = l.slice(i + 2).trim(); });
    return { ok: true, id: m[2], fields, text };
  }

  /* ---------- Окно «QR-код техники» ---------- */
  function qrSvgFor(text, px) { return QR.svg(QR.encode(text, { ecl: 'M' }), { size: px, label: 'QR-код техники' }); }

  function qrShowModal(id) {
    const v = getVehicle(id);
    if (!v) return;
    let level = (v.qr && v.qr.level) || 'basic', format = (v.qr && v.qr.format) || '50x50';
    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">QR-код техники</div>
          <div class="f-modal-sub">${esc(v.name)}${v.invNumber ? ' · инв. № ' + esc(v.invNumber) : ''}</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <div class="qr-show">
          <div class="qr-show-code" id="qrBig"></div>
          <div class="qr-show-side">
            <div class="f-form-grid" style="grid-template-columns:1fr">
              <div><label class="f-fld-label" for="qrLevel">Состав данных в коде</label>
                <select id="qrLevel" class="f-select">${Object.keys(QR_LEVELS).map(k => `<option value="${k}" ${k === level ? 'selected' : ''}>${esc(QR_LEVELS[k].label)}</option>`).join('')}</select>
                <div class="f-hint" id="qrLevelHint"></div></div>
              <div><label class="f-fld-label" for="qrFormat">Формат наклейки</label>
                <select id="qrFormat" class="f-select">${Object.keys(QR_FORMATS).map(k => `<option value="${k}" ${k === format ? 'selected' : ''}>${esc(QR_FORMATS[k].label)}</option>`).join('')}</select></div>
              <div><label class="f-fld-label" for="qrEnc">Вид кода</label>
                <select id="qrEnc" class="f-select">${Object.keys(QR_ENCS).map(k => `<option value="${k}" ${k === qrEnc() ? 'selected' : ''}>${esc(QR_ENCS[k].label)}</option>`).join('')}</select></div>
            </div>
            <div class="qr-meta" id="qrMeta"></div>
          </div>
        </div>
        <div class="f-fld-label" style="margin-top:14px">Содержимое кода</div>
        <pre class="qr-text" id="qrText"></pre>
        <div class="f-hint" id="qrCheck"></div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" id="qrCheckBtn">${FI.gauge(15)}<span>Проверить QR-код</span></button>
        <button type="button" class="f-btn" id="qrPng">${FI.download(15)}<span>Скачать PNG</span></button>
        <button type="button" class="f-btn" id="qrSvgBtn">${FI.download(15)}<span>Скачать SVG</span></button>
        <button type="button" class="f-btn f-btn-primary" id="qrPrintBtn">${FI.pdf(15)}<span>Печать QR-наклейки</span></button>
      </div>`);
    const r = m.root;
    let fit = null;
    const draw = () => {
      level = r.querySelector('#qrLevel').value; format = r.querySelector('#qrFormat').value;
      fit = qrFit(v, level, format);
      r.querySelector('#qrBig').innerHTML = QR.svg(fit.code, { label: 'QR-код: ' + esc(v.name) });
      r.querySelector('#qrText').textContent = fit.text;
      r.querySelector('#qrLevelHint').textContent = QR_LEVELS[level].hint;
      const s = qrState(v);
      r.querySelector('#qrMeta').innerHTML = `
        <div class="f-kv"><span class="f-kv-k">Идентификатор</span><span class="f-kv-v qr-mono">${esc(v.id)}</span></div>
        <div class="f-kv"><span class="f-kv-k">Версия кода / коррекция</span><span class="f-kv-v">${fit.code.version} / ${fit.code.ecl}</span></div>
        <div class="f-kv"><span class="f-kv-k">Объём</span><span class="f-kv-v">${fit.code.bytes} байт</span></div>
        <div class="f-kv"><span class="f-kv-k">Модуль на наклейке</span><span class="f-kv-v">${fmtNum(fit.moduleMm, 2)} мм</span></div>
        <div class="f-kv"><span class="f-kv-k">Наклейка</span><span class="f-kv-v">${esc(s.label)}${s.date ? ' (' + fmtDate(s.date.slice(0, 10)) + ')' : ''}</span></div>
        ${fit.reduced ? `<div class="f-hint" style="color:var(--st-urgent-text)">Выбранный состав не помещается в наклейку ${esc(QR_FORMATS[format].label)} с надёжным размером модуля (≥ ${fmtNum(QR_MIN_MODULE_MM, 1)} мм) — использован состав «${esc(QR_LEVELS[fit.level].label)}».</div>` : ''}`;
```

```
      r.querySelector('#qrCheck').textContent = '';
    };
    r.querySelector('#qrLevel').addEventListener('change', draw);
    r.querySelector('#qrFormat').addEventListener('change', draw);
    r.querySelector('#qrEnc').addEventListener('change', e => { qrSetEnc(e.target.value); draw(); });
    r.querySelector('#qrPng').addEventListener('click', () => { qrDownload(v, fit, 'png'); });
    r.querySelector('#qrSvgBtn').addEventListener('click', () => { qrDownload(v, fit, 'svg'); });
    r.querySelector('#qrCheckBtn').addEventListener('click', () => qrVerify(v, fit, r.querySelector('#qrCheck')));
    r.querySelector('#qrPrintBtn').addEventListener('click', () => { m.close(); qrPrintModal([v.id], { level, format });
   });
    draw();
  }

  function qrDownload(v, fit, kind) {
    fit = fit || qrFit(v, (v.qr && v.qr.level) || 'basic', (v.qr && v.qr.format) || '50x50');
    const name = 'qr-' + (qrOneLine(v.invNumber || v.regNumber || v.id, 40).replace(/[^\wA-Яа-яЁё-]+/g, '_')) + '.' +
  kind;
    let blob;
    if (kind === 'svg') blob = new Blob([QR.svg(fit.code, { size: 512 })], { type: 'image/svg+xml' });
    else {
      const c = QR.canvas(fit.code, 12);
      const bin = atob(c.toDataURL('image/png').split(',')[1]);
      const u8 = new Uint8Array(bin.length);
      for (let i = 0; i < bin.length; i++) u8[i] = bin.charCodeAt(i);
      blob = new Blob([u8], { type: 'image/png' });
    }
    const a = document.createElement('a');
    a.href = URL.createObjectURL(blob); a.download = name;
    document.body.appendChild(a); a.click();
    setTimeout(() => { URL.revokeObjectURL(a.href); a.remove(); }, 800);
    qrMark(v, fit.level, v.qr && v.qr.format); saveDB();
    toast('success', 'QR-код сохранён', name);
  }

  /* Проверка: (1) идентификатор корректен и уникален; (2) содержимое
     разбирается обратно в ту же запись; (3) если браузер умеет распознавать
     QR (BarcodeDetector) — код распознаётся с изображения */
  async function qrVerify(v, fit, out) {
    const msgs = [];
    const s = qrState(v);
    msgs.push((s.code === 'bad' || s.code === 'dup' ? '✗ ' : '✓ ') + 'Идентификатор: ' + (s.code === 'bad' || s.code ===
   'dup' ? s.label : 'корректен и уникален'));
    const p = qrParse(fit.text);
    msgs.push((p.ok && p.id === v.id ? '✓ ' : '✗ ') + 'Разбор содержимого: ' + (p.ok && p.id === v.id ? 'указывает на
  эту единицу техники' : (p.error || 'идентификатор не совпадает')));
    msgs.push((fit.moduleMm >= QR_MIN_MODULE_MM || fit.level === 'id' ? '✓ ' : '⚠ ') + 'Размер модуля на наклейке: ' +
  fmtNum(fit.moduleMm, 2) + ' мм');
    if ('BarcodeDetector' in window) {
      try {
        const det = new window.BarcodeDetector({ formats: ['qr_code'] });
        const res = await det.detect(QR.canvas(fit.code, 8));
        const ok = res.length && res[0].rawValue === fit.text;
        msgs.push((ok ? '✓ ' : '✗ ') + 'Распознавание с изображения: ' + (ok ? 'код читается, содержимое совпадает' :
  'код не распознан'));
      } catch (e) { msgs.push('— Распознавание с изображения недоступно в этом браузере'); }
    } else msgs.push('— Встроенного распознавания QR в этом браузере нет; проверьте напечатанный код камерой
  смартфона');
    if (out) out.innerHTML = msgs.map(esc).join('<br>');
    return msgs;
  }

  /* ---------- Печать наклеек ---------- */
  function qrStickerHtml(v, level, format) {
    const f = QR_FORMATS[format];
    const fit = qrFit(v, level, format);
    const inv = v.invNumber ? 'Инв. № ' + v.invNumber : (v.regNumber ? 'Гос. № ' + v.regNumber : '');
    const model = v.brand || (catNodeOf(v) ? catNodeOf(v).name : '');
    return { fit, html: `
      <div class="qr-stk qr-stk-${f.layout} qr-stk-${format}" style="width:${f.w}mm;height:${f.h}mm">
        <div class="qr-stk-code" style="width:${f.qr}mm;height:${f.qr}mm">${QR.svg(fit.code)}</div>
        <div class="qr-stk-text">
          <div class="qr-stk-name">${esc(qrOneLine(v.name, 70))}</div>
          ${inv ? `<div class="qr-stk-inv">${esc(qrOneLine(inv, 40))}</div>` : ''}
          ${model ? `<div class="qr-stk-model">${esc(qrOneLine(model, 50))}</div>` : ''}
          <div class="qr-stk-hint">Отсканируйте QR-код для просмотра карточки техники</div>
          <div class="qr-stk-id">${esc(v.id)}</div>
        </div>
      </div>` };
  }
  function qrSheetHtml(ids, opts) {
    const f = QR_FORMATS[opts.format];
    const list = ids.map(getVehicle).filter(Boolean);
    const stickers = [];
    list.forEach(v => { for (let k = 0; k < (opts.copies || 1); k++) stickers.push(qrStickerHtml(v, opts.level,
  opts.format).html); });
    if (opts.media === 'roll') return { pages: stickers.map(s => `<div class="qr-page qr-roll">${s}</div>`), css:
  `@page{size:${f.w}mm ${f.h}mm;margin:0}` };
    // Лист А4: сетка наклеек с полями 10 мм и линиями реза
    const cols = Math.floor(190 / f.w), rows = Math.floor(277 / f.h), per = cols * rows;
    const pages = [];
```

```
    for (let i = 0; i < stickers.length; i += per)
      pages.push(`<div class="qr-page qr-a4"><div class="qr-grid" style="grid-template-columns:repeat(${cols},${f.w}mm);grid-auto-rows:${f.h}mm">${stickers.slice(i, i + per).join('')}</div></div>`);
    return { pages, css: '@page{size:A4 portrait;margin:10mm}', perSheet: per };
  }
  function qrPrintModal(ids, init) {
    ids = ids.filter(id => getVehicle(id));
    if (!ids.length) { toast('warning', 'Не выбрана техника', 'Отметьте единицы техники для печати наклеек.'); return; }
    init = init || {};
    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">Печать QR-наклеек</div>
          <div class="f-modal-sub">Единиц техники: ${ids.length}</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <div class="f-form-grid">
          <div><label class="f-fld-label" for="qpFormat">Формат наклейки</label>
            <select id="qpFormat" class="f-select">${Object.keys(QR_FORMATS).map(k => `<option value="${k}" ${k === (init.format || '50x50') ? 'selected' : ''}>${esc(QR_FORMATS[k].label)}</option>`).join('')}</select></div>
          <div><label class="f-fld-label" for="qpMedia">Носитель</label>
            <select id="qpMedia" class="f-select"><option value="a4">Лист А4 (несколько наклеек, линии реза)</option><option value="roll" ${init.media === 'roll' ? 'selected' : ''}>Принтер этикеток (одна наклейка на лист)</option></select></div>
          <div><label class="f-fld-label" for="qpLevel">Состав данных в коде</label>
            <select id="qpLevel" class="f-select">${Object.keys(QR_LEVELS).map(k => `<option value="${k}" ${k === (init.level || 'basic') ? 'selected' : ''}>${esc(QR_LEVELS[k].label)}</option>`).join('')}</select></div>
          <div><label class="f-fld-label" for="qpCopies">Экземпляров на единицу</label>
            <input id="qpCopies" class="f-input" type="number" min="1" max="20" value="1"></div>
          <div><label class="f-fld-label" for="qpEnc">Вид кода</label>
               <select id="qpEnc" class="f-select">${Object.keys(QR_ENCS).map(k => `<option value="${k}" ${k === qrEnc() ? 'selected' : ''}>${esc(QR_ENCS[k].label)}</option>`).join('')}</select></div>
        </div>
        <div class="f-hint" id="qpInfo" style="margin-top:10px"></div>
        <div class="qr-preview" id="qpPreview" aria-label="Предварительный просмотр наклеек"></div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary" id="qpPrint">${FI.pdf(15)}<span>Печать</span></button>
      </div>`);
    const r = m.root;
    const read = () => ({ format: r.querySelector('#qpFormat').value, media: r.querySelector('#qpMedia').value,
      level: r.querySelector('#qpLevel').value, copies: clamp(parseInt(r.querySelector('#qpCopies').value, 10) || 1, 1, 20) });
    const draw = () => {
      const o = read();
      const sheet = qrSheetHtml(ids, o);
      const f = QR_FORMATS[o.format];
      const fits = ids.map(id => qrFit(getVehicle(id), o.level, o.format));
      const reduced = fits.filter(x => x.reduced).length;
      const minMm = Math.min.apply(null, fits.map(x => x.moduleMm));
      r.querySelector('#qpInfo').innerHTML = esc(`Наклеек: ${ids.length * o.copies}; ${o.media === 'a4' ? `листов А4: ${sheet.pages.length} (по ${sheet.perSheet} на листе)` : 'по одной на лист ' + f.w + '×' + f.h + ' мм'}; QR-код ${f.qr} мм, модуль от ${fmtNum(minMm, 2)} мм.`) +
        (reduced ? `<br><span style="color:var(--st-urgent-text)">Для ${reduced} ед. состав данных уменьшен, чтобы код уверенно считывался на этом формате.</span>` : '') +
        `<br>В окне печати выберите масштаб «100 %» (без «по размеру страницы»), иначе размеры наклеек изменятся.`;
      r.querySelector('#qpPreview').innerHTML = sheet.pages.slice(0, 2).join('');
    };
    ['#qpFormat', '#qpMedia', '#qpLevel', '#qpCopies'].forEach(s => r.querySelector(s).addEventListener('change', draw));
    r.querySelector('#qpEnc').addEventListener('change', e => { qrSetEnc(e.target.value); draw(); });
    r.querySelector('#qpPrint').addEventListener('click', () => {
      const o = read();
      qrPrint(ids, o);
      m.close();
    });
    draw();
  }
  let qrPrintStyle = null;
  function qrPrint(ids, o) {
    const sheet = qrSheetHtml(ids, o);
    if (!qrPrintStyle) { qrPrintStyle = document.createElement('style'); qrPrintStyle.id = 'qr-print-page'; document.head.appendChild(qrPrintStyle); }
    qrPrintStyle.textContent = sheet.css;
    const tst = document.getElementById('tst-print-page');
    if (tst) tst.textContent = '';                 // поля и колонтитулы окна печати документов здесь не нужны
    el('fleetPrint').innerHTML = sheet.pages.join('');
    ids.forEach(id => { const v = getVehicle(id); if (v) qrMark(v, qrFit(v, o.level, o.format).level, o.format); });
    saveDB();
    const done = () => { qrPrintStyle.textContent = ''; window.removeEventListener('afterprint', done); };
    window.addEventListener('afterprint', done);
    try { window.print(); } catch (e) { toast('warning', 'Печать недоступна', 'Браузер не поддерживает печать из программы.'); }
    if (ui.view === 'qr' || ui.view === 'vehicle') renderView();
  }

  /* ---------- Сканирование и карточка по QR ---------- */
```

```
  function qrOpenScanned(raw) {
    const p = qrParse(raw);
    if (!p.ok) { qrScanResult({ error: p.error, text: p.text }); return; }
    const v = getVehicle(p.id);
    if (!v) { qrScanResult({ error: 'Техника с идентификатором ' + p.id + ' не найдена в локальной базе этой программы.', text: p.text, notFound: true }); return; }
    // Сверка сведений наклейки с базой — расхождения показываются, а не исправляются
    const diffs = [];
    const cmp = (k, cur) => { if (p.fields[k] != null && qrOneLine(cur, 200) && p.fields[k] !== qrOneLine(cur, p.fields[k].length)) diffs.push(k + ': на наклейке «' + p.fields[k] + '», в базе «' + qrOneLine(cur, 80) + '»'); };
    cmp('Наименование', v.name); cmp('Инв. №', v.invNumber); cmp('Гос. №', v.regNumber); cmp('Зав. №', v.serialNumber);
    qrScanResult({ v, diffs, text: p.text });
  }
  function qrScanResult(r) {
    const NA = '<span class="f-td-muted">Не указано</span>';
    const row = (k, val) => `<div class="qr-card-row"><span>${esc(k)}</span><b>${val == null || val === '' ? NA : esc(val)}</b></div>`;
    let body;
    if (r.error) {
      body = `<div class="qr-alert ${r.notFound ? 'is-warn' : 'is-bad'}">${esc(r.error)}</div>
        ${r.text ? `<div class="f-fld-label" style="margin-top:12px">Считанное содержимое</div><pre class="qr-text">${esc(String(r.text).slice(0, 1500))}</pre>` : ''}`;
    } else {
      const v = r.v, st = computeStatus(v), ms = maintOf(v.id);
      const node = catNodeOf(v);
      const status = st.level === 'retired' ? 'Снято с учёта'
        : st.level === 'none' ? 'Недостаточно данных для определения срока ТО'
        : st.level === 'overdue' ? 'ТО просрочено'
        : (st.level === 'soon' || st.level === 'urgent') ? 'Приближается срок ТО (' + LEVELS[st.level].label + ')'
        : 'ТО по текущим данным не требуется';
      const nextTxt = st.level === 'none' || st.level === 'retired' ? '' :
        nextMaintKind(v) + (st.nextDate ? ' до ' + fmtDate(st.nextDate) : '') + (st.nextHours != null ? ' · при ' + fmtNum(st.nextHours, 0) + ' мото·ч' : '');
      const specs = node && node.specs && node.specs.groups && node.specs.groups[0] ? node.specs.groups[0].params.slice(0, 6) : [];
      body = `
        ${r.diffs && r.diffs.length ? `<div class="qr-alert is-warn">Сведения на наклейке отличаются от базы — наклейку нужно перепечатать:<br>${r.diffs.map(esc).join('<br>')}</div>` : ''}
        <div class="qr-status lvl-${st.level}"><span class="f-dot f-lvl-${st.level}"></span>${esc(status)}</div>
        <section class="qr-card-sec"><h4>Основные сведения</h4>
          ${row('Наименование', v.name)}${row('Тип по справочнику', node ? node.name : '')}${row('Марка и модель', v.brand)}
          ${row('Инвентарный номер', v.invNumber)}${row('Государственный номер', v.regNumber)}${row('Заводской номер', v.serialNumber)}
          ${row('Подразделение', v.unit)}${row('Идентификатор записи', v.id)}</section>
        <section class="qr-card-sec"><h4>Технические характеристики</h4>
          ${row('Год выпуска', v.year)}${row('Статус', v.status)}${row('Группа эксплуатации', v.expGroup)}
          ${specs.map(p => row(p.label, p.value)).join('')}</section>
        <section class="qr-card-sec"><h4>Учёт и эксплуатация</h4>
          ${row('Дата постановки на учёт', v.regDate ? fmtDate(v.regDate) : '')}
          ${row('Моточасы', v.currentHours != null && v.currentHours !== '' ? fmtNum(v.currentHours, 0) + ' мото·ч' : '')}
          ${row('Моточасы обновлены', v.hoursUpdatedAt ? fmtDate(v.hoursUpdatedAt) : '')}
          ${row('Последнее ТО', st.lastMaint ? st.lastMaint.kind + ' · ' + fmtDate(st.lastMaint.date) : '')}
          ${row('Очередное ТО', nextTxt)}
          ${row('Состояние ТО', LEVELS[st.level].label)}</section>
        <section class="qr-card-sec"><h4>Выполненные работы (последние записи журнала ТО)</h4>
          ${ms.length ? ms.slice(0, 5).map(x => `<div class="qr-card-row"><span>${esc(fmtDate(x.date))}</span><b>${esc(x.kind)}${x.works ? ' — ' + esc(qrOneLine(x.works, 90)) : ''}</b></div>`).join('') : `<div class="qr-card-row"><span>Записей нет</span><b></b></div>`}</section>`;
    }
    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">${r.v ? esc(r.v.name) : 'Результат сканирования'}</div>
          <div class="f-modal-sub">Карточка по QR-коду · только просмотр${r.v ? ' · данные из базы на ' + esc(fmtDate(todayISO())) : ''}</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body qr-card">${body}</div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Закрыть</button>
        ${r.v ? `<button type="button" class="f-btn f-btn-primary" id="qrOpenFull">${FI.file(15)}<span>Открыть карточку техники</span></button>` : ''}
      </div>`, { narrow: true });
    const b = m.root.querySelector('#qrOpenFull');
    if (b) b.addEventListener('click', () => { m.close(); go('vehicle', r.v.id); });
  }

  /* Камера / фото: встроенный BarcodeDetector (Chrome, Edge, Android). Если его
     нет — содержимое кода вставляется в поле (сканер штрихкодов или
     копирование из приложения камеры) */
  let qrStream = null;
  function qrStopCamera() {
    if (qrStream) { qrStream.getTracks().forEach(t => t.stop()); qrStream = null; }
    const box = el('qrCamBox'); if (box) box.hidden = true;
  }
  async function qrStartCamera() {
    if (!('BarcodeDetector' in window)) { toast('warning', 'Камера недоступна', 'В этом браузере нет встроенного
```

```
распознавания QR-кодов. Отсканируйте код камерой смартфона и вставьте текст в поле.'); return; }
5941      if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) { toast('warning', 'Камера недоступна',
'Браузер не предоставляет доступ к камере для этой страницы.'); return; }
5942      try {
5943        qrStream = await navigator.mediaDevices.getUserMedia({ video: { facingMode: 'environment' } });
5944      } catch (e) { toast('warning', 'Нет доступа к камере', 'Разрешите доступ к камере в браузере или используйте «Фото с
 QR-кодом».'); return; }
5945      const box = el('qrCamBox'), video = el('qrVideo');
5946      if (!box || !video) { qrStopCamera(); return; }
5947      box.hidden = false; video.srcObject = qrStream; await video.play().catch(() => {});
5948      const det = new window.BarcodeDetector({ formats: ['qr_code'] });
5949      const tick = async () => {
5950        if (!qrStream) return;
5951        try {
5952          const res = await det.detect(video);
5953          if (res.length) { qrStopCamera(); qrOpenScanned(res[0].rawValue); return; }
5954        } catch (e) { /* кадр ещё не готов */ }
5955        setTimeout(tick, 250);
5956      };
5957      tick();
5958    }
5959    async function qrFromPhoto(file) {
5960      if (!file) return;
5961      if (!('BarcodeDetector' in window)) { toast('warning', 'Распознавание недоступно', 'В этом браузере нет встроенного
распознавания QR-кодов.'); return; }
5962      try {
5963        const bmp = await createImageBitmap(file);
5964        const res = await new window.BarcodeDetector({ formats: ['qr_code'] }).detect(bmp);
5965        if (!res.length) { qrScanResult({ error: 'На фотографии QR-код не найден или он повреждён. Сфотографируйте код
ближе и ровнее, без бликов.' }); return; }
5966        qrOpenScanned(res[0].rawValue);
5967      } catch (e) { qrScanResult({ error: 'Не удалось прочитать изображение.' }); }
5968    }
5969
5970    /* ---------- Раздел «QR-маркировка» ---------- */
5971    function qrRows() {
5972      const q = (ui.qr.q || '').trim().toLowerCase();
5973      const rows = DB.vehicles.map(v => ({ v, s: qrState(v) }));
5974      return rows.filter(({ v, s }) => {
5975        if (ui.qr.filter && s.code !== ui.qr.filter && !(ui.qr.filter === 'need' && (s.code === 'none' || s.code ===
'stale'))) return false;
5976        if (!q) return true;
5977        const hay = [v.name, v.invNumber, v.regNumber, v.serialNumber, v.id, v.brand].filter(Boolean).join('
').toLowerCase();
5978        return hay.indexOf(q) >= 0 || hay.replace(/\s+/g, '').indexOf(q.replace(/\s+/g, '')) >= 0;
5979      }).sort((a, b) => cmp(a.v.name, b.v.name, 'asc'));
5980    }
5981    function renderQr() {
5982      const rows = qrRows(), sel = ui.qr.selected;
5983      const all = DB.vehicles.map(v => qrState(v));
5984      const cnt = c => all.filter(s => s.code === c).length;
5985      const bad = cnt('bad') + cnt('dup');
5986      const allSel = rows.length > 0 && rows.every(({ v }) => sel.has(v.id));
5987      return `
5988        <div class="f-page-head">
5989          <div>
5990            <div class="f-page-title" role="heading" aria-level="1">QR-маркировка техники</div>
5991            <div class="f-page-sub">Индивидуальные QR-коды, печать наклеек и просмотр карточки по коду — без
интернета</div>
5992          </div>
5993          <div class="f-page-actions">
5994            <button type="button" class="f-btn" data-act="help-qr">${FI.doc(15)}<span>Как это работает</span></button>
5995            <button type="button" class="f-btn f-btn-primary" data-act="qr-print-sel" ${sel.size ? '' :
'disabled'}>${FI.pdf(15)}<span>Печать выбранных (${sel.size})</span></button>
5996          </div>
5997        </div>
5998
5999        <div class="f-stats">
6000          <div class="f-stat"><div class="f-stat-label">Единиц на учёте</div><div class="f-stat-
value">${DB.vehicles.length}</div></div>
6001          <div class="f-stat is-clickable" data-act="qr-filter" data-f="ok"><div class="f-stat-label"><span class="f-dot
f-lvl-ok"></span>Наклейка актуальна</div><div class="f-stat-value">${cnt('ok')}</div></div>
6002          <div class="f-stat is-clickable" data-act="qr-filter" data-f="none"><div class="f-stat-label"><span class="f-dot
 f-lvl-none"></span>Не печаталась</div><div class="f-stat-value">${cnt('none')}</div></div>
6003          <div class="f-stat is-clickable" data-act="qr-filter" data-f="stale"><div class="f-stat-label"><span
class="f-dot f-lvl-urgent"></span>Устарела</div><div class="f-stat-value">${cnt('stale')}</div></div>
6004          <div class="f-stat"><div class="f-stat-label"><span class="f-dot f-lvl-overdue"></span>Ошибки
идентификатора</div><div class="f-stat-value" ${bad ? 'style="color:var(--st-overdue-text)"' : ''}>${bad}</div></div>
6005        </div>
6006
6007        <div class="f-panel">
6008          <div class="f-panel-title">Сканирование QR-кода</div>
6009          <div class="qr-scan">
6010            <textarea id="qrScanInput" class="f-textarea qr-scan-input" rows="2" autocomplete="off" spellcheck="false"
6011                  placeholder="Отсканируйте сканером или вставьте содержимое кода (строка TST:1:…)" aria-
label="Содержимое QR-кода"></textarea>
6012            <button type="button" class="f-btn f-btn-primary" data-act="qr-scan-text">Открыть карточку</button>
6013            <button type="button" class="f-btn" data-act="qr-cam">Камера</button>
6014            <label class="f-btn" style="cursor:pointer">Фото с QR-кодом<input type="file" id="qrPhoto" accept="image/*"
capture="environment" hidden></label>
```

```
        </div>
        <div class="qr-cam" id="qrCamBox" hidden><video id="qrVideo" playsinline muted></video>
          <button type="button" class="f-btn f-btn-sm" data-act="qr-cam-stop">Остановить камеру</button></div>
        <div class="f-hint" style="margin-top:8px">${'BarcodeDetector' in window ? 'Камера и фото распознаются браузером на этом устройстве.' : 'В этом браузере нет встроенного распознавания QR: используйте поле ввода — USB-сканер вводит код сам, со смартфона содержимое можно скопировать.'} Сканирование ничего не меняет в данных.</div>
      </div>

      <div class="f-toolbar">
        <div class="f-fld f-search"><label class="f-fld-label" for="qrQ">Поиск</label>
          <input id="qrQ" class="f-input" type="search" value="${esc(ui.qr.q)}" placeholder="Наименование, инв. №, гос. №, идентификатор"></div>
        <div class="f-fld"><label class="f-fld-label" for="qrF">Наклейка</label>
          <select id="qrF" class="f-select">${[['', 'Все'], ['need', 'Нужно напечатать'], ['ok', 'Актуальна'], ['none', 'Не печаталась'], ['stale', 'Устарела'], ['bad', 'Ошибка идентификатора']].map(o => `<option value="${o[0]}" ${o[0] === ui.qr.filter ? 'selected' : ''}>${o[1]}</option>`).join('')}</select></div>
      </div>

      ${!rows.length ? emptyState(FI.grid(28), 'Нет техники', DB.vehicles.length ? 'По условиям поиска ничего не найдено.' : 'Поставьте технику на учёт — QR-код появится автоматически.') : `
      <div class="f-table-wrap"><div class="f-table-scroll">
        <table class="f-table f-table-qr">
          <thead><tr>
            <th style="width:36px"><input type="checkbox" data-act="qr-sel-all" aria-label="Выбрать всё" ${allSel ? 'checked' : ''}></th>
            <th style="width:64px">QR</th><th>Техника</th><th>Идентификатор</th><th>Наклейка</th><th style="width:150px"><span class="sr-only">Действия</span></th>
          </tr></thead>
          <tbody>${rows.map(({ v, s }) => `
            <tr>
              <td><input type="checkbox" data-act="qr-sel" data-id="${esc(v.id)}" aria-label="Выбрать: ${esc(v.name)}" ${sel.has(v.id) ? 'checked' : ''}></td>
              <td><div class="qr-thumb" data-act="qr-show" data-id="${esc(v.id)}" title="Показать QR-код">${qrSvgFor(QR_PREFIX + ':' + QR_SCHEME + ':' + v.id)}</div></td>
              <td><div class="f-td-strong">${esc(v.name)}</div><div class="f-td-muted f-fit-sub">${esc([v.invNumber ? 'инв. № ' + v.invNumber : '', v.regNumber].filter(Boolean).join(' · ') || 'номера не указаны')}</div></td>
              <td><span class="qr-mono">${esc(v.id)}</span></td>
              <td><span class="f-badge ${s.code === 'ok' ? 'f-badge-ok' : s.code === 'stale' ? 'f-badge-urgent' : s.code === 'none' ? 'f-badge-none' : 'f-badge-overdue'}">${esc(s.label)}</span>
                ${s.date ? `<div class="f-td-muted f-fit-sub">${esc(fmtDate(s.date.slice(0, 10)))} · ${esc((QR_LEVELS[v.qr.level] || {}).label || '')}</div>` : ''}</td>
              <td style="white-space:nowrap"><button type="button" class="f-btn f-btn-sm" data-act="qr-show" data-id="${esc(v.id)}">Показать</button>
                <button type="button" class="f-btn f-btn-sm" data-act="qr-print" data-id="${esc(v.id)}">Печать</button></td>
            </tr>`).join('')}</tbody>
        </table>
      </div></div>`}`;
}
function bindQr() {
  const q = el('qrQ');
  if (q) q.addEventListener('input', () => { ui.qr.q = q.value; const pos = q.selectionStart; renderView(); const n = el('qrQ'); if (n) { n.focus(); n.setSelectionRange(pos, pos); } });
  const f = el('qrF');
  if (f) f.addEventListener('change', () => { ui.qr.filter = f.value; renderView(); });
  /* Сканер штрихкодов «печатает» код и нажимает Enter после каждой строки:
     ждём паузы во вводе и только тогда открываем карточку */
  const inp = el('qrScanInput');
  if (inp) {
    let timer = null;
    const later = () => { clearTimeout(timer); timer = setTimeout(() => { timer = null; if (inp.value.trim()) qrOpenScanned(inp.value); }, 350); };
    inp.addEventListener('keydown', e => {
      if (e.key === 'Enter' && !e.shiftKey) {
        e.preventDefault();
        const a = inp.selectionStart, b = inp.selectionEnd;
        inp.value = inp.value.slice(0, a) + '\n' + inp.value.slice(b);
        inp.selectionStart = inp.selectionEnd = a + 1;
        later();
      }
    });
    inp.addEventListener('input', () => { if (timer) later(); });
  }
  const ph = el('qrPhoto');
  if (ph) ph.addEventListener('change', () => { qrFromPhoto(ph.files && ph.files[0]); ph.value = ''; });
}

/* Блок в карточке техники («Основные сведения») */
function renderVehicleQr(v) {
  const s = qrState(v);
  return `
    <div class="f-panel qr-panel">
      <div class="f-panel-title">QR-маркировка</div>
      <div class="qr-panel-body">
        <div class="qr-thumb qr-thumb-lg" data-act="qr-show" data-id="${esc(v.id)}" title="Показать QR-код">${qrSvgFor(QR_PREFIX + ':' + QR_SCHEME + ':' + v.id)}</div>
        <div style="min-width:0;flex:1">
          <div class="f-kv"><span class="f-kv-k">Идентификатор</span><span class="f-kv-v qr-mono">${esc(v.id)}</span></div>
          <div class="f-kv"><span class="f-kv-k">Наклейка</span><span class="f-kv-v">${esc(s.label)}${s.date ? ' · ' +
```

```
 esc(fmtDate(s.date.slice(0, 10))) : ''}</span></div>
          <div class="qr-panel-btns">
            <button type="button" class="f-btn f-btn-sm" data-act="qr-show" data-id="${esc(v.id)}">Показать QR-код</button>
            <button type="button" class="f-btn f-btn-sm f-btn-primary" data-act="qr-print" data-id="${esc(v.id)}">Печать QR-наклейки</button>
            <button type="button" class="f-btn f-btn-sm" data-act="qr-download" data-id="${esc(v.id)}">Скачать QR-код</button>
            <button type="button" class="f-btn f-btn-sm" data-act="qr-check" data-id="${esc(v.id)}">Проверить QR-код</button>
          </div>
        </div>
      </div>
    </div>`;
}

/* ========================================================================
   27c. СОХРАННОСТЬ ДАННЫХ
   ------------------------------------------------------------------------
   Данные хранятся только в этом браузере на этом компьютере, поэтому защита
   от потери — отдельная подсистема, а не набор разрозненных кнопок:

     • точки восстановления — автоматические снимки базы в IndexedDB:
       ежедневно и перед каждой опасной операцией (восстановление из копии,
       очистка, обновление структуры данных, исправление ошибок, откат);
     • корзина — всё удалённое хранится trashDays дней (по умолчанию 30)
       и возвращается одной кнопкой, вместе со связанными записями;
     • журнал изменений важных полей карточки техники;
     • проверка целостности перед восстановлением из копии и по запросу;
     • версия структуры данных и безопасная миграция при обновлении
       программы (перед миграцией — точка восстановления).

   Вложения (PDF, изображения) в точки восстановления не копируются: они
   лежат в отдельном хранилище и не удаляются, пока на них ссылается хотя бы
   одна запись — текущая, в корзине или в точке восстановления.
   ======================================================================== */

/* --- Модель данных учёта для платформы --- */
function defineFleetModel() {
  const p = window.TST && window.TST.platform;
  if (!p) return;
  const drop = { vehicleId: 'drop-orphan' };
  p.defineCollection('vehicles', {
    owner: 'park', required: ['name'], unique: ['invNumber', 'regNumber', 'serialNumber'],
    labels: { name: 'Наименование', invNumber: 'Инвентарный номер', regNumber: 'Государственный номер', serialNumber: 'Заводской номер' },
    fixRequired: { name: 'set:name:Без наименования' },
    check(v, ctx, issue) {
      if (v.status && !STATUSES.includes(v.status)) issue('warn', `Неизвестный статус «${v.status}»`);
      if (v.catalogId && CAT() && !CAT().get(v.catalogId)) issue('warn', 'Привязка к справочнику указывает на отсутствующую позицию');
      if (v.regDate && !toDate(v.regDate)) issue('warn', 'Некорректная дата постановки на учёт');
      if (v.currentHours != null && v.currentHours !== '' && !validHours(Number(v.currentHours))) issue('warn', 'Недопустимое значение моточасов');
    }
  });
  p.defineCollection('maint', {
    owner: 'maint', required: ['vehicleId', 'kind', 'date'], refs: { vehicleId: 'vehicles' },
    labels: { vehicleId: 'Техника', kind: 'Вид ТО', date: 'Дата' }, fixRequired: drop,
    check(m, ctx, issue) {
      if (m.kind && !MAINT_KINDS.includes(m.kind)) issue('warn', `Неизвестный вид обслуживания «${m.kind}»`);
      if (m.date && !toDate(m.date)) issue('warn', 'Некорректная дата проведения');
    }
  });
  p.defineCollection('hours', {
    owner: 'trips', required: ['vehicleId'], refs: { vehicleId: 'vehicles', maintId: 'maint', tripId: 'trips' },
    labels: { vehicleId: 'Техника', maintId: 'Запись ТО', tripId: 'Путевой лист' }, fixRequired: drop,
    check(h, ctx, issue) { if (!validHours(Number(h.value))) issue('warn', 'Недопустимое показание счётчика'); }
  });
  p.defineCollection('trips', { owner: 'trips', required: ['vehicleId'], refs: { vehicleId: 'vehicles' }, labels: { vehicleId: 'Техника' }, fixRequired: drop });
  p.defineCollection('norms', { owner: 'catalog' });
  p.defineCollection('catalogEdits', { owner: 'catalog' });
  p.defineCollection('trash', { owner: 'safety' });
  p.defineCollection('history', { owner: 'safety' });
}

/* --- Версия структуры данных и миграции ---------------------------------
   Каждая миграция переводит базу на одну версию вперёд и принадлежит
   модулю-владельцу. Перед миграцией создаётся точка восстановления со
   старыми данными. База из более новой версии программы не понижается:
   неизвестные поля сохраняются как есть, пользователь получает
   предупреждение */
const MIGRATIONS = [
  { to: 2, module: 'safety', name: 'Корзина, журнал изменений, статус «Неисправна»',
    run(d) { d.trash = Array.isArray(d.trash) ? d.trash : []; d.history = Array.isArray(d.history) ? d.history : []; }
  }
];
let newerData = null;     // { from, to } — база создана более новой версией программы
let lastMigration = null; // { from, to } — миграция при этом запуске
```

```
6166
6167    function migrateDB(d, from, rawForSnapshot) {
6168      from = Number(from) || 1;
6169      if (from > SCHEMA_VERSION) { newerData = { from, to: SCHEMA_VERSION }; return d; }
6170      if (from >= SCHEMA_VERSION) { d.version = SCHEMA_VERSION; return d; }
6171      if (rawForSnapshot) snapCreate('migration', `Перед обновлением структуры данных (версия ${from} →
      ${SCHEMA_VERSION})`, rawForSnapshot).catch(() => null);
6172      MIGRATIONS.filter(m => m.to > from && m.to <= SCHEMA_VERSION).sort((a, b) => a.to - b.to).forEach(m => {
6173        m.run(d);
6174        d.migrations = (Array.isArray(d.migrations) ? d.migrations : []).concat([{ to: m.to, from, name: m.name, module:
      m.module, at: new Date().toISOString() }]);
6175      });
6176      d.version = SCHEMA_VERSION;
6177      lastMigration = { from, to: SCHEMA_VERSION };
6178      if (window.TST && window.TST.platform) window.TST.emit(window.TST.platform.EVENTS.MIGRATED, lastMigration);
6179      return d;
6180    }
6181
6182    /* --- Журнал изменений важных полей ------------------------------------ */
6183    const HISTORY_FIELDS = {
6184      name: 'Наименование', brand: 'Марка, модель', invNumber: 'Инвентарный номер', regNumber: 'Государственный номер',
6185      serialNumber: 'Заводской номер', unit: 'Подразделение', responsible: 'Ответственный', status: 'Статус',
6186      expGroup: 'Группа эксплуатации', catalogId: 'Тип по справочнику', regDate: 'Дата постановки на учёт', year: 'Год
      выпуска'
6187    };
6188    const HISTORY_MAX = 3000;
6189
6190    /* Разница между двумя состояниями списка техники — чистая функция */
6191    function historyDiff(beforeList, afterList, at) {
6192      const before = new Map((beforeList || []).filter(Boolean).map(v => [v.id, v]));
6193      const after = new Map((afterList || []).filter(Boolean).map(v => [v.id, v]));
6194      const out = [];
6195      const val = x => (x == null ? '' : String(x));
6196      after.forEach((v, id) => {
6197        const o = before.get(id);
6198        if (!o) { out.push({ id: uid('e'), at, vehicleId: id, name: v.name || '', action: 'create' }); return; }
6199        Object.keys(HISTORY_FIELDS).forEach(f => {
6200          if (val(o[f]) !== val(v[f])) out.push({ id: uid('e'), at, vehicleId: id, name: v.name || '', action: 'edit',
      field: f, from: val(o[f]), to: val(v[f]) });
6201        });
6202      });
6203      before.forEach((o, id) => { if (!after.has(id)) out.push({ id: uid('e'), at, vehicleId: id, name: o.name || '',
      action: 'delete' }); });
6204      return out;
6205    }
6206    function recordHistory() {
6207      let prev = null;
6208      try { prev = baseJSON ? JSON.parse(baseJSON) : null; } catch (e) { prev = null; }
6209      if (!prev) return;
6210      const add = historyDiff(prev.vehicles, DB.vehicles, new Date().toISOString());
6211      if (!add.length) return;
6212      DB.history = (DB.history || []).concat(add);
6213      if (DB.history.length > HISTORY_MAX) DB.history = DB.history.slice(-HISTORY_MAX);
6214    }
6215    function historyValue(field, value) {
6216      if (!value) return '—';
6217      if (field === 'catalogId') { const c = CAT(); const n = c && c.get(value); return n ? n.name : value; }
6218      if (field === 'regDate') return fmtDate(value);
6219      return value;
6220    }
6221    function historyText(e) {
6222      if (e.action === 'create') return 'Запись создана (постановка на учёт или восстановление)';
6223      if (e.action === 'delete') return 'Запись удалена (перемещена в корзину)';
6224      return `${HISTORY_FIELDS[e.field] || e.field}: «${historyValue(e.field, e.from)}» → «${historyValue(e.field,
      e.to)}»`;
6225    }
6226    const fmtStamp = iso => iso ? fmtDate(iso.slice(0, 10)) + ' ' + new Date(iso).toLocaleTimeString('ru-RU', { hour:
      '2-digit', minute: '2-digit' }) : '—';
6227
6228    /* --- Корзина ---------------------------------------------------------- */
6229    function trashDays() { return Math.max(1, Number(DB.settings.trashDays) || 30); }
6230    function trashPush(title, text, removed) {
6231      const records = {};
6232      let n = 0;
6233      COLLECTIONS.forEach(k => { if (removed[k] && removed[k].length) { records[k] = removed[k]; n += removed[k].length; }
       });
6234      if (!n) return null;
6235      const entry = { id: uid('r'), at: new Date().toISOString(), title, text: text || '', records };
6236      DB.trash = (DB.trash || []).concat([entry]);
6237      return entry.id;
6238    }
6239    const trashCount = t => COLLECTIONS.reduce((s, k) => s + ((t.records && t.records[k]) || []).length, 0);
6240    /* Возврат записей из корзины в базу db (чистая функция над переданной базой).
6241       Записи, чей «родитель» (техника) отсутствует, не возвращаются — иначе
6242       появились бы записи-сироты */
6243    function trashApply(db, entry) {
6244      const back = {}, skipped = [];
6245      COLLECTIONS.forEach(k => {
6246        const list = (entry.records && entry.records[k]) || [];
6247        if (!list.length) return;
```

```
      const have = new Set(db[k].map(r => r.id));
      back[k] = list.filter(r => !have.has(r.id));
    });
    db.vehicles = db.vehicles.concat(back.vehicles || []);
    const vids = new Set(db.vehicles.map(v => v.id));
    ['maint', 'trips', 'hours'].forEach(k => {
      if (!back[k]) return;
      back[k] = back[k].filter(r => { const ok = !r.vehicleId || vids.has(r.vehicleId); if (!ok) skipped.push(r); return ok; });
    });
    ['maint', 'trips', 'hours', 'norms', 'catalogEdits'].forEach(k => { if (back[k]) db[k] = db[k].concat(back[k]); });
    db.trash = db.trash.filter(t => t.id !== entry.id);
    const touched = new Set([].concat((back.vehicles || []).map(v => v.id), (back.maint || []).map(m => m.vehicleId),
      (back.hours || []).map(h => h.vehicleId), (back.trips || []).map(t => t.vehicleId)).filter(Boolean));
    touched.forEach(id => recomputeHours(id, db));
    return { restored: Object.keys(back).reduce((s, k) => s + back[k].length, 0), skipped: skipped.length };
  }
  function trashRestore(id) {
    const entry = (DB.trash || []).find(t => t.id === id);
    if (!entry) return;
    const r = trashApply(DB, entry);
    saveDB();
    rerender();
    toast('success', 'Восстановлено из корзины', `${entry.title}: возвращено записей — ${r.restored}` +
      (r.skipped ? `; не возвращено ${r.skipped} (связанная техника отсутствует — сначала восстановите её)` : ''));
  }
  async function trashDelete(id) {
    const entry = (DB.trash || []).find(t => t.id === id);
    if (!entry) return;
    const ok = await confirmDialog('Удалить навсегда?', `«${entry.title}» будет удалено без возможности возврата из корзины (останется только в точках восстановления).`, 'Удалить навсегда');
    if (!ok) return;
    DB.trash = DB.trash.filter(t => t.id !== id);
    saveDB(); rerender();
    setTimeout(cleanupOrphanFiles, 1500);
  }
  async function trashEmpty() {
    if (!(DB.trash || []).length) return;
    const ok = await confirmDialog('Очистить корзину?', `Будет окончательно удалено элементов: ${DB.trash.length}. Перед очисткой создаётся точка восстановления.`, 'Очистить корзину');
    if (!ok) return;
    await snapCreate('trash', 'Перед очисткой корзины').catch(() => null);
    DB.trash = [];
    saveDB(); rerender();
    setTimeout(cleanupOrphanFiles, 1500);
  }
  function trashPurgeOld() {
    const limit = new Date(Date.now() - trashDays() * MS_DAY).toISOString();
    const before = (DB.trash || []).length;
    DB.trash = (DB.trash || []).filter(t => (t.at || '') >= limit);
    if (DB.trash.length !== before) saveDB();
  }

  /* --- Точки восстановления (IndexedDB «tst-fleet-snapshots») -------------- */
  const SNAP_DB = 'tst-fleet-snapshots', SNAP_ST = 'snaps';
  const SNAP_KEEP_AUTO = 20, SNAP_KEEP_MANUAL = 20;
  const SNAP_REASONS = {
    manual: 'Создана вручную', daily: 'Ежедневная', import: 'Перед восстановлением из копии', wipe: 'Перед очисткой всех данных',
    migration: 'Перед обновлением структуры данных', rollback: 'Перед откатом', repair: 'Перед исправлением ошибок', trash: 'Перед очисткой корзины'
  };
  let snapPromise = null;
  function snapIdb() {
    if (snapPromise) return snapPromise;
    snapPromise = new Promise((resolve, reject) => {
      if (!('indexedDB' in window)) { reject(new Error('IndexedDB недоступна')); return; }
      const req = indexedDB.open(SNAP_DB, 1);
      req.onupgradeneeded = () => { if (!req.result.objectStoreNames.contains(SNAP_ST)) req.result.createObjectStore(SNAP_ST, { keyPath: 'id' }); };
      req.onsuccess = () => resolve(req.result);
      req.onerror = () => reject(req.error);
    });
    snapPromise.catch(() => { snapPromise = null; });
    return snapPromise;
  }
  function snapRun(mode, fn) {
    return snapIdb().then(d => new Promise((resolve, reject) => {
      const tx = d.transaction(SNAP_ST, mode);
      const req = fn(tx.objectStore(SNAP_ST));
      tx.oncomplete = () => resolve(req && req.result);
      tx.onerror = () => reject(tx.error);
      tx.onabort = () => reject(tx.error);
    }));
  }
  const dbCounts = d => ({ vehicles: (d.vehicles || []).length, maint: (d.maint || []).length, trips: (d.trips || []).length, hours: (d.hours || []).length });
  async function snapCreate(reason, label, raw) {
    const data = raw || JSON.stringify(DB);
    let counts = {};
```

```
    try { counts = dbCounts(raw ? JSON.parse(raw) : DB); } catch (e) { /* повреждённые данные тоже сохраняются как есть */ }
    const rec = { id: uid('s'), at: new Date().toISOString(), reason, label: label || SNAP_REASONS[reason] || reason,
                  auto: reason !== 'manual', size: data.length, counts, schema: SCHEMA_VERSION, data };
    await snapRun('readwrite', st => st.put(rec));
    nativeSnapPut(rec);
    await snapPrune();
    if (window.TST && window.TST.platform) window.TST.emit(window.TST.platform.EVENTS.SNAPSHOT, { reason });
    return rec;
  }
  async function snapAll() { return (await snapRun('readonly', st => st.getAll())) || []; }
  async function snapList() {
    return (await snapAll()).map(s => Object.assign({}, s, { data: undefined })).sort((a, b) => (b.at || '').localeCompare(a.at || ''));
  }
  async function snapPrune() {
    const list = await snapList();
    const kill = list.filter(s => s.auto).slice(SNAP_KEEP_AUTO).concat(list.filter(s => !s.auto).slice(SNAP_KEEP_MANUAL));
    for (const s of kill) { await snapRun('readwrite', st => st.delete(s.id)); nativeSnapDel(s.id); }
  }
  async function snapAutoDaily() {
    const hasData = DB.vehicles.length || DB.maint.length;
    if (!hasData) return;
    try {
      const list = await snapList();
      const last = list.find(s => s.reason === 'daily');
      if (!last || Date.now() - new Date(last.at).getTime() > 20 * 3600 * 1000) await snapCreate('daily');
    } catch (e) { /* IndexedDB недоступна — раздел «Сохранность данных» сообщит об этом */ }
  }
  async function snapRestore(id) {
    let rec, parsed;
    try { rec = await snapRun('readonly', st => st.get(id)); parsed = JSON.parse(rec.data); }
    catch (e) { toast('danger', 'Точка восстановления повреждена', 'Не удалось прочитать сохранённые данные.'); return; }
    const decision = await importReview(parsed, null, 'точка восстановления от ' + fmtStamp(rec.at), true);
    if (!decision) return;
    if (decision === 'repair') parsed = window.TST.platform.repair(normalizeDB(parsed)).db;
    await snapCreate('rollback', 'Перед откатом к точке от ' + fmtStamp(rec.at)).catch(() => null);
    DB = migrateDB(normalizeDB(parsed), parsed.version);
    migrateCatalogLinks();
    resetUiSelection();
    saveDB({ replace: true });
    toast('success', 'Данные восстановлены', `Состояние на ${fmtStamp(rec.at)}: ${DB.vehicles.length} ед. техники. Предыдущее состояние сохранено в точке «Перед откатом».`, 8000);
    go('safety');
  }
  async function snapDownload(id) {
    const rec = await snapRun('readonly', st => st.get(id)).catch(() => null);
    if (!rec) return;
    downloadBlob(new Blob([rec.data], { type: 'application/json' }), `tst_snapshot_${(rec.at || '').slice(0,16).replace(/[:T]/g, '-')}.json`);
    toast('info', 'Точка восстановления сохранена в файл', 'Файл принимается кнопкой «Восстановить из копии» (вложения в него не входят).');
  }
  async function snapDelete(id) {
    const ok = await confirmDialog('Удалить точку восстановления?', 'Откатиться к этому состоянию будет нельзя.', 'Удалить');
    if (!ok) return;
    await snapRun('readwrite', st => st.delete(id)).catch(() => null);
    nativeSnapDel(id);
    fillSafetyAsync();
  }

  /* Вложения, на которые ссылается хоть что-нибудь: текущие записи ТО,
     корзина и точки восстановления. Если точки прочитать не удалось —
     исключение: тогда очистка вложений не выполняется вовсе */
  async function referencedFileIds() {
    const used = new Set();
    const take = list => (list || []).forEach(m => (m && m.docs || []).forEach(d => d && used.add(d.id)));
    const takeV = list => (list || []).forEach(v => v && vfFileIds(v).forEach(id => used.add(id)));
    take(DB.maint); takeV(DB.vehicles);
    (DB.trash || []).forEach(t => { take(t.records && t.records.maint); takeV(t.records && t.records.vehicles); });
    if ('indexedDB' in window) {
      const all = await snapAll();
      all.forEach(s => { try { const d = JSON.parse(s.data); take(d.maint); takeV(d.vehicles); } catch (e) { /* повреждённая точка — пропускаем */ } });
    }
    return used;
  }

  /* --- Проверка перед восстановлением -----------------------------------
     Показывает, что именно будет загружено, и результат проверки
     целостности. Возвращает Promise: false — отмена, 'restore', 'repair' */
  function importReview(parsed, files, sourceName, isSnapshot) {
    const P = window.TST && window.TST.platform;
    let cand, rep;
    try { cand = normalizeDB(parsed); rep = P ? P.checkIntegrity(cand) : { errors: [], warnings: [], fixable: 0 }; }
    catch (e) { rep = { errors: [{ text: 'Структура данных не распознана' }], warnings: [], fixable: 0 }; cand = blankDB(); }
```

```
  const ver = Number(parsed && parsed.version) || 1;
  const verText = ver > SCHEMA_VERSION ? `версия данных ${ver} новее программы (${SCHEMA_VERSION}) — неизвестные поля сохранятся, но могут не отображаться`
      : ver < SCHEMA_VERSION ? `версия данных ${ver} — структура будет обновлена до версии ${SCHEMA_VERSION}` : `версия данных ${ver} — совпадает с программой`;
  const row = (label, k) => `<tr><td>${label}</td><td class="f-td-num">${fmtNum((DB[k] || []).length, 0)}</td><td class="f-td-num"><b>${fmtNum((cand[k] || []).length, 0)}</b></td></tr>`;
  const list = (arr, cls) => arr.slice(0, 8).map(x => `<li class="${cls}">${esc(x.text)}${x.id ? ` <span class="f-td-muted qr-mono">${esc(x.id)}</span>` : ''}</li>`).join('') +
    (arr.length > 8 ? `<li class="f-td-muted">… и ещё ${arr.length - 8}</li>` : '');
  return new Promise(resolve => {
    const m = openModal(`
      <div class="f-modal-head">
        <div>
          <div class="f-modal-title">${isSnapshot ? 'Откат к точке восстановления' : 'Проверка резервной копии'}</div>
          <div class="f-modal-sub">${esc(sourceName || '')} · ${esc(verText)}</div>
        </div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <table class="f-table sf-cmp"><thead><tr><th>Записи</th><th class="f-td-num">Сейчас</th><th class="f-td-num">Будет загружено</th></tr></thead><tbody>
          ${row('Единицы техники', 'vehicles')}${row('Записи ТО и ремонтов', 'maint')}${row('Путевые листы', 'trips')}${row('Показания моточасов', 'hours')}${row('Элементы корзины', 'trash')}
        </tbody></table>
        ${files ? `<p class="f-hint">Вложений в копии: ${files.length}. Уже сохранённые вложения не удаляются.</p>` : isSnapshot ? '<p class="f-hint">Вложения в точку восстановления не входят — сохранённые файлы останутся на месте.</p>' : '<p class="f-hint">Копия в формате JSON — вложений в ней нет, сохранённые файлы останутся.</p>'}
        <div class="sf-check ${rep.errors.length ? 'is-bad' : rep.warnings.length ? 'is-warn' : 'is-ok'}">
          <b>${rep.errors.length ? `Ошибки целостности: ${rep.errors.length}` : 'Связи и идентификаторы в порядке'}${rep.warnings.length ? ` · замечаний: ${rep.warnings.length}` : ''}</b>
          ${rep.errors.length || rep.warnings.length ? `<ul class="sf-issues">${list(rep.errors, 'is-err')}${list(rep.warnings, 'is-warn')}</ul>` : ''}
          ${rep.fixable ? '<p class="f-hint">«Исправить и загрузить» удалит записи без связанной техники и заменит повторяющиеся идентификаторы новыми.</p>' : ''}
        </div>
        <p class="f-hint">Текущие данные будут заменены. Перед заменой автоматически создаётся точка восстановления — вернуться можно в разделе «Сохранность данных».</p>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        ${rep.fixable ? `<button type="button" class="f-btn f-btn-primary" data-dec="repair">Исправить и загрузить</button>` : ''}
        <button type="button" class="f-btn ${rep.fixable ? 'f-btn-danger' : 'f-btn-primary'}" data-dec="restore">${rep.errors.length ? 'Загрузить без исправления' : 'Загрузить'}</button>
      </div>`, { onClose: () => resolve(false) });
    m.root.querySelectorAll('[data-dec]').forEach(b => b.addEventListener('click', () => { resolve(b.dataset.dec); m.close(); }));
  });
}

/* --- Проверка и исправление текущей базы --- */
async function integrityFix() {
  const P = window.TST.platform;
  const { db, fixes } = P.repair(DB);
  if (!fixes.length) { toast('info', 'Исправлять нечего', 'Автоматически исправимых ошибок не найдено.'); return; }
  const ok = await confirmDialog('Исправить ошибки целостности?', `Будет выполнено исправлений: ${fixes.length}. Перед исправлением создаётся точка восстановления.`, 'Исправить');
  if (!ok) return;
  await snapCreate('repair').catch(() => null);
  DB = normalizeDB(db);
  db.vehicles.forEach(v => recomputeHours(v.id));
  saveDB({ replace: true });
  ui.safety.report = P.checkIntegrity(DB);
  ui.safety.fixes = fixes;
  rerender();
  toast('success', 'Ошибки исправлены', `Исправлений: ${fixes.length}.`);
}

/* ========================================================================
   РАЗДЕЛ «СОХРАННОСТЬ ДАННЫХ»
   ======================================================================== */
function renderSafety() {
  const P = window.TST && window.TST.platform;
  const b = backupInfo();
  const rep = ui.safety.report || (P ? P.checkIntegrity(DB) : null);
  ui.safety.report = rep;
  const trash = (DB.trash || []).slice().sort((a, x) => (x.at || '').localeCompare(a.at || ''));
  const hist = (DB.history || []).slice().reverse().filter(e => !ui.safety.hq || (e.name + ' ' + historyText(e)).toLowerCase().includes(ui.safety.hq.toLowerCase())).slice(0, 150);
  const mods = P ? P.modules() : [];
  const tests = ui.safety.tests;
  const testFor = id => tests ? tests.filter(t => t.module === id) : null;
  return `
    <div class="f-page-head">
      <div>
        <div class="f-page-title" role="heading" aria-level="1">Сохранность данных</div>
        <div class="f-page-sub">Резервные копии, точки восстановления, корзина, журнал изменений, проверка целостности и состав платформы</div>
      </div>
```

```
        <div class="f-page-actions">
          <button type="button" class="f-btn" data-act="help-safety">${FI.doc(15)}<span>Как это работает</span></button>
        </div>
      </div>
      ${newerData ? `<div class="f-panel sf-alert is-bad"><b>Данные созданы более новой версией программы</b> (структура ${newerData.from}, программа понимает ${newerData.to}). Работать можно, но новые разделы данных не отображаются. Обновите файл программы.</div>` : ''}
      ${lastMigration ? `<div class="f-panel sf-alert is-ok"><b>Структура данных обновлена</b> с версии ${lastMigration.from} до ${lastMigration.to}. Прежнее состояние сохранено в точке восстановления «Перед обновлением структуры данных».</div>` : ''}

      <div class="f-stats">
        <div class="f-stat ${b.due ? 'sf-due' : ''}"><div class="f-stat-label">Резервная копия</div>
          <div class="f-stat-value sf-sm">${b.last ? (b.daysAgo <= 0 ? 'сегодня' : b.daysAgo + ' ' + plural(b.daysAgo, 'день', 'дня', 'дней') + ' назад') : 'не создавалась'}</div>
          <div class="f-hint">напоминание каждые ${b.remind} ${plural(b.remind, 'день', 'дня', 'дней')}</div></div>
        <div class="f-stat"><div class="f-stat-label">Точки восстановления</div><div class="f-stat-value" id="sfSnapCount">…</div><div class="f-hint">автоматически: ежедневно и перед опасными операциями</div></div>
        <div class="f-stat"><div class="f-stat-label">В корзине</div><div class="f-stat-value">${trash.length}</div><div class="f-hint">хранится ${trashDays()} ${plural(trashDays(), 'день', 'дня', 'дней')}</div></div>
        <div class="f-stat"><div class="f-stat-label">Целостность</div>
          <div class="f-stat-value" style="color:var(--st-${rep && rep.errors.length ? 'overdue' : rep && rep.warnings.length ? 'soon' : 'ok'}-text)">${rep ? (rep.errors.length ? rep.errors.length + ' ' + plural(rep.errors.length, 'ошибка', 'ошибки', 'ошибок') : 'в порядке') : '—'}</div>
          <div class="f-hint">${rep ? 'замечаний: ' + rep.warnings.length : ''}</div></div>
      </div>

      <div class="f-panel">
        <div class="f-panel-title">Резервная копия на внешний носитель</div>
        <p class="f-hint">${NATIVE() ? 'Папка данных защищает от очистки браузера и переустановки программы, но не от поломки компьютера: храните копию вне него' : 'Единственная защита от поломки компьютера или переустановки браузера — копия вне этого компьютера'} (флешка, сетевой диск). ${esc(backupAgoText(b).replace(/^./, c => c.toUpperCase()))}.</p>
        <div class="sf-actions">
          <button type="button" class="f-btn f-btn-primary" data-act="backup-export">${FI.download(15)}<span>Создать резервную копию</span></button>
          <button type="button" class="f-btn" data-act="backup-import">${FI.upload(15)}<span>Восстановить из копии</span></button>
        </div>
        <div class="f-info-grid" id="setStorageStatus" style="margin-top:12px"></div>
        <button type="button" class="f-btn" id="setPersistBtn" data-act="persist-request" hidden style="margin-top:8px">Включить постоянное хранение</button>
      </div>

      ${renderNativeStorage()}

      <div class="f-panel">
        <div class="f-panel-title">Точки восстановления</div>
        <p class="f-hint">${NATIVE() ? 'Снимки базы хранятся в папке данных (snapshots).' : 'Снимки базы в этом браузере.'} Позволяют вернуться к состоянию до сбоя, ошибочного импорта или неудачного обновления. Хранится до ${SNAP_KEEP_AUTO} автоматических и ${SNAP_KEEP_MANUAL} ручных точек. Вложения в снимки не входят.</p>
        <div class="sf-actions"><button type="button" class="f-btn" data-act="snap-create">${FI.save(15)}<span>Создать точку сейчас</span></button></div>
        <div id="sfSnaps"><div class="f-hint">Загрузка…</div></div>
      </div>

      <div class="f-panel">
        <div class="f-panel-title">Корзина</div>
        <p class="f-hint">Удалённые карточки, записи ТО, путевые листы и показания хранятся ${trashDays()} ${plural(trashDays(), 'день', 'дня', 'дней')} и возвращаются вместе со связанными записями. Срок хранения задаётся в настройках.</p>
        ${trash.length ? `
          <div class="f-table-wrap"><div class="f-table-scroll"><table class="f-table f-table-stack">
            <thead><tr><th>Удалено</th><th>Что</th><th class="f-td-num">Записей</th><th style="width:170px"></th></tr></thead>
            <tbody>${trash.map(t => `<tr>
              <td class="f-td-nowrap" data-label="Удалено">${esc(fmtStamp(t.at))}</td>
              <td class="is-head"><div class="f-td-strong">${esc(t.title)}</div><div class="f-td-muted f-fit-sub">${esc(t.text)}</div></td>
              <td class="f-td-num" data-label="Записей">${trashCount(t)}</td>
              <td class="is-acts"><div class="sf-row-acts">
                <button type="button" class="f-btn f-btn-sm" data-act="trash-restore" data-id="${t.id}">Восстановить</button>
                <button type="button" class="f-icon-btn" data-act="trash-delete" data-id="${t.id}" title="Удалить навсегда" aria-label="Удалить навсегда">${FI.trash(15)}</button></div></td>
            </tr>`).join('')}</tbody></table></div></div>
          <div class="sf-actions" style="margin-top:10px"><button type="button" class="f-btn" data-act="trash-empty">${FI.trash(15)}<span>Очистить корзину</span></button></div>`
        : '<div class="f-hint">Корзина пуста.</div>'}
      </div>

      <div class="f-panel">
        <div class="f-panel-title">Проверка целостности</div>
        <p class="f-hint">Связи между записями (ТО, путевые листы и моточасы ↔ техника), уникальность идентификаторов, обязательные поля, допустимые значения, повторы инвентарных, государственных и заводских номеров.</p>
        <div class="sf-actions">
          <button type="button" class="f-btn" data-act="integrity-run">${FI.gauge(15)}<span>Проверить сейчас</span></button>
          ${rep && rep.fixable ? `<button type="button" class="f-btn f-btn-primary" data-act="integrity-fix">${FI.wrench(15)}<span>Исправить автоматически (${rep.fixable})</span></button>` : ''}
        </div>
```

```
        ${rep ? `<div class="sf-check ${rep.errors.length ? 'is-bad' : rep.warnings.length ? 'is-warn' : 'is-ok'}">
          <b>${rep.errors.length ? 'Ошибок: ' + rep.errors.length : 'Ошибок нет'} · замечаний: ${rep.warnings.length}</b> <span class="f-td-muted">проверено ${esc(fmtStamp(rep.at))}</span>
          ${rep.errors.length || rep.warnings.length ? `<ul class="sf-issues">${rep.errors.concat(rep.warnings).slice(0, 40).map(x => `<li class="${x.level === 'error' ? 'is-err' : 'is-warn'}">${esc(x.text)}${x.coll === 'vehicles' && getVehicle(x.id) ? ` — <button type="button" class="link-btn" data-act="open-vehicle" data-id="${esc(x.id)}">${esc(getVehicle(x.id).name)}</button>` : x.id ? ` <span class="f-td-muted qr-mono">${esc(x.id)}</span>` : ''}</li>`).join('')}</ul>` : ''}
          ${ui.safety.fixes ? `<p class="f-hint">Последнее исправление: ${ui.safety.fixes.length} действий.</p>` : ''}
        </div>` : ''}
      </div>

      <div class="f-panel">
        <div class="f-panel-title">Журнал изменений карточек</div>
        <p class="f-hint">Фиксируются: ${Object.values(HISTORY_FIELDS).join(', ')}; создание и удаление карточек. Хранится до ${HISTORY_MAX} последних записей.</p>
        <input type="search" class="f-input" id="sfHistQ" placeholder="Поиск по технике или полю" value="${esc(ui.safety.hq || '')}" style="max-width:360px;margin-bottom:10px">
        ${hist.length ? `<div class="f-table-wrap"><div class="f-table-scroll"><table class="f-table f-table-stack">
          <thead><tr><th style="width:130px">Когда</th><th>Техника</th><th>Изменение</th></tr></thead>
          <tbody>${hist.map(e => `<tr ${getVehicle(e.vehicleId) ? `class="is-clickable" data-act="open-vehicle" data-id="${esc(e.vehicleId)}"` : ''}>
            <td class="f-td-nowrap" data-label="Когда">${esc(fmtStamp(e.at))}</td><td class="is-head">${esc(e.name || '—')}</td><td>${esc(historyText(e))}</td></tr>`).join('')}</tbody>
        </table></div></div>` : '<div class="f-hint">Изменений пока нет.</div>'}
      </div>

      <div class="f-panel">
        <div class="f-panel-title">Платформа и модули</div>
        <p class="f-hint">Ядро платформы ${esc(P ? P.version : '—')} · структура данных версии ${SCHEMA_VERSION}${(DB.migrations || []).length ? ` · миграций выполнено: ${DB.migrations.length}` : ''}. Каждый модуль владеет своими данными и проверяется своими самотестами; самотесты работают на копиях данных и ничего не меняют.</p>
        <div class="sf-actions"><button type="button" class="f-btn f-btn-primary" data-act="tests-run">${FI.gauge(15)}<span>Запустить самотесты</span></button>
          ${tests ? `<span class="sf-tests-sum ${tests.some(t => !t.ok) ? 'is-bad' : 'is-ok'}">${tests.filter(t => t.ok).length} из ${tests.length} пройдено</span>` : ''}</div>
        <div class="f-table-wrap"><div class="f-table-scroll"><table class="f-table f-table-stack">
          <thead><tr><th>Модуль</th><th style="width:80px">Версия</th><th>Данные</th><th style="width:140px">Состояние</th><th>Самотесты</th></tr></thead>
          <tbody>${mods.map(md => { const tr = testFor(md.id); return `<tr>
            <td class="is-head"><div class="f-td-strong">${esc(md.name)}</div><div class="f-td-muted f-fit-sub">${esc(md.desc || '')}</div></td>
            <td class="qr-mono" data-label="Версия">${esc(md.version)}</td>
            <td class="f-td-muted" data-label="Данные">${esc((md.owns || []).join(', ') || '—')}</td>
            <td data-label="Состояние">${esc(md.state)}</td>
            <td data-label="Самотесты">${!md.tests.length ? '<span class="f-td-muted">нет</span>' : !tr ? `<span class="f-td-muted">${md.tests.length} ${plural(md.tests.length, 'тест', 'теста', 'тестов')}</span>` :
              `<ul class="sf-tests">${tr.map(t => `<li class="${t.ok ? 'is-ok' : 'is-err'}">${t.ok ? '✓' : '✗'} ${esc(t.name)}${t.info ? ` <span class="f-td-muted">— ${esc(t.info)}</span>` : ''}</li>`).join('')}</ul>`}</td>
          </tr>`; }).join('')}</tbody></table></div></div>
      </div>`;
    }
    function bindSafety() {
      const q = el('sfHistQ');
      if (q) q.addEventListener('input', () => {
        ui.safety.hq = q.value;
        clearTimeout(bindSafety.t);
        bindSafety.t = setTimeout(() => { const pos = q.selectionStart; renderView(); const n = el('sfHistQ'); if (n) { n.focus(); n.setSelectionRange(pos, pos); } }, 250);
      });
      fillStorageStatus();
      fillSafetyAsync();
    }
    async function fillSafetyAsync() {
      const box = el('sfSnaps'), cnt = el('sfSnapCount');
      if (!box) return;
      let list;
      try { list = await snapList(); }
      catch (e) { box.innerHTML = '<div class="sf-check is-bad">Хранилище точек восстановления недоступно в этом браузере (IndexedDB отключена). Регулярно создавайте резервные копии на внешний носитель.</div>'; if (cnt) cnt.textContent = '—'; return; }
      if (cnt) cnt.textContent = String(list.length);
      box.innerHTML = list.length ? `<div class="f-table-wrap"><div class="f-table-scroll"><table class="f-table f-table-stack">
        <thead><tr><th style="width:130px">Создана</th><th>Причина</th><th class="f-td-num">Техники</th><th class="f-td-num">Записей ТО</th><th class="f-td-num">Размер</th><th style="width:190px"></th></tr></thead>
        <tbody>${list.map(s => `<tr>
          <td class="f-td-nowrap" data-label="Создана">${esc(fmtStamp(s.at))}</td>
          <td class="is-head">${esc(s.label)}${s.auto ? '' : ' <span class="f-badge">вручную</span>'}</td>
          <td class="f-td-num" data-label="Техники">${s.counts && s.counts.vehicles != null ? s.counts.vehicles : '—'}</td>
          <td class="f-td-num" data-label="Записей ТО">${s.counts && s.counts.maint != null ? s.counts.maint : '—'}</td>
          <td class="f-td-num f-td-nowrap" data-label="Размер">${esc(fmtSize(s.size))}</td>
          <td class="is-acts"><div class="sf-row-acts">
            <button type="button" class="f-btn f-btn-sm" data-act="snap-restore" data-id="${s.id}">Вернуться</button>
            <button type="button" class="f-icon-btn" data-act="snap-download" data-id="${s.id}" title="Сохранить в файл" aria-label="Сохранить в файл">${FI.download(15)}</button>
            <button type="button" class="f-icon-btn" data-act="snap-delete" data-id="${s.id}" title="Удалить точку" aria-label="Удалить точку">${FI.trash(15)}</button></div></td>
```

```
      </tr>`).join('')}</tbody></table></div></div>` : '<div class="f-hint">Точек восстановления пока нет — первая создаётся автоматически при запуске программы с данными.</div>';
    }

    /* Журнал изменений в карточке техники */
    function renderVehicleHistory(v) {
      const list = (DB.history || []).filter(e => e.vehicleId === v.id).slice(-12).reverse();
      if (!list.length) return '';
      return `
        <div class="f-panel">
          <div class="f-panel-title">Журнал изменений карточки</div>
          <ul class="sf-hist">${list.map(e => `<li><span class="f-td-muted f-td-nowrap">${esc(fmtStamp(e.at))}</span> ${esc(historyText(e))}</li>`).join('')}</ul>
          <button type="button" class="link-btn" data-act="nav" data-key="safety">Весь журнал — «Сохранность данных»</button>
        </div>`;
    }

    /* ============================================================================
       27d. КОМАНДНЫЙ ПУНКТ — стартовый экран учёта
       ----------------------------------------------------------------------------
       Состояние всего парка на одном экране. Каждое число — кнопка: открывает
       перечень именно этих машин, из которого можно перейти в карточку,
       провести ТО или сформировать задание (печатная форма).

       Техническая готовность: «неисправна» и «ремонт» — неготовая техника;
       КТГ = (на учёте − неготовые) / на учёте, где «на учёте» — без списанной.
       Норматив КТГ подразделения — средний по группам эксплуатации его техники
       (Настройки → нормативы КТГ); если группа не указана — норматив не считается.
       ============================================================================ */
    const CP_NOT_READY = new Set(['Ремонт', 'Неисправна']);
    const CP_RANGES = { today: { label: 'Сегодня', days: 0 }, week: { label: 'Неделя', days: 7 }, month: { label: 'Месяц', days: 30 } };

    function commandStats() {
      const active = DB.vehicles.filter(v => v.status !== 'Списано');
      const st = new Map(active.map(v => [v.id, computeStatus(v)]));
      const lvl = l => active.filter(v => st.get(v.id).level === l);
      const unitsMap = new Map();
      active.forEach(v => {
        const name = (v.unit || '').trim() || 'Подразделение не указано';
        if (!unitsMap.has(name)) unitsMap.set(name, []);
        unitsMap.get(name).push(v);
      });
      const units = Array.from(unitsMap.keys()).sort((a, b) => a.localeCompare(b, 'ru')).map(name => {
        const list = unitsMap.get(name);
        const notReady = list.filter(v => CP_NOT_READY.has(v.status));
        const grouped = list.filter(v => v.expGroup && DB.settings.ktgByGroup[v.expGroup] != null);
        const norm = grouped.length ? grouped.reduce((s, v) => s + Number(DB.settings.ktgByGroup[v.expGroup]), 0) / grouped.length : null;
        const ktg = list.length ? (list.length - notReady.length) / list.length : null;
        return { name, list, ready: list.filter(v => v.status === 'В эксплуатации'), faulty: list.filter(v => v.status === 'Неисправна'),
                 repair: list.filter(v => v.status === 'Ремонт'), overdue: list.filter(v => st.get(v.id).level === 'overdue'),
                 ktg, norm, below: ktg != null && norm != null && ktg + 1e-9 < norm };
      });
      const notReadyAll = active.filter(v => CP_NOT_READY.has(v.status)).length;
      return {
        active, st, units,
        total: active.length,
        ready: active.filter(v => v.status === 'В эксплуатации'),
        faulty: active.filter(v => v.status === 'Неисправна'),
        repair: active.filter(v => v.status === 'Ремонт'),
        stored: active.filter(v => v.status === 'На хранении' || v.status === 'Консервация'),
        ktg: active.length ? (active.length - notReadyAll) / active.length : null,
        overdue: lvl('overdue'), urgent: lvl('urgent'), soon: lvl('soon'), none: lvl('none'),
        stale: active.filter(hoursStale)
      };
    }

    /* Работы в окне дат: плановые ТО по сроку. «Сегодня» — срок наступил
       (включая просроченные), «Неделя»/«Месяц» — до 7/30 дней вперёд */
    function commandCalendar(S, range) {
      const today = todayISO();
      const end = addDays(today, CP_RANGES[range].days);
      return S.active
        .map(v => ({ v, st: S.st.get(v.id) }))
        .filter(x => x.st.nextDate && x.st.nextDate <= end && x.st.level !== 'retired')
        .sort((a, b) => a.st.nextDate.localeCompare(b.st.nextDate) || a.v.name.localeCompare(b.v.name, 'ru'));
    }

    /* Перечень по ключу кнопки */
    function commandList(key) {
      const S = commandStats();
      const m = /^u(\d+)-(\w+)$/.exec(key);
      if (m) {
        const u = S.units[Number(m[1])];
        if (!u) return null;
        const kinds = { all: ['вся техника', u.list], ready: ['исправная', u.ready], faulty: ['неисправная', u.faulty],
```

```
    repair: ['в ремонте', u.repair], overdue: ['ТО просрочено', u.overdue] };
      const k = kinds[m[2]] || kinds.all;
      return { title: `${u.name}: ${k[0]}`, list: k[1], task: m[2] === 'overdue' };
    }
    if (/^cal-/.test(key)) {
      const r = key.slice(4);
      if (!CP_RANGES[r]) return null;
      return { title: `Работы по ТО — ${CP_RANGES[r].label.toLowerCase()}`, list: commandCalendar(S, r).map(x => x.v),
    task: true };
    }
    if (key === 'season') {
      const info = seasonInfo();
      return { title: info.ev ? `Сезонное обслуживание: ${seasonTitle(info.ev).toLowerCase()} — не проведено` :
    'Сезонное обслуживание', list: info.todo, task: true, season: true };
    }
    const map = {
      total: ['Вся техника на учёте', S.active], ready: ['Исправная техника', S.ready], faulty: ['Неисправная техника',
    S.faulty],
      repair: ['Техника в ремонте', S.repair], stored: ['На хранении и консервации', S.stored],
      overdue: ['ТО просрочено', S.overdue, true], urgent: ['Требуется ТО', S.urgent, true], soon: ['Скоро ТО', S.soon,
    true],
      none: ['Интервал ТО не задан', S.none], stale: [`Моточасы не обновлялись ${DB.settings.hoursStaleDays} и более
    дней`, S.stale]
    };
    const x = map[key];
    return x ? { title: x[0], list: x[1], task: !!x[2] } : null;
  }

  const cpPct = k => k == null ? '—' : fmtNum(k * 100, 0) + ' %';
  const cpBtn = (key, value, label, opts) => `
    <button type="button" class="f-stat is-clickable cp-tile ${opts && opts.cls || ''}" data-act="cp-list" data-
    list="${key}" ${value ? '' : 'data-empty="1"'}>
      <div class="f-stat-label">${opts && opts.dot ? `<span class="f-dot f-lvl-${opts.dot}"></span>` :
    ''}${esc(label)}</div>
      <div class="f-stat-value"${opts && opts.color ? ` style="color:var(--st-${opts.color}-text)"` : ''}>${value}</div>
      ${opts && opts.hint ? `<div class="f-hint">${esc(opts.hint)}</div>` : ''}
    </button>`;
  const cpNum = (key, n) => n ? `<button type="button" class="cp-num" data-act="cp-list" data-
    list="${key}">${n}</button>` : '<span class="cp-num is-zero">0</span>';

  function renderCommand() {
    const S = commandStats();
    const range = ui.cp.range;
    const cal = commandCalendar(S, range);
    const today = todayISO();
    const b = backupInfo();
    const P = window.TST && window.TST.platform;
    const rep = P ? P.checkIntegrity(DB) : null;
    const ktgNorm = S.units.some(u => u.norm != null);
    return `
      <div class="f-page-head">
        <div>
          <div class="f-page-title" role="heading" aria-level="1">Пункт управления</div>
          <div class="f-page-sub">Состояние парка технических средств тыла на ${fmtDate(today)} · каждое число открывает
     перечень техники</div>
        </div>
        <div class="f-page-actions">
          <button type="button" class="f-btn" data-act="cp-report">${FI.pdf(15)}<span>Доклад о состоянии
    парка</span></button>
        </div>
      </div>

      <div class="cp-section-title">Техническое состояние</div>
      <div class="f-stats cp-stats">
        ${cpBtn('total', S.total, 'На учёте', { hint: 'без списанной техники' })}
        ${cpBtn('ready', S.ready.length, 'Исправна', { color: 'ok', dot: 'ok', hint: 'в эксплуатации' })}
        ${cpBtn('faulty', S.faulty.length, 'Неисправна', { color: S.faulty.length ? 'overdue' : '', dot: 'overdue' })}
        ${cpBtn('repair', S.repair.length, 'В ремонте', { color: S.repair.length ? 'urgent' : '', dot: 'urgent' })}
        ${cpBtn('stored', S.stored.length, 'Хранение и консервация')}
        <div class="f-stat cp-ktg"><div class="f-stat-label">КТГ парка</div>
          <div class="f-stat-value">${cpPct(S.ktg)}</div><div class="f-hint">(на учёте − неисправна − в ремонте) / на
    учёте</div></div>
      </div>

      <div class="cp-section-title">Техническое обслуживание</div>
      <div class="f-stats cp-stats">
        ${cpBtn('overdue', S.overdue.length, 'ТО просрочено', { color: S.overdue.length ? 'overdue' : '', dot: 'overdue'
     })}
        ${cpBtn('urgent', S.urgent.length, 'Требуется ТО', { color: S.urgent.length ? 'urgent' : '', dot: 'urgent' })}
        ${cpBtn('soon', S.soon.length, 'Скоро ТО', { color: S.soon.length ? 'soon' : '', dot: 'soon' })}
        ${cpBtn('none', S.none.length, 'Интервал не задан', { dot: 'none' })}
        ${cpBtn('stale', S.stale.length, 'Моточасы не обновлялись', { hint: `${DB.settings.hoursStaleDays} и более дней`
     })}
        ${(() => {
          const si = seasonInfo();
          if (!si.ev) return '';
          const todo = si.ev.active ? si.todo.length : 0;
          const hint = si.ev.active
            ? `${SEASONS[si.ev.season].cap} режим к ${fmtDate(si.ev.date)} · ${seasonLeftText(si.ev)}`
            : `Ближайшее: ${SEASONS[si.ev.season].label} режим ${fmtDate(si.ev.date)} · ${seasonLeftText(si.ev)}`;
```

```
        return cpBtn('season', todo, 'Сезонное ТО не проведено', { color: si.ev.active && si.ev.late && todo ? 'overdue' : (todo ? 'urgent' : ''), hint });
      })()}
    </div>

    <div class="cp-grid">
      <div class="f-panel cp-units">
        <div class="f-panel-title">Готовность по подразделениям</div>
        ${S.units.length ? `<div class="f-table-wrap"><div class="f-table-scroll"><table class="f-table f-table-stack cp-table">
          <thead><tr><th>Подразделение</th><th class="f-td-num">Всего</th><th class="f-td-num">Исправна</th><th class="f-td-num">Неиспр.</th><th class="f-td-num">Ремонт</th><th class="f-td-num">ТО просроч.</th><th style="width:110px">КТГ${ktgNorm ? ' / норматив' : ''}</th></tr></thead>
          <tbody>${S.units.map((u, i) => `<tr>
            <td class="f-td-strong is-head">${esc(u.name)}</td>
            <td class="f-td-num" data-label="Всего">${cpNum('u' + i + '-all', u.list.length)}</td>
            <td class="f-td-num" data-label="Исправна">${cpNum('u' + i + '-ready', u.ready.length)}</td>
            <td class="f-td-num" data-label="Неисправна">${cpNum('u' + i + '-faulty', u.faulty.length)}</td>
            <td class="f-td-num" data-label="Ремонт">${cpNum('u' + i + '-repair', u.repair.length)}</td>
            <td class="f-td-num" data-label="ТО просрочено">${cpNum('u' + i + '-overdue', u.overdue.length)}</td>
            <td data-label="КТГ"><div class="cp-ktg-cell ${u.below ? 'is-below' : ''}"><span>${cpPct(u.ktg)}${u.norm != null ? ' / ' + cpPct(u.norm) : ''}</span>
              <div class="cp-bar"><i style="width:${Math.round((u.ktg || 0) * 100)}%"></i>${u.norm != null ? `<b style="left:${Math.round(u.norm * 100)}%" title="Норматив"></b>` : ''}</div></div></td>
          </tr>`).join('')}</tbody></table></div></div>
          ${S.units.some(u => u.below) ? '<p class="f-hint" style="color:var(--st-overdue-text)">Красным отмечены подразделения с КТГ ниже норматива.</p>' : ''}`
        : '<div class="f-hint">Техники на учёте нет.</div>'}
      </div>

      <div class="f-panel cp-cal">
        <div class="cp-cal-head">
          <div class="f-panel-title">Календарь работ</div>
          <div class="f-tabs cp-tabs" role="tablist">${Object.keys(CP_RANGES).map(k => `<button type="button" class="f-tab ${k === range ? 'is-active' : ''}" data-act="cp-range" data-range="${k}" role="tab" aria-selected="${k === range}">${CP_RANGES[k].label}</button>`).join('')}</div>
        </div>
        ${cal.length ? `<div class="cp-cal-list">${cal.slice(0, 40).map(({ v, st }) => {
          const late = st.nextDate < today;
          return `<div class="cp-cal-row is-clickable" data-act="open-vehicle" data-id="${v.id}">
            <div class="cp-cal-date ${late ? 'is-late' : st.nextDate === today ? 'is-today' : ''}"><b>${fmtDate(st.nextDate).slice(0, 5)}</b><span>${late ? 'просрочено' : st.nextDate === today ? 'сегодня' : new Date(toDate(st.nextDate)).toLocaleDateString('ru-RU', { weekday: 'short' })}</span></div>
            <div class="cp-cal-main"><div class="f-td-strong">${esc(nextMaintKind(v))} · ${esc(v.name)}</div>
              <div class="f-td-muted f-fit-sub">${esc([v.regNumber, v.unit].filter(Boolean).join(' · ') || '—')}</div></div>
            ${levelBadge(st.level)}
            <button type="button" class="f-icon-btn" data-act="add-maint" data-id="${v.id}" title="Провести ТО" aria-label="Провести ТО">${FI.wrench(15)}</button>
          </div>`; }).join('')}</div>
          <div class="sf-actions" style="margin-top:10px">
            <button type="button" class="f-btn" data-act="cp-list" data-list="cal-${range}">${FI.list(15)}<span>Перечень (${cal.length})</span></button>
            <button type="button" class="f-btn f-btn-primary" data-act="cp-task" data-list="cal-${range}">${FI.pdf(15)}<span>Сформировать задание</span></button>
          </div>`
        : `<div class="f-hint">${range === 'today' ? 'На сегодня плановых работ нет.' : 'В этом периоде плановых ТО по сроку нет.'} Сроки по моточасам — в разделе «Ближайшие ТО».</div>`}
      </div>
    </div>

    <div class="f-panel cp-safety ${b.due || (rep && rep.errors.length) ? 'is-due' : ''}">
      <div class="cp-safety-row">
        <div><div class="f-panel-title">Сохранность данных</div>
          <div class="f-hint">${esc(backupAgoText(b).replace(/^./, c => c.toUpperCase()))}${b.due ? ' — пора сохранить копию на внешний носитель' : ''} · в корзине: ${(DB.trash || []).length} · проверка целостности: ${rep ? (rep.errors.length ? rep.errors.length + ' ' + plural(rep.errors.length, 'ошибка', 'ошибки', 'ошибок') : 'ошибок нет') : '—'}</div></div>
        <div class="sf-actions">
          <button type="button" class="f-btn ${b.due ? 'f-btn-primary' : ''}" data-act="backup-export">${FI.download(15)}<span>Резервная копия</span></button>
          <button type="button" class="f-btn" data-act="nav" data-key="safety">Подробнее</button>
        </div>
      </div>
    </div>`;
  }

  /* Перечень техники по нажатию на число */
  function commandListModal(key) {
    const L = commandList(key);
    if (!L) return;
    const rows = L.list.slice().sort((a, b) => a.name.localeCompare(b.name, 'ru'));
    const m = openModal(`
      <div class="f-modal-head">
        <div><div class="f-modal-title">${esc(L.title)}</div><div class="f-modal-sub">Единиц техники: ${rows.length} · на ${fmtDate(todayISO())}</div></div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        ${rows.length ? `<div class="f-table-wrap"><div class="f-table-scroll"><table class="f-table f-table-stack">
```

```
        <thead><tr><th>Техника</th><th>Подразделение</th><th>Статус</th><th>Состояние ТО</th><th>Очередное ТО</th></tr></thead>
        <tbody>${rows.map(v => { const st = computeStatus(v); return `<tr class="is-clickable" data-cp-open="${v.id}">
          <td class="is-head"><div class="f-td-strong">${esc(v.name)}</div>${v.regNumber ? `<span class="f-plate f-fit-sub">${esc(v.regNumber)}</span>` : ''}</td>
          <td data-label="Подразделение">${esc(v.unit || '—')}</td><td data-label="Статус">${esc(v.status || '—')}</td>
          <td data-label="Состояние ТО">${levelBadge(st.level)}</td>
          <td class="f-td-nowrap" data-label="Очередное ТО">${st.level === 'none' || st.level === 'retired' ? '—' : esc(nextMaintKind(v)) + (st.nextDate ? ' до ' + fmtDate(st.nextDate) : '')}</td>
        </tr>`; }).join('')}</tbody></table></div></div>` : '<div class="f-hint">Таких единиц техники нет.</div>'}
    </div>
    <div class="f-modal-foot">
      <button type="button" class="f-btn" data-close>Закрыть</button>
      ${rows.length ? `<button type="button" class="f-btn f-btn-primary" data-cp-task>${FI.pdf(15)}<span>${L.task ? 'Сформировать задание на ТО' : 'Печать перечня'}</span></button>` : ''}
    </div>`);
  m.root.addEventListener('click', e => {
    const r = e.target.closest('[data-cp-open]');
    if (r) { m.close(); go('vehicle', r.dataset.cpOpen); return; }
    if (e.target.closest('[data-cp-task]')) { m.close(); commandTask(key); }
  });
}

/* Задание на проведение ТО / перечень техники — печатная форма */
function commandTask(key) {
  if (key === 'season') { seasonTask(); return; }
  const L = commandList(key);
  if (!L || !L.list.length) { toast('info', 'Нет техники', 'Перечень пуст — задание не сформировано.'); return; }
  const rows = L.list.slice().sort((a, b) => (computeStatus(a).nextDate || '9').localeCompare(computeStatus(b).nextDate || '9') || a.name.localeCompare(b.name, 'ru'));
  const body = L.task ? `<table>
    <thead><tr><th style="width:22pt">№</th><th>Техника, гос. номер</th><th>Подразделение</th><th>Вид работ</th><th>Срок</th><th>Ответственный</th><th style="width:70pt">Отметка о выполнении</th></tr></thead>
    <tbody>${rows.map((v, i) => { const st = computeStatus(v); return `<tr><td>${i + 1}</td><td>${esc(v.name)}${v.regNumber ? ', ' + esc(v.regNumber) : ''}</td><td>${esc(v.unit || '—')}</td>
      <td>${esc(st.level === 'none' ? 'Определить интервал ТО' : nextMaintKind(v))}</td><td>${st.nextDate ? fmtDate(st.nextDate) : '—'}${st.remainingHours != null ? ` / ${fmtNum(st.nextHours, 0)} мото·ч` : ''}</td>
      <td>${esc(v.responsible || '')}</td><td></td><td></tr>`; }).join('')}</tbody></table>`
    : `<table><thead><tr><th style="width:22pt">№</th><th>Техника</th><th>Гос. номер</th><th>Инв. номер</th><th>Подразделение</th><th>Статус</th><th>Состояние ТО</th></tr></thead>
    <tbody>${rows.map((v, i) => `<tr><td>${i + 1}</td><td>${esc(v.name)}</td><td>${esc(v.regNumber || '—')}</td><td>${esc(v.invNumber || '—')}</td><td>${esc(v.unit || '—')}</td><td>${esc(v.status || '—')}</td><td>${esc(LEVELS[computeStatus(v).level].label)}</td></tr>`).join('')}</tbody></table>`;
  printCustom(L.task ? 'ЗАДАНИЕ' : 'ПЕРЕЧЕНЬ', `${L.task ? 'на проведение технического обслуживания: ' : ''}${/^[A-ЯЁA-Z][a-яёa-z]/.test(L.title) ? L.title.charAt(0).toLowerCase() + L.title.slice(1) : L.title} (на ${fmtDate(todayISO())})`, body,
    L.task ? '<span>Задание выдал: _____</span><span>Задание получил: _____</span><span>«____» _____ 20____ г.</span>'
           : '<span>Составил: _____</span><span>«____» _____ 20____ г.</span>');
}

/* Доклад о состоянии парка */
function commandReport() {
  const S = commandStats();
  if (!S.total) { toast('info', 'Нет данных', 'Техники на учёте нет.'); return; }
  const body = `
    <table><tbody>
      <tr><td>На учёте (без списанной)</td><td class="p-num">${S.total}</td></tr>
      <tr><td>Исправна (в эксплуатации)</td><td class="p-num">${S.ready.length}</td></tr>
      <tr><td>Неисправна</td><td class="p-num">${S.faulty.length}</td></tr>
      <tr><td>В ремонте</td><td class="p-num">${S.repair.length}</td></tr>
      <tr><td>На хранении и консервации</td><td class="p-num">${S.stored.length}</td></tr>
      <tr><td><b>Коэффициент технической готовности</b></td><td class="p-num"><b>${cpPct(S.ktg)}</b></td></tr>
      <tr><td>ТО просрочено / требуется / скоро</td><td class="p-num">${S.overdue.length} / ${S.urgent.length} / ${S.soon.length}</td></tr>
    </tbody></table>
    <h3 style="margin:12pt 0 6pt;font-size:11pt">Готовность по подразделениям</h3>
    <table><thead><tr><th>Подразделение</th><th class="p-num">Всего</th><th class="p-num">Исправна</th><th class="p-num">Неисправна</th><th class="p-num">Ремонт</th><th class="p-num">КТГ</th><th class="p-num">Норматив</th></tr></thead>
    <tbody>${S.units.map(u => `<tr><td>${esc(u.name)}</td><td class="p-num">${u.list.length}</td><td class="p-num">${u.ready.length}</td><td class="p-num">${u.faulty.length}</td><td class="p-num">${u.repair.length}</td><td class="p-num">${cpPct(u.ktg)}</td><td class="p-num">${u.norm != null ? cpPct(u.norm) : '—'}</td></tr>`).join('')}</tbody></table>
    ${S.overdue.length ? `<h3 style="margin:12pt 0 6pt;font-size:11pt">Техника с просроченным ТО</h3>
    <table><thead><tr><th>Техника</th><th>Подразделение</th><th>Вид ТО</th><th>Срок</th></tr></thead>
    <tbody>${S.overdue.map(v => { const st = S.st.get(v.id); return `<tr><td>${esc(v.name)}${v.regNumber ? ', ' + esc(v.regNumber) : ''}</td><td>${esc(v.unit || '—')}</td><td>${esc(nextMaintKind(v))}</td><td>${st.nextDate ? fmtDate(st.nextDate) : '—'}</td></tr>`; }).join('')}</tbody></table>` : ''}`;
  printCustom('ДОКЛАД', `о состоянии парка технических средств тыла на ${fmtDate(todayISO())}`, body,
    '<span>Докладывает: _____</span><span>«____» _____ 20____ г.</span>');
}

/* ============================================================================
   27е. МОДУЛИ УЧЁТА В РЕЕСТРЕ ПЛАТФОРМЫ
   ----------------------------------------------------------------------------
   Логические границы модулей внутри файла: кто владеет какими данными,
   от кого зависит, какой версии и чем проверяется. Самотесты работают
   на временной базе (withTestDB): реальная база подменяется на время
```

```
       теста, сохранение в хранилище в это время заблокировано
       ========================================================================== */
    let testDepth = 0;
    function withTestDB(tmp, fn) {
      const saved = DB, savedBase = baseJSON;
      DB = normalizeDB(tmp);
      testDepth++;
      try { return fn(); } finally { testDepth--; DB = saved; baseJSON = savedBase; }
    }
    function testFixture() {
      const t = todayISO();
      const iv = { byHours: true, byDate: true, hours: 250, days: 180 };
      return {
        version: SCHEMA_VERSION,
        vehicles: [
          { id: 'v_test_aaaa01', name: 'Тест-кухня 1', status: 'В эксплуатации', unit: '1 рота', expGroup: 'Строевая', regDate: addDays(t, -400), currentHours: 1000, interval: iv },
          { id: 'v_test_aaaa02', name: 'Тест-кухня 2', status: 'Ремонт', unit: '1 рота', expGroup: 'Строевая', regDate: addDays(t, -30), currentHours: 0, interval: iv },
          { id: 'v_test_aaaa03', name: 'Тест-кухня 3', status: 'Неисправна', unit: '2 рота', regDate: addDays(t, -10), currentHours: 0, interval: iv },
          { id: 'v_test_aaaa04', name: 'Тест-кухня 4', status: 'В эксплуатации', unit: '2 рота', regDate: addDays(t, -5), currentHours: 0, interval: iv }
        ],
        maint: [{ id: 'm_test_aaaa01', vehicleId: 'v_test_aaaa01', kind: 'ТО-1', date: addDays(t, -200), hours: 700 }],
        hours: [
          { id: 'h_test_aaaa01', vehicleId: 'v_test_aaaa01', date: addDays(t, -200), value: 700, maintId: 'm_test_aaaa01' },
          { id: 'h_test_aaaa02', vehicleId: 'v_test_aaaa01', date: addDays(t, -2), value: 1000 }
        ],
        trips: [], norms: [], catalogEdits: [], trash: [], history: [], settings: DB.settings
      };
    }

    function registerFleetModules() {
      const P = window.TST && window.TST.platform;
      if (!P) return;
      const A = P.assert;
      defineFleetModel();

      P.register({ id: 'catalog', name: 'Справочник ТС', version: '1.4.0', owns: ['catalogEdits', 'norms'], depends: ['platform'],
        desc: 'Образцы техники, регламенты ТО, нормативная база',
        tests: [{ name: 'Справочник загружен и отвечает по идентификатору', run() {
          const c = CAT();
          A(c && typeof c.get === 'function', 'API справочника недоступен');
          const any = c.match('КП-130') || c.match('АЦ');
          return any ? 'найдена позиция ' + any : 'справочник доступен';
        } }] });

      P.register({ id: 'park', name: 'Парк техники', version: '1.5.0', owns: ['vehicles'], depends: ['platform', 'catalog'],
        desc: 'Карточки техники, постоянные идентификаторы, массовые операции',
        tests: [
          { name: 'Новый идентификатор техники уникален', run() {
            return withTestDB(testFixture(), () => {
              const seen = new Set(DB.vehicles.map(v => v.id));
              for (let i = 0; i < 300; i++) { const id = uniqueVehicleId(); A(!seen.has(id) && P.ids.valid(id), 'повтор или неверный id ' + id); seen.add(id); DB.vehicles.push({ id, name: 'x' }); }
              return '300 идентификаторов без повторов';
            });
          } },
          { name: 'Повреждённые данные приводятся к корректной структуре', run() {
            const d = normalizeDB({ vehicles: 'мусор', maint: null, settings: 5 });
            A(Array.isArray(d.vehicles) && Array.isArray(d.maint) && Array.isArray(d.trash) && d.settings.defaultHours > 0, 'структура не восстановлена');
            return true;
          } }
        ] });

      P.register({ id: 'maint', name: 'Техническое обслуживание', version: '1.7.0', owns: ['maint'], depends: ['park'],
        desc: 'Журнал ТО, расчёт сроков и состояния по моточасам и календарю',
        tests: [{ name: 'Шаблон работ: операции регламента и отметки сохраняются по тексту', run() {
          const a = buildChecklist(['Осмотр', 'Замена масла', 'Проверка тормозов'], null);
          A(a.length === 3 && a.every(x => !x.done), 'новый шаблон должен быть без отметок');
          a[1].done = true;
          const b = buildChecklist(['Осмотр', 'Замена масла', 'Проверка тормозов', 'Новая операция'], a);
          A(b[1].done && !b[0].done && !b[3].done && b.length === 4, 'отметки не перенеслись по тексту операции');
          A(checklistSummary({ items: b }).done === 1 && checklistSummary({ items: b }).total === 4, 'неверная сводка');
          return 'отметки переносятся при изменении регламента';
        } }, { name: 'Срок ТО рассчитывается по моточасам и календарю', run() {
          return withTestDB(testFixture(), () => {
            const s1 = computeStatus(getVehicle('v_test_aaaa01'));
            A(s1.level === 'overdue', 'ожидалось «ТО просрочено», получено ' + s1.level);
            const s4 = computeStatus(getVehicle('v_test_aaaa04'));
            A(s4.level === 'ok', 'ожидалось «Норма», получено ' + s4.level);
            return 'просрочено и норма определены верно';
          });
        } }] });
```

```
  P.register({ id: 'repairs', name: 'Ремонты и готовность', version: '1.0.0', owns: [], depends: ['park', 'maint'],
    desc: 'Статусы «Неисправна» и «Ремонт», записи о ремонтах в журнале ТО, КТГ',
    tests: [{ name: 'КТГ учитывает неисправную технику и ремонт', run() {
      return withTestDB(testFixture(), () => {
        const S = commandStats();
        A(S.total === 4 && S.faulty.length === 1 && S.repair.length === 1, 'неверный подсчёт состояний');
        A(Math.abs(S.ktg - 0.5) < 1e-9, 'КТГ парка ' + S.ktg + ', ожидалось 0,5');
        return 'КТГ = 50 % при 2 неготовых из 4';
      });
    } }] });

  P.register({ id: 'trips', name: 'Путевые листы и моточасы', version: '1.3.0', owns: ['trips', 'hours'], depends: ['park'],
    desc: 'Выходы техники, показания счётчика, наработка',
    tests: [{ name: 'Приросты и текущие моточасы пересчитываются', run() {
      return withTestDB(testFixture(), () => {
        recomputeHours('v_test_aaaa01');
        const v = getVehicle('v_test_aaaa01');
        A(v.currentHours === 1000, 'текущие моточасы ' + v.currentHours);
        A(DB.hours.find(h => h.id === 'h_test_aaaa02').delta === 300, 'неверный прирост');
        return true;
      });
    } }] });

  P.register({ id: 'warehouse', name: 'Склад (ЗИП, ГСМ)', version: '0.0.0', state: 'запланирован', owns: [], depends: ['park'],
    desc: 'Место для будущего модуля: собственная коллекция данных и самотесты добавятся при его разработке' });

  P.register({ id: 'documents', name: 'Документы и печать', version: '1.4.0', owns: [], depends: ['park', 'maint', 'trips'],
    desc: 'Печатные формы, отчёты, окно предпросмотра',
    tests: [{ name: 'Отчёт «Перечень техники» формируется', run() {
      return withTestDB(testFixture(), () => {
        const spec = captureDocQuiet('report-vehicles');
        A(spec && spec.title, 'документ не сформирован');
        return 'документ: ' + spec.title;
      });
    } }] });

  P.register({ id: 'analytics', name: 'Пункт управления', version: '1.1.0', owns: [], depends: ['park', 'maint', 'repairs'],
    desc: 'Стартовый экран: состояние парка, КТГ по подразделениям, календарь работ',
    tests: [{ name: 'Перечни по нажатию на числа совпадают со счётчиками', run() {
      return withTestDB(testFixture(), () => {
        const S = commandStats();
        ['overdue', 'faulty', 'repair', 'total'].forEach(k => { const L = commandList(k); A(L && L.list.length === (k === 'total' ? S.total : S[k].length), 'перечень ' + k + ' не совпадает'); });
        const u = S.units.findIndex(x => x.name === '1 рота');
        A(commandList('u' + u + '-repair').list.length === 1, 'перечень подразделения не совпадает');
        return true;
      });
    } }] });

  P.register({ id: 'seasons', name: 'Сезонное обслуживание', version: '1.0.0', owns: [], depends: ['park', 'maint'],
    desc: 'Перевод техники на летний и зимний режим: даты, напоминания, задание',
    tests: [
      { name: 'Окно напоминания и отметка выполнения по сезону', run() {
        return withTestDB(Object.assign(testFixture(), { settings: Object.assign({}, DB.settings, { seasons: { summer: addDays(todayISO(), 180).slice(5), winter: addDays(todayISO(), 10).slice(5), leadDays: 30 } }) }), () => {
          let info = seasonInfo();
          A(info.ev && info.ev.active && info.ev.season === 'winter' && info.ev.daysLeft === 10, 'ожидалось активное зимнее событие через 10 дней');
          A(info.fleet.length === 4 && info.todo.length === 4, 'СО не проведено ни на одной машине');
          DB.maint.push({ id: 'm_test_season1', vehicleId: 'v_test_aaaa01', kind: SEASON_KIND, season: 'winter', date: todayISO(), hours: 1000 });
          info = seasonInfo();
          A(info.done.length === 1 && info.todo.length === 3, 'отметка о СО не засчитана');
          DB.maint.push({ id: 'm_test_season2', vehicleId: 'v_test_aaaa04', kind: SEASON_KIND, season: 'summer', date: todayISO(), hours: 0 });
          A(seasonInfo().todo.length === 3, 'СО другого сезона засчитано ошибочно');
          const n = seasonNotification();
          A(n && n.act === 'season' && /зимний/.test(n.title), 'нет уведомления о сезоне');
          return 'зимний режим: 1 из 4 проведено';
        });
      } },
      { name: 'Вне окна напоминаний уведомления нет; дата 30 февраля не ломает расчёт', run() {
        return withTestDB(Object.assign(testFixture(), { settings: Object.assign({}, DB.settings, { seasons: { summer: addDays(todayISO(), 100).slice(5), winter: addDays(todayISO(), 200).slice(5), leadDays: 30 } }) }), () => {
          const info = seasonInfo();
          A(info.ev && !info.ev.active && !seasonNotification(), 'уведомление вне окна');
          A(/^\d{4}-02-2[89]$/.test(seasonDate('02-30', 2026)), 'дата 30 февраля не приведена к концу месяца');
          return true;
        });
      } }
    ] });

  P.register({ id: 'qr', name: 'QR-маркировка', version: '1.2.0', owns: [], depends: ['park'],
    desc: 'Постоянные QR-коды, наклейки, сканирование без интернета',
    tests: [{ name: 'Код читается обратно, чужие коды отклоняются', run() {
      return withTestDB(testFixture(), () => {
```

```
        const v = getVehicle('v_test_aaaa01');
        ['vcard', 'text'].forEach(enc => { const r = qrParse(qrPayload(v, 'basic', todayISO(), enc)); A(r.ok && r.id === v.id, 'не разобран код вида ' + enc); });
        A(!qrParse('TST:1:<img src=x>').ok && !qrParse('https://example.com').ok, 'принят чужой код');
        return true;
      });
    } }] });

  P.register({ id: 'vfiles', name: 'Фото и документы техники', version: '1.0.0', owns: [], depends: ['park'],
    desc: 'Фотографии единицы техники, паспорт, формуляр, акты приёма-передачи (поля photos, vdocs)',
    tests: [{ name: 'Файлы техники: учёт ссылок и удаление вместе с машиной', run() {
      return withTestDB(testFixture(), () => {
        const v = getVehicle('v_test_aaaa01');
        v.photos = [{ id: 'file_p1', thumbId: 'file_t1', name: 'a.jpg' }, { id: 'file_p2', name: 'b.jpg' }];
        v.vdocs = [{ id: 'doc_1', kind: 'Паспорт', files: [{ id: 'file_d1', name: 'p.pdf' }, { id: 'file_d2', name: 'p2.png' }] }, { id: 'doc_2', kind: 'Формуляр', files: [] }];
        const ids = vfFileIds(v);
        A(ids.length === 5 && ids.indexOf('file_t1') >= 0 && ids.indexOf('file_d2') >= 0, 'не собраны идентификаторы файлов');
        A(vfThumbId(v) === 'file_t1', 'миниатюра главного фото не определена');
        A(vfFileIds({ id: 'v_x' }).length === 0, 'у машины без файлов есть ссылки');
        const drop = [];
        removeVehicle(v, drop);
        A(drop.length === 5 && drop.indexOf('file_p2') >= 0, 'файлы не переданы на удаление');
        return '5 файлов, 2 документа';
      });
    } }] });

  P.register({ id: 'safety', name: 'Сохранность данных', version: '1.0.0', owns: ['trash', 'history'], depends: ['platform'],
    desc: 'Точки восстановления, корзина, журнал изменений, проверка перед импортом, миграции',
    tests: [
      { name: 'Миграция структуры данных 1 → ' + SCHEMA_VERSION, run() {
        const keepNewer = newerData, keepLast = lastMigration;
        try {
          const old = { version: 1, vehicles: [{ id: 'v_old_000001', name: 'Старая' }] };
          const d = migrateDB(normalizeDB(old), 1);
          A(d.version === SCHEMA_VERSION && Array.isArray(d.trash) && Array.isArray(d.history) && d.vehicles.length === 1, 'миграция не выполнена');
          const nd = migrateDB(normalizeDB({ version: 99, vehicles: [], zzz: 1 }), 99);
          A(nd.version === 99 && nd.zzz === 1, 'данные новой версии изменены');
          return true;
        } finally { newerData = keepNewer; lastMigration = keepLast; }
      } },
      { name: 'Журнал фиксирует изменение важного поля', run() {
        const a = [{ id: 'v_x_000001', name: 'A', invNumber: '1' }], b = [{ id: 'v_x_000001', name: 'A', invNumber: '2' }, { id: 'v_x_000002', name: 'Б' }];
        const d = historyDiff(a, b, new Date().toISOString());
        A(d.some(e => e.action === 'edit' && e.field === 'invNumber' && e.from === '1' && e.to === '2'), 'правка не найдена');
        A(d.some(e => e.action === 'create' && e.vehicleId === 'v_x_000002'), 'создание не найдено');
        return true;
      } },
      { name: 'Удалённая техника возвращается из корзины со связанными записями', run() {
        return withTestDB(testFixture(), () => {
          const v = getVehicle('v_test_aaaa01');
          const removed = { vehicles: [v], maint: DB.maint.filter(m => m.vehicleId === v.id), hours: DB.hours.filter(h => h.vehicleId === v.id), trips: [] };
          removeVehicle(v, []);
          const id = trashPush('тест', '', removed);
          A(!getVehicle(v.id) && DB.trash.length === 1, 'запись не попала в корзину');
          const r = trashApply(DB, DB.trash[0]);
          A(getVehicle(v.id) && DB.maint.length === 1 && DB.hours.length === 2 && !DB.trash.length, 'восстановлено не всё');
          return 'возвращено записей: ' + r.restored + (id ? '' : '');
        });
      } },
      { name: 'Проверка перед импортом находит записи без техники', run() {
        const f = testFixture();
        f.trips.push({ id: 't_test_orphan1', vehicleId: 'v_net_takoi1', date: todayISO() });
        const rep = P.checkIntegrity(normalizeDB(f));
        A(rep.errors.some(x => x.coll === 'trips'), 'сирота не найдена');
        A(P.checkIntegrity(P.repair(normalizeDB(f)).db).errors.length === 0, 'исправление не помогло');
        return true;
      } }
    ] });
}

/* ========================================================================
   27f. ШАБЛОНЫ РАБОТ ТО И СЕЗОННОЕ ОБСЛУЖИВАНИЕ
   ------------------------------------------------------------------------
   Шаблон работ — перечень операций регламента из справочника для выбранного
   вида ТО. В форме записи ТО операции показываются списком с галочками;
   снимок (операция + отметка) сохраняется в записи, поэтому история не
   меняется, даже если регламент в справочнике потом отредактируют.

   Сезонное обслуживание — перевод техники на летний и зимний режим к
   заданным датам (Настройки → «Сезонное обслуживание»). За leadDays дней
   до даты программа напоминает, кому ещё не проведено СО с отметкой
   сезона. Напоминание остаётся SEASON_GRACE дней после даты (с пометкой
```

```
      «просрочено») и затем снимается. Даты задаются пользователем (приказом
      или решением командира): программа их не выдумывает, значения по
      умолчанию — только исходная подстановка.
      ========================================================================== */
  const SEASON_KIND = 'Сезонное обслуживание (СО)';
  const SEASONS = {
    summer: { label: 'летний', title: 'весенне-летний период', cap: 'Летний' },
    winter: { label: 'зимний', title: 'осенне-зимний период',  cap: 'Зимний' }
  };
  const SEASON_GRACE = 60;        // сколько дней после даты напоминание ещё показывается
  const SEASON_MATCH = 120;       // СО в пределах ±N дней от даты засчитывается за этот сезон
  const SEASON_DEFAULTS = { summer: '04-15', winter: '10-15', leadDays: 30 };

  /* --- Шаблон работ: снимок перечня операций с отметками --- */
  function buildChecklist(actions, prevItems) {
    const prev = new Map((prevItems || []).map(x => [x.t, !!x.done]));
    return (actions || []).map(t => ({ t: String(t), done: !!prev.get(String(t)) }));
  }
  function checklistSummary(cl) {
    const items = (cl && cl.items) || [];
    return { total: items.length, done: items.filter(x => x.done).length };
  }

  /* --- Расчёт сезонных событий --- */
  function seasonDate(mmdd, year) {
    const m = /^(\d{2})-(\d{2})$/.exec(String(mmdd || ''));
    if (!m) return null;
    const mo = Number(m[1]), d = Number(m[2]);
    if (mo < 1 || mo > 12 || d < 1 || d > 31) return null;
    const dt = new Date(year, mo - 1, d);
    if (dt.getMonth() !== mo - 1) return isoOf(new Date(year, mo, 0));   // 30 февраля → 28/29
    return isoOf(dt);
  }
  function seasonSettings() {
    const s = DB.settings.seasons || {};
    return {
      summer: s.summer || SEASON_DEFAULTS.summer,
      winter: s.winter || SEASON_DEFAULTS.winter,
      leadDays: Math.max(0, Number(s.leadDays) >= 0 && s.leadDays !== '' ? Number(s.leadDays) :
SEASON_DEFAULTS.leadDays)
    };
  }
  /* Событие, которое сейчас в работе (окно напоминания), иначе ближайшее будущее */
  function seasonEvent(today) {
    today = today || todayISO();
    const cfg = seasonSettings();
    const y = Number(today.slice(0, 4));
    const all = [];
    Object.keys(SEASONS).forEach(season => {
      [y - 1, y, y + 1].forEach(yr => {
        const date = seasonDate(cfg[season], yr);
        if (date) all.push({ season, date });
      });
    });
    all.sort((a, b) => a.date.localeCompare(b.date));
    const mk = e => {
      const daysLeft = daysBetween(today, e.date);
      return Object.assign({ daysLeft, late: daysLeft < 0, leadDays: cfg.leadDays }, e);
    };
    const active = all.find(e => {
      const d = daysBetween(today, e.date);
      return d <= cfg.leadDays && d >= -SEASON_GRACE;
    });
    if (active) return Object.assign(mk(active), { active: true });
    const next = all.find(e => e.date > today);
    return next ? Object.assign(mk(next), { active: false }) : null;
  }
  /* Техника, к которой применимо СО: не списана и не на хранении/консервации */
  function seasonFleet() {
    return DB.vehicles.filter(v => v.status !== 'Списано' && v.status !== 'На хранении' && v.status !== 'Консервация');
  }
  function seasonDone(v, ev) {
    const from = addDays(ev.date, -SEASON_MATCH), to = addDays(ev.date, SEASON_MATCH);
    return DB.maint.some(m => m.vehicleId === v.id && m.kind === SEASON_KIND && m.season === ev.season && m.date >= from
 && m.date <= to);
  }
  function seasonInfo(today) {
    const ev = seasonEvent(today);
    const fleet = seasonFleet();
    if (!ev) return { ev: null, fleet, done: [], todo: [] };
    const done = fleet.filter(v => seasonDone(v, ev));
    const todo = fleet.filter(v => !seasonDone(v, ev));
    return { ev, fleet, done, todo };
  }
  function seasonLeftText(ev) {
    if (ev.daysLeft === 0) return 'сегодня';
    if (ev.daysLeft > 0) return `через ${ev.daysLeft} ${plural(ev.daysLeft, 'день', 'дня', 'дней')}`;
    const d = Math.abs(ev.daysLeft);
    return `просрочено на ${d} ${plural(d, 'день', 'дня', 'дней')}`;
  }
```

```
  function seasonTitle(ev) {
    return `Перевод на ${SEASONS[ev.season].label} режим к ${fmtDate(ev.date)}`;
  }

  /* --- Уведомление и сводка --- */
  function seasonNotification() {
    const info = seasonInfo();
    if (!info.ev || !info.ev.active || !info.todo.length) return null;
    const ev = info.ev;
    return {
      level: ev.late ? 'overdue' : ev.daysLeft <= 7 ? 'urgent' : 'soon',
      vehicleId: '', act: 'season',
      title: seasonTitle(ev),
      text: `Не проведено СО: ${info.todo.length} из ${info.fleet.length} · ${seasonLeftText(ev)}`
    };
  }

  /* --- Поля настроек --- */
  const MONTHS_RU = ['января', 'февраля', 'марта', 'апреля', 'мая', 'июня', 'июля', 'августа', 'сентября', 'октября', 'ноября', 'декабря'];
  function seasonField(prefix, mmdd) {
    const [mo, d] = String(mmdd).split('-').map(Number);
    return `<div style="display:flex;gap:8px">
      <input id="${prefix}Day" class="f-input" type="number" min="1" max="31" value="${d}" style="width:84px" aria-label="День">
      <select id="${prefix}Month" class="f-select" aria-label="Месяц">${MONTHS_RU.map((n, i) => `<option value="${i + 1}" ${i + 1 === mo ? 'selected' : ''}>${n}</option>`).join('')}</select>
    </div>`;
  }
  function renderSeasonSettings() {
    const c = seasonSettings();
    return `
      <div class="f-panel">
        <div class="f-panel-title">Сезонное обслуживание</div>
        <div class="f-hint" style="margin-bottom:10px">Даты перевода техники на ${SEASONS.summer.label} и ${SEASONS.winter.label} режим. Задаются приказом или решением командира — значения ниже подставлены по умолчанию, проверьте их. За указанное число дней до даты программа напоминает (колокол, «Пункт управления»), у кого СО ещё не проведено.</div>
        <div class="f-form-grid">
          <div><label class="f-fld-label" for="setSumDay">Перевод на летний режим — до</label>${seasonField('setSum', c.summer)}</div>
          <div><label class="f-fld-label" for="setWinDay">Перевод на зимний режим — до</label>${seasonField('setWin', c.winter)}</div>
          <div><label class="f-fld-label" for="setSeasonLead">Напоминать за, дней</label>
            <input id="setSeasonLead" class="f-input" type="number" min="0" max="365" value="${c.leadDays}"></div>
        </div>
        <div class="f-hint" style="margin-top:10px">СО засчитывается, если в журнале ТО есть запись вида «Сезонное обслуживание (СО)» с отметкой сезона в пределах ${SEASON_MATCH} дней от даты. Не требуется для техники на хранении, консервации и списанной.</div>
      </div>`;
  }
  function bindSeasonSettings() {
    const save = () => {
      const cfg = Object.assign({}, DB.settings.seasons || {});
      const read = p => {
        const d = parseInt(el(p + 'Day').value, 10), m = parseInt(el(p + 'Month').value, 10);
        const maxD = new Date(2024, m, 0).getDate();          // 2024 — високосный: 29 февраля допустимо
        if (isNaN(d) || d < 1 || d > maxD) return null;
        return String(m).padStart(2, '0') + '-' + String(d).padStart(2, '0');
      };
      const sum = read('setSum'), win = read('setWin');
      if (!sum || !win) { toast('warning', 'Некорректная дата', 'Проверьте день и месяц.'); return false; }
      cfg.summer = sum; cfg.winter = win;
      const lead = parseInt(el('setSeasonLead').value, 10);
      cfg.leadDays = isNaN(lead) || lead < 0 || lead > 365 ? SEASON_DEFAULTS.leadDays : lead;
      DB.settings.seasons = cfg;
      saveDB(); renderNav(); renderNotifPanel();
      toast('success', 'Настройки сохранены', '');
      return true;
    };
    ['setSumDay', 'setSumMonth', 'setWinDay', 'setWinMonth', 'setSeasonLead'].forEach(id => {
      const n = el(id);
      if (n) n.addEventListener('change', () => { if (!save()) renderView(); });
    });
  }

  /* --- Задание на перевод на сезонный режим (печатная форма) --- */
  function seasonTask() {
    const info = seasonInfo();
    if (!info.ev || !info.todo.length) { toast('info', 'Нет техники', 'Сезонное обслуживание проведено на всей технике.'); return; }
    const ev = info.ev;
    const rows = info.todo.slice().sort((a, b) => (a.unit || '').localeCompare(b.unit || '', 'ru') || a.name.localeCompare(b.name, 'ru'));
    const body = `<table>
      <thead><tr><th style="width:22pt">№</th><th>Техника, гос. номер</th><th>Подразделение</th><th>Вид работ</th><th>Срок</th><th>Ответственный</th><th style="width:70pt">Отметка о выполнении</th></tr></thead>
      <tbody>${rows.map((v, i) => `<tr><td>${i + 1}</td><td>${esc(v.name)}${v.regNumber ? ', ' + esc(v.regNumber) : ''}</td><td>${esc(v.unit || '—')}</td>
        <td>Сезонное обслуживание — перевод на ${esc(SEASONS[ev.season].label)}
```

```
    режим</td><td>${fmtDate(ev.date)}</td><td>${esc(v.responsible || '')}</td><td></td></tr>`).join('')}</tbody></table>`;
      printCustom('ЗАДАНИЕ', `на проведение сезонного обслуживания: перевод техники на ${SEASONS[ev.season].label} режим эксплуатации (${SEASONS[ev.season].title}) к ${fmtDate(ev.date)}`, body,
        '<span>Задание выдал: _____</span><span>Задание получил: _____</span><span>«____» _____ 20____ г.</span>');
    }

    /* ========================================================================
       27g. ПРИЛОЖЕНИЕ ДЛЯ WINDOWS (Electron): ХРАНЕНИЕ В ФАЙЛАХ
       ------------------------------------------------------------------------
       Когда программа запущена внутри приложения, доступен мост window.TSTNative
       (app/preload.js). Тогда основное хранилище — файлы в папке данных:
       db.json (записи), files/ (вложения), snapshots/ (точки восстановления),
       backups/ (суточные копии). localStorage браузера для записей не
       используется, поэтому очистка профиля и лимит localStorage не страшны.
       В обычном браузере моста нет, и всё работает как раньше.
       ======================================================================== */
    const NATIVE = () => (window.TSTNative && window.TSTNative.isNative) ? window.TSTNative : null;
    let nativeSaveWarned = false;
    const nativeWarn = (what, e) => console.warn('[ТСТ] приложение: ' + what, e);

    function nativeSaveDb(raw) {
      const N = NATIVE();
      if (!N) return;
      N.saveDb(raw).then(() => { nativeSaveWarned = false; }).catch(e => {
        nativeWarn('не удалось записать файл данных', e);
        if (!nativeSaveWarned) {
          nativeSaveWarned = true;
          toast('danger', 'Данные не записаны в файл',
            `Не удалось записать файл в папку данных (${N.dataDir}). Проверьте место на диске и права доступа, затем сохраните резервную копию.`, 12000);
        }
      });
    }
    function nativeFilePut(rec) {
      const N = NATIVE();
      if (!N || !rec || !rec.blob) return;
      rec.blob.arrayBuffer()
        .then(buf => N.putFile(rec.id, { name: rec.name || '', type: rec.type || '' }, new Uint8Array(buf)))
        .catch(e => nativeWarn('вложение не записано', e));
    }
    function nativeFileGet(id) {
      const N = NATIVE();
      if (!N) return null;
      return N.getFile(id).then(r => r ? { id, name: r.meta.name || '', type: r.meta.type || '', size: r.meta.size || r.bytes.length,
        blob: new Blob([r.bytes], { type: r.meta.type || '' }) } : null).catch(() => null);
    }
    function nativeFileDel(id) { const N = NATIVE(); if (N) N.delFile(id).catch(e => nativeWarn('вложение не удалено', e)); }
    function nativeFileList() { const N = NATIVE(); return N ? N.listFiles().catch(() => []) : Promise.resolve([]); }
    function nativeSnapPut(rec) { const N = NATIVE(); if (N && rec) N.putSnapshot(rec.id, rec).catch(e => nativeWarn('точка не записана', e)); }
    function nativeSnapDel(id) { const N = NATIVE(); if (N) N.delSnapshot(id).catch(e => nativeWarn('точка не удалена', e)); }

    /* Точки восстановления: файлы ↔ IndexedDB приложения (в обе стороны) */
    async function nativeSync() {
      const N = NATIVE();
      if (!N) return;
      try {
        const files = await N.listSnapshots();
        const mine = await snapAll().catch(() => []);
        const haveIdb = new Set(mine.map(s => s.id)), haveFile = new Set(files.map(s => s.id));
        for (const rec of files) if (!haveIdb.has(rec.id)) await snapRun('readwrite', st => st.put(rec));
        for (const rec of mine) if (!haveFile.has(rec.id)) await N.putSnapshot(rec.id, rec);
        if (files.some(s => !haveIdb.has(s.id)) && window.TST && window.TST.workspace === 'fleet' && ui.view === 'safety') fillSafetyAsync();
      } catch (e) { nativeWarn('синхронизация точек восстановления', e); }
    }
    function nativeInit() {
      const N = NATIVE();
      if (!N) return;
      document.documentElement.classList.add('is-native');
      if (N.recovered) {
        const r = N.recovered;
        setTimeout(() => toast('warning', 'Файл данных восстановлен из копии',
          r.from ? `Основной файл был ${r.reason}. Загружена суточная копия ${r.from}; повреждённый файл сохранён в папке данных.`
                 : `Основной файл повреждён, исправных копий нет. Повреждённый файл сохранён в папке данных (${r.corruptFile || ''}); восстановите данные из резервной копии ZIP.`, 15000), 1500);
      }
      setTimeout(nativeSync, 600);
    }

    function renderNativeStorage() {
      const N = NATIVE();
      if (!N) return '';
      return `
        <div class="f-panel">
```

```
        <div class="f-panel-title">Файлы данных приложения</div>
        <p class="f-hint">Приложение хранит данные в обычных файлах: очистка браузера, переустановка приложения или смена профиля Windows их не затрагивают. Папку целиком можно копировать на флешку или другой компьютер.</p>
        <div class="f-info-grid">
          <div><div class="f-info-k">Папка данных</div><div class="f-info-v qr-mono" style="word-break:break-all">${esc(N.dataDir)}</div></div>
          <div><div class="f-info-k">Версия приложения</div><div class="f-info-v">${esc(N.version)}</div></div>
        </div>
        <div class="sf-actions" style="margin-top:12px">
          <button type="button" class="f-btn" data-act="native-open">${FI.file(15)}<span>Открыть папку данных</span></button>
          <button type="button" class="f-btn" data-act="native-choose">${FI.upload(15)}<span>Сменить папку…</span></button>
        </div>
        <div class="f-hint" style="margin-top:10px">Внутри: <b>db.json</b> — записи учёта, <b>backups</b> — суточные копии (последние 14), <b>files</b> — вложения, <b>snapshots</b> — точки восстановления. Если db.json окажется повреждённым, приложение само возьмёт последнюю исправную копию.</div>
      </div>`;
    }
    async function nativeChooseDir() {
      const N = NATIVE();
      if (!N) return;
      const ok = await confirmDialog('Сменить папку данных?', 'Выберите новую папку: если в ней уже есть данные приложения, будут использованы они, иначе текущие данные будут скопированы туда. Приложение перезапустится. Старая папка останется нетронутой.', 'Выбрать папку');
      if (!ok) return;
      const r = await N.chooseDataDir().catch(e => ({ ok: false, error: e.message }));
      if (r && r.ok) toast('success', 'Папка данных изменена', `${r.dir}${r.copied ? ' — данные скопированы' : ' — найдены существующие данные'}. Приложение перезапускается…`, 6000);
      else if (r && r.error) toast('danger', 'Не удалось сменить папку', r.error);
    }

    /* ========================================================================
       27h. ФОТО И ДОКУМЕНТЫ НА ТЕХНИКУ
       Модуль «Файлы техники» (vfiles). Данные — поля единицы техники:
         v.photos  = [{ id, thumbId, name, w, h, size, addedAt, caption }]
         v.vdocs   = [{ id, kind, title, number, date, issuer, note, files:[{ id, name, type, size }] }]
         v.mainPhotoId — фото, показываемое в шапке и в списке
       Сами файлы лежат там же, где вложения ТО (filePut / fileGet / fileDel), поэтому
       резервная копия, точки восстановления и файловое хранилище приложения работают без
       отдельной настройки. Фотографии сжимаются в браузере; миниатюры — отдельные файлы.
       ======================================================================== */
    const VF_DOC_KINDS = ['Паспорт', 'Формуляр', 'Акт приёма-передачи', 'Другой документ'];
    const VF_OTHER = 'Другой документ';
    const VF_MAX_FILE = 20 * 1024 * 1024;
    const VF_MAX_PHOTOS = 100;
    const VF_PHOTO_PX = 1600, VF_THUMB_PX = 320, VF_SCAN_PX = 2600;
    const VF_DOC_ACCEPT = '.pdf,.doc,.docx,.xls,.xlsx,.png,.jpg,.jpeg,.webp,.gif,.txt';
    const vfUrls = new Map();          // id файла → blob-адрес (для предпросмотра)

    const vfPhotos = v => Array.isArray(v && v.photos) ? v.photos : [];
    const vfDocs   = v => Array.isArray(v && v.vdocs) ? v.vdocs : [];

    /* Все идентификаторы файлов единицы техники — для удаления и учёта ссылок */
    function vfFileIds(v) {
      const ids = [];
      vfPhotos(v).forEach(p => { if (p && p.id) ids.push(p.id); if (p && p.thumbId) ids.push(p.thumbId); });
      vfDocs(v).forEach(d => (d && d.files || []).forEach(f => { if (f && f.id) ids.push(f.id); }));
      return ids;
    }

    function vfUrl(id) {
      if (vfUrls.has(id)) return Promise.resolve(vfUrls.get(id));
      return fileGet(id).then(rec => {
        if (!rec || !rec.blob) return '';
        const u = URL.createObjectURL(rec.blob);
        vfUrls.set(id, u);
        return u;
      }).catch(() => '');
    }
    function vfForget(ids) {
      ids.forEach(id => { const u = vfUrls.get(id); if (u) { URL.revokeObjectURL(u); vfUrls.delete(id); } });
    }

    /* Миниатюры подгружаются лениво: в разметке только data-vf-img, адрес ставится после показа */
    function vfHydrate(root) {
      (root || document).querySelectorAll('img[data-vf-img]:not([src])').forEach(img => {
        vfUrl(img.dataset.vfImg).then(u => { if (u) img.src = u; else img.classList.add('is-missing'); });
      });
    }
    let vfHydrateQueued = false;
    function vfWatch() {
      if (typeof MutationObserver === 'undefined') return;
      new MutationObserver(() => {
        if (vfHydrateQueued) return;
        vfHydrateQueued = true;
        requestAnimationFrame(() => { vfHydrateQueued = false; vfHydrate(document); });
      }).observe(document.body, { childList: true, subtree: true });
    }
```

```
  /* Сжатие изображения: длинная сторона не больше maxPx, JPEG */
  async function vfScale(file, maxPx, quality) {
    let bmp;
    try { bmp = await createImageBitmap(file); }
    catch (e) { throw new Error('Формат изображения не поддерживается'); }
    const k = Math.min(1, maxPx / Math.max(bmp.width, bmp.height));
    const w = Math.max(1, Math.round(bmp.width * k)), h = Math.max(1, Math.round(bmp.height * k));
    const c = document.createElement('canvas');
    c.width = w; c.height = h;
    const g = c.getContext('2d');
    g.fillStyle = '#fff'; g.fillRect(0, 0, w, h);      // прозрачность PNG → белый фон
    g.drawImage(bmp, 0, 0, w, h);
    if (bmp.close) bmp.close();
    const blob = await new Promise(res => c.toBlob(res, 'image/jpeg', quality));
    if (!blob) throw new Error('Не удалось сжать изображение');
    return { blob, w, h };
  }
  const vfBase = n => String(n || 'photo').replace(/\.[^.]+$/, '').slice(0, 80) || 'photo';

  async function vfAddPhotos(v, files) {
    const list = Array.from(files || []);
    let added = 0, bad = 0;
    for (const f of list) {
      if (vfPhotos(v).length >= VF_MAX_PHOTOS) { toast('warning', 'Достигнут предел', `На одну единицу техники — не более ${VF_MAX_PHOTOS} фотографий.`); break; }
      if (!/^image\//.test(f.type)) { bad++; continue; }
      try {
        const big = await vfScale(f, VF_PHOTO_PX, 0.82);
        const th = await vfScale(f, VF_THUMB_PX, 0.7);
        const id = uid('file'), thumbId = uid('file');
        const name = vfBase(f.name) + '.jpg';
        await filePut({ id, name, type: 'image/jpeg', size: big.blob.size, blob: big.blob });
        await filePut({ id: thumbId, name: 'thumb_' + name, type: 'image/jpeg', size: th.blob.size, blob: th.blob });
        if (!Array.isArray(v.photos)) v.photos = [];
        v.photos.push({ id, thumbId, name, w: big.w, h: big.h, size: big.blob.size, addedAt: new Date().toISOString(), caption: '' });
        if (!v.mainPhotoId) v.mainPhotoId = id;
        added++;
      } catch (e) { bad++; }
    }
    if (added) { v.updatedAt = new Date().toISOString(); saveDB(); }
    if (bad) toast('warning', 'Не все файлы добавлены', `Пропущено: ${bad}. Нужны изображения JPG, PNG, WebP или GIF.`);
    if (added) toast('success', 'Фотографии добавлены', `${added} ${plural(added, 'фото', 'фото', 'фото')} · ${v.name}`);
    return added;
  }

  function vfRemovePhoto(v, pid) {
    const p = vfPhotos(v).find(x => x.id === pid);
    if (!p) return;
    v.photos = v.photos.filter(x => x.id !== pid);
    if (v.mainPhotoId === pid) v.mainPhotoId = v.photos.length ? v.photos[0].id : '';
    vfForget([p.id, p.thumbId]);
    saveDB();
    cleanupOrphanFiles();           // файл удаляется, только если на него не ссылается корзина или точка восстановления
  }

  function vfThumbId(v) {
    const ph = vfPhotos(v);
    const p = ph.find(x => x.id === v.mainPhotoId) || ph[0];
    return p ? (p.thumbId || p.id) : '';
  }
  function vfThumbHtml(v, cls) {
    const id = vfThumbId(v);
    return id ? `<img class="${cls || 'vf-mini'}" data-vf-img="${esc(id)}" alt="Фото: ${esc(v.name)}" loading="lazy">` : '';
  }

  /* ---------- Вкладка «Фото и документы» ---------- */
  const vfIsImg = f => /^image\//.test((f && f.type) || '') || /\.(png|jpe?g|webp|gif)$/i.test((f && f.name) || '');

  function renderVehicleFiles(v) {
    const ph = vfPhotos(v), dc = vfDocs(v);
    const coarse = typeof matchMedia === 'function' && matchMedia('(pointer: coarse)').matches;
    const photos = ph.length ? `<div class="vf-grid">${ph.map(p => `
        <button type="button" class="vf-ph ${p.id === v.mainPhotoId ? 'is-main' : ''}" data-act="vf" data-do="photo" data-pid="${esc(p.id)}" title="${esc(p.caption || p.name)}">
          <img data-vf-img="${esc(p.thumbId || p.id)}" alt="${esc(p.caption || p.name)}" loading="lazy">
          ${p.id === v.mainPhotoId ? '<span class="vf-main-tag">Главное</span>' : ''}
        </button>`).join('')}</div>`
      : `<div class="vf-empty">Фотографий пока нет. Добавьте общий вид, номера, табличку с заводским номером, повреждения.</div>`;
    const order = k => { const i = VF_DOC_KINDS.indexOf(k); return i < 0 ? VF_DOC_KINDS.length : i; };
    const docs = dc.length ? `<div class="vf-docs">${dc.slice().sort((a, b) => order(a.kind) - order(b.kind)).map(d => {
        const head = d.kind === VF_OTHER && d.title ? d.title : d.kind;
        const meta = [d.number ? '№ ' + d.number : '', d.date ? 'от ' + fmtDate(d.date) : '', d.issuer || ''].filter(Boolean).join(' · ');
        return `<div class="vf-doc">
          <div class="vf-doc-main">
            <div class="vf-doc-head"><span class="f-badge f-badge-
```

```
neutral">${esc(d.kind)}</span><b>${esc(head)}</b></div>
7554            ${meta ? `<div class="vf-doc-meta">${esc(meta)}</div>` : ''}
7555            ${d.note ? `<div class="vf-doc-note">${esc(d.note)}</div>` : ''}
7556            <div class="vf-doc-files">${(d.files || []).length ? d.files.map(f => `
7557              <button type="button" class="f-btn f-btn-sm" data-act="vf" data-do="docfile" data-did="${esc(d.id)}" data-
fid="${esc(f.id)}" title="${vfIsImg(f) ? 'Открыть' : 'Скачать'}: ${esc(f.name)}">
7558                ${vfIsImg(f) ? FI.file(13) : FI.download(13)}<span>${esc(f.name)}</span><small>${fmtSize(f.size ||
0)}</small></button>`).join('')
7559                : '<span class="f-td-muted">Скан не приложен — документ учтён по бумажному экземпляру</span>'}</div>
7560          </div>
7561          <div class="f-row-actions">
7562            <button type="button" class="f-icon-btn" data-act="vf" data-do="doc-edit" data-did="${esc(d.id)}"
title="Редактировать">${FI.edit(15)}</button>
7563            <button type="button" class="f-icon-btn is-danger" data-act="vf" data-do="doc-del" data-did="${esc(d.id)}"
title="Удалить">${FI.trash(15)}</button>
7564          </div>
7565        </div>`; }).join('')}</div>`
7566      : `<div class="vf-empty">Документов на машину нет. Добавьте паспорт, формуляр, акты приёма-передачи — со сканами
или без них.</div>`;
7567    return `
7568      <div class="f-panel vf-panel" data-vf-drop>
7569        <div class="f-panel-head">
7570          <div class="f-panel-title">Фотографии (${ph.length})</div>
7571          <div class="sf-actions">
7572            <button type="button" class="f-btn f-btn-sm f-btn-primary" data-act="vf" data-do="pick-
photo">${FI.upload(14)}<span>Добавить фото</span></button>
7573            ${coarse ? `<button type="button" class="f-btn f-btn-sm" data-act="vf" data-do="pick-cam">Снять на
камеру</button>` : ''}
7574          </div>
7575        </div>
7576        <input type="file" hidden multiple accept="image/*" data-vf-in="photo" id="vfPhotoIn">
7577        <input type="file" hidden accept="image/*" capture="environment" data-vf-in="photo" id="vfCamIn">
7578        ${photos}
7579        <p class="f-hint">Перетащите фотографии сюда или нажмите «Добавить фото». Снимки сжимаются до ${VF_PHOTO_PX} px
и хранятся на этом компьютере.</p>
7580      </div>
7581      <div class="f-panel vf-panel">
7582        <div class="f-panel-head">
7583          <div class="f-panel-title">Документы на машину (${dc.length})</div>
7584          <div class="sf-actions"><button type="button" class="f-btn f-btn-sm f-btn-primary" data-act="vf" data-do="doc-
add">${FI.plus(14)}<span>Добавить документ</span></button></div>
7585        </div>
7586        ${docs}
7587      </div>`;
7588  }
7589
7590  /* ---------- Просмотр фотографий и изображений документов ---------- */
7591  function vfViewer(items, idx, ctx) {
7592    let i = idx;
7593    const m = openModal(`
7594      <div class="f-modal-head">
7595        <div><div class="f-modal-title" id="vwTitle"></div><div class="f-modal-sub" id="vwSub"></div></div>
7596        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
7597      </div>
7598      <div class="f-modal-body vf-view">
7599        <button type="button" class="vf-nav vf-prev" aria-label="Предыдущее">‹</button>
7600        <img id="vwImg" alt="">
7601        <button type="button" class="vf-nav vf-next" aria-label="Следующее">›</button>
7602      </div>
7603      <div class="f-modal-foot vf-view-foot">
7604        <input type="text" class="f-input" id="vwCap" placeholder="Подпись к фото" maxlength="120" ${ctx && ctx.photos ?
 '' : 'hidden'}>
7605        <button type="button" class="f-btn" id="vwMain" ${ctx && ctx.photos ? '' : 'hidden'}>Сделать главным</button>
7606        <button type="button" class="f-btn" id="vwDl">${FI.download(15)}<span>Скачать</span></button>
7607        <button type="button" class="f-btn f-btn-danger" id="vwDel" ${ctx && ctx.photos ? '' :
'hidden'}>${FI.trash(15)}<span>Удалить</span></button>
7608      </div>`, { onClose: () => document.removeEventListener('keydown', onKey) });
7609    const q = s => m.root.querySelector(s);
7610    const draw = () => {
7611      const it = items[i];
7612      q('.f-modal-title').textContent = it.title;
7613      q('#vwSub').textContent = `${i + 1} из ${items.length}${it.sub ? ' · ' + it.sub : ''}`;
7614      const img = q('#vwImg');
7615      img.removeAttribute('src');
7616      vfUrl(it.id).then(u => { if (items[i] === it) img.src = u; });
7617      q('.vf-prev').hidden = q('.vf-next').hidden = items.length < 2;
7618      if (ctx && ctx.photos) {
7619        q('#vwCap').value = it.caption || '';
7620        const v = getVehicle(ui.vehicleId);
7621        q('#vwMain').disabled = !!(v && v.mainPhotoId === it.id);
7622      }
7623    };
7624    const step = d => { i = (i + d + items.length) % items.length; draw(); };
7625    function onKey(e) { if (e.key === 'ArrowLeft') step(-1); else if (e.key === 'ArrowRight') step(1); }
7626    document.addEventListener('keydown', onKey);
7627    q('.vf-prev').addEventListener('click', () => step(-1));
7628    q('.vf-next').addEventListener('click', () => step(1));
7629    q('#vwDl').addEventListener('click', () => fileGet(items[i].id).then(r => r && r.blob ? downloadBlob(r.blob,
items[i].name) : toast('warning', 'Файл недоступен', 'Не найден в локальном хранилище.')));
7630    if (ctx && ctx.photos) {
```

```
      q('#vwCap').addEventListener('change', e => {
        const v = getVehicle(ui.vehicleId), p = v && vfPhotos(v).find(x => x.id === items[i].id);
        if (!p) return;
        p.caption = e.target.value.trim().slice(0, 120); items[i].caption = p.caption; saveDB();
      });
      q('#vwMain').addEventListener('click', () => {
        const v = getVehicle(ui.vehicleId); if (!v) return;
        v.mainPhotoId = items[i].id; saveDB(); draw(); rerender();
        toast('success', 'Главное фото выбрано', 'Оно показывается в карточке и в списке техники.');
      });
      q('#vwDel').addEventListener('click', async () => {
        if (!(await confirmDialog('Удалить фото?', items[i].title, 'Удалить'))) return;
        const v = getVehicle(ui.vehicleId); if (!v) return;
        vfRemovePhoto(v, items[i].id);
        m.close(); rerender();
      });
    }
    draw();
  }

  /* ---------- Форма документа ---------- */
  function vfDocModal(v, docId) {
    const cur = docId ? vfDocs(v).find(d => d.id === docId) : null;
    const rec = cur || { kind: VF_DOC_KINDS[0], title: '', number: '', date: '', issuer: '', note: '', files: [] };
    let files = (rec.files || []).map(f => Object.assign({}, f));
    const newBlobs = new Map();
    const fileRow = f => `<div class="f-file"><span class="f-file-name">${esc(f.name)}</span><span class="f-td-
muted">${fmtSize(f.size || 0)}</span>
      <button type="button" class="f-icon-btn is-danger" data-rm="${esc(f.id)}" title="Убрать" aria-label="Убрать файл
 ${esc(f.name)}">${FI.x(14)}</button></div>`;
    const m = openModal(`
      <div class="f-modal-head">
        <div><div class="f-modal-title">${cur ? 'Изменить документ' : 'Добавить документ'}</div><div class="f-modal-
sub">${esc(v.name)}</div></div>
        <button type="button" class="f-modal-close" data-close aria-label="Закрыть">${FI.x(16)}</button>
      </div>
      <div class="f-modal-body">
        <div class="f-form-grid">
          <div><label class="f-fld-label" for="vdKind">Вид документа</label>
            <select id="vdKind" class="f-select">${VF_DOC_KINDS.map(k => `<option ${k === rec.kind ? 'selected' :
''}>${esc(k)}</option>`).join('')}</select></div>
          <div id="vdTitleBox"><label class="f-fld-label" for="vdTitle">Название</label>
            <input id="vdTitle" class="f-input" maxlength="120" value="${esc(rec.title)}" placeholder="Например:
свидетельство о регистрации"></div>
          <div><label class="f-fld-label" for="vdNumber">Номер</label><input id="vdNumber" class="f-input"
maxlength="60" value="${esc(rec.number)}"></div>
          <div><label class="f-fld-label" for="vdDate">Дата</label><input id="vdDate" class="f-input" type="date"
value="${esc(rec.date)}"></div>
          <div class="f-span2"><label class="f-fld-label" for="vdIssuer">Кем выдан, составлен или подписан</label><input
 id="vdIssuer" class="f-input" maxlength="160" value="${esc(rec.issuer)}"></div>
          <div class="f-span2"><label class="f-fld-label" for="vdNote">Примечание</label><textarea id="vdNote"
class="f-textarea" maxlength="500">${esc(rec.note)}</textarea></div>
          <div class="f-span2">
            <label class="f-fld-label">Сканы и файлы</label>
            <div class="f-drop" id="vdDrop">${FI.upload(18)} Перетащите файлы сюда или нажмите для выбора<br>
              <span style="font-size:11.5px">PDF, Word, Excel, изображения · до ${VF_MAX_FILE / 1048576} МБ</span></div>
            <input type="file" id="vdFiles" multiple hidden accept="${VF_DOC_ACCEPT}">
            <div class="f-files" id="vdList"></div>
          </div>
        </div>
      </div>
      <div class="f-modal-foot">
        <button type="button" class="f-btn" data-close>Отмена</button>
        <button type="button" class="f-btn f-btn-primary" id="vdSave">${FI.save(15)}<span>Сохранить</span></button>
      </div>`);
    const q = id => m.root.querySelector('#' + id);
    const kindUi = () => { q('vdTitleBox').hidden = q('vdKind').value !== VF_OTHER; };
    q('vdKind').addEventListener('change', kindUi); kindUi();
    const list = q('vdList');
    const redraw = () => {
      list.innerHTML = files.map(fileRow).join('');
      list.querySelectorAll('[data-rm]').forEach(b => b.addEventListener('click', () => {
        files = files.filter(f => f.id !== b.dataset.rm); newBlobs.delete(b.dataset.rm); redraw();
      }));
    };
    redraw();
    const add = fl => {
      Array.from(fl).forEach(f => {
        if (f.size > VF_MAX_FILE) { toast('warning', 'Файл слишком большой', `${f.name} — более ${VF_MAX_FILE / 1048576}
 МБ, пропущен.`); return; }
        const id = uid('file');
        files.push({ id, name: f.name, type: f.type, size: f.size });
        newBlobs.set(id, f);
      });
      redraw();
    };
    const drop = q('vdDrop');
    drop.addEventListener('click', () => q('vdFiles').click());
    q('vdFiles').addEventListener('change', e => { add(e.target.files); e.target.value = ''; });
    ['dragenter', 'dragover'].forEach(ev => drop.addEventListener(ev, e => { e.preventDefault(); drop.classList.add('is-
```

```
over'); }));
  ['dragleave', 'drop'].forEach(ev => drop.addEventListener(ev, e => { e.preventDefault(); drop.classList.remove('is-over'); }));
  drop.addEventListener('drop', e => { if (e.dataTransfer && e.dataTransfer.files) add(e.dataTransfer.files); });

  let saving = false;
  q('vdSave').addEventListener('click', async () => {
    if (saving) return;
    saving = true;
    try {
      const kind = q('vdKind').value;
      const title = kind === VF_OTHER ? q('vdTitle').value.trim() : '';
      if (kind === VF_OTHER && !title) { toast('warning', 'Укажите название', 'Для «Другого документа» название обязательно.'); saving = false; return; }
      // Новые файлы записываются в хранилище; изображения-сканы сжимаются
      for (const f of files) {
        const blob = newBlobs.get(f.id);
        if (!blob) continue;
        let out = blob, name = f.name, type = f.type;
        if (/^image\/(png|jpeg|webp)$/.test(blob.type) && blob.size > 1.5 * 1048576) {
          try { const s = await vfScale(blob, VF_SCAN_PX, 0.85); out = s.blob; type = 'image/jpeg'; name = vfBase(f.name) + '.jpg'; } catch (e) { /* оставляем как есть */ }
        }
        await filePut({ id: f.id, name, type, size: out.size, blob: out });
        f.name = name; f.type = type; f.size = out.size;
      }
      const data = { kind, title, number: q('vdNumber').value.trim(), date: q('vdDate').value,
        issuer: q('vdIssuer').value.trim(), note: q('vdNote').value.trim(), files };
      if (!Array.isArray(v.vdocs)) v.vdocs = [];
      if (cur) {
        const dropped = (cur.files || []).filter(f => !files.some(x => x.id === f.id)).map(f => f.id);
        Object.assign(cur, data);
        vfForget(dropped);
      } else {
        v.vdocs.push(Object.assign({ id: uid('doc') }, data));
      }
      v.updatedAt = new Date().toISOString();
      saveDB();
      m.close();
      toast('success', cur ? 'Документ изменён' : 'Документ добавлен', `${kind === VF_OTHER ? title : kind} · ${v.name}`);
      rerender();
      cleanupOrphanFiles();
    } catch (e) {
      saving = false;
      toast('danger', 'Не удалось сохранить', 'Файл не записан в хранилище. Проверьте свободное место и повторите.');
    }
  });
}

/* ---------- Действия ---------- */
function vfAction(node) {
  const v = getVehicle(ui.vehicleId);
  if (!v) return;
  const d = node.dataset;
  switch (d.do) {
    case 'pick-photo': document.getElementById('vfPhotoIn')?.click(); break;
    case 'pick-cam':   document.getElementById('vfCamIn')?.click(); break;
    case 'open-files': ui.vehicleTab = 'files'; rerender(); break;
    case 'photo': {
      const items = vfPhotos(v).map(p => ({ id: p.id, name: p.name, caption: p.caption || '', title: p.caption || p.name,
        sub: `${p.w || '?'}×${p.h || '?'} · ${fmtSize(p.size || 0)}` }));
      const at = items.findIndex(x => x.id === d.pid);
      if (at >= 0) vfViewer(items, at, { photos: true });
      break;
    }
    case 'doc-add':  vfDocModal(v, null); break;
    case 'doc-edit': vfDocModal(v, d.did); break;
    case 'doc-del': {
      const doc = vfDocs(v).find(x => x.id === d.did);
      if (!doc) break;
      confirmDialog('Удалить документ?', `${doc.kind}${doc.number ? ' № ' + doc.number : ''} — вместе со сканами`, 'Удалить').then(ok => {
        if (!ok) return;
        v.vdocs = v.vdocs.filter(x => x.id !== doc.id);
        vfForget((doc.files || []).map(f => f.id));
        v.updatedAt = new Date().toISOString();
        saveDB(); rerender(); cleanupOrphanFiles();
        toast('success', 'Документ удалён', v.name);
      });
      break;
    }
    case 'docfile': {
      const doc = vfDocs(v).find(x => x.id === d.did);
      const f = doc && (doc.files || []).find(x => x.id === d.fid);
      if (!f) break;
      if (vfIsImg(f)) {
        const imgs = (doc.files || []).filter(vfIsImg).map(x => ({ id: x.id, name: x.name, title: x.name, sub: doc.kind + (doc.number ? ' № ' + doc.number : '') }));
```

```
          vfViewer(imgs, Math.max(0, imgs.findIndex(x => x.id === f.id)), { photos: false });
        } else {
          fileGet(f.id).then(r => r && r.blob ? downloadBlob(r.blob, r.name || f.name) : toast('warning', 'Файл недоступен', 'Не найден в локальном хранилище.'));
        }
        break;
      }
    }
  }

  /* Выбор файлов и перетаскивание фотографий — общий обработчик документа */
  function vfBindInputs() {
    document.addEventListener('change', e => {
      const t = e.target;
      if (!(t && t.matches && t.matches('input[data-vf-in="photo"]'))) return;
      const v = getVehicle(ui.vehicleId);
      const fl = Array.from(t.files || []);
      t.value = '';
      if (v && fl.length) vfAddPhotos(v, fl).then(() => rerender());
    });
    const zone = e => e.target && e.target.closest ? e.target.closest('[data-vf-drop]') : null;
    document.addEventListener('dragover', e => { const z = zone(e); if (z) { e.preventDefault(); z.classList.add('is-over'); } });
    document.addEventListener('dragleave', e => { const z = zone(e); if (z) z.classList.remove('is-over'); });
    document.addEventListener('drop', e => {
      const z = zone(e);
      if (!z) return;
      e.preventDefault(); z.classList.remove('is-over');
      const v = getVehicle(ui.vehicleId);
      if (v && e.dataTransfer && e.dataTransfer.files.length) vfAddPhotos(v, e.dataTransfer.files).then(() => rerender());
    });
  }
  vfBindInputs();
  if (document.body) vfWatch(); else document.addEventListener('DOMContentLoaded', vfWatch);


  /* ============================================================================
     28а. ПУБЛИЧНЫЙ API МОДУЛЯ — window.TST.fleet
     Только чтение сводок и переходы; изменение данных — через формы модуля
     ============================================================================ */
  function unitInfo(v, st) {
    st = st || computeStatus(v);
    const kind = st.level === 'retired' ? '' : nextMaintKind(v);
    const nextText = st.level === 'retired' ? 'снято с учёта'
      : st.nextDate ? kind + ' до ' + fmtDate(st.nextDate)
      : st.nextHours != null ? kind + ' при ' + fmtNum(st.nextHours, 0) + ' мото·ч' : 'интервал не задан';
    return {
      id: v.id, name: v.name, regNumber: v.regNumber || '', status: v.status,
      catalogId: v.catalogId || '', level: st.level, levelLabel: LEVELS[st.level].label,
      pct: st.pct, nextText
    };
  }

  const FLEET_API = {
    unitsFor(catalogId) {
      if (!DB || !catalogId) return [];
      return DB.vehicles.filter(v => v.catalogId === catalogId)
        .map(v => unitInfo(v))
        .sort((a, b) => LEVELS[a.level].order - LEVELS[b.level].order || a.name.localeCompare(b.name, 'ru'));
    },
    worstLevel(units) {
      return units.reduce((w, u) => (LEVELS[u.level].order < LEVELS[w].order ? u.level : w), 'retired');
    },
    levelLabel: level => (LEVELS[level] || LEVELS.none).label,
    summary() {
      const rows = DB.vehicles.filter(v => v.status !== 'Списано').map(v => ({ v, st: computeStatus(v) }));
      const cnt = lvl => rows.filter(r => r.st.level === lvl).length;
      return {
        total: rows.length,
        overdue: cnt('overdue'), urgent: cnt('urgent'), soon: cnt('soon'), ok: cnt('ok'),
        linked: rows.filter(r => catNodeOf(r.v)).length,
        noNorm: rows.filter(r => catNodeOf(r.v) && !catMaintOf(r.v)).length,
        upcoming: rows.filter(r => ['overdue', 'urgent', 'soon'].indexOf(r.st.level) !== -1)
          .sort((a, b) => a.st.pct - b.st.pct).slice(0, 5).map(r => unitInfo(r.v, r.st))
      };
    },
    openVehicle(id) {
      if (!getVehicle(id)) return;
      ui.view = 'vehicle'; ui.vehicleId = id; ui.vehicleTab = 'info';
      window.TST.setWorkspace('fleet');           // событие 'workspace' перерисует раздел
      el('fleetScroll').scrollTo({ top: 0 });
    },
    addVehicle(prefill) {
      window.TST.setWorkspace('fleet');
      vehicleModal(null, prefill);
    },
    /* Поиск техники: наименование, марка, гос./инв./зав. номер, подразделение,
       ответственный. Номера сравниваются и без пробелов («АА1234» = «АА 1234 77») */
    search(q, limit) {
```

```
      const norm = s => String(s || '').toLowerCase().replace(/ё/g, 'е');
      const s = norm(q).trim();
      if (!s || !DB) return { items: [], total: 0 };
      const tight = s.replace(/\s+/g, '');
      const fields = [['regNumber', 'гос. номер'], ['invNumber', 'инв. номер'], ['serialNumber', 'зав. номер'],
                      ['name', ''], ['brand', 'марка'], ['unit', 'подразделение'], ['responsible', 'ответственный']];
      const hits = [];
      DB.vehicles.forEach(v => {
        for (const [f, label] of fields) {
          const val = norm(v[f]);
          if (val && (val.includes(s) || (tight.length >= 3 && val.replace(/\s+/g, '').includes(tight)))) {
            hits.push(Object.assign(unitInfo(v), { field: label, fieldValue: v[f], rank: f === 'name' || f === 'regNumber' ? 0 : 1 }));
            break;
          }
        }
      });
      hits.sort((a, b) => a.rank - b.rank || a.name.localeCompare(b.name, 'ru'));
      return { items: hits.slice(0, limit || 6), total: hits.length };
    },
    backupInfo,
    backupAgoText,
    backup: backupExport,
    customMaint,
    editNorm: (catalogId, type) => normModal(catalogId, type),
    catalogEdits: () => (DB ? DB.catalogEdits : []),
    editCatalogNode: id => catalogModal({ id }),
    addCatalogNode: parentId => catalogModal({ parentId }),
    deleteCatalogNode,
    resetCatalogNode,
    hideCatalogNode,
    manageCatalog: () => catalogManager(),
    deleteNorm,
    printChecklist,
    openPrint: () => openPrintWindow(),
    openView(view) { if (!NAV.some(n => n.key === view)) return; ui.view = view; window.TST.setWorkspace('fleet'); },
    commandSummary() { const S = commandStats(); return { total: S.total, ready: S.ready.length, faulty: S.faulty.length, repair: S.repair.length, ktg: S.ktg }; },
    qrPayload(id, level, format) {
      const v = getVehicle(id);
      if (!v) return null;
      return format ? qrFit(v, level || 'basic', format).text : qrPayload(v, level || 'basic');
    },
    qrState: id => { const v = getVehicle(id); return v ? qrState(v).code : null; },
    openScanned: text => qrOpenScanned(text),

    /* Данные для справки по учёту: нормативы, пороги, перечни и движок
       расчёта состояния ТО — примеры справки считаются той же функцией */
    helpData() {
      if (!DB) return null;
      const clone = o => JSON.parse(JSON.stringify(o));
      return {
        today: todayISO(),
        settings: clone(DB.settings), defaults: clone(DEFAULT_SETTINGS),
        statuses: STATUSES.slice(), maintKinds: MAINT_KINDS.slice(),
        resetKinds: Array.from(RESET_KINDS), to2ResetKinds: Array.from(TO2_RESET_KINDS),
        catalogKinds: Object.keys(KIND_TO_CATALOG),
        expGroups: EXP_GROUPS.slice(), levels: clone(LEVELS), to2Every: to2Every(),
        reports: REPORTS.map(r => ({ title: r.title, desc: r.desc })),
        qrFormats: Object.keys(QR_FORMATS).map(k => ({ key: k, label: QR_FORMATS[k].label, qr: QR_FORMATS[k].qr })),
        qrMinModule: QR_MIN_MODULE_MM,
        counts: { vehicles: DB.vehicles.length, maint: DB.maint.length, hours: DB.hours.length, trips: DB.trips.length, norms: DB.norms.length },
        /* Условная единица без записей в журнале: поставлена на учёт regAgo
           дней назад при 0 мото·ч, сейчас current мото·ч */
        example({ hours, days, regAgo, current }) {
          return computeStatus({
            id: '__help_example__', status: 'В эксплуатации', currentHours: current,
            regDate: addDays(todayISO(), -regAgo),
            interval: { byHours: true, hours, byDate: true, days }
          });
        }
      };
    }
  };

  /* ============================================================================
     29. ИНИЦИАЛИЗАЦИЯ МОДУЛЯ
     ============================================================================ */
  function initFleet() {
    registerFleetModules();
    loadDB();
    nativeInit();
    if (lastMigration) saveDB();          // обновлённая структура сохраняется сразу
    trashPurgeOld();
    setTimeout(snapAutoDaily, 4000);
    if (newerData) setTimeout(() => toast('warning', 'Данные из более новой версии программы',
      `Структура данных ${newerData.from}, программа понимает ${newerData.to}. Неизвестные сведения сохранятся, но могут не отображаться — обновите файл программы.`, 12000), 1500);

```

```
    // Первый запуск — наполняем демонстрационными записями
    if (!DB.seeded && !DB.vehicles.length) seedDemo();

    // Один обработчик на #fleetRoot — навбар является его потомком.
    // Панель уведомлений вынесена из #fleetRoot, поэтому слушается отдельно.
    el('fleetRoot').addEventListener('click', onFleetClick);
    el('fleetNotifPanel').addEventListener('click', onFleetClick);

    if (window.TST) {
      /* Маршрут учёта: #fleet/<раздел> или #fleet/vehicle/<id> */
      window.TST.routeProviders.fleet = () =>
        ui.view === 'vehicle' && getVehicle(ui.vehicleId) ? 'fleet/vehicle/' + encodeURIComponent(ui.vehicleId) :
    'fleet/' + ui.view;
      window.TST.routeHandlers.fleet = ([view, id]) => {
        if (view === 'scan') {
          ui.view = 'qr';
          if (id) setTimeout(() => qrOpenScanned(QR_PREFIX + ':' + QR_SCHEME + ':' + id), 0);
          return;
        }
        if (view === 'vehicle') {
          // Карточка удалена или ссылка неверна — открываем список техники
          if (getVehicle(id)) { ui.view = 'vehicle'; ui.vehicleId = id; ui.vehicleTab = 'info'; }
          else ui.view = 'park';
        } else if (NAV.some(n => n.key === view)) ui.view = view;
      };
      window.TST.fleet = FLEET_API;
      window.TST.toast = (type, title, text) => toast(type, title, text);
      window.TST.on('workspace', onWorkspace);
      // Постоянное хранилище запрашивается один раз — при первом входе в
      // приложение (после действия пользователя: так браузер охотнее разрешает)
      let persistAsked = false;
      try { persistAsked = !!localStorage.getItem(LS_PERSIST_ASKED); } catch (e) { /* приватный режим */ }
      if (!persistAsked) {
        let once = false;
        window.TST.on('workspace', () => { if (!once) { once = true; requestPersist(false); } });
      }
    }
    refreshStorageInfo();

    // Другая вкладка сохранила данные. Если открыта форма — ждём её закрытия:
    // форма держит ссылки на текущие объекты, их нельзя подменять на лету
    window.addEventListener('storage', e => {
      if (e.key !== LS_KEY && e.key !== null) return;
      if (document.querySelector('#fleetModalHost .f-modal-backdrop')) { pendingExternal = true; return; }
      applyExternal();
    });

    // Закрытие вкладки: откладывать стирание вложений больше некому
    window.addEventListener('pagehide', () => { pendingFileDeletes.clear(); });
    setTimeout(cleanupOrphanFiles, UNDO_MS + 7000);

    // Колокол уведомлений
    const bell = el('fleetBell');
    bell.addEventListener('click', e => {
      e.stopPropagation();
      const panel = el('fleetNotifPanel');
      const open = panel.classList.toggle('is-open');
      bell.setAttribute('aria-expanded', String(open));
      if (open) renderNotifPanel();
    });
    document.addEventListener('click', e => {
      const panel = el('fleetNotifPanel');
      if (panel.classList.contains('is-open') && !panel.contains(e.target) && !bell.contains(e.target)) closeNotif();
    });

    renderNotifPanel();
    // Раздел не открывается здесь: его выбирает TST.showApp() при выходе с
    // лендинга. Раньше модуль сам «открывался» при загрузке под лендингом,
    // показывал скрытые стартовые уведомления и помечал их как показанные —
    // при реальном входе в раздел сводка уже не появлялась
  }

  if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', initFleet);
  } else {
    initFleet();
  }

  })();
```

5. Обзор и быстрая навигация (index.html, скрипт 5)

```
/* Dashboard rebuild: overview, KPI snapshot and quick navigation. */
(function(){
  function leafCount(nodes){return nodes.reduce((n,x)=>n+(x.children?leafCount(x.children):1),0)}
  function allLeaves(nodes){return nodes.flatMap(x=>x.children?allLeaves(x.children):[x])}
  function esc(v){return String(v).replace(/[&<>"']/g,m=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[m]))}
  // Палитра донат-диаграммы — переиспользуем цвета, уже принятые в теме
  // приложения (акцент, индиго градиентов, оранжевый статус), а не вводим новые
  const DONUT_COLORS=['var(--accent)','var(--accent-2)','var(--warning)','var(--st-ok-text)','var(--accent-3)'];
  function buildDonut(segments,size,thickness){
    const r=(size-thickness)/2, c=2*Math.PI*r, total=segments.reduce((s,x)=>s+x.value,0)||1;
    let offset=0;
    const circles=segments.map(seg=>{
      const len=seg.value/total*c, dash=`${len} ${c-len}`, dashoffset=-offset;
      offset+=len;
      return `<circle cx="${size/2}" cy="${size/2}" r="${r}" fill="none" stroke="${seg.color}" stroke-width="${thickness}" stroke-dasharray="${dash}" stroke-dashoffset="${dashoffset}"/>`;
    }).join('');
    return `<svg width="${size}" height="${size}" viewBox="0 0 ${size} ${size}" style="transform:rotate(-90deg)">${circles}</svg>`;
  }
  function renderOverview(){
    const leaves=allLeaves(CATALOG),
actions=leaves.reduce((n,x)=>n+Object.values(x.maintenance||{}).reduce((s,m)=>s+(m.actions||[]).length,0),0);
    const k=document.getElementById('overviewKpis');
    k.innerHTML=[['технических средств',leaves.length],['карт обслуживания',leaves.reduce((n,x)=>n+Object.keys(x.maintenance||{}).length,0)],['регламентных действий',actions]].map(x=>`<div class="overview-fleet-stat"><b>${x[1].toLocaleString('ru-RU')}</b><span>${x[0]}</span></div>`).join('');

    // Структура по службам: донат + полосы, один и тот же цвет на сегменте и на маркере полосы
    const structure=CATALOG.map((x,i)=>({name:x.name.replace(/^Технические средства (?:службы )?/,'').replace('службы',''),full:x.name,value:leafCount(x.children||[x]),color:DONUT_COLORS[i%DONUT_COLORS.length]}));
    const maxStruct=Math.max(...structure.map(s=>s.value),1);
    document.getElementById('overviewDonut').innerHTML=buildDonut(structure,110,18)+`<div class="overview-donut-center"><b>${leaves.length.toLocaleString('ru-RU')}</b><span>средств</span></div>`;
    document.getElementById('overviewBars').innerHTML=structure.map(s=>`<div class="overview-bar-row"><span title="${esc(s.full)}"><i class="overview-bar-dot" style="background:${s.color}"></i><span class="overview-bar-label-text">${esc(s.name)}</span></span><div class="overview-bar-track"><div class="overview-bar-fill" style="width:${Math.round(s.value/maxStruct*100)}%;background:${s.color}"></div></div><b>${s.value}</b></div>`).join('');

    document.getElementById('overviewDate').textContent=new Intl.DateTimeFormat('ru-RU',{dateStyle:'long'}).format(new Date());
  }
  /* Состояние парка из модуля учёта (TST.fleet) — связующий блок обзора */
  function renderFleet(){
    const box=document.getElementById('overviewFleet');
    const F=window.TST&&TST.fleet;
    if(!box)return;
    if(!F){box.innerHTML='<div class="overview-empty">Модуль учёта недоступен.</div>';return;}
    const s=F.summary();
    const stat=(v,l,lvl)=>`<div class="overview-fleet-stat"><b${lvl?` style="color:var(--st-${lvl}-text)"`:''}>${v.toLocaleString('ru-RU')}</b><span>${l}</span></div>`;
    box.innerHTML=`
      <div>
        <div class="overview-fleet-stats is-row">
          ${stat(s.overdue,'ТО просрочено','overdue')}
          ${stat(s.urgent+s.soon,'ТО в ближайшее время','urgent')}
          ${stat(s.ok,'ТО в норме','ok')}
        </div>
        <p class="overview-fleet-note">Всего на учёте: ${s.total.toLocaleString('ru-RU')}${F.commandSummary&&F.commandSummary().ktg!=null?` · КТГ ${Math.round(F.commandSummary().ktg*100)}%`:''}${s.noNorm?` · <b style="color:var(--st-urgent-text)">без норматива ТО: ${s.noNorm}</b>`:''}</p>
        ${F.openView?'<button type="button" class="link-btn" data-overview-command>Пункт управления →</button>':''}
      </div>
      <div class="overview-fleet-list">
        <h3>Ближайшие ТО</h3>
        ${s.upcoming.length?`<div class="link-units">${s.upcoming.map(u=>`
          <button type="button" class="link-unit" data-overview-vehicle="${esc(u.id)}">
            <span class="f-dot f-lvl-${u.level}"></span>
            <span class="link-unit-name">${esc(u.name)}${u.regNumber?` <span class="link-unit-plate">${esc(u.regNumber)}</span>`:''}</span>
            <span class="link-unit-meta">${esc(u.nextText)}</span>
            <span class="f-badge f-badge-${u.level}">${esc(u.levelLabel)}</span>
          </button>`).join('')}</div>`:'<div class="overview-empty">Вся техника обслужена в срок.</div>'}
      </div>`;
  }
  /* Напоминание о резервной копии — данные хранятся только в браузере */
  function renderBackup(){
    const box=document.getElementById('overviewBackup');
    const F=window.TST&&TST.fleet;
    if(!box||!F){ if(box)box.innerHTML=''; return; }
    const b=F.backupInfo();
    if(!b.hasData&&!b.last){ box.innerHTML=''; return; }
    const ago=F.backupAgoText(b);
    const persist=!window.TSTNative&&b.persistSupported&&!b.persisted?' Постоянное хранение браузера не включено (Настройки учёта).':'';
    box.innerHTML=`<div class="overview-backup ${b.due?'is-due':''}">
      <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" aria-hidden="true"><path d="M19 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11l5 5v11a2
```

5. Обзор и быстрая навигация (index.html, скрипт 5) (продолжение)

```
 2 0 0 1-2 2z"/><polyline points="17 21 17 13 7 13 7 21"/><polyline points="7 3 7 8 15 8"/></svg>
      <div class="overview-backup-text"><b>${esc(ago.charAt(0).toUpperCase()+ago.slice(1))}</b>
        <span>${window.TSTNative?'Данные учёта хранятся в файлах на этом компьютере — сохраните копию на флешку или сетевой диск.':'Данные учёта хранятся только в этом браузере — сохраните копию на флешку или сетевой диск.'}${esc(persist)}</span></div>
      <button type="button" class="link-btn" data-overview-backup>Сохранить копию</button>
    </div>`;
  }
  function initOverview(){
    // Переключение разделов — общий роутер TST; обзор лишь перерисовывается при входе
    TST.on('workspace',({ws})=>{ if(ws==='overview'){renderOverview();renderFleet();renderBackup();} });
    TST.on('fleet-changed',()=>{ if(TST.workspace==='overview'){renderFleet();renderBackup();} });
    document.getElementById('overviewBackup').addEventListener('click',e=>{
      if(e.target.closest('[data-overview-backup]')&&TST.fleet)TST.fleet.backup();
    });
    document.querySelectorAll('[data-overview-go]').forEach(b=>b.addEventListener('click',()=>TST.setWorkspace(b.dataset.overviewGo)));
    document.getElementById('overviewFleet')?.addEventListener('click',e=>{
      if(e.target.closest('[data-overview-command]')&&TST.fleet){TST.fleet.openView('command');return;}
      const b=e.target.closest('[data-overview-vehicle]');
      if(b&&TST.fleet)TST.fleet.openVehicle(b.dataset.overviewVehicle);
    });
    document.getElementById('overviewDate').textContent=new Intl.DateTimeFormat('ru-RU',{dateStyle:'long'}).format(new Date());
  }
  if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', initOverview);
  } else {
    initOverview();
  }
})();
```

6. Общий поиск в шапке (index.html, скрипт 6)

```
/* ========================================================================
   ОБЩИЙ ПОИСК В ШАПКЕ
   ------------------------------------------------------------------------
   Одно поле для обоих блоков: техника из учёта (TST.fleet.search) и
   позиции справочника (TST.catalog.search). В разделе «Справочник» поле
   по-прежнему фильтрует дерево — панель тогда показывает только технику.
   Клавиатура: ↑/↓ — выбор, Enter — открыть, Escape — закрыть.
   ======================================================================== */
(() => {
  const input = document.getElementById('searchInput');
  const panel = document.getElementById('globalSearchPanel');
  const wrap = document.getElementById('catalogSearchWrap');
  if (!input || !panel) return;
  const esc = s => String(s == null ? '' : s).replace(/[&<>"']/g, m => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[m]));
  const truck = '<svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M10 17h4V5H2v12h3"/><path d="M20 17h2v-3.34a4 4 0 0 0-1.17-2.83L19 9h-5v8h2"/><circle cx="7.5" cy="17.5" r="2.5"/><circle cx="17.5" cy="17.5" r="2.5"/></svg>';
  const book = '<svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>';
  let active = -1, timer = null;

  const items = () => Array.from(panel.querySelectorAll('.gs-item'));
  const setActive = i => {
    const list = items();
    active = list.length ? (i + list.length) % list.length : -1;
    list.forEach((n, k) => { n.classList.toggle('is-active', k === active); n.setAttribute('aria-selected', String(k === active)); });
    if (active >= 0) { list[active].scrollIntoView({ block: 'nearest' }); input.setAttribute('aria-activedescendant', list[active].id); }
  };
  const close = () => { panel.hidden = true; active = -1; input.setAttribute('aria-expanded', 'false'); input.removeAttribute('aria-activedescendant'); };

  function render() {
    const q = input.value.trim();
    if (q.length < 2 || !TST.isApp()) { close(); return; }
    const inCatalog = TST.workspace === 'catalog';
    const v = TST.fleet ? TST.fleet.search(q, 6) : { items: [], total: 0 };
    const c = inCatalog ? { items: [], total: 0 } : TST.catalog.search(q, 5);
    if (inCatalog && !v.items.length) { close(); return; }
    let n = 0;
    const item = (attrs, ico, title, sub, extra) => `<button type="button" class="gs-item" role="option" id="gs-opt-${n++}" ${attrs}>
      <span class="gs-item-ico">${ico}</span>
      <span class="gs-item-main"><span class="gs-item-title">${title}</span><span class="gs-item-sub">${sub}</span></span>${extra || ''}</button>`;
    let html = '';
    if (v.items.length) {
      html += `<div class="gs-group">Техника${v.total > v.items.length ? ` · ${v.total}` : ''}</div>`;
      html += v.items.map(u => item(`data-gs-vehicle="${esc(u.id)}"`, truck,
        esc(u.name) + (u.regNumber ? ` · ${esc(u.regNumber)}` : ''),
        esc(u.field ? `${u.field}: ${u.fieldValue}` : u.nextText),
        `<span class="f-badge f-badge-${esc(u.level)}">${esc(u.levelLabel)}</span>`)).join('');
    }
    if (c.items.length) {
      html += `<div class="gs-group">Справочник${c.total > c.items.length ? ` · ${c.total}` : ''}</div>`;
      html += c.items.map(l => item(`data-gs-catalog="${esc(l.id)}"`, book, esc(l.name), esc(l.path.slice(1).join(' › ') || l.service))).join('');
      if (c.total > c.items.length) html += item('data-gs-all-catalog', book, `<span class="gs-more">Все совпадения в справочнике (${c.total}) →</span>`, 'откроется каталог с отфильтрованным деревом');
    }
    if (!html) html = '<div class="gs-empty">Ничего не найдено ни в учёте, ни в справочнике</div>';
    panel.innerHTML = html;
    panel.hidden = false;
    input.setAttribute('aria-expanded', 'true');
    setActive(-1);
    active = -1;
  }

  function choose(node) {
    if (!node) return;
    const q = input.value;
    close();
    if (node.dataset.gsVehicle !== undefined) {
      TST.catalog.clearSearch();
      wrap.classList.remove('is-mobile-open');
      TST.fleet.openVehicle(node.dataset.gsVehicle);
    } else if (node.dataset.gsCatalog !== undefined) {
      wrap.classList.remove('is-mobile-open');
      TST.catalog.open(node.dataset.gsCatalog);          // сам сбрасывает фильтр дерева
    } else if (node.dataset.gsAllCatalog !== undefined) {
      TST.setWorkspace('catalog');                       // дерево уже отфильтровано этим запросом
      input.value = q;
    }
  }

  input.addEventListener('input', () => { clearTimeout(timer); timer = setTimeout(render, 140); });
  input.addEventListener('focus', () => { if (input.value.trim().length >= 2) render(); });
```

6. Общий поиск в шапке (index.html, скрипт 6) (продолжение)

```
      input.addEventListener('keydown', e => {
        if (panel.hidden) { if (e.key === 'ArrowDown' && input.value.trim().length >= 2) { render(); e.preventDefault(); }
     return; }
        if (e.key === 'ArrowDown') { e.preventDefault(); setActive(active + 1); }
        else if (e.key === 'ArrowUp') { e.preventDefault(); setActive(active - 1); }
        else if (e.key === 'Enter') { e.preventDefault(); choose(items()[active >= 0 ? active : 0]); }
        else if (e.key === 'Escape') { close(); }
      });
      // mousedown, а не click: иначе поле теряет фокус и панель закрывается раньше выбора
      panel.addEventListener('mousedown', e => { const n = e.target.closest('.gs-item'); if (n) { e.preventDefault();
    choose(n); } });
      document.addEventListener('mousedown', e => { if (!panel.hidden && !wrap.contains(e.target)) close(); });
      TST.on('workspace', close);
    })();
```

7. Окно печати и справка (index.html, скрипт 7)

```
/* ========================================================================
   ОКНО ПЕЧАТИ И СПРАВКА ПО УЧЁТУ (перенесено из программы КРПД)
   ------------------------------------------------------------------------
   TSTPrint.open({ spec, docs, capture }) — предпросмотр документа учёта:
     миниатюры страниц, масштаб, параметры листа (принтер, копии, бумага,
     ориентация, поля, двусторонняя и ч/б печать, колонтитулы), шаблоны
     печати, журнал печати; экспорт в PDF (системное окно печати),
     Word (.docx) и Excel (.xlsx), отправка по e-mail.
     spec    — документ { key, title, subtitle, bodyHtml, footHtml, meta, file }
     docs    — список документов учёта для переключения [{ key, label, group }]
     capture — key → spec|null: построить другой документ, не открывая окно
   TSTHelp.open(sectionId) — справочные данные по модулю учёта (F1).
   Данные справка берёт у модуля учёта (TST.fleet.helpData), поэтому
   нормативы, пороги и примеры совпадают с текущими расчётами программы.
   ======================================================================== */
(function () {
'use strict';
const NS = 'spravochnik-tst-';
const $ = id => document.getElementById(id);
const RM = window.matchMedia ? window.matchMedia('(prefers-reduced-motion: reduce)') : null;

function E(tag, attrs, kids) {
  const n = document.createElement(tag);
  if (attrs) Object.keys(attrs).forEach(k => {
    const v = attrs[k];
    if (v == null || v === false) return;
    if (k === 'class') n.className = v;
    else if (k === 'text') n.textContent = v;
    else if (k === 'html') n.innerHTML = v;
    else if (k.slice(0, 2) === 'on') n.addEventListener(k.slice(2), v);
    else if (k === 'style') n.style.cssText = v;
    else n.setAttribute(k, v === true ? '' : v);
  });
  (kids || []).forEach(c => { if (c != null && c !== false) n.appendChild(typeof c === 'string' ?
document.createTextNode(c) : c); });
  return n;
}
function toast(title, text, type) {
  if (window.TST && typeof window.TST.toast === 'function') window.TST.toast(type || 'info', title, text || '');
}
function lsGet(k, d) { try { const v = localStorage.getItem(NS + k); return v ? JSON.parse(v) : d; } catch (e) {
return d; } }
function lsSet(k, v) { try { localStorage.setItem(NS + k, JSON.stringify(v)); return true; } catch (e) { return false;
 } }
const pad2 = n => (n < 10 ? '0' : '') + n;
function nowStr() { const d = new Date(); return pad2(d.getDate()) + '.' + pad2(d.getMonth() + 1) + '.' +
d.getFullYear() + ' ' + pad2(d.getHours()) + ':' + pad2(d.getMinutes()); }
function fmtIso(iso) { const d = new Date(iso); return isNaN(d) ? '' : pad2(d.getDate()) + '.' + pad2(d.getMonth() +
1) + '.' + d.getFullYear() + ' ' + pad2(d.getHours()) + ':' + pad2(d.getMinutes()); }
function fullscreen() {
  try {
    if (document.fullscreenElement) document.exitFullscreen();
    else if (document.documentElement.requestFullscreen) document.documentElement.requestFullscreen();
  } catch (e) { toast('Полный экран недоступен', 'Браузер не разрешил полноэкранный режим.'); }
}
/* Фон приложения недоступен для фокуса и чтения, пока открыто окно */
const BG_SEL = '.header, #overviewRoot, #catalogMain, #fleetRoot, #fleetNotifPanel';
let bgLocks = 0;
function lockBg(on) {
  bgLocks = Math.max(0, bgLocks + (on ? 1 : -1));
  const lock = bgLocks > 0;
  document.querySelectorAll(BG_SEL).forEach(e => {
    try { e.inert = lock; } catch (x) { /* старый браузер */ }
    if (lock) e.setAttribute('aria-hidden', 'true'); else e.removeAttribute('aria-hidden');
  });
}

/* ---------- печать через скрытую область #fleetPrint ---------- */
let printStyle = null;
function sendToPrinter(nodes, css) {
  const root = $('fleetPrint');
  if (!root) return false;
  if (!printStyle) { printStyle = E('style', { id: 'tst-print-page' }); document.head.appendChild(printStyle); }
  printStyle.textContent = css;
  root.replaceChildren.apply(root, nodes);
  document.documentElement.classList.add('printing');
  try { window.print(); }
  catch (e) {
    document.documentElement.classList.remove('printing');
    root.replaceChildren();
    toast('Печать недоступна', 'Браузер не поддерживает печать из программы.', 'warning');
    return false;
  }
  return true;
}
window.addEventListener('afterprint', () => document.documentElement.classList.remove('printing'));

/* ---------- журнал печати ---------- */
const ACT = { print: 'Печать', pdf: 'PDF', word: 'Word', excel: 'Excel', email: 'E-mail' };
function logPrint(name, action, detail) {
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
  const l = lsGet('printlog', []);
  l.unshift({ t: new Date().toISOString(), name, action, detail: detail || '' });
  lsSet('printlog', l.slice(0, 100));
}

/* ---------- нативные файлы DOCX / XLSX (ZIP без сжатия) ---------- */
const CRC_T = (() => { const t = []; for (let n = 0; n < 256; n++) { let c = n; for (let k = 0; k < 8; k++) c = c & 1 ? 0xEDB88320 ^ (c >>> 1) : c >>> 1; t[n] = c >>> 0; } return t; })();
function crc32(b) { let c = 0xFFFFFFFF; for (let i = 0; i < b.length; i++) c = CRC_T[(c ^ b[i]) & 255] ^ (c >>> 8); return (c ^ 0xFFFFFFFF) >>> 0; }
function zipStore(files) {
  const enc = new TextEncoder(), parts = [], central = [];
  let off = 0;
  const u16 = v => [v & 255, (v >>> 8) & 255];
  const u32 = v => [v & 255, (v >>> 8) & 255, (v >>> 16) & 255, (v >>> 24) & 255];
  files.forEach(f => {
    const name = enc.encode(f.name), data = enc.encode(f.data), crc = crc32(data);
    const lh = [].concat(u32(0x04034b50), u16(20), u16(0x0800), u16(0), u16(0), u16(0x21), u32(crc), u32(data.length), u32(data.length), u16(name.length), u16(0));
    parts.push(new Uint8Array(lh), name, data);
    central.push({ name, crc, size: data.length, off });
    off += lh.length + name.length + data.length;
  });
  const cdStart = off;
  let cdLen = 0;
  central.forEach(c => {
    const h = [].concat(u32(0x02014b50), u16(20), u16(20), u16(0x0800), u16(0), u16(0), u16(0x21), u32(c.crc), u32(c.size), u32(c.size), u16(c.name.length), u16(0), u16(0), u16(0), u16(0), u32(0), u32(c.off));
    parts.push(new Uint8Array(h), c.name); cdLen += h.length + c.name.length;
  });
  parts.push(new Uint8Array([].concat(u32(0x06054b50), u16(0), u16(0), u16(central.length), u16(central.length), u32(cdLen), u32(cdStart), u16(0))));
  return new Blob(parts, { type: 'application/zip' });
}
function downloadBlob(name, blob) {
  const a = E('a', { href: URL.createObjectURL(blob), download: name });
  document.body.appendChild(a); a.click();
  setTimeout(() => { URL.revokeObjectURL(a.href); a.remove(); }, 800);
}
const xe = t => String(t).replace(/[&<>"\u0000-\u0008\u000b\u000c\u000e-\u001f]/g, c => c === '&' ? '&amp;' : c === '<' ? '&lt;' : c === '>' ? '&gt;' : c === '"' ? '&quot;' : '');

/* DOCX из листа документа: абзацы, заголовки разделов, таблицы с объединениями */
function docxBlob(paper, g, px) {
  const tw = mm => Math.round(mm * 56.7);
  function para(text, o) {
    o = o || {};
    const jc = o.center ? '<w:jc w:val="center"/>' : o.right ? '<w:jc w:val="right"/>' : '';
    return '<w:p><w:pPr>' + jc + '<w:spacing w:before="' + (o.before || 0) + '" w:after="' + (o.after == null ? 60 : o.after) + '"/></w:pPr>' +
      (text === '' ? '' : '<w:r><w:rPr><w:rFonts w:ascii="Times New Roman" w:hAnsi="Times New Roman" w:cs="Times New Roman"/>' + (o.bold ? '<w:b/>' : '') + '<w:sz w:val="' + (o.sz || 21) + '"/></w:rPr><w:t xml:space="preserve">' + xe(text) + '</w:t></w:r>') + '</w:p>';
  }
  function table(t) {
    const rows = [].slice.call(t.rows), occ = [];
    let out = '', ncols = 0;
    rows.forEach(tr => { let n = 0; [].forEach.call(tr.cells, c => { n += c.colSpan; }); ncols = Math.max(ncols, n); });
    ncols = Math.max(ncols, 1);
    out += '<w:tbl><w:tblPr><w:tblW w:w="5000" w:type="pct"/><w:jc w:val="center"/><w:tblBorders>' +
      ['top', 'left', 'bottom', 'right', 'insideH', 'insideV'].map(b => '<w:' + b + ' w:val="single" w:sz="4" w:space="0" w:color="000000"/>').join('') +
      '</w:tblBorders><w:tblLayout w:type="autofit"/><w:tblCellMar><w:left w:w="60" w:type="dxa"/><w:right w:w="60" w:type="dxa"/></w:tblCellMar></w:tblPr><w:tblGrid>' +
      new Array(ncols + 1).join('<w:gridCol w:w="' + Math.floor(9000 / ncols) + '"/>') + '</w:tblGrid>';
    rows.forEach((tr, ri) => {
      occ[ri] = occ[ri] || [];
      let col = 0;
      out += '<w:tr>' + (tr.parentNode && tr.parentNode.tagName === 'THEAD' ? '' : '');
      const fillCont = () => { while (occ[ri][col]) { out += '<w:tc><w:tcPr><w:vMerge/></w:tcPr><w:p/></w:tc>'; col += occ[ri][col].span; } };
      [].forEach.call(tr.cells, c => {
        fillCont();
        const cs = c.colSpan, rs = c.rowSpan;
        if (rs > 1) for (let k = 1; k < rs; k++) { occ[ri + k] = occ[ri + k] || []; occ[ri + k][col] = { span: cs }; }
        const isH = c.tagName === 'TH', right = c.classList.contains('p-num');
        out += '<w:tc><w:tcPr>' + (cs > 1 ? '<w:gridSpan w:val="' + cs + '"/>' : '') + (rs > 1 ? '<w:vMerge w:val="restart"/>' : '') +
          (isH ? '<w:shd w:val="clear" w:color="auto" w:fill="ECECEC"/>' : '') + '<w:vAlign w:val="center"/></w:tcPr>' +
          para(c.textContent.trim(), { right, center: isH && !right, bold: isH || !!c.querySelector('b'), sz: 19, after: 0 }) + '</w:tc>';
        col += cs;
      });
      fillCont();
      out += '</w:tr>';
    });
    return out + '</w:tbl>' + para('', { after: 80 });
  }
```

```
      function walk(n) {
        let out = '';
        [].forEach.call(n.children, c => {
          const tag = c.tagName, cl = c.classList;
          if (tag === 'TABLE') out += table(c);
          else if (cl.contains('pfoot')) { [].forEach.call(c.children, s => { out += para(s.textContent.trim(), { before:
    160 }); }); if (!c.children.length) out += para(c.textContent.trim(), { before: 160 }); }
          else if (tag === 'DIV' && c.querySelector('p,table,h3,div')) out += walk(c);
          else if (/^H[1-6]$/.test(tag)) out += para(c.textContent.trim(), { bold: true, sz: 22, before: 160 });
          else if (tag === 'P' || tag === 'DIV') {
            out += para(c.textContent.trim(), {
              center: cl.contains('pt') || cl.contains('pt-sub') || cl.contains('porg'),
              bold: cl.contains('pt'),
              sz: cl.contains('pt') ? 30 : cl.contains('pmeta') ? 18 : 21
            });
          }
        });
        return out;
      }
      const body = walk(paper);
      const land = g.orient === 'landscape';
      const sect = '<w:sectPr><w:pgSz w:w="' + tw(land ? g.h : g.w) + '" w:h="' + tw(land ? g.w : g.h) + '"' + (land ? '
    w:orient="landscape"' : '') +
          '/><w:pgMar w:top="' + tw(px.mt) + '" w:right="' + tw(px.mr) + '" w:bottom="' + tw(px.mb) + '" w:left="' +
    tw(px.ml) + '" w:header="708" w:footer="708" w:gutter="0"/></w:sectPr>';
      const W = 'xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main"';
      return zipStore([
        { name: '[Content_Types].xml', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Types
    xmlns="http://schemas.openxmlformats.org/package/2006/content-types"><Default Extension="rels"
    ContentType="application/vnd.openxmlformats-package.relationships+xml"/><Default Extension="xml"
    ContentType="application/xml"/><Override PartName="/word/document.xml" ContentType="application/vnd.openxmlformats-
    officedocument.wordprocessingml.document.main+xml"/></Types>' },
        { name: '_rels/.rels', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Relationships
    xmlns="http://schemas.openxmlformats.org/package/2006/relationships"><Relationship Id="rId1"
    Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument"
    Target="word/document.xml"/></Relationships>' },
        { name: 'word/document.xml', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><w:document ' + W +
    '><w:body>' + body + sect + '</w:body></w:document>' }
      ]);
    }
    /* XLSX: blocks = [{ title, rows: [[{ t, rs, cs, h }]] }] */
    function xlsxBlob(blocks, sheetName) {
      function colName(n) { let s = ''; n++; while (n > 0) { const m = (n - 1) % 26; s = String.fromCharCode(65 + m) + s;
    n = Math.floor((n - 1) / 26); } return s; }
      let rowsXml = '', r = 0, maxc = 1;
      const merges = [];
      function row(cells, bold) {
        r++;
        let x = '<row r="' + r + '">';
        cells.forEach(c => {
          if (c.t !== '' && c.t != null) x += '<c r="' + colName(c.c) + r + '" t="inlineStr"' + (bold || c.h ? ' s="1"' :
    (c.plain ? '' : ' s="2"')) + '><is><t xml:space="preserve">' + xe(c.t) + '</t></is></c>';
          if (c.cs > 1 || c.rs > 1) merges.push(colName(c.c) + r + ':' + colName(c.c + (c.cs || 1) - 1) + (r + (c.rs || 1)
     - 1));
          maxc = Math.max(maxc, c.c + (c.cs || 1));
        });
        rowsXml += x + '</row>';
      }
      blocks.forEach(b => {
        if (b.title) row([{ c: 0, t: b.title, cs: 1, plain: !b.bold }], !!b.bold);
        const occ = {};
        b.rows.forEach(tr => {
          let col = 0;
          const cells = [], rowIdx = r + 1;
          tr.forEach(c => {
            while (occ[rowIdx + ',' + col]) col++;
            cells.push({ c: col, t: c.t, cs: c.cs || 1, rs: c.rs || 1, h: c.h });
            if ((c.rs || 1) > 1) for (let k = 1; k < c.rs; k++) for (let j = 0; j < (c.cs || 1); j++) occ[(rowIdx + k) +
    ',' + (col + j)] = 1;
            col += c.cs || 1;
          });
          row(cells);
        });
        if (b.rows.length) row([]);
      });
      const cols = '<cols><col min="1" max="' + maxc + '" width="24" customWidth="1"/></cols>';
      const S = 'xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main"';
      const sheet = xe(String(sheetName || 'Документ').replace(/[\\/?*[\]:]/g, ' ').slice(0, 31));
      return zipStore([
        { name: '[Content_Types].xml', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Types
    xmlns="http://schemas.openxmlformats.org/package/2006/content-types"><Default Extension="rels"
    ContentType="application/vnd.openxmlformats-package.relationships+xml"/><Default Extension="xml"
    ContentType="application/xml"/><Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-
    officedocument.spreadsheetml.sheet.main+xml"/><Override PartName="/xl/worksheets/sheet1.xml"
    ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/><Override
    PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-
    officedocument.spreadsheetml.styles+xml"/></Types>' },
        { name: '_rels/.rels', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Relationships
    xmlns="http://schemas.openxmlformats.org/package/2006/relationships"><Relationship Id="rId1"
    Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument"
```

```
Target="xl/workbook.xml"/></Relationships>' },
    { name: 'xl/workbook.xml', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><workbook ' + S + '
xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships"><sheets><sheet name="' + sheet + '"
sheetId="1" r:id="rId1"/></sheets></workbook>' },
    { name: 'xl/_rels/workbook.xml.rels', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><Relationships
 xmlns="http://schemas.openxmlformats.org/package/2006/relationships"><Relationship Id="rId1"
Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/worksheet"
Target="worksheets/sheet1.xml"/><Relationship Id="rId2"
Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/styles"
Target="styles.xml"/></Relationships>' },
    { name: 'xl/styles.xml', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><styleSheet ' + S +
'><fonts count="2"><font><sz val="11"/><name val="Times New Roman"/></font><font><b/><sz val="11"/><name val="Times
New Roman"/></font></fonts><fills count="2"><fill><patternFill patternType="none"/></fill><fill><patternFill
patternType="gray125"/></fill></fills><borders
count="2"><border><left/><right/><top/><bottom/><diagonal/></border><border><left style="thin"/><right
style="thin"/><top style="thin"/><bottom style="thin"/><diagonal/></border></borders><cellStyleXfs count="1"><xf
numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs count="3"><xf numFmtId="0" fontId="0"
fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1" fillId="0" borderId="1" xfId="0" applyFont="1"
applyBorder="1" applyAlignment="1"><alignment horizontal="center" vertical="center" wrapText="1"/></xf><xf
numFmtId="0" fontId="0" fillId="0" borderId="1" xfId="0" applyBorder="1" applyAlignment="1"><alignment
vertical="center" wrapText="1"/></xf></cellXfs></styleSheet>' },
    { name: 'xl/worksheets/sheet1.xml', data: '<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet ' + S
 + '>' + cols + '<sheetData>' + rowsXml + '</sheetData>' + (merges.length ? '<mergeCells count="' + merges.length +
'">' + merges.map(m => '<mergeCell ref="' + m + '"/>').join('') + '</mergeCells>' : '') + '</worksheet>' }
  ]);
}
function blocksFromPaper(paper) {
  const out = [];
  [].forEach.call(paper.querySelectorAll('.porg,.pt,.pt-sub,.pmeta,.p-h,table,.pbody > p,.pfoot > span'), n => {
    if (n.tagName === 'TABLE') out.push({ rows: [].map.call(n.rows, tr => [].map.call(tr.cells, c => ({ t:
c.textContent.trim(), rs: c.rowSpan, cs: c.colSpan, h: c.tagName === 'TH' }))) });
    else out.push({ title: n.textContent.trim(), bold: n.classList.contains('pt') || n.classList.contains('p-h'),
rows: [] });
  });
  return out;
}

/* ---------- лист документа ---------- */
function paperFromSpec(spec, d) {
  const p = E('div', { class: 'paper tpaper' });
  if (d.org) p.appendChild(E('p', { class: 'porg', text: d.org }));
  p.appendChild(E('p', { class: 'pt', text: spec.title }));
  if (spec.subtitle) p.appendChild(E('p', { class: 'pt-sub', text: spec.subtitle }));
  p.appendChild(E('p', { class: 'pmeta', text: 'Дата формирования: ' + nowStr() + (spec.meta ? ' · ' + spec.meta : '')
 }));
  p.appendChild(E('div', { class: 'pbody', html: spec.bodyHtml || '' }));
  const foot = E('div', { class: 'pfoot' });
  if (spec.footHtml) foot.innerHTML = spec.footHtml;
  else {
    const who = [d.role, '_____________________', d.sign].filter(Boolean).join(' ');
    foot.appendChild(E('span', { text: 'Исполнитель: ' + who }));
    foot.appendChild(E('span', { text: '«____» _____________ 20____ г.' }));
  }
  p.appendChild(foot);
  return p;
}

/* ---------- универсальное окно (справка) ---------- */
const stack = [];
function xwin(o) {
  const last = document.activeElement;
  const ov = E('div', { class: 'xov' });
  const head = E('div', { class: 'xw-head' }, [E('h2', { text: o.title }), E('button', { type: 'button', class:
'xw-x', 'aria-label': 'Закрыть', title: 'Закрыть (Esc)', text: '×', onclick: () => api.close() })]);
  const body = E('div', { class: 'xw-body' }, o.body ? [o.body] : []);
  const win = E('div', { class: 'xw ' + (o.cls || ''), role: 'dialog', 'aria-modal': 'true', 'aria-label': o.title },
[head, o.bar || null, body]);
  ov.appendChild(win);
  ov.addEventListener('mousedown', e => { if (e.target === ov) api.close(); });
  document.body.appendChild(ov);
  lockBg(true);
  const api = {
    el: win, body,
    close() {
      const i = stack.indexOf(api);
      if (i < 0) return;
      stack.splice(i, 1); ov.remove(); lockBg(false);
      if (o.onClose) o.onClose();
      if (last && last.focus) try { last.focus(); } catch (e) { /* элемент удалён */ }
    }
  };
  stack.push(api);
  return api;
}

/* ========================================================================
   ОКНО ПРЕДПРОСМОТРА И ПЕЧАТИ
   ======================================================================== */
let PW = null;
const PAPERS = { A4: [210, 297, 'A4'], A3: [297, 420, 'A3'], A5: [148, 210, 'A5'], Letter: [215.9, 279.4, 'letter'],
```

```
    Legal: [215.9, 355.6, 'legal'] };
291 const MM = 96 / 25.4;
292 function defaultPx() { return { printer: 'system', copies: 1, orient: 'landscape', paper: 'A4', mt: 12, mr: 12, mb:
    12, ml: 15, duplex: false, color: 'color', header: true, pageno: true, nup: 1, zoomMode: 'fitw', presets: [] }; }
293
294 function openPrintWin(opts) {
295   opts = opts || {};
296   if (PW) { PW.apply(opts); return; }
297   const docs = opts.docs || [];
298   const capture = opts.capture || (() => null);
299   const px = Object.assign(defaultPx(), lsGet('print-settings', {}));
300   const pd = Object.assign({ org: '', role: '', sign: '' }, lsGet('print-doc', {}));
301   let spec = opts.spec || null, extra = null;
302   let pages = 1, curPage = 1, zoom = 1, paperEl = null;
303   const ov = E('div', { class: 'pw', role: 'dialog', 'aria-label': 'Предпросмотр и печать', 'aria-modal': 'true' });
304   const opener = document.activeElement;
305   const w0 = Math.min(1440, window.innerWidth - 32), h0 = Math.min(900, window.innerHeight - 32);
306   ov.style.cssText = 'left:' + Math.max(8, (window.innerWidth - w0) / 2) + 'px;top:' + Math.max(8, (window.innerHeight
     - h0) / 2) + 'px;width:' + w0 + 'px;height:' + h0 + 'px';
307   const R = {};
308   function saveP() { lsSet('print-settings', Object.assign({}, px)); }
309   function saveD() { lsSet('print-doc', { org: R.org.value.trim(), role: R.role.value.trim(), sign:
    R.sign.value.trim() }); }
310
311   /* --- Документ --- */
312   const docSel = E('select', { id: 'pw-doc' });
313   function fillDocs() {
314     docSel.replaceChildren();
315     if (spec && !docs.some(d => d.key === spec.key)) docSel.appendChild(E('option', { value: '__current', text:
    'Текущий: ' + spec.title.replace(/\s+/g, ' ').slice(0, 70) }));
316     const groups = [];
317     docs.forEach(d => { if (groups.indexOf(d.group) < 0) groups.push(d.group); });
318     groups.forEach(g => docSel.appendChild(E('optgroup', { label: g }, docs.filter(d => d.group === g).map(d =>
    E('option', { value: d.key, text: d.label })))));
319     docSel.value = spec && docs.some(d => d.key === spec.key) ? spec.key : '__current';
320   }
321   docSel.addEventListener('change', () => {
322     if (docSel.value === '__current') { spec = extra; refresh(); return; }
323     const s = capture(docSel.value);
324     if (s) { spec = s; refresh(); }
325     else { spec = null; paperEl = null; pagesBox.replaceChildren(E('p', { class: 'xempty', text: 'Для этого документа
    нет данных. Заполните учёт или выберите другой документ.' })); thumbs.replaceChildren(); pages = 1; updNav();
    statusEl.textContent = ''; }
326   });
327   function fld(label, key, extraAttrs) {
328     const inp = E('input', Object.assign({ type: 'text', id: 'pw-' + key, value: pd[key] || '', maxlength: '200' },
    extraAttrs || {}));
329     R[key] = inp;
330     inp.addEventListener('input', () => { saveD(); refresh(); });
331     return E('label', { class: 'xfield' }, [E('span', { text: label }), inp]);
332   }
333   const docFs = E('fieldset', null, [E('legend', { text: 'Документ' }),
334     E('label', { class: 'xfield' }, [E('span', { text: 'Печатная форма' }), docSel]),
335     fld('Организация (воинская часть, подразделение)', 'org', { placeholder: 'Печатается над заголовком и в
    колонтитуле' }),
336     fld('Должность исполнителя', 'role'),
337     fld('Звание и фамилия исполнителя', 'sign'),
338     E('p', { class: 'xnote', text: 'Должность и фамилия подставляются в строку «Исполнитель» документов, где она
    предусмотрена. Значения запоминаются.' })]);
339
340   /* --- Параметры печати --- */
341   function chk(label, key) {
342     const c = E('input', { type: 'checkbox', id: 'pw-' + key });
343     c.checked = !!px[key]; R[key] = c;
344     c.addEventListener('change', () => { px[key] = c.checked; saveP(); rebuild(); });
345     return E('label', { class: 'chk' }, [c, label]);
346   }
347   const paperSel = E('select', { id: 'pw-paper' }, Object.keys(PAPERS).map(k => E('option', { value: k, text: k ===
    'Letter' ? 'Letter (216×279 мм)' : k === 'Legal' ? 'Legal (216×356 мм)' : k + ' (' + PAPERS[k][0] + '×' + PAPERS[k][1]
     + ' мм)' })));
348   paperSel.value = px.paper;
349   paperSel.addEventListener('change', () => { px.paper = paperSel.value; saveP(); rebuild(); });
350   const printerSel = E('select', { id: 'pw-printer' }, [E('option', { value: 'system', text: 'Принтер — выбор в
    системном окне печати' }), E('option', { value: 'pdf', text: 'Сохранить как PDF (в системном окне)' })]);
351   printerSel.value = px.printer;
352   printerSel.addEventListener('change', () => { px.printer = printerSel.value; saveP(); });
353   const copies = E('input', { type: 'number', min: '1', max: '99', value: String(px.copies), id: 'pw-copies', class:
    'xin', 'aria-label': 'Количество копий', style: 'width:56px;text-align:center' });
354   function setCopies(n) { n = Math.max(1, Math.min(99, n || 1)); copies.value = String(n); px.copies = n; saveP(); }
355   copies.addEventListener('change', () => setCopies(parseInt(copies.value, 10)));
356   const oriP = E('button', { type: 'button', id: 'pw-op', onclick: () => { px.orient = 'portrait'; syncOri(); saveP();
     rebuild(); } }, [E('span', { class: 'ori p' }), 'Книжная']);
357   const oriL = E('button', { type: 'button', id: 'pw-ol', onclick: () => { px.orient = 'landscape'; syncOri();
    saveP(); rebuild(); } }, [E('span', { class: 'ori l' }), 'Альбомная']);
358   function syncOri() { oriP.classList.toggle('on', px.orient === 'portrait'); oriL.classList.toggle('on', px.orient
    === 'landscape'); oriP.setAttribute('aria-pressed', String(px.orient === 'portrait')); oriL.setAttribute('aria-
    pressed', String(px.orient === 'landscape')); }
359   function marg(label, key) {
360     const i = E('input', { type: 'number', min: '0', max: '60', step: '1', value: String(px[key]), class: 'xin',
    'aria-label': label });
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
      i.addEventListener('change', () => { let v = parseFloat(i.value); if (!(v >= 0 && v <= 60)) v = 12; i.value = String(v); px[key] = v; saveP(); rebuild(); });
      return E('label', { class: 'xfield' }, [E('span', { text: label }), i]);
    }
    const colorBtns = ['color', 'bw'].map(v => { const b = E('button', { type: 'button', text: v === 'color' ? 'Цветная' : 'Чёрно-белая', onclick: () => { px.color = v; syncColor(); saveP(); rebuild(); } }); b.setAttribute('data-v', v); return b; });
    function syncColor() { colorBtns.forEach(b => { const on = b.getAttribute('data-v') === px.color; b.classList.toggle('on', on); b.setAttribute('aria-pressed', String(on)); }); }
    const marginInputs = [marg('Верх, мм', 'mt'), marg('Право, мм', 'mr'), marg('Низ, мм', 'mb'), marg('Лево, мм', 'ml')];
    const nupSel = E('select', { id: 'pw-nup' }, [['1', '1 страница на листе'], ['2', '2 страницы на листе'], ['4', '4 страницы на листе']].map(o => E('option', { value: o[0], text: o[1] })));
    nupSel.value = String(px.nup);
    nupSel.addEventListener('change', () => { px.nup = parseInt(nupSel.value, 10); saveP(); layoutPages(); fitIfMode(); });
    const prFs = E('fieldset', { id: 'pw-settings' }, [E('legend', { text: 'Параметры печати' }),
      E('label', { class: 'xfield' }, [E('span', { text: 'Принтер' }), printerSel]),
      E('p', { class: 'xnote', text: 'Браузер не раскрывает список принтеров: конкретный принтер выбирается в системном окне, которое откроется при печати.' }),
      E('div', { class: 'row' }, [E('span', { text: 'Копии' }), E('span', { class: 'cnt' }, [E('button', { type: 'button', 'aria-label': 'Меньше копий', text: '−', onclick: () => setCopies(px.copies - 1) }), copies, E('button', { type: 'button', 'aria-label': 'Больше копий', text: '+', onclick: () => setCopies(px.copies + 1) })])]),
      E('div', { class: 'row' }, [E('span', { class: 'seg2', role: 'group', 'aria-label': 'Ориентация' }, [oriP, oriL])]),
      E('label', { class: 'xfield' }, [E('span', { text: 'Размер бумаги' }), paperSel]),
      E('div', { class: 'xgrid2' }, marginInputs),
      chk('Двусторонняя печать (каждая копия с нового листа)', 'duplex'),
      E('div', { class: 'row' }, [E('span', { class: 'seg2', role: 'group', 'aria-label': 'Цветность' }, colorBtns)]),
      chk('Колонтитул (организация, документ, дата)', 'header'), chk('Номера страниц', 'pageno'),
      E('label', { class: 'xfield' }, [E('span', { text: 'Несколько страниц на листе (предпросмотр)' }), nupSel])]);

    /* --- Шаблоны печати --- */
    const presetSel = E('select', { id: 'pw-presets', 'aria-label': 'Шаблоны печати' });
    const presetName = E('input', { type: 'text', class: 'xin', placeholder: 'Название шаблона печати', maxlength: '60', 'aria-label': 'Название шаблона печати' });
    function fillPresets() {
      presetSel.replaceChildren(E('option', { value: '', text: 'Шаблоны печати…' }));
      (px.presets || []).forEach((p, i) => presetSel.appendChild(E('option', { value: String(i), text: p.name })));
    }
    presetSel.addEventListener('change', () => {
      const p = (px.presets || [])[parseInt(presetSel.value, 10)];
      if (!p) return;
      Object.assign(px, p.s); setUiFromPx(); saveP(); rebuild();
      toast('Шаблон применён', p.name, 'success');
    });
    function savePreset() {
      const n = presetName.value.trim();
      if (!n) { toast('Введите название шаблона', ''); presetName.focus(); return; }
      const s = {};
      ['orient', 'paper', 'mt', 'mr', 'mb', 'ml', 'duplex', 'color', 'header', 'pageno', 'copies', 'nup'].forEach(k => { s[k] = px[k]; });
      px.presets = (px.presets || []).filter(p => p.name !== n).concat([{ name: n, s }]);
      saveP(); fillPresets(); presetName.value = '';
      toast('Шаблон сохранён', '«' + n + '»', 'success');
    }
    function setUiFromPx() {
      paperSel.value = px.paper; copies.value = String(px.copies); syncOri(); syncColor();
      ['duplex', 'header', 'pageno'].forEach(k => { R[k].checked = !!px[k]; });
      nupSel.value = String(px.nup);
      marginInputs.forEach((m, i) => { m.querySelector('input').value = String(px[['mt', 'mr', 'mb', 'ml'][i]]); });
    }
    const tplFs = E('fieldset', null, [E('legend', { text: 'Шаблоны печати' }), presetSel, E('div', { class: 'row' }, [presetName, E('button', { type: 'button', class: 'xbtn sm', text: 'Сохранить как шаблон', onclick: savePreset })])]);

    /* --- Журнал печати --- */
    const logList = E('ul', { class: 'pw-log' });
    function drawLog() {
      const l = lsGet('printlog', []).slice(0, 8);
      logList.replaceChildren.apply(logList, l.length ? l.map(x => E('li', null, [E('span', { text: (ACT[x.action] || x.action) + ' · ' + x.name }), E('small', { text: fmtIso(x.t) })])) : [E('li', { class: 'xnote', text: 'Документы ещё не печатались.' })]);
    }
    const logFs = E('fieldset', null, [E('legend', { text: 'Журнал печати' }), logList]);

    const side = E('div', { class: 'pw-side' }, [docFs, prFs, tplFs, logFs]);
    const thumbs = E('div', { class: 'pw-thumbs', 'aria-label': 'Миниатюры страниц' });
    const pagesBox = E('div', { class: 'pw-pages' });
    const scroll = E('div', { class: 'pw-scroll', tabindex: '0', 'aria-label': 'Предварительный просмотр' }, [pagesBox]);
    /* панель масштаба и страниц */
    const slider = E('input', { type: 'range', min: '25', max: '300', value: '100', 'aria-label': 'Масштаб', id: 'pw-zoom' });
    const zlabel = E('span', { text: '100%', style: 'width:42px;text-align:right' });
    const zBtns = [50, 75, 100, 150, 200].map(z => E('button', { type: 'button', text: z + '%', 'data-z': String(z), onclick: () => { px.zoomMode = 'fixed'; setZoom(z / 100); } }));
    const fitBtns = [['fitw', 'По ширине'], ['fith', 'По высоте'], ['fit', 'Вместить']].map(f => E('button', { type: 'button', text: f[1], 'data-m': f[0], onclick: () => { px.zoomMode = f[0]; fitIfMode(); saveP(); } }));
    const pgIn = E('input', { type: 'number', class: 'pg', min: '1', value: '1', 'aria-label': 'Номер страницы' });
    const pgInfo = E('span', { text: 'Страница 1 из 1' });
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
431       const navBtns = [['⏮', 'Первая страница', () => goPage(1)], ['◀', 'Предыдущая страница', () => goPage(curPage - 1)],
     ['▶', 'Следующая страница', () => goPage(curPage + 1)], ['⏭', 'Последняя страница', () => goPage(pages)]]
432         .map(b => E('button', { type: 'button', 'aria-label': b[1], title: b[1], text: b[0], onclick: b[2] }));
433       pgIn.addEventListener('change', () => goPage(parseInt(pgIn.value, 10)));
434       const bar = E('div', { class: 'pw-bar' }, [E('div', { class: 'grp' }, zBtns.concat([slider, zlabel])), E('div', {
    class: 'grp' }, fitBtns), E('div', { class: 'grp' }, [navBtns[0], navBtns[1], pgIn, navBtns[2], navBtns[3],
    pgInfo])]);
435       const statusEl = E('span', { class: 'pw-status', id: 'pw-status', 'aria-live': 'polite' });
436       const xbtn = (label, fn) => E('button', { type: 'button', class: 'xbtn', text: label, onclick: fn });
437       const foot = E('div', { class: 'pw-foot' }, [xbtn('Экспорт в PDF', () => doPrint('pdf')), xbtn('Экспорт в Word
    (.docx)', () => doExport('word')), xbtn('Экспорт в Excel (.xlsx)', () => doExport('excel')), xbtn('Отправить по
    e-mail', doMail),
438         statusEl, E('span', { class: 'sp' }), E('button', { type: 'button', class: 'xbtn', text: 'Закрыть', onclick: () =>
     close() }), E('button', { type: 'button', class: 'xbtn fill', id: 'pw-print', text: 'Печать', onclick: () =>
    doPrint('print') })]);
439       /* заголовок окна */
440       const maxBtn = E('button', { type: 'button', title: 'Развернуть / восстановить', 'aria-label': 'Развернуть', text:
    '▢', onclick: () => toggleMax() });
441       const fsBtn = E('button', { type: 'button', title: 'Полный экран (F11)', 'aria-label': 'Полный экран', text: '⛶',
    onclick: fullscreen });
442       const titleH = E('h2', { text: 'Предпросмотр и печать — учёт технических средств тыла' });
443       const title = E('div', { class: 'pw-title' }, [titleH, fsBtn, maxBtn, E('button', { type: 'button', 'aria-label':
    'Закрыть', title: 'Закрыть (Esc)', text: '×', onclick: () => close() })]);
444       ov.appendChild(title); ov.appendChild(bar); ov.appendChild(E('div', { class: 'pw-main' }, [thumbs, scroll, side]));
    ov.appendChild(foot);
445       document.body.appendChild(ov);
446       function toggleMax() { ov.classList.toggle('max'); setTimeout(fitIfMode, 60); }
447       /* перетаскивание за заголовок */
448       title.addEventListener('pointerdown', e => {
449         if (e.target.closest('button') || ov.classList.contains('max') || window.innerWidth < 900) return;
450         const r = ov.getBoundingClientRect(), dx = e.clientX - r.left, dy = e.clientY - r.top;
451         title.setPointerCapture(e.pointerId);
452         const mv = ev => { ov.style.left = Math.min(Math.max(-r.width + 120, ev.clientX - dx), window.innerWidth - 120) +
    'px'; ov.style.top = Math.min(Math.max(0, ev.clientY - dy), window.innerHeight - 40) + 'px'; };
453         const up = () => { title.removeEventListener('pointermove', mv); title.removeEventListener('pointerup', up); };
454         title.addEventListener('pointermove', mv); title.addEventListener('pointerup', up);
455       });
456       title.addEventListener('dblclick', e => { if (!e.target.closest('button')) toggleMax(); });
457       const ro = window.ResizeObserver ? new ResizeObserver(() => fitIfMode()) : null;
458       if (ro) ro.observe(scroll);
459
460       /* геометрия листа */
461       function geom() {
462         const p = PAPERS[px.paper] || PAPERS.A4, w = px.orient === 'landscape' ? p[1] : p[0], h = px.orient ===
    'landscape' ? p[0] : p[1];
463         return { w, h, wp: w * MM, hp: h * MM, pw: (w - px.ml - px.mr) * MM, ph: (h - px.mt - px.mb) * MM, css: p[2] + ' '
     + px.orient };
464       }
465       function readDoc() { return { org: R.org.value.trim(), role: R.role.value.trim(), sign: R.sign.value.trim() }; }
466       function docName() { return spec ? spec.title.replace(/\s+/g, ' ') + (spec.subtitle && spec.title.length < 12 ? ' '
    + spec.subtitle : '') : ''; }
467       function refresh() {
468         if (!spec) return;
469         titleH.textContent = 'Предпросмотр и печать — ' + docName().slice(0, 90);
470         paperEl = paperFromSpec(spec, readDoc());
471         rebuild();
472       }
473       function headerText() { return (R.org.value.trim() || 'Регламент-ТС') + ' · ' + (spec ? spec.title.replace(/\s+/g, '
     ') : '') + ' · ' + nowStr().slice(0, 10); }
474       function rebuild() {
475         if (!paperEl) return;
476         const g = geom();
477         const meas = E('div', { class: 'pv pv-meas', style: 'width:' + g.pw + 'px' }, [paperEl.cloneNode(true)]);
478         document.body.appendChild(meas);
479         const H = meas.scrollHeight;
480         meas.remove();
481         pages = Math.max(1, Math.ceil(H / g.ph - 0.02));
482         pagesBox.replaceChildren(); thumbs.replaceChildren();
483         for (let i = 0; i < pages; i++) {
484           const inner = E('div', { class: 'pv-in pv', style: 'left:' + px.ml * MM + 'px;top:' + px.mt * MM + 'px;width:' +
     g.pw + 'px;height:' + g.ph + 'px' }, [E('div', { style: 'width:' + g.pw + 'px;transform:translateY(' + (-i * g.ph) +
    'px)' }, [paperEl.cloneNode(true)])]);
485           const sheet = E('div', { class: 'pv-sheet' + (px.color === 'bw' ? ' bw' : ''), 'data-p': String(i + 1), style:
    'width:' + g.wp + 'px;height:' + g.hp + 'px' }, [inner]);
486           if (px.header) sheet.appendChild(E('div', { class: 'pv-hd', style: 'top:' + Math.max(2, px.mt * MM / 2 - 7) +
    'px', text: headerText() }));
487           if (px.pageno) sheet.appendChild(E('div', { class: 'pv-ft', style: 'bottom:' + Math.max(2, px.mb * MM / 2 - 7) +
     'px', text: 'Страница ' + (i + 1) + ' из ' + pages }));
488           pagesBox.appendChild(sheet);
489           const sc = 100 / g.wp;
490           const tb = E('div', { class: 'tb', style: 'width:100px;height:' + g.hp * sc + 'px' }, [E('div', { style:
    'width:' + g.wp + 'px;height:' + g.hp + 'px;transform:scale(' + sc + ')' }, [sheet.cloneNode(true)])]);
491           const n = i + 1;
492           thumbs.appendChild(E('button', { type: 'button', class: 'pw-thumb', 'data-p': String(n), 'aria-label': 'Страница
     ' + n, onclick: () => goPage(n) }, [tb, E('span', { text: String(n) })]));
493         }
494         layoutPages(); curPage = Math.min(curPage, pages); updNav(); fitIfMode();
495         statusEl.textContent = pages + ' стр. · разбивка приблизительная' + (px.orient === 'portrait' ? ' · широкие
    таблицы удобнее печатать на альбомном листе' : '');
496       }
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
  function layoutPages() { const cols = px.nup === 1 ? 1 : 2; pagesBox.style.gridTemplateColumns = 'repeat(' + cols + ',auto)'; }
  function setZoom(z) {
    zoom = Math.max(0.25, Math.min(3, z));
    pagesBox.style.zoom = String(zoom);
    slider.value = String(Math.round(zoom * 100)); zlabel.textContent = Math.round(zoom * 100) + '%';
    zBtns.forEach(b => b.classList.toggle('on', Math.round(zoom * 100) === parseInt(b.getAttribute('data-z'), 10)));
    fitBtns.forEach(b => b.classList.toggle('on', b.getAttribute('data-m') === px.zoomMode));
  }
  function fitIfMode() {
    if (!paperEl) return;
    const g = geom(), cols = px.nup === 1 ? 1 : 2, rows = Math.ceil(Math.min(pages, px.nup === 1 ? 1 : 4) / cols) || 1;
    const aw = scroll.clientWidth - 48, ah = scroll.clientHeight - 48;
    const tw = cols * g.wp + (cols - 1) * 22, th = rows * g.hp + (rows - 1) * 22;
    if (aw <= 0 || ah <= 0) return;
    if (px.zoomMode === 'fitw') setZoom(aw / tw); else if (px.zoomMode === 'fith') setZoom(ah / th); else if (px.zoomMode === 'fit') setZoom(Math.min(aw / tw, ah / th)); else setZoom(zoom);
  }
  slider.addEventListener('input', () => { px.zoomMode = 'fixed'; setZoom(parseInt(slider.value, 10) / 100); });
  function updNav() {
    pgIn.max = String(pages); pgIn.value = String(curPage); pgInfo.textContent = 'Страница ' + curPage + ' из ' + pages;
    navBtns[0].disabled = navBtns[1].disabled = curPage <= 1; navBtns[2].disabled = navBtns[3].disabled = curPage >= pages;
    thumbs.querySelectorAll('.pw-thumb').forEach(t => t.classList.toggle('cur', t.getAttribute('data-p') === String(curPage)));
  }
  function goPage(n) {
    if (!(n >= 1)) n = 1;
    n = Math.min(n, pages); curPage = n; updNav();
    const s = pagesBox.querySelector('.pv-sheet[data-p="' + n + '"]');
    if (s) s.scrollIntoView({ block: 'start', behavior: RM && RM.matches ? 'auto' : 'smooth' });
  }
  scroll.addEventListener('scroll', () => {
    const top = scroll.getBoundingClientRect().top + 40;
    let best = 1;
    pagesBox.querySelectorAll('.pv-sheet').forEach(s => { if (s.getBoundingClientRect().top <= top) best = parseInt(s.getAttribute('data-p'), 10); });
    if (best !== curPage) { curPage = best; updNav(); }
  });

  /* печать и экспорт */
  function ready() {
    if (!spec || !paperEl) { toast('Нет документа', 'Выберите документ, для которого есть данные.', 'warning'); return false; }
    saveD();
    return true;
  }
  function pageCss(g) {
    let css = '@page{size:' + g.css + ';margin:' + px.mt + 'mm ' + px.mr + 'mm ' + px.mb + 'mm ' + px.ml + 'mm;';
    if (px.header) css += '@top-center{content:' + JSON.stringify(headerText()) + ';font:9pt "Times New Roman",serif;color:#444}';
    if (px.pageno) css += '@bottom-center{content:"Страница " counter(page) " из " counter(pages);font:9pt "Times New Roman",serif;color:#444}';
    css += '}';
    css += '@media print{#fleetPrint{' + (px.color === 'bw' ? 'filter:grayscale(1);' : '') + '}#fleetPrint .pw-copy{' + (px.duplex ? 'break-before:right;' : '') + '}#fleetPrint .pw-copy:first-child{break-before:auto}}';
    return css;
  }
  function doPrint(mode) {
    if (!ready()) return;
    const g = geom(), nodes = [];
    for (let i = 0; i < px.copies; i++) nodes.push(E('div', { class: 'pw-copy' }, [paperFromSpec(spec, readDoc())]));
    logPrint(docName(), mode === 'pdf' ? 'pdf' : 'print', px.paper + ' ' + (px.orient === 'landscape' ? 'альбомная' : 'книжная') + ', копий: ' + px.copies);
    drawLog();
    if (mode === 'pdf' || px.printer === 'pdf') toast('Сохранение в PDF', 'В окне печати выберите «Сохранить как PDF».', 'success');
    sendToPrinter(nodes, pageCss(g));
  }
  function fileName(ext) {
    const d = new Date();
    return 'tst-' + (spec.file || spec.key || 'dokument') + '-' + d.getFullYear() + '-' + pad2(d.getMonth() + 1) + '-' + pad2(d.getDate()) + '.' + ext;
  }
  function doExport(kind) {
    if (!ready()) return;
    const paper = paperFromSpec(spec, readDoc()), g = geom();
    const ext = kind === 'word' ? 'docx' : 'xlsx';
    const blob = kind === 'word' ? docxBlob(paper, { w: g.w, h: g.h, orient: px.orient }, px) : xlsxBlob(blocksFromPaper(paper), spec.sheet || spec.title);
    downloadBlob(fileName(ext), blob);
    logPrint(docName(), kind, fileName(ext)); drawLog();
    toast('Файл сформирован', fileName(ext), 'success');
  }
  function doMail() {
    if (!ready()) return;
    const d = readDoc();
    const body = docName() + (spec.subtitle ? '\n' + spec.subtitle : '') + (d.org ? '\nОрганизация: ' + d.org : '') +
```

```
    '\nСформировано: ' + nowStr() +
572         '\n\nФайл документа (PDF, Word или Excel) прикрепите вручную: почтовая программа не принимает вложения от веб-
    страницы.';
573       logPrint(docName(), 'email', d.org); drawLog();
574       location.href = 'mailto:?subject=' + encodeURIComponent('Регламент-ТС: ' + docName()) + '&body=' +
    encodeURIComponent(body);
575     }
576     function close() {
577       if (!PW) return;
578       document.removeEventListener('keydown', onKey, true);
579       if (ro) ro.disconnect();
580       lockBg(false); ov.remove(); PW = null;
581       if (opener && opener.focus && document.contains(opener)) { try { opener.focus(); } catch (e) { /* элемент удалён
    */ } }
582       if (document.fullscreenElement) { try { document.exitFullscreen(); } catch (e) { /* уже вышли */ } }
583     }
584     function onKey(e) {
585       if (stack.length || document.querySelector('.f-modal-backdrop')) return;
586       if (e.key === 'Tab') {
587         const f = [].slice.call(ov.querySelectorAll('button:not(:disabled),input:not(:disabled),select:not(:disabled),[t
    abindex="0"]')).filter(x => x.offsetParent !== null);
588         if (f.length) {
589           const i = f.indexOf(document.activeElement);
590           if (e.shiftKey && i <= 0) { e.preventDefault(); f[f.length - 1].focus(); }
591           else if (!e.shiftKey && (i === f.length - 1 || i < 0)) { e.preventDefault(); f[0].focus(); }
592         }
593         return;
594       }
595       const inField = e.target.closest && e.target.closest('input,select,textarea');
596       if (e.key === 'Escape') { e.preventDefault(); e.stopPropagation(); close(); }
597       else if (e.key === 'F11') { e.preventDefault(); fullscreen(); }
598       else if (e.key === 'PageDown' && !inField) { e.preventDefault(); goPage(curPage + 1); }
599       else if (e.key === 'PageUp' && !inField) { e.preventDefault(); goPage(curPage - 1); }
600       else if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'p') { e.preventDefault(); doPrint('print'); }
601       else if ((e.ctrlKey || e.metaKey) && (e.key === '+' || e.key === '=')) { e.preventDefault(); px.zoomMode =
    'fixed'; setZoom(zoom + 0.1); }
602       else if ((e.ctrlKey || e.metaKey) && e.key === '-') { e.preventDefault(); px.zoomMode = 'fixed'; setZoom(zoom -
    0.1); }
603     }
604     document.addEventListener('keydown', onKey, true);
605     PW = {
606       apply(o) {
607         if (o.spec) { spec = o.spec; if (!docs.some(d => d.key === spec.key)) extra = spec; fillDocs(); refresh(); }
608         if (o.focus === 'settings') { prFs.scrollIntoView({ behavior: 'smooth' }); paperSel.focus(); }
609       },
610       close
611     };
612     if (spec && !docs.some(d => d.key === spec.key)) extra = spec;
613     fillDocs(); fillPresets(); setUiFromPx(); setZoom(1); drawLog();
614     lockBg(true);
615     if (spec) refresh();
616     else pagesBox.replaceChildren(E('p', { class: 'xempty', text: 'Выберите документ в списке «Печатная форма».' }));
617     setTimeout(() => { const pb = $('pw-print'); if (pb && opts.focus !== 'settings') pb.focus(); }, 40);
618     if (opts.focus === 'settings') PW.apply({ focus: 'settings' });
619   }
620
621   /* ============================================================================
622      СПРАВОЧНЫЕ ДАННЫЕ ПО УЧЁТУ
623      ============================================================================ */
624   const nf = (v, d) => Number(v || 0).toLocaleString('ru-RU', { minimumFractionDigits: d || 0, maximumFractionDigits: d
    || 0 });
625   function src(t) { return E('span', { class: 'ref-src', text: t }); }
626   function p(t, s) { return E('p', null, [t, s ? ' ' : null, s ? src(s) : null]); }
627   function h3(t) { return E('h3', { text: t }); }
628   function h4(t) { return E('h4', { text: t }); }
629   function ul(items) { return E('ul', null, items.map(i => E('li', null, [i]))); }
630   function ol(items) { return E('ol', null, items.map(i => E('li', null, [i]))); }
631   function tbl(head, rows) {
632     const t = E('table', { class: 'ref-table' });
633     t.appendChild(E('thead', null, [E('tr', null, head.map(c => E('th', { text: c }))) ]));
634     t.appendChild(E('tbody', null, rows.map(r => E('tr', null, r.map(c => (c && c.nodeType) ? E('td', null, [c]) :
    E('td', { text: c == null ? '' : String(c) }))))));
635     return E('div', { class: 'ref-scroll' }, [t]);
636   }
637   function formula(title, math, vars, steps, extra) {
638     return E('div', { class: 'ref-formula' }, [h4(title), E('p', { class: 'ref-math', text: math }), vars && vars.length
     ? ul(vars) : null, steps && steps.length ? ol(steps) : null].concat(extra || []));
639   }
640   function example(n, title, lines, result) {
641     return E('div', { class: 'ref-example' }, [h4('Пример ' + n + '. ' + title), ol(lines), result ? E('p', { class:
    'ref-result', text: result }) : null]);
642   }
643   function dot(color) { return E('span', { class: 'ref-dot', style: 'background:' + color }); }
644
645   function buildHelp() {
646     const F = window.TST && window.TST.fleet;
647     const D = F && F.helpData ? F.helpData() : null;
648     if (!D) return null;
649     const s = D.settings, df = D.defaults, sections = [];
650     function sec(id, title, nodes) {
```

```
      const el = E('section', { class: 'ref-sec', id: 'ref-' + id }, [E('h2', { text: title })].concat(nodes));
      sections.push({ id, title, el });
    }
    const today = D.today;

    /* 1 */
    sec('s1', 'Раздел 1. Общие сведения о модуле учёта', [
      E('div', { class: 'ref-lead' }, [
        p('Модуль «Технические средства тыла» автоматизированной системы учёта и контроля «Регламент-ТС» ведёт электронный учёт техники служб тыла: карточки единиц техники, журнал технического обслуживания (ТО), показания счётчиков моточасов, путевые листы, планирование эксплуатации и ремонта, печатные документы и отчёты.'),
        p('Программа работает без подключения к сети. Данные хранятся только в этом браузере на этом компьютере; перенос и сохранность обеспечиваются резервной копией (раздел 9).', 'Настройки → Сохранность данных')]),
      h3('1.1 Разделы модуля'),
      tbl(['Раздел', 'Назначение'], [
        ['Пункт управления', 'Стартовый экран: состояние всего парка, КТГ по подразделениям, ТО и календарь работ. Каждое число открывает перечень этих машин (раздел 12).'],
        ['Парк техники', 'Реестр зарегистрированной техники: поиск, фильтры, сортировка, таблица или карточки, групповые действия; переход в карточку единицы техники.'],
        ['Ближайшие ТО', 'Техника, требующая внимания: просроченные, требующие и приближающиеся ТО, давно не обновлявшиеся моточасы.'],
        ['Путевые листы', 'Ежедневный учёт выходов техники с показаниями счётчика на начало и конец выхода.'],
        ['Планирование', 'Группы эксплуатации и КТГ, наличие моторесурса, потребность в ТО и трудозатратах, фонд времени ремонтного подразделения, экономика.'],
        ['Документы', 'Годовой план, план-график ТО на месяц, заявка на материалы и специалистов, акты.'],
        ['Отчёты', 'Табличные отчёты с предпросмотром, выгрузкой в Excel и печатью.'],
        ['Сохранность данных', 'Резервные копии, точки восстановления, корзина, журнал изменений, проверка целостности, модули и самотесты (раздел 12).'],
        ['Настройки', 'Интервалы ТО по умолчанию, пороги цветовой индикации, напоминания, резервная копия, управление данными.']]),
      h3('1.2 Связь со справочником'),
      p('Единица техники привязывается к позиции справочника («Тип по справочнику» в карточке). Тогда в карточке доступен регламент ТО из учебного пособия: перечень работ, личный состав, инструмент, материалы и нормативная трудоёмкость. Из справочника видно, сколько техники этого типа стоит на учёте и в каком она состоянии.'),
      p('Если в пособии регламент для позиции не приведён, его можно задать самостоятельно — «норматив пользователя». Он хранится вместе с данными учёта и попадает в резервную копию.', 'Справочник → карточка позиции'),
      h3('1.3 Порядок работы'),
      ol(['Поставьте технику на учёт: «Парк техники» → «Добавить технику». Укажите текущие моточасы, дату постановки на учёт и интервалы ТО.',
        'Регулярно вносите показания счётчика (кнопка «Моточасы» в карточке) или путевые листы — моточасы обновятся автоматически.',
        'После каждого обслуживания добавляйте «Запись ТО»: вид, дату, показание счётчика, выполненные работы.',
        'Следите за разделом «Ближайшие ТО» и колоколом уведомлений в шапке.',
        'Формируйте документы и отчёты; печатайте их через окно предпросмотра (Ctrl+P).',
        'Регулярно сохраняйте резервную копию (ZIP) вне этого компьютера.'])
    ]);

    /* 2 */
    sec('s2', 'Раздел 2. Парк техники и карточка техники', [
      h3('2.1 Регистрация техники'),
      tbl(['Поле', 'Назначение'], [
        ['Тип по справочнику', 'Позиция справочника — источник регламента ТО и связи «Справочник ↔ Учёт».'],
        ['Наименование техники *', 'Обязательное поле; отображается во всех списках и документах.'],
        ['Марка и модель', 'Используется в группировке «марка × группа эксплуатации» при планировании.'],
        ['Гос. номер, инвентарный и заводской номера', 'Идентификация единицы; по ним работает поиск (гос. номер сравнивается и без пробелов).'],
        ['Подразделение, ответственное лицо', 'Фильтры парка, отчёт «Статистика по подразделениям».'],
        ['Год выпуска, дата постановки на учёт', 'Дата постановки — точка отсчёта срока первого ТО, пока в журнале нет записей.'],
        ['Статус', D.statuses.join(', ') + '. Списанная техника исключается из расчётов и уведомлений.'],
        ['Текущие моточасы', 'Показание счётчика; при постановке на учёт сохраняется в истории моточасов как база отсчёта.'],
        ['Интервал ТО', 'По моточасам и/или по календарной дате — периодичность ТО-1. Можно задать отдельные интервалы ТО-2.'],
        ['Группа эксплуатации', D.expGroups.join(', ') + ' — определяет норматив КТГ.'],
        ['Плановая годовая наработка', 'Для годового плана, потребности в ТО, план-графика и среднесуточной наработки.'],
        ['Годовая норма расхода ресурса', 'Для расчёта наличия моторесурса и акта списания.']]),
      h3('2.2 Список техники'),
      ul(['Поиск — по наименованию, марке, гос. номеру, инвентарному и заводскому номерам, подразделению и ответственному.',
        'Фильтры — подразделение, ответственный, статус, состояние ТО, период проведения ТО («ТО с даты» / «ТО по дату»).',
        'Сортировка — щелчок по заголовку столбца; повторный щелчок меняет направление.',
        'Режимы — таблица или карточки с кольцом остатка ресурса.',
        'Групповые действия — отметьте строки флажками: смена статуса, назначение интервала ТО, удаление (с возможностью отмены).']),
      h3('2.3 Карточка единицы техники'),
      p('Верхняя панель показывает кольцо остатка ресурса, текущие моточасы, наработку с последнего ТО, даты последнего и следующего ТО, остаток моточасов и дней. Вкладки: «Основные сведения», «Журнал ТО», «Моточасы». Кнопки: «Моточасы», «Изменить», «Запись ТО».'),
      p('Удаление записей выполняется сразу; в уведомлении есть кнопка «Отменить» — она возвращает удалённые записи в течение нескольких секунд.')
    ]);

    /* 3 */
    sec('s3', 'Раздел 3. Техническое обслуживание и моточасы', [
      h3('3.1 Виды ТО в журнале'),
      tbl(['Вид', 'Сбрасывает срок ТО-1', 'Начинает отсчёт ТО-2', 'Регламент в справочнике'], D.maintKinds.map(k => [
        k, D.resetKinds.indexOf(k) >= 0 ? 'да' : 'нет', D.to2ResetKinds.indexOf(k) >= 0 ? 'да' : 'нет',
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
    D.catalogKinds.indexOf(k) >= 0 ? 'да' : '—'])),
716     p('ЕО (ежедневное обслуживание) проводится ежедневно и не сбрасывает межсервисный интервал. Текущий ремонт учитывается в журнале и в экономических расчётах, но срок ТО не сбрасывает.'),
717     h3('3.2 Запись ТО'),
718     ul(['Вид ТО, дата проведения и показание счётчика — обязательны. Показание переносится в историю моточасов.',
719       'Исполнитель, подразделение, стоимость, выполненные работы, запасные части, примечание — по необходимости.',
720       'Прикреплённые документы (акты, фото) хранятся в браузере и попадают в резервную копию.',
721       'Если техника привязана к справочнику, в карточке («Основные сведения») видны регламенты ТО, и по каждому виду можно распечатать карту ТО с отметками о выполнении.']),
722     h3('3.3 Моточасы'),
723     p('Показание вносится с датой снятия. Прирост наработки считается как разность соседних показаний; отрицательные скачки (замена счётчика) в расход не включаются. Самое первое показание единицы — база отсчёта, а не наработка.'),
724     p('Если показания не обновлялись ' + s.hoursStaleDays + ' дн. и более, программа напоминает об этом (только для техники в статусе «В эксплуатации»).', 'Настройки → Пороги цветовой индикации'),
725     h3('3.4 Фото и документы на технику'),
726     ul(['Карточка техники → вкладка «Фото и документы». Фотографии добавляются кнопкой или перетаскиванием (на телефоне — «Снять на камеру»); снимки сжимаются до 1600 px, миниатюры хранятся отдельно. До 100 фото на единицу, у одного из них — отметка «Главное»: оно видно в шапке карточки и в списке техники.',
727       'Документы на машину в целом: паспорт, формуляр, акты приёма-передачи и другие (с названием). Номер, дата, кем выдан, примечание и несколько сканов на документ (PDF, Word, Excel, изображения до 20 МБ). Документ можно учесть и без скана — по бумажному экземпляру.',
728       'Изображения открываются в просмотре со стрелками, подписью и скачиванием; PDF и прочие файлы скачиваются. Все файлы входят в резервную копию и удаляются вместе с техникой (отмена возвращает их).'])
729   ]);
730
731   /* 4: формулы и примеры считаются движком программы */
732   const ex1 = D.example({ hours: 250, days: 180, regAgo: 60, current: 190 });
733   const ex2 = D.example({ hours: 250, days: 180, regAgo: 170, current: 120 });
734   const ex3 = D.example({ hours: 250, days: 180, regAgo: 40, current: 270 });
735   const lvlName = k => D.levels[k] ? D.levels[k].label : k;
736   sec('s4', 'Раздел 4. Расчёт срока ТО и цветовая индикация', [
737     p('Срок очередного ТО считается для каждой единицы техники отдельно по моточасам и/или по календарю — в зависимости от включённых в карточке критериев. Итоговое состояние определяется наиболее «тревожным» критерием.'),
738     formula('Ф1. Срок ТО по моточасам', 'Lслед = Lбаз + Lинт;  ΔL = Lслед − Lтек', [
739       'Lбаз — показание счётчика при последнем ТО, сбрасывающем срок (при отсутствии записей — показание на дату постановки на учёт)',
740       'Lинт — интервал ТО по моточасам', 'Lтек — текущие моточасы', 'ΔL — остаток моточасов (отрицательный — перерасход)']),
741     formula('Ф2. Срок ТО по календарю', 'Дслед = Дбаз + Динт;  ΔД = Дслед − сегодня', [
742       'Дбаз — дата последнего ТО (при отсутствии записей — дата постановки на учёт)', 'Динт — интервал ТО в днях']),
743     formula('Ф3. Остаток ресурса', 'R = min(ΔL / Lинт; ΔД / Динт) × 100 %', [
744       'Берётся наименьший остаток среди включённых критериев и видов ТО (ТО-1 и ТО-2)']),
745     formula('Ф4. Интервал ТО-2', 'Lинт(ТО-2) = Lинт(ТО-1) × N;  N = Пто-2 / Пто-1 = ' + nf(s.periodTO2) + ' / ' + nf(s.periodTO1) + ' = ' + D.to2Every, [
746       'Пто-1, Пто-2 — периодичность ТО-1 и ТО-2 из раздела «Планирование»', 'В карточке можно задать собственный интервал ТО-2 (по моточасам и дням)',
747       'Срок ТО-2 отсчитывается только от ТО-2, ТО-3 или капитального ремонта. ТО-2 включает объём ТО-1, поэтому оно очередное, если наступает не позже ТО-1']),
748     h3('4.1 Шкала состояний'),
749     tbl(['Состояние', 'Условие', 'Цвет'], [
750       [lvlName('ok'), 'R ≥ ' + s.warnPct + ' %', E('span', null, [dot('var(--st-ok)'), 'зелёный'])],
751       [lvlName('soon'), s.urgentPct + ' % < R < ' + s.warnPct + ' %', E('span', null, [dot('var(--st-soon)'), 'жёлтый'])],
752       [lvlName('urgent'), '0 % < R ≤ ' + s.urgentPct + ' %', E('span', null, [dot('var(--st-urgent)'), 'оранжевый'])],
753       [lvlName('overdue'), 'R ≤ 0 % (ресурс исчерпан)', E('span', null, [dot('var(--st-overdue)'), 'красный'])],
754       [lvlName('none'), 'интервал ТО в карточке не задан', E('span', null, [dot('var(--st-none)'), 'серый'])],
755       [lvlName('retired'), 'статус «Списано»', E('span', null, [dot('var(--st-none)'), 'серый'])]]),
756     p('Пороги жёлтого (' + s.warnPct + ' %) и оранжевого (' + s.urgentPct + ' %) цвета задаются в настройках; красный устанавливается автоматически.', 'Настройки → Пороги цветовой индикации'),
757     h3('4.2 Примеры'),
758     p('Примеры считаются той же функцией, что и состояние техники в программе, с текущими порогами из настроек. Интервал ТО-1 — 250 мото·ч и 180 дней.'),
759     example(1, 'Приближается срок ТО', [
760       'Техника поставлена на учёт 60 дней назад при 0 мото·ч, записей ТО нет. Текущие моточасы — 190.',
761       'Ф1: Lслед = 0 + 250 = 250; ΔL = 250 − 190 = ' + nf(ex1.remainingHours) + ' мото·ч → ' + nf(ex1.remainingHours) + ' / 250 = ' + nf(ex1.remainingHours / 250 * 100, 1) + ' %.',
762       'Ф2: ΔД = 180 − 60 = ' + ex1.remainingDays + ' дн. → ' + nf(ex1.remainingDays / 180 * 100, 1) + ' %.',
763       'Ф3: R = min(…) = ' + nf(ex1.pct, 1) + ' %.'], 'Состояние: ' + lvlName(ex1.level) + '.'),
764     example(2, 'Срок по календарю', [
765       'Поставлена на учёт 170 дней назад, текущие моточасы — 120.',
766       'По моточасам: ΔL = 250 − 120 = ' + nf(ex2.remainingHours) + ' → ' + nf(ex2.remainingHours / 250 * 100, 1) + ' %.',
767       'По календарю: ΔД = 180 − 170 = ' + ex2.remainingDays + ' дн. → ' + nf(ex2.remainingDays / 180 * 100, 1) + ' %.',
768       'R = ' + nf(ex2.pct, 1) + ' % — решает календарный критерий.'], 'Состояние: ' + lvlName(ex2.level) + '.'),
769     example(3, 'Перерасход ресурса', [
770       'Поставлена на учёт 40 дней назад, текущие моточасы — 270.',
771       'ΔL = 250 − 270 = ' + nf(ex3.remainingHours) + ' мото·ч (перерасход ' + nf(Math.abs(ex3.remainingHours)) + ').',
772       'R = ' + nf(ex3.pct, 1) + ' % ≤ 0.'], 'Состояние: ' + lvlName(ex3.level) + '.'),
773     h3('Типичные ошибки'),
774     ul(['Не внесено показание счётчика при постановке на учёт — тогда отсчёт идёт от нуля, и техника с наработкой сразу считается просроченной.',
775       'Запись ТО внесена без показания счётчика или с ошибочной датой — срок следующего ТО смещается.',
776       'Нет записи о последнем ТО-2, проведённом до начала ведения журнала, — карточка попросит внести его дату; до этого срок ТО-2 не ухудшает состояние.',
777       'Отключены оба критерия интервала — состояние «' + lvlName('none') + '», уведомления не формируются.'])
778   ]);
779
780   /* 5 */
781   sec('s5', 'Раздел 5. Ближайшие ТО и уведомления', [
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
      p('Раздел «Ближайшие ТО» показывает счётчики «Просрочено», «Требуется ТО», «Скоро ТО», «Обновить моточасы» и список техники, требующей внимания, — от самых срочных.'),
      h3('5.1 Уведомления'),
      tbl(['Уведомление', 'Когда формируется'], [
        ['ТО просрочено', 'R ≤ 0 %: перерасход моточасов и/или просрочка по календарю'],
        ['Требуется ТО в ближайшее время', 'R ≤ ' + s.urgentPct + ' %'],
        ['Приближается срок ТО', 'R < ' + s.warnPct + ' %'],
        ['Необходимо обновить моточасы', 'показания не обновлялись ' + s.hoursStaleDays + ' дн. и более (техника «В эксплуатации»)']]),
      p('Колокол в шапке показывает число уведомлений; щелчок по уведомлению открывает карточку техники. Сводка при открытии раздела включается в настройках («Показывать сводку уведомлений при открытии раздела»).'),
      p('Если резервная копия не создавалась более ' + s.backupRemindDays + ' дн., программа напоминает об этом.', 'Настройки → Сохранность данных')
    ]);

    /* 6 */
    sec('s6', 'Раздел 6. Путевые листы', [
      p('Путевой лист — ежедневный документ учёта выхода техники. На его основании ведётся карточка учёта работы машины и сводная ведомость за месяц.', 'методика ВАМТО, 2025, гл. 3'),
      tbl(['Поле', 'Назначение'], [
        ['Техника *', 'Единица техники из парка'], ['Дата выхода *', 'Дата путевого листа'], ['Водитель (расчёт)', 'Кто использовал технику'],
        ['Маршрут / задача', 'Куда и для чего'], ['Показание на начало / конец выхода *', 'Показания счётчика, мото·ч; конец не меньше начала'], ['Примечание', 'Произвольный текст']]),
      formula('Ф5. Наработка за выход', 'Lвых = Lконец − Lначало', ['Наработка за месяц — сумма Lвых по путевым листам месяца (отчёт «Карточка учёта работы машины»)']),
      p('Каждый путевой лист синхронизирован с историей моточасов: показание на конец выхода записывается в историю, а текущие моточасы пересчитываются по последнему показанию. Поэтому расход ресурса из путевых листов автоматически учитывается в сроке ТО, наличии моторесурса и потребности в ТО.')
    ]);

    /* 7 */
    const ktgS = s.ktgByGroup['Строевая'] != null ? s.ktgByGroup['Строевая'] : 0.9;
    const ktgN = Math.max(0, Math.floor(20 * (1 - ktgS)) - 1);
    const L = 6000, nTO2 = Math.floor(L / s.periodTO2), nTO1 = Math.max(0, Math.floor(L / s.periodTO1) - nTO2), nTR = Math.floor(L / s.periodTR);
    const lab = s.laborNorms, tSum = nTO1 * lab['ТО-1'] + nTO2 * lab['ТО-2'] + nTR * lab['ТР'];
    const ru = s.repairUnit, fm = ru.workDaysMonth * ru.hoursPerDay, fy = fm * 12 * (1 - ru.nonProdPct / 100);
    sec('s7', 'Раздел 7. Планирование эксплуатации и ремонта', [
      p('Расчёты выполняются по методике планирования эксплуатации и ремонта ВВСТ (ВАМТО, 2025) и опираются на нормативы из настроек и карточек техники. Они предназначены для оценки и не заменяют официальные формы.', 'методика ВАМТО, 2025, гл. 3'),
      formula('Ф6. Допустимое количество машин в ремонте', 'Мплан.рем = Мсписочн × (1 − КТГ) − 1', [
        'Мсписочн — списочный состав группы эксплуатации (без списанной техники)', 'КТГ — норматив коэффициента технической готовности группы',
        'Результат округляется вниз; превышение фактического числа машин в ремонте выделяется красным'],
        null, [p('Пример: 20 машин строевой группы, КТГ = ' + nf(ktgS, 2) + ' → 20 × ' + nf(1 - ktgS, 2) + ' − 1 = ' + ktgN + ' машин.', 'формула (4)')]),
      formula('Ф7. Наличие моторесурса', 'Rразр = Nгод − 0,05 × Nгод;  Rост = Rразр − Rфакт', [
        'Nгод — годовая норма расхода ресурса из карточки', '5 % — резерв на инспектирование и итоговые проверки', 'Rфакт — израсходовано за календарный год по истории моточасов'],
        null, [p('Пример: норма 500 мото·ч → резерв 25, разрешено 475; израсходовано 300 → остаток 175 мото·ч.', 'п. 3.7 методики')]),
      formula('Ф8. Потребность в ТО и ремонте на год', 'N(ТО-2) = Lгод / Пто-2;  N(ТО-1) = Lгод / Пто-1 − N(ТО-2);  N(ТР) = Lгод / Птр', [
        'Lгод — суммарная плановая годовая наработка группы «марка × группа эксплуатации»',
        'Пто-1 = ' + nf(s.periodTO1) + ', Пто-2 = ' + nf(s.periodTO2) + ', Птр = ' + nf(s.periodTR) + ' мото·ч — периодичность из настроек планирования',
        'Количество округляется вниз до целого']),
      formula('Ф9. Трудозатраты', 'T = N(ТО-1) × tто-1 + N(ТО-2) × tто-2 + N(ТР) × tтр', [
        'Нормативная трудоёмкость: ТО-1 — ' + nf(lab['ТО-1'], 1) + ', ТО-2 — ' + nf(lab['ТО-2'], 1) + ', ТР — ' + nf(lab['ТР'], 1) + ' чел.-ч'],
        null, [p('Пример: Lгод = ' + nf(L) + ' → N(ТО-2) = ' + nTO2 + ', N(ТО-1) = ' + nTO1 + ', N(ТР) = ' + nTR + '; T = ' + nTO1 + '×' + nf(lab['ТО-1'], 1) + ' + ' + nTO2 + '×' + nf(lab['ТО-2'], 1) + ' + ' + nTR + '×' + nf(lab['ТР'], 1) + ' = ' + nf(tSum, 1) + ' чел.-ч.', 'формулы (5)–(10)')]),
      formula('Ф10. Фонд рабочего времени ремонтного подразделения', 'Фмес = Драб × tдн;  Фгод = Фмес × 12 × (1 − Пвын / 100)', [
        'Драб — рабочих дней в месяце на ТО и ремонт (' + ru.workDaysMonth + ')', 'tдн — продолжительность рабочего дня (' + ru.hoursPerDay + ' ч)', 'Пвын — вынужденные потери (' + ru.nonProdPct + ' %)', 'Фонд подразделения = фонд одного специалиста × число производственников (' + ru.specialists + ')'],
        null, [p('Сейчас: Фмес = ' + nf(fm) + ' ч, Фгод = ' + nf(fy, 1) + ' ч на одного специалиста; на подразделение — ' + nf(fy * ru.specialists, 1) + ' ч в год.', 'формулы (21)–(24), п. 3.12')]),
      formula('Ф11. Экономическая эффективность и загрузка', 'Эф = Трт × Снч;  Загрузка = Трт / (Фмес × М × месяцев) × 100 %', [
        'Трт — нормативные трудозатраты на ТО-1, ТО-2 и ТР по журналу за период', 'Снч — стоимость нормочаса (' + nf(s.normHourCost) + ' руб.)', 'М — число производственников']),
      formula('Ф12. План-график ТО на месяц', 'Дто = (Lто-1 − Lф) / Lсс;  Lсс = Lгод / 365', [
        'Lто-1 — периодичность ТО-1 (интервал из карточки или норматив планирования)', 'Lф — наработка с последнего ТО', 'Lсс — среднесуточная наработка по плановой годовой наработке'],
        null, [p('Пример: Lто-1 = 250, Lф = 190, Lгод = 730 → Lсс = 2, Дто = (250 − 190) / 2 = 30 дней.', 'формула (18)')])
    ]);

    /* 8 */
    sec('s8', 'Раздел 8. Документы, отчёты и печать', [
      h3('8.1 Документы'),
      tbl(['Документ', 'Источник данных'], [
        ['Годовой план эксплуатации и ремонта', 'Плановая годовая наработка и нормативы (Ф8, Ф9)'],
        ['План-график ТО и ремонта на месяц', 'Наработка с ТО и среднесуточная наработка (Ф12)'],
        ['Заявка на материалы, инструмент и специалистов', 'Техника с ТО в выбранный период и регламенты справочника'],
```

```
      ['Акт списания моторесурса', 'Техника с перерасходом годовой нормы (Ф7)'],
      ['Акт внезапной проверки наличия техники', 'Список техники, кроме списанной'],
      ['Акт проверки использования техники и расхода ГСМ', 'Путевые листы за выбранный период'],
      ['Карта ТО (чек-лист)', 'Регламент вида ТО из справочника — из карточки позиции справочника или карточки техники']]),
    h3('8.2 Отчёты'),
    tbl(['Отчёт', 'Содержание'], D.reports.map(r => [r.title, r.desc])),
    p('Отчёт можно просмотреть, выгрузить в Excel или напечатать. Фильтры периода и подразделения применяются ко всем форматам.'),
    h3('8.3 Окно предпросмотра и печати'),
    ul(['Открывается кнопками «Сформировать и распечатать», «Печать / PDF» или сочетанием Ctrl+P в разделе учёта.',
      'Слева — миниатюры страниц, в центре — лист, справа — параметры. Документ можно сменить в списке «Печатная форма».',
      'Параметры: принтер (выбирается в системном окне), число копий, ориентация, размер бумаги, поля, двусторонняя и чёрно-белая печать, колонтитул и номера страниц, несколько страниц на листе (в предпросмотре).',
      'Организация, должность, звание и фамилия исполнителя подставляются в документ и запоминаются.',
      'Шаблоны печати сохраняют набор параметров листа под именем.',
      'Экспорт: PDF (через системное окно печати), Word (.docx), Excel (.xlsx); отправка по e-mail — письмо с описанием, файл прикладывается вручную.',
      'Журнал печати хранит последние операции.',
      'Клавиши: PageUp/PageDown — страницы, Ctrl + «+»/«−» — масштаб, Ctrl+P — печать, F11 — полный экран, Esc — закрыть.']),
    p('Разбивка на страницы в предпросмотре приблизительная: окончательную формирует браузер при печати.', 'окно печати')
  ]);

  /* 9 */
  const rowS = (name, key, unit, where) => [name, nf(df[key]) + (unit || ''), nf(s[key]) + (unit || ''), where];
  sec('s9', 'Раздел 9. Настройки и сохранность данных', [
    h3('9.1 Параметры'),
    tbl(['Параметр', 'По умолчанию', 'Сейчас', 'Где изменить'], [
      rowS('Интервал ТО-1 по моточасам', 'defaultHours', ' мото·ч', 'Настройки'),
      rowS('Интервал ТО-1 по календарю', 'defaultDays', ' дн.', 'Настройки'),
      rowS('Жёлтый порог', 'warnPct', ' %', 'Настройки'),
      rowS('Оранжевый порог', 'urgentPct', ' %', 'Настройки'),
      rowS('Напоминание о моточасах', 'hoursStaleDays', ' дн.', 'Настройки'),
      rowS('Напоминание о резервной копии', 'backupRemindDays', ' дн.', 'Настройки'),
      rowS('Периодичность ТО-1', 'periodTO1', ' мото·ч', 'Планирование'),
      rowS('Периодичность ТО-2', 'periodTO2', ' мото·ч', 'Планирование'),
      rowS('Периодичность ТР', 'periodTR', ' мото·ч', 'Планирование'),
      rowS('Стоимость нормочаса', 'normHourCost', ' руб.', 'Планирование → Экономика')].concat(
      Object.keys(df.ktgByGroup).map(g => ['КТГ: ' + g, nf(df.ktgByGroup[g], 2), nf(s.ktgByGroup[g], 2), 'Планирование → Группы и КТГ'])),
    p('Интервалы по умолчанию подставляются в новые карточки; кнопка «Применить ко всей технике» назначает их всем единицам. Жёлтый порог должен быть больше оранжевого.'),
    h3('9.2 Резервная копия'),
    ul(['«Сохранить копию (ZIP)» — архив со всеми записями, нормативами, настройками и прикреплёнными файлами, а также инструкцией по восстановлению.',
      '«Восстановить из копии» — загрузка ZIP (или старой копии в формате JSON без вложений).',
      'Данные хранятся только в этом браузере. Очистка данных браузера удаляет их — храните копию вне компьютера.',
      'Программу можно открывать в нескольких вкладках: изменения синхронизируются между ними автоматически.',
      '«Включить постоянное хранение» просит браузер не удалять данные при нехватке места.']),
    h3('9.3 Управление данными'),
    p('Сейчас в учёте: техники — ' + D.counts.vehicles + ', записей ТО — ' + D.counts.maint + ', записей моточасов — ' + D.counts.hours + ', путевых листов — ' + D.counts.trips + ', своих нормативов ТО — ' + D.counts.norms + '. Демонстрационные записи удаляются одной кнопкой; «Очистить все данные» удаляет всё без возможности восстановления (кроме резервной копии).')
  ]);

  /* 10 */
  const gl = [['ТО', 'Техническое обслуживание: ЕО, ТО-1, ТО-2, ТО-3, сезонное (СО).'], ['Межсервисный интервал', 'Наработка (мото·ч) или срок (дни) между плановыми ТО.'],
    ['Моточас (мото·ч)', 'Единица наработки по счётчику; для автомобильной техники допускается учёт в километрах при тех же расчётах.'],
    ['Остаток ресурса', 'Доля интервала, оставшаяся до следующего ТО (Ф3).'], ['КТГ', 'Коэффициент технической готовности — доля исправной техники, установленная нормативом для группы эксплуатации.'],
    ['Группа эксплуатации', 'Боевая, строевая, учебно-боевая, учебно-строевая — определяет норматив КТГ и нормы расхода ресурса.'],
    ['Годовая норма расхода ресурса', 'Предельная наработка за календарный год; 5 % — резерв.'], ['Плановая годовая наработка', 'Ожидаемая наработка на год — основа годового плана и план-графика.'],
    ['ТР', 'Текущий ремонт.'], ['Норматив пользователя', 'Регламент ТО, заданный вручную для позиции справочника, у которой в пособии регламента нет.']];
  const dl = E('dl', { class: 'ref-gloss' });
  gl.forEach(g => { dl.appendChild(E('dt', { text: g[0] })); dl.appendChild(E('dd', { text: g[1] })); });
  const faq = [['Почему новая техника сразу «просрочена»?', 'При постановке на учёт не указаны текущие моточасы, и отсчёт идёт от нуля. Внесите показание на дату постановки или запись последнего ТО.'],
    ['Как перенести данные на другой компьютер?', 'Настройки → «Сохранить копию (ZIP)», затем на другом компьютере — «Восстановить из копии».'],
    ['Почему не формируется годовой план?', 'В карточках техники не указана плановая годовая наработка (раздел «Планирование эксплуатации» в форме техники).'],
    ['Как получить PDF?', 'Откройте окно печати и нажмите «Экспорт в PDF», затем в системном окне выберите «Сохранить как PDF».'],
    ['Что будет со сроком ТО после путевого листа?', 'Показание на конец выхода попадает в историю моточасов, текущие моточасы и остаток ресурса пересчитываются автоматически.'],
    ['Можно ли отменить удаление?', 'Да, в течение нескольких секунд — кнопкой «Отменить» в уведомлении.']];
  const fq = E('div', { class: 'ref-faq' });
  faq.forEach(q => fq.appendChild(E('details', null, [E('summary', { text: q[0] }), E('p', { text: q[1] })])));
  sec('s10', 'Раздел 10. Пояснения', [h3('Глоссарий'), dl, h3('Часто задаваемые вопросы'), fq, h3('Горячие клавиши'),
    tbl(['Клавиши', 'Действие'], [['F1', 'Справка по учёту'], ['Ctrl+P', 'Окно предпросмотра и печати (в разделе учёта)'], ['/', 'Поиск в шапке'],
```

```
      ['Ctrl+F (в справке)', 'Поиск по справке; Enter / Shift+Enter — следующее / предыдущее совпадение'], ['Ctrl+B (в справке)', 'Закладка на текущий раздел'], ['Esc', 'Закрыть окно']]),
    h3('Связанные документы'),
    ul(['Учебное пособие «Методика планирования эксплуатации и ремонта ВВСТ» (ВАМТО, 2025), гл. 3 — формулы планирования, путевые листы, карточка учёта работы машины.',
      'Регламенты технического обслуживания технических средств служб тыла — раздел «Справочник» программы.'])]);


  /* 11 */
  const qrFm = (D.qrFormats || []).map(x => [x.label, x.qr + ' мм']);
  sec('s11', 'Раздел 11. QR-маркировка техники', [
    E('div', { class: 'ref-lead' }, [
      p('Каждая единица техники при постановке на учёт получает постоянный идентификатор (например, v_mgq2k3x9_a1b2c3d). Он не меняется при правке наименования, подразделения, моточасов и других сведений и сохраняется в резервной копии. QR-код строится из этого идентификатора — повторная печать даёт тот же код.'),
      p('Программа работает без интернета: QR-коды формируются встроенным генератором, наклейки печатаются через окно печати браузера.', 'раздел «QR-маркировка»')]),
    h3('11.1 Что записано в QR-коде'),
    p('По умолчанию код оформлен как карточка контакта (vCard): имя — наименование техники, в заметке — номера, очередное ТО, дата «Сведения на …» и последней строкой служебная метка TST:1:<идентификатор> (TST — программа, 1 — версия формата). Так сделано потому, что камера iPhone показывает обычный текст из QR-кода лишь фрагментом (например, один номер, принятый за телефон), а карточку контакта открывает целиком и без интернета. Вид «Обычный текст» (строки «Поле: значение») выбирается в окнах «QR-код техники» и «Печать QR-наклеек» — он удобен для Android и сканеров. Ответственное лицо, пароли и другие закрытые сведения в код не записываются: наклейку может прочитать любой человек.'),
    tbl(['Состав данных', 'Что входит', 'Где применять'], [
      ['Только идентификатор', 'Регламент-ТС и строка TST:1:<идентификатор>', 'Самый надёжный код. Сведения открываются в программе, где есть база (сценарий 2)'],
      ['Основные сведения', 'наименование, инв. №, гос. №, зав. №, очередное ТО, дата формирования', 'Наклейки 50 × 50 и 60 × 40 мм. Читаются камерой любого смартфона (сценарий 1)'],
      ['Расширенные сведения', 'плюс марка, подразделение, год выпуска, дата постановки, моточасы, последнее ТО, состояние', 'Только наклейка 70 × 70 мм']]),
    p('Если выбранный состав не помещается в наклейку с надёжным размером модуля (не менее ' + nf(D.qrMinModule || 0.5, 1) + ' мм), программа автоматически уменьшает состав и предупреждает об этом. Порог получен испытаниями: код с модулем 0,3 мм в имитации снимка камерой не читается.'),
    h3('11.2 Два сценария просмотра со смартфона'),
    E('div', { class: 'ref-formula' }, [h4('Сценарий 1. Сведения внутри QR-кода (основной)'),
      ul(['Смартфон не нужно подключать к сети и устанавливать программу: камера показывает текст, записанный в коде.',
        'Это снимок на дату печати (строка «Сведения на …»). После изменения карточки наклейка не обновляется сама: в разделе «QR-маркировка» такая наклейка помечается «Устарела — перепечатайте».',
        'Актуальные моточасы, история ТО и все поля в коде не помещаются — их показывает сценарий 2.'])]),
    E('div', { class: 'ref-formula' }, [h4('Сценарий 2. Полная актуальная карточка'),
      ul(['Нужна программа с базой на том устройстве, которым сканируют: компьютер с этой программой (USB-сканер штрихкодов, веб-камера) или смартфон с копией программы и восстановленной резервной копией.',
        'Раздел «QR-маркировка» → «Сканирование QR-кода»: поле ввода (сканер вводит текст сам, со смартфона текст можно вставить), кнопки «Камера» и «Фото с QR-кодом» — там, где браузер умеет распознавать QR (Chrome, Edge, браузеры Android).',
        'Открывается карточка «только просмотр»: основные сведения, характеристики, учёт и эксплуатация, состояние ТО, последние записи журнала. Сканирование ничего не меняет в данных.',
        'Если на наклейке сведения отличаются от базы, программа показывает расхождение и не подставляет похожую запись. Неизвестный идентификатор — «не найдена в локальной базе».'])]),
    p('Ограничение: QR-код не может сам открыть базу, которая хранится в браузере другого компьютера. Сайта или сервера у программы нет, поэтому ссылку «открыть карточку в сети» QR-код не содержит.', 'архитектура'),
    h3('11.3 Печать наклеек'),
    ol(['Карточка техники → блок «QR-маркировка» → «Печать QR-наклейки» (или раздел «QR-маркировка» → отметить технику → «Печать выбранных»).',
      'Выберите формат наклейки, носитель (лист А4 с линиями реза или принтер этикеток — одна наклейка на страницу), состав данных и число экземпляров.',
      'В окне печати браузера: масштаб 100 % (отключите «По размеру страницы»), поля — «По умолчанию» или «Нет». Иначе размеры наклеек изменятся.',
      'Проверьте напечатанную наклейку камерой смартфона, затем наклейте её на ровную чистую поверхность без бликов.']),
    tbl(['Формат', 'Размер QR-кода'], qrFm),
    h3('11.4 Сканирование'),
    ul(['Android: откройте «Камеру» (или Google Объектив) и наведите на код — появится контакт с наименованием техники (для вида «Обычный текст» — сам текст); нажмите, чтобы прочитать сведения полностью. Сохранять контакт не нужно.',
      'iPhone: приложение «Камера» → нажмите жёлтую плашку «Контакт» под кодом — откроется карточка: наименование и заметка со сведениями. Сохранять контакт не нужно — нажмите «Отменить». Если камера показывает только цифры или «Нет пригодных данных», наклейка напечатана в виде «Обычный текст» или в первом выпуске программы — перепечатайте её в виде «Карточка контакта».',
      'Любое приложение — сканер QR-кодов, работающее без сети, тоже подходит. Работу конкретной модели смартфона проверьте заранее на напечатанной наклейке.',
      'Наводите камеру с расстояния 10–20 см, держите код целиком в кадре, избегайте бликов от плёнки и лампы.']),
    h3('11.5 Проверка и обслуживание'),
    ul(['«Проверить QR-код» — проверяет, что идентификатор корректен и уникален, содержимое разбирается обратно в ту же запись, модуль не меньше нормы; в Chrome и Edge код дополнительно распознаётся с изображения.',
      'Раздел «QR-маркировка» показывает по каждой единице: «Наклейка актуальна», «Не печаталась», «Устарела», «Ошибка идентификатора».',
      'Резервная копия сохраняет идентификаторы и отметки о печати; после восстановления на другом компьютере старые наклейки остаются действительными.'])
  ]);
  /* 12 */
  sec('s12', 'Раздел 12. Пункт управления и сохранность данных', [
    E('div', { class: 'ref-lead' }, [
      p('«Пункт управления» — стартовый экран учёта. «Сохранность данных» защищает реестр от потери: данные хранятся только в этом браузере на этом компьютере, поэтому программа сама делает снимки и хранит удалённое.')]),
    h3('12.1 Пункт управления'),
    ul(['Техническое состояние: на учёте (без списанной), исправна (статус «В эксплуатации»), неисправна, в ремонте, хранение и консервация, КТГ парка.',
```

```
      'КТГ = (на учёте − неисправна − в ремонте) / на учёте. Норматив подразделения — средний норматив КТГ по группам эксплуатации его техники (Планирование → Группы и КТГ); техника без группы в норматив не входит.',
      'Техническое обслуживание: просрочено, требуется, скоро, интервал не задан, моточасы не обновлялись дольше заданного в настройках срока.',
      'Календарь работ: плановые ТО по календарному сроку — на сегодня (включая просроченные), на неделю, на месяц. Сроки по моточасам показывает раздел «Ближайшие ТО».',
      'Каждое число — кнопка: открывается перечень этих машин. Из перечня можно открыть карточку или «Сформировать задание» — печатную форму со сроками, ответственными и графой отметки о выполнении.',
      '«Доклад о состоянии парка» — печатная сводка: состояние, КТГ по подразделениям, техника с просроченным ТО.']),
    h3('12.2 Как защищены данные'),
    tbl(['Механизм', 'Что делает'], [
      ['Резервная копия (ZIP)', 'Полная копия с вложениями. Единственная защита от поломки компьютера — храните её вне компьютера. Напоминание — в пункте управления, обзоре и при входе.'],
      ['Точки восстановления', 'Снимки базы в браузере: ежедневно и автоматически перед восстановлением из копии, очисткой данных, обновлением структуры, исправлением ошибок, откатом и очисткой корзины. Хранится до 20 автоматических и 20 ручных. Вложения в снимки не входят, но не удаляются, пока на них ссылается хоть одна точка.'],
      ['Корзина', 'Удалённые карточки, записи ТО, путевые листы и показания хранятся заданное в настройках число дней (по умолчанию 30) и возвращаются вместе со связанными записями.'],
      ['Журнал изменений', 'Кто бы ни правил карточку, сохраняются прежнее и новое значение важных полей: наименование, номера, подразделение, ответственный, статус, группа, тип по справочнику, даты. Последние изменения видны в карточке техники.'],
      ['Проверка перед импортом', 'Перед восстановлением из копии или откатом показывается, сколько записей будет загружено, и результат проверки целостности. Ошибки можно исправить автоматически.'],
      ['Версия структуры данных', 'При обновлении программы данные переводятся на новую структуру автоматически; перед этим создаётся точка восстановления. Данные из более новой версии программы не понижаются — показывается предупреждение.']]),
    h3('12.3 Восстановление после сбоя'),
    ol(['Ошибочно удалили — «Сохранность данных» → «Корзина» → «Восстановить».',
      'Ошибочно восстановили не ту копию или обновление прошло неудачно — «Точки восстановления» → «Вернуться» к точке «Перед восстановлением из копии» или «Перед обновлением структуры данных».',
      'Браузер очищен или компьютер заменён — откройте программу и «Восстановить из копии» последний ZIP-файл.',
      'Проверка целостности нашла ошибки — «Исправить автоматически» (перед исправлением создаётся точка восстановления).']),
    h3('12.4 Модули платформы'),
    p('Программа разделена на модули: справочник, парк, ТО, ремонты и готовность, путевые листы и моточасы, склад (запланирован), документы, пункт управления, QR-маркировка, сохранность данных. Каждый модуль владеет своими данными, имеет версию и самотесты. «Запустить самотесты» проверяет модули на копиях данных — реальные данные при этом не меняются.'),
    p('Идентификатор записи: <вид>_<время>_<случайная часть> (например, v_… — техника, m_… — запись ТО). Он присваивается один раз и не меняется: на нём держатся связи, журнал изменений и QR-код.'),
    h3('12.5 Шаблоны работ ТО'),
    p('В форме записи ТО, если для типа техники и выбранного вида ТО в справочнике есть регламент, показывается перечень его операций с галочками. Отметьте выполненные работы: счётчик «выполнено N из M» ведётся автоматически. Если поле «Выполненные работы» оставить пустым, в него подставятся отмеченные операции. Перечень сохраняется в записи как снимок: если регламент в справочнике потом изменят, история обслуживания останется прежней. В журнале ТО у записи видно «По регламенту: N из M», а при наведении — какие операции не выполнены. Кнопка «Подставить материалы» переносит в запись перечень материалов регламента.'),
    h3('12.6 Сезонное обслуживание'),
    p('В настройках задаются даты перевода всей техники на летний и зимний режим и число дней, за которое нужно напоминать. Даты задаются приказом или решением командира; значения по умолчанию (15 апреля и 15 октября) — лишь начальная подстановка, их нужно проверить. За заданное число дней до даты в колоколе, при входе в учёт и в «Пункте управления» появляется напоминание: сколько машин ещё не прошли сезонное обслуживание. Напоминание остаётся 60 дней после даты с пометкой «просрочено», затем снимается.'),
    ul(['СО засчитывается, если в журнале ТО есть запись вида «Сезонное обслуживание (СО)» с отметкой сезона (летний или зимний) в пределах 120 дней от даты.',
      'Не учитываются списанная техника и техника на хранении и консервации.',
      'Плитка «Сезонное ТО не проведено» в «Пункте управления» открывает перечень машин, из него можно сформировать печатное «Задание на проведение сезонного обслуживания».']),
    h3('12.7 Приложение для Windows и файлы данных'),
    p('В установленном приложении данные хранятся не в браузере, а в обычных файлах — в папке «Документы\\Регламент-ТС» (переносная версия — рядом с программой). Основной файл db.json, суточные копии (последние 14), вложения и точки восстановления лежат в подпапках; всю папку можно скопировать на флешку или на другой компьютер. Если db.json окажется повреждённым, приложение само возьмёт последнюю исправную суточную копию, а повреждённый файл сохранит рядом. Папку данных можно открыть и сменить в разделе «Сохранность данных → Файлы данных приложения».'),
    p('Данные из обычного браузера переносятся один раз: в браузере — «Создать резервную копию», в приложении — «Восстановить из копии».')
  ]);
  sections.forEach(x => { x.text = x.el.textContent; });
  return sections;
}

let RW = null;
function copyText(t) {
  if (navigator.clipboard && navigator.clipboard.writeText) return navigator.clipboard.writeText(t);
  return new Promise((res, rej) => {
    const ta = E('textarea', { style: 'position:fixed;left:-999px' });
    ta.value = t; document.body.appendChild(ta); ta.select();
    try { document.execCommand('copy') ? res() : rej(); } catch (e) { rej(e); }
    ta.remove();
  });
}
function openRef(secId, anchor) {
  if (RW) { RW.go(secId, anchor); return; }
  const secs = buildHelp();
  if (!secs) { toast('Справка недоступна', 'Модуль учёта ещё не загружен.', 'warning'); return; }
  let marks = [], mi = -1, curSecId = secs[0].id;
  let hist = lsGet('refhist', []), books = lsGet('refbooks', []);
  const known = id => secs.some(x => x.id === id);
  hist = hist.filter(h => known(h.id)); books = books.filter(b => known(b.id));
  const search = E('input', { type: 'search', id: 'rw-search', placeholder: 'Поиск по справке (Ctrl+F)', 'aria-label':
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
 'Поиск по справке' });
1014      const count = E('span', { class: 'xnote', style: 'min-width:90px', 'aria-live': 'polite' });
1015      const content = E('div', { class: 'rw-content', tabindex: '0' }, secs.map(x => x.el));
1016      const toc = E('nav', { class: 'rw-toc', 'aria-label': 'Оглавление' });
1017      const btn = (label, fn, cls, title) => E('button', { type: 'button', class: 'xbtn sm ' + (cls || ''), text: label,
    title: title || label, onclick: fn });
1018      const bookBtn = btn('☆ Закладка', () => toggleBook(), '', 'Добавить в закладки (Ctrl+B)');
1019      const short = t => t.replace(/^Раздел \d+\. /, '');
1020      function drawToc() {
1021        toc.replaceChildren(E('h5', { text: 'Разделы' }));
1022        secs.forEach(x => {
1023          const n = marks.filter(m => x.el.contains(m)).length;
1024          toc.appendChild(E('button', { type: 'button', class: x.id === curSecId ? 'cur' : '', 'aria-current': x.id ===
    curSecId ? 'true' : null, onclick: () => go(x.id) }, [E('span', { text: short(x.title) }), n ? E('span', { class: 'n',
     text: String(n) }) : null]));
1025        });
1026        toc.appendChild(E('h5', { text: 'Закладки' }));
1027        if (!books.length) toc.appendChild(E('span', { class: 'xnote', style: 'padding:0 10px', text: 'Нет закладок' }));
1028        books.forEach(b => toc.appendChild(E('button', { type: 'button', onclick: () => go(b.id) }, [E('span', { text: '★
    ' + short(b.title) })])));
1029        toc.appendChild(E('h5', { text: 'История просмотров' }));
1030        if (!hist.length) toc.appendChild(E('span', { class: 'xnote', style: 'padding:0 10px', text: 'Пока пусто' }));
1031        hist.slice(0, 6).forEach(b => toc.appendChild(E('button', { type: 'button', onclick: () => go(b.id) }, [E('span',
    { text: short(b.title) })])));
1032        const isB = books.some(b => b.id === curSecId);
1033        bookBtn.textContent = (isB ? '★' : '☆') + ' Закладка';
1034      }
1035      function pushHist(id) {
1036        const x = secs.filter(y => y.id === id)[0];
1037        hist = [{ id, title: x.title }].concat(hist.filter(h => h.id !== id)).slice(0, 15);
1038        lsSet('refhist', hist);
1039      }
1040      function go(id, a) {
1041        const x = secs.filter(y => y.id === id)[0] || secs[0];
1042        curSecId = x.id; pushHist(x.id);
1043        const t = a && document.getElementById(a) ? document.getElementById(a) : x.el;
1044        t.scrollIntoView({ block: 'start' }); drawToc();
1045      }
1046      function toggleBook() {
1047        const i = books.findIndex(b => b.id === curSecId);
1048        const x = secs.filter(y => y.id === curSecId)[0];
1049        if (i >= 0) { books.splice(i, 1); toast('Закладка удалена', short(x.title)); }
1050        else { books.push({ id: x.id, title: x.title }); toast('Раздел добавлен в закладки', short(x.title), 'success'); }
1051        lsSet('refbooks', books); drawToc();
1052      }
1053      function clearMarks() {
1054        content.querySelectorAll('mark.hit').forEach(m => { const pn = m.parentNode;
    pn.replaceChild(document.createTextNode(m.textContent), m); pn.normalize(); });
1055        marks = []; mi = -1;
1056      }
1057      function doSearch() {
1058        clearMarks();
1059        const q = search.value.trim();
1060        if (q.length < 2) { count.textContent = ''; drawToc(); return; }
1061        const re = new RegExp(q.replace(/[.*+?^${}()|[\]\\]/g, '\\$&'), 'gi');
1062        const walker = document.createTreeWalker(content, NodeFilter.SHOW_TEXT, { acceptNode: n =>
    n.parentNode.closest('button,script,style') ? NodeFilter.FILTER_REJECT : (re.lastIndex = 0, re.test(n.nodeValue) ?
    NodeFilter.FILTER_ACCEPT : NodeFilter.FILTER_REJECT) });
1063        const nodes = [];
1064        while (walker.nextNode()) nodes.push(walker.currentNode);
1065        nodes.forEach(n => {
1066          const txt = n.nodeValue, frag = document.createDocumentFragment();
1067          let last = 0, m;
1068          re.lastIndex = 0;
1069          while ((m = re.exec(txt))) {
1070            frag.appendChild(document.createTextNode(txt.slice(last, m.index)));
1071            const mk = E('mark', { class: 'hit', text: m[0] });
1072            frag.appendChild(mk); marks.push(mk);
1073            last = m.index + m[0].length;
1074            if (m[0].length === 0) re.lastIndex++;
1075          }
1076          frag.appendChild(document.createTextNode(txt.slice(last)));
1077          n.parentNode.replaceChild(frag, n);
1078        });
1079        marks.forEach(mk => { const d = mk.closest('details'); if (d) d.open = true; });
1080        count.textContent = marks.length ? 'Найдено: ' + marks.length : 'Не найдено';
1081        drawToc();
1082        if (marks.length) step(1);
1083      }
1084      function step(d) {
1085        if (!marks.length) return;
1086        if (mi >= 0) marks[mi].classList.remove('cur');
1087        mi = (mi + d + marks.length) % marks.length;
1088        marks[mi].classList.add('cur'); marks[mi].scrollIntoView({ block: 'center' });
1089        count.textContent = (mi + 1) + ' из ' + marks.length;
1090      }
1091      search.addEventListener('input', doSearch);
1092      search.addEventListener('keydown', e => { if (e.key === 'Enter') { e.preventDefault(); step(e.shiftKey ? -1 : 1); }
    });
1093      const secText = ids => secs.filter(x => ids.indexOf(x.id) >= 0).map(x => x.el.innerText).join('\n\n');
```

7. Окно печати и справка (index.html, скрипт 7) (продолжение)

```
  function printRef(all, pdf) {
    const ids = all ? secs.map(x => x.id) : [curSecId];
    clearMarks(); count.textContent = '';
    const nodes = secs.filter(x => ids.indexOf(x.id) >= 0).map(x => {
      const c = x.el.cloneNode(true);
      c.removeAttribute('id');
      c.querySelectorAll('details').forEach(d => { d.open = true; });
      return c;
    });
    logPrint(all ? 'Справка по учёту целиком' : 'Справка: ' + short(secs.filter(x => x.id === curSecId)[0].title), pdf ? 'pdf' : 'print');
    if (pdf) toast('Сохранение в PDF', 'В окне печати выберите «Сохранить как PDF».', 'success');
    sendToPrinter(nodes, '@page{size:A4 portrait;margin:15mm 12mm 15mm 15mm;@bottom-center{content:"Страница " counter(page) " из " counter(pages);font:9pt serif}}');
  }
  const bar = E('div', { class: 'rw-bar' }, [search, count, btn('▲', () => step(-1), '', 'Предыдущее совпадение'), btn('▼', () => step(1), '', 'Следующее совпадение'), bookBtn,
    btn('Копировать', () => copyText(secText([curSecId])).then(() => toast('Текст раздела скопирован', '')), () => toast('Не удалось скопировать', '', 'warning')), '', 'Копировать текст текущего раздела'),
    btn('Печать раздела', () => printRef(false)), btn('Печать всего', () => printRef(true)), btn('Экспорт в PDF', () => printRef(true, true), 'fill')]);
  const main = E('div', { class: 'rw-main', style: 'height:100%;margin:0' }, [toc, content]);
  const api = xwin({ title: 'Справка — учёт технических средств тыла', cls: 'rw', bar, body: main, onClose: () => { RW = null; document.removeEventListener('keydown', onKey, true); } });
  api.body.style.cssText = 'padding:0;overflow:hidden;display:block';
  const obs = window.IntersectionObserver ? new IntersectionObserver(es => es.forEach(en => {
    if (en.isIntersecting) { const id = en.target.id.replace('ref-', ''); if (id !== curSecId) { curSecId = id; drawToc(); } }
  }), { root: content, rootMargin: '0px 0px -75% 0px' }) : null;
  if (obs) secs.forEach(x => obs.observe(x.el));
  function onKey(e) {
    if (stack[stack.length - 1] !== api) return;
    if (e.key === 'Escape') { e.preventDefault(); e.stopPropagation(); api.close(); return; }
    if (e.key === 'F1') { e.preventDefault(); RW.focusSearch(); return; }
    if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'f') { e.preventDefault(); RW.focusSearch(); }
    else if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'b') { e.preventDefault(); toggleBook(); }
    else if (e.key === 'Tab') {
      const f = [].slice.call(api.el.querySelectorAll('button:not(:disabled),input,summary,[tabindex="0"]')).filter(x => x.offsetParent !== null);
      const i = f.indexOf(document.activeElement);
      if (e.shiftKey && i <= 0) { e.preventDefault(); f[f.length - 1].focus(); }
      else if (!e.shiftKey && (i === f.length - 1 || i < 0)) { e.preventDefault(); f[0].focus(); }
    }
  }
  document.addEventListener('keydown', onKey, true);
  RW = { go: (id, a) => { if (id) go(id, a); }, focusSearch: () => { search.focus(); search.select(); }, close: api.close };
  drawToc(); go(secId || secs[0].id, anchor);
  setTimeout(() => search.focus(), 60);
}

/* ---------- горячие клавиши: F1 — справка, Ctrl+P — печать в учёте ---------- */
document.addEventListener('keydown', e => {
  if (!window.TST || !window.TST.isApp() || PW || stack.length) return;
  if (document.querySelector('.f-modal-backdrop')) return;
  if (e.key === 'F1') { e.preventDefault(); openRef(); return; }
  if ((e.ctrlKey || e.metaKey) && !e.altKey && e.key.toLowerCase() === 'p' && window.TST.workspace === 'fleet' && window.TST.fleet && window.TST.fleet.openPrint) {
    e.preventDefault(); window.TST.fleet.openPrint();
  }
});

window.TSTPrint = { open: openPrintWin, isOpen: () => !!PW };
window.TSTHelp = { open: openRef };
})();
```

8. Витрина и навигация по разделам (index.html, скрипт 8)

```
/* ========================================================================
   ЛЕНДИНГ: навигация по разделам витрины и вход в приложение.
   Переход лендинг ⇄ приложение — только через TST.showApp / TST.showLanding
   ======================================================================== */
(() => {
  const root = document.getElementById('landing');
  if (!root) return;
  const sections = root.querySelectorAll('.ld-section');
  const links = root.querySelectorAll('.ld-nav-link, .ld-mobile-link');
  const btn = root.querySelector('.ld-menu-btn');
  const overlay = root.querySelector('.ld-overlay');
  const menu = root.querySelector('.ld-mobile-menu');
  let open = false;

  const select = t => {
    sections.forEach(x => x.classList.toggle('active', x.id === 'section-' + t));
    links.forEach(x => {
      const on = x.dataset.target === t;
      x.classList.toggle('active', on);
      if (on) x.setAttribute('aria-current', 'page'); else x.removeAttribute('aria-current');
    });
  };
  const toggle = force => {
    open = typeof force === 'boolean' ? force : !open;
    btn?.classList.toggle('open', open);
    overlay?.classList.toggle('show', open);
    menu?.classList.toggle('show', open);
    btn?.setAttribute('aria-expanded', String(open));
  };

  links.forEach(x => x.addEventListener('click', () => { select(x.dataset.target); if (open) toggle(false); }));
  btn?.addEventListener('click', () => toggle());
  overlay?.addEventListener('click', () => toggle(false));
  document.addEventListener('keydown', e => { if (e.key === 'Escape' && open) toggle(false); });

  // Вторичные кнопки лендинга: переход между его разделами
  root.querySelectorAll('[data-go-section]').forEach(b =>
    b.addEventListener('click', () => select(b.dataset.goSection)));

  // Контакты: копирование в буфер обмена (с запасным вариантом для старых браузеров)
  const status = root.querySelector('.ld-copy-status');
  root.querySelectorAll('[data-copy]').forEach(b => b.addEventListener('click', async () => {
    const text = b.dataset.copy;
    let ok = false;
    try { await navigator.clipboard.writeText(text); ok = true; }
    catch (e) {
      const t = document.createElement('textarea');
      t.value = text; t.setAttribute('readonly', ''); t.style.cssText = 'position:fixed;opacity:0';
      document.body.appendChild(t); t.select();
      try { ok = document.execCommand('copy'); } catch (e2) { ok = false; }
      t.remove();
    }
    if (status) status.textContent = ok ? 'Скопировано: ' + text : 'Не удалось скопировать — выделите текст вручную';
    clearTimeout(status?._t); if (status) status._t = setTimeout(() => { status.textContent = ''; }, 2600);
  }));

  // Фоновое видео (~13 МБ) подгружается после отрисовки и только там, где оно уместно:
  // широкий экран, нет «экономии трафика» и prefers-reduced-motion. Иначе остаётся градиент
  const video = root.querySelector('.ld-bg-video');
  const loadVideo = () => {
    const vdata = document.getElementById('ldVideoData');   // лежит в конце файла — ищем уже после загрузки
    if (!video || !vdata || video.getAttribute('src')) return;
    const conn = navigator.connection || {};
    if (conn.saveData || matchMedia('(prefers-reduced-motion: reduce)').matches || !matchMedia('(min-width:
768px)').matches) return;
    try {
      const bin = atob(vdata.textContent.trim());
      const u8 = new Uint8Array(bin.length);
      for (let i = 0; i < bin.length; i++) u8[i] = bin.charCodeAt(i);
      video.src = URL.createObjectURL(new Blob([u8], { type: 'video/mp4' }));
      vdata.textContent = '';
      if (!TST.isApp()) video.play().catch(() => {});
    } catch (e) { /* без видео остаётся градиентный фон */ }
  };
  const idle = fn => (window.requestIdleCallback ? requestIdleCallback(fn, { timeout: 1500 }) : setTimeout(fn, 400));
  if (document.readyState === 'complete') idle(loadVideo);
  else window.addEventListener('load', () => idle(loadVideo));

  // «Начать использовать» → Обзор; вторичные кнопки — сразу в модуль
  root.querySelectorAll('[data-open-app]').forEach(b =>
    b.addEventListener('click', () => { toggle(false); TST.showApp(b.dataset.openApp); }));
  document.getElementById('backToLanding')?.addEventListener('click', () => TST.showLanding());

  // Адрес страницы → экран. Запуск после инициализации всех модулей
  // (их обработчики DOMContentLoaded зарегистрированы раньше этого)
  const startRouter = () => TST.initRouter();
  if (document.readyState === 'loading') document.addEventListener('DOMContentLoaded', startRouter);
  else startRouter();
})();
```

9. app/main.js

```
'use strict';
const { app, BrowserWindow, Menu, dialog, ipcMain, shell, session } = require('electron');
const path = require('path');
const fs = require('fs');
const { pathToFileURL } = require('url');
const { createStorage } = require('./storage');

app.setAppUserModelId('ru.reglament.ts');
/* Переносной режим: рядом с программой лежит файл portable.flag (или запуск из portable-exe
   electron-builder). Данные и профиль приложения хранятся рядом — переносятся вместе с флешкой */
const exeDir = process.env.PORTABLE_EXECUTABLE_DIR || path.dirname(process.execPath);
const PORTABLE = !!process.env.PORTABLE_EXECUTABLE_DIR || fs.existsSync(path.join(exeDir, 'portable.flag'));
const PORTABLE_DIR = path.join(exeDir, 'Данные Регламент-ТС');
if (PORTABLE) app.setPath('userData', path.join(PORTABLE_DIR, 'profile'));
if (process.env.TST_USER_DATA) app.setPath('userData', process.env.TST_USER_DATA);   // для автотестов
if (process.env.TST_TEST) app.disableHardwareAcceleration();

if (!app.requestSingleInstanceLock()) { app.quit(); process.exit(0); }

const WWW = path.join(__dirname, 'www', 'index.html');
const WWW_URL = pathToFileURL(WWW).href;

/* --- настройки приложения (папка данных) --- */
const cfgPath = () => path.join(app.getPath('userData'), 'config.json');
const readCfg = () => { try { return JSON.parse(fs.readFileSync(cfgPath(), 'utf8')); } catch (e) { return {}; } };
const writeCfg = c => { fs.mkdirSync(path.dirname(cfgPath()), { recursive: true }); fs.writeFileSync(cfgPath(),
JSON.stringify(c, null, 1)); };
function defaultDataDir() {
  if (PORTABLE) return path.join(PORTABLE_DIR, 'db');
  return path.join(app.getPath('documents'), 'Регламент-ТС');
}
const resolveDataDir = () => process.env.TST_DATA_DIR || readCfg().dataDir || defaultDataDir();

let storage = null, win = null, dataDir = '';

/* Принимаем запросы только от нашей страницы */
const trusted = e => { try { return !!e.senderFrame && e.senderFrame.url.startsWith(WWW_URL.split('#')[0]); } catch
(x) { return false; } };
const guard = fn => (e, ...a) => { if (!trusted(e)) throw new Error('Запрос отклонён'); return fn(e, ...a); };

function registerIpc() {
  ipcMain.on('tst:init', e => {
    if (!trusted(e)) { e.returnValue = null; return; }
    let r = { raw: null };
    try { r = storage.readDb(); } catch (err) { r = { raw: null, error: String(err && err.message || err) }; }
    e.returnValue = Object.assign({ version: app.getVersion(), platform: process.platform, dataDir }, r);
  });
  ipcMain.handle('tst:db-write', guard((e, raw) => storage.writeDb(raw)));
  ipcMain.handle('tst:file-put', guard((e, id, meta, bytes) => storage.putFile(id, meta, bytes)));
  ipcMain.handle('tst:file-get', guard((e, id) => storage.getFile(id)));
  ipcMain.handle('tst:file-del', guard((e, id) => storage.delFile(id)));
  ipcMain.handle('tst:file-list', guard(() => storage.listFiles()));
  ipcMain.handle('tst:snap-put', guard((e, id, rec) => storage.putSnapshot(id, rec)));
  ipcMain.handle('tst:snap-list', guard(() => storage.listSnapshots()));
  ipcMain.handle('tst:snap-del', guard((e, id) => storage.delSnapshot(id)));
  ipcMain.handle('tst:open-dir', guard(() => shell.openPath(dataDir)));
  ipcMain.handle('tst:choose-dir', guard(async () => {
    if (process.env.TST_DATA_DIR) return { ok: false, error: 'Папка задана переменной окружения' };
    const r = await dialog.showOpenDialog(win, { title: 'Папка данных «Регламент-ТС»', defaultPath: dataDir,
properties: ['openDirectory', 'createDirectory'] });
    if (r.canceled || !r.filePaths[0]) return { ok: false, canceled: true };
    const target = r.filePaths[0];
    if (path.resolve(target) === path.resolve(dataDir)) return { ok: false, canceled: true };
    // Если в выбранной папке уже есть данные — используем их; иначе переносим текущие
    const hasData = fs.existsSync(path.join(target, 'db.json'));
    if (!hasData) storage.copyAllTo(target);
    writeCfg(Object.assign(readCfg(), { dataDir: target }));
    setTimeout(() => { app.relaunch(); app.exit(0); }, 300);
    return { ok: true, dir: target, copied: !hasData };
  }));
}

function buildMenu() {
  const template = [
    { label: 'Файл', submenu: [
      { label: 'Открыть папку данных', click: () => shell.openPath(dataDir) },
      { type: 'separator' },
      { label: 'Выход', role: 'quit' }] },
    { label: 'Правка', submenu: [
      { role: 'undo', label: 'Отменить' }, { role: 'redo', label: 'Повторить' }, { type: 'separator' },
      { role: 'cut', label: 'Вырезать' }, { role: 'copy', label: 'Копировать' }, { role: 'paste', label: 'Вставить' },
      { role: 'selectAll', label: 'Выделить всё' }] },
    { label: 'Вид', submenu: [
      { role: 'resetZoom', label: 'Обычный масштаб' }, { role: 'zoomIn', label: 'Увеличить' }, { role: 'zoomOut',
label: 'Уменьшить' },
      { type: 'separator' }, { role: 'togglefullscreen', label: 'Во весь экран' }].concat(app.isPackaged ? [] : [{
role: 'toggleDevTools', label: 'Инструменты разработчика' }]) },
    { label: 'Справка', submenu: [
      { label: 'О программе', click: () => dialog.showMessageBox(win, { type: 'info', title: 'О программе', message:
'Регламент-ТС ' + app.getVersion(),
```

9. app/main.js (продолжение)

```
            detail: 'Автоматизированная система учёта и контроля технических средств тыла.\nРаботает без интернета.\n\nПапка данных:\n' + dataDir, buttons: ['Закрыть'] }) }] }
    ];
    Menu.setApplicationMenu(Menu.buildFromTemplate(template));
  }

  function createWindow() {
    const bounds = readCfg().bounds || {};
    win = new BrowserWindow({
      width: bounds.width || 1440, height: bounds.height || 900, minWidth: 1000, minHeight: 640,
      show: false, autoHideMenuBar: true, backgroundColor: '#0b0b0d', title: 'Регламент-ТС',
      icon: path.join(__dirname, 'build', 'icon.png'),
      webPreferences: { preload: path.join(__dirname, 'preload.js'), contextIsolation: true, nodeIntegration: false, sandbox: true, spellcheck: false }
    });
    if (bounds.maximized) win.maximize();
    win.once('ready-to-show', () => win.show());
    win.on('close', () => {
      try { const b = win.getNormalBounds(); writeCfg(Object.assign(readCfg(), { bounds: { width: b.width, height: b.height, maximized: win.isMaximized() } })); } catch (e) { /* не критично */ }
    });
    // Внешние ссылки (mailto, https) открываются в системе; сама страница никуда не уходит
    win.webContents.setWindowOpenHandler(({ url }) => { if (/^(mailto:|https:)/i.test(url)) shell.openExternal(url); return { action: 'deny' }; });
    win.webContents.on('will-navigate', (e, url) => {
      if (!url.startsWith(WWW_URL.split('#')[0])) { e.preventDefault(); if (/^(mailto:|https:)/i.test(url)) shell.openExternal(url); }
    });
    win.loadFile(WWW);
  }

  app.on('second-instance', () => { if (win) { if (win.isMinimized()) win.restore(); win.focus(); } });
  app.on('window-all-closed', () => app.quit());

  app.whenReady().then(() => {
    dataDir = resolveDataDir();
    try { storage = createStorage(dataDir); }
    catch (e) {
      dialog.showErrorBox('Папка данных недоступна', `Не удалось открыть папку данных:\n${dataDir}\n\n${e.message}\n\nПроверьте путь или удалите настройку в ${cfgPath()}.`);
      app.exit(1); return;
    }
    // Камера нужна только для сканирования QR-кодов; остальные разрешения запрещены
    session.defaultSession.setPermissionRequestHandler((wc, permission, cb) => cb(permission === 'media'));
    session.defaultSession.setPermissionCheckHandler((wc, permission) => permission === 'media');
    if (process.env.TST_DOWNLOAD_DIR) session.defaultSession.on('will-download', (e, item) => item.setSavePath(path.join(process.env.TST_DOWNLOAD_DIR, item.getFilename())));
    registerIpc();
    buildMenu();
    createWindow();
  });

```

10. app/preload.js

```
'use strict';
/* Мост между программой (index.html) и приложением. Работает в изолированном
   контексте: программе отдаются только перечисленные ниже функции. */
const { contextBridge, ipcRenderer } = require('electron');

const init = ipcRenderer.sendSync('tst:init') || {};
const call = (ch, ...args) => ipcRenderer.invoke(ch, ...args);

contextBridge.exposeInMainWorld('TSTNative', {
  isNative: true,
  version: init.version || '',
  platform: init.platform || '',
  dataDir: init.dataDir || '',
  initialDb: typeof init.raw === 'string' ? init.raw : null,   // основной файл данных, прочитанный до старта программы
  recovered: init.recovered || null,                           // если файл был повреждён и взят из копии
  saveDb: raw => call('tst:db-write', raw),
  putFile: (id, meta, bytes) => call('tst:file-put', id, meta, bytes),
  getFile: id => call('tst:file-get', id),
  delFile: id => call('tst:file-del', id),
  listFiles: () => call('tst:file-list'),
  putSnapshot: (id, rec) => call('tst:snap-put', id, rec),
  listSnapshots: () => call('tst:snap-list'),
  delSnapshot: id => call('tst:snap-del', id),
  openDataDir: () => call('tst:open-dir'),
  chooseDataDir: () => call('tst:choose-dir')
});

```

11. app/storage.js

```
'use strict';
/* Файловое хранилище приложения «Регламент-ТС» (без зависимостей от Electron —
   проверяется обычными тестами Node).

   Папка данных:
     db.json            — все записи учёта (основной файл)
     db.prev.json       — предыдущая версия db.json (перед каждой записью)
     backups/           — суточные копии db.json (последние 14)
     files/<id>.bin     — вложения (содержимое) и files/<id>.json (имя, тип, размер)
     snapshots/<id>.json — точки восстановления

   Запись db.json атомарная: во временный файл с fsync, затем переименование.
   Повреждённый db.json не удаляется, а откладывается как db.corrupt-*.json;
   данные берутся из db.prev.json (теряется только последнее изменение), а если
   и он не годится — из последней исправной суточной копии. */
const fs = require('fs');
const path = require('path');

const ID_RE = /^[A-Za-z0-9_-]{4,96}$/;
const KEEP_BACKUPS = 14;

function assertId(id) {
  if (typeof id !== 'string' || !ID_RE.test(id)) throw new Error('Недопустимый идентификатор: ' + String(id).slice(0, 40));
  return id;
}
const pad = n => String(n).padStart(2, '0');
const dayStamp = (d = new Date()) => `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;
const fullStamp = (d = new Date()) => `${dayStamp(d)}_${pad(d.getHours())}${pad(d.getMinutes())}${pad(d.getSeconds())}`;

function isValidDb(raw) {
  if (typeof raw !== 'string' || raw.length < 2 || raw[0] !== '{') return false;
  try { const o = JSON.parse(raw); return !!o && typeof o === 'object' && !Array.isArray(o); } catch (e) { return false; }
}

function createStorage(dir) {
  const P = {
    db: path.join(dir, 'db.json'),
    prev: path.join(dir, 'db.prev.json'),
    tmp: path.join(dir, 'db.json.tmp'),
    backups: path.join(dir, 'backups'),
    files: path.join(dir, 'files'),
    snaps: path.join(dir, 'snapshots')
  };
  [dir, P.backups, P.files, P.snaps].forEach(d => fs.mkdirSync(d, { recursive: true }));

  const listBackups = () => fs.readdirSync(P.backups).filter(n => /^db-.*\.json$/.test(n)).sort().reverse();

  function readValid(file) {
    try { const raw = fs.readFileSync(file, 'utf8'); return isValidDb(raw) ? raw : null; } catch (e) { return null; }
  }

  /* Прочитать базу: { raw, recovered? }. raw === null — данных ещё нет */
  function readDb() {
    const exists = fs.existsSync(P.db);
    if (exists) {
      const raw = readValid(P.db);
      if (raw) return { raw };
      // Файл есть, но повреждён: сохраняем его для разбора и берём копию
      const moved = `db.corrupt-${fullStamp()}.json`;
      try { fs.renameSync(P.db, path.join(dir, moved)); } catch (e) { /* не критично */ }
      const rec = fromBackups('повреждён');
      return rec ? Object.assign(rec, { recovered: Object.assign(rec.recovered, { corruptFile: moved }) }) : { raw: null, recovered: { reason: 'повреждён', from: null, corruptFile: moved } };
    }
    const rec = fromBackups('отсутствует');
    return rec || { raw: null };
  }
  function fromBackups(reason) {
    const candidates = [{ name: 'db.prev.json', file: P.prev }].concat(listBackups().map(n => ({ name: n, file: path.join(P.backups, n) })));
    for (const c of candidates) {
      const name = c.name;
      const raw = readValid(c.file);
      if (raw) {
        // восстановленное состояние сразу становится основным файлом
        fs.writeFileSync(P.db, raw);
        return { raw, recovered: { reason, from: name } };
      }
    }
    return null;
  }

  /* Суточная копия — перед первой записью за день */
  function rotate() {
    if (!fs.existsSync(P.db)) return;
    const name = `db-${dayStamp()}.json`;
    const dest = path.join(P.backups, name);
```

11. app/storage.js (продолжение)

```
86          if (!fs.existsSync(dest) && readValid(P.db)) fs.copyFileSync(P.db, dest);
87          listBackups().slice(KEEP_BACKUPS).forEach(n => { try { fs.unlinkSync(path.join(P.backups, n)); } catch (e) { /* ок
        */ } });
88        }
89
90        function writeDb(raw) {
91          if (!isValidDb(raw)) throw new Error('Данные для записи не являются корректным JSON-объектом');
92          rotate();
93          const fd = fs.openSync(P.tmp, 'w');
94          try { fs.writeSync(fd, raw, null, 'utf8'); fs.fsyncSync(fd); } finally { fs.closeSync(fd); }
95          // Прежняя исправная версия остаётся как db.prev.json
96          if (readValid(P.db)) fs.copyFileSync(P.db, P.prev);
97          fs.renameSync(P.tmp, P.db);
98          return true;
99        }
100
101       /* --- вложения --- */
102       function putFile(id, meta, bytes) {
103         assertId(id);
104         const buf = Buffer.isBuffer(bytes) ? bytes : Buffer.from(bytes);
105         fs.writeFileSync(path.join(P.files, id + '.bin'), buf);
106         fs.writeFileSync(path.join(P.files, id + '.json'), JSON.stringify({ name: String((meta && meta.name) || ''), type:
       String((meta && meta.type) || ''), size: buf.length }));
107         return true;
108       }
109       function getFile(id) {
110         assertId(id);
111         const bin = path.join(P.files, id + '.bin');
112         if (!fs.existsSync(bin)) return null;
113         let meta = {};
114         try { meta = JSON.parse(fs.readFileSync(path.join(P.files, id + '.json'), 'utf8')); } catch (e) { /* без
    метаданных */ }
115         return { meta, bytes: fs.readFileSync(bin) };
116       }
117       function delFile(id) {
118         assertId(id);
119         ['.bin', '.json'].forEach(ext => { try { fs.unlinkSync(path.join(P.files, id + ext)); } catch (e) { /* уже нет */
    } });
120         return true;
121       }
122       const listFiles = () => fs.readdirSync(P.files).filter(n => n.endsWith('.bin')).map(n => n.slice(0, -4));
123
124       /* --- точки восстановления --- */
125       function putSnapshot(id, rec) {
126         assertId(id);
127         fs.writeFileSync(path.join(P.snaps, id + '.json'), JSON.stringify(rec));
128         return true;
129       }
130       function listSnapshots() {
131         return fs.readdirSync(P.snaps).filter(n => n.endsWith('.json')).map(n => {
132           try { return JSON.parse(fs.readFileSync(path.join(P.snaps, n), 'utf8')); } catch (e) { return null; }
133         }).filter(x => x && x.id);
134       }
135       function delSnapshot(id) {
136         assertId(id);
137         try { fs.unlinkSync(path.join(P.snaps, id + '.json')); } catch (e) { /* уже нет */ }
138         return true;
139       }
140
141       /* Копия всех данных в другую папку (при смене папки данных) */
142       function copyAllTo(target) {
143         fs.mkdirSync(target, { recursive: true });
144         const copy = (from, to) => {
145           fs.mkdirSync(to, { recursive: true });
146           for (const n of fs.readdirSync(from)) {
147             const a = path.join(from, n), b = path.join(to, n);
148             if (fs.statSync(a).isDirectory()) copy(a, b); else fs.copyFileSync(a, b);
149           }
150         };
151         copy(dir, target);
152       }
153
154       return { dir, readDb, writeDb, putFile, getFile, delFile, listFiles, putSnapshot, listSnapshots, delSnapshot,
    copyAllTo, isValidDb };
155     }
156
157     module.exports = { createStorage, ID_RE, isValidDb };
158
```

12. app/package.json

```
{
  "name": "reglament-ts",
  "productName": "Регламент-ТС",
  "version": "1.0.0",
  "description": "Автоматизированная система учёта и контроля технических средств тыла",
  "main": "main.js",
  "author": "Регламент-ТС",
  "license": "UNLICENSED",
  "private": true,
  "scripts": {
    "prepare-www": "node scripts/prepare-www.js",
    "start": "npm run prepare-www && electron .",
    "test": "node test/storage.test.js",
    "dist": "npm run prepare-www && electron-builder --win --x64 --publish never",
    "dist:dir": "npm run prepare-www && electron-builder --win --x64 --dir --publish never"
  },
  "devDependencies": {
    "electron": "^45.0.0",
    "electron-builder": "^26.15.0"
  },
  "build": {
    "appId": "ru.reglament.ts",
    "productName": "Регламент-ТС",
    "copyright": "Регламент-ТС",
    "directories": { "output": "dist", "buildResources": "build" },
    "files": ["main.js", "preload.js", "storage.js", "build/icon.png", "www/index.html", "package.json"],
    "asar": true,
    "electronLanguages": ["ru", "en-US"],
    "compression": "normal",
    "win": {
      "icon": "build/icon.ico",
      "target": [
        { "target": "nsis", "arch": ["x64"] },
        { "target": "portable", "arch": ["x64"] }
      ]
    },
    "nsis": {
      "oneClick": false,
      "perMachine": false,
      "allowToChangeInstallationDirectory": true,
      "createDesktopShortcut": true,
      "createStartMenuShortcut": true,
      "shortcutName": "Регламент-ТС",
      "uninstallDisplayName": "Регламент-ТС",
      "deleteAppDataOnUninstall": false,
      "language": "1049",
      "installerIcon": "build/icon.ico",
      "uninstallerIcon": "build/icon.ico",
      "artifactName": "Reglament-TS-Setup-${version}.exe"
    },
    "portable": {
      "artifactName": "Reglament-TS-Portable-${version}.exe"
    }
  }
}

```

13. app/scripts/prepare-www.js

```
'use strict';
/* Кладёт программу (index.html из корня репозитория) в app/www/ для упаковки */
const fs = require('fs'), path = require('path');
const src = path.resolve(__dirname, '..', '..', 'index.html');
const dst = path.resolve(__dirname, '..', 'www', 'index.html');
if (!fs.existsSync(src)) { console.error('Не найден ' + src); process.exit(1); }
fs.mkdirSync(path.dirname(dst), { recursive: true });
fs.copyFileSync(src, dst);
console.log('www/index.html: ' + (fs.statSync(dst).size / 1048576).toFixed(1) + ' МБ');

```

14. app/scripts/build-installer.js

```
'use strict';
/* Собирает установщик (NSIS) и переносной zip из dist/win-unpacked.
   Работает на Linux/macOS без Wine: makensis берётся из кэша electron-builder или из PATH.
   Порядок: npm run dist:dir  →  node scripts/build-installer.js  */
const fs = require('fs'), path = require('path'), os = require('os'), cp = require('child_process');
const root = path.resolve(__dirname, '..');
const pkg = require(path.join(root, 'package.json'));
const unpacked = path.join(root, 'dist', 'win-unpacked');
if (!fs.existsSync(unpacked)) { console.error('Нет dist/win-unpacked — сначала: npm run dist:dir'); process.exit(1); }

function findMakensis() {
  if (process.env.MAKENSIS) return { bin: process.env.MAKENSIS, dir: process.env.NSISDIR };
  const base = path.join(os.homedir(), '.cache', 'electron-builder');
  try {
    for (const v of fs.readdirSync(base).filter(n => /^nsis-\d/.test(n))) for (const d of fs.readdirSync(path.join(base, v))) {
      const bin = path.join(base, v, d, process.platform === 'darwin' ? 'mac' : 'linux', 'makensis');
      if (fs.existsSync(bin)) return { bin, dir: path.join(base, v, d) };
    }
  } catch (e) { /* нет кэша */ }
  return { bin: 'makensis', dir: process.env.NSISDIR };
}

// Рабочая копия: исполняемый файл с ASCII-именем (имена с кириллицей makensis на Linux читает ненадёжно)
const stage = path.join(root, 'dist', 'stage');
fs.rmSync(stage, { recursive: true, force: true });
fs.cpSync(unpacked, stage, { recursive: true });
// Языковые файлы Chromium: оставляем русский и английский (около 45 МБ экономии)
const loc = path.join(stage, 'locales');
if (fs.existsSync(loc)) for (const n of fs.readdirSync(loc)) if (!/^(ru|en-US)\.pak$/.test(n)) fs.rmSync(path.join(loc, n));
const exeSrc = fs.readdirSync(stage).find(n => n.toLowerCase().endsWith('.exe'));
fs.renameSync(path.join(stage, exeSrc), path.join(stage, 'Reglament-TS.exe'));

const out = path.join(root, 'dist', `Reglament-TS-Setup-${pkg.version}.exe`);
const { bin, dir } = findMakensis();
const env = Object.assign({}, process.env, dir ? { NSISDIR: dir } : {});
const args = [`-DVERSION=${pkg.version}`, `-DOUTFILE=${out}`, `-DSRCDIR=${stage}`, `-DICON=${path.join(root, 'build', 'icon.ico')}`, path.join(root, 'installer', 'installer.nsi')];
const r = cp.spawnSync(bin, args, { env, stdio: 'inherit' });
if (r.status !== 0) { console.error('makensis завершился с ошибкой', r.error ? r.error.message : '', r.signal || '', 'статус:', r.status, '\nкоманда:', bin, args.join(' ')); process.exit(r.status || 1); }
console.log(`Установщик: ${out} (${(fs.statSync(out).size / 1048576).toFixed(1)} МБ)`);

// Переносная версия: та же папка + метка portable.flag (данные рядом, в «Данные Регламент-ТС»)
fs.writeFileSync(path.join(stage, 'portable.flag'), 'Переносной режим: данные хранятся в папке «Данные Регламент-ТС» рядом с программой.\n');
fs.writeFileSync(path.join(stage, 'ПРОЧТИТЕ.txt'), 'Регламент-ТС — переносная версия.\r\n\nЗапуск: Reglament-TS.exe\r\n\nДанные хранятся в папке «Данные Регламент-ТС» рядом с программой; копируйте её вместе с программой.\r\n', 'utf8');
const zip = path.join(root, 'dist', `Reglament-TS-Portable-${pkg.version}.zip`);
fs.rmSync(zip, { force: true });
const z = cp.spawnSync('zip', ['-qr', zip, '.'], { cwd: stage, stdio: 'inherit' });
if (z.status === 0) console.log(`Переносная версия: ${zip} (${(fs.statSync(zip).size / 1048576).toFixed(1)} МБ)`);
else console.log('zip не найден — переносную версию соберите архиватором из dist/stage');

```

15. app/installer/installer.nsi

```
    Unicode true
; Установщик «Регламент-ТС» для Windows (без прав администратора).
; Сборка: node scripts/build-installer.js (makensis из кэша electron-builder или из PATH).
!include "MUI2.nsh"
!include "LogicLib.nsh"
!include "FileFunc.nsh"

!define APPNAME "Регламент-ТС"
!define APPEXE "Reglament-TS.exe"
!define APPID "ReglamentTS"
!define UNKEY "Software\Microsoft\Windows\CurrentVersion\Uninstall\${APPID}"
!ifndef VERSION
  !define VERSION "1.0.0"
!endif
!ifndef OUTFILE
  !define OUTFILE "Reglament-TS-Setup.exe"
!endif
!ifndef SRCDIR
  !define SRCDIR "stage"
!endif

Name "${APPNAME}"
OutFile "${OUTFILE}"
InstallDir "$LOCALAPPDATA\Programs\${APPNAME}"
InstallDirRegKey HKCU "Software\${APPID}" "InstallDir"
RequestExecutionLevel user
SetCompressor /SOLID lzma
BrandingText "${APPNAME} ${VERSION}"
Icon "${ICON}"
UninstallIcon "${ICON}"
VIProductVersion "${VERSION}.0"
VIAddVersionKey /LANG=1049 "ProductName" "${APPNAME}"
VIAddVersionKey /LANG=1049 "FileDescription" "Установщик ${APPNAME}"
VIAddVersionKey /LANG=1049 "FileVersion" "${VERSION}"
VIAddVersionKey /LANG=1049 "ProductVersion" "${VERSION}"

!define MUI_ICON "${ICON}"
!define MUI_UNICON "${ICON}"
!define MUI_ABORTWARNING
!define MUI_FINISHPAGE_RUN "$INSTDIR\${APPEXE}"
!define MUI_FINISHPAGE_RUN_TEXT "Запустить ${APPNAME}"

!insertmacro MUI_PAGE_WELCOME
!insertmacro MUI_PAGE_DIRECTORY
!insertmacro MUI_PAGE_INSTFILES
!insertmacro MUI_PAGE_FINISH
!insertmacro MUI_UNPAGE_CONFIRM
!insertmacro MUI_UNPAGE_INSTFILES
!insertmacro MUI_LANGUAGE "Russian"

; Если программа запущена, её файлы заняты: просим закрыть и повторяем
Function CheckRunning
  ${If} ${FileExists} "$INSTDIR\${APPEXE}"
    retry:
    ClearErrors
    FileOpen $0 "$INSTDIR\${APPEXE}" a
    ${If} ${Errors}
      MessageBox MB_RETRYCANCEL|MB_ICONEXCLAMATION "${APPNAME} запущена. Закройте программу и нажмите «Повтор»." IDRETRY retry
      Abort
    ${EndIf}
    FileClose $0
  ${EndIf}
FunctionEnd

Section "Install"
  ; Всегда ставим в собственную подпапку: выбранная пользователем папка не должна пострадать при удалении
  ${GetFileName} "$INSTDIR" $R0
  ${If} $R0 != "${APPNAME}"
    StrCpy $INSTDIR "$INSTDIR\${APPNAME}"
  ${EndIf}
  Call CheckRunning
  SetOutPath "$INSTDIR"
  File /r "${SRCDIR}\*.*"
  File "/oname=icon.ico" "${ICON}"
  FileOpen $0 "$INSTDIR\.reglament-ts" w
  FileWrite $0 "${APPNAME} ${VERSION}"
  FileClose $0
  WriteUninstaller "$INSTDIR\Uninstall.exe"

  ; Ярлыки (прежние ярлыки версии на базе браузера убираются)
  Delete "$DESKTOP\Регламент-ТСТ.lnk"
  Delete "$SMPROGRAMS\Регламент-ТСТ.lnk"
  CreateShortCut "$SMPROGRAMS\${APPNAME}.lnk" "$INSTDIR\${APPEXE}" "" "$INSTDIR\icon.ico" 0
  CreateShortCut "$DESKTOP\${APPNAME}.lnk" "$INSTDIR\${APPEXE}" "" "$INSTDIR\icon.ico" 0

  WriteRegStr HKCU "Software\${APPID}" "InstallDir" "$INSTDIR"
  WriteRegStr HKCU "${UNKEY}" "DisplayName" "${APPNAME}"
  WriteRegStr HKCU "${UNKEY}" "DisplayVersion" "${VERSION}"
  WriteRegStr HKCU "${UNKEY}" "Publisher" "${APPNAME}"
```

15. app/installer/installer.nsi (продолжение)

```
      WriteRegStr HKCU "${UNKEY}" "DisplayIcon" "$INSTDIR\icon.ico"
      WriteRegStr HKCU "${UNKEY}" "InstallLocation" "$INSTDIR"
      WriteRegStr HKCU "${UNKEY}" "UninstallString" '"$INSTDIR\Uninstall.exe"'
      WriteRegDWORD HKCU "${UNKEY}" "NoModify" 1
      WriteRegDWORD HKCU "${UNKEY}" "NoRepair" 1
      ${GetSize} "$INSTDIR" "/S=0K" $0 $1 $2
      IntFmt $0 "0x%08X" $0
      WriteRegDWORD HKCU "${UNKEY}" "EstimatedSize" "$0"
    SectionEnd

    Section "Uninstall"
      Call un.CheckRunning
      Delete "$DESKTOP\${APPNAME}.lnk"
      Delete "$SMPROGRAMS\${APPNAME}.lnk"
      ; Удаляем папку целиком только если это наша установка (метка), иначе — только свои файлы
      ${If} ${FileExists} "$INSTDIR\.reglament-ts"
        RMDir /r "$INSTDIR"
      ${Else}
        Delete "$INSTDIR\${APPEXE}"
        Delete "$INSTDIR\Uninstall.exe"
        Delete "$INSTDIR\icon.ico"
      ${EndIf}
      DeleteRegKey HKCU "${UNKEY}"
      DeleteRegKey HKCU "Software\${APPID}"
      MessageBox MB_OK|MB_ICONINFORMATION "${APPNAME} удалена.$\r$\n$\r$\nВаши данные сохранены: папка
    «Документы\${APPNAME}»."
    SectionEnd

    Function un.CheckRunning
      ${If} ${FileExists} "$INSTDIR\${APPEXE}"
        retry:
        ClearErrors
        FileOpen $0 "$INSTDIR\${APPEXE}" a
        ${If} ${Errors}
          MessageBox MB_RETRYCANCEL|MB_ICONEXCLAMATION "${APPNAME} запущена. Закройте программу и нажмите «Повтор»."
    IDRETRY retry
          Abort
        ${EndIf}
        FileClose $0
      ${EndIf}
    FunctionEnd
```

16. .github/workflows/build-windows.yml

```yaml
name: Сборка приложения для Windows

# Собирает установщик и переносную версию «Регламент-ТС» на сервере с Windows.
# Запускается при изменении программы или приложения, вручную и при создании тега v*.
on:
  push:
    branches: ['**']
    paths:
      - 'index.html'
      - 'app/**'
      - '.github/workflows/build-windows.yml'
    tags: ['v*']
  workflow_dispatch:

jobs:
  windows:
    runs-on: windows-latest
    defaults:
      run:
        working-directory: app
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Установка зависимостей
        run: npm ci

      - name: Тест файлового хранилища
        run: npm test

      - name: Сборка установщика и переносной версии
        run: npm run dist
        env:
          CSC_IDENTITY_AUTO_DISCOVERY: 'false'   # подпись кодом не используется

      - name: Файлы сборки
        shell: bash
        run: ls -la dist/*.exe

      - uses: actions/upload-artifact@v4
        with:
          name: reglament-ts-windows
          path: |
            app/dist/Reglament-TS-Setup-*.exe
            app/dist/Reglament-TS-Portable-*.exe
          if-no-files-found: error

      - name: Выпуск (только для тегов v*)
        if: startsWith(github.ref, 'refs/tags/v')
        uses: softprops/action-gh-release@v2
        with:
          files: |
            app/dist/Reglament-TS-Setup-*.exe
            app/dist/Reglament-TS-Portable-*.exe

```

17. tests/platform.spec.js

```
// Запуск: node tests/platform.spec.js [папка-для-результатов]
// Нужен Playwright с Chromium (npm i -D playwright && npx playwright install chromium).
const path = require('path');
const fs = require('fs');
let pw; try { pw = require('playwright'); } catch (e) { pw = require('/opt/node22/lib/node_modules/playwright'); }
const { chromium } = pw;
const OUT = process.argv[2] || path.join(__dirname, 'out');
fs.mkdirSync(OUT, { recursive: true });
const PAGE_URL = require('url').pathToFileURL(path.resolve(__dirname, '..', 'index.html')).href;
// Функциональная проверка платформы, сохранности данных и пункта управления
const LS = 'spravochnik-tst-fleet-v1';
const res = [];
const check = (name, ok, info) => { res.push(ok); console.log((ok ? 'PASS ' : 'FAIL ') + name + (info ? ' — ' + info : ''
)); };
(async () => {
  const b = await chromium.launch();
  const ctx = await b.newContext({ viewport: { width: 1440, height: 900 }, acceptDownloads: true });
  const net = [];
  await ctx.route('**/*', r => { const u = r.request().url(); if (/^(file|data|blob):/.test(u)) return r.continue();
net.push(u); r.abort(); });
  const p = await ctx.newPage();
  const errs = []; p.on('pageerror', e => errs.push(e.message));
  await p.addInitScript(() => { window.print = () => { window.__printed = (window.__printed || 0) + 1; }; });
  const URL0 = PAGE_URL;
  // Настоящий перезапуск программы: смена только #… не перезагружает страницу
  const load = async (hash) => { await p.goto(URL0 + (hash || '#fleet/command')); await p.reload(); await
p.waitForTimeout(1500); await p.evaluate(() => document.querySelectorAll('.f-toast-close').forEach(x => x.click()));
};
  const db = () => p.evaluate(k => JSON.parse(localStorage.getItem(k)), LS);
  const act = (a, id, extra) => p.evaluate(([a, id, extra]) => {
    const btn = document.createElement('button'); btn.dataset.act = a; if (id) btn.dataset.id = id;
    Object.entries(extra || {}).forEach(([k, v]) => { btn.dataset[k] = v; });
    document.getElementById('fleetView').appendChild(btn); btn.click(); btn.remove();
  }, [a, id, extra]);
  const closeModals = () => p.evaluate(() => document.querySelectorAll('.f-modal-backdrop').forEach(x => x.remove()));
  const snaps = () => p.evaluate(() => new Promise(res => {
    const r = indexedDB.open('tst-fleet-snapshots', 1);
    r.onsuccess = () => { const tx = r.result.transaction('snaps', 'readonly'); const q =
tx.objectStore('snaps').getAll(); q.onsuccess = () => res(q.result.map(s => ({ id: s.id, reason: s.reason, at: s.at,
vehicles: s.counts && s.counts.vehicles }))); };
    r.onerror = () => res([]);
  }));

  // 0. Старая база (структура 1) → миграция, точка восстановления, ничего не потеряно
  await load('#fleet/park');
  const seeded = await db();
  const old = Object.assign({}, seeded, { version: 1 }); delete old.trash; delete old.history; delete old.migrations;
  old.vehicles[0].customField = 'сохранить';
  await p.evaluate(([k, v]) => localStorage.setItem(k, JSON.stringify(v)), [LS, old]);
  await load('#fleet/safety');
  const m1 = await db();
  check('Миграция 1 → 2: версия обновлена, записи и неизвестные поля сохранены',
    m1.version === 2 && m1.vehicles.length === old.vehicles.length && m1.vehicles[0].customField === 'сохранить' &&
Array.isArray(m1.trash) && m1.migrations.length === 1,
    `версия ${m1.version}, техники ${m1.vehicles.length}`);
  await p.waitForTimeout(500);
  const s0 = await snaps();
  check('Перед миграцией создана точка восстановления со старыми данными', s0.some(s => s.reason === 'migration'),
s0.map(s => s.reason).join(', '));
  check('Раздел «Сохранность данных» сообщает об обновлении структуры', await p.evaluate(() => /Структура данных
обновлена/.test(document.body.innerText)));

  // 1. Данные новой версии программы не понижаются
  const newer = Object.assign({}, m1, { version: 7, futureCollection: [{ id: 'x_1', a: 1 }] });
  await p.evaluate(([k, v]) => localStorage.setItem(k, JSON.stringify(v)), [LS, newer]);
  await p.goto(URL0 + '#fleet/command'); await p.reload(); await p.waitForTimeout(2200);
  const warned = await p.evaluate(() => [...document.querySelectorAll('.f-toast-title')].some(t => /более новой
версии/.test(t.textContent)));
  await act('snap-create'); await p.waitForTimeout(400);
  await p.evaluate(() => document.querySelectorAll('.f-toast-close').forEach(x => x.click()));
  // любое сохранение не должно терять неизвестную коллекцию и номер версии
  await act('edit-vehicle', m1.vehicles[0].id); await p.waitForSelector('#vSave'); await p.click('#vSave'); await
p.waitForTimeout(300);
  const n1 = await db();
  check('База новой версии: предупреждение, версия и неизвестные данные сохранены', warned && n1.version === 7 &&
n1.futureCollection && n1.futureCollection.length === 1, `версия ${n1.version}`);
  await p.evaluate(([k, v]) => localStorage.setItem(k, JSON.stringify(v)), [LS, m1]);
  await load('#fleet/park');

  // 2. Журнал изменений
  const v0 = (await db()).vehicles[0];
  await act('edit-vehicle', v0.id); await p.waitForSelector('#vInv');
  await p.fill('#vInv', 'ИН-ЖУРНАЛ-1'); await p.click('#vSave'); await p.waitForTimeout(300);
  const h1 = await db();
  const he = h1.history.find(e => e.vehicleId === v0.id && e.field === 'invNumber');
  check('Журнал: изменение инвентарного номера записано (было → стало)', he && he.from === (v0.invNumber || '') &&
he.to === 'ИН-ЖУРНАЛ-1', he ? `${he.from} → ${he.to}` : 'нет записи');
  await p.evaluate(id => location.hash = '#fleet/vehicle/' + id, v0.id); await p.waitForTimeout(500);
  check('Журнал виден в карточке техники', await p.evaluate(() => /Журнал изменений карточки[\s\S]*ИН-
ЖУРНАЛ-1/i.test(document.getElementById('fleetView').innerText)));
```

17. tests/platform.spec.js (продолжение)

```
77
78        // 3. Корзина: удаление техники и возврат со всеми записями
79        const before = await db();
80        const victim = before.vehicles.find(v => before.maint.some(m => m.vehicleId === v.id)) || before.vehicles[0];
81        const cnt = d => ({ m: d.maint.filter(x => x.vehicleId === victim.id).length, h: d.hours.filter(x => x.vehicleId ===
           victim.id).length, t: d.trips.filter(x => x.vehicleId === victim.id).length });
82        const c0 = cnt(before);
83        await act('del-vehicle', victim.id); await p.waitForTimeout(400);
84        await p.evaluate(() => document.querySelectorAll('.f-toast-close').forEach(x => x.click()));
85        const afterDel = await db();
86        check('Удалённая техника попала в корзину вместе с журналом ТО, моточасами и путевыми листами',
87          !afterDel.vehicles.some(v => v.id === victim.id) && afterDel.trash.length === 1 &&
          afterDel.trash[0].records.vehicles[0].id === victim.id,
88          `ТО ${c0.m}, моточасов ${c0.h}, путевых ${c0.t}`);
89        await p.waitForTimeout(8500);   // срок отмены истёк — запись должна остаться в корзине
90        await load('#fleet/safety');
91        check('После истечения срока отмены и перезапуска запись в корзине', (await db()).trash.length === 1);
92        await act('trash-restore', afterDel.trash[0].id); await p.waitForTimeout(400);
93        const back = await db();
94        const c1 = cnt(back);
95        check('Возврат из корзины: техника и все связанные записи на месте', back.vehicles.some(v => v.id === victim.id) &&
          c1.m === c0.m && c1.h === c0.h && c1.t === c0.t && back.trash.length === 0, JSON.stringify(c1));
96        check('ID техники после возврата не изменился (QR-наклейка действительна)', back.vehicles.some(v => v.id ===
          victim.id && v.qr === victim.qr || v.id === victim.id));
97
98        // 4. Удаление с отменой не оставляет мусора в корзине
99        await act('del-vehicle', victim.id); await p.waitForTimeout(300);
100       await p.evaluate(() => document.querySelector('.f-toast-action')?.click()); await p.waitForTimeout(300);
101       const undone = await db();
102       check('«Отменить» в уведомлении возвращает запись и убирает её из корзины', undone.vehicles.some(v => v.id ===
          victim.id) && undone.trash.length === 0);
103
104       // 5. Проверка перед импортом: копия с ошибками → отчёт → «Исправить и загрузить»
105       const broken = JSON.parse(JSON.stringify(undone));
106       broken.trips.push({ id: 't_orphan_00001', vehicleId: 'v_net_takoy_tehniki', date: '2026-01-01', startHours: 1,
          endHours: 2 });
107       broken.maint.push(Object.assign({}, broken.maint[0]));            // повтор идентификатора
108       broken.vehicles.push({ id: 'v_bad id!', name: '' });               // неверный id и пустое имя
109       fs.writeFileSync(OUT + '/broken.json', JSON.stringify(broken));
110       await load('#fleet/safety');
111       const [chooser] = await Promise.all([p.waitForEvent('filechooser'), act('backup-import')]);
112       await chooser.setFiles(OUT + '/broken.json'); await p.waitForTimeout(600);
113       const review = await p.evaluate(() => { const m = document.querySelector('.f-modal-backdrop .f-modal'); return m ?
          m.innerText : ''; });
114       check('Перед импортом показан отчёт проверки с ошибками целостности', /Проверка резервной копии/.test(review) &&
          /Ошибки целостности: \d+/.test(review) && /Исправить и загрузить/.test(review), (review.match(/Ошибки целостности:
          \d+/) || [''])[0]);
115       await p.screenshot({ path: OUT + '/review.png' });
116       await p.evaluate(() => document.querySelector('[data-dec="repair"]').click()); await p.waitForTimeout(800);
117       const fixed = await db();
118       const integ = await p.evaluate(() => TST.platform.checkIntegrity(JSON.parse(localStorage.getItem('spravochnik-tst-
          fleet-v1'))));
119       check('После «Исправить и загрузить» ошибок целостности нет', integ.errors.length === 0 && !fixed.trips.some(t =>
          t.id === 't_orphan_00001'), `ошибок ${integ.errors.length}, замечаний ${integ.warnings.length}`);
120       const s1 = await snaps();
121       check('Перед восстановлением создана точка «Перед восстановлением из копии»', s1.some(s => s.reason === 'import'));
122
123       // 6. Откат к точке восстановления возвращает состояние до импорта
124       const imp = s1.filter(s => s.reason === 'import').sort((a, b) => b.at.localeCompare(a.at))[0];
125       await load('#fleet/safety');
126       await act('snap-restore', imp.id); await p.waitForTimeout(700);
127       await p.evaluate(() => document.querySelector('[data-dec="restore"]').click()); await p.waitForTimeout(900);
128       const rolled = await db();
129       check('Откат к точке «Перед восстановлением из копии» вернул прежние данные', rolled.vehicles.length ===
          undone.vehicles.length && !rolled.vehicles.some(v => v.id === 'v_bad id!'), `техники ${rolled.vehicles.length}`);
130       check('Перед откатом создана точка «Перед откатом»', (await snaps()).some(s => s.reason === 'rollback'));
131
132       // 7. Очистка всех данных — с точкой восстановления
133       await act('wipe'); await p.waitForTimeout(400);
134       await p.evaluate(() => document.querySelector('[data-yes]').click()); await p.waitForTimeout(800);
135       const wiped = await db();
136       const sw = (await snaps()).filter(s => s.reason === 'wipe');
137       check('После очистки данных есть точка «Перед очисткой» с прежним количеством техники', wiped.vehicles.length === 0
          && sw.length === 1 && sw[0].vehicles === rolled.vehicles.length, `в точке ${sw[0] && sw[0].vehicles}`);
138       await load('#fleet/safety');
139       await act('snap-restore', sw[0].id); await p.waitForTimeout(700);
140       await p.evaluate(() => document.querySelector('[data-dec="restore"]').click()); await p.waitForTimeout(900);
141       check('Данные возвращены из точки «Перед очисткой»', (await db()).vehicles.length === rolled.vehicles.length);
142
143       // 8. Самотесты не меняют реальные данные
144       await load('#fleet/safety');
145       const raw0 = await p.evaluate(k => localStorage.getItem(k), LS);
146       await act('tests-run'); await p.waitForTimeout(1200);
147       const raw1 = await p.evaluate(k => localStorage.getItem(k), LS);
148       const tr = await p.evaluate(() => document.querySelector('.sf-tests-sum')?.textContent || '');
149       check('Самотесты модулей пройдены и не изменили базу', /20 из 20/.test(tr) && raw0 === raw1, tr);
150
151       // 9. Пункт управления: числа совпадают с перечнями, перечень → карточка, задание печатается
152       await load('#fleet/command');
153       const tiles = await p.evaluate(() => [...document.querySelectorAll('[data-act="cp-list"].cp-tile')].map(b =>
```

17. tests/platform.spec.js (продолжение)

```
    [b.dataset.list, Number(b.querySelector('.f-stat-value').textContent)]));
154   let allOk = true; const info = [];
155   for (const [key, n] of tiles) {
156     await act('cp-list', null, { list: key }); await p.waitForTimeout(150);
157     const rows = await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop [data-cp-open]').length);
158     if (rows !== n) { allOk = false; info.push(`${key}: ${n}≠${rows}`); }
159     await closeModals();
160   }
161   check('Каждое число пункта управления открывает перечень ровно из этих машин', allOk && tiles.length >= 10,
    info.join('; ') || `${tiles.length} показателей`);
162   const units = await p.evaluate(() => [...document.querySelectorAll('.cp-table [data-act="cp-list"]')].map(b =>
    [b.dataset.list, Number(b.textContent)]));
163   let uOk = true;
164   for (const [key, n] of units) { await act('cp-list', null, { list: key }); await p.waitForTimeout(100); const rows =
     await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop [data-cp-open]').length); if (rows !== n) uOk =
    false; await closeModals(); }
165   check('Числа таблицы подразделений совпадают с перечнями', uOk && units.length > 0, units.length + ' чисел');
166   await act('cp-list', null, { list: 'total' }); await p.waitForTimeout(200);
167   const firstId = await p.evaluate(() => document.querySelector('[data-cp-open]').dataset.cpOpen);
168   await p.evaluate(() => document.querySelector('[data-cp-open]').click()); await p.waitForTimeout(400);
169   check('Из перечня открывается карточка выбранной машины', (await p.evaluate(() => location.hash)) ===
    '#fleet/vehicle/' + firstId);
170   await load('#fleet/command');
171   await act('cp-task', null, { list: 'overdue' }); await p.waitForTimeout(700);
172   const pw = await p.evaluate(() => { const t = document.querySelector('.tpaper, .pw-page, #pwStage'); return t ?
    t.innerText.slice(0, 200) : document.body.innerText.includes('ЗАДАНИЕ'); });
173   check('«Сформировать задание» открывает печатную форму «Задание»', /ЗАДАНИЕ/.test(String(pw)) || pw === true);
174   await p.keyboard.press('Escape'); await p.waitForTimeout(200);
175
176   // 10. Статус «Неисправна» влияет на КТГ
177   const d10 = await db();
178   const vv = d10.vehicles.find(v => v.status === 'В эксплуатации');
179   await act('edit-vehicle', vv.id); await p.waitForSelector('#vStatus');
180   await p.selectOption('#vStatus', 'Неисправна'); await p.click('#vSave'); await p.waitForTimeout(300);
181   await load('#fleet/command');
182   const faulty = await p.evaluate(() => Number(document.querySelector('[data-list="faulty"] .f-stat-
    value').textContent));
183   const ktg = await p.evaluate(() => document.querySelector('.cp-ktg .f-stat-value').textContent);
184   const S = await p.evaluate(() => TST.fleet.commandSummary());
185   check('Статус «Неисправна» учитывается в счётчике и КТГ', faulty >= 1 && Math.abs(S.ktg - (S.total - S.faulty -
    S.repair) / S.total) < 1e-9, `неисправна ${faulty}, КТГ ${ktg}`);
186
187   check('Без сетевых запросов и ошибок JavaScript', net.length === 0 && errs.length === 0, `запросов ${net.length},
    ошибок ${errs.length}${errs[0] ? ': ' + errs[0] : ''}`);
188   console.log(`ИТОГ: ${res.filter(Boolean).length} из ${res.length}`);
189   await b.close();
190 })();
191
```

18. tests/qr.spec.js

```
// Запуск: node tests/qr.spec.js [папка-для-результатов]
// Нужен Playwright с Chromium (npm i -D playwright && npx playwright install chromium).
const path = require('path');
const fs = require('fs');
let pw; try { pw = require('playwright'); } catch (e) { pw = require('/opt/node22/lib/node_modules/playwright'); }
const { chromium } = pw;
const OUT = process.argv[2] || path.join(__dirname, 'out');
fs.mkdirSync(OUT, { recursive: true });
const PAGE_URL = require('url').pathToFileURL(path.resolve(__dirname, '..', 'index.html')).href;
const results = [];
const check = (n, name, ok, info) => { results.push({ n, name, ok: !!ok, info: info || '' }); console.log((ok ? 'PASS' : 'FAIL') + ' #' + n + ' ' + name + (info ? ' — ' + info : '')); };
(async () => {
  const b = await chromium.launch();
  const ctx = await b.newContext({ viewport: { width: 1440, height: 900 }, acceptDownloads: true });
  // Полная изоляция от сети: любой не-file запрос блокируется и учитывается
  const netReq = [];
  await ctx.route('**/*', r => { const u = r.request().url(); if (u.startsWith('file:') || u.startsWith('data:') || u.startsWith('blob:')) return r.continue(); netReq.push(u); return r.abort(); });
  const p = await ctx.newPage();
  const errs = [];
  p.on('pageerror', e => errs.push(e.message));
  await p.addInitScript(() => { window.__printed = 0; window.print = () => { window.__printed++; }; });
  const URL0 = PAGE_URL;
  await p.goto(URL0 + '#fleet/park'); await p.waitForTimeout(1200);
  const closeToasts = () => p.evaluate(() => document.querySelectorAll('.f-toast-close').forEach(x => x.click()));
  await closeToasts();
  const ids0 = await p.evaluate(() => JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')).vehicles.map(v => v.id));

  // 1. Постановка на учёт → идентификатор и QR
  async function addVehicle(f) {
    await p.evaluate(() => { document.querySelectorAll('.f-modal-backdrop').forEach(x => x.remove()); location.hash = '#fleet/park'; });
    await p.waitForTimeout(250);
    await p.evaluate(() => { document.querySelector('[data-act="add-vehicle"]').click(); });
    await p.waitForSelector('#vName');
    for (const [k, v] of Object.entries(f)) await p.fill('#' + k, String(v));
    await p.click('#vSave'); await p.waitForTimeout(300);
    const db = await p.evaluate(() => JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')));
    return db.vehicles.find(v => v.name === f.vName);
  }
  const v1 = await addVehicle({ vName: 'Испытательная кухня КП-130 №1', vBrand: 'КП-130-11Л', vInv: 'ИН-777001', vReg: 'ТТ 0001 77', vSer: '3В-0001', vUnit: '1-я рота МО', vHours: 120 });
  check(1, 'Новая техника получает постоянный идентификатор', v1 && /^v_[a-z0-9]+_[a-z0-9]+$/.test(v1.id), v1 && v1.id);
  await p.evaluate(id => location.hash = '#fleet/vehicle/' + id, v1.id); await p.waitForTimeout(500);
  const hasPanel = await p.evaluate(() => !!document.querySelector('.qr-panel svg') && [...document.querySelectorAll('.qr-panel button')].map(b => b.textContent.trim()));
  check(1, 'В карточке есть QR-код и 4 кнопки', hasPanel && hasPanel.length === 4, JSON.stringify(hasPanel));

  // 2. Уникальность
  const more = [];
  for (let i = 2; i <= 4; i++) more.push(await addVehicle({ vName: 'Испытательная кухня КП-130 №' + i, vInv: 'ИН-77700' + i }));
  const v5 = await addVehicle({ vName: 'Пустая карточка' });   // только обязательное поле — для проверки пустых полей
  const all = await p.evaluate(() => JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')).vehicles.map(v => v.id));
  check(2, 'Идентификаторы всех записей уникальны', new Set(all).size === all.length, all.length + ' записей');
  const payloads = await p.evaluate(ids => ids.map(id => TST.fleet.qrPayload(id)), all);

  // 3. Правка без смены идентификатора
  await p.evaluate(id => location.hash = '#fleet/vehicle/' + id, v1.id); await p.waitForTimeout(400);
  await p.evaluate(() => document.querySelector('[data-act="edit-vehicle"]').click()); await p.waitForSelector('#vName');
  await p.fill('#vName', 'Испытательная кухня КП-130 №1 (изм.)'); await p.fill('#vUnit', '2-я рота МО');
  await p.click('#vSave'); await p.waitForTimeout(300);
  const v1e = await p.evaluate(id => JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')).vehicles.find(v => v.id === id), v1.id);
  check(3, 'После правки идентификатор не изменился', v1e && v1e.name.endsWith('(изм.)') && v1e.id === v1.id, v1e && v1e.id);

  // 5. Печать одной наклейки (принтер этикеток 50×50)
  await p.evaluate(() => document.querySelector('.qr-panel [data-act="qr-print"]').click()); await p.waitForSelector('#qpFormat');
  await p.selectOption('#qpMedia', 'roll'); await p.selectOption('#qpFormat', '50x50'); await p.selectOption('#qpLevel', 'basic');
  await p.screenshot({ path: OUT + '/print-modal.png' });
  await p.click('#qpPrint'); await p.waitForTimeout(300);
  const printed1 = await p.evaluate(() => ({ n: window.__printed, pages: document.querySelectorAll('#fleetPrint .qr-page').length }));
  await p.emulateMedia({ media: 'print' }); await p.pdf({ path: OUT + '/one-50x50.pdf', preferCSSPageSize: true });
  await p.emulateMedia({ media: 'screen' });
  check(5, 'Печать одной наклейки: вызвано окно печати, 1 страница', printed1.n === 1 && printed1.pages === 1, JSON.stringify(printed1));
  const exp1 = await p.evaluate(id => TST.fleet.qrPayload(id, 'basic'), v1.id);
  fs.writeFileSync(OUT + '/expect-one.json', JSON.stringify([exp1]));

  // 13. Моточасы: срок «Основных сведений» от них не зависит, «Расширенные» — устаревают
  const st1 = await p.evaluate(id => TST.fleet.qrState(id), v1.id);
```

18. tests/qr.spec.js (продолжение)

```
74        const fullBefore = await p.evaluate(id => TST.fleet.qrPayload(id, 'full'), v1.id);
75        await p.evaluate(() => document.querySelector('[data-act="add-hours"]').click()); await p.waitForSelector('#hVal');
76        await p.fill('#hVal', '400'); await p.click('#hSave'); await p.waitForTimeout(300);
77        const st2 = await p.evaluate(id => TST.fleet.qrState(id), v1.id);
78        const fullAfter = await p.evaluate(id => TST.fleet.qrPayload(id, 'full'), v1.id);
79        check(13, 'Новые моточасы попадают в расширенный состав кода; наклейка «Основные сведения» остаётся актуальной (срок ТО не изменился)',
80          st1 === 'ok' && st2 === 'ok' && /Моточасы: 120/.test(fullBefore) && /Моточасы: 400/.test(fullAfter), st1 + ' → ' + st2);
81        // 4. Перезапуск
82        await p.reload(); await p.waitForTimeout(1200);
83        const after = await p.evaluate(id => { const v = JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')).vehicles.find(x => x.id === id); return v && v.qr && v.id; }, v1.id);
84        check(4, 'После перезапуска запись, идентификатор и отметка о печати сохранились', after === v1.id);
85
86        // 6. А4 — массовая печать всех единиц
87        await p.evaluate(() => location.hash = '#fleet/qr'); await p.waitForTimeout(500); await closeToasts();
88        await p.evaluate(() => document.querySelector('[data-act="qr-sel-all"]').click()); await p.waitForTimeout(200);
89        await p.evaluate(() => document.querySelector('[data-act="qr-print-sel"]').click()); await p.waitForSelector('#qpFormat');
90        await p.selectOption('#qpMedia', 'a4'); await p.selectOption('#qpFormat', '50x50'); await p.selectOption('#qpLevel', 'basic');
91        await p.screenshot({ path: OUT + '/print-a4-modal.png' });
92        await p.click('#qpPrint'); await p.waitForTimeout(300);
93        const a4 = await p.evaluate(() => ({ pages: document.querySelectorAll('#fleetPrint .qr-page').length, stickers: document.querySelectorAll('#fleetPrint .qr-stk').length }));
94        await p.emulateMedia({ media: 'print' }); await p.pdf({ path: OUT + '/a4-50x50.pdf', preferCSSPageSize: true }); await p.emulateMedia({ media: 'screen' });
95        const allIds = await p.evaluate(() => JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')).vehicles.map(v => v.id));
96        check(6, 'Лист А4: все отмеченные наклейки на листе', a4.stickers === allIds.length, JSON.stringify(a4));
97        fs.writeFileSync(OUT + '/expect-a4.json', JSON.stringify(await p.evaluate(ids => ids.map(id => TST.fleet.qrPayload(id, 'basic', '50x50')), allIds)));
98        // формат 60×40 и 70×70 (расширенные)
99        for (const [fmt, lvl] of [['60x40', 'basic'], ['70x70', 'full']]) {
100         await p.evaluate(() => document.querySelector('[data-act="qr-print-sel"]').click()); await p.waitForSelector('#qpFormat');
101         await p.selectOption('#qpMedia', 'roll'); await p.selectOption('#qpFormat', fmt); await p.selectOption('#qpLevel', lvl);
102         await p.click('#qpPrint'); await p.waitForTimeout(300);
103         await p.emulateMedia({ media: 'print' }); await p.pdf({ path: OUT + '/roll-' + fmt + '.pdf', preferCSSPageSize: true }); await p.emulateMedia({ media: 'screen' });
104         fs.writeFileSync(OUT + '/expect-' + fmt + '.json', JSON.stringify(await p.evaluate(([ids, f, l]) => ids.map(id => TST.fleet.qrPayload(id, l, f)), [allIds, fmt, lvl])));
105       }
106
107       // 9, 10. Неизвестный и повреждённый код; HTML/JS в содержимом не выполняется
108       const scan = async txt => {
109         await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop').forEach(x => x.remove()));
110         await p.fill('#qrScanInput', txt); await p.evaluate(() => document.querySelector('[data-act="qr-scan-text"]').click()); await p.waitForTimeout(250);
111         return p.evaluate(() => { const m = document.querySelector('.f-modal-backdrop .qr-card'); return m ? m.innerText.slice(0, 300) : null; });
112       };
113       const r9 = await scan('TST:1:v_neizvestnyi_000');
114       check(9, 'Неизвестный идентификатор → «не найдена в локальной базе»', r9 && /не найдена в локальной базе/.test(r9), r9 && r9.split('\n')[0]);
115       await p.evaluate(() => { window.__pwn = 0; });
116       const bad = ['просто текст', 'TST:2:v_abc_def', 'TST:1:<img src=x onerror="window.__pwn=1">', 'TST:1:v_ab\n<script>window.__pwn=1</script>', 'https://example.com/'];
117       const rb = []; for (const t of bad) rb.push(await scan(t));
118       const pwn = await p.evaluate(() => window.__pwn);
119       check(10, 'Повреждённые и чужие коды отклоняются, содержимое не выполняется', rb.every(x => x && !/основные сведения/i.test(x)) && pwn === 0, rb.map(x => (x || '').split('\n')[0]).join(' | '));
120
121       // 7 (часть). Содержимое напечатанного кода → карточка именно этой единицы
122       const card1 = await scan(exp1);
123       check(7, 'Содержимое напечатанного кода открывает карточку именно этой единицы', card1 && card1.includes('Испытательная кухня КП-130 №1 (изм.)') && !/отличаются от базы/.test(card1), (card1 || '').split('\n').slice(0, 2).join(' / '));
124       const oneLine = await scan(exp1.replace(/\n/g, ' '));   // строки склеены при вставке
125       check(7, 'Склеенное в одну строку содержимое тоже распознаётся', oneLine && /основные сведения/i.test(oneLine) && oneLine.includes('Испытательная кухня КП-130 №1 (изм.)'));
126       // Наклейка напечатана, затем название изменено → при сканировании видно расхождение
127       await p.evaluate(id => { location.hash = '#fleet/vehicle/' + id; }, v1.id); await p.waitForTimeout(400);
128       await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop').forEach(x => x.remove()));
129       await p.evaluate(() => document.querySelector('[data-act="edit-vehicle"]').click()); await p.waitForSelector('#vName');
130       await p.fill('#vName', 'Кухня КП-130 — переименована'); await p.click('#vSave'); await p.waitForTimeout(300);
131       const stale = await p.evaluate(id => TST.fleet.qrState(id), v1.id);
132       await p.evaluate(() => location.hash = '#fleet/qr'); await p.waitForTimeout(400);
133       const card1b = await scan(exp1);
134       check(13, 'После правки названия наклейка помечена «устарела», при сканировании показано расхождение (без подмены записи)', stale === 'stale' && card1b && /отличаются от базы/.test(card1b), stale);
135       await p.screenshot({ path: OUT + '/scan-card.png' });
136       const card5 = await scan('TST:1:' + v5.id);
137       const naCount = card5 ? (card5.match(/Не указано/g) || []).length : 0;
138       check(12, 'Пустые поля показываются как «Не указано»', naCount >= 5, naCount + ' полей «Не указано»');
139       const scanUnchanged = await p.evaluate(() => localStorage.getItem('spravochnik-tst-fleet-v1').length);
140
```

18. tests/qr.spec.js (продолжение)

```
    // 11. Резервная копия: экспорт → очистка → восстановление
    await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop').forEach(x => x.remove()));
    await p.evaluate(() => location.hash = '#fleet/settings'); await p.waitForTimeout(500);
    const [dl] = await Promise.all([p.waitForEvent('download'), p.evaluate(() => document.querySelector('[data-act="backup-export"]').click())]);
    await dl.saveAs(OUT + '/backup.zip');
    const before = await p.evaluate(() => JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')).vehicles.map(v => v.id + '|' + (v.qr ? v.qr.digest : '')).sort().join(','));
    await p.evaluate(() => { const d = JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')); d.vehicles = []; localStorage.setItem('spravochnik-tst-fleet-v1', JSON.stringify(d)); });
    await p.reload(); await p.waitForTimeout(1000); await p.evaluate(() => location.hash = '#fleet/settings'); await p.waitForTimeout(400);
    const [fc] = await Promise.all([p.waitForEvent('filechooser'), p.evaluate(() => document.querySelector('[data-act="backup-import"]').click())]);
    await fc.setFiles(OUT + '/backup.zip'); await p.waitForTimeout(600);
    await p.evaluate(() => { const y = document.querySelector('[data-dec="restore"]') || document.querySelector('[data-yes]'); if (y) y.click(); }); await p.waitForTimeout(1200);
    const restored = await p.evaluate(() => JSON.parse(localStorage.getItem('spravochnik-tst-fleet-v1')).vehicles.map(v => v.id + '|' + (v.qr ? v.qr.digest : '')).sort().join(','));
    check(11, 'Экспорт/импорт сохраняет идентификаторы и отметки QR', restored === before && before.length > 0, restored.split(',').length + ' записей');

    // 8, 14. Без сети
    const ext = await p.evaluate(() =>
[...document.querySelectorAll('script[src],link[href],img[src],iframe[src]')].map(n => n.src || n.href).filter(u => /^https?:/.test(u)));
    check(8, 'Ни одного сетевого запроса за весь прогон (все внешние запросы заблокированы и учтены)', netReq.length === 0, netReq.length + ' запросов: ' + netReq.slice(0, 3).join(', '));
    check(14, 'Нет внешних скриптов, стилей, изображений', ext.length === 0, ext.join(', '));

    // 15. Регрессия: все разделы учёта, справочник, обзор, окно печати, справка
    const views = ['park', 'dashboard', 'trips', 'planning', 'documents', 'reports', 'qr', 'settings'];
    for (const v of views) { await p.evaluate(x => location.hash = '#fleet/' + x, v); await p.waitForTimeout(250); }
    await p.evaluate(() => location.hash = '#catalog/ka-220/to1'); await p.waitForTimeout(300);
    await p.evaluate(() => location.hash = '#overview'); await p.waitForTimeout(300);
    await p.evaluate(() => location.hash = '#fleet/reports'); await p.waitForTimeout(300);
    await p.keyboard.press('Control+p'); await p.waitForTimeout(500);
    const pw = await p.evaluate(() => !!document.querySelector('.pw')); await p.keyboard.press('Escape');
    await p.keyboard.press('F1'); await p.waitForTimeout(500);
    const help = await p.evaluate(() => !!document.querySelector('#ref-s11')); await p.keyboard.press('Escape');
    check(15, 'Регрессия: разделы, окно печати документов и справка (с разделом 11) работают без ошибок', errs.length === 0 && pw && help, 'ошибок JS: ' + errs.length + (errs[0] ? ' (' + errs[0] + ')' : ''));
    fs.writeFileSync(OUT + '/results.json', JSON.stringify(results, null, 1));
    await b.close();
})().catch(e => { console.error('SUITE ERROR', e); process.exit(1); });
```

19. tests/season.spec.js

```
// Запуск: node tests/season.spec.js [папка-для-снимков]
const path = require('path'), fs = require('fs');
let pw; try { pw = require('playwright'); } catch (e) { pw = require('/opt/node22/lib/node_modules/playwright'); }
const { chromium } = pw;
const OUT = process.argv[2] || path.join(__dirname, 'out'); fs.mkdirSync(OUT, { recursive: true });
const PAGE_URL = require('url').pathToFileURL(path.resolve(__dirname, '..', 'index.html')).href; const LS = 'spravochnik-tst-fleet-v1';
const res = []; const check = (n, ok, i) => { res.push(ok); console.log((ok ? 'PASS ' : 'FAIL ') + n + (i ? ' — ' + i : '')); };
(async () => {
  const b = await chromium.launch(); const ctx = await b.newContext({ viewport: { width: 1440, height: 900 } });
  const net = []; await ctx.route('**/*', r => { const u = r.request().url(); if (/^(file|data|blob):/.test(u)) return r.continue(); net.push(u); r.abort(); });
  const p = await ctx.newPage(); const errs = []; p.on('pageerror', e => errs.push(e.message));
  await p.addInitScript(() => { window.print = () => {}; });
  const URL0 = PAGE_URL;
  const load = async h => { await p.goto(URL0 + h); await p.reload(); await p.waitForTimeout(1500); await p.evaluate(() => document.querySelectorAll('.f-toast-close').forEach(x => x.click())); };
  const db = () => p.evaluate(k => JSON.parse(localStorage.getItem(k)), LS);
  const act = (a, id, ex) => p.evaluate(([a, id, ex]) => { const b = document.createElement('button'); b.dataset.act = a; if (id) b.dataset.id = id; Object.entries(ex || {}).forEach(([k, v]) => { b.dataset[k] = v; }); document.getElementById('fleetView').appendChild(b); b.click(); b.remove(); }, [a, id, ex]);
  await load('#fleet/park');
  const d0 = await db();
  const v = d0.vehicles.find(x => x.catalogId && /КА-220/.test(x.name)) || d0.vehicles.find(x => x.catalogId);
  check('Есть техника, привязанная к справочнику', !!v, v && v.name);

  // 1. Шаблон работ в форме ТО
  await act('add-maint', v.id); await p.waitForSelector('#mKind'); await p.waitForTimeout(300);
  await p.selectOption('#mKind', 'ТО-1'); await p.waitForTimeout(200);
  const rows = await p.evaluate(() => document.querySelectorAll('#mCheck [data-mc]').length);
  check('Перечень операций регламента показан с галочками', rows > 0, rows + ' операций');
  const hidden0 = await p.evaluate(() => document.getElementById('mCheck').hidden);
  check('Блок виден, пока есть регламент', !hidden0);
  await p.click('#mCheck [data-mc="0"]'); await p.click('#mCheck [data-mc="1"]');
  const cnt = await p.evaluate(() => document.getElementById('mcCount').textContent);
  check('Счётчик выполненных работ', /выполнено 2 из \d+/.test(cnt), cnt);
  await p.click('[data-mc-all]');
  check('«Отметить все» отмечает всё', await p.evaluate(() => [...document.querySelectorAll('#mCheck [data-mc]')].every(x => x.checked)));
  await p.click('[data-mc-none]'); await p.click('#mCheck [data-mc="0"]'); await p.click('#mCheck [data-mc="2"]');
  await p.screenshot({ path: OUT + '/form.png' });
  await p.selectOption('#mKind', 'ТО-2'); await p.waitForTimeout(200);
  const after2 = await p.evaluate(() => document.getElementById('mcCount').textContent);
  check('При смене вида ТО перечень сменяется, отметки не переносятся', /выполнено 0 из/.test(after2), after2);
  await p.selectOption('#mKind', 'ТО-1'); await p.waitForTimeout(200);
  const back = await p.evaluate(() => document.getElementById('mcCount').textContent);
  check('Возврат к ТО-1 вернул прежние отметки', /выполнено 2 из 7/.test(back), back);
  await p.click('#mSave'); await p.waitForTimeout(500);
  const d1 = await db();
  const rec = d1.maint.filter(m => m.vehicleId === v.id).sort((a, b) => (b.createdAt || '').localeCompare(a.createdAt || ''))[0];
  check('Запись ТО хранит снимок перечня с отметками', rec.checklist && rec.checklist.kind === 'ТО-1' && rec.checklist.items.filter(x => x.done).length === 2, rec.checklist && rec.checklist.items.length + ' операций');
  check('Пустое «Выполненные работы» заполнено отмеченными операциями', rec.works && rec.works.split('\n').length === 2, rec.works.slice(0, 60));
  await load('#fleet/vehicle/' + v.id);
  await p.evaluate(() => [...document.querySelectorAll('.f-tab')].find(t => /ТО/.test(t.textContent) && !/Моточасы/.test(t.textContent))?.click()); await p.waitForTimeout(400);
  const badge = await p.evaluate(() => document.querySelector('.mc-badge')?.textContent || '');
  check('В журнале ТО видно «По регламенту: 2 из N»', /По регламенту: 2 из \d+/.test(badge), badge);
  // повторное открытие записи: отметки сохранились
  await act('edit-maint', rec.id); await p.waitForSelector('#mKind'); await p.waitForTimeout(300);
  const reopened = await p.evaluate(() => document.getElementById('mcCount').textContent);
  check('При редактировании записи отметки восстановлены', /выполнено 2 из/.test(reopened), reopened);
  await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop').forEach(x => x.remove()));

  // 2. Сезонное обслуживание: настройки → напоминание
  await load('#fleet/settings');
  const winter = await p.evaluate(() => { const d = new Date(); d.setDate(d.getDate() + 10); return [d.getDate(), d.getMonth() + 1]; });
  await p.fill('#setWinDay', String(winter[0])); await p.selectOption('#setWinMonth', String(winter[1])); await p.dispatchEvent('#setWinMonth', 'change'); await p.waitForTimeout(300);
  await p.evaluate(() => { const d = new Date(); d.setDate(d.getDate() + 180); document.getElementById('setSumDay').value = d.getDate(); document.getElementById('setSumMonth').value = d.getMonth() + 1; document.getElementById('setSumMonth').dispatchEvent(new Event('change')); });
  await p.waitForTimeout(300);
  await p.screenshot({ path: OUT + '/settings.png', fullPage: true });
  const s1 = (await db()).settings.seasons;
  check('Даты сезонов сохраняются в настройках', s1 && /^\d\d-\d\d$/.test(s1.winter) && /^\d\d-\d\d$/.test(s1.summer), JSON.stringify(s1));
  await p.fill('#setWinDay', '31'); await p.selectOption('#setWinMonth', '2'); await p.dispatchEvent('#setWinMonth', 'change'); await p.waitForTimeout(300);
  check('Некорректная дата (31 февраля) не сохраняется', (await db()).settings.seasons.winter === s1.winter);
  await load('#fleet/command');
  const tile = await p.evaluate(() => { const t = document.querySelector('[data-list="season"]'); return t ? t.innerText.replace(/\n+/g, ' | ') : null; });
  check('В «Пункте управления» есть плитка сезонного ТО', !!tile && /Сезонное ТО/i.test(tile), tile);
  const total = await p.evaluate(() => TST.fleet.helpData ? 0 : 0);
  const bell = await p.evaluate(() => { document.getElementById('fleetBell').click(); return
```

19. tests/season.spec.js (продолжение)

```
    document.getElementById('fleetNotifPanel').innerText; });
73    check('В колоколе есть напоминание о переводе на зимний режим', /Перевод на зимний режим/.test(bell),
    (bell.match(/Перевод на зимний[^\n]*/) || [''])[0]);
74    await p.evaluate(() => document.getElementById('fleetBell').click());
75    await act('season'); await p.waitForTimeout(400);
76    const listN = await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop [data-cp-open]').length);
77    const tileN = Number(await p.evaluate(() => document.querySelector('[data-list="season"] .f-stat-
    value').textContent));
78    check('Перечень по плитке совпадает со счётчиком', listN === tileN && tileN > 0, tileN + ' машин');
79    await p.screenshot({ path: OUT + '/season-list.png' });
80    await p.evaluate(() => document.querySelector('[data-cp-task]').click()); await p.waitForTimeout(800);
81    const doc = await p.evaluate(() => document.body.innerText);
82    check('Задание на сезонное обслуживание формируется', /сезонного обслуживания/.test(doc) && /зимний
    режим/.test(doc));
83    await p.keyboard.press('Escape'); await p.evaluate(() => document.querySelectorAll('.f-modal-backdrop').forEach(x =>
     x.remove()));
84
85    // 3. СО с отметкой сезона снимает машину из перечня
86    await load('#fleet/command');
87    await act('add-maint', v.id); await p.waitForSelector('#mKind');
88    await p.selectOption('#mKind', 'Сезонное обслуживание (СО)'); await p.waitForTimeout(200);
89    check('Для СО появляется выбор сезона, по умолчанию — ближайший (зимний)', await p.evaluate(() =>
    !document.getElementById('mSeasonBox').hidden && document.getElementById('mSeason').value === 'winter'));
90    await p.screenshot({ path: OUT + '/form-so.png' });
91    await p.click('#mSave'); await p.waitForTimeout(500);
92    await load('#fleet/command');
93    const tile2 = Number(await p.evaluate(() => document.querySelector('[data-list="season"] .f-stat-
    value').textContent));
94    check('После СО с отметкой сезона плитка уменьшилась на 1', tile2 === tileN - 1, tileN + ' → ' + tile2);
95    await p.screenshot({ path: OUT + '/command.png', fullPage: false });
96
97    check('Без сетевых запросов и ошибок JavaScript', net.length === 0 && errs.length === 0, `запросов ${net.length},
    ошибок ${errs.length}${errs[0] ? ': ' + errs[0] : ''}`);
98    console.log(`ИТОГ: ${res.filter(Boolean).length} из ${res.length}`);
99    await b.close();
100 })();
101
```

20. tests/vfiles.spec.js

```
// Запуск: node tests/vfiles.spec.js [папка-для-снимков]
// Фото единицы техники и документы на машину (паспорт, формуляр, акты приёма-передачи)
const path = require('path'), fs = require('fs');
let pw; try { pw = require('playwright'); } catch (e) { pw = require('/opt/node22/lib/node_modules/playwright'); }
const { chromium } = pw;
const OUT = process.argv[2] || path.join(__dirname, 'out'); fs.mkdirSync(OUT, { recursive: true });
const PAGE_URL = require('url').pathToFileURL(path.resolve(__dirname, '..', 'index.html')).href; const LS = 'spravochnik-tst-fleet-v1';
const res = []; const check = (n, ok, i) => { res.push(ok); console.log((ok ? 'PASS ' : 'FAIL ') + n + (i ? ' — ' + i : '')); };
(async () => {
  const b = await chromium.launch(); const ctx = await b.newContext({ viewport: { width: 1440, height: 900 } });
  const net = []; await ctx.route('**/*', r => { const u = r.request().url(); if (/^(file|data|blob):/.test(u)) return r.continue(); net.push(u); r.abort(); });
  const p = await ctx.newPage(); const errs = []; p.on('pageerror', e => errs.push(e.message));
  const load = async h => { await p.goto(PAGE_URL + h); await p.reload(); await p.waitForTimeout(1500); await p.evaluate(() => document.querySelectorAll('.f-toast-close').forEach(x => x.click())); };
  const db = () => p.evaluate(k => JSON.parse(localStorage.getItem(k)), LS);
  // Большое изображение 4000×3000 из холста — проверка сжатия
  const bigPng = () => p.evaluate(async () => {
    const c = document.createElement('canvas'); c.width = 4000; c.height = 3000; const g = c.getContext('2d');
    const gr = g.createLinearGradient(0, 0, 4000, 3000); gr.addColorStop(0, '#2a6'); gr.addColorStop(1, '#e83'); g.fillStyle = gr; g.fillRect(0, 0, 4000, 3000);
    for (let i = 0; i < 400; i++) { g.fillStyle = `hsl(${i * 7},70%,50%)`; g.fillRect((i * 97) % 3900, (i * 53) % 2900, 90, 60); }
    const blob = await new Promise(r => c.toBlob(r, 'image/png')); return Array.from(new Uint8Array(await blob.arrayBuffer()));
  });
  await load('#fleet/park');
  const v = (await db()).vehicles[0];
  await load('#fleet/vehicle/' + v.id);
  const tab = await p.evaluate(() => [...document.querySelectorAll('.f-tab')].map(t => t.textContent.trim()));
  check('Во вкладках карточки есть «Фото и документы»', tab.some(t => /^Фото и документы \(0\)/.test(t)), tab.join(' | '));
  await p.evaluate(() => [...document.querySelectorAll('.f-tab')].find(t => /Фото и документы/.test(t.textContent)).click()); await p.waitForTimeout(300);

  // 1. Фотографии
  const png = await bigPng();
  await p.setInputFiles('#vfPhotoIn', [{ name: 'общий вид.png', mimeType: 'image/png', buffer: Buffer.from(png) }, { name: 'табличка.png', mimeType: 'image/png', buffer: Buffer.from(png) }, { name: 'заметка.txt', mimeType: 'text/plain', buffer: Buffer.from('x') }]);
  await p.waitForFunction(() => document.querySelectorAll('.vf-ph').length === 2, null, { timeout: 15000 }).catch(() => {});
  let d = await db(); let vv = d.vehicles.find(x => x.id === v.id);
  check('Добавлены 2 фото, текстовый файл отклонён', (vv.photos || []).length === 2, (vv.photos || []).length + '');
  const ph = vv.photos[0];
  check('Фото сжато до 1600 px по длинной стороне', ph.w === 1600 && ph.h === 1200, ph.w + '×' + ph.h);
  check('Размер сжатого фото меньше исходного PNG', ph.size < png.length / 2, ph.size + ' < ' + png.length);
  check('Главное фото назначено первым', vv.mainPhotoId === ph.id);
  check('Миниатюра — отдельный файл, в записи нет данных изображения', !!ph.thumbId && JSON.stringify(vv).length < 4000, JSON.stringify(vv).length + ' байт');
  const thumbOk = await p.evaluate(async () => { await new Promise(r => setTimeout(r, 600)); const im = document.querySelector('.vf-ph img'); return !!(im && im.src.startsWith('blob:') && im.naturalWidth > 0 && im.naturalWidth <= 320); });
  check('Миниатюры загружаются и не больше 320 px', thumbOk);
  await p.screenshot({ path: OUT + '/photos.png' });

  // 2. Просмотр, подпись, главное фото
  await p.click('.vf-ph >> nth=1'); await p.waitForFunction(() => { const i = document.getElementById('vwImg'); return i && i.naturalWidth > 0; }, null, { timeout: 10000 });
  const shown = await p.evaluate(() => { const i = document.getElementById('vwImg'); return i.naturalWidth; });
  check('Фото открывается в просмотре', shown === 1600, shown + '');
  await p.fill('#vwCap', 'Табличка с заводским номером'); await p.dispatchEvent('#vwCap', 'change');
  await p.click('#vwMain'); await p.waitForTimeout(200);
  d = await db(); vv = d.vehicles.find(x => x.id === v.id);
  check('Подпись и главное фото сохранены', vv.photos[1].caption === 'Табличка с заводским номером' && vv.mainPhotoId === vv.photos[1].id);
  await p.keyboard.press('ArrowRight'); await p.waitForTimeout(150);
  check('Стрелки листают фото', /1 из 2/.test(await p.textContent('#vwSub')));
  await p.click('.f-modal-close'); await p.waitForTimeout(200);

  // 3. Документы
  let nDocs = 0;
  for (const [kind, num, fname] of [['Паспорт', 'АБ 123456', 'паспорт.pdf'], ['Формуляр', 'Ф-77', 'формуляр.pdf'], ['Акт приёма-передачи', '12', 'акт.png']]) {
    await p.click('[data-do="doc-add"]'); await p.waitForSelector('#vdKind');
    await p.selectOption('#vdKind', kind); await p.fill('#vdNumber', num); await p.fill('#vdDate', '2026-03-15');
    const buf = fname.endsWith('.png') ? Buffer.from(png) : Buffer.from('%PDF-1.4 test');
    await p.setInputFiles('#vdFiles', [{ name: fname, mimeType: fname.endsWith('.png') ? 'image/png' : 'application/pdf', buffer: buf }]);
    await p.click('#vdSave'); nDocs++; await p.waitForFunction(n => document.querySelectorAll('.vf-doc').length === n, nDocs, { timeout: 15000 }).catch(() => {});
  }
  d = await db(); vv = d.vehicles.find(x => x.id === v.id);
  check('Три документа сохранены', (vv.vdocs || []).length === 3 && vv.vdocs.map(x => x.kind).join() === 'Паспорт,Формуляр,Акт приёма-передачи');
  check('Скан-изображение акта сжато (PNG 4000 px → JPEG)', vv.vdocs[2].files[0].type === 'image/jpeg' && vv.vdocs[2].files[0].size < png.length, vv.vdocs[2].files[0].size + '');
  check('Файлы документов лежат в хранилище, а не в записи', JSON.stringify(vv).length < 5000);
```

20. tests/vfiles.spec.js (продолжение)

```
69        const rows = await p.evaluate(() => document.querySelectorAll('.vf-doc').length);
70        check('Документы показаны в списке', rows === 3, rows + '');
71        await p.click('[data-do="doc-add"]'); await p.waitForSelector('#vdKind'); await p.selectOption('#vdKind', 'Другой документ'); await p.click('#vdSave'); await p.waitForTimeout(300);
72        check('«Другой документ» без названия не сохраняется', (await db()).vehicles.find(x => x.id === v.id).vdocs.length === 3);
73        await p.fill('#vdTitle', 'Свидетельство о регистрации'); await p.click('#vdSave'); await p.waitForTimeout(500);
74        check('«Другой документ» с названием сохранён', (await db()).vehicles.find(x => x.id === v.id).vdocs.length === 4);
75        await p.screenshot({ path: OUT + '/docs.png' });
76
77        // 4. Скачивание PDF и открытие скана
78        const dlp = p.waitForEvent('download');
79        await p.click('[data-do="docfile"][data-did] >> nth=0'); const dl = await dlp;
80        check('PDF скачивается с исходным именем', dl.suggestedFilename().length > 0, dl.suggestedFilename());
81        const content = fs.readFileSync(await dl.path(), 'latin1');
82        check('Содержимое скачанного файла совпадает', content === '%PDF-1.4 test');
83
84        // 5. Шапка, список
85        await p.evaluate(() => document.querySelector('[data-act="back-park"]').click()); await p.waitForTimeout(700);
86        const mini = await p.evaluate(() => { const i = document.querySelector('img.vf-mini'); return !!(i && i.src.startsWith('blob:')); });
87        check('Фото показано в списке техники', mini);
88
89        // 6. Редактирование и удаление
90        await load('#fleet/vehicle/' + v.id);
91        await p.evaluate(() => [...document.querySelectorAll('.f-tab')].find(t => /Фото и документы/.test(t.textContent)).click()); await p.waitForTimeout(300);
92        await p.click('[data-do="doc-edit"] >> nth=0'); await p.waitForSelector('#vdKind');
93        await p.fill('#vdNumber', 'АБ 999999'); await p.click('#vdSave'); await p.waitForTimeout(500);
94        check('Документ редактируется', (await db()).vehicles.find(x => x.id === v.id).vdocs[0].number === 'АБ 999999');
95        const before = await p.evaluate(() => new Promise(r => { const q = indexedDB.open('tst-fleet-files'); q.onsuccess = () => { const t = q.result.transaction('files').objectStore('files').getAllKeys(); t.onsuccess = () => r(t.result.length); }; }));
96        await p.click('[data-do="doc-del"] >> nth=0'); await p.click('[data-yes]'); await p.waitForTimeout(1200);
97        d = await db(); vv = d.vehicles.find(x => x.id === v.id);
98        const after = await p.evaluate(() => new Promise(r => { const q = indexedDB.open('tst-fleet-files'); q.onsuccess = () => { const t = q.result.transaction('files').objectStore('files').getAllKeys(); t.onsuccess = () => r(t.result.length); }; }));
99        check('Документ удалён', vv.vdocs.length === 3);
100       check('Файл удалённого документа убран из хранилища', after < before, before + ' → ' + after);
101
102       // 7. Резервная копия содержит файлы техники
103       await load('#fleet/settings');
104       const dlb = p.waitForEvent('download'); await p.evaluate(() => document.querySelector('[data-act="backup-export"]').click()); const zip = await dlb;
105       const zb = fs.readFileSync(await zip.path());
106       const names = (zb.toString('latin1').match(/files\/[A-Za-z0-9_-]+\.[a-z]+/g) || []);
107       const mine = (await db()).vehicles.find(x => x.id === v.id);
108       const need = mine.photos.length * 2 + mine.vdocs.reduce((s, x) => s + x.files.length, 0);
109       check('В резервной копии есть все фото, миниатюры и сканы', new Set(names).size >= need, new Set(names).size + ' из ' + need);
110
111       // 8. Удаление техники вместе с файлами, отмена возвращает
112       await load('#fleet/park');
113       await p.evaluate(id => { const b = document.createElement('button'); b.dataset.act = 'del-vehicle'; b.dataset.id = id; document.getElementById('fleetView').appendChild(b); b.click(); }, v.id);
114       await p.waitForTimeout(800);
115       const gone = (await db()).vehicles.some(x => x.id === v.id);
116       check('Техника удалена в корзину', !gone);
117       const undo = await p.waitForSelector('.f-toast-action', { timeout: 4000 }).catch(() => null);
118       if (undo) { await undo.click(); await p.waitForTimeout(500); }
119       const back = (await db()).vehicles.find(x => x.id === v.id);
120       check('Отмена удаления возвращает технику вместе с фото и документами', !!back && (back.photos || []).length === 2 && (back.vdocs || []).length === 3, `undo=${!!undo} back=${!!back} ph=${back && (back.photos || []).length}`);
121       const st = await p.evaluate(() => TST.platform.runTests());
122       const badT = st.filter(r => r.ok === false || r.pass === false);
123       check('Самотесты платформы проходят (включая «Файлы техники»)', badT.length === 0 && st.some(r => r.module === 'vfiles'), st.length + ' тестов' + (badT.length ? '; упали: ' + badT.map(r => r.name).join(',') : ''));
124       check('Внешних запросов нет', net.length === 0, net.join(','));
125       check('Ошибок на странице нет', errs.length === 0, errs.join('; '));
126       await b.close();
127       const bad = res.filter(x => !x).length; console.log(`\nИтого: ${res.length - bad}/${res.length}`); process.exit(bad ? 1 : 0);
128    })();
129
```

21. app/test/storage.test.js

```
'use strict';
/* Проверка файлового хранилища без Electron: node test/storage.test.js */
const fs = require('fs'), os = require('os'), path = require('path'), assert = require('assert');
const { createStorage } = require('../storage');
let n = 0, failed = 0;
const t = (name, fn) => { try { fn(); n++; console.log('PASS ' + name); } catch (e) { failed++; console.log('FAIL ' + name + ' — ' + e.message); } };
const tmp = () => fs.mkdtempSync(path.join(os.tmpdir(), 'tst-storage-'));
const db = v => JSON.stringify({ version: 2, vehicles: [{ id: 'v_aaaa_0001', name: v }] });

t('Пустая папка: данных нет', () => { const s = createStorage(tmp()); assert.strictEqual(s.readDb().raw, null); });
t('Запись и чтение базы, русский текст не портится', () => {
  const s = createStorage(tmp()); s.writeDb(db('Кухня КА-220 «№1»')); assert.strictEqual(s.readDb().raw, db('Кухня КА-220 «№1»'));
});
t('Временный файл не остаётся после записи', () => { const d = tmp(); createStorage(d).writeDb(db('x')); assert.ok(!fs.existsSync(path.join(d, 'db.json.tmp'))); });
t('Некорректные данные не записываются, прежний файл цел', () => {
  const s = createStorage(tmp()); s.writeDb(db('ok'));
  assert.throws(() => s.writeDb('не json')); assert.throws(() => s.writeDb('[1,2]')); assert.throws(() => s.writeDb(undefined));
  assert.strictEqual(s.readDb().raw, db('ok'));
});
t('Суточная копия создаётся перед первой записью за день', () => {
  const d = tmp(), s = createStorage(d); s.writeDb(db('v1')); s.writeDb(db('v2'));
  const b = fs.readdirSync(path.join(d, 'backups'));
  assert.strictEqual(b.length, 1); assert.strictEqual(fs.readFileSync(path.join(d, 'backups', b[0]), 'utf8'), db('v1'));
});
t('Хранится не больше 14 суточных копий', () => {
  const d = tmp(), s = createStorage(d); s.writeDb(db('a'));
  for (let i = 1; i <= 20; i++) fs.writeFileSync(path.join(d, 'backups', `db-2020-01-${String(i).padStart(2,'0')}.json`), db('old' + i));
  s.writeDb(db('b'));
  assert.ok(fs.readdirSync(path.join(d, 'backups')).length <= 14);
});
t('Повреждённый db.json: берётся копия, повреждённый файл сохранён отдельно', () => {
  const d = tmp(), s = createStorage(d); s.writeDb(db('v1')); s.writeDb(db('v2'));
  fs.writeFileSync(path.join(d, 'db.json'), '{"version":2,"vehicles":[{"id":');   // оборван на середине записи
  const r = createStorage(d).readDb();
  assert.strictEqual(r.raw, db('v1')); assert.ok(r.recovered && r.recovered.reason === 'повреждён' && r.recovered.from === 'db.prev.json');
  assert.ok(fs.readdirSync(d).some(n => n.startsWith('db.corrupt-')));
  assert.strictEqual(createStorage(d).readDb().raw, db('v1'));   // восстановленное стало основным
});
t('Файл удалён, копии есть: данные восстанавливаются', () => {
  const d = tmp(), s = createStorage(d); s.writeDb(db('v1')); s.writeDb(db('v2')); fs.unlinkSync(path.join(d, 'db.json'));
  const r = createStorage(d).readDb(); assert.strictEqual(r.raw, db('v1')); assert.strictEqual(r.recovered.reason, 'отсутствует');
});
t('Предыдущая версия теряет только последнее изменение', () => {
  const d = tmp(), s = createStorage(d); s.writeDb(db('v1')); s.writeDb(db('v2')); s.writeDb(db('v3'));
  fs.writeFileSync(path.join(d, 'db.json'), '{');
  assert.strictEqual(createStorage(d).readDb().raw, db('v2'));
});
t('Повреждены db.json и db.prev.json: берётся суточная копия', () => {
  const d = tmp(), s = createStorage(d); s.writeDb(db('v1')); s.writeDb(db('v2')); s.writeDb(db('v3'));
  fs.writeFileSync(path.join(d, 'db.json'), '{'); fs.writeFileSync(path.join(d, 'db.prev.json'), 'мусор');
  const r = createStorage(d).readDb(); assert.strictEqual(r.raw, db('v1')); assert.ok(/^db-/.test(r.recovered.from));
});
t('Повреждены и файл, и копии: данных нет, но файлы не потеряны', () => {
  const d = tmp(), s = createStorage(d); s.writeDb(db('v1')); s.writeDb(db('v2'));
  fs.readdirSync(path.join(d, 'backups')).forEach(n => fs.writeFileSync(path.join(d, 'backups', n), 'мусор'));
  fs.writeFileSync(path.join(d, 'db.json'), 'мусор');
  fs.writeFileSync(path.join(d, 'db.prev.json'), 'мусор');
  const r = createStorage(d).readDb(); assert.strictEqual(r.raw, null); assert.ok(r.recovered && r.recovered.corruptFile);
});
t('Вложения: запись, чтение, список, удаление', () => {
  const s = createStorage(tmp()); const bytes = Buffer.from([0, 1, 2, 250, 255]);
  s.putFile('file_abc_123', { name: 'Акт №1.pdf', type: 'application/pdf' }, bytes);
  const g = s.getFile('file_abc_123'); assert.deepStrictEqual([...g.bytes], [...bytes]);
  assert.strictEqual(g.meta.name, 'Акт №1.pdf'); assert.strictEqual(g.meta.size, 5);
  assert.deepStrictEqual(s.listFiles(), ['file_abc_123']); s.delFile('file_abc_123');
  assert.strictEqual(s.getFile('file_abc_123'), null);
});
t('Недопустимые идентификаторы (выход за папку) отклоняются', () => {
  const s = createStorage(tmp());
  ['../x', '..\\x', 'a/b', '', 'ab', 'x'.repeat(200), null, 'file/../../db'].forEach(id => {
    assert.throws(() => s.putFile(id, {}, Buffer.alloc(1)), /идентификатор/i); assert.throws(() => s.getFile(id));
  assert.throws(() => s.delFile(id)); assert.throws(() => s.putSnapshot(id, {}));
  });
});
t('Точки восстановления: запись, список, удаление', () => {
  const s = createStorage(tmp()); s.putSnapshot('s_1111_aaaa', { id: 's_1111_aaaa', at: 'x', data: db('s') });
  s.putSnapshot('s_2222_bbbb', { id: 's_2222_bbbb', data: '{}' });
  assert.strictEqual(s.listSnapshots().length, 2); s.delSnapshot('s_1111_aaaa');
  assert.deepStrictEqual(s.listSnapshots().map(x => x.id), ['s_2222_bbbb']);
});
```

21. app/test/storage.test.js (продолжение)

```
76    t('Копирование всех данных в другую папку', () => {
77      const d = tmp(), s = createStorage(d); s.writeDb(db('v1')); s.putFile('file_abc_123', { name: 'a' },
    Buffer.from('z')); s.putSnapshot('s_1111_aaaa', { id: 's_1111_aaaa' });
78      const target = path.join(tmp(), 'новая папка'); s.copyAllTo(target);
79      const s2 = createStorage(target); assert.strictEqual(s2.readDb().raw, db('v1'));
    assert.ok(s2.getFile('file_abc_123')); assert.strictEqual(s2.listSnapshots().length, 1);
80    });
81    t('Большая база (10 МБ) пишется и читается целиком', () => {
82      const big = JSON.stringify({ version: 2, vehicles: Array.from({ length: 40000 }, (_, i) => ({ id: 'v_' + i +
    '_abcd', name: 'Машина ' + i, note: 'я'.repeat(200) })) });
83      const s = createStorage(tmp()); s.writeDb(big); assert.strictEqual(s.readDb().raw, big); assert.ok(big.length >
    8e6);
84    });
85    console.log(`ИТОГ: ${n} из ${n + failed}`); process.exit(failed ? 1 : 0);
86
```

22. app/test/e2e.test.js

```
'use strict';
/* Сквозной тест приложения на Electron (нужен виртуальный экран на Linux):
     xvfb-run -a node test/e2e.test.js
   Проверяет главное: данные лежат в файлах и переживают потерю профиля браузера. */
const path = require('path'), fs = require('fs'), os = require('os');
let pw; try { pw = require('playwright'); } catch (e) { pw = require('/opt/node22/lib/node_modules/playwright'); }
const electronPath = require('electron');
const APP = path.resolve(__dirname, '..');
const res = []; const check = (n, ok, i) => { res.push(!!ok); console.log((ok ? 'PASS ' : 'FAIL ') + n + (i ? ' — ' + i : '')); };
const tmp = p => fs.mkdtempSync(path.join(os.tmpdir(), p));

(async () => {
  const dataDir = path.join(tmp('tst-e2e-'), 'Данные'), dlDir = tmp('tst-dl-');
  const launch = async () => {
    const app = await pw._electron.launch({ executablePath: electronPath, args: [APP, '--no-sandbox'],
      env: Object.assign({}, process.env, { TST_USER_DATA: tmp('tst-profile-'), TST_DATA_DIR: dataDir, TST_TEST: '1', TST_DOWNLOAD_DIR: dlDir }) });
    const page = await app.firstWindow();
    const errs = []; page.on('pageerror', e => errs.push(e.message));
    await page.waitForLoadState('domcontentloaded'); await page.waitForTimeout(2500);
    await page.evaluate(() => document.querySelectorAll('.f-toast-close').forEach(x => x.click()));
    return { app, page, errs };
  };
  const readDb = () => JSON.parse(fs.readFileSync(path.join(dataDir, 'db.json'), 'utf8'));
  const act = (page, a, id) => page.evaluate(([a, id]) => { const b = document.createElement('button'); b.dataset.act = a; if (id) b.dataset.id = id; document.getElementById('fleetView').appendChild(b); b.click(); b.remove(); }, [a, id]);

  // 1. Первый запуск
  let { app, page, errs } = await launch();
  check('Приложение запущено, мост TSTNative доступен', await page.evaluate(() => !!(window.TSTNative && window.TSTNative.isNative)));
  check('Заголовок окна', /Регламент-ТС/.test(await page.title()), await page.title());
  check('Папка данных — заданная', await page.evaluate(d => window.TSTNative.dataDir === d, dataDir));
  await page.evaluate(() => TST.setWorkspace('fleet')); await page.waitForTimeout(800);
  check('Файл db.json создан сразу (демо-данные)', fs.existsSync(path.join(dataDir, 'db.json')) && readDb().vehicles.length > 0, readDb().vehicles.length + ' машин');
  check('Записи не дублируются в localStorage браузера', await page.evaluate(() => localStorage.getItem('spravochnik-tst-fleet-v1') === null));

  // 2. Добавить технику через интерфейс
  await page.evaluate(() => { location.hash = '#fleet/park'; }); await page.waitForTimeout(500);
  await page.evaluate(() => document.querySelector('[data-act="add-vehicle"]').click()); await page.waitForSelector('#vName');
  await page.fill('#vName', 'Тестовая кухня «Файл»'); await page.fill('#vInv', 'ИН-E2E-1'); await page.click('#vSave'); await page.waitForTimeout(800);
  const veh = readDb().vehicles.find(v => v.name === 'Тестовая кухня «Файл»');
  check('Новая техника записана в db.json', !!veh, veh && veh.id);

  // 3. Вложение к записи ТО
  const bytes = Buffer.from([37, 80, 68, 70, 0, 1, 2, 200, 255, 10, 13]);   // файл с русским именем передаётся из памяти
  await act(page, 'add-maint', veh.id); await page.waitForSelector('#mFiles', { state: 'attached' });
  await page.setInputFiles('#mFiles', { name: 'акт.pdf', mimeType: 'application/pdf', buffer: bytes }); await page.waitForTimeout(300);
  await page.click('#mSave'); await page.waitForTimeout(1200);
  const filesDir = path.join(dataDir, 'files'); const bins = fs.readdirSync(filesDir).filter(n => n.endsWith('.bin'));
  check('Вложение записано в папку files', bins.length === 1 && Buffer.compare(fs.readFileSync(path.join(filesDir, bins[0])), bytes) === 0, bins.join(','));
  const maint = readDb().maint.find(m => m.vehicleId === veh.id); const docId = maint && maint.docs && maint.docs[0] && maint.docs[0].id;
  check('Запись ТО ссылается на вложение', !!docId && bins[0] === docId + '.bin');

  // 4. Точка восстановления
  await page.evaluate(() => { location.hash = '#fleet/safety'; }); await page.waitForTimeout(600);
  await act(page, 'snap-create'); await page.waitForTimeout(1200);
  const snaps = () => fs.readdirSync(path.join(dataDir, 'snapshots')).filter(n => n.endsWith('.json'));
  check('Точка восстановления записана в папку snapshots', snaps().length >= 1, snaps().length + ' шт.');
  check('Панель «Файлы данных приложения» показана', await page.evaluate(() => /файлы данных приложения/i.test(document.getElementById('fleetView').innerText) && document.getElementById('fleetView').innerText.includes('db.json')));
  check('Нет ошибок JavaScript (сессия 1)', errs.length === 0, errs[0]);
  await app.close();

  // 5. Главное: новый профиль браузера (как после очистки или переустановки), та же папка данных
  ({ app, page, errs } = await launch());
  await page.evaluate(() => { location.hash = '#fleet/park'; }); await page.waitForTimeout(800);
  check('После перезапуска с ЧИСТЫМ профилем техника на месте', await page.evaluate(() => document.getElementById('fleetView').innerText.includes('Тестовая кухня «Файл»')));
  check('Идентификатор техники не изменился', readDb().vehicles.some(v => v.id === veh.id));
  await page.evaluate(() => { location.hash = '#fleet/safety'; }); await page.waitForTimeout(1500);
  const nSnap = await page.evaluate(() => new Promise(r => { const q = indexedDB.open('tst-fleet-snapshots', 1); q.onsuccess = () => { const g = q.result.transaction('snaps').objectStore('snaps').getAll(); g.onsuccess = () => r(g.result.length); }; q.onerror = () => r(-1); }));
  check('Точки восстановления подтянуты из файлов в новый профиль', nSnap >= 1, nSnap + ' шт.');
  check('Точка видна в списке раздела «Сохранность данных»', await page.evaluate(() => /Вернуться/.test(document.getElementById('sfSnaps').innerText)));
  // вложение открывается из файла (в IndexedDB нового профиля его нет)
  await page.evaluate(id => { location.hash = '#fleet/vehicle/' + id; }, veh.id); await page.waitForTimeout(800);
```

22. app/test/e2e.test.js (продолжение)

```
72        await page.evaluate(() => [...document.querySelectorAll('.f-tab')].find(t => /ТО/.test(t.textContent) &&
    !/Моточасы/.test(t.textContent))?.click()); await page.waitForTimeout(500);
73        await page.evaluate(() => document.querySelector('[data-act="download-file"]').click()); await
    page.waitForTimeout(1500);
74        // Имя файла зависит от локали ОС (в тестовой среде без UTF-8 имя заменяется на «download»), поэтому сверяем
    содержимое
75        const dlFiles = fs.readdirSync(dlDir); const got = dlFiles.length ? fs.readFileSync(path.join(dlDir, dlFiles[0])) :
    null;
76        const dlToast = await page.evaluate(() => [...document.querySelectorAll('.f-toast')].map(t =>
    t.innerText.replace(/\n/g, ' ')).join(' | '));
77        check('Вложение читается из файла при чистом профиле (байты совпадают)', got && Buffer.compare(got, bytes) === 0,
    got ? got.length + ' байт' : 'не скачан; в папке: ' + fs.readdirSync(dlDir).join(',') + '; уведомления: ' + dlToast);
78        // правка после перезапуска снова пишет в файл
79        await page.evaluate(() => { location.hash = '#fleet/park'; }); await page.waitForTimeout(500);
80        await page.evaluate(id => { const b = document.createElement('button'); b.dataset.act = 'edit-vehicle'; b.dataset.id
     = id; document.getElementById('fleetView').appendChild(b); b.click(); }, veh.id);
81        await page.waitForSelector('#vInv'); await page.fill('#vInv', 'ИН-E2E-2'); await page.click('#vSave'); await
    page.waitForTimeout(800);
82        check('Правка после перезапуска записана в файл', readDb().vehicles.find(v => v.id === veh.id).invNumber ===
    'ИН-E2E-2');
83        check('Предыдущая версия сохранена в db.prev.json', fs.existsSync(path.join(dataDir, 'db.prev.json')) &&
    JSON.parse(fs.readFileSync(path.join(dataDir, 'db.prev.json'), 'utf8')).vehicles.find(v => v.id === veh.id).invNumber
    === 'ИН-E2E-1');
84        check('Нет ошибок JavaScript (сессия 2)', errs.length === 0, errs[0]);
85        await app.close();
86
87        // 6. Сбой: db.json оборван на середине записи
88        const full = fs.readFileSync(path.join(dataDir, 'db.json'), 'utf8');
89        fs.writeFileSync(path.join(dataDir, 'db.json'), full.slice(0, Math.floor(full.length / 2)));
90        ({ app, page, errs } = await launch());
91        const toasts = await page.evaluate(() => [...document.querySelectorAll('.f-toast-title')].map(t => t.textContent));
92        await page.waitForTimeout(1200);
93        const toasts2 = await page.evaluate(() => [...document.querySelectorAll('.f-toast')].map(t => t.innerText).join(' |
    '));
94        check('Оборванный db.json: программа предупредила о восстановлении', /Файл данных восстановлен из
    копии/.test(toasts.concat(toasts2).join(' ')), toasts2.slice(0, 120));
95        await page.evaluate(() => { location.hash = '#fleet/park'; }); await page.waitForTimeout(800);
96        check('Данные восстановлены из db.prev.json (техника на месте)', await page.evaluate(() =>
    document.getElementById('fleetView').innerText.includes('Тестовая кухня «Файл»')));
97        check('Повреждённый файл сохранён рядом', fs.readdirSync(dataDir).some(n => n.startsWith('db.corrupt-')));
98        check('Новый db.json корректен', (() => { try { return readDb().vehicles.length > 0; } catch (e) { return false; }
    })());
99        check('Нет ошибок JavaScript (сессия 3)', errs.length === 0, errs[0]);
100       await app.close();
101
102       console.log(`ИТОГ: ${res.filter(Boolean).length} из ${res.length}`);
103       process.exit(res.every(Boolean) ? 0 : 1);
104     })().catch(e => { console.error('E2E ERROR', e); process.exit(2); });
105
```

23. app/test/portable.test.js

```
'use strict';
/* Переносной режим и проверка упакованного приложения:
     xvfb-run -a node test/portable.test.js [путь-к-app.asar]
   Без аргумента запускается исходная папка app. */
const path = require('path'), fs = require('fs'), os = require('os');
let pw; try { pw = require('playwright'); } catch (e) { pw = require('/opt/node22/lib/node_modules/playwright'); }
const electronPath = require('electron');
const target = process.argv[2] ? path.resolve(process.argv[2]) : path.resolve(__dirname, '..');
const ok = []; const check = (n, v, i) => { ok.push(!!v); console.log((v ? 'PASS ' : 'FAIL ') + n + (i ? ' — ' + i : ''')); };
(async () => {
  const exeDir = fs.mkdtempSync(path.join(os.tmpdir(), 'tst-portable-'));
  const env = Object.assign({}, process.env, { PORTABLE_EXECUTABLE_DIR: exeDir, TST_TEST: '1' });
  delete env.TST_DATA_DIR; delete env.TST_USER_DATA;
  const app = await pw._electron.launch({ executablePath: electronPath, args: [target, '--no-sandbox'], env });
  const page = await app.firstWindow(); const errs = []; page.on('pageerror', e => errs.push(e.message));
  await page.waitForLoadState('domcontentloaded'); await page.waitForTimeout(3000);
  const base = path.join(exeDir, 'Данные Регламент-ТС');
  check('Приложение стартует' + (process.argv[2] ? ' из упакованного app.asar' : ''), await page.evaluate(() => !!(window.TSTNative && window.TSTNative.isNative)));
  check('Папка данных рядом с программой', await page.evaluate(d => window.TSTNative.dataDir === d, path.join(base, 'db')), await page.evaluate(() => window.TSTNative.dataDir));
  check('db.json создан в переносной папке', fs.existsSync(path.join(base, 'db', 'db.json')));
  check('Профиль приложения тоже рядом (флешка не оставляет следов)', fs.existsSync(path.join(base, 'profile')));
  check('Нет ошибок JavaScript', errs.length === 0, errs[0]);
  await app.close();
  console.log(`ИТОГ: ${ok.filter(Boolean).length} из ${ok.length}`); process.exit(ok.every(Boolean) ? 0 : 1);
})().catch(e => { console.error('ERROR', e); process.exit(2); });

```